LLD Handbook — Vol 2: Design Patterns and Concurrency

OOP · UML · SOLID · Clean Code · Refactoring · Design Patterns · Concurrency · 45+ case studies in Java

6 parts · one place to study LLD: DesignGurus courses + classic books + GitHub problem sets, combined

what it is | problem | how it works | trade-offs | real world | interview | self-check | TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 8 — Design Patterns: Introduction & Creational Patterns**
2. **Part A — Design patterns: the big picture**
   - 8.1 What is a design pattern?
   - 8.2 A short history: from buildings to the "Gang of Four"
   - 8.3 The three categories and all 23 GoF patterns
   - 8.4 The OO principles behind every pattern
   - 8.5 How to pick a pattern
   - 8.6 Anti-patterns: when patterns go wrong
3. **Part B — Creational patterns**
   - 8.7 Creational patterns at a glance
   - 8.8 Singleton
   - 8.9 Simple Factory (an idiom, not a GoF pattern)
   - 8.10 Factory Method
   - 8.11 Abstract Factory
   - 8.12 Builder
   - 8.13 Prototype
   - 8.14 Bonus — Object Pool
   - 8.15 Creational patterns compared
   - 8.16 Chapter self-check
   - 8.17 Chapter TL;DR
4. **Chapter 9 — Structural Design Patterns**
   - 9.0 What structural patterns are
   - 9.1 Adapter Pattern
   - 9.2 Bridge Pattern
   - 9.3 Composite Pattern
   - 9.4 Decorator Pattern
   - 9.5 Facade Pattern
   - 9.6 Flyweight Pattern
   - 9.7 Proxy Pattern
   - 9.8 Putting it all together — comparison
5. **Chapter 10 — Behavioral Design Patterns (Part 1)**
   - 10.0 Behavioral patterns: the big picture
   - 10.1 Chain of Responsibility
   - 10.2 Command
   - 10.3 Iterator
   - 10.4 Mediator
   - 10.5 Memento
   - 10.6 Interpreter
   - 10.7 Chapter recap: the six patterns side by side
6. **Chapter 11 — Behavioral Design Patterns (Part 2) & Beyond GoF**
   - 11.1 Observer
   - 11.2 State
   - 11.3 Strategy
   - 11.4 Template Method
   - 11.5 Visitor
   - 11.6 GoF behavioral patterns: quick-reference summary
   - 11.7 Beyond GoF: patterns every LLD interview expects
   - 11.8 Choose the right behavioral pattern
   - 11.9 Architectural patterns, and how they differ from design patterns
7. **Chapter 12 — Concurrency & Multithreading for LLD (Java)**
8. **Part A — Foundations**
   - 12.1 Concurrency vs parallelism
   - 12.2 Processes vs threads
   - 12.3 Thread lifecycle & states
   - 12.4 Creating threads: `Thread`, `Runnable`, `Callable`, `ExecutorService`, virtual threads
9. **Part B — What goes wrong**
   - 12.5 Race conditions & critical sections
   - 12.6 Java Memory Model basics: visibility, happens-before, `volatile`
10. **Part C — Locks**
    - 12.7 Mutex (mutual exclusion) and `synchronized`
    - 12.8 `ReentrantLock` and reentrancy
    - 12.9 Try-lock & timed locking
    - 12.10 Read-write locks: `ReadWriteLock` and `StampedLock`
    - 12.11 Coarse-grained vs fine-grained locking
11. **Part D — Coordination**
    - 12.12 Semaphores
    - 12.13 Condition variables: `wait/notify` and `Condition`
    - 12.14 `CountDownLatch` and `CyclicBarrier`
12. **Part E — Lock-free**
    - 12.15 Compare-and-swap (CAS) & atomics
    - 12.16 Concurrent collections
13. **Part F — Liveness failures**
    - 12.17 Deadlock
    - 12.18 Livelock
    - 12.19 Starvation (and fairness)
14. **Part G — Concurrency patterns**
    - 12.20 Signaling pattern
    - 12.21 Thread pool pattern
    - 12.22 Producer-consumer pattern
    - 12.23 Reader-writer pattern
    - 12.24 Immutability & thread confinement
15. **Part H — Concurrency in LLD interviews**

- 12.25 How concurrency shows up in LLD interviews (parking, seat booking, inventory)
- 12.26 Choosing the right tool (cheat sheet)
- 12.27 Chapter self-check
- 12.28 Chapter TL;DR

16. **Chapter 13 — Classic Concurrency Problems**
    - 13.1 Print FooBar Alternately (LeetCode 1115)
    - 13.2 Print Zero Even Odd (LeetCode 1116)
    - 13.3 Fizz Buzz Multithreaded (LeetCode 1195)
    - 13.4 Building H2O (LeetCode 1117)
    - 13.5 Dining Philosophers (LeetCode 1226)
    - 13.6 Thread-safe cache with TTL
    - 13.7 Design a (simplified) Concurrent HashMap — lock striping
    - 13.8 Thread-safe bounded Blocking Queue (two ways)
    - 13.9 Concurrent Bloom Filter
    - 13.10 Multi-threaded Merge Sort (Fork/Join)
    - 13.11 Rate limiter — token bucket (thread-safe)
    - 13.12 Chapter summary
    - 13.13 Chapter self-check
    - 13.14 Chapter TL;DR

Chapter 8 — Design Patterns: Introduction & Creational Patterns

What you will learn. What a design pattern is and where the idea came from, the full Gang of Four catalogue of 23 patterns, the OO principles underneath them, how to choose (and *not* choose) a pattern, the anti-patterns of overuse — and then, in depth, the five creational patterns (Singleton, Factory Method, Abstract Factory, Builder, Prototype), the Simple Factory idiom and the Object Pool bonus pattern.

How to read it. Part A is the map. Part B covers each creational pattern with the same template: *what it is → problem → how it works → Java code (real output) → trade-offs → when not to use it → mistakes → real-world examples → interview answers → self-check → TL;DR*. The chapter ends with one comparison table.

Code. Everything lives in `chapters/08-patterns-creational/code/lld/ch08/<pattern>/`. Each package (a named folder of related classes that also acts as a namespace) has a `Main` (the class whose `main` method starts the program). Listings below show the essential lines; the full runnable files are in that folder. Compile and run with:

```
cd chapters/08-patterns-creational/code
javac --release 21 -d /tmp/lld_08 $(find . -name '*.java')
java -cp /tmp/lld_08 lld.ch08.singleton.Main        # or .simplefactory / .factorymethod / .abstractfactory / .builder / .prototype / .objectpool
```

Part A — Design patterns: the big picture

8.1 What is a design pattern?

What it is

A **design pattern** is a named, reusable description of a good solution to a problem that keeps recurring in software design. It is not a library or code you paste in — it is a *recipe* you adapt, the written-down experience of many developers so you do not have to rediscover it (as builders learned over hundreds of houses where a front door works best).

- **Intent** — the one-sentence purpose of a pattern ("ensure a class has only one instance").
- **Participants** — the roles (classes/interfaces — an **interface** is a type listing method signatures without bodies, which classes implement) in the pattern, such as *Creator*, *Product*, *Client*.

The problem it solves

Without patterns every team re-solves the same recurring problems (creating objects without hard-coding classes, adding behaviour without editing old code, notifying many listeners), each in a slightly different, slightly buggy way. Patterns give you:

Benefit	What it means in practice
Solve common design problems	Start from a solution refined over years.
Avoid reinventing the wheel	Less design time, fewer surprises.
Readability and maintainability	A known shape is easier for the next person.
Shared vocabulary	"This is a Factory" replaces a paragraph. The biggest win in interviews.
Best practices built in	Patterns embody loose coupling (parts know little about each other) and open/closed (extend without editing), so code tends to be more testable.
Adaptability and scalability	Many patterns exist so code can absorb change later.
Faster development	Tested paradigms speed up decisions.

How it works

Every classic pattern description (the Gang of Four framing) has four parts:

1. **Name** — a word or two that becomes team vocabulary.
2. **Problem** — when to apply it: the situation and the forces in tension.
3. **Solution** — participants, responsibilities and collaborations; an abstract *template*, not concrete code.
4. **Consequences** — costs and benefits. A pattern without its trade-offs is half a pattern.

You **adapt** the template to your language and problem; two correct implementations can look quite different.

When not to use it

Not every problem needs a pattern. Forcing one adds classes, indirection (extra layers between you and the code doing the real work) and confusion — worse than no pattern. See 8.6 Anti-patterns.

TL;DR

A design pattern = **name + problem + solution template + consequences**. Shared experience, not copy-paste code. Biggest value: vocabulary and proven trade-offs; biggest risk: overuse.

8.2 A short history: from buildings to the "Gang of Four"

Year	Event	Why it matters
1977	Architect **Christopher Alexander** publishes *A Pattern Language*, solutions for towns and buildings.	The "pattern" idea comes from architecture, not software.
1987	**Kent Beck** and **Ward Cunningham** apply pattern languages to programming (UI design in Smalltalk, an early OO language).	First bridge to software.
1994	**Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides** publish *Design Patterns: Elements of Reusable Object-Oriented Software* — the **Gang of Four (GoF)**.	Catalogues **23 patterns** in three categories, with C++ and Smalltalk examples.
2004	*Head First Design Patterns* (Freeman & Robson, with Sierra and Bates) teaches them in Java with stories (pizza store for factories, chocolate boiler for Singleton, Starbuzz coffee for Decorator).	Made GoF approachable and put the **OO principles** front and centre.

Two GoF principles run through the whole catalogue:

1. **Program to an interface, not an implementation.** Type variables and parameters by abstractions (interfaces or abstract classes that say *what*, not *how*), so callers do not care which concrete class (a fully implemented class you can `new`) they get.
2. **Favour object composition over class inheritance.** Build behaviour by *holding* objects ("has-a"), not only by extending classes ("is-a"). Composition is "black-box" reuse; inheritance exposes a parent's internals.

Creational patterns make principle 1 practical: if code must not name concrete classes, *someone* still has to create the objects — creational patterns decide who and how.

8.3 The three categories and all 23 GoF patterns

What it is

The GoF grouped patterns by **purpose**:

Category	Question it answers	In one line
Creational	*How do objects get made?*	Control creation so the system does not depend on concrete classes or messy construction details.
Structural	*How are objects put together?*	Compose classes and objects into larger, flexible structures — simple ways to realise relationships between entities.
Behavioral	*How do objects talk and share work?*	Assign responsibilities and design communication between objects, rather than thinking in raw control flow.

A second axis is **scope**: **class patterns** use inheritance, fixed at compile time (Factory Method, class Adapter, Template Method, Interpreter); **object patterns** use composition, changeable at run time (almost everything else).

The full catalogue

#	Pattern	Category	Intent in one line	Chapter
1	**Abstract Factory**	Creational	Create *families* of related objects without naming their concrete classes.	8
2	**Builder**	Creational	Build a complex object step by step, separating construction from the final representation.	8
3	**Factory Method**	Creational	Let subclasses decide which concrete class to instantiate through an overridable creation method.	8
4	**Prototype**	Creational	Create new objects by copying a pre-configured instance.	8
5	**Singleton**	Creational	Ensure a class has exactly one instance and give a global access point to it.	8
6	**Adapter** (also called Wrapper)	Structural	Convert one interface into another that clients expect.	9
7	**Bridge**	Structural	Split an abstraction from its implementation so both can vary independently.	9
8	**Composite**	Structural	Treat individual objects and trees of objects uniformly.	9
9	**Decorator**	Structural	Attach extra responsibilities to an object dynamically by wrapping it.	9
10	**Facade**	Structural	Give a simple, unified entry point to a complex subsystem.	9
11	**Flyweight**	Structural	Share common (intrinsic) state to support huge numbers of fine-grained objects cheaply.	9
12	**Proxy**	Structural	Provide a stand-in that controls access to another object (lazy, remote, protective…).	9
13	**Chain of Responsibility**	Behavioral	Pass a request along a chain of handlers until one handles it.	10/11
14	**Command**	Behavioral	Turn a request into an object so it can be queued, logged or undone.	10/11
15	**Interpreter**	Behavioral	Represent a small language's grammar as classes and evaluate sentences in it.	10/11
16	**Iterator**	Behavioral	Traverse a collection without exposing how it is stored.	10/11
17	**Mediator**	Behavioral	Centralise complex communication between objects in one mediator object.	10/11
18	**Memento**	Behavioral	Capture and restore an object's state without breaking encapsulation (keeping internals private).	10/11
19	**Observer**	Behavioral	Notify many dependents automatically when one object changes.	10/11
20	**State**	Behavioral	Let an object change behaviour when its internal state changes, as if it changed class.	10/11
21	**Strategy**	Behavioral	Make a family of algorithms interchangeable behind one interface.	10/11
22	**Template Method**	Behavioral	Fix an algorithm's skeleton in a base class and let subclasses fill in steps.	10/11
23	**Visitor**	Behavioral	Add new operations to a set of classes without changing those classes.	10/11

Count check: 5 creational + 7 structural + 11 behavioral = **23**. (Some course lists show ten behavioral patterns, omitting the rarely used Interpreter — it is still one of the 23.)

Not in the GoF 23 but you will meet them: *Simple Factory* (an idiom), *Object Pool* (end of this chapter), *Dependency Injection* (handing an object its dependencies from outside instead of letting it create them), *Null Object*, *Repository*, *MVC* (Model–View–Controller: data, display and input handling split apart; Head First's "compound pattern" of Observer, Strategy and Composite).

Self-check

- **Q:** Which creational pattern is a *class* pattern (uses inheritance)? **A:** Factory Method — subclasses override the creation method.
- **Q:** How many behavioral GoF patterns are there? **A:** Eleven.

8.4 The OO principles behind every pattern

Patterns are principles applied to recurring situations. *Head First* collects these as it goes; together with SOLID (Chapter 4 — Single responsibility, Open–closed, Liskov substitution, Interface segregation, Dependency inversion) they explain *why* each pattern is shaped as it is.

Principle (plain words)	What it means	Patterns that apply it (examples)
Encapsulate what varies	Put the part that changes behind its own abstraction, apart from what stays the same.	Factory Method (the created type varies), Strategy, State, Builder
Favour composition over inheritance	Get behaviour by holding objects, not only by subclassing.	Abstract Factory (client *holds* a factory), Strategy, Decorator
Program to interfaces, not implementations	Depend on the supertype.	Every creational pattern returns an interface type
Strive for loosely coupled designs between objects that interact	Objects know as little about each other as possible.	Observer, Mediator, factories
Open–Closed Principle — open for extension, closed for modification	Add behaviour by adding code, not editing tested code.	Factory Method (new product = new subclass), Decorator
Dependency Inversion — depend on abstractions, not concrete classes	High- and low-level code both depend on interfaces.	The core of all factory patterns: the client never calls `new ConcreteThing()`
Least Knowledge ("only talk to your friends")	A method calls only its own object, its parameters, objects it creates and its fields.	Facade
Hollywood Principle ("don't call us, we'll call you")	High-level components decide when low-level ones run.	Template Method, Factory Method (base class calls the subclass's hook), frameworks/IoC (inversion of control)
Single Responsibility — one reason to change	Each class does one job.	Builder (construction separated from the product), Iterator

Why creational patterns exist, in one sentence: Dependency Inversion says "do not depend on concrete classes", but `new PdfDocument()` *is* such a dependency — so creational patterns move that `new` somewhere controlled (a factory, a subclass, a builder, a prototype, a container).

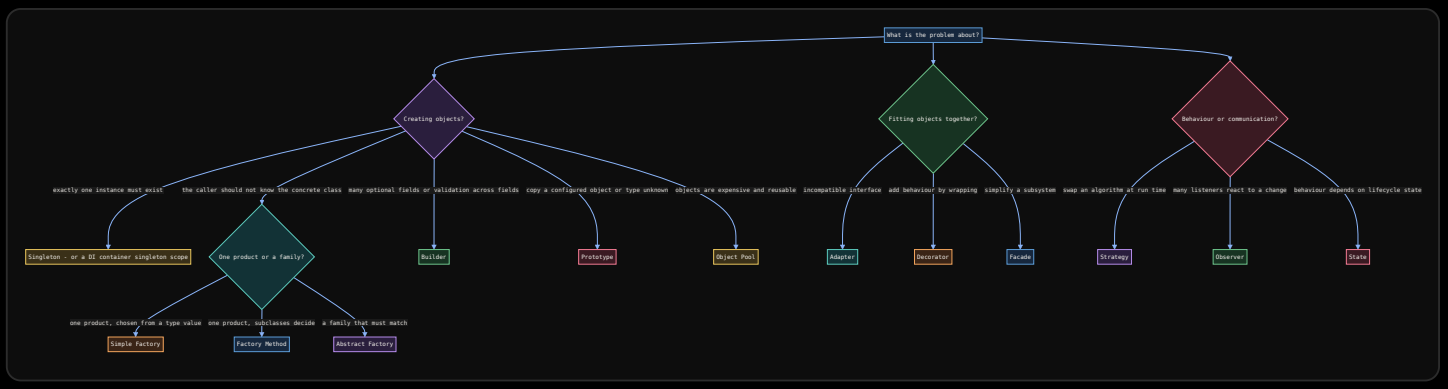
8.5 How to pick a pattern

The problem it solves

Learning 23 patterns and then hunting for places to use them is backwards. Start from a **signal** in the requirements, then reach for the pattern that answers it.

How it works

1. **Name the problem** in plain words: "the pricing rule will change", "there must be exactly one", "the constructor has ten arguments".
2. **Decide the category**: creating, structuring, or behaving/communicating?
3. **Match the signal** to a pattern (tables below).
4. **Check the cost**: how many classes does it add? Is there a simpler option — an enum (a type with a fixed list of named constants), a lambda (a short unnamed function), a constructor, a DI container (a framework that creates objects and hands them their dependencies)?
5. **Say what you rejected and why** — in an interview, worth as much as the pick.



The six patterns that come up in almost every LLD interview

Six patterns recur in object-oriented design rounds. A name replaces a long explanation and signals you have met the problem before — but forcing a pattern where it is not needed costs more than not knowing it. So learn each with its **signal**: the clue in the problem statement that says it fits.

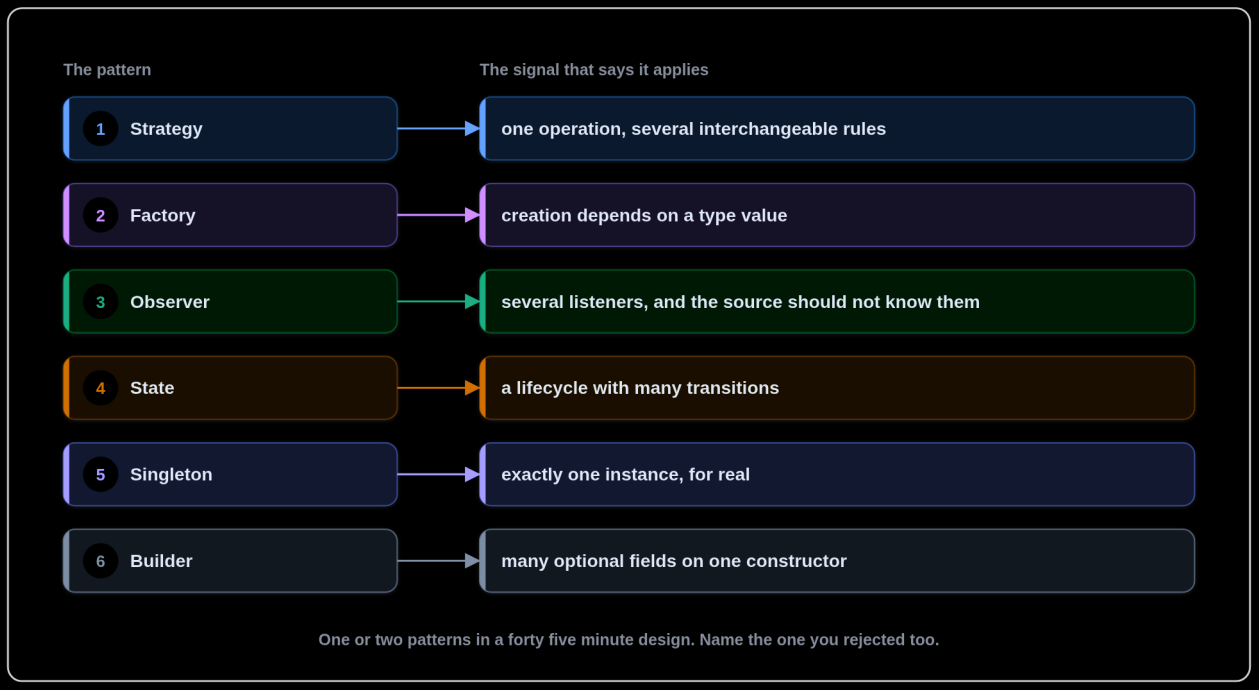


Figure — the six interview patterns and the signal that tells you each one applies (course diagram)

Pattern	The signal	Where it typically applies	Use it when	Do not use it when
Strategy	One operation comes in several swappable versions, chosen at run time	Pricing, matching, sorting, ranking, routing	One operation with several interchangeable rules	Only one rule exists and nothing hints at a second
Factory	Something must be created and the caller should not care which concrete class	Type value → vehicle, spot, payment or notification	The class to create depends on a type value	Only one concrete class exists
Observer	One object changes and several others must react, without it knowing who they are	Display boards, notifications, live updates, event logs	Several listeners the source should stay ignorant of	A single listener the source already references
State	Behaviour depends on current status, and allowed moves between statuses are fixed	Tickets, orders, bookings, loans — anything with a lifecycle	Many states and transitions	Three or four states handled by one guarded method
Singleton	Genuinely one and only one instance	The top-level object, e.g. `ParkingLot`, `Library`	There really must be exactly one	You just want an easy global handle
Builder	Many optional fields; the constructor keeps growing	A book copy, a booking, a search query	Lots of optional fields	A handful of required fields

How each signal shows up in a parking-lot or library design:

- **Strategy.** Price starts as "per hour"; later come a weekend tariff and a monthly pass. Without a pattern the fee method grows a `switch` edited for every tariff. Strategy defines an interface `ParkingRate` (hours → amount); `HourlyRate`, `WeekendRate`, `MonthlyPass` implement it and the lot holds the one in force. A new tariff is one new class, no edits. This answers the most common follow-up: "what if the pricing rules change?"
- **Factory.** A string or enum must become a `Vehicle`. If that lives in the entrance panel, the panel knows every type and changes with each new one. A factory takes the type and returns the object — the single registration point for new types (Open–Closed in practice). Keep it lean: in 45 minutes a creation method on an existing class is usually enough; one factory class per vehicle type is too much.
- **Observer.** When a spot is taken, every display board on that floor must refresh. The spot is the *subject*; boards, entrance panels and floor portals *subscribe*; on each change it notifies them without knowing what they are, so a new board needs no change to the spot. Contrast the library: a returned reserved book must notify exactly one member — a direct call suffices; one listener does not justify Observer.
- **State.** A parking ticket goes active → paid → closed; a book copy is available, reserved, on loan or lost. Without the pattern every method opens with "if status is…" and legality rules scatter. State gives each status a small class knowing its legal next steps, so an illegal move cannot even be written. Cost: with four states and three transitions an `enum` plus one guarded method reads better — reach for State when the transition table is big, and say why.
- **Singleton.** A car park is one physical place, so one `ParkingLot`: private constructor plus one accessor. Interviewers probe two points, so raise them yourself: fetching the instance must be correct when several threads (independent paths of execution in one program) ask at once (use a lock — a guard letting one thread at a time run a piece of code — or create it eagerly), and a singleton is global state (data any part of the program can read and change) that makes tests harder, which is why many teams create one instance and pass it around. Stating that trade-off earns more than naming the pattern.
- **Builder.** A `BookItem` needs barcode, price, format, purchase date, publication date and rack — an eight-argument constructor is easy to misuse, since two adjacent `String` parameters can be swapped silently. A builder names every field and builds at the end. Raise it once a constructor passes about six arguments, then move on: it improves readability, not structure.

Rules of thumb for the interview:

- **One or two patterns** in a 45-minute design, placed exactly where their signal appears. Six means the design has drifted from the requirements.
- **Compare out loud:** "Strategy for rates, because a second tariff is clearly coming; no State for the ticket, because one guard handles three statuses." Explaining what you *rejected* shows as much judgement as what you chose.

8.6 Anti-patterns: when patterns go wrong

What it is

An **anti-pattern** is a common "solution" that looks reasonable but reliably causes more problems than it solves.

Anti-pattern	What it looks like	Why it hurts	The fix
God Object (God class, the Blob)	One class knows and does almost everything: `SystemManager` with 80 methods and 40 fields.	Violates Single Responsibility; every change touches it, merge conflicts (clashing edits to the same lines) everywhere, untestable in isolation.	Split by responsibility (Extract Class — Chapter 7); push behaviour to the objects that own the data.
Golden Hammer	"When all you have is a hammer, everything looks like a nail" — the favourite tool or pattern applied everywhere.	Forced onto problems it does not fit: every object a Singleton, every `if` a Strategy.	Start from the problem's signal (8.5), not the tool.
Pattern overuse ("patternitis")	Factories creating one class, single-implementation interfaces "just in case", five layers of indirection for a CRUD (create, read, update, delete) screen.	More classes and hops, no benefit. Fowler's smell: **Speculative Generality** (building for imagined needs).	YAGNI ("You Aren't Gonna Need It", Chapter 5): add the pattern when the *second* variant arrives. Refactor towards a pattern; do not start from one.
Singletonitis	Singletons as convenient global variables.	Hidden dependencies, order-dependent tests, hard to replace.	Dependency injection; pass the object in (8.8).
Cargo-cult patterns	Copying structure without the intent (a "Builder" that only copies fields with no validation, a "Factory Method" that is really a switch).	Ceremony with none of the guarantees.	Know the *intent*: what property does it give you? If you cannot name it, drop it.

TL;DR

Patterns are a vocabulary, not a checklist. The God Object comes from ignoring design; Golden Hammer and pattern overuse from over-applying it.

Part B — Creational patterns

8.7 Creational patterns at a glance

What it is

Creational patterns control *how objects are created* so the rest of the system is not tied to concrete classes or construction details. They hide *which* class is instantiated, *how* it is assembled, and sometimes *whether* a new object is made at all — keeping the system **independent** of how objects are created, composed and represented, **hiding the complexity** of creation (long setup, validation, choosing a subtype), and keeping **flexibility** over *what*, *who*, *how* and *when*.

Pattern	Intent in one line
Singleton	Exactly one instance of a class, with one global way to reach it.
Factory Method	A base class defines a creation method; subclasses decide which concrete class it returns.
Abstract Factory	One factory object creates a whole *family* of related objects that must match.
Builder	Assemble one complex object step by step; the same process can produce different representations.
Prototype	Create new objects by cloning a prototypical instance.
Simple Factory (idiom)	One method with a switch/registry decides which class to create.
Object Pool (bonus)	Reuse a bounded set of expensive objects instead of creating and destroying them.

8.8 Singleton

What it is

The **Singleton** pattern restricts a class to **exactly one instance** (per process — one running program — or, strictly, per class loader, the part of the JVM (Java Virtual Machine) that loads classes into memory) and gives every caller **one well-known way to reach it** — like a country's one official government that everyone reaches through a known channel.

- **Instance** — one object created from a class.
- **Eager initialization** — create the object as soon as the class is loaded.
- **Lazy initialization** — create it the first time someone asks.
- **Thread-safe** — correct even when many threads call it at the same moment.

GoF intent and participants. Intent: a class never has more than one instance, and every client has a well-known way to reach it. One participant, **Singleton**, which (a) defines a class-level operation (`getInstance()`) to reach the unique instance and (b) usually creates that instance itself. Clients just call that operation.

The problem it solves

Some things really are "one per process": settings loaded at start-up, the pool of database connections (open connections kept ready for reuse), the metrics registry (counters and timings the app records about itself). If every module creates its own, settings are parsed many times and metrics split across registries nobody can combine. E.g. a `ConfigStore` (region, retry count, timeout) read by billing, the nightly report job and the mailer: three private copies mean three parses, and a change in one never reaches the others.

A plain global variable has two problems: tests share it (a result depends on which test ran before), and *some* code must assign it before anyone reads it — get start-up order wrong and a caller finds `null`. Singleton moves both jobs into the class: **the class decides how many instances may exist and creates the one it allows**, on demand, so there is no ordering mistake to make.

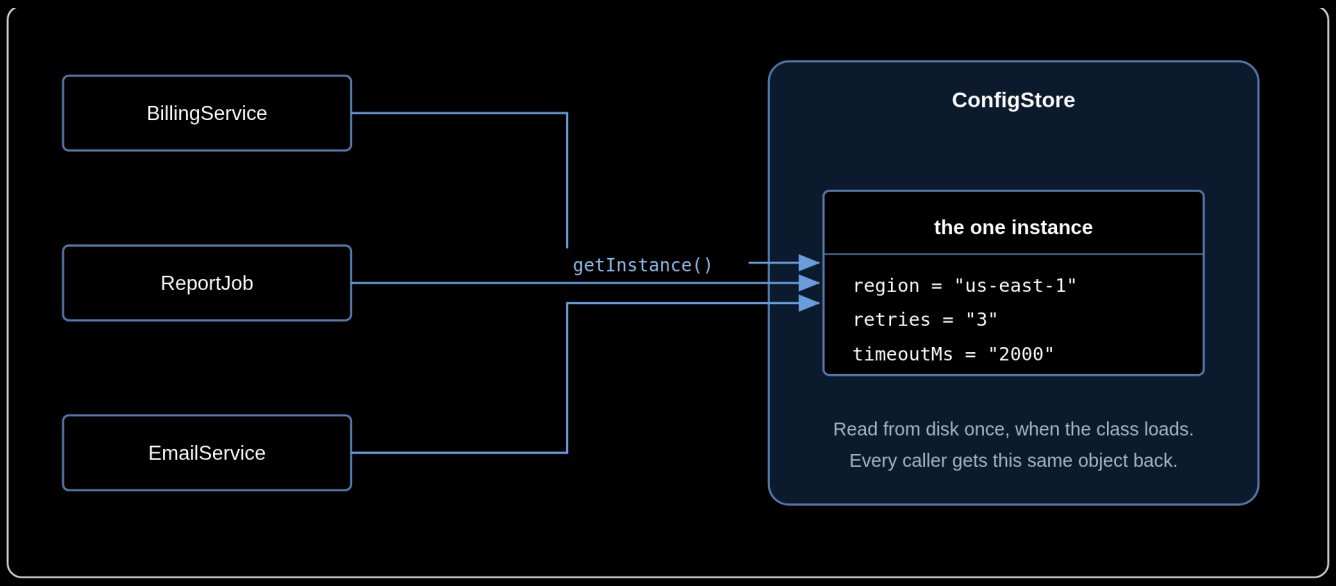


Figure — BillingService, ReportJob and EmailService all call getInstance() and receive the same ConfigStore, which was read from disk once (course diagram)

How it works

1. Make the **constructor** `private` so outside code cannot write `new ConfigStore()`. That access modifier (`public` / `protected` / `private` — who may use a member) is the whole enforcement; make it public and anyone can create a second one.
2. Keep the instance in a `private static` **field** — owned by the class, not by any object.
3. Expose a `public static getInstance()` — static because a caller has no instance yet.
4. Inside it, return the stored instance, creating it first if needed (safely when threads are involved).
5. Add ordinary instance methods (`get`, `set`, …) for the real work.

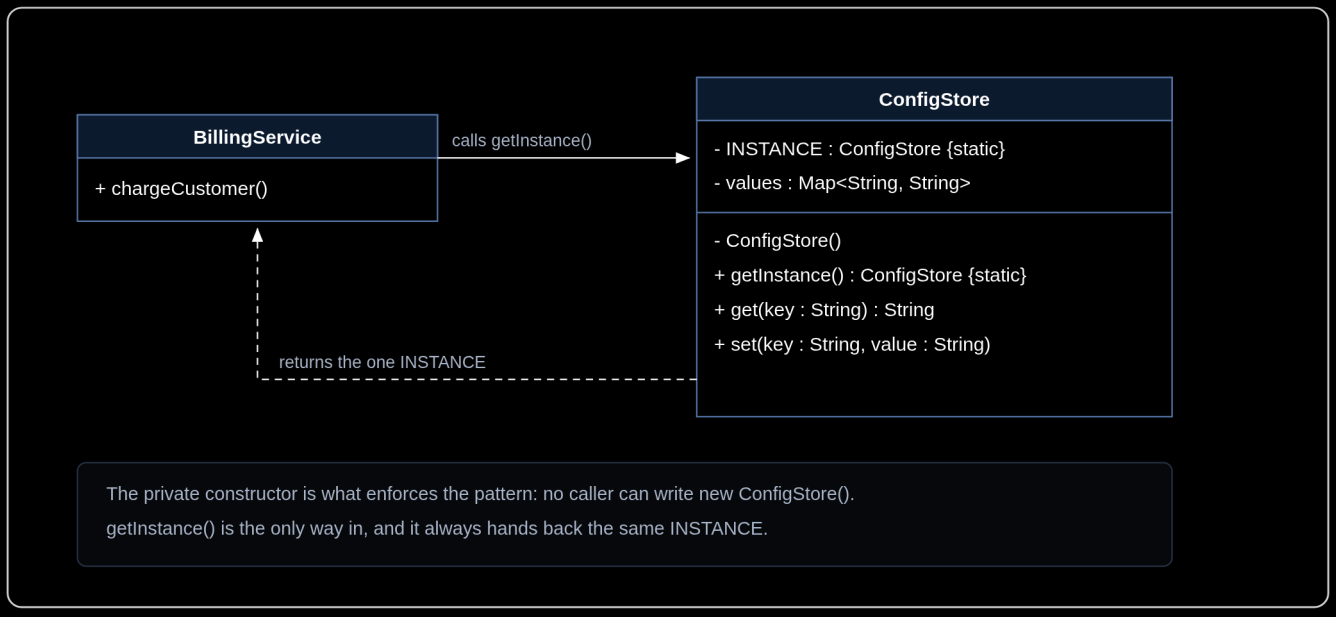
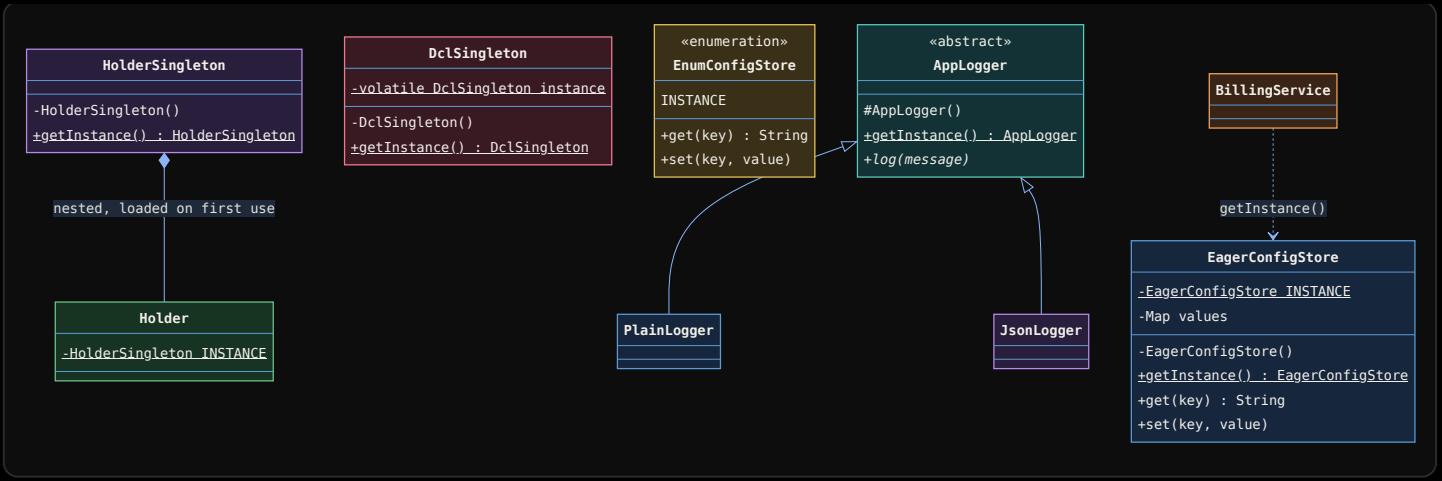


Figure — ConfigStore with a private static INSTANCE, a private constructor and a static getInstance(); BillingService only calls getInstance() (course diagram)

Our Java design (all the variants in the code package):



The six ways to write a Singleton in Java

Terms: **serialization** turns an object into bytes to save or send (**deserialization** rebuilds it); **reflection** is Java's ability to inspect and call a class's members — even private ones — at run time; **contention** is many threads competing for one lock.

#	Variant	Lazy?	Thread-safe?	Cost on each call	Survives serialization / reflection?	Verdict
1	**Eager** `static final` field	No (built when the class is initialized)	Yes — the JVM initializes a class exactly once	None	No / No (needs defences)	Default choice when construction is cheap
2	**Naive lazy** `if (instance == null)`	Yes	**No**	None	No / No	Never in multi-threaded code
3	`synchronized` **getInstance()**	Yes	Yes	A lock on *every* call	No / No	Correct but slow under contention
4	**Double-checked locking** + `volatile`	Yes	Yes (Java 5+)	One volatile read	No / No	When the instance depends on run-time input
5	**Bill Pugh / holder idiom**	Yes	Yes — class-loading guarantee	None	No / No	Best lazy option in plain Java
6	`enum` **with one constant**	No (created when the enum class initializes)	Yes	None	**Yes / Yes** (JVM guarantees)	Shortest and most robust; recommended by *Effective Java* (Joshua Bloch's best-practice book)

1 — Eager. The `static final` field is assigned when the class is *initialized* — on its first active use (typically the first `getInstance()`, but also any other static method or non-constant static field), not at JVM start-up. The Java Language Specification (§12.4.2) guarantees initialization runs exactly once, under an internal lock, and every thread then sees the fully built object — thread safety for free. Cost: the object is built when the class is touched for *any* reason; matters if the constructor opens a file or socket (a network connection endpoint).

2 — Naive lazy (broken). "Create on first use" written as *check, then act*:

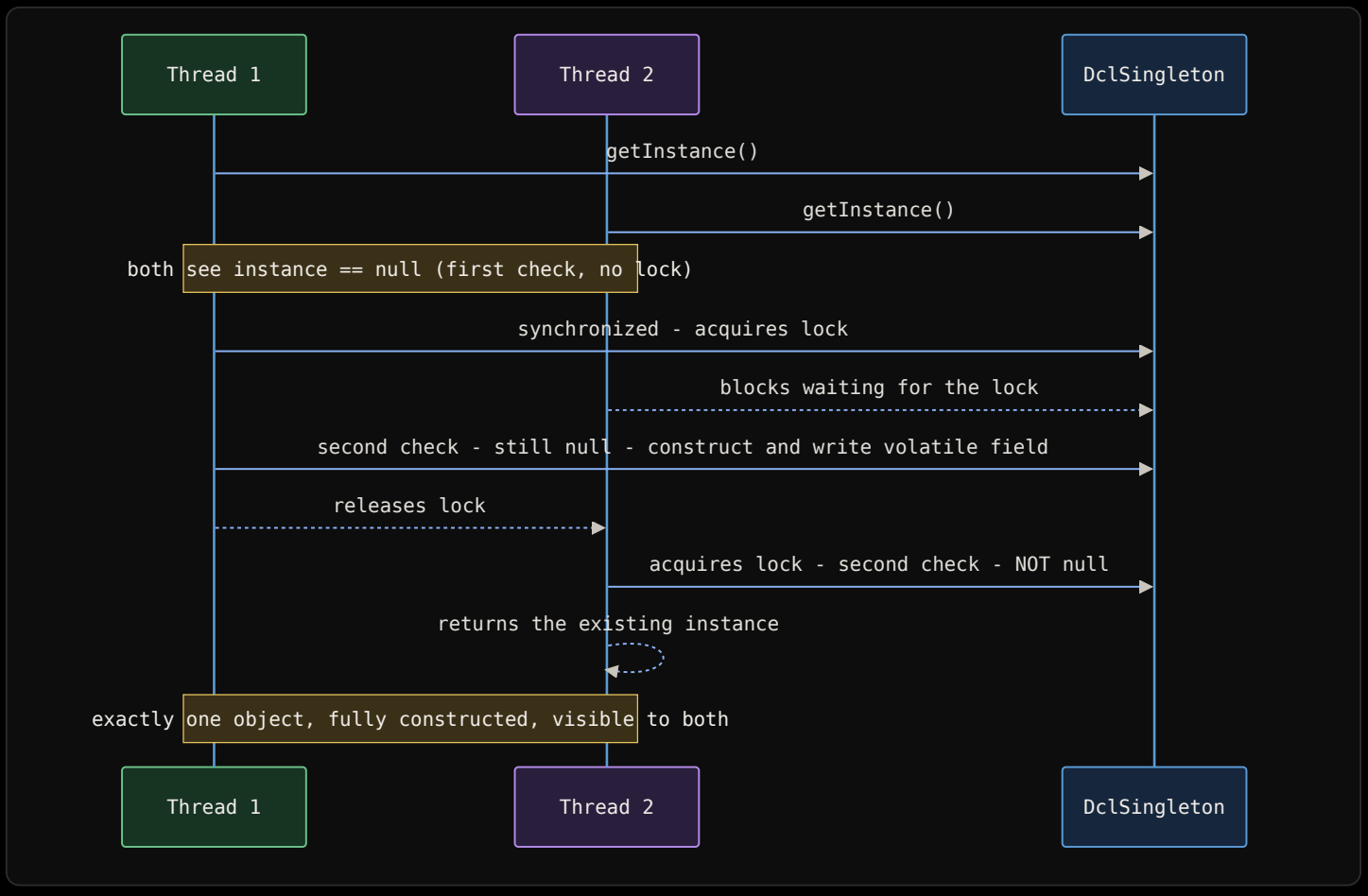
```
// Do NOT use this in multi-threaded code.
public static ConfigStore getInstance() {
    if (instance == null) {            // two threads can both see null here...
        instance = new ConfigStore();  // ...and both construct
    }
    return instance;
}
```

Threads A and B can both read `null` before either stores its object; each builds its own store and writes to one never show in the other. It passes every single-threaded test, which is why it slips through review. Our demo releases eight threads at once: the naive version built **more than one** object (eight in the run below).

3 — `synchronized` method. Fixes the race (a **race condition**: a bug whose outcome depends on unlucky thread timing) — one thread inside at a time — but takes the lock on *every* call forever, though only the first needs it.

4 — Double-checked locking (DCL). Check *without* the lock (fast path); only if missing, take the lock and **check again** (another thread may have created it meanwhile):

- The second check prevents two constructions.
- The field **must be** `volatile` (makes each write immediately visible to all threads and forbids reordering around it). Creating an object is three steps — allocate, run constructor, store reference. Without `volatile` the Java Memory Model (rules for when one thread's writes become visible to others) lets another thread see the reference *before* the constructor's writes — a half-built object. A `volatile` write/read creates a **happens-before** relationship: everything the constructor did is visible to any thread that reads the reference. (Before Java 5's memory model, DCL was broken even with `volatile`.)
- Copying the field into a local (`local`) means the common path reads the volatile field once — an *Effective Java* optimization.



5 — Bill Pugh / initialization-on-demand holder. Put the instance in a private *nested* class (declared inside another class). The JVM does not initialize `Holder` until `getInstance()` first touches `Holder.INSTANCE` (lazy), and class initialization runs once and is safely published (visible to other threads only once fully built) — thread-safe with **no locks and no** `volatile`. Named after Bill Pugh, who led the Java memory model work. The usual lazy choice.

6 — Enum. `enum EnumConfigStore { INSTANCE; ... }`. The JVM guarantees each constant exists once; `Constructor.newInstance` refuses enum objects; enum serialization returns the existing constant. Shortest correct version. Limits: cannot extend a class (can implement interfaces); created when the enum class initializes, not truly on demand.

Which one to pick. Default to **eager** (fewest lines, no race possible; drawback only when construction is costly). Switch to **holder** once you have *measured* a start-up cost worth avoiding. Use **DCL** when construction needs a value only known at run time (a holder's static initializer cannot take a parameter) — always `volatile`. Use **enum** for serialization and reflection safety with no defensive code.

Breaking a Singleton — and the fixes

A `private` constructor is a *language* rule, not a JVM guarantee:

Attack	How it creates a second instance	Fix
Reflection	`getDeclaredConstructor().setAccessible(true).newInstance()` calls the private constructor.	Throw from the constructor if the instance already exists (reliable for eager/holder, racy for lazy). Or use an **enum** — the JVM refuses.
Serialization	`ObjectInputStream.readObject()` builds a new object without calling your constructor.	Add `private Object readResolve() { return INSTANCE; }` — the deserialized copy is replaced by the real one. Mark instance fields `transient` (skipped by serialization) if they should not travel. Or use an **enum**.
Cloning	If the class (or a parent) implements `Cloneable` and exposes `clone()`, `clone()` copies the object.	Override `clone()` to throw `CloneNotSupportedException` (or return `INSTANCE`). If no ancestor is `Cloneable`, `Object.clone()` is `protected` and this attack does not apply.
Multiple class loaders	Each class loader loads its own copy of the class, each with its own "single" instance (common in app servers and plugin systems).	Load the class from a shared parent loader, or let a container own the instance.
Multiple processes / servers	One per JVM: four servers, four instances.	Use an external coordinator (database, Redis — an in-memory data store, ZooKeeper — a coordination service) for anything unique system-wide, e.g. ID generation or locks.

The demo attacks an undefended singleton, a hardened one, then the enum (see output).

Subclassing a Singleton

GoF notes a singleton *can* be subclassed if its constructor is `protected`, with `getInstance()` deciding **once, at run time**, which subclass becomes "the" instance. `AppLogger` picks `PlainLogger` or `JsonLogger` from a system property (a key=value setting passed at launch, e.g. `-Dlog.format=json`). With a `private` constructor (the usual case) it cannot be subclassed — the flexibility exists only if designed in.

The dependency-injection alternative

In modern code the best singleton is often **not a Singleton class at all**: a normal class with a public constructor, **one** instance created at start-up (the *composition root*), and **passed in** through constructors. "One instance" becomes a wiring decision, not a property of the class.

- Dependencies are visible in constructors (no hidden `getInstance()` calls).
- Tests pass a fake (a simple stand-in written for tests): `new BillingService(testConfig)` — no reset hooks.
- Spring, Guice, Micronaut, Dagger, NestJS (DI frameworks) do exactly this: in Spring every bean (an object Spring creates and manages) is **singleton-scoped by default** — *one shared instance per bean definition per container* (`ApplicationContext`), not one per JVM, and with no private constructor. Two contexts hold two instances, and nothing stops other code calling `new`. The opposite, **`prototype` scope**, creates a fresh instance on every lookup.

Java implementation

Full code: `code/lld/ch08/singleton/` — `EagerConfigStore`, `LazyVariants` (naive, synchronized, DCL, holder), `EnumConfigStore`, `BreakableSingleton`, `HardenedSingleton`, `AppLogger`, `DiAlternative`, and `Main` (the demo driver).

```java
// EagerConfigStore.java — 1. Eager: created when the class is initialised. Thread-safe for free.
public final class EagerConfigStore {
    private static final EagerConfigStore INSTANCE = new EagerConfigStore();
    private final Map<String, String> values = new ConcurrentHashMap<>();

    private EagerConfigStore() {                        // private: nobody else can call new
        System.out.println("  [EagerConfigStore] reading settings from disk (runs once)");
        values.put("region", "us-east-1");
        values.put("retries", "3");
    }
    public static EagerConfigStore getInstance() { return INSTANCE; }
    public String get(String key)             { return values.get(key); }
    public void set(String key, String value) { values.put(key, value); }
}
```

```java
// LazyVariants.java (excerpt) — 4. DCL: lock only on the first calls. volatile is mandatory.
private static volatile DclSingleton instance;
static DclSingleton getInstance() {
    DclSingleton local = instance;           // one volatile read on the fast path
    if (local == null) {                     // 1st check, no lock
        synchronized (DclSingleton.class) {
            local = instance;
            if (local == null) {             // 2nd check, under the lock
                local = new DclSingleton();
                instance = local;            // volatile write publishes a fully built object
            }
        }
    }
    return local;
}

// 5. Bill Pugh holder: lazy, lock-free, thread-safe via class loading.
private static final class Holder {          // not initialised until first touched
    private static final HolderSingleton INSTANCE = new HolderSingleton();
}
static HolderSingleton getInstance() { return Holder.INSTANCE; }
```

```java
// EnumConfigStore.java — 6. Safe against reflection and serialization by the JVM itself.
public enum EnumConfigStore {
    INSTANCE;
    private final Map<String, String> values = new ConcurrentHashMap<>(Map.of("region", "us-east-1"));
    public String get(String key)             { return values.get(key); }
    public void set(String key, String value) { values.put(key, value); }
}
```

```java
// HardenedSingleton.java — BreakableSingleton is identical minus these three defences.
// CloneableBase is a parent that (unwisely) exposes a public clone().
public final class HardenedSingleton extends CloneableBase implements Serializable {
    public static final HardenedSingleton INSTANCE = new HardenedSingleton();

    private HardenedSingleton() {
        // Defence 1 - reflection (reliable with eager init: INSTANCE is set during class init)
        if (INSTANCE != null) throw new IllegalStateException("Singleton already exists - use INSTANCE");
    }
    @Serial private Object readResolve() { return INSTANCE; }   // Defence 2 - serialization
    @Override public Object clone() throws CloneNotSupportedException {   // Defence 3 - cloning
        throw new CloneNotSupportedException("Singleton cannot be cloned");
    }
}
```

```java
// AppLogger.java — subclassable singleton: protected constructor, subclass chosen once at run time.
public abstract class AppLogger {
    protected AppLogger() { }
    private static final class Holder {
        private static final AppLogger INSTANCE =
                "json".equals(System.getProperty("log.format")) ? new JsonLogger() : new PlainLogger();
    }
    public static AppLogger getInstance() { return Holder.INSTANCE; }
    public abstract void log(String message);
}

// DiAlternative.java — "one instance" decided by wiring code; dependency visible in the constructor.
final class BillingService {
    private final Config config;
    BillingService(Config config) { this.config = config; }
    String charge(int cents) { return "charged " + cents + " cents in " + config.get("region"); }
}
```

Explaining the parts:

- `EagerConfigStore` — Section 1 of `Main` shows the point: `reports` sets the region and `billing` immediately reads the new value — they share one store without referencing each other.
- `LazyVariants.java` — each constructor sleeps 20 ms to widen the race window and counts its runs; `Main.race(...)` releases eight threads at once with a `CountDownLatch`.
- `HolderSingleton` prints "constructed so far: 0" before first use, proving laziness.
- `Main.attack(...)` tries reflection, a serialization round-trip and `clone()` on each version.

Output:

```
== 1. Eager singleton: every caller shares one store ==
  [EagerConfigStore] reading settings from disk (runs once)
  billing reads region = us-east-1
  after reports.set, billing reads = eu-west-1
  same object? true

== 2-4. Lazy variants hit by 8 threads at the same instant ==
  naive lazy (unsafe)      constructor ran 8 time(s), callers saw 8 distinct object(s)
  synchronized lazy        constructor ran 1 time(s), callers saw 1 distinct object(s)
  double-checked locking   constructor ran 1 time(s), callers saw 1 distinct object(s)

== 5. Bill Pugh holder: nothing built until first use ==
  constructed so far: 0
  [HolderSingleton] constructed on first getInstance()
  constructed now: 1, same? true

== 6. Enum singleton ==
  retries = 5

== 7. Breaking a naive singleton ==
  reflection    -> new object, same? false
  serialization -> same? false
  clone         -> same? false

== 8. The hardened version ==
  reflection    -> blocked: Singleton already exists - use INSTANCE
  serialization -> same? true
  clone         -> blocked: Singleton cannot be cloned

== 9. Enum resists reflection and serialization ==
  reflection    -> blocked: Cannot reflectively create enum objects
  serialization -> same? true

== 10. Subclassable singleton (protected constructor) ==
  LOG chosen subclass = PlainLogger

== 11. DI alternative: one instance created by the composition root ==
  charged 499 cents in ap-south-1
  report for ap-south-1
  in a test: charged 1 cents in test-region
```

(The naive-lazy line is a race, so its count varies run to run — anything above 1 is the bug.)

Trade-offs

Pros	Cons
Controlled access — one guaranteed instance, one documented door.	**Global state** — tends to become shared mutable (changeable) state anything can change.
You choose when it is built — eagerly (simplest) or lazily (cheaper start-up).	**Awkward to test** — swapping in a fake needs a reset/injection hook that then ships in production code, where someone may misuse it.
Right fit for a shared resource — a pool, cache or device handle where a second copy is a bug.	**Invisible dependencies** — a class calling `getInstance()` internally hides that dependency from its constructor, readers and tests.
Setup cost paid once per process instead of per caller.	**Uniqueness ends at the process** (strictly, the class loader) — cannot coordinate across machines.
Subclassing is possible if designed in — `protected` constructor, subclass picked at run time (`AppLogger`). Not with a `private` one.	**Hard to undo** — `getInstance()` calls end up everywhere. It also mixes two jobs (SRP — Single Responsibility Principle): managing its lifecycle *and* its real work.

Often confused with:

Looks like	The actual difference
Static utility class (`Math`)	Only static members — not an object, so it cannot implement an interface or be passed as an argument. A Singleton *is* an object: it can hide behind an interface and tests can substitute another implementation — the main argument for it over a static class.
Prototype	Opposite intents: Singleton caps the count at one; Prototype churns out copies.
DI container singleton scope	Same "one shared instance", better mechanism: the container manages the lifecycle, classes declare needs in constructors, tests swap replacements freely.
Multiton / registry (`Logger.getLogger(name)`)	One instance *per key*, not per class.

In other languages: Python — module top-level code runs once, so a module-level object is shared by every importer (a hand-written `__new__` version has the naive-lazy race and needs a `threading.Lock` with a double check). JavaScript/TypeScript — export an already-constructed object; the single event loop (runs tasks one after another) never interleaves synchronous code, so no locking — but each Web Worker (a background browser thread) has its own memory and copy. Go — a package-level variable initialized via `sync.Once`. C++ — a function-local `static` (the "Meyers Singleton"), created on first call, guaranteed once even under concurrent calls. Java — the single-element `enum` is the shortest thread-safe form and, unlike field-based versions, survives serialization with no extra code.

When not to use it

No creational pattern is misapplied more often.

- **The real motive is convenience.** Reaching an object from anywhere is not a requirement — a constructor or method parameter does it just as well. This explains most singletons that should never have existed.
- **It holds mutable state tests read or write** — tests then share it, giving the test that passes alone but fails in the full suite.
- **A framework already manages lifecycles** (Spring, Guice, NestJS) — declare singleton scope and write no singleton code.
- **More than one process is involved** — four servers, four "singletons". Relying on one for a distributed lock (shared by several machines) or unique IDs is a **correctness bug**, not style.

Common mistakes

- Using the **naive lazy** version in a web server or any multi-threaded code.
- Writing double-checked locking **without** `volatile`.
- Forgetting that `implements Serializable` silently breaks the guarantee unless you add `readResolve()`.
- **Mutable state** without thread-safe structures (`ConfigStore` uses `ConcurrentHashMap` because many threads may `set` concurrently).
- A `public static void reset()` "for tests" — it will be called in production one day.

- Assuming "one instance" means one in the whole system (it is one per JVM / class loader).

Real-world examples

- **JDK** (Java Development Kit — compiler, tools, standard library): `Runtime.getRuntime()` is the textbook case (private constructor, one instance per JVM); `java.awt.Desktop.getDesktop()` also returns one shared instance. `Collections.emptyList()` returns one shared immutable constant — singleton-*like* caching rather than the full pattern. `Logger.getLogger(name)` (java.util.logging) returns one logger per name — a multiton.
- **Not singletons despite `getInstance()`:** `Calendar.getInstance()`, `NumberFormat.getInstance()`, `Cipher.getInstance(...)` return a **new object each call** — static factory methods (8.9). What matters is whether only one instance can ever exist.
- **Spring:** beans are singleton-scoped by default — one per bean definition per `ApplicationContext`, container-managed; "one per container", not "one per JVM".
- **Everyday infrastructure:** connection pools (HikariCP, usually one per data source), metrics registries (Micrometer's global registry), feature-flag clients (switch features on/off without redeploying) — each stands for something that truly exists once. The browser's `window` is a singleton per page.
- **Head First's example:** a chocolate boiler controller — one physical boiler, so two controllers could both fill it and overflow it. Their fix walks the same lazy → synchronized → double-checked / eager progression.

In the interview

> **30-second answer:** "Singleton guarantees one instance per JVM (strictly, per class loader) with a single access point: private constructor, static accessor. I'd use eager initialization — the class loader makes it thread-safe for free — or the holder idiom if construction is expensive. If it must survive serialization or reflection I'd use an enum. But it's global state with hidden dependencies, so in a Spring app I'd just make it a singleton-scoped bean and inject it."

Follow-ups:

- *Q: Why is the naive lazy version broken?* — A: Check-then-act without synchronization; two threads can both see `null` and both construct.
- *Q: Why does DCL need `volatile`?* — A: Without it the reference can be published before the constructor's writes are visible, so another thread may use a half-initialized object. `volatile` gives a happens-before edge between the write and later reads.
- *Q: Why is the holder idiom thread-safe without locks?* — A: The JVM initializes a class once and publishes its static fields safely; the nested class initializes only on first access.
- *Q: How can a singleton be broken?* — A: Reflection, serialization, cloning, multiple class loaders. Fixes: constructor guard, `readResolve`, throwing `clone`, or an enum.
- *Q: Singleton vs static class?* — A: A singleton is an object: it can implement interfaces, be passed around, be lazily created and be mocked (replaced in a test by a fake that records calls). A static class can do none of that.
- *Q: Is Singleton an anti-pattern?* — A: Often *used* as one (glorified global). Fine for genuinely single resources; prefer DI to enforce "one instance" from outside.
- *Q: Unique ID generator across 10 servers?* — A: Not a Singleton — it is one per JVM. Use a database sequence (an increasing counter), a Snowflake-style ID (timestamp + machine id + counter), or UUIDs (128-bit random IDs that practically never repeat).

Self-check

- **Q:** Which single keyword enforces the pattern? **A:** `private` on the constructor.
- **Q:** Which Java variant is lazy, lock-free and thread-safe? **A:** The Bill Pugh holder idiom.
- **Q:** Which variant resists both reflection and serialization with no extra code? **A:** The single-element enum.
- **Q:** What does `readResolve` do? **A:** Replaces the deserialized object with the one you return — the real instance.
- **Q:** In Spring, how many instances of a default bean exist? **A:** One per application context (container), not one per JVM.

TL;DR

Private constructor + static accessor = one instance per JVM. Prefer **eager**, then **holder**, then **enum** (bullet-proof); **DCL needs `volatile`**; never naive lazy. Know the four ways to break it. Mention the costs (global state, hidden deps, tests) and that DI singleton scope is usually the better tool.

8.9 Simple Factory (an idiom, not a GoF pattern)

What it is

A **Simple Factory** is one class (usually one static method) that takes a type value — a string or enum — and returns the matching object, typed as a common interface. Like a vending machine: press "B3", get a snack, never mind how it is picked. Extremely common and often the right answer, but **not** one of the 23 GoF patterns — *Head First* calls it a programming idiom.

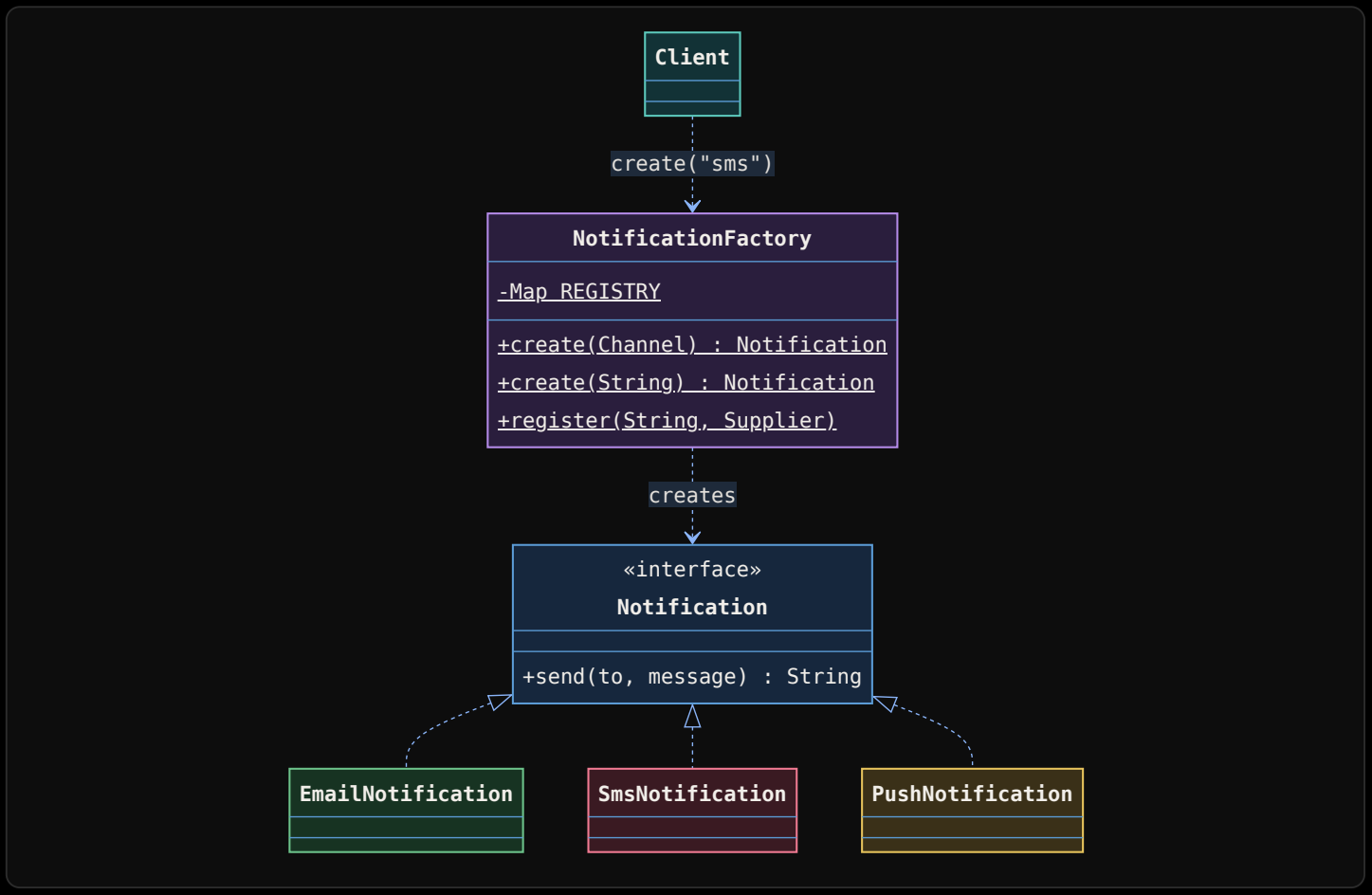
The problem it solves

Without it, the `if/else` or `switch` turning `"sms"` into `new SmsNotification()` is copied everywhere a notification is needed, and a new channel means hunting every copy. A simple factory **centralizes** that decision; callers depend only on the `Notification` interface.

It **centralizes** the decision; it does not **remove** it — a new channel still edits the factory, which still knows every type. The registry version softens this: new types are *registered* at start-up rather than coded into a switch.

How it works

1. Define a product **interface** (`Notification`).
2. Write the concrete classes (`EmailNotification`, `SmsNotification`, `PushNotification`).
3. Write one `create(type)` mapping the type to a class — an exhaustive `switch` over an enum, or a `Map<String, Supplier<Notification>>` registry (a lookup table from names to `Supplier`s; a `Supplier` is Java's built-in "give me a new value" function type).
4. Callers hold the result as the interface and never write `new ConcreteClass()`.
5. Reject unknown types loudly (`IllegalArgumentException`).



Java implementation

Full code: `code/lld/ch08/simplefactory/` (`Notification` with three record implementations, `NotificationFactory`, `Main`).

```java
// NotificationFactory.java (core)
public enum Channel { EMAIL, SMS, PUSH }

/** Version 1: a switch. Adding a channel means editing this method (and the enum). */
public static Notification create(Channel channel) {
    return switch (channel) {                    // exhaustive switch: the compiler checks every enum value
        case EMAIL -> new EmailNotification();
        case SMS   -> new SmsNotification();
        case PUSH  -> new PushNotification();
    };
}

/** Version 2: a registry of suppliers. New types are registered, not coded into a switch. */
private static final Map<String, Supplier<? extends Notification>> REGISTRY = new ConcurrentHashMap<>(Map.of(
        "email", EmailNotification::new, "sms", SmsNotification::new, "push", PushNotification::new));

public static void register(String type, Supplier<? extends Notification> supplier) {
    REGISTRY.put(type.toLowerCase(Locale.ROOT), supplier);
}
public static Notification create(String type) {
    Supplier<? extends Notification> s = REGISTRY.get(type.toLowerCase(Locale.ROOT));
    if (s == null) throw new IllegalArgumentException("Unknown notification type: " + type);
    return s.get();
}

// Main.java: a new channel is registered at start-up - the factory's code is not edited.
NotificationFactory.register("whatsapp", () -> (to, msg) -> "WHATSAPP to " + to + ": " + msg);
```

- The enum `switch` has **no** `default`: the compiler checks every `Channel` is covered, so adding `WHATSAPP` to the enum without handling it is a compile error.
- The registry maps lower-cased names to **constructor references** (`EmailNotification::new` — "call this constructor"). Because `Notification` has one abstract method (a *functional interface*), a lambda can implement it.
- `Locale.ROOT` avoids surprising lower-casing in some locales (language-and-region settings) — the Turkish dotless "i" problem.

Output:

```
== Simple Factory with an enum switch ==
  EMAIL to ada: Your order shipped
  SMS to ada: Your order shipped
  PUSH to ada: Your order shipped

== Simple Factory with a registry (string keys from config) ==
  SMS to +91-98xxxx: OTP 4821
  rejected: Unknown notification type: whatsapp
  WHATSAPP to grace: Table for 2 confirmed
```

Trade-offs

Simple Factory	Factory Method (next section)
One method, usually a switch	The decision is spread across subclasses, one each
Adding a type = edit the factory	Adding a type = add a class
No inheritance needed	Needs a creator subclass per product
Not a GoF pattern	A GoF pattern

Which to use: a short, rarely changing list selected by a configuration value → Simple Factory, shorter and perfectly respectable. Move to Factory Method when *other* developers (perhaps in separate modules) will add types without modifying your class.

When not to use it

- Only **one** concrete class — call the constructor.
- Objects differ only in **data**, not behaviour — one class with a field.

Common mistakes

- Calling a switch-based factory a "Factory Method" in an interview.
- A `default` branch that silently returns `null` or a "default" object instead of failing.
- Letting the factory grow into a God class that also configures, caches and logs.

Real-world examples

These are **static factory methods** (*Effective Java* item 1): named static methods returning an instance, possibly of a hidden subclass. Simple-Factory style — **not** GoF Factory Method (no Creator whose subclasses override a hook) and **not** Singletons (most return a new object per call).

- `Calendar.getInstance()` — a **new** `GregorianCalendar` per call in most locales (a different calendar for a few, e.g. Japanese imperial or Buddhist). The name suggests Singleton; it is not.
- `NumberFormat.getInstance()` / `getCurrencyInstance(locale)` — a fresh (locale-specific, usually `DecimalFormat`) object per call.
- `EnumSet.noneOf(type)` — returns a `RegularEnumSet` (≤ 64 constants, one `long` bit-vector) or a `JumboEnumSet`; you never see either class.
- `List.of(...)`, `Map.of(...)`, `Optional.of(...)`, `Integer.valueOf(...)`, `Path.of(...)`.
- `Executors.newFixedThreadPool(n)`, `Executors.newVirtualThreadPerTaskExecutor()` — return configured `ExecutorService` implementations (Java's interface for running tasks on reusable threads).
- **Spring:** `@Bean` methods (an annotation — a `@` tag a framework reads) in a `@Configuration` class are factory methods the container calls to create beans.

In the interview

> **30-second answer:** "A simple factory is one method that maps a type value to a concrete class and returns it as an interface. It centralizes the switch so callers don't depend on concrete classes. It's an idiom, not a GoF pattern — adding a type still edits the factory. If I want new types without editing it, I use a registry of suppliers or Factory Method."

- *Q: Is it Open–Closed?* — A: Not strictly: the factory is edited per new type. A registry makes it open for extension via registration.
- *Q: Static factory method vs constructor?* — A: Static factories have names, can cache/return existing instances (`Integer.valueOf`), and can return subtypes. Constructors cannot.

Self-check

- **Q:** What does a Simple Factory centralize but not remove? **A:** The switch (knowledge of every concrete type).
- **Q:** Why does the enum `switch` have no `default`? **A:** So the compiler flags any unhandled enum constant.

TL;DR

One `create(type)` method, returns an interface. Great default for small, stable sets of types. Not GoF. Registry of `Supplier`s = extensible variant.

8.10 Factory Method

What it is

Factory Method defines a creation method in a base class but lets **subclasses decide which concrete class** it returns; the base class uses the object without naming its concrete type. Like a franchise manual — take order, *prepare the dish*, serve, bill — where only "prepare" differs per branch (*Head First*'s pizza store: `PizzaStore.orderPizza()` is fixed, each regional store overrides `createPizza()`).

- **Product** — the interface of the created object (`Document`).
- **Concrete product** — an implementation (`PdfDocument`).
- **Creator** — the base class declaring the factory method and holding the shared logic (`ReportGenerator`).
- **Concrete creator** — a subclass overriding the factory method (`PdfReportGenerator`).

GoF intent and participants. Declare an operation that creates an object but leave the concrete class to subclasses — instantiation is postponed to them. **Product**; **ConcreteProduct**; **Creator** (declares the factory

method returning a Product, may give a default, and — crucially — *calls* it from its own operations); **ConcreteCreator** (overrides it to return a ConcreteProduct).

> **Factory Method is not a "static factory method".** A *static factory method* (`List.of`, `Integer.valueOf`, `Calendar.getInstance`) and a *Simple Factory* (`NotificationFactory.create(type)`, 8.9) are **one static method that decides**. They cannot be overridden (static methods are not polymorphic — a call cannot be redirected to a subclass's version at run time), and there is no Creator hierarchy. GoF **Factory Method** is an **instance method, overridden in subclasses**, that the base class itself calls. No inheritance and no overriding → not this pattern.

The problem it solves

Many classes must create an object whose exact type they should not decide (a report generator needs a document, a parser tree nodes). Once `ReportGenerator` contains `new PdfDocument()` it is hard-wired to PDF, and HTML means reopening finished, tested code.

`ReportGenerator` gathers rows, creates a document, renders it, announces the file name — **only "create a document" depends on the format**. A `switch` on a format string holds until a second and third method need the same `switch`, after which each format means hunting every copy. Factory Method leaves a deliberate **gap**: the full procedure lives in the base class, which calls an abstract method at the point it needs a document; each subclass supplies just that method.

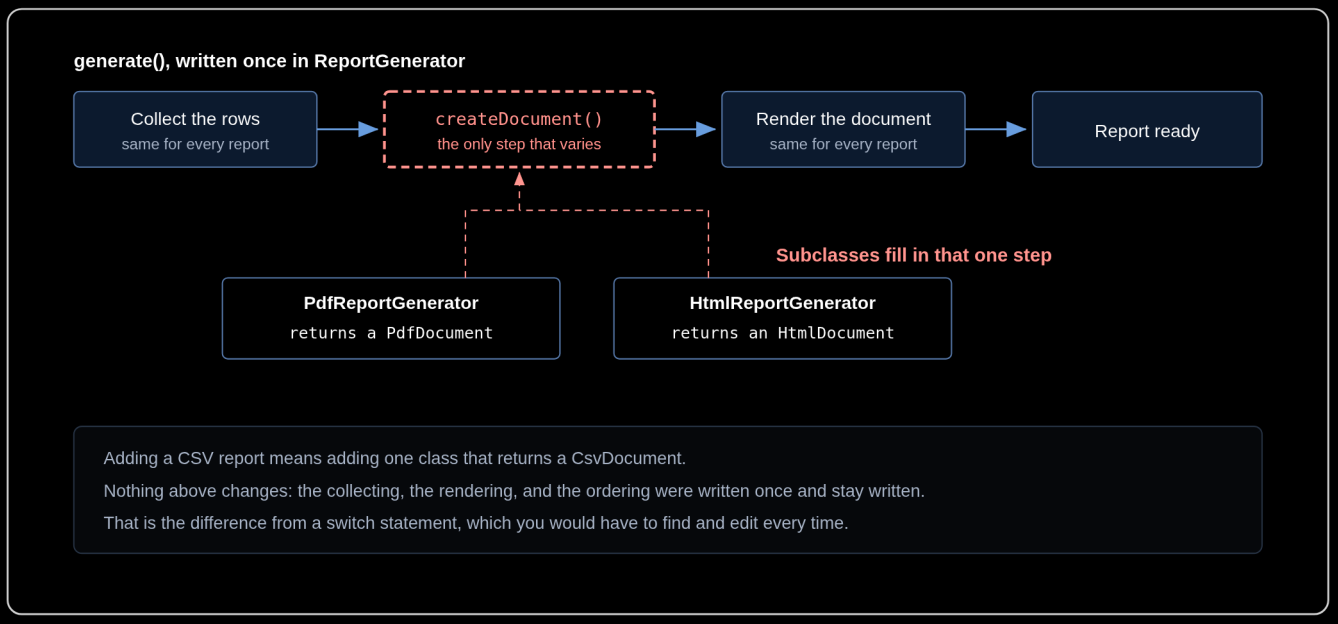


Figure — generate() is written once in ReportGenerator; only createDocument() varies, filled in by PdfReportGenerator or HtmlReportGenerator (course diagram)

Supporting CSV needs only one more subclass returning a `CsvDocument`.

How it works

1. Define the **product interface** (`Document`: `render(rows)`, `fileName()`).
2. Write **concrete products** (`PdfDocument`, `HtmlDocument`, later `CsvDocument`).
3. Write the **creator** as an abstract class with an **abstract, `protected` factory method** (`createDocument()`) — `protected` marks an extension point for subclasses; outside callers call `generate()`.
4. Write the **shared logic** (`generate()`) in the creator, calling `createDocument()`; it depends only on `Document`.
5. Write one **concrete creator** per product overriding only the factory method — the *only* place a concrete type is named.
6. The **client** picks a creator once, then works through `ReportGenerator` and `Document`.

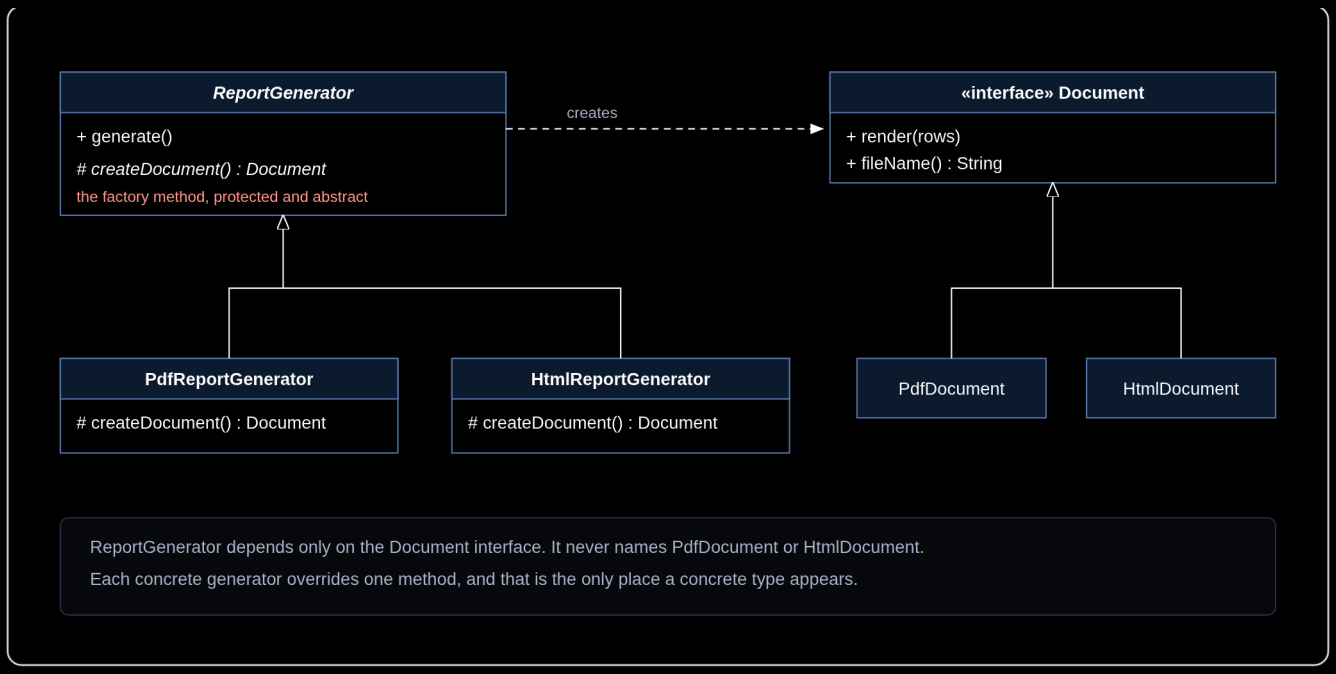
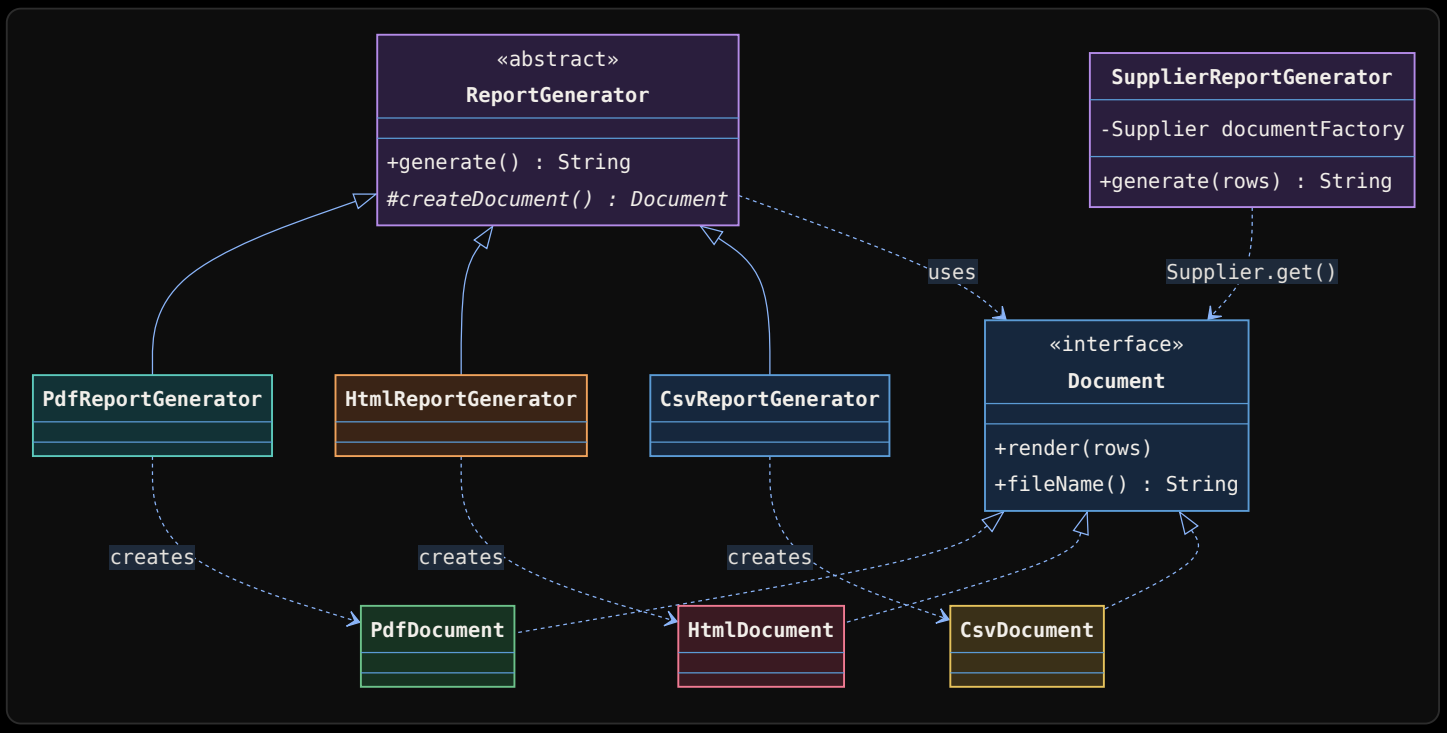


Figure — ReportGenerator declares the protected factory method createDocument(); each concrete generator overrides it to return its own Document (course diagram)



Java implementation

Full code: `code/lld/ch08/factorymethod/` (`Document` + PDF/HTML/CSV products, `ReportGenerator` + three creators, `SupplierReportGenerator`, `Main`).

```java
// ReportGenerator.java — Creator: writes the whole procedure once and leaves one hole.
public abstract class ReportGenerator {
    /** The factory method. Protected: a hook for subclasses, not public API. */
    protected abstract Document createDocument();

    /** The template that USES the factory method. It never names a concrete document. */
    public final String generate() {
        System.out.println("  Collecting rows for the monthly sales report...");
        List<String> rows = List.of("north,120", "south,95", "east,140");
        Document document = createDocument();
        document.render(rows);
        System.out.println("  Report ready: " + document.fileName());
        return document.fileName();
    }
}
final class PdfReportGenerator extends ReportGenerator {
    @Override protected Document createDocument() { return new PdfDocument(); }
}
// HtmlReportGenerator likewise; CsvReportGenerator + CsvDocument "added later" - nothing edited.

// SupplierReportGenerator.java — composition variant: pass the "factory method" in.
public final class SupplierReportGenerator {
    private final Supplier<? extends Document> documentFactory;
    public SupplierReportGenerator(Supplier<? extends Document> documentFactory) { this.documentFactory = documentFactory;
}
    public String generate(List<String> rows) {
        Document d = documentFactory.get();
        d.render(rows);
        return d.fileName();
    }
}
// Main: new SupplierReportGenerator(HtmlDocument::new); a test passes a lambda returning a fake Document.
```

- `generate()` is `final`: subclasses change *what* is created, not the procedure; it never mentions PDF, HTML or CSV.
- `CsvDocument` + `CsvReportGenerator` were added later — **no existing class was edited** (Open–Closed).
- `SupplierReportGenerator` — pass a `Supplier<Document>` instead of subclassing; composes better and a test can supply a lambda.

Output:

```
== PdfReportGenerator ==
  Collecting rows for the monthly sales report...
  Rendering as PDF: 3 rows
  Report ready: sales.pdf
== HtmlReportGenerator ==
  Collecting rows for the monthly sales report...
  Rendering as HTML table: 3 rows
  Report ready: sales.html
== CsvReportGenerator ==
  Collecting rows for the monthly sales report...
  csv> north,120
  csv> south,95
  csv> east,140
  Report ready: sales.csv

== Supplier-based variant (composition instead of subclassing) ==
  Rendering as HTML table: 1 rows
  produced sales.html
  (fake) captured [row-1]
  produced fake.txt
```

The identical "Collecting rows…" line comes from code that exists once, in `ReportGenerator`; only rendering changes.

Trade-offs

Pros	Cons
Extensible without edits — another product is another class; nothing is reopened.	**More classes** — a creator subclass per product type.
Common procedure in one place — everything around creation is written once.	**Indirection** — the concrete type is not visible where used.
Built-in test seam (a spot a test can plug in its own object) — subclass the creator, return a fake.	**Relies on inheritance** — one superclass only; combines less flexibly than passing a function.
Nobody holds the full list of types.	**Tempting too early** — with one product type it is all cost.

Often confused with:

Looks like	The actual difference
Simple Factory	One static `create` with a `switch`; not GoF, often better for small stable sets; gathers the `switch` rather than eliminating it.
Abstract Factory	Factory Method: **one** product, varied by **subclassing** the creator. Abstract Factory: a **family**, varied by **passing in** a different factory object (composition). Abstract Factories are often built from factory methods.
Template Method	Same mechanism (base method calls an overridable hook), different goal: vary *a step of an algorithm* vs vary *which object gets created*. Factory Method is a special case — `generate()` here *is* a template method.
Builder	Factory Method chooses a type in one call; Builder assembles one complex object across many calls.
"Static factory method" (`List.of`)	A named static method instead of a constructor; cannot be overridden, so not this pattern.

In other languages (and modern Java): where functions are values, hand the creator a *creation function* instead of subclassing. Python: `ReportGenerator(create_document=PdfDocument)` (a class is callable). JS/TS: pass `() => new PdfDocument()` (arrow-function syntax); a test passes its own. Java: a `Supplier<Document>` field, as in `SupplierReportGenerator`. Usually the better default (no new class); keep subclassing for framework base classes other teams extend — where the pattern was born.

When not to use it

- **One product type, no second in sight** — call the constructor; introduce the pattern the day a second type appears. The most frequent over-design in the creational group.
- **The base class has no shared work.** If `generate()` only called `createDocument()`, the abstract class would be an interface in costume.
- **Types differ by data, not behaviour** — a `Map<String, Supplier<Document>>` beats a subclass per type.
- **Several objects must match** — Abstract Factory's job.

Common mistakes

- Making the factory method **public** — it joins the API (Application Programming Interface — the methods others are meant to call) and callers bypass the shared procedure.
- Calling the factory method from the base **constructor** — subclass fields are not yet initialized when the override runs.
- Calling a switch-in-one-method a Factory Method. (A learner asked exactly this: one method deciding with `if/switch` = *simple factory*; subclasses overriding a creation hook used by shared base logic = *factory method*.)

Real-world examples

- **JDK:** `Iterable.iterator()` / `Collection.iterator()` — the interface declares the creation method (Creator); `ArrayList`, `LinkedList`, `HashSet` … (ConcreteCreators) return their own `Iterator` (ConcreteProduct); the enhanced `for` loop and code written against `Iterator` (Product) never learn which. `AbstractList` implements `iterator()` on its abstract `get(int)` — template/factory mix.

- **JDK XML:** `DocumentBuilderFactory.newDocumentBuilder()` and `TransformerFactory.newTransformer()` are abstract methods each parser's factory subclass overrides — factory methods inside what is, as a whole, an Abstract Factory (8.11). `DocumentBuilderFactory.newInstance()` itself is a *static* factory method.
- **Spring:** `FactoryBean<T>.getObject()` — a bean that creates another object (proxies, `SqlSessionFactoryBean` in MyBatis-Spring). `AbstractApplicationContext` exposes overridable creation hooks.
- **Frameworks:** Android's `Fragment.onCreateView()`; JUnit/TestNG fixture (shared test setup) base classes with overridable `create…` methods.

In the interview

> **30-second answer:** "Factory Method puts a creation hook in a base class. The base class writes the whole procedure once against a product interface; each subclass overrides the protected factory method to return its concrete product. New types are new classes, so it's Open–Closed. It only pays off when there's genuine shared logic in the base class — otherwise a constructor or a simple factory is simpler. In modern Java I'd often pass a `Supplier` instead of subclassing."

- *Q: Difference from Simple Factory?* — A: Simple Factory = one (static) method with a switch knowing every type. Factory Method = subclasses override an instance hook the base class calls; nothing knows the full list. Raise this yourself — calling a `switch` a "Factory Method" is a classic slip.
- *Q: Difference from Abstract Factory?* — A: One product via inheritance vs a family of matched products via a factory object passed in.
- *Q: Why protected?* — A: An extension point for subclasses, not for callers.
- *Q: How do you test `generate()`?* — A: Subclass the creator in the test (or pass a `Supplier`) returning a fake `Document` that records calls.

Self-check

- **Q:** Where is the only place a concrete product class is named? **A:** Inside each concrete creator's override of the factory method.
- **Q:** What must exist in the base class for this pattern to be worthwhile? **A:** Shared logic that uses the product (here, `generate()`).
- **Q:** Which principle does adding `CsvReportGenerator` demonstrate? **A:** Open–Closed.

TL;DR

Base class = procedure + abstract `protected createX()`; subclasses decide the concrete type. One product, varied by inheritance. Worth it only with shared logic and multiple (growing) product types; `Supplier` is the lightweight modern form.

8.11 Abstract Factory

What it is

Abstract Factory provides an interface for creating **families of related objects** that must be used together, without the client naming any concrete class. Choosing a family = choosing a factory; everything created afterwards matches — like ordering chair, sofa and table from one "Victorian" catalogue. (The course uses one GUI (graphical user interface) factory per OS theme — Windows or macOS — producing matching buttons, checkboxes and scrollbars; *Head First* a pizza *ingredient* factory per region: New York vs Chicago dough, sauce and cheese.)

- **Family** — products designed to work together (Postgres connection + Postgres query builder; Postgres = PostgreSQL, a popular open-source database; a query builder writes the SQL text for you).
- **Abstract product** — an interface per product kind (`Connection`, `QueryBuilder`).
- **Abstract factory** — an interface with one `create…` method per product kind (`StorageFactory`).
- **Concrete factory** — implements *every* create method for one family (`PostgresFactory`).

GoF intent and participants. One interface through which a client creates a whole set of related or interdependent objects without naming their concrete classes. **AbstractFactory** (`StorageFactory`), **ConcreteFactory** (`PostgresFactory`), **AbstractProduct** (`Connection`, `QueryBuilder`), **ConcreteProduct** (`PostgresQueryBuilder`), **Client** (uses *only* abstract types — `ReportService`). GoF notes an application usually needs one ConcreteFactory per family, so it is often made a Singleton (though passing it in is better).

The problem it solves

Some objects are only correct with partners from the same variant. A Postgres connection needs a Postgres-flavoured query builder (Postgres wraps identifiers — table and column names — in double quotes, MySQL in backticks); a macOS button belongs next to a macOS checkbox; a test wants a fake payment client, mailer *and* clock — never a mix.

Constructed independently, nothing prevents a bad combination: `PostgresConnection` + `MySqlQueryBuilder` compiles silently and surfaces later as an SQL (the standard query language for relational databases) syntax error whose stack trace (the method calls active when the error happened) points nowhere near the wrong choice. (Factory Method varies **one** product; Abstract Factory varies a **set** that must stay consistent.)

Abstract Factory gives responsibility for the **entire set to one object**; each concrete factory returns only its own family. *It does not help you choose the right combination — it removes the ability to build a wrong one.*

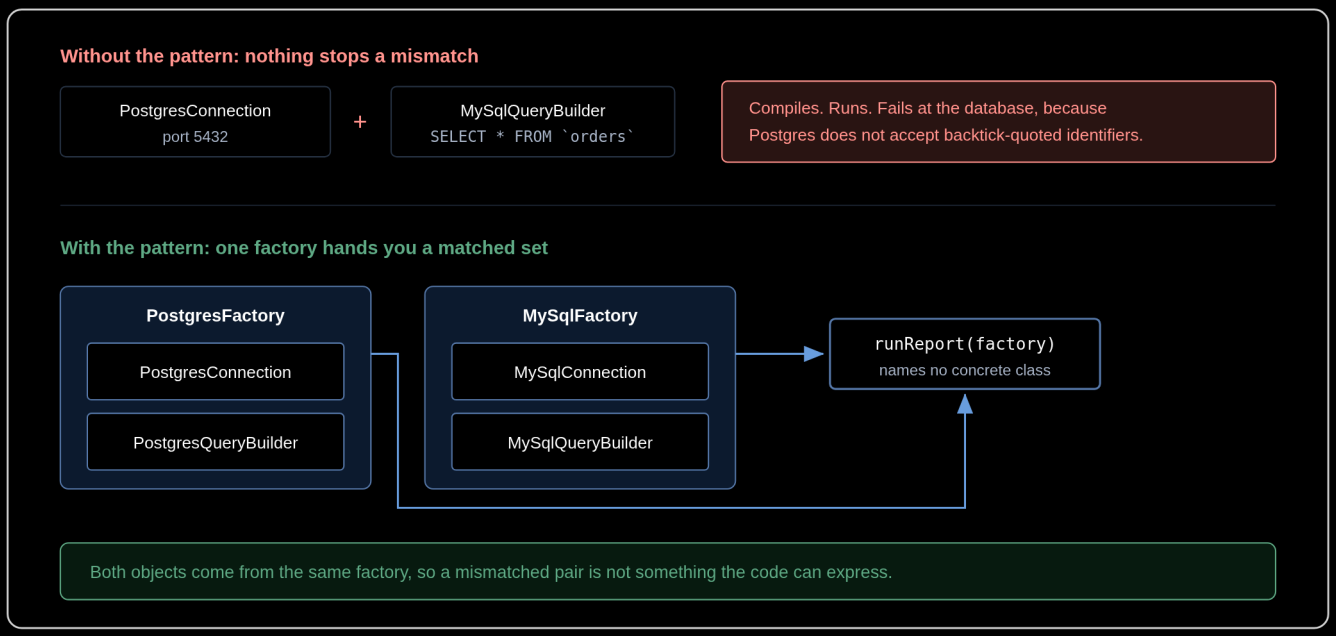


Figure — without the pattern a PostgresConnection can be paired with a MySqlQueryBuilder and fail at the database; with it, each factory hands out a matched set (course diagram)

Top: the code compiles and starts, then Postgres rejects the backtick-quoted table name. Bottom: `runReport(factory)` gets both objects from one factory.

How it works

1. Identify the **product kinds** and give each an interface (`Connection`, `QueryBuilder`).
2. Write the **abstract factory** with one creation method per kind.
3. Per variant, write the **concrete products** and a **concrete factory** implementing *every* method, returning only that family (Java's covariant return types — an override may declare a more specific return type — let `PostgresFactory.createConnection()` declare `PostgresConnection`, documenting the family).
4. The **client** receives a factory (composition, usually via its constructor) and uses only abstract types.
5. Choose the concrete factory **once**, at the **composition root** — the spot near `main` where the application wires its concrete parts — and pass it down.

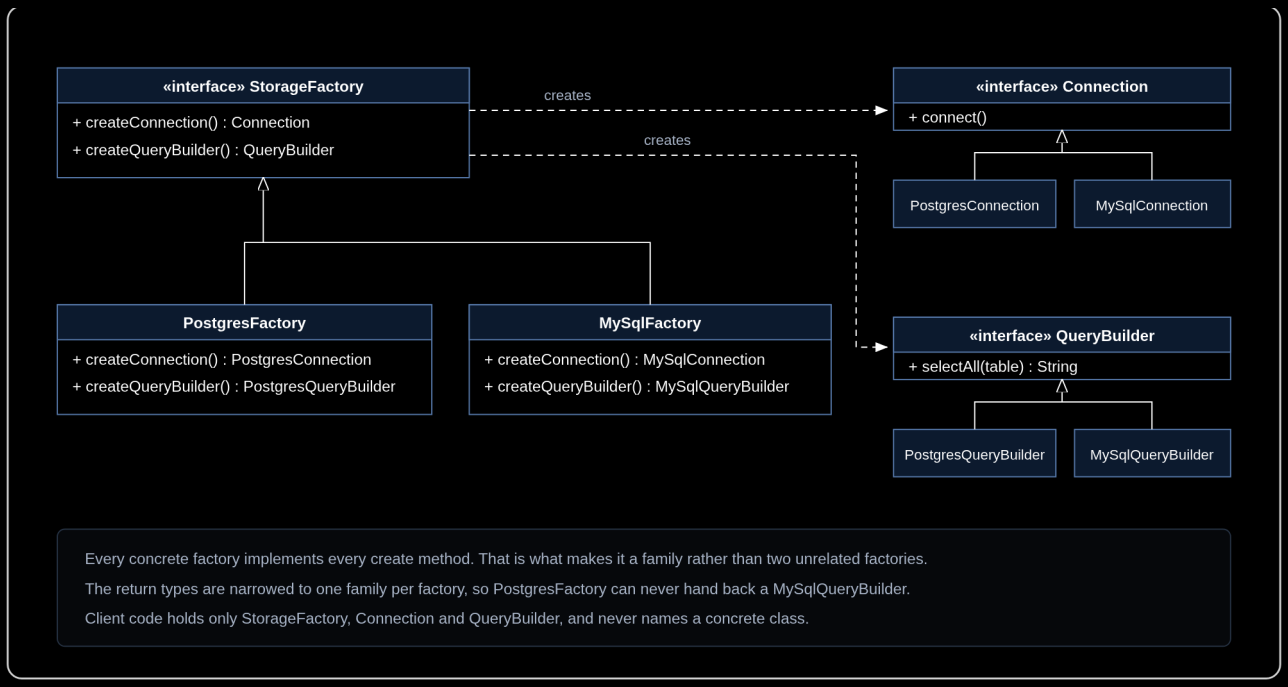
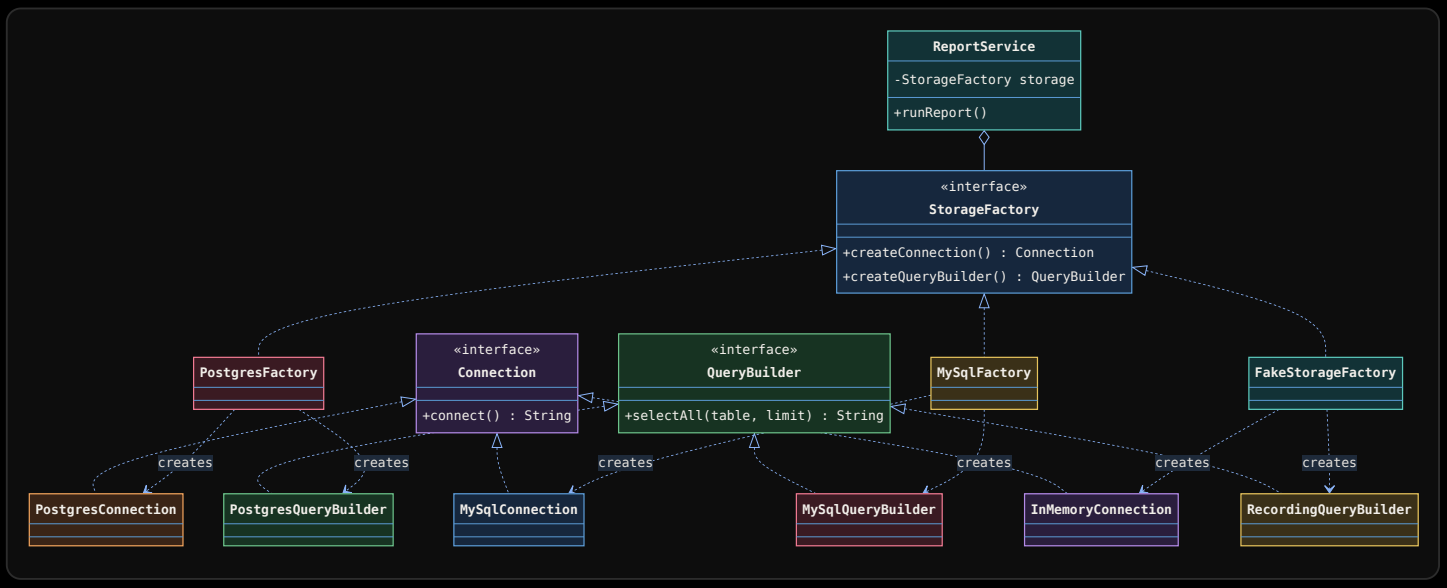


Figure — StorageFactory with createConnection() and createQueryBuilder(); PostgresFactory and MySqlFactory each create their own Connection and QueryBuilder (course diagram)

Every concrete factory implements *every* create method — that makes it a family rather than unrelated factories. (A learner noted an older café example whose factories returned one product each; a correct Abstract Factory returns **all** products of its family.)



The call order: `Main` (composition root) creates `PostgresFactory`, passes it to `new ReportService(factory)`, and `runReport()` asks the same factory for both the connection and the query builder — so they always match.

Java implementation

Full code: `code/lld/ch08/abstractfactory/` (`Products` — Postgres, MySQL and fake families; `StorageFactory` + three factories; `ReportService`; `Main`).

```java
// StorageFactory.java — one creation method per product in the family.
public interface StorageFactory {
    Connection createConnection();
    QueryBuilder createQueryBuilder();
}
/** Concrete factories implement EVERY method and return only their own family (covariant return types). */
final class PostgresFactory implements StorageFactory {
    @Override public PostgresConnection createConnection()     { return new PostgresConnection(); }
    @Override public PostgresQueryBuilder createQueryBuilder() { return new PostgresQueryBuilder(); }
}
// MySqlFactory likewise; FakeStorageFactory returns InMemoryConnection + RecordingQueryBuilder.
// PostgresQueryBuilder quotes with "double quotes"; MySqlQueryBuilder with `backticks`.

// ReportService.java — Client: holds only abstract types. It cannot mix families.
public void runReport() {
    Connection connection = storage.createConnection();
    QueryBuilder queries = storage.createQueryBuilder();
    System.out.println("  " + connection.connect());
    System.out.println("  " + queries.selectAll("orders", 10));
}

// Main.java — Composition root: the ONE place that chooses the family.
static StorageFactory chooseFactory(String db) {
    return switch (db) {
        case "postgres" -> new PostgresFactory();
        case "mysql"    -> new MySqlFactory();
        case "fake"     -> new FakeStorageFactory();
        default -> throw new IllegalArgumentException("unknown db " + db);
    };
}
// main: for db in {env DB or "postgres", "mysql", "fake"}: new ReportService(chooseFactory(db)).runReport();
```

- `ReportService` has **no branch** on the database — the SQL differs purely because the factory differs.
- `chooseFactory(...)` holds the program's only `switch` on the database. Inside `runReport` it would merely be relocated, and every later class needing a connection would grow a duplicate.
- `FakeStorageFactory` is the most valuable use: swap the **whole environment** for tests with one line.

Output (no `DB` environment variable set — a named setting the OS passes to a program):

```
== Using postgres ==
  connected to postgres://localhost:5432/app
  SELECT * FROM "orders" LIMIT 10
== Using mysql ==
  connected to mysql://localhost:3306/app
  SELECT * FROM `orders` LIMIT 10
== Using fake ==
  connected to in-memory store (no network)
  [recorded] selectAll(orders, 10)
```

Trade-offs

Pros	Cons
Wrong combinations cannot be written — the family is fixed when the factory is chosen.	**Many types** — an interface per product kind plus a class per product per family.
One switch for a whole environment — real ↔ fake, Postgres ↔ MySQL, light ↔ dark theme.	**New product kinds are expensive** — the interface grows and every factory is edited.
Client never names a concrete class (Dependency Inversion).	**Another layer to trace** — which factory was picked?
Test seam — one fake factory stands in for everything outside the process.	**Pointless with one family.**

The asymmetry to remember: a new **family** (`SqliteFactory`) is easy and Open–Closed; a new **product kind** (`createMigrationRunner()`) is hard — every factory changes.

Abstract Factory vs Factory Method

	Factory Method	Abstract Factory
Creates	**One** product	A **family** of related products
Mechanism	**Inheritance** — subclass overrides a method	**Composition** — client holds a factory object
Who chooses the concrete type	The subclass of the creator	The concrete factory passed in
Shape	One abstract method in a class that has other logic	An interface whose *only* job is creation (several methods)
Guarantees	New types without editing the creator	Products from one factory are always compatible
Adding a new variant	New creator subclass + new product	New factory + one new product per kind
Adding a new product kind	n/a	Edit the interface and **every** factory
Relationship	—	Each `create…` method in a concrete factory is often itself a factory method
Typical example	`Collection.iterator()`	`DocumentBuilderFactory`, UI toolkits, DB dialects

Often confused with: *Builder* (Abstract Factory returns several finished products at once; Builder assembles one step by step); a *DI container* (same outcome, more general — Spring profiles or Guice modules bind a whole set as configuration; prefer it if you have one); a *Facade* over factories (simplifies an interface vs guarantees consistency — similar shape, unrelated purpose).

In other languages: in Python or JavaScript a dictionary/object of creation functions — `{"connection": PostgresConnection, "queries": PostgresQueryBuilder}` — already is a factory. In Java, a `record` (short syntax for an immutable data class) of `Supplier`s (`record StorageKit(Supplier<Connection> connection, Supplier<QueryBuilder> queries)`) is often shorter than interface + class per family. With a DI container, hand-write the pattern only when building the framework.

When not to use it

- **The objects do not belong together** — `createLogger()` + `createInvoice()` is a grab-bag, not a matched set; inject them separately.
- **Only one variant** and none expected.
- **Product kinds keep growing** (beyond roughly five or six) — each edits every factory; a container or registry scales better.
- **Your DI container already does it** (Spring `@Profile`, Guice modules, NestJS providers) — that *is* this pattern as configuration.

Common mistakes

- A concrete factory implementing only some methods, or returning another family's products.
- Choosing the factory deep in the call stack (the chain of calls leading to the current one) instead of once at the composition root.
- Making the factory a Singleton by reflex — pass it in.

Real-world examples

- **JDK:** `javax.xml.parsers.DocumentBuilderFactory`, `javax.xml.transform.TransformerFactory`, `XPathFactory` — clients hold the abstract factory; static `newInstance()` locates one implementation (the JDK's built-in Xerces/Xalan-based ones, or Saxon if configured), whose factory hands out objects of the same implementation (`TransformerFactory` creates both `Transformer` and `Templates`). JDBC's (Java Database Connectivity) `java.sql.Connection` is a factory for a driver-specific family (a driver is the library for one database): `createStatement()`, `prepareStatement()`, `prepareCall()`, `createBlob()`.
- **UI toolkits:** the original GoF example — a factory per look-and-feel; Swing's pluggable look-and-feel follows it.
- **ORMs** (object-relational mappers, mapping classes to tables): Hibernate's `Dialect` bundles SQL generation, type mapping and identifier quoting that must agree.

- **Spring:** `@Profile("test")` vs `@Profile("prod")` swap a whole set of beans (fake mailer, fake payments, in-memory DB) together.
- **Cloud SDKs** (a provider's client libraries): a multi-cloud abstraction switching AWS vs GCP storage + queue + secrets clients per provider.

In the interview

> **30-second answer:** "Abstract Factory is an interface with one create method per product in a family; each concrete factory returns a matched set. The client holds only the factory and the abstract products, so it can't mix, say, a Postgres connection with a MySQL query builder. I pick the factory once at the composition root. The cost is that adding a new product kind means editing every factory."

- *Q: Factory Method vs Abstract Factory?* — A: One product via inheritance vs a family via composition; Abstract Factory often uses factory methods internally.
- *Q: How would you add SQLite?* — A: `SqliteConnection`, `SqliteQueryBuilder`, `SqliteFactory`, one line in the composition root. Nothing else changes.
- *Q: How would you add a `MigrationRunner` product?* — A: Add `createMigrationRunner()` to the interface and every factory — the pattern's weak spot.
- *Q: How is this used in tests?* — A: A fake factory returns in-memory/recording products; one line swaps the entire outside world.

Self-check

- **Q:** What property does Abstract Factory guarantee? **A:** Products from one factory are always from the same family — mismatches cannot be expressed.
- **Q:** What is the composition root? **A:** The one place near start-up where concrete choices (like which factory) are made and passed down.
- **Q:** Which change is easy and which is hard? **A:** New family = easy; new product kind = edit every factory.

TL;DR

One factory object → a matched family of products. Chosen once, passed down. Easy to add families, hard to add product kinds. Prefer a DI container's profiles if you already have one.

8.12 Builder

What it is

Builder separates the *construction* of a complex object from the object itself: a builder collects values **one named step at a time** and a final `build()` produces the finished — usually **immutable** — product. GoF adds that the same construction process can create different representations. Like ordering at Subway (or the course's fast-food counter): bread → cheese → vegetables → sauce, each named, many optional, the sandwich only at the end.

- **Product** — the object being built (`HttpRequest`).
- **Builder** — collects the values and has `build()`.
- **Fluent interface** — each method returns `this` so calls chain: `.method("POST").timeoutSeconds(5)`.
- **Director** — optional class that knows a *recipe* (the order of steps) and runs it on a builder.
- **Immutable** — cannot change after creation (all fields `final`, no setters).

Two different "Builders" share the name — know both:

	GoF Builder (1994)	**Bloch's Builder (*Effective Java*, item 2)**
Problem it targets	One *construction process* should produce *different representations* (GoF: one RTF reader converting a document to plain ASCII, TeX, or an editable text widget)	A constructor with many optional parameters (telescoping constructors) and the wish for an immutable, validated object
Participants	**Builder** (abstract interface, one method per part, e.g. `buildParagraph()`), **ConcreteBuilder** (implements parts for one representation, keeps the result, `getResult()`), **Director** (runs the steps through the Builder interface), **Product** (type can differ per ConcreteBuilder)	One product class plus a `static` nested `Builder`: fluent setters returning `this`, `build()` that validates and calls the product's private constructor. No Director, no abstract Builder
What varies	*Which* ConcreteBuilder is plugged in — same steps, different output type/format	Only the *values* the caller supplies
Examples	The GoF RTF converter; a report exporter whose one traversal drives a PDF, HTML or CSV builder	`HttpRequest.newBuilder()`, `Locale.Builder`, Lombok `@Builder`, this chapter's `HttpRequest.Builder`

Most code is Bloch's fluent builder; interviews often expect you to *name* the GoF participants too. Shared core insight: separate step-by-step assembly from the finished object.

The problem it solves

A class gathers fields, most optional, until a call site reads:

```
new HttpRequest(url, "GET", headers, null, 30, 3, true, false, null);
```

Which `null` is the body? What does the first `true` switch on? Swap the two booleans and it compiles, surfacing in production as, say, requests that follow redirects when they should not. The two usual "fixes" are both bad:

Approach	**What goes wrong**
Telescoping constructors — one overload per useful combination: `Pizza(size)`, `Pizza(size, crust)`, `Pizza(size, crust, cheese)`, …	Overloads explode with every field; call sites stay positional and unreadable; same-typed parameters are easy to swap.
JavaBeans style — empty constructor + setters (methods that each change one field)	The object exists **half-built**; nothing stops use before it is ready; it **cannot be immutable**; no single place to validate rules spanning several fields.

Builder fixes both: values **named** at the call site, optional fields **defaulted**, product **immutable**, and `build()` is **one gate** that refuses invalid objects. Our rule "a GET must not carry a body" (HTTP — the web's request/response protocol, see System Design Fundamentals — gives a GET body no defined meaning, RFC 9110 §9.3.1, and many servers and proxies (forwarding middlemen) reject or drop it) spans `method` and `body`, so no single setter can enforce it — `build()` can.

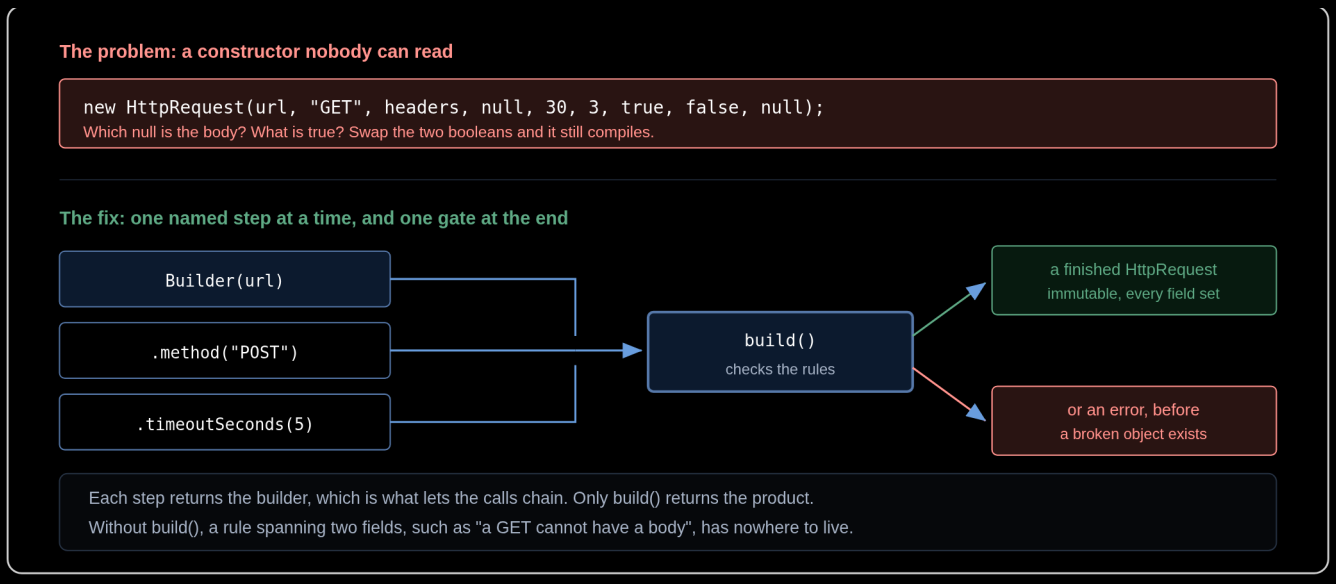


Figure — the unreadable positional constructor versus named builder steps that end in one build() gate which validates and returns an immutable HttpRequest or an error (course diagram)

Every step returns the builder (enabling chaining); only `build()` returns a product — or throws, so an invalid request never exists.

How it works

1. Product fields `final`, constructor `private` taking the builder — no half-filled product can be made directly.
2. A `static` **nested** `Builder` with the same fields, mutable, optional ones **defaulted**.
3. **Required** values in the builder's constructor (or `builder(url)`) so they cannot be forgotten.
4. Each setter-like method **named for what it sets**, `return this`.
5. `build()`: validate cross-field rules, **copy mutable collections**, call the private constructor. It alone returns the product.
6. Optionally a **Director** for reusable recipes, `toBuilder()` for copy-and-modify, or a **staged builder** so the compiler enforces required steps.

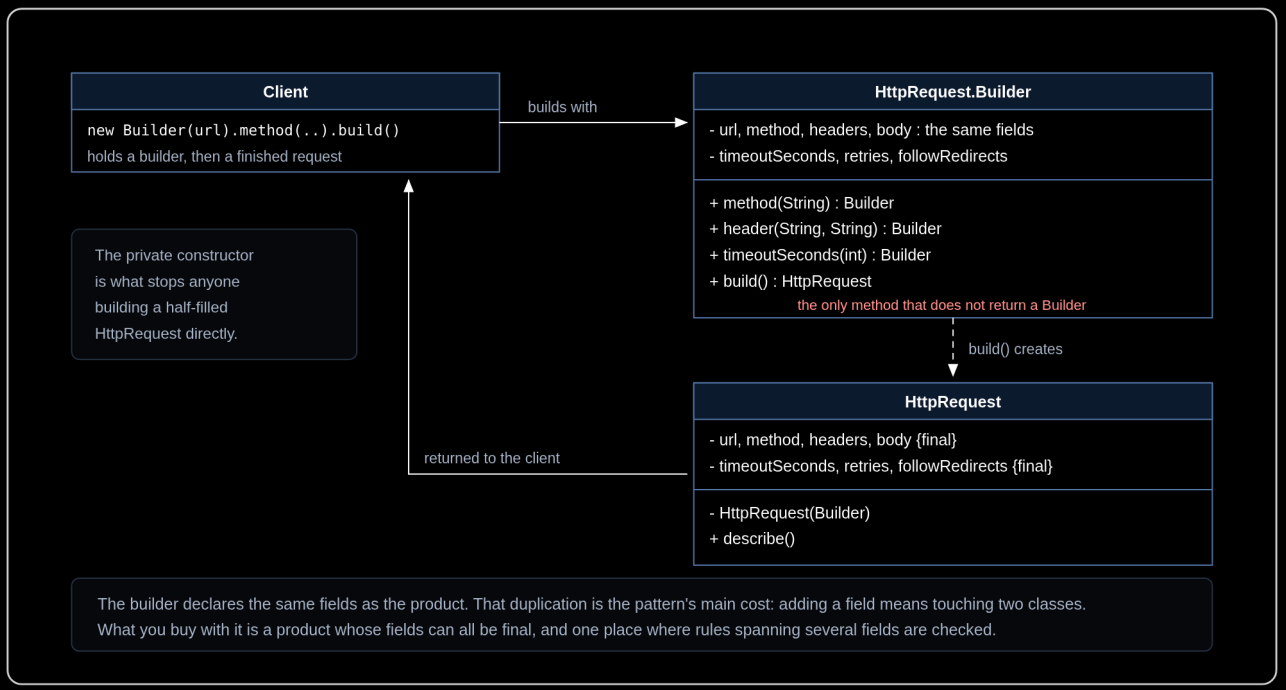
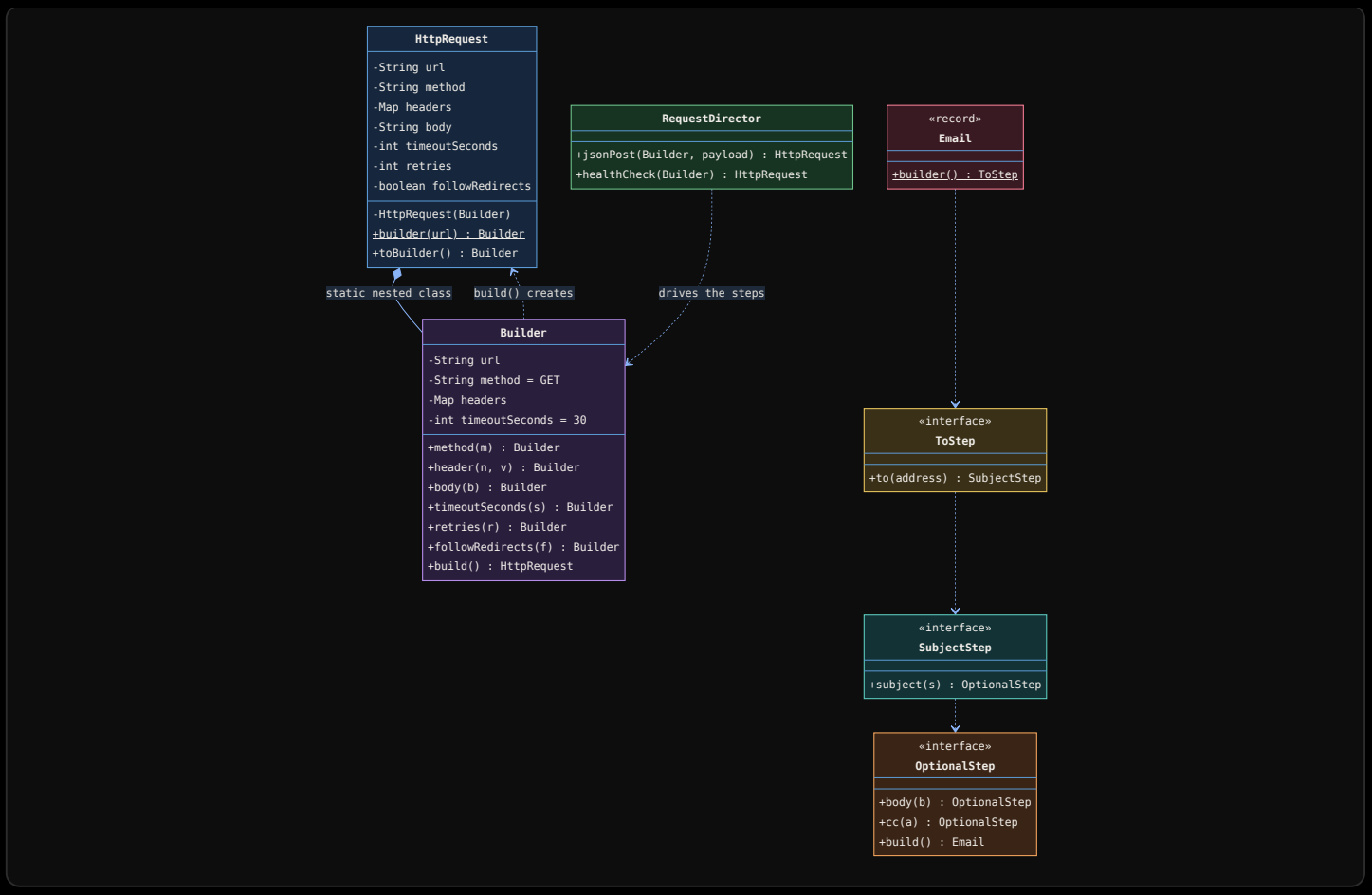


Figure — HttpRequest with final fields and a private constructor; HttpRequest.Builder with the same mutable fields, setters returning Builder, and build() returning HttpRequest (course diagram)

The price is visible: every field is repeated in the builder (two edits per new field). In exchange the product has no setters, all fields `final`, and one validation point.



The Director — and why you rarely see one

The GoF **Director** knows the **order of steps** and runs them on whichever builder it is given, making a *sequence* reusable — "a JSON POST with our usual headers and timeout" deserves a name. (GoF: one RTF reader drives builders outputting plain text or TeX — same process, different representations.)

Day to day you see the **fluent** builder, because what varies between call sites is usually *values*, not order — and then a Director adds nothing a chained expression could not. **Quick test:** will this Director ever get a *different* kind of builder? If yes, keep it; if not, remove it. (Our `RequestDirector` only accepts `HttpRequest.Builder`, so it shows only the "named recipe" half; a full GoF Director is typed against an abstract Builder with several implementations.)

The staged (step) builder

When steps are required or ordered, encode it in **types**: each step returns an interface exposing only the legal next calls. `Email.builder()` returns `ToStep` (only `to(...)`) → `SubjectStep` (only `subject(...)`) → `OptionalStep`, which finally has `build()`. `Email.builder().build()` **does not compile** — the answer to "how would you stop a caller forgetting a required field?". Heavier than most code needs.

Lombok-style builders and records

- **Lombok** `@Builder` (Lombok writes Java boilerplate at compile time) generates a static `builder()`, a nested builder with one method per field, and `build()`. `@Builder(toBuilder = true)` adds `toBuilder()`; `@Singular` generates `header(name, value)`-style collection adders. It does **not** validate by default — add validation in the constructor it calls.
- **Java records** give the immutable product for free; record + static nested builder (like `Email`) is now common. Validate in the record's compact constructor (no parameter list, runs on every creation) so it runs however the record is created.
- **Immutability checklist:** `final` class or private constructor, `final` fields, no setters, defensive copies (private copies so outside code cannot change your internals) of mutable inputs (`Map.copyOf`, `List.copyOf`), getters that do not leak mutable internals.

Java implementation

Full code: `code/lld/ch08/builder/` (`TelescopingPizza` — the anti-example, five constructors for five fields; `HttpRequest`; `RequestDirector`; `Email`; `Main`).

```java
// HttpRequest.java — immutable product; only the nested Builder can create one.
public final class HttpRequest {
    private final String url, method, body;
    private final Map<String, String> headers;
    private final int timeoutSeconds, retries;
    private final boolean followRedirects;

    private HttpRequest(Builder b) {
        this.url = b.url;  this.method = b.method;
        this.headers = Map.copyOf(b.headers);      // defensive, unmodifiable copy
        this.body = b.body;  this.timeoutSeconds = b.timeoutSeconds;
        this.retries = b.retries;  this.followRedirects = b.followRedirects;
    }
    public static Builder builder(String url) { return new Builder(url); }
    /** Lombok-style toBuilder(): a new builder pre-filled from this object. */
    public Builder toBuilder() {
        Builder b = new Builder(url).method(method).timeoutSeconds(timeoutSeconds)
                .retries(retries).followRedirects(followRedirects);
        headers.forEach(b::header);
        if (body != null) b.body(body);
        return b;
    }

    public static final class Builder {
        private final String url;                                  // required -> constructor
        private String method = "GET";                             // optional -> defaults
        private final Map<String, String> headers = new LinkedHashMap<>();
        private String body;
        private int timeoutSeconds = 30;
        private int retries = 3;
        private boolean followRedirects = true;

        private Builder(String url) { this.url = Objects.requireNonNull(url, "url"); }
        public Builder method(String m)           { this.method = m; return this; }
        public Builder header(String n, String v) { headers.put(n, v); return this; }
        public Builder body(String b)             { this.body = b; return this; }
        // timeoutSeconds(int), retries(int), followRedirects(boolean) likewise

        /** The single gate: cross-field validation happens here, before any product exists. */
        public HttpRequest build() {
            if (url.isBlank()) throw new IllegalStateException("url is required");
            if (method.equals("GET") && body != null) throw new IllegalStateException("a GET request cannot have a body");
            if (retries < 0) throw new IllegalStateException("retries cannot be negative");
            if (timeoutSeconds <= 0) throw new IllegalStateException("timeout must be positive");
            return new HttpRequest(this);
        }
    }
}
```

```java
// RequestDirector.java — a named, reusable RECIPE (order of steps).
public HttpRequest jsonPost(HttpRequest.Builder builder, String payload) {
    return builder.method("POST").header("Content-Type", "application/json")
            .header("Accept", "application/json").body(payload).timeoutSeconds(5).build();
}
public HttpRequest healthCheck(HttpRequest.Builder builder) {
    return builder.method("HEAD").timeoutSeconds(1).retries(0).followRedirects(false).build();
}

// Email.java — staged builder: the compiler forces "to" and "subject" before build() is visible.
public record Email(String to, String subject, String body, List<String> cc) {
    public Email { cc = List.copyOf(cc); }                    // records + builder = immutable product
    public interface ToStep      { SubjectStep to(String address); }
    public interface SubjectStep { OptionalStep subject(String subject); }
    public interface OptionalStep { OptionalStep body(String body); OptionalStep cc(String address); Email build(); }
    public static ToStep builder() { return new Steps(); }   // Steps implements all three interfaces
}
// Email.builder().build();   // does not compile: ToStep has no build()
```

- `TelescopingPizza("large", "thin", true, false, true)` hides which boolean means what.

- `Map.copyOf(b.headers)`: without it, building twice from one builder hands both products **the same map**, and a later header appears in the earlier product. The reuse demo proves `first` did not pick up `X-Debug`. `Map.copyOf` is also unmodifiable, so `headers().put(...)` throws.
- `build()` enforces four rules, including the cross-field one (GET + body); `toBuilder()` gives Lombok-style "same as this, but…" copies.

Output:

```
== The problem: telescoping constructors ==
  Pizza[size=large, crust=thin, cheese=true, olives=false, jalapenos=true]

== Fluent builder: every value is named ==
  GET https://api.example.com/orders headers={Accept=application/json} timeout=30s retries=3 followRedirects=true
  POST https://api.example.com/orders headers={Content-Type=application/json} body={"item":"widget"} timeout=5s retries=0
followRedirects=false

== build() refuses broken objects ==
  rejected: a GET request cannot have a body

== Reusing a builder does not leak state into earlier products ==
  first headers  = {X-Trace=1}
  second headers = {X-Debug=true, X-Trace=1}

== Product is immutable ==
  headers().put -> UnsupportedOperationException

== toBuilder(): copy-and-change, like Lombok @Builder(toBuilder = true) ==
  POST https://api.example.com/orders headers={Content-Type=application/json} body={"item":"widget"} timeout=60s retries=0
followRedirects=false

== Director: named recipes over any builder ==
  POST https://api.example.com/payments headers={Accept=application/json, Content-Type=application/json} body=
{"amount":499} timeout=5s retries=3 followRedirects=true
  HEAD https://api.example.com/health headers={} timeout=1s retries=0 followRedirects=false

== Staged builder: the compiler enforces required steps ==
  Email[to=ada@example.com, subject=Welcome, body=Glad to have you, cc=[team@example.com]]
```

Trade-offs

Pros	Cons
Self-describing call sites — each value labelled; parameter order no longer matters.	**Duplicated fields** — two edits per new field.
Immutable products — fields fixed before the object exists; safe to share between threads.	**Forgettable last step** — omit `build()` and you hold a builder.
A home for validation of multi-field rules in `build()`.	**Easily overused** — for a three-field value object (defined only by its values, like `Money`) it is noise.
Free optional fields — defaults in the builder; callers mention only changes.	**Reuse pitfall** — two products sharing one mutable collection unless copied.
Different representations (GoF) — a Director drives different ConcreteBuilders through the same steps.	One extra short-lived object per construction — almost never measurable.

Often confused with:

Looks like	The actual difference
Factory Method	Chooses *which type*, in one call. Builder assembles *one known type* over many calls.
Abstract Factory	Several finished products straight away vs one product after a sequence of steps.
Fluent interface	Just a calling style. A chain that modifies and returns the *same* object (a jQuery chain, a `StringBuilder`-style mutator used as the result) has no separate finished product, so it is not Builder by itself.
Prototype	Starts from a ready-made object and duplicates it; Builder starts from nothing. Both avoid a huge constructor.

In other languages: Python, Kotlin and C# have **named arguments with defaults** — `HttpRequest(url, method="POST", timeout=5)`; Python's `@dataclass(frozen=True)` and Kotlin's `data class` (with `.copy(...)`) give the immutable product. Use a builder there only for cross-field validation or staged construction. JS/TS usually pass an options object: `new HttpRequest({ url, method: "POST" })`.

When not to use it

- **A few fields, all required** — a plain constructor; the most common misuse.
- **The language has named/default arguments.**
- **The builder merely copies fields**, validates nothing, and the product is mutable anyway — add validation or remove it.
- **The object is mutable and lives inside one method** — setters are fine.

Common mistakes

- Setters that **don't return the builder** (breaks chaining); only `build()` returns the product.
- **No validation** in `build()`.
- **Sharing mutable collections** between builder and products.
- A public product constructor (bypasses the builder and its validation).
- A Director when the order never varies.

Real-world examples

- **JDK:** `StringBuilder` (append repeatedly, `toString()` once); `java.net.http.HttpClient.newBuilder()` and `HttpRequest.newBuilder()` (Java 11+); `Locale.Builder`; `Calendar.Builder` (Java 8+); `Stream.builder()`; `Thread.ofVirtual().name("w").start(task)` — `Thread.Builder` in Java 21; `ProcessBuilder`.
- **Spring:** `UriComponentsBuilder`, `ResponseEntity.ok().header(...).body(...)`, `WebClient.builder()`, `MockMvcRequestBuilders.get("/x").param(...)`, Spring Security's `HttpSecurity` DSL (domain-specific language — a mini-language for one job).
- **Libraries:** OkHttp `Request.Builder`; Protocol Buffers (immutable messages + generated builders); Guava `ImmutableList.builder()`; Lombok `@Builder`; test-data builders like `aUser().withEmail("x@y.com").build()`.

In the interview

> **30-second answer:** "When a constructor has many optional parameters, I use a Builder: required values in the builder's constructor, one named method per optional field that returns the builder, and `build()` that validates cross-field rules and creates an immutable product through a private constructor. I'd mention the classic Director but usually skip it because only values vary. For tiny objects or languages with named arguments, I wouldn't bother."

- *Q: Why not setters?* — A: Half-built objects, no immutability, nowhere for cross-field validation.
- *Q: How do you enforce required fields?* — A: Builder constructor, validation in `build()`, or a staged builder so the compiler enforces them.
- *Q: Is `StringBuilder` a GoF Builder?* — A: Not the full structure (no Director, one representation), but the everyday example of the idea: accumulate parts, produce the `String` once.
- *Q: Is the built object thread-safe?* — A: The product is, if immutable. The builder is not — do not share it across threads.

Self-check

- **Q:** What is the only builder method that does not return the builder? **A:** `build()`.
- **Q:** What bug does copying the headers map in `build()`/the constructor prevent? **A:** Two products built from one builder sharing (and mutating) the same map.
- **Q:** When is a Director worth it? **A:** When the same recipe (order of steps) is reused, possibly with different builders.

TL;DR

Named steps + defaults + one validating `build()` → immutable product. Solves telescoping constructors and half-built JavaBeans. Director optional; staged builder for compile-time required steps; Lombok/records remove the boilerplate.

8.13 Prototype

What it is

Prototype creates new objects by **copying an existing, already-configured object** (the prototype) instead of building from scratch. The object offers a `copy()` method, so the caller need not know its concrete class — like photocopying a contract template legal drafted once and changing only the name and date.

- **Prototype** — the original object that gets copied.
- **Clone / copy** — the new object produced.
- **Shallow copy** — copies the fields; a reference field in the copy points to the *same* object.
- **Deep copy** — copies the fields *and* the objects they point to, so nothing mutable is shared.
- **Copy constructor** — a constructor taking another instance of the same class and copying it.
- **Prototype registry** — a map of named, pre-configured prototypes you ask for copies of.

GoF intent and participants. Use a ready-configured instance as the model and make new objects by copying it. **Prototype** (declares the self-copy operation — `Shape` with `copy()`), **ConcretePrototype** (implements it — `Circle`, `Rectangle`, `Group`, or `NotificationTemplate`, which plays both roles), **Client** (asks a prototype to copy itself — `Main`, or `ShapeRegistry`, which GoF calls a *prototype manager*).

The problem it solves

1. **Construction is expensive** — loading a file, hitting a database, parsing a template, filling dozens of defaults. A `NotificationTemplate` parsed from disk at start-up, where each welcome email differs only in subject and one channel: re-parsing per recipient repeats the slowest work for nothing.
2. **You do not know the concrete class** — the more important case. Code holding a `Shape` or `Template` needs *one more of the same* but cannot write `new` without a class name. The object always knows its own class, so asking it to copy itself needs no factory or `switch`, and the copy starts from already-correct values.

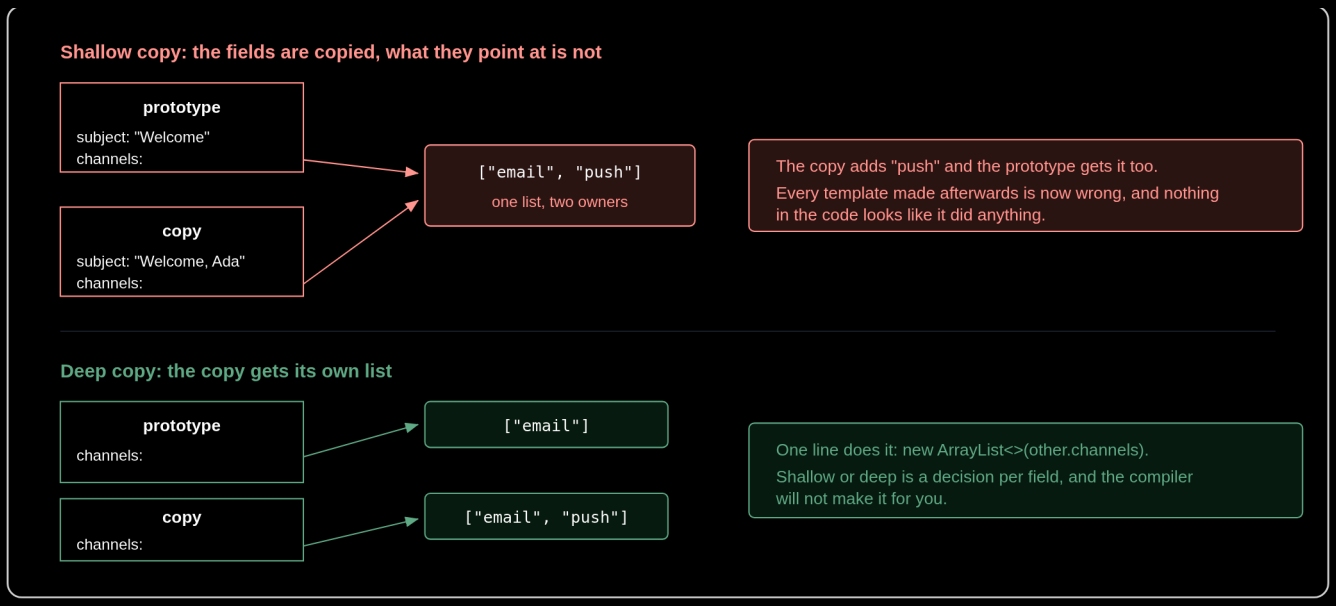


Figure — a shallow copy shares the channels list with the prototype, so adding "push" to the copy corrupts the prototype; a deep copy gets its own list (course diagram)

The one decision that needs thought: **how far down the object graph (every object reachable through references) the copy reaches.** The structure is simple: an ordinary constructor doing the expensive work, and a copy path doing none of it.

How it works

1. Two ways in: the **normal constructor** (expensive setup) and a **copy path** (none of it).
2. Implement the copy path, preferably as a **private copy constructor** `NotificationTemplate(NotificationTemplate other)`.
3. Per field: immutable → share it (`String`, records of immutables); mutable → copy it (`new ArrayList<>(other.channels)`), recursing for nested mutable objects.
4. Expose a public **`copy()`** returning a new, independent instance.
5. If clients hold several unrelated types, add a **prototype interface** (`Shape { Shape copy(); }`) for polymorphic copying.
6. Optionally a **registry** of named prototypes that always hands out copies, never the original.

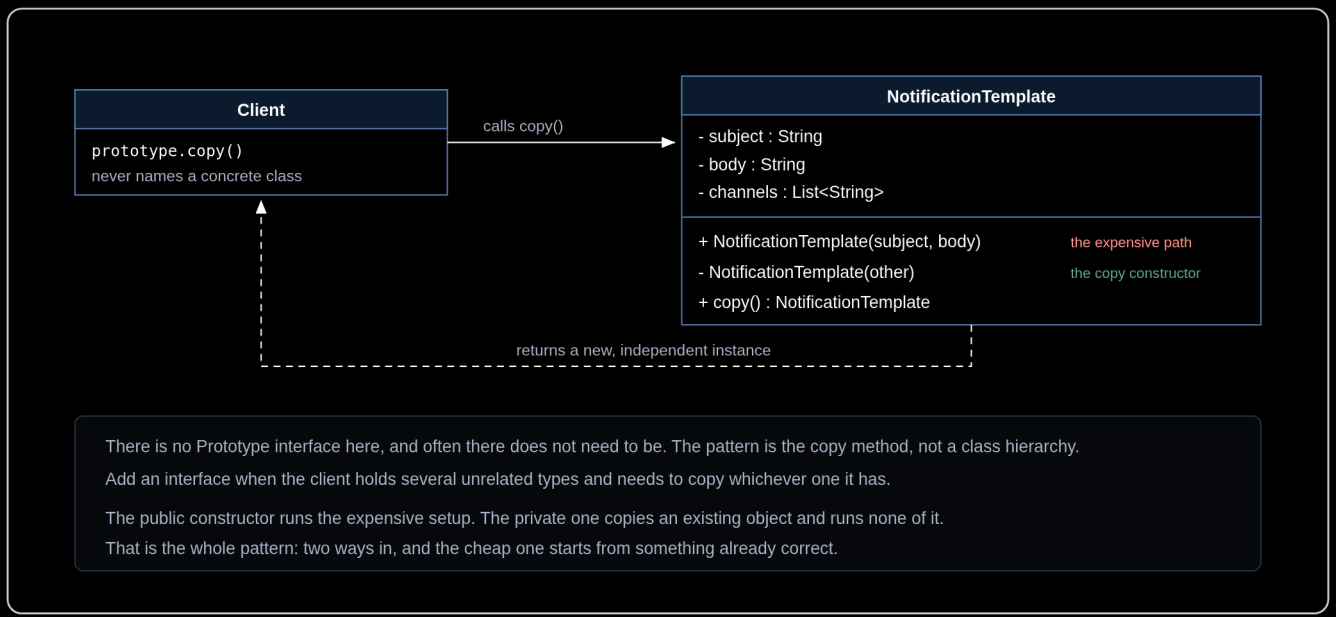
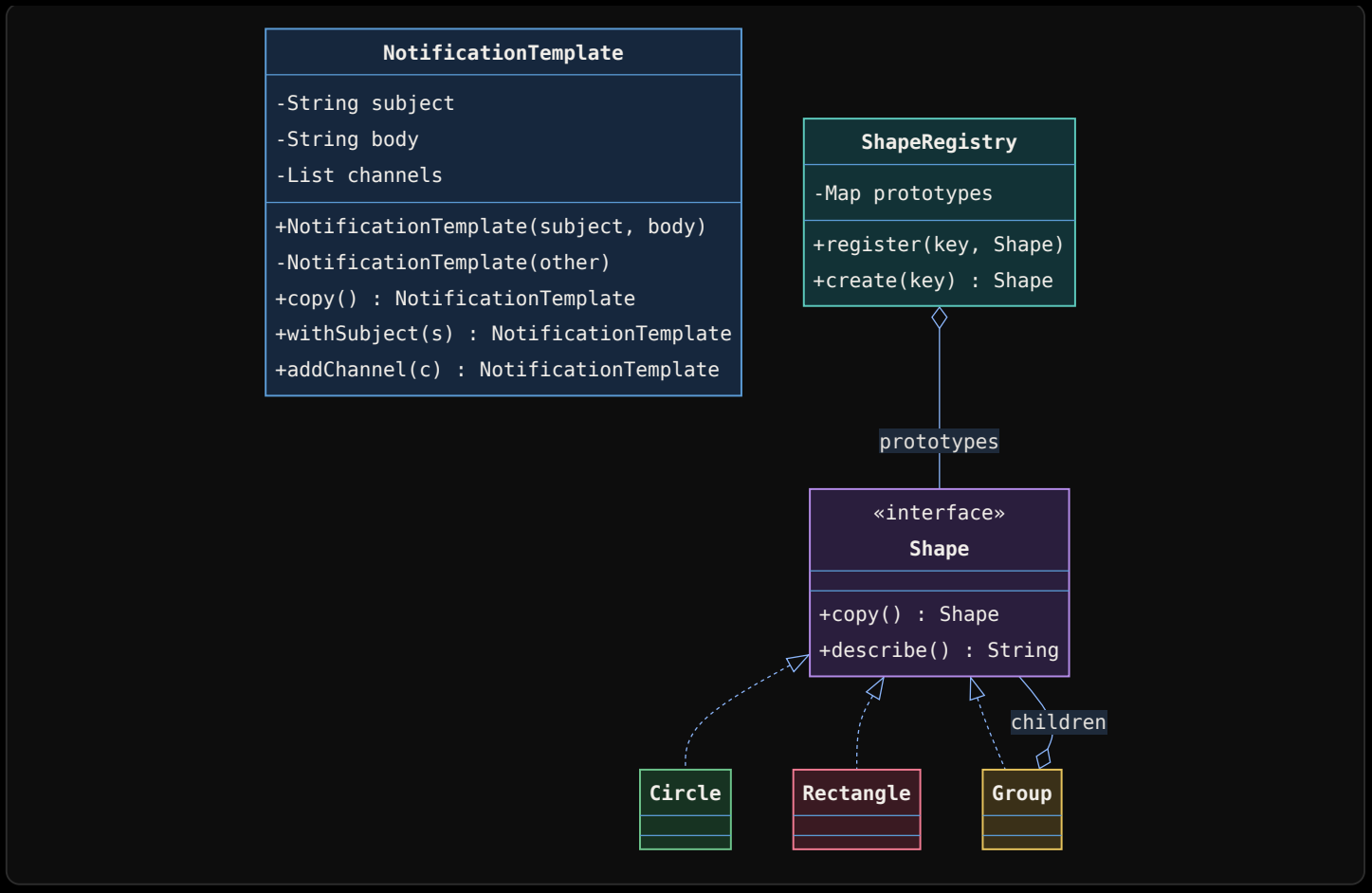


Figure — NotificationTemplate is both prototype and product: a public expensive constructor, a private copy constructor, and copy() that the Client calls (course diagram)

There is **no separate creator class and no Prototype interface** here — the product creates its own copies; the copy method, not a hierarchy, makes it Prototype. Add an interface (like `Shape`) only when the client must copy whichever of several unrelated types it holds.



Shallow vs deep copy

	Shallow copy	Deep copy
Primitives (`int`, `double`, `boolean`…), `String`, other immutables	Copied / safely shared	Copied / safely shared
Mutable fields (lists, maps, objects)	**Shared** — both objects point to the same one	**Duplicated** — each has its own
Cost	Cheap	Proportional to the size of the object graph
Risk	Changing the copy silently changes the original	Cycles, copying things you meant to share (caches), copying resources
Java default	`Object.clone()`	You write it (per field)

Had the copy constructor written `this.channels = other.channels`, adding "push" to the copy would add it to the prototype — and **every later copy would inherit the stray channel**, though no visible line touched the prototype. This shared-mutable-collection bug is the one you will meet most. The cure is one line per mutable field (`new ArrayList<>(other.channels)`); mutable elements would need copying too, and so on down. **Copy depth is a design choice made field by field**; Java will not make it for you.

Three related traps:

- **A new field added without updating the copy** — silently dropped; no compiler error, rarely a test, so it drifts unnoticed for months.
- **Copying something that owns a resource** (connection, file handle) — two objects think they own it and both close it.
- **Cycles** — naive recursive deep copy of mutually referencing objects never terminates; real utilities track visited objects.

Why Java's `Cloneable` is a trap

Java's built-in Prototype, `Object.clone()` + `Cloneable`, is widely considered broken (*Effective Java*, item 13):

- `Cloneable` is a **marker interface with no methods** — it does not declare `clone()`; it only changes `Object.clone()`'s behaviour (without it, `clone()` throws `CloneNotSupportedException`).
- `Object.clone()` is **`protected`** — you must override it as `public`.
- It makes a **shallow** field-by-field copy.
- It **creates the object without running any constructor**, skipping validation and invariants (rules that must always hold).
- Deep-copying means reassigning fields after `super.clone()` — **impossible for `final` fields**.
- It throws a **checked** exception (one the compiler forces you to catch or declare) even when it cannot happen.

Prefer a copy constructor or static copy factory (`static Foo copyOf(Foo other)`): ordinary code, runs your validation, works with `final` fields, behaviour visible. Exception: `array.clone()` works well for primitive arrays.

Java implementation

Full code: `code/lld/ch08/prototype/` (`NotificationTemplate`, `Shapes` — `Circle`, `Rectangle`, `Group`; `ShapeRegistry`; `CloneablePitfall` — `ShallowPlaylist`/`DeepPlaylist`; `Main`).

```
// NotificationTemplate.java — one expensive path, one cheap copy path.
public NotificationTemplate(String subject, String body) {        // the expensive path
    parses++;
    System.out.println("  Parsing template from disk... (expensive)");
    this.subject = subject;
    this.body = body;
    this.channels = new ArrayList<>(List.of("email"));
}
private NotificationTemplate(NotificationTemplate other) {         // the copy path: no parsing
    this.subject = other.subject;                    // String is immutable -> sharing is safe
    this.body = other.body;
    this.channels = new ArrayList<>(other.channels); // NEW list -> the copy owns its own
}
public NotificationTemplate copy() { return new NotificationTemplate(this); }
```

```
// Shapes.java — prototype interface lets code copy an object whose class it does not know.
interface Shape { Shape copy(); String describe(); }

final class Circle implements Shape {
    private Circle(Circle other) { this(other.radius, other.color); }   // copy constructor
    @Override public Circle copy() { return new Circle(this); }         // covariant return
    ...
}
final class Group implements Shape {           // composite prototype: deep copy must copy children
    private final List<Shape> children = new ArrayList<>();
    @Override public Group copy() {
        Group g = new Group();
        for (Shape s : children) g.add(s.copy());       // recursive deep copy
        return g;
    }
    ...
}

// ShapeRegistry.java — prototype manager: named prototypes handed out as copies.
Shape create(String key) {
    Shape p = prototypes.get(key);
    if (p == null) throw new IllegalArgumentException("no prototype named " + key);
    return p.copy();                                    // never hand out the original
}
```

```
// CloneablePitfall.java — Object.clone() is shallow, so mutable fields are shared.
class ShallowPlaylist implements Cloneable {
    String name;
    List<String> songs = new ArrayList<>();
    @Override public ShallowPlaylist clone() {
        try { return (ShallowPlaylist) super.clone(); }   // 'songs' reference is shared
        catch (CloneNotSupportedException e) { throw new AssertionError(e); }  // cannot happen
    }
}
// DeepPlaylist.clone(): copy = (DeepPlaylist) super.clone(); copy.songs = new ArrayList<>(songs);
// re-assigning is why 'songs' cannot be final.
```

- "Parsing template from disk" prints **once** though three templates exist; the prototype keeps only `[email]` — each copy owns its list.
- `ShallowPlaylist`: editing the clone changed the original. `DeepPlaylist` fixes it only because `songs` is not `final`.
- `Circle.copy()` returns `Circle` (covariant) — handy when you know the type; `Shape` callers still get a `Shape`.
- `Group.copy()` deep-copies a composite; `ShapeRegistry.create()` returns `p.copy()`, so callers cannot corrupt the stored prototype.

Output:

```
== Copy instead of re-parsing ==
  Parsing template from disk... (expensive)
  ada       : subject="Welcome, Ada" channels=[email, push]
  grace     : subject="Welcome, Grace" channels=[email, sms]
  prototype : subject="Welcome" channels=[email]   <- unchanged (deep copy of the list)
  templates parsed: 1

== Cloneable pitfall: shallow copy shares the list ==
  original songs after editing the CLONE: [Song A, Song B]   <- bug
  deep clone -> original [Song A], copy [Song A, Song B]

== Polymorphic copy: the caller only knows 'Shape' ==
  copied a Circle: red circle r=5, same object? false

== Prototype registry ==
  blue rectangle 120x40
  grey circle r=24
  group[gold circle r=8, gold rectangle 30x10], two badges distinct? true
```

Trade-offs

Pros	Cons
Costly setup paid once — the prototype parses/loads; each copy is cheap.	**Every class carries copy code** — out of date as soon as a field is added and the copy forgotten.
Create without naming a class — the only option when you hold an interface.	**Shallow vs deep decided per field** — a wrong call makes objects secretly share state.
New kinds at run time — configured prototypes registered and copied without a class each (fewer subclasses).	**Cyclic graphs** need care or a naive deep copy recurses forever.
Saved starting points — users keep a template, filter or layout to start from.	**Hazardous for objects with identity** — primary keys (unique DB row IDs), handles, connections must not be duplicated.
	Uneven language support — some languages have no built-in cloning and Java's (`Cloneable`) is flawed, so you write copy logic yourself.

Often confused with:

Looks like	The actual difference
Factory Method	Fresh object from nothing, type chosen by a subclass. Prototype duplicates an instance, so the type comes along automatically.
Builder	Assembles from caller-supplied parts; Prototype adjusts a finished object.
Copy constructor	The usual *implementation*; becomes the pattern when callers use a polymorphic `copy()` without knowing the type.
Object Pool	Lends **existing** objects and takes them back (identity reused); Prototype produces **new** objects (identity never shared).
Spring `prototype` scope	Despite the name, Spring constructs a **new instance** on every `getBean()` — no cloning.
JavaScript prototypes	Prototypal inheritance (the prototype chain) is a language feature for sharing behaviour, not this pattern.

In other languages: Python has `copy.copy()` / `copy.deepcopy()` and `dataclasses.replace(obj, subject="x")`. JavaScript's spread `{...original, subject: "x"}` is *shallow* (same trap); `structuredClone()` deep-copies plain data. Kotlin/Scala `data class` gives `.copy(subject = "x")`. C++ has the copy constructor built in; the "rule of zero" means the compiler-generated one copies every member correctly if the class owns no raw resources. In Java a `record` is a natural copy point (all components available to a copy constructor or `withX(...)`); still prefer a hand-written copy constructor to `Cloneable`.

When not to use it

- **Creating from scratch is cheap** — a constructor is clearer.
- **The object has identity** — copying something holding a DB primary key, socket or open file yields an object that *looks* fine but is not; two rows sharing one ID is far worse than a slow constructor.
- **The object graph is large** — a deep copy may cost more than rebuilding and duplicate things meant to be shared (a cache).
- **The object is immutable** — just share the instance.

Common mistakes

- Shallow-copying a mutable collection (the classic bug).
- Implementing `Cloneable` and trusting `super.clone()`.
- Forgetting to update the copy constructor when adding a field.
- Handing out the registry's original instead of a copy.

Real-world examples

- **JDK:** `Object.clone()` and arrays' `clone()`; `ArrayList.clone()` and `HashMap.clone()` (documented **shallow** — elements not copied); copy constructors everywhere: `new ArrayList<>(collection)`, `new HashMap<>(map)`, `new TreeMap<>(sortedMap)`, `new EnumMap<>(enumMap)`; `List.copyOf(...)` for immutable snapshots.
- **Game engines:** Unity's `Instantiate(prefab)` clones a configured "prefab" — load model, stats and animations once, copy for each of hundreds of enemies or bullets.
- **Products:** "Duplicate" in Google Docs/Slides/Figma, "Save as template" / "Use template" in Notion or Jira, saved search filters and report layouts.
- **Tests:** a valid base fixture copied and nudged per test.

In the interview

> **30-second answer:** "Prototype creates objects by copying a configured instance through a `copy()` method on the object. It saves expensive setup and — the deeper reason — it lets me create an object when I only hold an interface. The main decision is shallow vs deep copy per field; sharing a mutable list is the classic bug. In Java I'd write a copy constructor rather than use `Cloneable`, and I'd refuse to copy anything with identity like a DB id or a connection."

- *Q: Why avoid `Cloneable`?* — A: No `clone()` in the interface, protected `Object.clone()`, shallow by default, skips constructors, cannot deep-copy `final` fields, checked exception.
- *Q: How do you deep-copy a graph with cycles?* — A: An identity map original → copy; check it before copying and reuse an existing copy.
- *Q: Prototype vs Spring prototype scope?* — A: Spring's scope means "new instance per request for the bean" — it constructs, it does not clone.
- *Q: Serialization-based deep copy?* — A: Possible (serialize then deserialize) but slow, needs everything `Serializable`, bypasses constructors — a last resort.

Self-check

- **Q:** What is the difference between shallow and deep copy? **A:** Shallow copies references (mutable parts are shared); deep also copies what they point to.
- **Q:** Why can't `DeepPlaylist` make `songs` final? **A:** `clone()` must reassign it after `super.clone()`, which is impossible for a final field.
- **Q:** What should a prototype registry return? **A:** A copy of the stored prototype, never the original.

TL;DR

Copy a configured object instead of building one — saves setup and works when you only know the interface. Decide shallow vs deep per field. Copy constructors, not `Cloneable`. Never copy things with identity.

8.14 Bonus — Object Pool

What it is

An **Object Pool** keeps a **bounded set of reusable objects**: a client **borrows** one (`acquire`), uses it and **returns** it (`release`) for the next client, instead of create–use–discard. Not one of the GoF 23, but a well-known creational pattern. Like a bike-sharing dock: fixed bikes, take one, ride, dock it; if all are out you wait (or give up after a while).

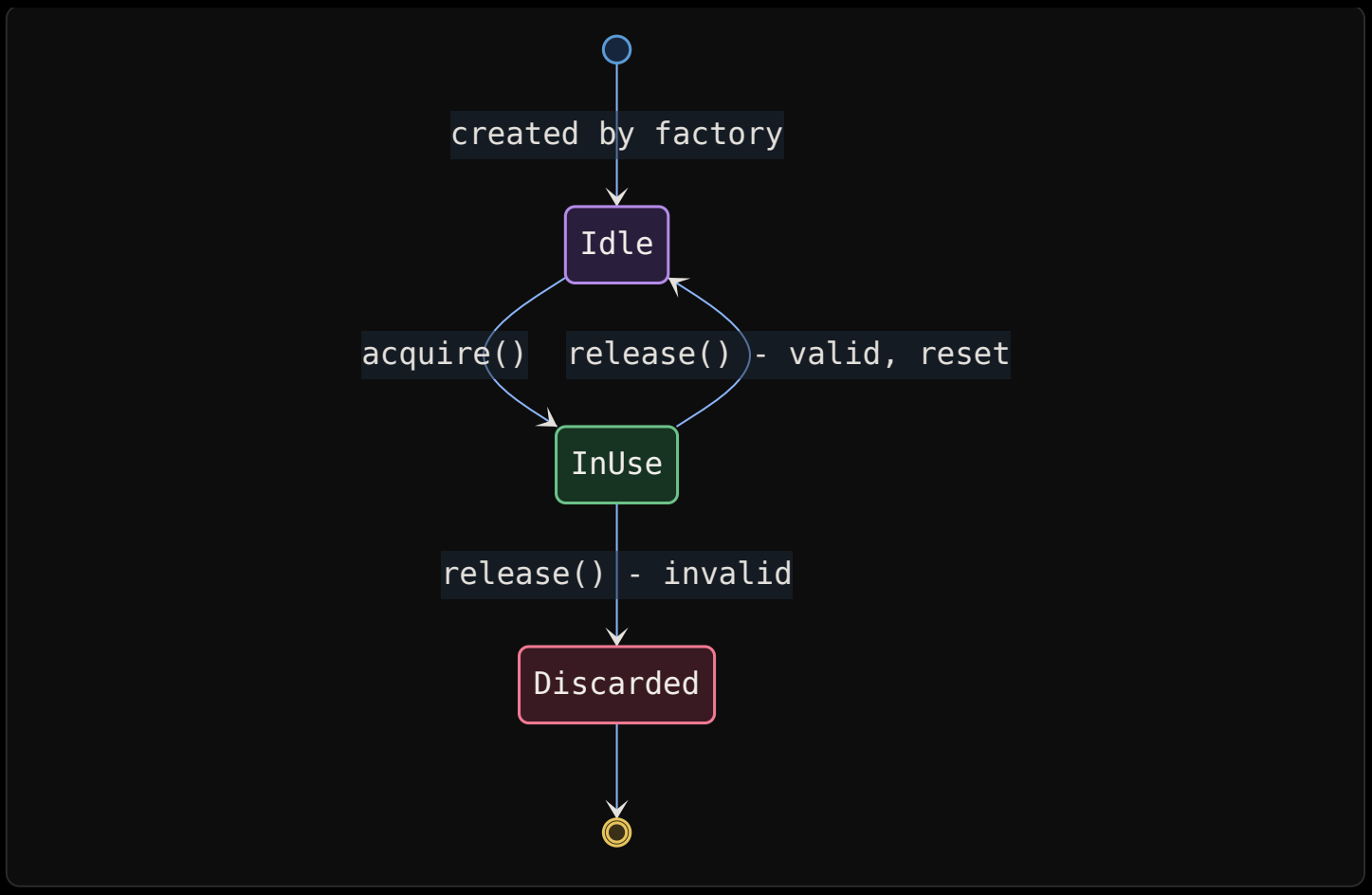
The problem it solves

Some objects are **expensive to create** (a database connection needs a TCP handshake — the exchange that opens a network connection — then TLS, the encryption behind `https`, then authentication; see System Design Fundamentals) and/or **limited** (the database accepts only so many connections). One per request is slow and can overwhelm the server. A pool pays creation a few times and **caps** the total.

How it works

1. Keep an **idle queue** (first-in, first-out) of ready objects, a count of how many are alive, and a cap on how many may be lent at once (here a `Semaphore` — a counter of permits a thread takes before going on and hands back afterwards — one permit per object).
2. **`acquire()`**: wait (with a timeout) for a free slot; take an idle object if any, else create one. No slot before the timeout → fail.
3. The client **always releases in a `finally` block** (runs even if an exception was thrown) — a leaked (never returned) object is gone from the pool forever.
4. **`release()`**: **validate** (connection still alive?); if broken, discard so a fresh one can be made later; else **reset** per-borrower state and put it back. Either way, free the slot.
5. Make it all **thread-safe** — pools exist *because* threads compete.





Java implementation

Full code: `code/lld/ch08/objectpool/` (`ObjectPool`, and `Main` with a `FakeConnection` stand-in).

```java
// ObjectPool.java — bounded, thread-safe; one Semaphore permit per object that may be lent out.
public final class ObjectPool<T> {
    private final Semaphore permits;
    private final ConcurrentLinkedQueue<T> idle = new ConcurrentLinkedQueue<>();
    private final Supplier<T> factory;        // how to make a new object
    private final Consumer<T> reset;          // how to clean an object before reuse
    private final Predicate<T> isValid;       // is a returned object still usable?
    private final AtomicInteger created = new AtomicInteger();    // alive (idle + borrowed)

    public ObjectPool(int maxSize, Supplier<T> factory, Consumer<T> reset, Predicate<T> isValid) {
        if (maxSize <= 0) throw new IllegalArgumentException("maxSize must be positive");
        this.permits = new Semaphore(maxSize, true);              // fair: waiters served in order
        this.factory = factory; this.reset = reset; this.isValid = isValid;
    }

    public T acquire(long timeout, TimeUnit unit) throws InterruptedException {
        if (!permits.tryAcquire(timeout, unit)) {
            throw new IllegalStateException("pool exhausted: timed out after " + timeout + " " + unit);
        }
        T obj = idle.poll();                      // holding a permit = within the cap
        if (obj != null) return obj;
        try {
            obj = factory.get();
            created.incrementAndGet();
            return obj;
        } catch (RuntimeException e) {
            permits.release();                    // creation failed: give the permit back
            throw e;
        }
    }

    public void release(T obj) {
        try {
            if (isValid.test(obj)) { reset.accept(obj); idle.offer(obj); }
            else created.decrementAndGet();       // discard it; its permit still comes back
        } finally {
            permits.release();                    // wakes one waiting borrower, if any
        }
    }
}
// Main: pool of 3 FakeConnections (reset c.queriesRun = 0; valid if !c.broken); 10 tasks on 6 threads,
// each does acquire(2, SECONDS) ... finally { pool.release(c); }
```

- The `Semaphore` enforces the cap: a borrower needs a permit first (`tryAcquire(timeout)` waits with a deadline), so at most `maxSize` objects are ever out however many threads race; holding one means "reuse idle, or it is safe to create".
- `release()` returns the permit in `finally` **even when discarding** — a waiting borrower wakes and can create a replacement. (Waiting only on the idle queue would leave it stuck until timeout though capacity is free.) A failed factory call also returns its permit, so failures never shrink the pool.
- Handing an object between threads through `ConcurrentLinkedQueue` (and the semaphore) also makes the previous borrower's writes visible to the next.
- Validation + reset on `release()` keep broken or dirty objects out. 10 tasks on 6 threads with a pool of 3 open only 3 connections.

Output:

```
== 10 tasks on 6 threads share at most 3 connections ==
  (opening a new connection - expensive)
  (opening a new connection - expensive)
  (opening a new connection - expensive)
  tasks finished: 10, connections ever created: 3, distinct connections used: 3

== A broken connection is discarded, not reused ==
  created after discarding: 2, idle: 2

== Exhaustion: borrowers wait, then time out ==
  (opening a new connection - expensive)
  pool exhausted: timed out after 100 MILLISECONDS
  after returning all: idle = 3
```

Trade-offs

Pros	Cons
Avoids repeated expensive creation	Complexity: sizing, validation, timeouts, leak detection
Caps resource usage (protects the DB)	Leaked (never released) objects shrink the pool until it deadlocks (everyone waits forever)
Predictable latency (time per request) after warm-up	Stale state if reset is incomplete — one user's data leaks to the next
	For cheap objects it is **slower** than `new` — modern JVM allocation and GC (garbage collection — automatic memory clean-up) are very fast

When not to use it

- **Cheap** objects (small value objects, `StringBuilder`s) — the JVM allocates faster than a pool hands out.
- **Virtual threads** (Java 21; lightweight JVM-managed threads, cheap enough to create by the million) — create one per task instead of pooling.
- A **well-tested library** exists — HikariCP, Apache Commons Pool 2 or your framework's pool, not your own in production.

Common mistakes

- Not releasing in `finally` (or not using try-with-resources — Java's `try (...)` that closes things automatically — wrappers).
- Not resetting borrower state.
- Unbounded waits — always use a timeout.
- Double release. In our teaching pool it would put one object in the idle queue twice *and* add a permit, silently raising the cap; production pools track borrowed objects and reject a second release.

Real-world examples

- **Database connection pools:** HikariCP (Spring Boot's default), Apache DBCP, c3p0.
- **Thread pools:** `ThreadPoolExecutor` / `Executors.newFixedThreadPool` reuse worker threads.
- **Apache Commons Pool 2** `GenericObjectPool` (used by Jedis for Redis connections).
- **Netty** `PooledByteBufAllocator` pools buffers to reduce GC pressure.
- Not pools, though often confused: the `String` intern pool and the `Integer` cache (-128..127) share immutable instances — **Flyweight** (Chapter 9); nothing is borrowed or returned.

In the interview

> **30-second answer:** "An object pool reuses a bounded set of expensive objects like DB connections. acquire takes an idle one, creates one if under the cap, or waits with a timeout; release validates, resets and returns it. It must be thread-safe, and clients must release in finally. For cheap objects it's counter-productive, and in production I'd use HikariCP."

- *Q: How do you size a pool?* — A: From measured concurrency and downstream limits (capacity of the services you call, e.g. the database); HikariCP's guidance: small pools (roughly core-count scale) often beat big ones, since the database can only do so much in parallel.
- *Q: How do you detect leaks?* — A: Record who borrowed each object and when; log if held past a threshold (HikariCP's `leakDetectionThreshold`).

Self-check

- **Q:** Object Pool vs Flyweight? **A:** Pool lends mutable objects exclusively and takes them back; Flyweight shares immutable objects among everyone at once.
- **Q:** Why validate on release? **A:** So broken objects (dead connections) are discarded instead of handed to the next borrower.

TL;DR

Borrow → use → return (in `finally`). Bounded, thread-safe, validated, reset. Worth it only for expensive or limited resources; use a mature library in production.

8.15 Creational patterns compared

At a glance

Pattern	Distinctive feature	Use when	Course example	JDK / Spring example	Pros	Cons
Singleton	Only one instance of the class exists	One shared resource: configuration, pool, registry	One `ConfigStore` (or one DB connection pool)	`Runtime.getRuntime()`, Spring singleton-scoped beans	Global access; single initialization	Hurts unit testing (testing one class on its own); hidden dependencies; global state
Simple Factory *(idiom)*	One method maps a type value to a class	Small, stable set of types chosen by config	Notification by channel	`Calendar.getInstance()`, `EnumSet.noneOf`, `@Bean` methods	Short, centralizes the switch	Edit it for every new type
Factory Method	Subclasses decide the created type via an overridable method	Shared procedure, varying product; others will add types	PDF / HTML report generator (documents of different types — PDF, Word, HTML…)	`Collection.iterator()`, Spring `FactoryBean`	Open–Closed; decouples creator from products	More classes; needs subclasses
Abstract Factory	Creates *families* of related objects without concrete classes	Products must be compatible; swap a whole set at once	Postgres vs MySQL storage; Windows vs macOS widgets	`DocumentBuilderFactory`, JDBC `Connection`, Spring profiles	Guarantees compatibility; Open–Closed for families	Complex; adding a product kind edits every factory
Builder	Separates construction of a complex object from its representation	Many optional fields, validation, immutability, step-by-step control	`HttpRequest`; building a custom meal	`StringBuilder`, `HttpClient.newBuilder()`, `UriComponentsBuilder`	Step-by-step, readable, immutable, validated	Verbose for simple objects; field duplication; Director optional
Prototype	Creates objects by copying an existing one	Setup is expensive, or only the interface is known	`NotificationTemplate`; cloning a customized car configuration	Copy constructors, `ArrayList.clone()`, Unity `Instantiate`	Skips setup; fewer subclasses; runtime-configured kinds	Deep copy is hard; `Cloneable` is broken; identity issues
Object Pool *(bonus)*	Reuses a bounded set of objects	Expensive or limited resources	DB connections	HikariCP, `ThreadPoolExecutor`	Fast reuse; caps load	Leaks, stale state, pointless for cheap objects

Pick by the question you are asking

You are asking…	Pattern
"How do I make sure there is only one?"	Singleton (or DI singleton scope)
"How do I avoid `new ConcreteClass()` scattered around?"	Simple Factory
"How do I let subclasses/plug-ins decide what to create?"	Factory Method
"How do I make sure these objects always match?"	Abstract Factory
"How do I make this 10-argument constructor readable and safe?"	Builder
"How do I make another one of *this*, cheaply or without knowing its class?"	Prototype
"How do I stop creating and destroying this expensive thing?"	Object Pool

How they combine

- An **Abstract Factory** is often implemented with **Factory Methods**, may use **Prototypes** (a factory configured with prototype objects it copies), and is frequently a **Singleton** (one factory per family) — though passing it in is better.
- A **Builder** can use a **Factory** to pick which parts to build, and a **Director** can be a Singleton-like stateless (keeps no data between calls) service.
- Designs often *start* with Factory Method (simple, subclass-based) and *evolve* towards Abstract Factory, Prototype or Builder as more flexibility is needed.

8.16 Chapter self-check

1. **Q:** Name the three GoF categories and how many patterns each has. **A:** Creational 5, Structural 7, Behavioral 11 = 23.
2. **Q:** Which two principles does the GoF book put at the centre? **A:** Program to an interface, not an implementation; favour composition over inheritance.
3. **Q:** What is the Golden Hammer? **A:** Applying a favourite tool/pattern to every problem regardless of fit.
4. **Q:** Why must the DCL field be `volatile`? **A:** To prevent another thread from seeing a reference to a not-yet-fully-constructed object (it creates a happens-before edge).
5. **Q:** Simple Factory vs Factory Method in one line? **A:** A switch in one method vs a creation hook overridden by subclasses.
6. **Q:** Factory Method vs Abstract Factory in one line? **A:** One product by inheritance vs a family of matching products by composition.
7. **Q:** Builder's single most important method, and why? **A:** `build()` — the one gate that validates cross-field rules and creates the immutable product.
8. **Q:** What is the classic Prototype bug? **A:** A shallow copy sharing a mutable collection with the original.
9. **Q:** Why is `enum` the most robust Java singleton? **A:** The JVM guarantees one instance per constant and blocks reflective creation and serialization duplicates.
10. **Q:** In an interview, how many patterns should a 45-minute design use? **A:** Usually one or two — and name the one you rejected.

8.17 Chapter TL;DR

- **Patterns** are named, proven solution templates with trade-offs — vocabulary first, code second. GoF (1994) catalogued 23 in three categories.
- **Pick from the signal, not the catalogue.** One or two patterns per interview design; always state the cost and the alternative.
- **Singleton** — one per JVM; eager / holder / enum; DCL needs `volatile`; prefer DI.
- **Simple Factory** — one `create(type)`; idiom, not GoF.
- **Factory Method** — base-class procedure + subclass-chosen product.
- **Abstract Factory** — one factory, one matching family; chosen at the composition root.
- **Builder** — named steps, one validating `build()`, immutable product.
- **Prototype** — copy a configured object; decide shallow vs deep; avoid `Cloneable`.

- **Object Pool** — borrow/return expensive objects; use a real library in production.

Chapter 9 — Structural Design Patterns

> **Where this fits.** Chapter 8 was about *creating* objects. This chapter is about *putting objects together*: connecting, wrapping, nesting and sharing them so larger structures stay flexible. Chapters 10–11 cover how objects *talk to each other* (behavioral patterns).

What you will learn

#	Pattern	One-line intent	Everyday analogy
9.1	Adapter	Translate a class's interface into the one your code already uses	A travel plug adapter
9.2	Bridge	Separate two independently varying dimensions into two hierarchies joined by a reference	A TV remote (control) and the TV (device) are sold separately
9.3	Composite	Treat one object and a tree of objects the same way	A folder can hold files and other folders
9.4	Decorator	Add behaviour to one object at run time by wrapping it	Toppings on a coffee
9.5	Facade	One simple entry point in front of a complicated subsystem	A hotel concierge
9.6	Flyweight	Share the repeating part of many objects to save memory	One font glyph "e" reused for every "e" on a page
9.7	Proxy	A stand-in with the same interface that controls access	A receptionist who decides whether you may see the manager
9.8	Comparison	Adapter vs Decorator vs Proxy vs Facade vs Bridge	—

How to run every example in this chapter

```
cd chapters/09-patterns-structural/code
javac --release 21 -d /tmp/lld_09 $(find . -name '*.java')
java -cp /tmp/lld_09 lld.ch09.adapter.Main     # or bridge, composite, decorator, facade, flyweight, proxy
```

All seven `Main` classes compile with Java 21 (standard library only) and run; each pasted output is that program's real output.

> **Code-style note.** In `code/` every top-level type has its own file and is package-private (visible only within its package — a named folder of related classes). The folder compiles with `javac --release 21 -Xlint:all` with **zero warnings**. The listings below show the essential lines of those files; open the folder for the full runnable code.

9.0 What structural patterns are

What it is

A **design pattern** is a named, reusable solution to a recurring design problem. The "Gang of Four" (GoF) book — *Design Patterns: Elements of Reusable Object-Oriented Software* (Gamma, Helm, Johnson, Vlissides, 1994) — sorted its 23 patterns into three families:

Family	Question it answers	Examples
Creational (Ch. 8)	How do I create objects?	Singleton, Factory Method, Builder
Structural (this chapter)	**How do I combine classes and objects into bigger structures?**	**Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy**
Behavioral (Ch. 10–11)	How do objects share work and talk?	Strategy, Observer, Command

Structural patterns combine small objects and classes into larger structures while keeping them flexible and efficient. Used well, they **simplify complex structures** (a Facade hides many steps; a Composite hides nesting), **promote reuse** (an Adapter reuses a class whose interface does not fit), **improve flexibility by decoupling** classes — reducing how much each depends on the others' details (Bridge, Decorator, Proxy) — and **improve maintainability**, since relationships are clear and changes stay local. The seven the GoF named, and interviewers ask about, are **Adapter, Bridge, Composite, Decorator, Facade, Flyweight and Proxy**.

Three words you need first

- **Composition** ("has-a"): an object holds and uses a reference to another. Almost every structural pattern is built on it.
- **Delegation**: forwarding a call to the object you hold. `wrapper.pay()` calls `inner.charge()`.
- **Interface** (design sense): the set of methods a caller can use on an object — often, not always, a Java `interface` type.

> **Key insight.** Adapter, Decorator, Proxy and Facade have the *same physical shape*: one object holds another and forwards calls. **Intent** — what changes as the call passes through — tells them apart. Section 9.8 puts them side by side.

9.1 Adapter Pattern

What it is

The **Adapter** lets your code use a class whose methods have the "wrong" shape — one you cannot change — by placing a small translator between them, like a travel plug adapter between a US charger and an Indian socket. GoF's intent: *convert the interface of a class into another interface clients expect; Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.* GoF also calls it **Wrapper**.

Term	Meaning
Target	The interface your code wants to use (`PaymentGateway`).
Adaptee	The existing class with the "wrong" interface (`StripeClient`). You usually do not own it.
Adapter	The class that implements the Target and translates calls to the Adaptee (`StripeGateway`).
Client	Code that uses the Target (`checkout`).

The problem it solves

The shape your code calls and the shape the other side offers do not match:

- checkout wants `pay(Money)`, but the payment SDK (software development kit — the vendor's client library) offers `charge(amountInCents, currencyCode)`;
- an old service answers in XML while every consumer works with JSON (two common text formats for structured data);
- an analytics library wants `track(String, Map)` while your code produces an `Event`.

The key constraint: **the other side is not yours to change** — a third-party library (owned by an outside company or project), another team's service, or code with dozens of callers. Bending *your* code to its shape touches every call site (each place that calls the method), and you repeat it the day the vendor is swapped.

Concretely: checkout must support Stripe now and perhaps PayPal later (both online payment providers). Stripe wants whole **cents** plus a **lowercase** currency; PayPal a **decimal string** like "42.50" plus an **uppercase** code. Your code would rather reason in `Money`.

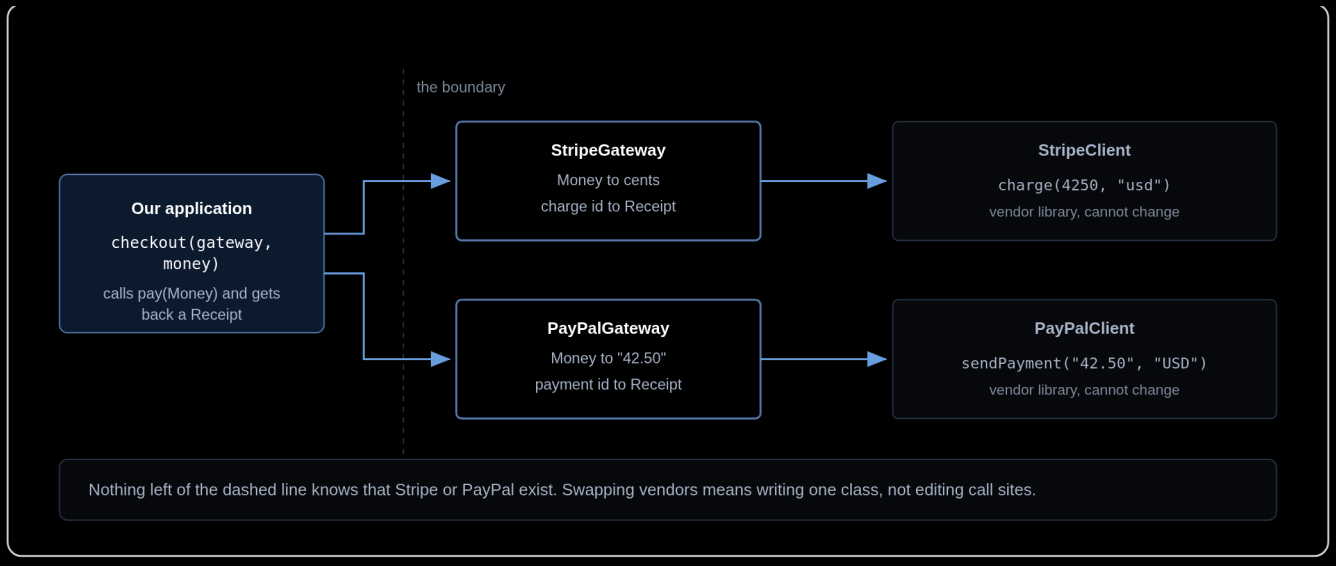


Figure — Two adapters (StripeGateway, PayPalGateway) translate our `pay(Money)` into each vendor's own call; nothing on the application side knows which vendor exists (course diagram)

Left of the dashed boundary everything speaks `Money` and `Receipt`; swapping vendors means one new class, no call-site changes.

How it works

1. **Define the Target in your own vocabulary** (`PaymentGateway.pay(Money) -> Receipt`), mentioning no vendor.
2. **One adapter class per adaptee**: it `implements` the Target and **holds** the vendor object.
3. **Translate inputs** (`Money` → cents + lowercase code).
4. **Call the adaptee.**
5. **Translate outputs and errors back**: vendor result → `Receipt`; vendor exception → *your* exception.
6. **Clients depend only on the Target** and never learn which vendor answered.

> **The rule that makes it work:** the adapter's public methods mention **only your types** — parameters, return values *and* exceptions. Once `pay` returns a `StripeCharge`, or lets a `StripeCardException` propagate (travel up, uncaught, to callers), callers depend on Stripe again and nothing is decoupled.

Structure

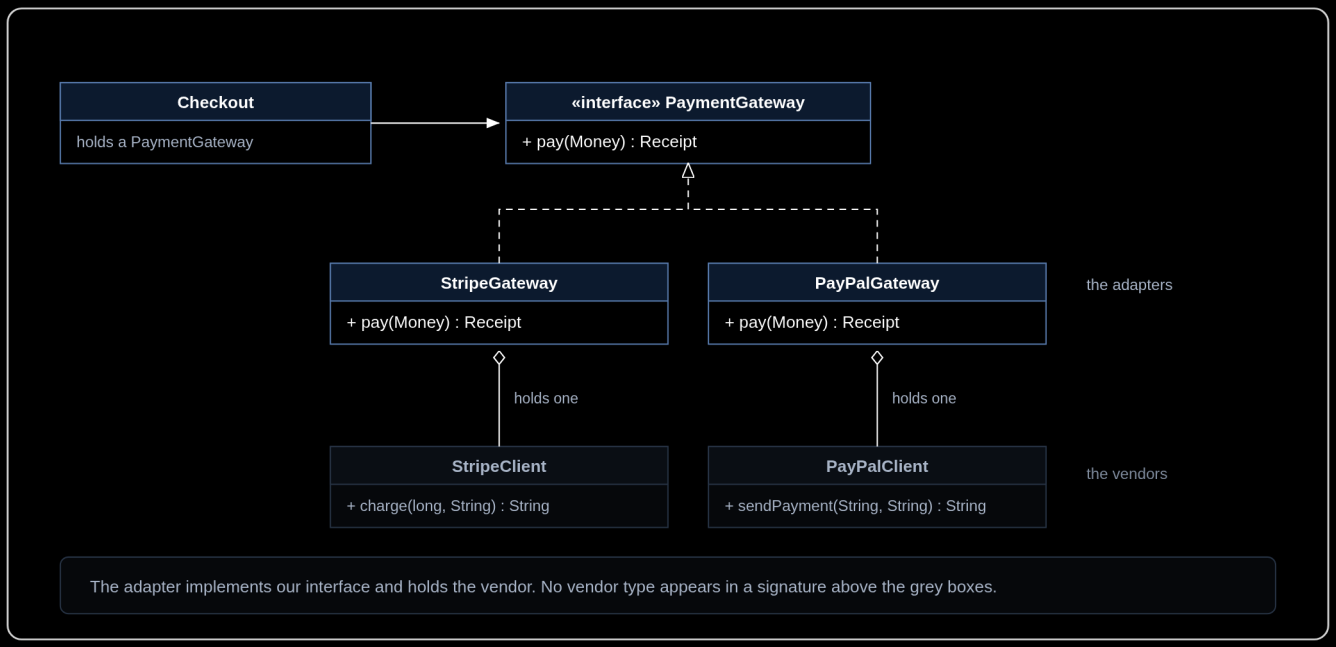
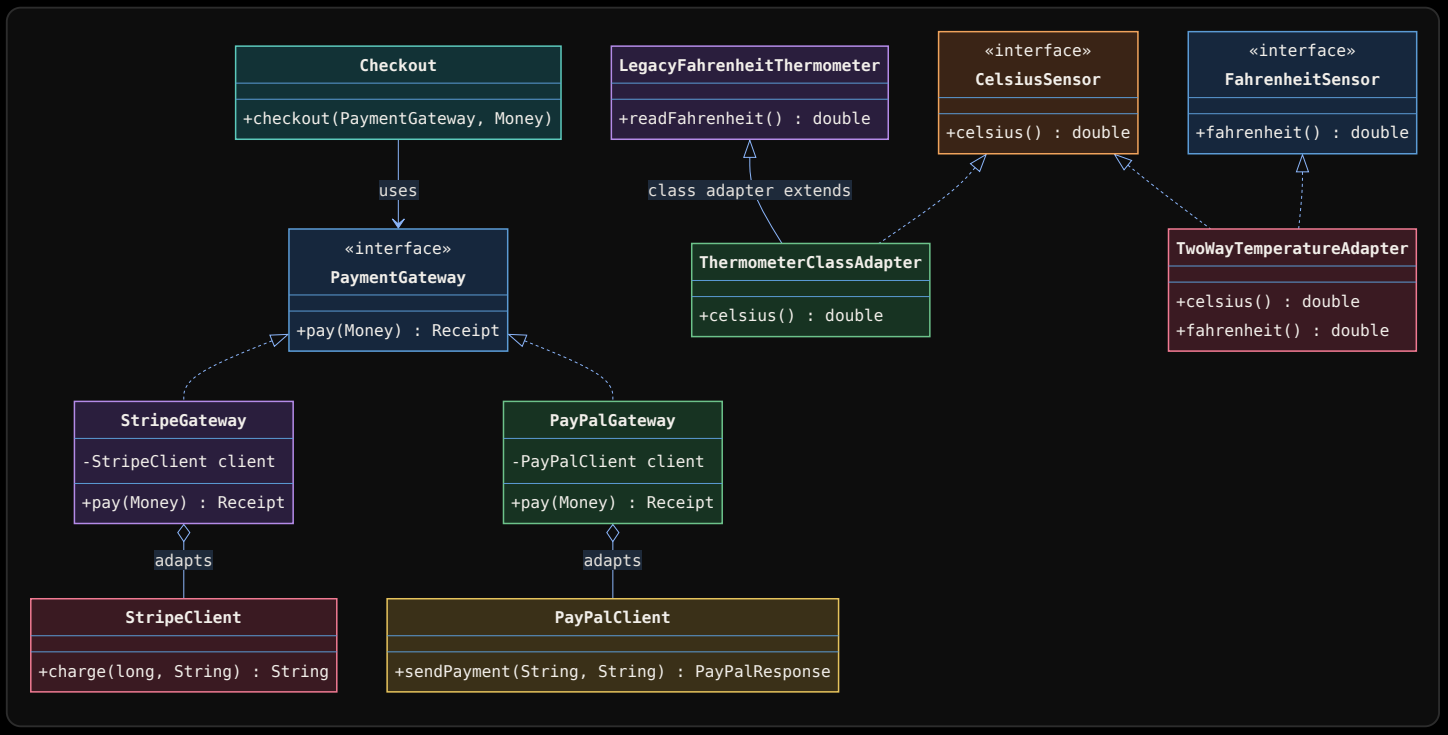


Figure — Adapter class diagram: `Checkout` holds a `PaymentGateway`; `StripeGateway` and `PayPalGateway` implement it and each wrap a vendor client, shown in grey because we do not own them (course diagram)

`PaymentGateway` is the Target (yours); `StripeGateway`/`PayPalGateway` the Adapters; `StripeClient`/`PayPalClient` the Adaptees (third-party); `Checkout` any Client. Every mismatch — names, parameter types, casing — is soaked up

inside the adapters.

Our Java design (including the class and two-way adapters):



Java example

Full code: `code/lld/ch09/adapter/` — our vocabulary (`Money`, `Receipt`, `PaymentGateway`, `PaymentFailedException`), the adaptees (`StripeClient` + `StripeCardException`, `PayPalClient`), the adapters, the temperature sensors, and `Main`.

```
// Money.java — our own vocabulary; no vendor type ever appears here.
record Money(long cents, String currency) {
    Money {
        if (cents < 0) throw new IllegalArgumentException("negative amount");
        currency = currency.toUpperCase(Locale.ROOT);
    }
    String asDecimal() { return String.format(Locale.ROOT, "%d.%02d", cents / 100, cents % 100); }  // 4250 -> "42.50"
}
interface PaymentGateway { Receipt pay(Money amount); }   // TARGET

// ADAPTEES (pretend third-party jars we cannot edit):
//   StripeClient.charge(long amountInCents, String currencyCode) -> charge id; throws StripeCardException over 100_000
//   PayPalClient.sendPayment(String total, String currency) -> PayPalResponse(paymentId, state)

// StripeGateway.java — OBJECT ADAPTER: implements our interface, HOLDS the vendor object.
final class StripeGateway implements PaymentGateway {
    private final StripeClient client;
    StripeGateway(StripeClient client) { this.client = client; }

    @Override public Receipt pay(Money amount) {
        try {
            String chargeId = client.charge(amount.cents(), amount.currency().toLowerCase(Locale.ROOT));
            return new Receipt(chargeId, "SUCCEEDED");
        } catch (StripeCardException e) {              // translate the vendor's error too
            throw new PaymentFailedException("Stripe declined: " + e.getMessage(), e);
        }
    }
}

// PayPalGateway.java
@Override public Receipt pay(Money amount) {
    PayPalClient.PayPalResponse r = client.sendPayment(amount.asDecimal(), amount.currency());
    return new Receipt(r.paymentId(), "approved".equals(r.state()) ? "SUCCEEDED" : "FAILED");
}
```

```java
// ThermometerClassAdapter.java — CLASS ADAPTER: inherits the adaptee, implements the target
// (works in Java only because the target is an interface).
final class ThermometerClassAdapter extends LegacyFahrenheitThermometer implements CelsiusSensor {
    ThermometerClassAdapter(double fahrenheit) { super(fahrenheit); }
    @Override public double celsius() { return (readFahrenheit() - 32) * 5.0 / 9.0; }
    // Downside: readFahrenheit() is still public on this object -> the adaptee's API leaks.
}

// TwoWayTemperatureAdapter.java — implements BOTH interfaces for new (Celsius) and old (Fahrenheit) code.
final class TwoWayTemperatureAdapter implements CelsiusSensor, FahrenheitSensor {
    private final double celsius;
    static TwoWayTemperatureAdapter from(CelsiusSensor s)    { return new TwoWayTemperatureAdapter(s.celsius()); }
    static TwoWayTemperatureAdapter from(FahrenheitSensor s) { return new TwoWayTemperatureAdapter((s.fahrenheit() - 32) *
5.0 / 9.0); }
    @Override public double celsius()    { return celsius; }
    @Override public double fahrenheit() { return celsius * 9.0 / 5.0 + 32; }
}

// Main.java — the CLIENT talks only to OUR interface and never names a vendor.
static void checkout(PaymentGateway gateway, Money amount) {
    try {
        Receipt r = gateway.pay(amount);
        System.out.println("  receipt " + r.id() + " " + r.status());
    } catch (PaymentFailedException e) {
        System.out.println("  payment failed -> " + e.getMessage());
    }
}
// main also demos Arrays.asList (array -> List view, no copy) and InputStreamReader (bytes -> chars).
```

Output:

```
== Object adapters (payment) ==
  [stripe] charge(4250, "usd")
  receipt ch_1001 SUCCEEDED
  [paypal] sendPayment("42.50", "USD")
  receipt PAY-4250 SUCCEEDED
  [stripe] charge(250000, "usd")
  payment failed -> Stripe declined: card_declined
== Class adapter ==
  98.6F -> 37.0C
== Two-way adapter ==
  new code sees 100.0C, old code sees 212.0F
== JDK adapters ==
  Arrays.asList sees array change: [z, b, c]
  cannot resize an array-backed list: UnsupportedOperationException
  InputStreamReader decoded: héllo
```

What each part does

- `Money`, `Receipt`, `PaymentGateway`, `PaymentFailedException` — **our** vocabulary. The compact constructor (no parameter list, runs on every creation) of the `Money` record (short syntax for an immutable data class) upper-cases the currency once, so every adapter can rely on it.
- `StripeGateway` — an **object adapter** converting `Money` → `(cents, "usd")`, charge id → `Receipt`, `StripeCardException` → `PaymentFailedException` (third output call: the client sees *our* error type). `PayPalGateway` translates differently (`Money.asDecimal()` → "42.50"; `"approved"` → `"SUCCEEDED"`).
- `checkout(...)` — the **client**: one function, two vendors, no `if (stripe)` branch. The JDK section shows two adapters you already use: `Arrays.asList` and `InputStreamReader`.

Object adapter vs class adapter

	Object adapter (composition)	**Class adapter (inheritance)**
How	Adapter *holds* an adaptee instance and implements the Target	Adapter *extends* the adaptee class and implements the Target
In our code	`StripeGateway`, `PayPalGateway`	`ThermometerClassAdapter extends LegacyFahrenheitThermometer implements CelsiusSensor`
Works when adaptee is `final`?	Yes	**No** — you cannot extend a final class (our `StripeClient` is `final`)
Can adapt a whole family (adaptee + subclasses)?	Yes — pass any subclass instance in	No — bound to one concrete class at compile time
Can override adaptee behaviour?	Harder — subclass the adaptee and adapt that subclass	Yes (it is a subclass)
Leaks the adaptee's API (its public methods)?	No — only the Target methods are visible	**Yes** — every public adaptee method (e.g. `readFahrenheit()`) is also on the adapter
Java constraint	none	Java has no multiple class inheritance (extending two classes at once), so the **Target must be an interface**

Rule of thumb: prefer the **object adapter** — composition over inheritance, works with final classes, can wrap several objects, inherits nothing unwanted. Use a class adapter only to override adaptee behaviour when the adaptee is designed for extension.

Two-way adapter

A **two-way adapter** implements *both* interfaces so one object serves clients of either side (GoF: two clients needing to view one object in two ways). `TwoWayTemperatureAdapter implements CelsiusSensor, FahrenheitSensor` is built from either kind of sensor; new dashboard code sees `100.0C`, old US reporting code `212.0F` — same object. Handy during a **migration** (gradually moving code from an old system to a new one).

Adapter vs Facade vs Decorator vs Proxy (quick view)

In one line each (full table in 9.8): **Adapter** *translates* the interface, same work; **Decorator** keeps the interface and *extends* the work; **Proxy** keeps interface and work but *gates* it (postponed, served from a cache — a store of earlier results for quick reuse — or rejected); **Facade** *replaces* many interfaces with one simpler one, fewer calls for the client.

A useful test: *adapters exist because someone else picked the interface.* If the vendor rewrote its API to match yours tomorrow and your class would lose its reason to exist, it was an Adapter; the other three exist because you chose to add something. Bridge uses the same boxes with reversed timing: planned *before* the classes exist so two hierarchies never fuse; an Adapter is bolted on *after* to reconcile two existing things.

Trade-offs

Pros	**Cons**
The foreign format is contained — one class knows the vendor's conventions.	**An extra class and call** — stack traces (calls listed when an error happens) gain a frame whose only job is conversion.
Changing vendor is local — PayPal beside Stripe is a new class, not edited callers (Open/Closed Principle).	**Conversions can drop information** invisibly to callers.
Easy to test — faking your one-method interface beats faking a vendor SDK.	**Two data models must stay aligned** — each vendor API change forces a mapping change.
Your code keeps its own language (`Money`, `Receipt`); conversion has one home (Single Responsibility).	**Logic creeps in** — retries (automatic re-tries of failed calls) and pricing rules drift in unless reviews stop them.

When not to use it

- **Both sides are yours** — change one interface; an adapter between your own code is overhead paid forever for a removable mismatch.

- **The two models really disagree** — if mapping drops data or forces guesses, the "adapter" becomes a buggy mini-application; redraw the boundary.
- **You want extra behaviour, not translation** — a Decorator's job.
- **The conversion happens in exactly one place** — a few inline lines read better.

Common mistakes

- **Leaking vendor types** (`StripeCharge` return type, `StripeCardException` escaping) — nothing is decoupled.
- **Business logic in the adapter** (discounts, retries). Adapters translate; they do not decide.
- **Class adapter by default** — exposes the adaptee's whole API and breaks on `final` classes.
- **One giant adapter** with `if (vendor == STRIPE)` — the branch the pattern removes. One adapter per adaptee.
- **Silently dropping information** (vendor "pending" mapped to "SUCCEEDED"). Map every case deliberately.

Real-world examples

- **`java.util.Arrays.asList(T...)`** adapts an array to `List` *without copying* — writes go through to the array and the list cannot grow (`add` throws `UnsupportedOperationException`).
- **`java.io.InputStreamReader`** / **`OutputStreamWriter`** adapt byte streams to character streams using a charset (the byte↔character rule, e.g. UTF-8).
- **`java.util.Collections.enumeration(Collection)`** adapts a `Collection` to the legacy `Enumeration` interface. (*Head First* builds the reverse, Enumeration-to-Iterator.)
- **SLF4J providers/bindings** — SLF4J (Simple Logging Facade for Java) is one logging API with an adapter module per backend (the library that writes log lines: Logback, Log4j, `java.util.logging`).
- **JDBC (and ODBC) drivers** adapt each database's wire protocol (the bytes it sends over the network) to the standard `java.sql` interfaces, so the same code runs on several databases.
- **Spring MVC** `HandlerAdapter` lets the `DispatcherServlet` (Spring MVC's central object receiving every web request) invoke very different handlers (annotated controllers, `HttpRequestHandler`, etc.) through one interface.
- **Almost every "client" wrapper around a vendor SDK** — if it exists so the rest of the code does not import the SDK, it is an adapter.

In modern languages

- **First-class functions** (functions stored and passed like values): for a single-method Target a **lambda** is a complete adapter — `PaymentGateway g = m -> new Receipt(stripe.charge(m.cents(), "usd"), "SUCCEEDED");` (our `FahrenheitSensor oldSensor = () -> 212.0;` is the same trick).
- **Python**: duck typing ("if it has the right methods, it fits", checked at run time) — a small wrapper class or even a `types.SimpleNamespace` suffices.
- **TypeScript**: structural typing accepts anything of the right shape; adapters are often plain object literals (`{ ... }`).
- **Go**: interfaces are satisfied implicitly (no `implements`), so the adapter is often a small named type with one method.

In the interview

> **30-second answer:** "An Adapter converts one interface into another that the client expects. I use it at boundaries I do not control — a vendor SDK, a legacy service. The adapter implements *my* interface, holds the vendor object, and translates inputs, outputs and exceptions, so no vendor type leaks into my code. Swapping vendors becomes one new class. I prefer the object adapter over the class adapter because it uses composition and works with final classes."

Follow-ups

- *Q: Adapter vs Decorator?* — A: Adapter changes the interface, keeps behaviour. Decorator keeps the interface, adds behaviour.
- *Q: Adapter vs Facade?* — A: Adapter converts **one** interface to **one** other, usually because you own only one side. Facade simplifies a **whole subsystem** into one entry point, usually over code you own.
- *Q: Why not just change the other side?* — A: Say it unprompted: vendor library, another team's API, or many callers. That constraint justifies the pattern — without it you are pattern-matching out of habit.
- *Q: What ruins an adapter?* — A: A vendor type or exception in its public signature.

- *Q: How does it help testing?* — A: Checkout depends on a small interface I own, so tests pass a fake `PaymentGateway`; no faking the vendor SDK.
- *Q: What is a two-way adapter?* — A: One class implementing both interfaces to serve either side, useful during migrations.

Self-check

1. **Q:** Name the four roles in the Adapter pattern. **A:** Target, Adaptee, Adapter, Client.
2. **Q:** Why is `StripeClient` impossible to class-adapt in our code? **A:** It is `final`; a class adapter must extend the adaptee.
3. **Q:** What does `Arrays.asList(arr).add(x)` do, and why? **A:** Throws `UnsupportedOperationException`, because the list is a fixed-size view (adapter) over the array.
4. **Q:** You own both classes and their interfaces differ slightly. Adapter? **A:** No — change the interface; an adapter would be permanent overhead.
5. **Q:** Where should the vendor's exception be handled? **A:** Inside the adapter, translated into your own exception type.

TL;DR

- **Adapter = translator** between the interface you have and the one you need.
- Implements **your** interface, **holds** the vendor object, translates **in, out and errors**.
- Prefer **object adapter** (composition) over **class adapter** (inheritance). Two-way adapters implement both sides.
- Use at boundaries you **don't control**; skip it when you own both sides.

9.2 Bridge Pattern

What it is

The **Bridge** pattern splits one class that varies in *two independent ways* into **two separate hierarchies** connected by a reference (the "bridge"), so each side grows on its own — like universal TV remotes (basic, advanced with mute) and TV brands (Sony, Samsung): no "BasicSonyRemote"; any remote drives any TV through a common protocol (agreed set of signals).

GoF's intent: *decouple an abstraction from its implementation so that the two can vary independently.* Also called **Handle/Body**. The vocabulary interviewers expect:

Term	Meaning	In our code
Abstraction	The high-level side the client uses; holds a reference to an Implementor	`Notification`
Refined Abstraction	Subclasses of the abstraction	`AlertNotification`, `ReminderNotification`, `DigestNotification`
Implementor	The interface of the low-level side that does the physical work	`Channel`
Concrete Implementor	Real implementations of that low-level work	`EmailChannel`, `SmsChannel`, `PushChannel`

Naming trap. In Bridge, "implementation" is *not* the everyday Java meaning ("a class that implements an interface"). It names the **low-level half of the split** — the platform-specific, physical work (sending bytes, drawing pixels). Awkward, but these are the terms you will be asked about.

The problem it solves — the class explosion

Three notification kinds (alert, reminder, digest) × three channels (email, SMS, push) in **one** hierarchy need a class per combination:

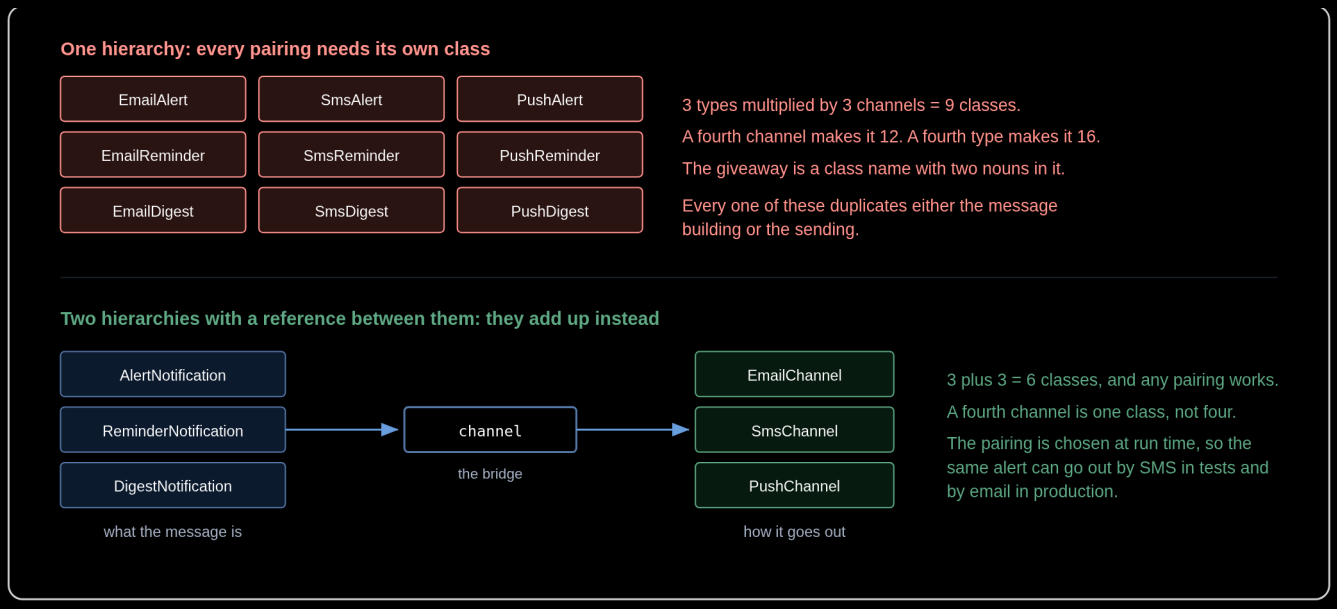


Figure — Top: one hierarchy needs 3 × 3 = 9 classes such as EmailAlert and SmsDigest. Bottom: two hierarchies joined by one `channel` reference need 3 + 3 = 6 (course diagram)

In one hierarchy concrete classes **multiply**: 3 × 3 = 9; one more channel → 12; one more type too → 16. With a bridge they **add**: 3 + 3 = 6, 4 + 4 = 8. (Counts are concrete classes; the `Notification` base and `Channel` interface exist in both.)

The **smell** (code smell: a surface symptom of a deeper design problem) is **a class name made of two nouns** — `EmailAlert`, `SmsDigest`, `PushReminder`. Each repeats the "build the message" or "send it" code, so changing SMS delivery edits three otherwise unrelated classes.

Splitting also makes the pairing a **run-time choice**: the same alert with an SMS channel in a test, email in production (the live system). The code is small; the hard part is deciding **which dimension is the abstraction and which the implementation** (see "Getting the split wrong").

How it works

1. **Find the two dimensions** varying independently (what the message *is* vs how it *goes out*).
2. **Make the lower-level one an interface** — the Implementor (`Channel.deliver(title, body)`), small and general.
3. **Make the higher-level one an abstract class** — the Abstraction — that **holds a reference** to the Implementor. That field is the bridge.
4. **Refined abstractions** build their content and **delegate** the physical work across the bridge, never checking which implementor they have.
5. **Pair at run time**: `new AlertNotification(new SmsChannel(...), ...)`.

Structure

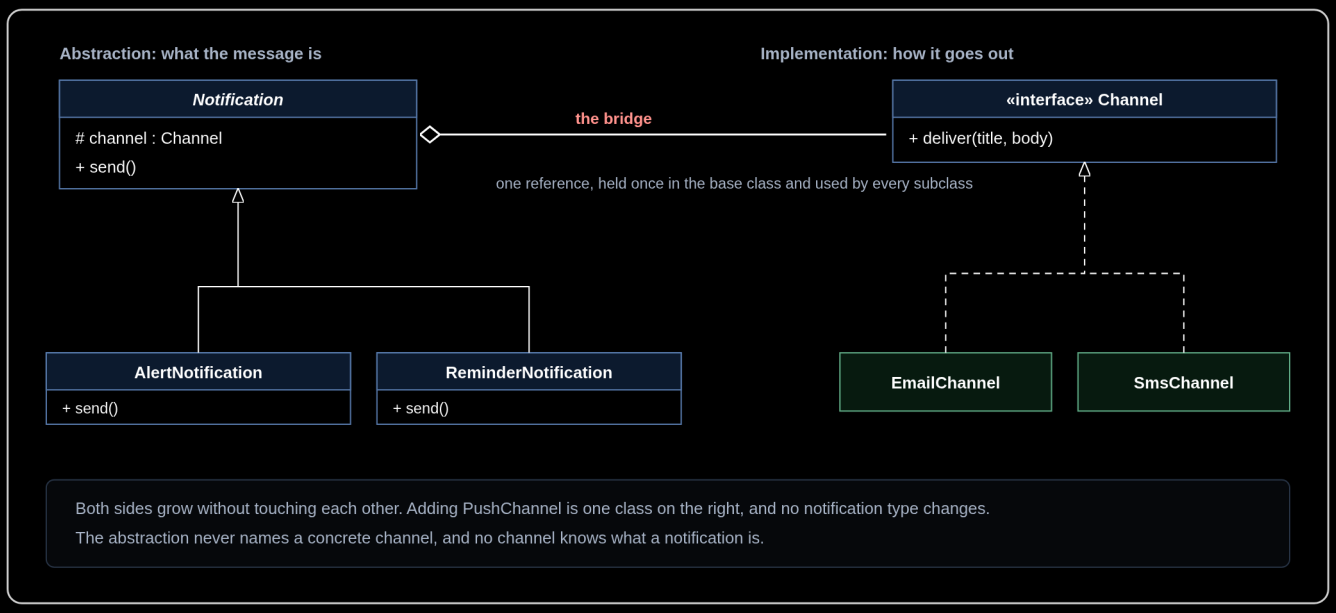
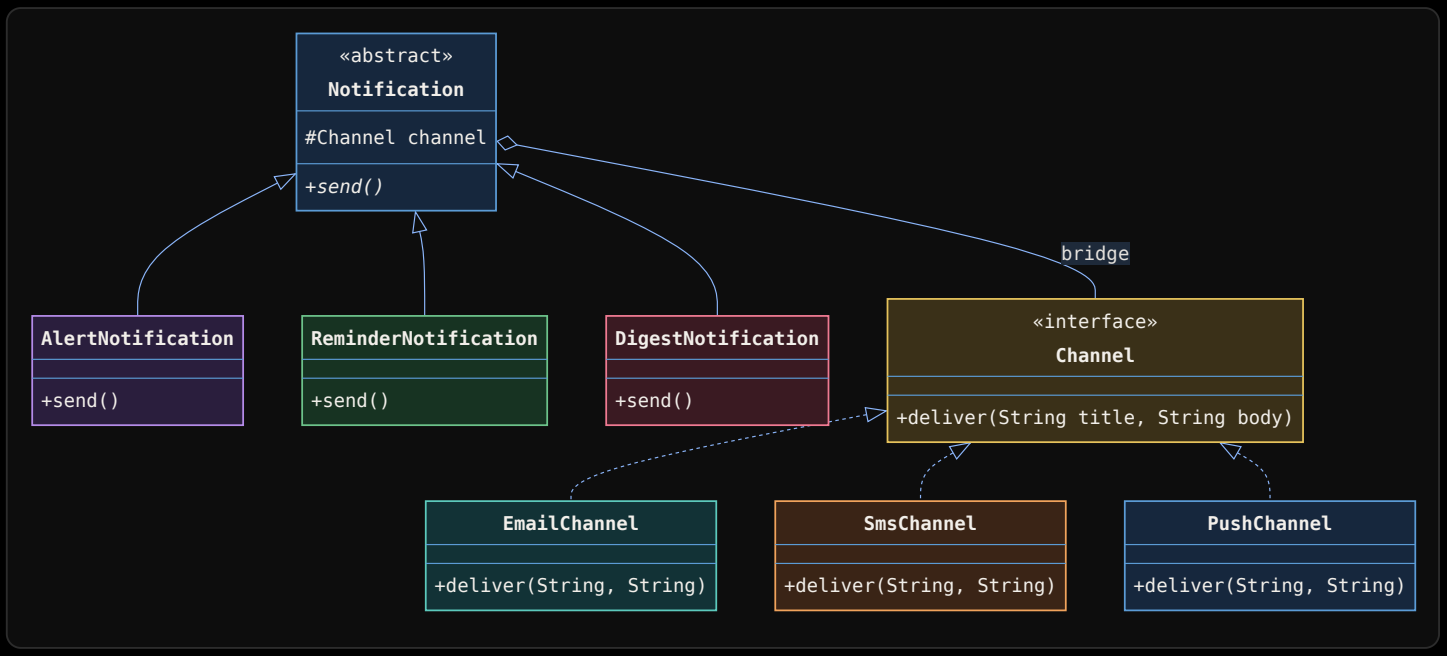


Figure — Bridge class diagram: `Notification` (abstraction) holds a `Channel` (implementor); alert/reminder notifications refine the abstraction; email/SMS channels implement the channel (course diagram)

The bridge is the one `channel` field, declared in the base class and used by every subclass. Channel classes know nothing about notifications.



Java example

Full code: `code/lld/ch09/bridge/` (one file per type, plus `Main`).

```java
// Channel.java — IMPLEMENTOR: the low-level "how it goes out" side; two plain strings, nothing channel-specific.
interface Channel { void deliver(String title, String body); }

// SmsChannel.java — CONCRETE IMPLEMENTOR (EmailChannel, PushChannel likewise); knows nothing of alerts or digests.
final class SmsChannel implements Channel {
    private static final int LIMIT = 40;          // tiny limit so the demo shows truncation
    private final String phone;
    SmsChannel(String phone) { this.phone = phone; }
    @Override public void deliver(String title, String body) {
        String text = title + ": " + body;
        if (text.length() > LIMIT) text = text.substring(0, LIMIT - 3) + "...";
        System.out.println("  [sms to " + phone + "] " + text);
    }
}

// Notification.java — ABSTRACTION: the protected 'channel' field IS the bridge.
abstract class Notification {
    protected final Channel channel;
    protected Notification(Channel channel) { this.channel = channel; }
    /** Builds the message, then hands it across the bridge. Never branches on channel type. */
    abstract void send();
}

// AlertNotification.java — REFINED ABSTRACTION (ReminderNotification, DigestNotification likewise).
final class AlertNotification extends Notification {
    private final String system, problem;
    AlertNotification(Channel channel, String system, String problem) {
        super(channel); this.system = system; this.problem = problem;
    }
    @Override void send() { channel.deliver("ALERT " + system, problem + " - act now"); }
}
// Digest: channel.deliver("Daily digest (" + items.size() + ")", String.join("; ", items));

// Main.java — any type x any channel, paired at run time.
List<Notification> outbox = List.of(
        new AlertNotification(email, "payments", "error rate 12%"),
        new AlertNotification(sms, "payments", "error rate 12%"),
        new ReminderNotification(push, "Stand-up", "09:30"),
        new DigestNotification(email, List.of("3 orders", "1 refund", "0 incidents")),
        new DigestNotification(sms, List.of("3 orders", "1 refund", "0 incidents")));
outbox.forEach(Notification::send);
```

Output:

```
== Any type x any channel, paired at run time ==
  [email to ops@shop.com] Subject: ALERT payments | error rate 12% - act now
  [sms to +1-555-0100] ALERT payments: error rate 12% - act now
  [push to pixel-7] (Reminder) Stand-up at 09:30
  [email to ops@shop.com] Subject: Daily digest (3) | 3 orders; 1 refund; 0 incidents
  [sms to +1-555-0100] Daily digest (3): 3 orders; 1 refund;...
== Class count ==
  one hierarchy (type x channel): 9 classes
  bridge (type + channel):        6 classes
  with 4 types and 4 channels:    16 vs 8
```

What each part does

- Channels know *only* delivery (SMS truncation lives inside SMS, not in any notification); `deliver` takes plain strings so every channel can support it.
- Notifications build title and body and call `channel.deliver(...)` — no `instanceof` (Java's concrete-type check), no `switch` on channel type.
- `SlackChannel` = one new class; `PromoNotification` = one new class. Nothing else changes.

Bridge vs Strategy

Written down they are **indistinguishable** — an object holds an interface reference and delegates. Interviewers and reviewers ask, so be ready: the difference is the **reason the reference is there**.

	Bridge	Strategy
What varies	A whole **dimension** of the design	**One algorithm** or policy
When chosen	Usually once, at construction, then it stays	Often swapped repeatedly, even per call
Why it exists	To stop two hierarchies multiplying	To make one behaviour replaceable
When decided	Up front, as a structural decision	Later, when a second algorithm appears
Example	`Notification` → `Channel`	`list.sort(comparator)`

Practical test: picture one more implementation. *A new kind of thing the system supports* (another delivery channel) → Bridge. *One more way to perform a single step* (another compression algorithm) → Strategy. If you truly can't decide, either name is understood since the code is identical; what matters is not building either while an axis has one member.

Getting the split wrong

Quick to write, costly to re-cut once both sides have several classes:

- **Low-level details leaking up.** `if (channel instanceof SmsChannel)` inside `AlertNotification` means the bridge has failed — move that SMS rule into `SmsChannel`.
- **An implementor interface shaped around one implementation.** Add `setEmailReplyTo(...)` to `Channel` and every non-email channel must implement a method it can only throw from. Keep it minimal and generic — why `deliver` takes two plain strings, not a rich message type.

Trade-offs

Pros	Cons
Class counts add rather than multiply — 3 types and 3 channels need 6 classes, not 9.	**Two hierarchies** to understand instead of one.
Each side extends independently (Open/Closed) — a new channel touches no notification class.	**More indirection** — one call spans two files.
Pairing picked at run time — real channel in production, fake in tests, per-environment config.	**Hard to undo** — moving the split line later is a large refactor (restructuring without changing behaviour).
Platform code kept apart — only one class knows SMTP (the email-sending protocol) (Single Responsibility).	**Pointless with one member** — a single implementor is an interface plus ceremony.

When not to use it

- **An axis has one member** — introduce it when the second channel actually shows up.
- **The axes are not truly independent.** If some combinations are forbidden (digests never by SMS), Bridge happily builds them and you add run-time checks that separate subclasses would have made impossible.
- **You can't say in one sentence which side is the abstraction** — the design isn't ready; a wrong split costs more than a few extra classes.
- **The second axis is a behaviour, not a kind of thing** — use Strategy.

Common mistakes

- Building the bridge "just in case" with a single implementor.
- Letting the abstraction branch on the concrete implementor type.
- Making the implementor interface mirror one specific implementation.
- Confusing Bridge with Adapter: Adapter is **retrofitted** to make existing things fit; Bridge is **designed in** so the sides never fuse.

Real-world examples

- **JDBC**: your code uses `java.sql.Connection` / `Statement`; each driver supplies the implementation, so the same query code runs on PostgreSQL or MySQL. (Often cited as both Bridge — two independent hierarchies — and Adapter — each driver adapts a native protocol. Both fair.)
- **SLF4J and similar logging APIs**: the API you call is one side, the backend writing log lines the other. (SLF4J calls itself a *facade*; the API/backend split underneath is Bridge-shaped.)

- **Window systems and device/graphics drivers**: GoF's motivating example — a `Window` abstraction whose `WindowImp` is written per platform (X Window System, IBM Presentation Manager); OS drawing APIs over per-card graphics drivers likewise.
- **`java.awt` peers** (the textbook Java example): each AWT component held a platform-specific peer doing the native drawing.
- **Notification / payment services**: a `NotificationService` interface with pluggable providers (SES, Twilio, FCM — Amazon's email service, an SMS provider, Google's push service) is this pattern, named or not.
- **Dependency-injection containers** (frameworks that create objects and hand each its dependencies) binding an interface to a different implementation per environment give configured, not coded, Bridge pairing.

In modern languages

- **First-class functions**: a one-method implementor can be a function — `Notification(channel = send_email)`.
- **Python / JavaScript**: duck typing — any object with a `deliver` method will do; no declared interface.
- **Go**: a small interface declared by the *consumer*, implemented in other packages, is idiomatic (conventional-style) Go — Bridge without the name.
- **DI containers** (Spring, Guice, .NET): pairing wired by configuration per environment.

Other patterns it is confused with

Looks like	Actual difference
Strategy / Adapter	Same code shape — see Bridge vs Strategy and the Adapter timing point in Common mistakes.
Abstract Factory	Decides *which family of objects gets created*; Bridge describes *how an object hands off work once it exists*. They combine well — a factory often supplies the implementor a bridge receives.
Decorator	Shares the inner object's interface and adds behaviour; in a Bridge the two sides have unrelated interfaces and neither wraps the other.

In the interview

30-second answer: "Bridge separates an abstraction from its implementation so both can vary independently. If notifications vary by type and by channel, one hierarchy needs type × channel classes — 3 × 3 = 9, and 16 with a fourth of each. I split it: a `Notification` hierarchy holding a reference to a `Channel` interface. Now it is type + channel classes — 6, then 8 — and the pairing is chosen at run time. I wouldn't build it while there is only one channel."

Follow-ups

- *Q: How is it different from Strategy?* — A: Same code, different intent. Bridge splits a permanent dimension decided up front; Strategy makes one algorithm replaceable, often per call.
- *Q: What is the smell that says you need it?* — A: Two-noun class names (`SmsDigest`) with duplicated code.
- *Q: When would you not use it?* — A: With one channel today — add it when the second lands. Also when some combinations are invalid.
- *Q: Who decides which implementor a notification gets?* — A: Whoever builds it — often a factory or the DI container, configured per environment.

Self-check

1. **Q:** With 5 notification types and 4 channels, how many classes without and with Bridge? **A:** 20 without, 9 with.
2. **Q:** Which field is "the bridge"? **A:** The `Channel channel` reference held by the `Notification` base class.
3. **Q:** An `AlertNotification` checks `channel instanceof SmsChannel`. What is wrong? **A:** Implementation detail has leaked upward; move that behaviour into `SmsChannel`.
4. **Q:** Is Bridge added before or after the classes exist? **A:** Before — it is an up-front structural decision (Adapter is the after-the-fact one).

TL;DR

- **Two dimensions varying → two hierarchies + one reference.** Multiplication becomes addition.
- Abstraction (what) holds Implementor (how). Keep the implementor small and general.

- Same code as Strategy; different intent. Don't build it for one implementor.

9.3 Composite Pattern

What it is

The **Composite** pattern gives an individual item and a whole nested group **one common interface**, so code never checks whether it has a **single object** or an **entire tree** — like asking anyone on an org chart "what is your team's total salary?": a manager asks each report and sums; an individual answers with their own.

Term	Meaning	In our code
Component	The shared interface for leaves and containers	`FileSystemNode`, `MenuComponent`
Leaf	An object with no children; does the work itself	`FileNode`, `MenuItem`
Composite	A container that holds children (typed as Component) and delegates to them	`FolderNode`, `Menu`
Client	Code that works with Components only	`report(...)`

GoF's intent: compose objects into tree structures representing **part-whole hierarchies**, and let clients treat individual objects and compositions **uniformly**.

The problem it solves

Much data nests: folders hold files *and* folders; an invoice holds items *and* bundles of items; a UI (user interface) panel holds widgets (buttons, text boxes and other controls) *and* sub-panels. Without the pattern every operation (`size`, `print`, `search`, `copy`) opens with the same type test:

```
if (node instanceof Folder f) { /* loop over children, recurse */ }
else if (node instanceof File f) { /* read the size */ }
```

Add a third node kind — a symbolic link (an entry pointing to another file or folder) — and you hunt every copy; miss one and it quietly gets the wrong answer.

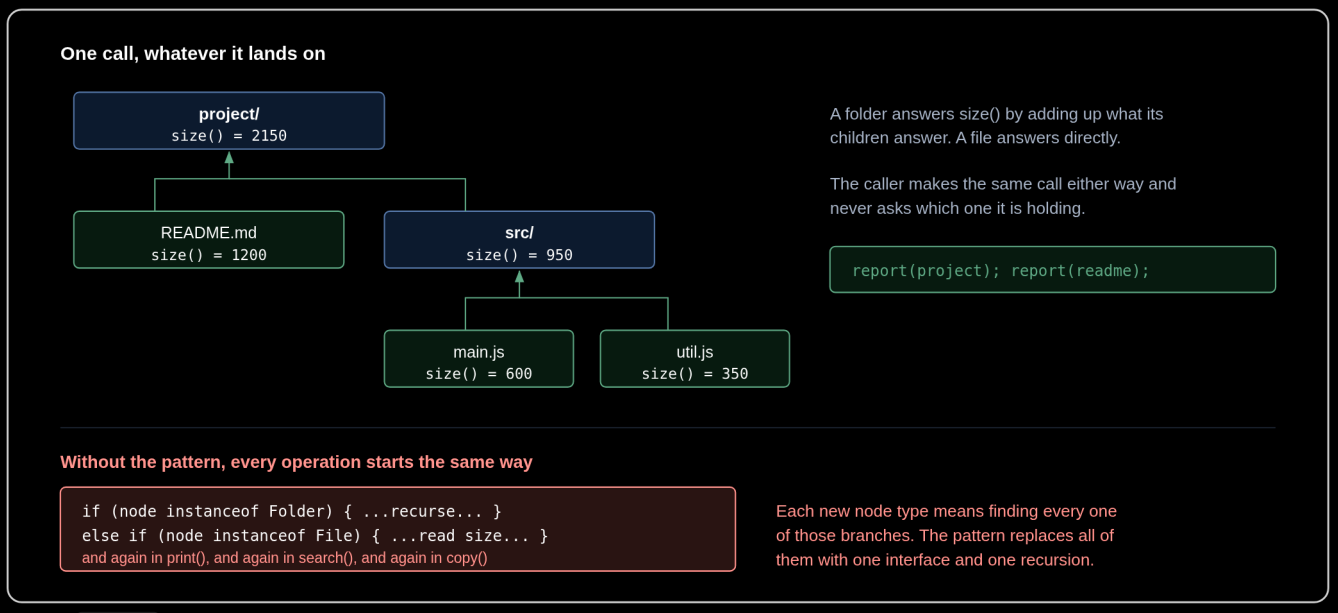


Figure — `size()` on the project folder (2150) is the sum of README.md (1200) and src/ (950), which itself sums main.js and util.js. The caller makes the same call on a tree or a single file (course diagram)

A file answers `size()` with its byte count; a folder asks each child and sums. Since a child may be either and both answer the same call, one piece of recursive code (code calling itself on smaller parts) handles any depth; the caller writes `report(project)` or `report(readme)` alike.

How it works

1. **Define a Component interface** with operations meaningful for *both* leaves and containers (`name()`, `size()`, `print()`).

2. **Leaf** implements them directly.
3. **Composite** stores a `List<Component>` — the **interface type**, not the leaf type — and implements each operation by **asking each child and combining results**.
4. Because a child can be a composite, the recursion is written **once** and works at any depth.
5. **Clients** hold a `Component` and never type-check.

Structure

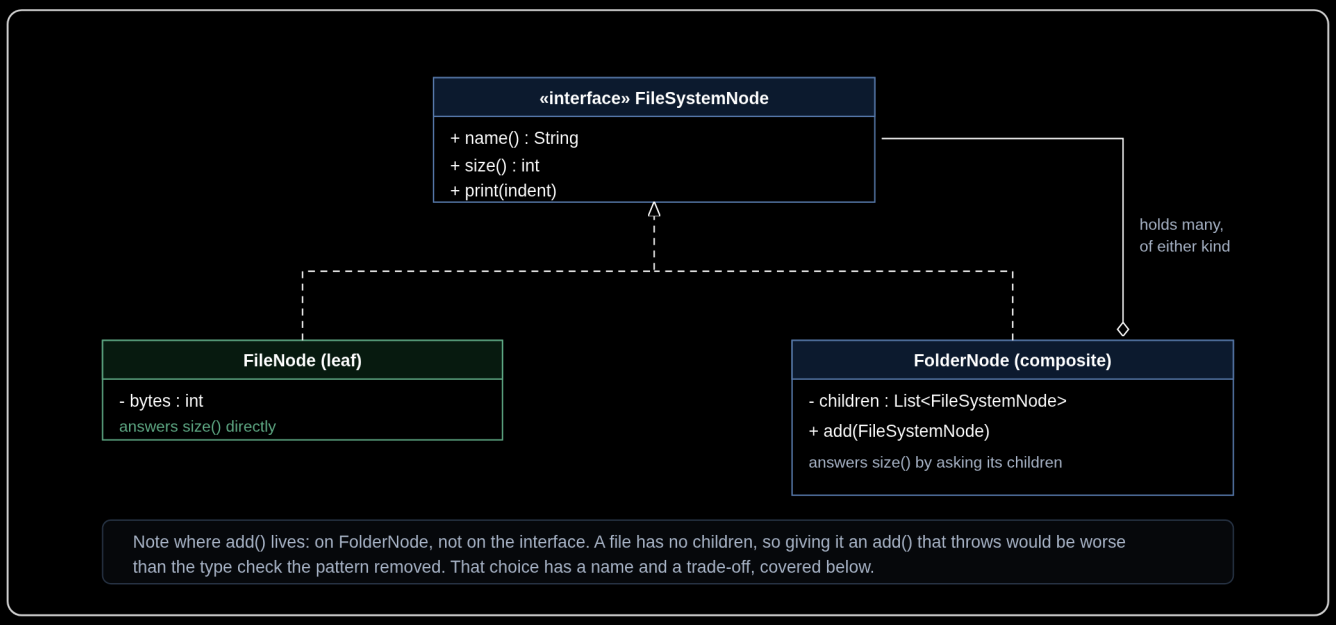
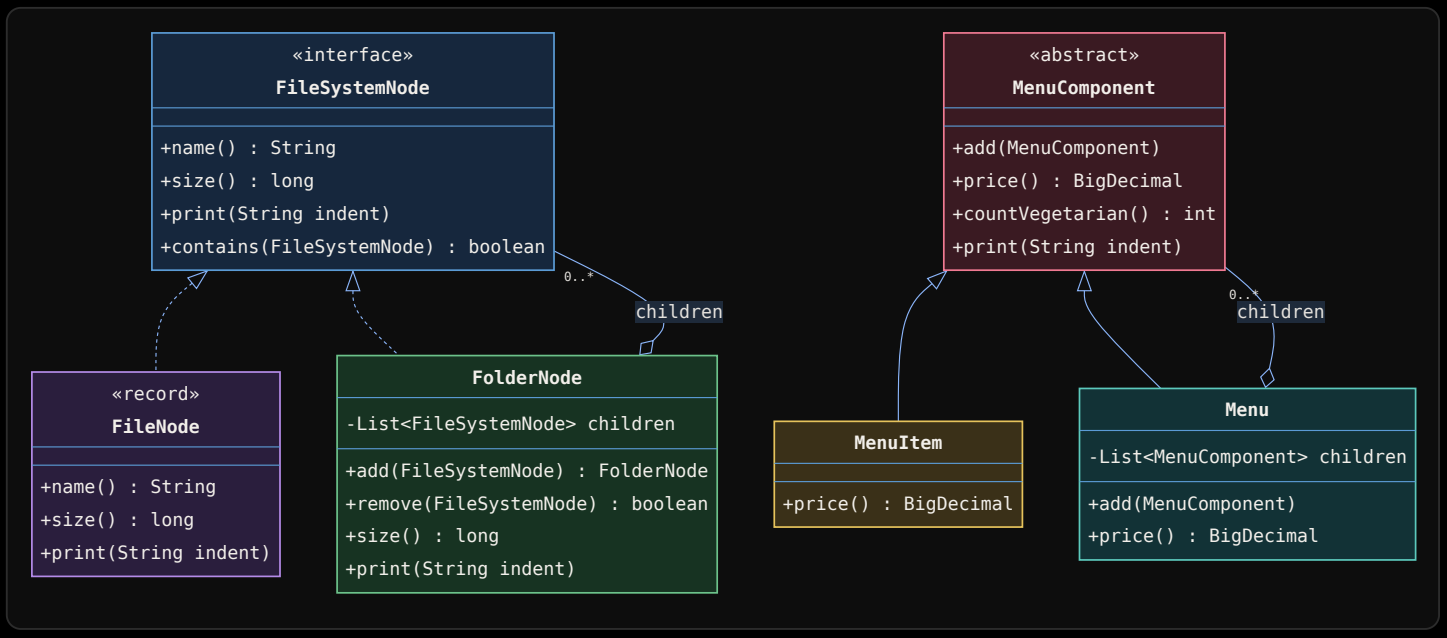


Figure — Composite class diagram: `FileNode` (leaf) and `FolderNode` (composite) implement `FileSystemNode`; the folder holds a list of `FileSystemNode` and has `add()` (course diagram)

Two details matter: the child list is typed as the **interface** (so folders hold folders), and `add()` is on `FolderNode`, not the shared interface — the deliberate "safe" design, covered after the code.



Java example

Full code: `code/lld/ch09/composite/` (file-system tree, restaurant menu, `Main`).

```java
// FileSystemNode.java — COMPONENT (safe version): only operations valid for BOTH; no add/remove here.
interface FileSystemNode {
    String name();
    long size();
    void print(String indent);
    /** True if 'target' is this node or somewhere below it. Used for the cycle check. */
    default boolean contains(FileSystemNode target) { return this == target; }
}

// FileNode.java — LEAF: no children, answers directly.
record FileNode(String name, long size) implements FileSystemNode {
    @Override public void print(String indent) { System.out.println(indent + "- " + name + " (" + size + " B)"); }
}

// FolderNode.java — COMPOSITE: children typed as the INTERFACE, so folders can hold folders.
final class FolderNode implements FileSystemNode {
    private final List<FileSystemNode> children = new ArrayList<>();

    /** Returns this so trees can be built fluently. Rejects cycles at insert time. */
    FolderNode add(FileSystemNode child) {
        if (child.contains(this)) throw new IllegalArgumentException("cycle: " + child.name() + " already contains " +
name);
        children.add(child);
        return this;
    }
    List<FileSystemNode> children() { return Collections.unmodifiableList(children); }

    @Override public long size() {                        // the recursion lives in ONE place
        long total = 0;
        for (FileSystemNode child : children) total += child.size();
        return total;
    }
    @Override public void print(String indent) {
        System.out.println(indent + "+ " + name + "/ (" + size() + " B)");
        for (FileSystemNode child : children) child.print(indent + "   ");
    }
    @Override public boolean contains(FileSystemNode target) {
        if (this == target) return true;
        for (FileSystemNode child : children) if (child.contains(target)) return true;
        return false;
    }
    ...
}
```

```java
// MenuComponent.java — COMPONENT (transparent version): child management on the shared type,
// so callers never cast - but a leaf must reject it at run time.
abstract class MenuComponent {
    void add(MenuComponent c) { throw new UnsupportedOperationException(name() + " is a leaf"); }
    abstract String name();
    abstract BigDecimal price();      // total under this node (money: BigDecimal, never double)
    abstract int countVegetarian();
    abstract void print(String indent);
}
// MenuItem: price = new BigDecimal("3.49") (from a String, so exactly 3.49).
// Menu: overrides add(); price() = children.stream().map(MenuComponent::price).reduce(BigDecimal.ZERO, BigDecimal::add)

// Main.java — client: one parameter type, no instanceof anywhere.
static void report(FileSystemNode node) { System.out.println("  " + node.name() + " -> " + node.size() + " bytes"); }
// report(project); report(readme); src.add(project) is rejected (project already contains src);
// new MenuItem("Tea", "1.00", true).add(...) throws.
```

Output:

```
== Safe composite: file system ==
  + project/ (2150 B)
     - README.md (1200 B)
     + src/ (950 B)
        - main.js (600 B)
        - util.js (350 B)
  project -> 2150 bytes
  README.md -> 1200 bytes
  rejected: cycle: project already contains src
== Transparent composite: restaurant menu ==
  [All menus]
    [Breakfast]
      Pancakes (v) $3.49
      Bacon & eggs $4.99
      [Desserts]
        Apple pie (v) $2.59
    Coffee (v) $1.50
  total $12.57, vegetarian items: 3
  transparent cost: Tea is a leaf
```

What each part does

- `FileSystemNode` (Component) / `FileNode` (immutable record Leaf) / `FolderNode` (Composite) — `FolderNode.size()` is the four-line recursion and the **only** place that knows a tree nests; `add()` returns `this` for fluent building.
- `contains(...)` + the check in `add()` — **cycle protection** at insert time (putting `project` inside its own child `src` is rejected).
- `MenuComponent` / `Menu` / `MenuItem` — the *transparent* variant (*Head First*'s restaurant menu): `add` on the shared abstract class, a leaf throws. `report(...)` is the client, called with a tree and a single file.

Transparent vs safe composite

Where do `add` and `remove` belong? GoF named both answers:

	Transparent	Safe
Where `add/remove` live	On the shared Component	Only on the Composite
Uniformity	Full — every node looks the same; no casts (forcing a value to a more specific type)	Partial — builders must hold a `FolderNode`
Type safety	Weaker — `leaf.add(...)` compiles, then **throws at run time**	Strong — the compiler rejects `file.add(...)`
Our code	`MenuComponent` (`Tea is a leaf` in the output)	`FileSystemNode` / `FolderNode`
Choose when	Callers add children to arbitrary nodes without knowing their type (e.g. a UI editor where anything might be a container)	The tree is built in one place and read in many (most cases)

Transparent trades a compile-time guarantee for a possible run-time exception. Safe keeps the read-side interface small, but building/editing code must hold a `FolderNode`, and code that walks then inserts needs a cast after all. Neither is wrong; in review check the choice was **made on purpose** — an unintended throwing `add` is the real defect. *Head First* notes the transparent composite knowingly bends Single Responsibility (the component both manages children and does work) to gain transparency.

Two traps worth knowing

- **Cycles.** A folder placed inside its own descendant makes `size()` recurse forever (`StackOverflowError`, when nested calls exhaust the call stack — the memory tracking calls in progress). **Validate when inserting**, not during traversal (walking every node) — by then you are stuck. Our `add()` does this.
- **Repeated work.** `print()` calls `size()` at every level, recomputing subtotals — roughly quadratic (work grows with the square of depth) for a deep chain. For big trees **store each subtree's total** and invalidate upward on change (needs a parent pointer).

Trade-offs

Pros	Cons
Callers never type-check — same call on a leaf or a tree.	**The shared interface bloats** with operations that half-fit one kind.
The recursion lives in one place, reused by each new operation.	**Weaker type safety** (transparent) — adding a child to a leaf compiles.
New node types don't touch callers (Open/Closed) — e.g. a `SymlinkNode`.	**Deep trees mean deep call stacks**; a cycle shows as endless recursion, not a clear error.
Nesting comes free — a composite storing the interface stores composites.	**Totals recomputed** each call unless cached.

When not to use it

- **Nothing nests** — a flat list gains nothing.
- **Leaves and containers share little** — a forced interface fills with throwing methods, worse than the type test it replaced.
- **Deep tree on a hot path** (code run so often its speed matters) — each level costs a stack frame and likely CPU cache misses (fetching from slow main memory instead of the processor's fast cache); precomputed totals or a loop with its own work-list can be faster.
- **The tree is modified during traversal** — concurrent changes (by another thread at the same time) during recursion are dangerous; Composite offers no protection.

Common mistakes

- Typing the child list as the **leaf** type (`List<FileNode>`) — folders cannot hold folders.
- Returning the internal mutable child list — callers bypass `add()` and its cycle check (we return `Collections.unmodifiableList(children)`).
- Putting `add()` on the interface by accident (transparent without meaning to).
- No cycle check.
- Using Composite when one recursive function with one type check would do — it pays off when **operations multiply**.

Real-world examples

- **The HTML DOM** (Document Object Model — the browser's in-memory tree of page elements) — elements hold child nodes; `textContent` walks the subtree.
- **UI toolkits** — Swing/AWT's `java.awt.Container` *is a* `Component` and holds `Component`s, so panels nest without limit; React (a JavaScript UI library) component trees and Android's `ViewGroup` (a `View` holding `View`s) likewise.
- **File systems** — the example used here.
- **Abstract syntax trees** (the tree a compiler builds from source) — expression nodes hold sub-expressions; evaluating a node evaluates its subtree.
- **`java.io.File`** represents files and directories with one class — the transparent cost: `listFiles()` returns `null` on a regular file.
- **JUnit 3 test suites** (JUnit: the standard Java testing framework; a suite is a named group of tests) — `TestSuite` implements `Test` and holds `Test`s; running a suite runs everything beneath. (JUnit 5 keeps a tree of test descriptors.)
- **Spring's `CompositeCacheManager`** delegates to a list of `CacheManager`s while being one.
- **Invoices / bundles** — a "bundle" priced as the sum of items that may themselves be bundles.

In modern languages

- **Python / JavaScript**: duck typing — leaf and container just share method names; a composite is often a class with a `children` list.
- **Sealed types + pattern matching** (Java 21, Kotlin, Scala, Rust) — a sealed type lists exactly which subtypes exist and a `switch` branches on the subtype: `sealed interface Node permits File, Folder` plus an exhaustive `switch`. The compiler reports any node type a `switch` forgets — a guarantee Composite lacks. Prefer it when node types are closed.

- **Functional languages**: a recursive algebraic data type ("one of these shapes", a shape may contain the type itself) is Composite with language support; a `fold` (combine all parts into one result) is the recursion written once.
- **Java streams** (pipeline API for collections): `flatMap` (each element → several, flattened) makes walking a composite read nicely.

Other patterns it is confused with

Looks like	Actual difference
Decorator	Both hold objects of their own interface; Decorator wraps **one** and adds behaviour, Composite holds **many** and combines results.
Iterator	Composite gives the tree a uniform interface; Iterator walks it without exposing its structure. Often used together.
Visitor	Composite makes the structure uniform; Visitor adds new operations without editing node classes — the usual next step when operations multiply.
A plain recursive function	If one function with one type check does the job, skip the pattern. Composite pays off when **many** operations would repeat that branch.

In the interview

30-second answer: "Composite lets a client treat a single object and a tree of objects the same way. Leaf and container share one interface; the container stores children *typed as that interface* — which is what allows arbitrary nesting — and implements each operation by delegating to its children and combining the results. Its value is what it deletes: the `instanceof` branch in every operation. I choose between the transparent and safe variants depending on whether callers need `add` on arbitrary nodes, I reject cycles on insert, and I cache subtree totals if the tree is large."

Follow-ups

- *Q: Transparent or safe?* — A: Safe if built in one place and read in many; transparent if callers add children to arbitrary nodes and casts would spread.
- *Q: What about cycles?* — A: Check on insert: reject `add(child)` if the child already contains the parent.
- *Q: Performance on a big tree?* — A: Cache subtree totals, invalidate upward on change; or walk iteratively to avoid deep recursion.
- *Q: Composite vs Decorator?* — A: Decorator holds **one** and adds behaviour; Composite holds **many** and combines results.
- *Q: How do you add a new operation without touching every node?* — A: Visitor over the composite structure.

Self-check

1. **Q:** Why must the child list be `List<FileSystemNode>`, not `List<FileNode>`? **A:** So a folder can contain other folders — arbitrary nesting.
2. **Q:** In the transparent variant, what happens when you call `add` on a leaf? **A:** It compiles but throws at run time (`UnsupportedOperationException`).
3. **Q:** Where should a cycle be detected? **A:** At insertion time.
4. **Q:** Name a JDK class that is both a component and a container of components. **A:** `java.awt.Container`.

TL;DR

- **Part-whole trees, one interface.** Leaf answers directly; composite asks children and combines.
- Child list holds the **interface**. Recursion lives in one place.
- **Safe** (add only on composite, compile-time safe) vs **transparent** (add everywhere, run-time throw). Guard against **cycles**; cache big totals.

9.4 Decorator Pattern

What it is

The **Decorator** pattern extends what **a single object** does, **while the program runs**, by enclosing it in a wrapper that implements **the same interface**. The original class is never edited and you need no subclass per mix of extras — like an espresso wrapped in milk, mocha and whipped cream, each topping adding to price and description while the result is still "a drink".

Term	Meaning	In our code
Component	The shared interface	`UserRepository`, `Beverage`, `InputStream`
Concrete Component	The real object that does the core work	`DatabaseUserRepository`, `Espresso`
Decorator (base)	Implements the Component, holds a Component, forwards everything	`RepositoryDecorator`, `java.io.FilterInputStream`
Concrete Decorator	Adds one behaviour around the forwarded call	`LoggingRepository`, `CachingRepository`, `Mocha`, `UpperCaseInputStream`

GoF's intent: *attach additional responsibilities to an object dynamically — a flexible alternative to subclassing for extending functionality.* Also called **Wrapper** (Adapter's nickname too).

The key property: **a decorator IS-A Component and HAS-A Component.** So it can wrap the real object, another decorator, or a test double (any test stand-in — fake, stub or mock), and interface-only code cannot tell which.

The problem it solves

A repository (an object that loads and saves data, hiding the database behind methods) needs logging, then caching, later retries and metrics (counters and timings the app records). Each caller wants its own mix, sometimes in its own order.

- **Subclassing can't keep up.** `LoggingCachingRetryingUserRepository` serves one mix; four optional concerns give 2^4 = **16** mixes, doubling per new concern — a class explosion.
- **One class with on/off flags** lasts until someone wants "everything except the cache": `new Repo(true, false, true, true)`, unreadable.

Decorator gives **each concern its own class** with the component's interface: four concerns, **four classes**, each caller assembling its stack by wrapping at run time rather than inheriting at compile time — Open/Closed: new behaviour as new classes, existing ones untouched.

How it works

1. Start from a **Component interface** (`UserRepository`).
2. Write the **concrete component** doing the real work (`DatabaseUserRepository`).
3. Write an **abstract base decorator** implementing the interface, holding an `inner` of the **interface type**, **forwarding every method** by default.
4. Each **concrete decorator** extends the base and overrides only what it changes, acting **before and/or after** calling `inner`.
5. **Compose by wrapping**: `new Logging(new Caching(new Database()))`; the caller holds the outermost.

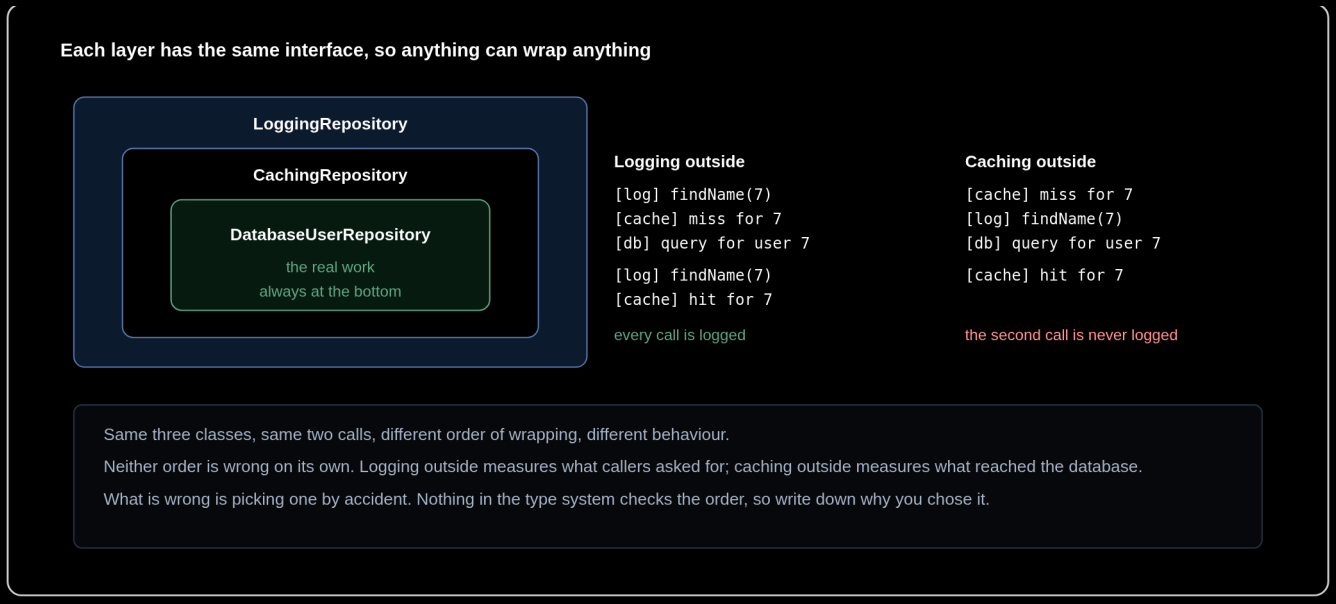


Figure — The same three classes stacked in two orders. Logging outside caching logs every call; caching outside logging means the second call (a cache hit) is never logged (course diagram)

Structure

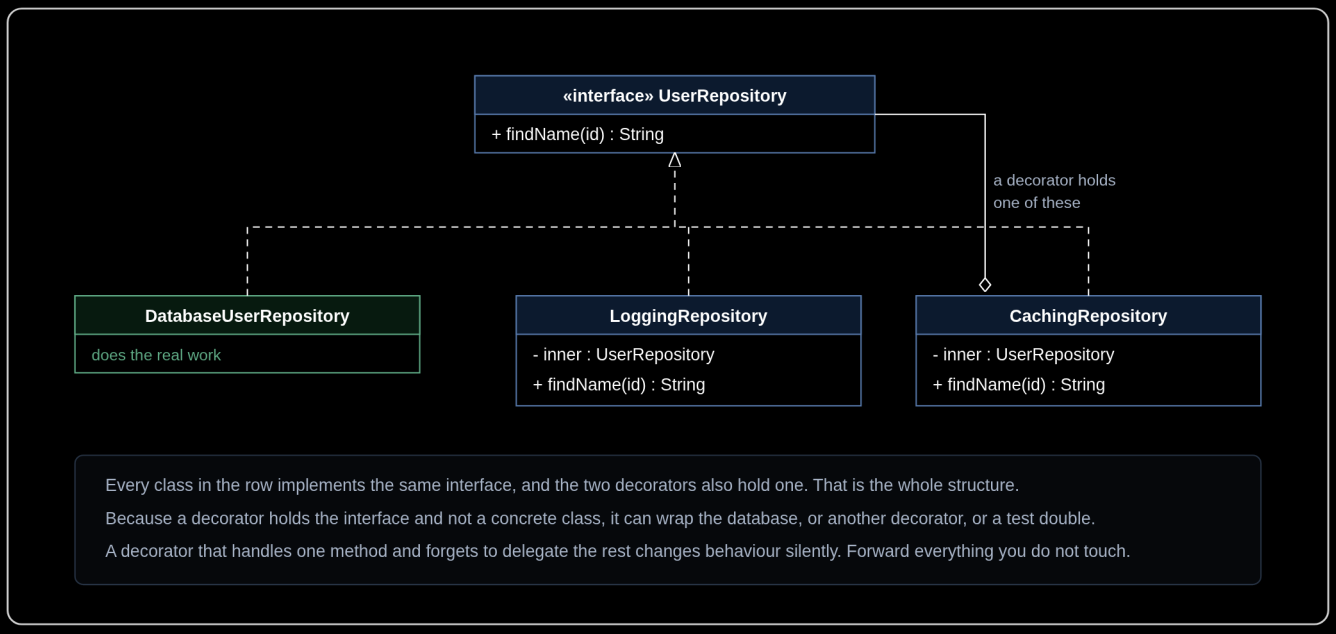
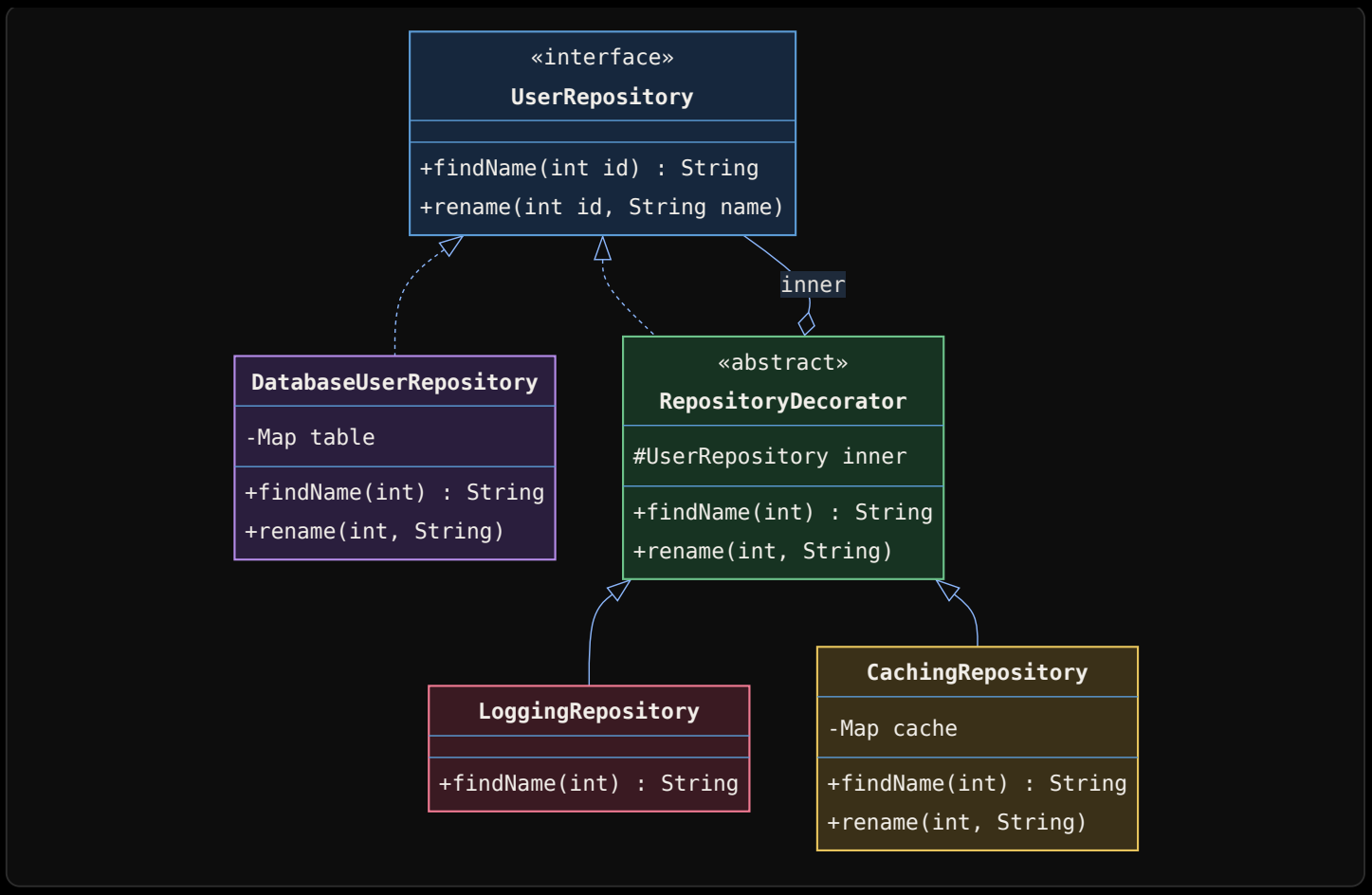


Figure — Decorator class diagram: `DatabaseUserRepository`, `LoggingRepository` and `CachingRepository` all implement `UserRepository`; each decorator holds an `inner` UserRepository (course diagram)

Every class implements `UserRepository`; `DatabaseUserRepository` sits at the bottom wrapping nothing. Because `inner` is typed as the interface, decorators wrap each other in any combination and order.

What bites in practice is **delegation**: a decorator must forward every method it is not deliberately changing. One overriding `findName` but silently swallowing `rename` changes behaviour invisibly. Java's usual guard is an abstract base decorator forwarding everything (`RepositoryDecorator`).



A decorated call, logging outside caching: `Logging.findName(7)` logs → `Caching.findName(7)` misses → `Database.findName(7)` returns "Ada" → caching stores it → "Ada" flows back to the caller (first block of the output below).

Java example

Full code: `code/lld/ch09/decorator/` (repository stack, coffee `Beverage` records, `UpperCaseInputStream`, `Main`).

```
// RepositoryDecorator.java — BASE DECORATOR: holds the INTERFACE and forwards everything,
// so subclasses override only what they change and no method is ever forgotten.
abstract class RepositoryDecorator implements UserRepository {
    protected final UserRepository inner;
    protected RepositoryDecorator(UserRepository inner) { this.inner = inner; }
    @Override public String findName(int id)          { return inner.findName(id); }
    @Override public void rename(int id, String name) { inner.rename(id, name); }
}

// LoggingRepository.java — CONCRETE DECORATOR 1
@Override public String findName(int id) {
    System.out.println("    [log]   findName(" + id + ")");
    return super.findName(id);
}

// CachingRepository.java — CONCRETE DECORATOR 2: must also invalidate on writes.
private final Map<Integer, String> cache = new HashMap<>();
@Override public String findName(int id) {
    if (cache.containsKey(id)) { System.out.println("    [cache] hit for " + id); return cache.get(id); }
    System.out.println("    [cache] miss for " + id);
    String name = super.findName(id);
    cache.put(id, name);
    return name;
}
@Override public void rename(int id, String name) { cache.remove(id); super.rename(id, name); }
```

```java
// Beverage decorators — prices in whole cents (int), no floating-point money.
interface Beverage { String description(); int costCents(); }
record Espresso() implements Beverage { public String description() { return "Espresso"; } public int costCents() { return
199; } }
/** Add-ons ARE a Beverage and HAVE a Beverage. (Milk +10, Whip +10 likewise; HouseBlend costs 89.) */
record Mocha(Beverage inner) implements Beverage {
    public String description() { return inner.description() + ", Mocha"; }
    public int costCents() { return inner.costCents() + 20; }
}

// UpperCaseInputStream.java — our own java.io decorator; FilterInputStream is the JDK's base decorator.
// Works byte-by-byte, so only correct for ASCII text.
final class UpperCaseInputStream extends FilterInputStream {
    UpperCaseInputStream(InputStream in) { super(in); }
    @Override public int read() throws IOException {
        int b = super.read();
        return b == -1 ? -1 : Character.toUpperCase((char) b);
    }
    // read(byte[], int, int) upper-cases each byte read likewise
}

// Main.java
lookUpTwice("Logging outside caching (logs every caller request):",
        new LoggingRepository(new CachingRepository(new DatabaseUserRepository())));
lookUpTwice("Caching outside logging (logs only what reaches the db):",
        new CachingRepository(new LoggingRepository(new DatabaseUserRepository())));
Beverage order = new Whip(new Mocha(new Mocha(new Espresso())));
InputStream in = new UpperCaseInputStream(new BufferedInputStream(new ByteArrayInputStream(...)));
```

Output:

```
== Same layers, different order ==
  Logging outside caching (logs every caller request):
    [log]   findName(7)
    [cache] miss for 7
    [db]    query for user 7
    [log]   findName(7)
    [cache] hit for 7
  Caching outside logging (logs only what reaches the db):
    [cache] miss for 7
    [log]   findName(7)
    [db]    query for user 7
    [cache] hit for 7
== Write goes through every layer and invalidates the cache ==
    [log]   findName(8)
    [cache] miss for 8
    [db]    query for user 8
    [db]    update user 8
    [log]   findName(8)
    [cache] miss for 8
    [db]    query for user 8
  now: Linus T.
== Coffee: stack add-ons at run time ==
  Espresso, Mocha, Mocha, Whip = $2.49
  House Blend, Milk = $0.99
== java.io is built from decorators ==
  DECORATORS WRAP STREAMS
```

What each part does

- `RepositoryDecorator` — the **abstract base decorator** forwarding *every* method, so a decorator caring only about `findName` cannot drop `rename` — the standard Java defence against forgotten delegation.
- `LoggingRepository` — prints, then delegates.
- `CachingRepository` — answers from its map on a hit (value already cached; a *miss* means not), otherwise delegates and stores (more on caches in System Design Fundamentals). Its `rename` **invalidates** (throws away the now-stale copy) — the third output block shows the next read hitting the database.
- `Beverage` records — *Head First*'s coffee example, rewritten: `new Whip(new Mocha(new Mocha(new Espresso())))` = 1.99 + 0.20 + 0.20 + 0.10 = **$2.49**. Double mocha is wrapping twice — impossible with one subclass per combination.
- `UpperCaseInputStream extends FilterInputStream` — `FilterInputStream` holds an `InputStream` and forwards to it.

Layer order changes behaviour

- **Logger outside, cache inside**: every request is logged, even cache-served ones → the log records *what callers requested*.
- **Cache outside, logger inside**: only misses are logged → the log records *what hit the database*.

Both are valid; they answer different questions. The mistake is choosing **without meaning to** and reading the log as answering the other. The same choice recurs wherever layers stack:

Pair	Outside	Inside
Retry vs cache	Repeats the entire lookup	Repeats only the database call, and a retry that succeeds populates the cache. Rule of thumb: put retry **next to the thing that fails**.
Metrics vs cache	Times what the caller experiences	Times the database alone — the number an on-call engineer (whoever currently answers alerts) is paged on. Many teams record both.
Authorization (checking the caller is allowed) vs cache	**Correct** — every request is checked	**Security bug** — a cached answer is returned without the check ever running

The type checker accepts every order. **Record why in a comment where the stack is built**, or the next person adding a layer will guess.

Two related traps

- **Stateful decorators (keeping data between calls) that assume one wrap.** Apply a counting decorator twice and every call counts twice.
- **Lost identity.** The wrapper is a different object, so `==` (same object?), `equals` (equal value?) and `instanceof` surprise callers. Code digging through layers to "reach the real object" fights the pattern.

Decorator vs inheritance

	Subclassing	Decorator
When behaviour is chosen	Compile time	Run time
How many at once	One subclass	Any number of layers, any order
Classes for N optional features	Up to 2^N	N
Works on `final` classes / classes you don't own	No	Yes (only needs the interface)
Can reach private state	Protected members, yes	No — only the public interface

Java I/O streams: the decorator you already use

```
new BufferedReader(new InputStreamReader(new FileInputStream("data.txt"), UTF_8))
```

- `FileInputStream` — concrete component (reads bytes from a file).
- `InputStreamReader` — strictly an **adapter** (bytes → characters), not a decorator.
- `BufferedReader` — a **decorator** on `Reader` (adds buffering — reading big chunks to serve many small reads — and `readLine()`).

On the byte side `FilterInputStream` is the abstract decorator; `BufferedInputStream`, `DataInputStream`, `PushbackInputStream`, `java.util.zip.GZIPInputStream` (via `InflaterInputStream`) and `java.security.DigestInputStream` are concrete decorators. *Head First* notes the downside: many small classes that overwhelm newcomers until they see the pattern.

Trade-offs

Pros	Cons
Any mix without a class per mix — 4 concerns need 4 classes, not 16.	**Deeper stack traces** — the real work is buried under every layer.
Concerns tested in isolation — logging without a database, caching without a logger (Single Responsibility).	**Identity surprises** — `equals` / `instanceof` on the wrapper don't match the wrapped object.
Assembled at run time — per caller, environment or feature flag (an on/off feature switch).	**Ordering matters for correctness**, and no compiler checks it.
The wrapped class is never edited (Open/Closed) — works on classes you don't own.	**Over-layering** — eight wrappers deep, nobody knows which changed a value.

When not to use it

- **Exactly one concern, forever** — put it in the class; a decorator for one behaviour is a class wrapped around an `if`.
- **The concern needs private state** — a decorator only sees the public interface.
- **The framework already offers it** — middleware and interceptors (code a framework runs around every request or call), Spring AOP aspects (aspect-oriented programming: code applied automatically around chosen methods) are decorators with tooling.
- **The stack would get deep** — each layer is another stack-trace frame and "which one changed this?" becomes a long debugging session.

Common mistakes

- Holding the **concrete class** (`private DatabaseUserRepository inner`) — decorators cannot stack.
- **Forgetting to forward** some methods (use an abstract base decorator).
- A caching decorator that **doesn't invalidate on writes** (ours does in `rename`).
- Putting authorization **inside** a cache.
- Relying on `instanceof` of the outer object.

Real-world examples

- **`java.io` streams** — above.
- **`java.util.Collections.unmodifiableList/synchronizedList/checkedList`** — same interface, one changed behaviour (read-only, synchronized, type-checked). Some call `unmodifiableList` a protection **proxy**; both labels are defensible.
- **`javax.servlet.http.HttpServletRequestWrapper`** (Jakarta: `jakarta.servlet...`) — a base decorator you subclass to change request methods in a filter (code running before and after each web request in a Java servlet application).
- **HTTP middleware** — Express, Django, ASP.NET and Go's `http.Handler` wrap a handler in layers (logging, auth, compression, rate limiting — capping requests per client per time window): this pattern with framework support.
- **Resilience4j (Java) and Polly (.NET)** — compose retry, circuit breaker (stops calling a failing service for a while), timeout and bulkhead (separate limited resources so one slow dependency cannot use them all) as stacked wrappers, e.g. `Decorators.ofSupplier(...).withCircuitBreaker(...).withRetry(...).withBulkhead(...).decorate()`; both document ordering carefully because it changes behaviour.
- **Python's `@functools.lru_cache`** (and `@retry` from libraries such as tenacity) — function decorators (LRU = least recently used: a full cache drops the longest-unused entry): the same idea on functions, hence Python's name.

In modern languages

- **Java**: abstract forwarding base class (as here), or a dynamic proxy (a class generated at run time; see 9.7) generating the forwarding.
- **Python**: function decorators (`@lru_cache`) are this pattern with syntax; for objects, `__getattr__` forwarding (and `functools.wraps` for functions) avoids hand-writing every delegating method.
- **JavaScript / TypeScript**: a higher-order function (takes or returns functions) returning a wrapped function; a `Proxy` object can forward every method automatically.

- **Go**: a struct (Go's record-like type) embedding the interface value gets delegation free — why Go middleware reads so cleanly.
- **C#**: `DelegatingHandler` in `HttpClient` is a framework-provided decorator chain.

In the interview

> **30-second answer:** "Decorator adds responsibilities to an object at run time by wrapping it in another object with the same interface. Without it, four optional concerns mean sixteen subclasses; with it, each concern — logging, caching, retries — is one class and I compose the stack I need. The decorator holds the interface, not the concrete class, which is what lets layers stack. And order matters: retry goes closest to what fails, and authorization must sit outside a cache or a cache hit skips the check."

Follow-ups

- *Q: Decorator vs Proxy?* — A: Same structure; different intent. Decorator **adds behaviour** and normally forwards; the client hands it the object to wrap and often stacks several. Proxy **controls access** — may defer, cache or refuse — often creates or locates the real object itself, and is handed to a client that doesn't know it isn't the real thing. The boundary is genuinely fuzzy (a caching wrapper is fairly called either); saying so beats pretending it is sharp.
- *Q: Decorator vs Adapter?* — A: Decorator keeps the interface, changes behaviour; Adapter changes the interface, keeps behaviour.
- *Q: Decorator vs Chain of Responsibility?* — A: Both chain objects. In Decorator every layer normally does its work and passes the call to the core (a cache hit is the usual exception); in Chain of Responsibility typically one handler handles the request and the rest never see it.
- *Q: Decorator vs Composite?* — A: Decorator wraps exactly one child and adds behaviour; Composite holds many and aggregates.
- *Q: Give a JDK example.* — A: `new BufferedInputStream(new FileInputStream(f))`; `FilterInputStream` is the base decorator.

Self-check

1. **Q:** How many classes for 5 optional features with decorators vs one subclass per combination? **A:** 5 vs up to 32.
2. **Q:** Why does `RepositoryDecorator` forward every method? **A:** So concrete decorators cannot silently drop methods they don't override.
3. **Q:** Why is "authorization inside cache" a bug? **A:** A cache hit returns data without ever running the authorization check.
4. **Q:** Is `InputStreamReader` a decorator? **A:** No — it changes the interface (bytes to chars), so it is an adapter. `BufferedReader` is the decorator.

TL;DR

> - **Wrap to add behaviour, keep the interface.** IS-A and HAS-A the same type.
> - N features = N classes, stacked at run time, in any order — and **order matters**.
> - Java I/O, `Collections.synchronizedList`, middleware, Resilience4j are all decorators. A decorator normally forwards every call; one that short-circuits (a cache hit) is drifting towards Proxy.

9.5 Facade Pattern

What it is

The **Facade** pattern gives a complicated subsystem **one simple front door**, so most callers never touch the machinery. GoF's intent: *provide a unified interface to a set of interfaces in a subsystem; the facade defines a higher-level interface that makes the subsystem easier to use.* Like a hotel concierge: "book me dinner and a taxi" and they call restaurant, taxi and front desk in order — though you can still phone the restaurant yourself.

Term	Meaning	In our code
Facade	Knows which subsystem classes to call, in what order, and how to clean up	`OrderFacade`
Subsystem classes	Do the real work; unaware of the facade	`InventoryService`, `PaymentService`, `ShippingService`, `EmailService`
Client	Uses the facade instead of the subsystem	`Main` / a checkout page

The problem it solves

One useful action needs several objects in the right order: **reserve stock → charge the card → create a shipment → send a confirmation**. Every place that places an order repeats those four calls — and must know what to do if the card is declined **after** stock was reserved.

That recovery step is where bugs live: copying the success path is easy, copying the undo correctly is not. A caller forgetting to release the reservation leaves units locked for an order never placed. The subsystem is not wrong — it is **flexible**, meaning many knobs — but nearly every caller wants the same standard sequence and writes it out again.

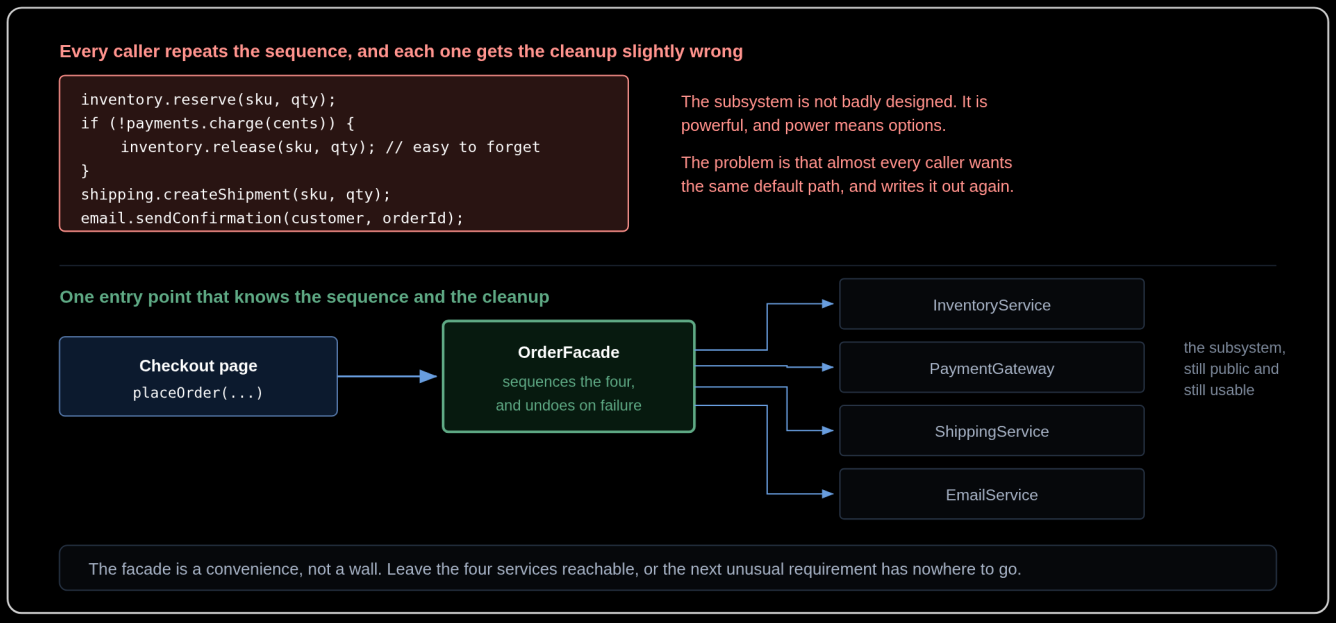


Figure — Top: each caller repeats the reserve/charge/release/ship/email sequence and may forget the release. Bottom: the checkout page calls `OrderFacade.placeOrder(...)`, which sequences the four services and undoes on failure; the services stay public (course diagram)

How it works

1. **Identify the common path** many callers repeat.
2. **Create a facade class** holding the subsystem objects.
3. **Expose one intention-revealing method** (`placeOrder`) calling them in order.
4. **Own the failure handling** — if payment fails, release the reservation (a *compensating action*).
5. **Keep the subsystem public** for unusual callers.
6. **Add no business rules** — the facade *sequences*, it does not *decide*.

Structure

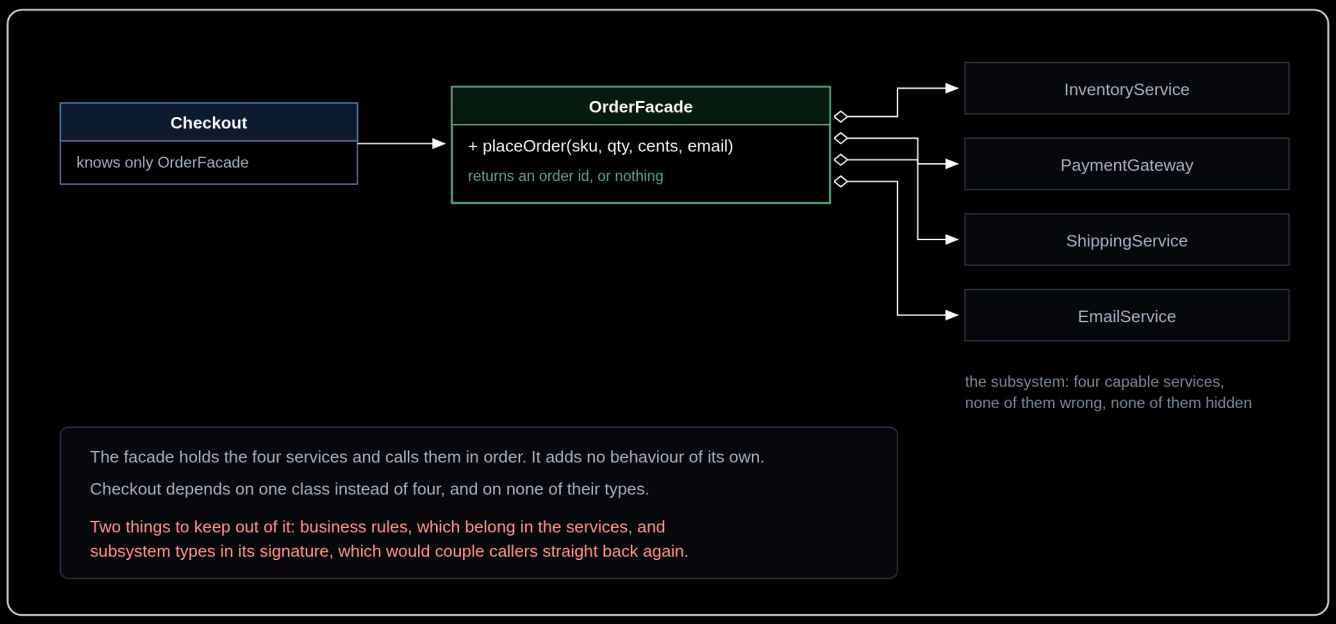
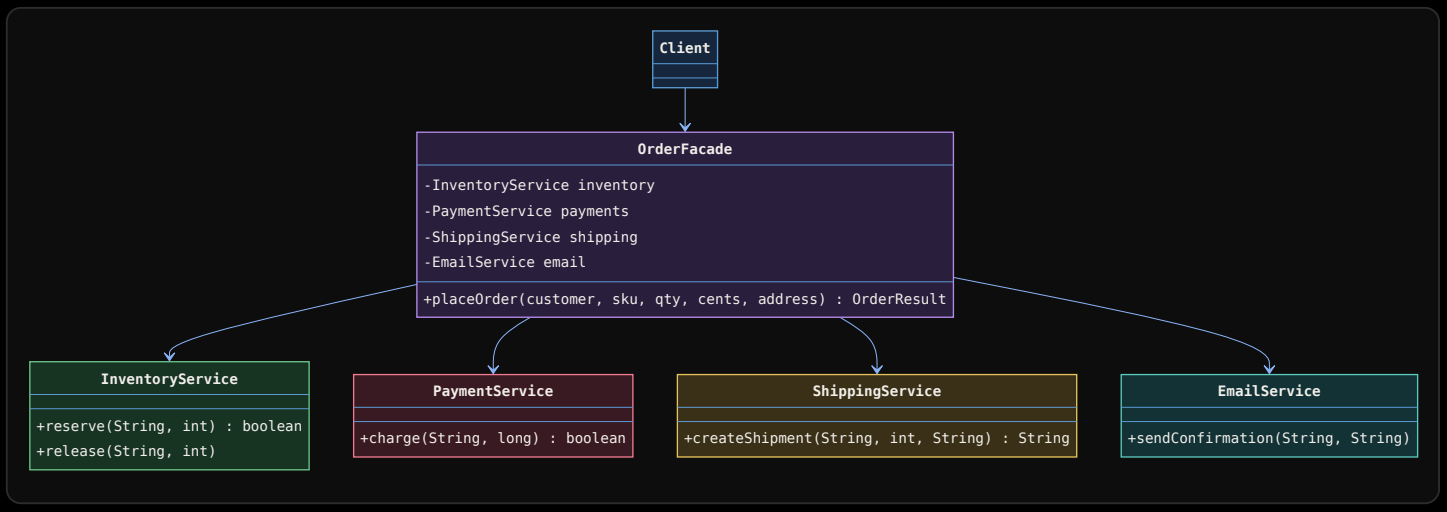


Figure — Facade class diagram: `Checkout` depends only on `OrderFacade`, which holds the four services (shown in grey) and exposes `placeOrder`. The figure labels the payment service `PaymentGateway` and shows a shorter `placeOrder` signature; our code calls it `PaymentService` and also passes customer and address (course diagram)

Dependencies run **one way**: no service knows the facade exists, so each stays usable alone. There is no interface — a facade is normally a concrete convenience class; add one only to substitute the entire subsystem, a different job.



Java example

Full code: `code/lld/ch09/facade/` — four independent, still-public subsystem services (`InventoryService` starts with 5 × `BOOK-42`; `PaymentService` declines anything over $100 to show the failure path), `OrderResult`, `OrderFacade`, `Main`.

```java
// OrderResult.java — callers get our own small result type.
record OrderResult(boolean success, String orderId, String message) { }

// OrderFacade.java — one method that knows the correct SEQUENCE and the CLEANUP.
// It sequences; it does not decide business rules (no pricing, no fraud logic here).
OrderResult placeOrder(String customer, String sku, int qty, long cents, String address) {
    String orderId = "ORD-" + orderSeq++;
    if (!inventory.reserve(sku, qty)) {
        return new OrderResult(false, orderId, "out of stock");
    }
    if (!payments.charge(customer, cents)) {
        inventory.release(sku, qty);            // compensating step callers used to forget
        return new OrderResult(false, orderId, "payment declined");
    }
    String shipmentId = shipping.createShipment(sku, qty, address);
    email.sendConfirmation(customer, orderId);
    return new OrderResult(true, orderId, "shipped as " + shipmentId);
}

// Main.java
orders.placeOrder("ada@x.com", "BOOK-42", 2, 5_000, "London");   // happy path
orders.placeOrder("bob@x.com", "BOOK-42", 1, 25_000, "Paris");   // declined -> released
orders.placeOrder("cy@x.com", "BOOK-42", 99, 1_000, "Rome");     // out of stock
inventory.reserve("BOOK-42", 1);   // subsystem still reachable, e.g. a warehouse tool
```

Output:

```
== Happy path: one call instead of four ==
  [inventory] reserved 2 x BOOK-42 (left 3)
  [payment]   charge ada@x.com 5000c -> OK
  [shipping]  SHP-1: 2 x BOOK-42 to London
  [email]     confirmation for ORD-100 sent to ada@x.com
  -> OrderResult[success=true, orderId=ORD-100, message=shipped as SHP-1]
== Declined card: the facade undoes the reservation ==
  [inventory] reserved 1 x BOOK-42 (left 2)
  [payment]   charge bob@x.com 25000c -> DECLINED
  [inventory] released 1 x BOOK-42 (left 3)
  -> OrderResult[success=false, orderId=ORD-101, message=payment declined]
== Out of stock ==
  -> OrderResult[success=false, orderId=ORD-102, message=out of stock]
== The subsystem is still reachable for unusual needs ==
  [inventory] reserved 1 x BOOK-42 (left 2)
```

What each part does

- The four services are ordinary classes with their own responsibilities; none mention `OrderFacade`.
- `OrderFacade.placeOrder` is the whole pattern: the **sequence** plus the **compensating step** (`inventory.release`) on a declined payment — second output block: reserved, declined, released, with no caller code.
- `OrderResult` — callers need no subsystem types.
- The last line calls `inventory.reserve` directly: the facade is **a convenience, not a wall**.

Facade vs Adapter vs Mediator

	Facade	Adapter	Proxy	Mediator (Ch. 10–11)
Intent	Simplify a subsystem	Convert one interface to another	Control access to one object	Coordinate peers that would otherwise talk directly
Objects wrapped	**Several**	**One**	**One**	Several colleagues
Interface offered	New, smaller one	The client's (Target)	**Same** as the real object	Mediator's own
Who chose the interface	**You** — a simpler one over things you usually control	**Someone else** — you must fit it	Already fixed by the real object	You
Do the wrapped objects know about it?	**No** — one-way	No	No	**Yes** — colleagues talk *through* the mediator
Direction of communication	Client → facade → subsystem	Client → adapter → adaptee	Client → proxy → real subject	Colleague ↔ mediator ↔ colleague (both ways)
Example	`OrderFacade.placeOrder`	`StripeGateway.pay`	`LazyReport`	An air-traffic control tower; a chat room

Quickest tell is **"objects wrapped"**: Adapter, Decorator and Proxy front exactly one object; a Facade fronts **several**. Another test: if the subsystem changed its API to exactly what callers want, an **Adapter** becomes pointless but a **Facade** stays useful — its value was *simplification*, not translation.

Facade vs "service layer". (A service layer is the classes exposing the app's use cases — like placing an order — to its UI or API.) Usually the same thing under two names: a service that only *sequences* is a facade; once it holds business rules it is a domain/application service with logic of its own.

The failure mode to watch for

Facades **pull in features**: four calls on day one, then a discount rule ("orders are placed here, after all"), then a fraud check, and a year later a **god object** (one class that knows and does almost everything) everyone fears editing. One rule: **a facade orchestrates; it does not decide.** If it makes a business decision instead of calling the thing that does, that logic belongs elsewhere — and keeping it out now is far easier than moving it later.

Head First pairs Facade with the **Principle of Least Knowledge** (Law of Demeter): "talk only to your immediate friends" — clients talk to the facade, not a chain of subsystem objects, reducing coupling.

Trade-offs

Pros	Cons
One call replaces a multi-step sequence; the common path is no longer duplicated.	**One more class to maintain** as the subsystem evolves.
The order and the undo logic live in one place — exactly what callers got wrong.	**Feature magnet** — grows into a god object unless someone pushes back.
One seam for swapping the subsystem later — only the facade changes.	**Opinionated** — some caller will need an option it left out.
Smaller public surface for the module; lower coupling.	**Can mask problems** — a neat front over a still-failing subsystem; everything funnels through it, so it can become a single point of failure.

When not to use it

- **The subsystem is already simple** — wrapping two calls adds a file.
- **Callers need the full range** — if every caller bypasses it for some option, its shape is wrong: widen it on purpose or remove it.

- **You are covering a mess instead of fixing it** — same failures behind a neat front; sometimes repair the subsystem.
- **You want it to be the only entrance** — hiding the subsystem forces every unusual need into the facade, which is how god objects are born.

Common mistakes

- Business rules (pricing, fraud) in the facade.
- Making the subsystem inaccessible.
- Returning subsystem types, so callers stay coupled to them.
- Forgetting the failure path — a happy-path-only (nothing-goes-wrong) facade misses its biggest value.
- Confusing it with Mediator (see table).

Real-world examples

- **Application service layers** — almost every `OrderService` or `RegistrationService` is a facade over repositories, gateways and notifiers (as long as it only sequences).
- **`java.net.URL`** — `url.openStream()` hides protocol handlers, connection setup and content handlers. (Since Java 20 the `URL` constructors are deprecated — officially discouraged, due for removal; use `URI.create(s).toURL()`. The facade idea is unchanged.)
- **SLF4J** **`LoggerFactory.getLogger(...)`** — hides configuration, appenders (log destinations: files, console) and formatters. SLF4J literally calls itself a "Simple Logging Facade".
- **`javax.faces.context.FacesContext`**, **jQuery** (historically the clearest: one `$` in front of DOM, events, animation and XHR — the browser's API for HTTP requests from JavaScript — each still usable directly), and Python's **`requests`** covering common "just fetch this URL" cases over `urllib3`.
- **E-commerce checkout**: "Place order" on Amazon or Flipkart is one backend (server-side) call orchestrating inventory, payment, shipping and notifications — often called an orchestrator or backend-for-frontend (a server layer built for one particular UI).

In modern languages

- **Python / JavaScript**: a module exporting one function, the rest private — a facade with no class.
- **Go**: a package's exported functions are the facade; lowercase identifiers stay internal.
- **Java**: a package with one public class (package-private subsystem), or a JPMS (Java Platform Module System) module exporting only the facade's package.
- **Any language**: if `placeOrder` needs no state of its own, one well-named function is the whole pattern.

In the interview

> **30-second answer:** "A Facade gives one simple entry point to a complex subsystem. For order placement, `OrderFacade.placeOrder` reserves stock, charges, ships and emails in the right order, and if payment fails it releases the reservation — so no caller has to remember that cleanup. It wraps several objects, unlike Adapter, Decorator and Proxy which wrap one. I keep the subsystem public, because a facade is a convenience, not a wall, and I keep business rules out of it — the main risk is that it grows into a god object."

Follow-ups

- *Q: How is it different from Adapter?* — A: Adapter converts one interface someone else chose; Facade simplifies many objects behind a new interface you chose.
- *Q: And from Mediator?* — A: Subsystem classes don't know the facade exists; Mediator colleagues know it and communicate through it, both ways.
- *Q: Risk?* — A: God object. A facade should sequence, not decide.
- *Q: Should the subsystem be hidden?* — A: Usually no; leave it reachable for unusual needs.
- *Q: Can a system have several facades?* — A: Yes — e.g. one per use case, or layered facades over sub-subsystems.

Self-check

1. **Q:** What is the fastest structural way to tell a Facade from Adapter/Decorator/Proxy? **A:** It wraps several objects; the others wrap one.

2. **Q:** In our code, what does the facade do when payment is declined? **A:** Releases the stock reservation and returns a failed `OrderResult`.
3. **Q:** A teammate adds a 10% discount rule inside `placeOrder`. Good idea? **A:** No — business rules belong in a service the facade calls; the facade only sequences.
4. **Q:** Which OO principle is closely related to Facade? **A:** Principle of Least Knowledge (Law of Demeter).

TL;DR

> - **One door into a complex subsystem.** Names the common path and owns the cleanup.
> - Wraps **several** objects; subsystem stays **public**; facade **sequences, never decides**.
> - Different from Adapter (translation of one) and Mediator (two-way coordination of peers).

9.6 Flyweight Pattern

What it is

The **Flyweight** pattern **shares** the part of each object's state that is the same across many objects, so huge numbers of them fit in memory. GoF's intent: *use sharing to support large numbers of fine-grained objects efficiently.* Like printing: every "e" uses the same type design (shared) at a different position (unique).

Two terms carry the whole pattern:

Term	Meaning	In the map example
Intrinsic state	Part of the thing's own identity and **identical for every holder**. Lives inside the shared flyweight.	icon bitmap, colour, size
Extrinsic state	**Varies with the context** the object is used in; different for each holder. Stays outside and is **handed in** as method arguments.	latitude, longitude

Remember: "intrinsic = inside the shared object", "extrinsic = supplied from outside" — GoF's words, used by interviewers.

Role	In our code
Flyweight (shared, immutable, intrinsic only)	`MarkerStyle`
Flyweight factory (owns the pool, returns existing or creates once)	`MarkerStyleFactory`
Context (one per logical object: extrinsic state + reference to a flyweight)	`Marker`
Client (asks the factory, supplies extrinsic state)	`Main`

GoF's participants are *Flyweight* (interface accepting extrinsic state), *ConcreteFlyweight* (shared implementation — `MarkerStyle` plays both since we need no interface), *UnsharedConcreteFlyweight* (same interface but **not** shared, e.g. a row object holding its own children), *FlyweightFactory* and *Client*. "Context" is later sources' (e.g. refactoring.guru) name for the object storing extrinsic state.

The problem it solves

A map app shows **100,000 markers**, each with latitude and longitude — plus icon bitmap, colour and size. Positions differ, but thousands share an icon, by far the largest field. 100,000 copies of a 2 MB bitmap need ~195 GB: **the program simply runs out of memory.** Likewise a game's million trees sharing one mesh and texture (3D shape and the image painted on it), a text editor's per-character font data — anywhere many objects **differ slightly but share a large common part**.

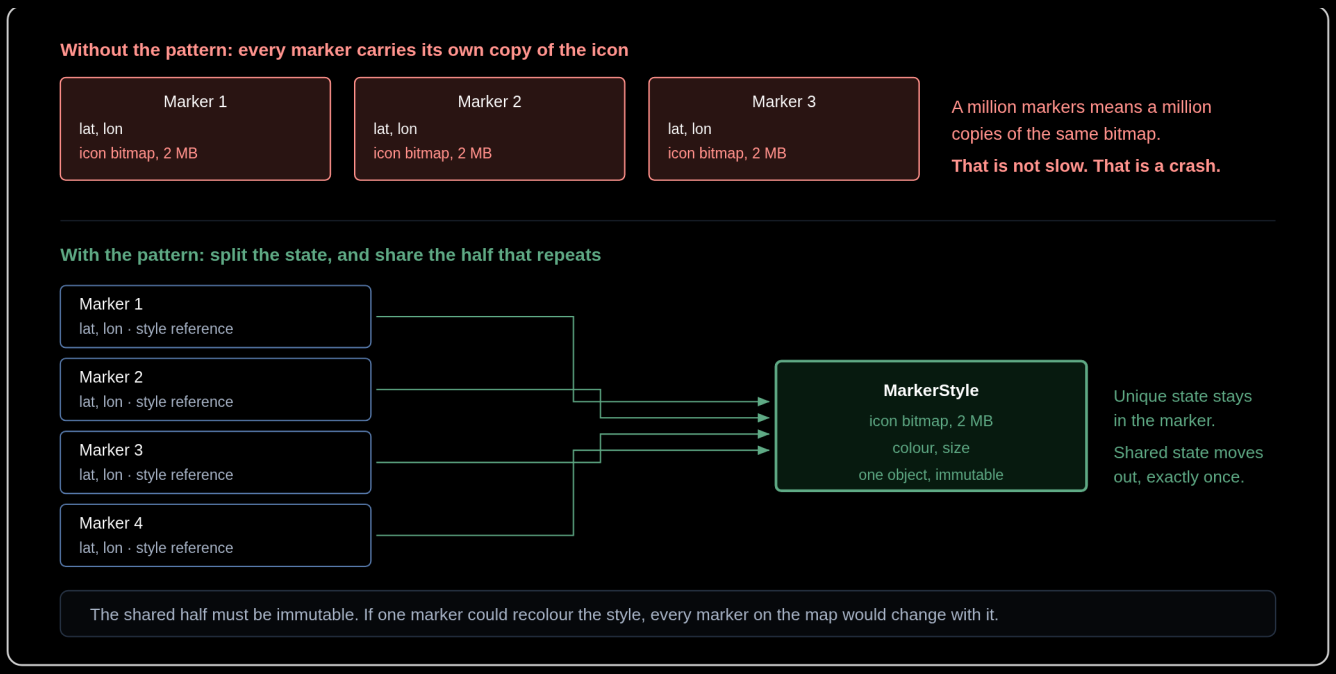


Figure — Top: every marker carries its own 2 MB icon. Bottom: each marker keeps only lat/lon and a style reference; one immutable `MarkerStyle` holds the icon, colour and size (course diagram)

How it works

1. **Split the state**: intrinsic (shared, invariant) vs extrinsic (per holder).
2. **Move intrinsic state into an immutable flyweight** — all fields `final`, no setters.
3. **Pass extrinsic state as method arguments** (`draw(lat, lon)`), never stored in the flyweight.
4. **A factory owns the pool**, keyed by intrinsic values; `get(...)` returns the existing object or creates it once.
5. **Clients must go through the factory**, never the constructor (package-private/private constructors).
6. **Context objects** keep extrinsic state plus a **reference** (not a copy) to the flyweight.

> **Two rules — break either and the pattern silently stops working.**
>
> 1. **Flyweights never change.** Many holders point at one object, so a change for one is seen by all.
> 2. **Only the factory creates them.** Direct construction creates identical copies and the saving is gone.

Structure

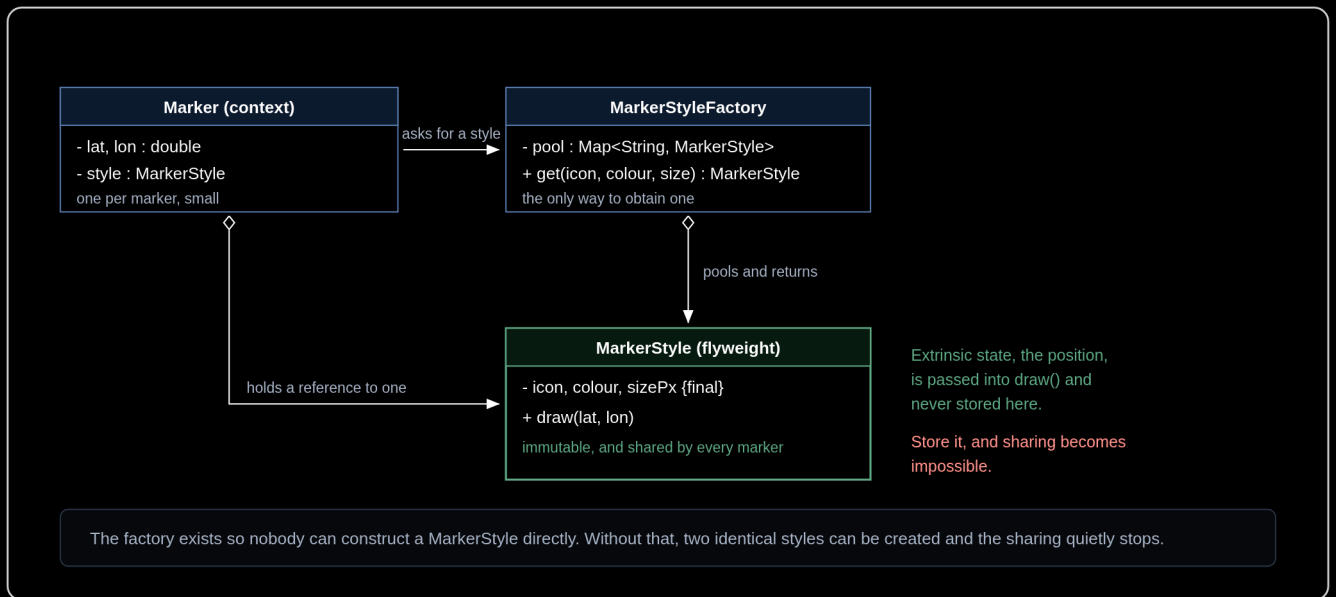
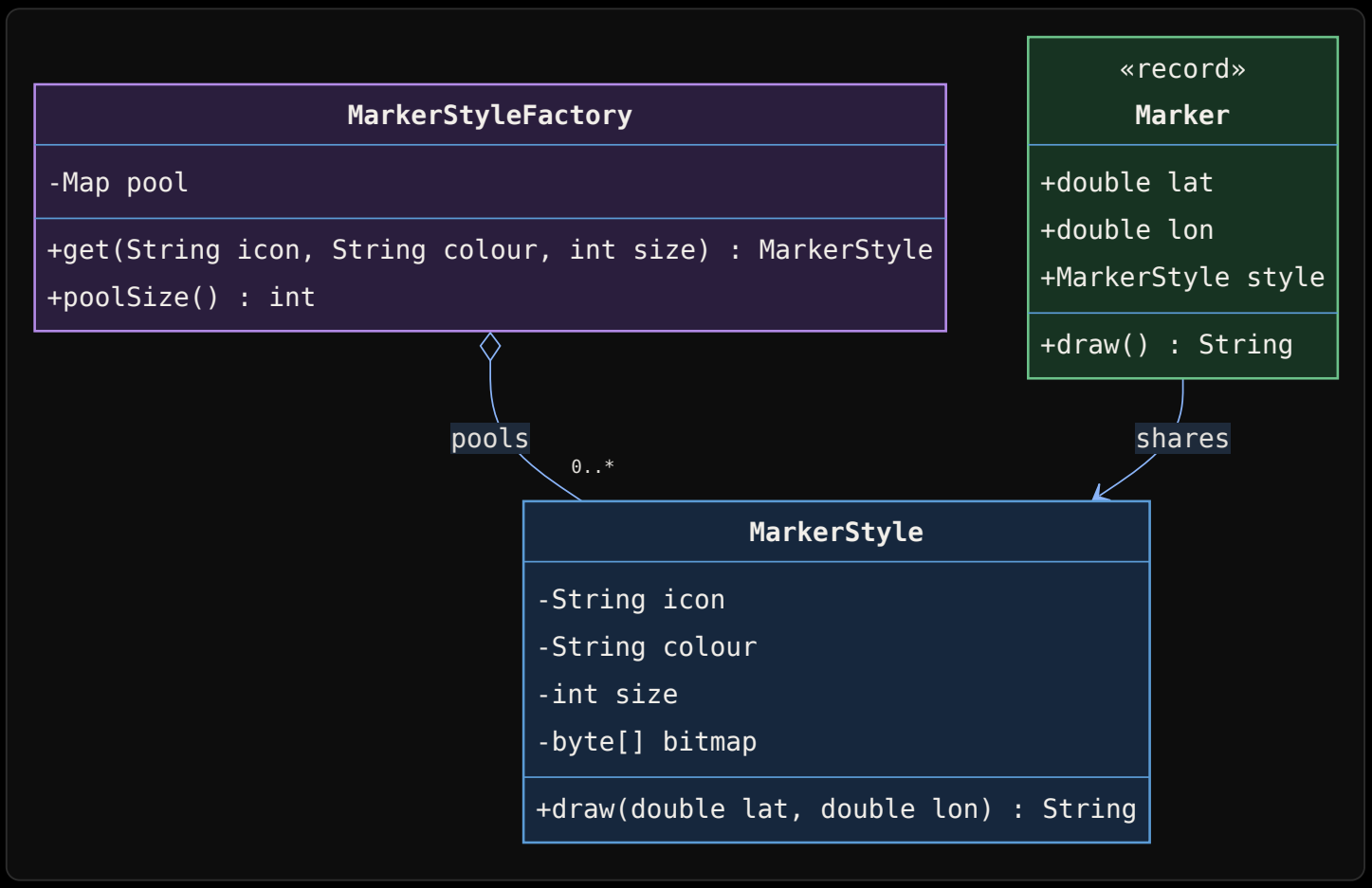


Figure — Flyweight class diagram: `MarkerStyle` (final fields, `draw(lat, lon)`), `MarkerStyleFactory` with a pool and `get()`, and `Marker` holding position plus a style reference (course diagram)

`draw` receives position as arguments; `Marker` holds a *reference* to a style; clients get styles only from the factory — enforced by a package-private (or private) constructor, or at least convention and review.



Java example

Full code: `code/lld/ch09/flyweight/` (`MarkerStyle`, `MarkerStyleFactory`, `Marker`, `Main`).

```java
// MarkerStyle.java — FLYWEIGHT: INTRINSIC state only. Immutable: final fields, no setters, bitmap never leaves.
// Package-private constructor: outside code cannot bypass the factory (inside the package it is a convention).
final class MarkerStyle {
    static final int ICON_BYTES = 2 * 1024 * 1024;   // pretend every icon bitmap is 2 MB
    private final String icon, colour;
    private final int size;
    private final byte[] bitmap;                     // the heavy part we want exactly one copy of

    MarkerStyle(String icon, String colour, int size) {
        System.out.println("  loading icon '" + icon + "' (" + colour + ", " + size + "px)");
        this.icon = icon; this.colour = colour; this.size = size;
        this.bitmap = new byte[ICON_BYTES];
    }
    /** EXTRINSIC state (the position) is passed in, never stored. */
    String draw(double lat, double lon) {
        return String.format("%s %s marker at (%.4f, %.4f)", colour, icon, lat, lon);
    }
}

// MarkerStyleFactory.java — owns the pool; thread-safe via ConcurrentHashMap.computeIfAbsent.
final class MarkerStyleFactory {
    private record Key(String icon, String colour, int size) { }   // the combination of intrinsic values
    private final Map<Key, MarkerStyle> pool = new ConcurrentHashMap<>();
    MarkerStyle get(String icon, String colour, int size) {
        return pool.computeIfAbsent(new Key(icon, colour, size), k -> new MarkerStyle(k.icon(), k.colour(), k.size()));
    }
    int poolSize() { return pool.size(); }
}

// Marker.java — CONTEXT: one per marker; extrinsic state + a REFERENCE to a shared style.
record Marker(double lat, double lon, MarkerStyle style) {
    String draw() { return style.draw(lat, lon); }
}
// Main: 5 markers over 3 styles, then 100,000 random markers; then JDK flyweight checks (Integer, Boolean, String).
```

Output:

```
== Five markers, three styles ==
  loading icon 'cafe' (brown, 24px)
  loading icon 'fuel' (green, 24px)
  loading icon 'hospital' (red, 32px)
  brown cafe marker at (51.5074, -0.1278)
  brown cafe marker at (48.8566, 2.3522)
  green fuel marker at (40.7128, -74.0060)
  red hospital marker at (35.6762, 139.6503)
  green fuel marker at (52.5200, 13.4050)
  same shared object? true
== Scale to 100,000 markers ==
  markers: 100000, styles in pool: 3
  icon memory without flyweight: ~200000 MB
  icon memory with flyweight:    ~6 MB
== Flyweights already in the JDK ==
  Integer 127 == 127 -> true  (cached -128..127)
  Integer 128 == 128 -> false (outside default cache)
  equals() is always right -> true
  Boolean.valueOf(true) == Boolean.TRUE -> true
  "hello" == "hel" + "lo" -> true (compile-time constant, pooled)
  "hello" == new String("hello") -> false
  "hello" == built.intern() -> true
```

What each part does

- `MarkerStyleFactory` — `ConcurrentHashMap.computeIfAbsent` makes "get or create once" **thread-safe and atomic** (correct when many threads — independent paths of execution — use it, done as one indivisible step), so two threads can't create duplicate styles.
- `Key` record — intrinsic values as a value object (defined only by its values); records give correct `equals` / `hashCode` (what a hash map uses to find a key) free.
- "loading icon" prints **3 times for 5 markers**, still 3 styles for 100,000 markers — ~6 MB of icons instead of ~200,000 MB (≈195 GB).

Flyweights already in Java

- `Integer.valueOf(int)` (used by autoboxing — automatic wrapping of an `int` into an `Integer`) returns cached objects for **`-128 to 127`**, a range the Java Language Specification requires. Hence `Integer a = 127, b = 127; a == b` is `true` but with 128 is `false` on a default JVM (the upper bound can be raised with `-XX:AutoBoxCacheMax`, so never rely on it). **Always compare boxed numbers with `equals`.** `Short`, `Byte`, `Long` (same range) and `Character` (0–127) have similar caches; `Boolean.valueOf` returns the shared `Boolean.TRUE` / `Boolean.FALSE`.
- **The String pool**: literals and compile-time constant expressions (`"hel" + "lo"`) are interned (stored once in a shared pool and reused). `new String("hello")` always creates a new object; `intern()` returns the pooled one.
- **Enums**: exactly one instance per constant, shared everywhere.

Flyweight is not a cache

Mixed up constantly — and the difference decides what you may do with the pool:

	Flyweight	Cache
Purpose	Make the data *fit in memory* at all	Make repeated work *quicker*
Contents	One copy of immutable state that many holders point to	Stored results of costly computations or lookups
May entries be removed?	Not while any holder still points to one	Yes, at any moment
Effect of removing one	Holders lose their shared object, or duplicates get created	None, apart from a slower next request

An LRU cache in front of a database is not a flyweight; a pool of shared marker styles is not a cache.

Two mistakes that undo the pattern

- **A flyweight that can change.** One caller updates a colour and every marker changes — the classic failure, hard to debug because the effect appears far from the cause. Fix: `final` fields, no setters, no leaked internal arrays.
- **An ever-growing pool keyed by open-ended input.** User-chosen keys mean the pool never stops growing — a memory leak (memory never released) disguised as optimisation. Flyweight is safe when intrinsic values are **small and bounded** (3 styles, a few hundred glyphs). An open key space needs weak references (which the garbage collector may clear) and eviction — at which point it is a cache, so call it one.

Measure first

Flyweight trades readability for memory, so using it without profiling (measuring where a program spends time or memory) is textbook premature optimisation. A thousand objects never need it; a million might. Answer **with numbers**: *how many of these objects exist at once*, and *how much of each is identical across all?* No numbers → too early; mostly-unique objects → nothing to share.

Trade-offs

Pros	Cons
Memory can fall by orders of magnitude — sometimes the difference between running and not.	**State is split across two places** — harder to follow and change than one class.
Costly construction once per distinct value, not per object.	**The pool keeps strong references** (ordinary references that stop the garbage collector freeing an object), so it can leak itself.
Friendlier to CPU caches — shared data stays hot (recently used, fast to reach).	**Thread safety is on you** — thread-safe pool, immutable flyweight.
"Load once" logic gets one obvious home.	**Tempting too early**, and callers pass extrinsic state on every call (some CPU cost).

When not to use it

- You **haven't measured**.
- The shared part **cannot be immutable**.

- The **counts are small**.
- The **unique state is most of the object**.
- The **key space is unbounded** (that's a cache).

Common mistakes

- Storing extrinsic state inside the flyweight (then it can't be shared).
- Public constructor so callers bypass the factory.
- Non-thread-safe pool (`HashMap` + check-then-put from many threads → duplicates or worse).
- Comparing boxed `Integer`s or `String`s with `==` because "it worked in the test" with small values or literals.

Other patterns it is confused with

Looks like	Actual difference
Cache	See table above.
Singleton	Singleton = exactly one instance of a class. Flyweight = one instance **per distinct intrinsic value** (3, or 300). A pool that happens to hold one entry only *looks* like a Singleton.
Object pool	An object pool **lends out mutable** objects and takes them back (e.g. DB connections). Flyweights are **immutable**, shared, never returned.
Prototype	Prototype makes many objects by copying; Flyweight avoids making them at all — opposite answers to "many similar objects".

Real-world examples

- **`Integer.valueOf`**, **`Boolean.valueOf`**, **`String.intern()`**, **enums** — above. (Python also interns small integers and many short strings automatically.)
- **Text rendering / word processors** — one glyph object per font character, shared by every occurrence (the classic GoF example).
- **Game engines and map renderers** — meshes, textures and sprite atlases loaded once and referenced by every entity (a forest of identical trees, rendered with GPU *instancing* — the graphics card drawing one model many times in one pass).
- **Map apps** — thousands of pins share a handful of icon styles (this section's example).
- **`java.time.ZoneOffset`** caches offsets that are whole multiples of 15 minutes, so `ZoneOffset.ofHours(5)` returns a shared instance.

In modern languages

- **Java**: `Integer.valueOf`, `String.intern` and enums are built-in flyweights; a `record` is an immutable flyweight in one line, and `Map` + `computeIfAbsent` is the whole factory.
- **Python**: small ints and short strings interned automatically; `functools.lru_cache` on a factory function gives a size-bounded pool.
- **JavaScript**: `Object.freeze` on the shared object plus a `Map` pool — freezing enforces immutability.
- **C++**: `std::shared_ptr<const Style>` expresses sharing and immutability in the type.
- **Any language with value types**: an immutable value object plus a memoising (remembers results already produced) factory is this pattern, usually unnamed.

In the interview

> **30-second answer:** "Flyweight saves memory when there are huge numbers of similar objects. I split state into intrinsic — shared and invariant, like an icon — and extrinsic — per object, like a position — which is passed in to methods. The intrinsic part lives in an immutable flyweight handed out by a factory that pools one instance per distinct value. `Integer.valueOf` and the String pool are JDK examples. I'd only do it after measuring."

Follow-ups

- *Q: Why must the flyweight be immutable?* — A: It is shared; mutating it changes every holder.
- *Q: How do you make the factory thread-safe?* — A: `ConcurrentHashMap.computeIfAbsent`, which runs the creation function at most once per key.

- *Q: Flyweight vs cache?* — A: A cache may evict anything and exists for speed; a flyweight exists because otherwise the program doesn't fit, and entries can't be dropped while referenced.
- *Q: Why is `Integer 128 == 128` false?* — A: Autoboxing uses `Integer.valueOf`, which only guarantees caching for −128..127; outside it you get distinct objects on a default JVM. Use `equals`.
- *Q: Flyweight vs Singleton?* — A: One instance per distinct value vs one instance total.

Self-check

1. **Q:** In a chess game with a million saved positions, what is intrinsic and extrinsic for a piece? **A:** Intrinsic: type and colour (and image). Extrinsic: the square it is on.
2. **Q:** What goes wrong if `MarkerStyle` had `setColour`? **A:** Changing one marker's colour would change every marker sharing that style.
3. **Q:** Is `"a" + "b" == "ab"` true in Java? **A:** Yes — it is a compile-time constant expression, so it is interned. With a non-final variable in the expression, it would be a new object.
4. **Q:** When does a flyweight pool become a leak? **A:** When its key space is unbounded (e.g. keyed by user input) and it never evicts.

TL;DR

- **Share what repeats, pass in what varies.** Intrinsic inside, extrinsic outside.
- **Immutable** flyweight + **factory-owned** pool (thread-safe).
- JDK: `Integer.valueOf` (−128..127), String pool, enums. **Measure first**; not a cache.

9.7 Proxy Pattern

What it is

The **Proxy** pattern places a **stand-in with the identical interface** in front of a real object to control **who** may use it, **when** it is created, or **how much** each use costs. GoF's intent: *provide a surrogate or placeholder for another object to control access to it* (also called **Surrogate**). Like a credit card standing in for your bank account: accepted wherever cash is (same "interface"), but adding limits and fraud checks, moving money only when needed.

Term	Meaning	In our code
Subject	The interface shared by the proxy and the real object	`Report`, `RateService`, `Device`
Real Subject	The object that does the real work	`RealReport`, `LiveRateService`, `RealDevice`
Proxy	Holds (or creates) the real subject, same interface, decides what happens before/instead of/around the call	`LazyReport`, `AuditedReport`, `CachingRateProxy`, `RemoteRateProxy`, the `SharedDevice` handle

The problem it solves

Sometimes callers should **not touch the real object directly — or not yet**:

- **Building it is costly and usually unnecessary** — loading a 100 MB report at start-up wastes work on runs that only show its title.
- **Only some callers are permitted**, and permission logic doesn't belong inside the object.
- **It runs on another machine**, and callers shouldn't write networking code.
- **Each call must be logged or counted**, but the class belongs to someone else.

With **the very same interface**, a proxy can be inserted **without editing any caller**. That is also its danger: a slow or remote call **looks just like a cheap local one**.

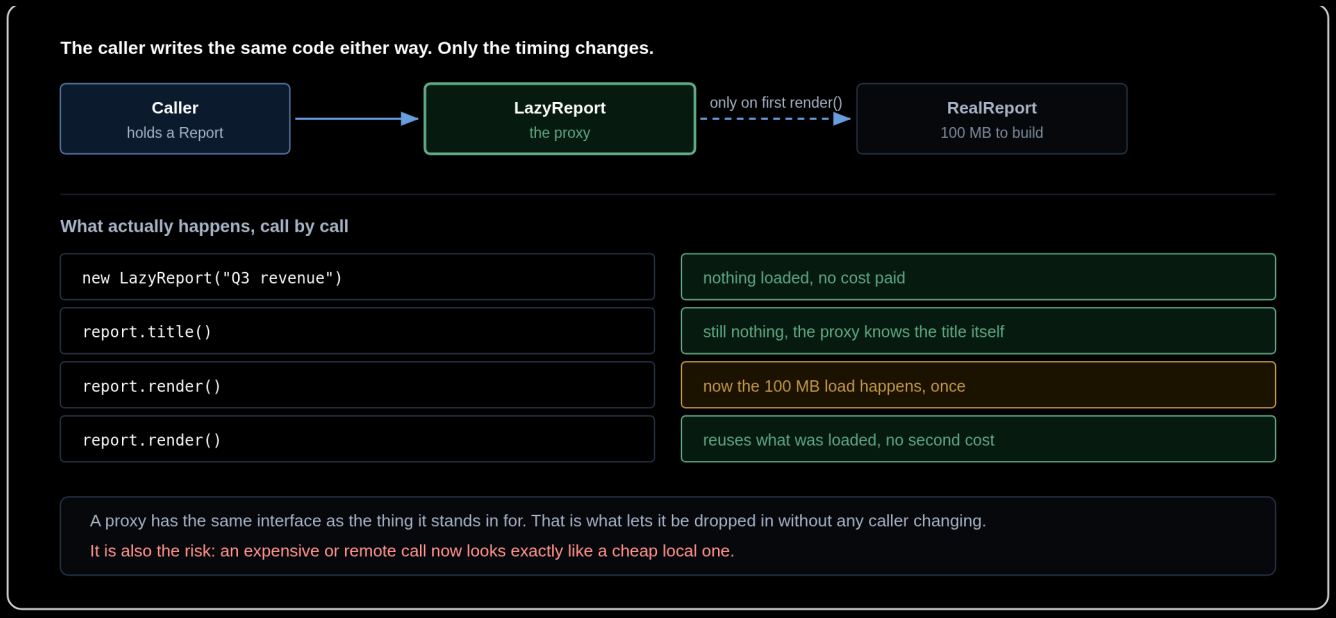


Figure — The caller holds a `Report`. Creating the `LazyReport` and calling `title()` cost nothing; the 100 MB load happens once on the first `render()`, and later renders reuse it (course diagram)

How it works

1. **Define (or reuse) the Subject interface** callers already use.
2. **Write a proxy implementing it**, holding the real subject — or the information to create it later.
3. In each method, **do the proxy's job first** — create lazily, check permission, look in a cache, marshal (convert into a sendable message) over a network, count references — then **delegate** (or refuse).
4. **Give callers the proxy** in place of the real object (often via a factory or DI container, so they never see the difference).

Kinds of proxy

Kind	What it controls	Our code / real example
Virtual proxy	*When* the expensive object is created (lazy initialisation)	`LazyReport`; Hibernate (a Java ORM) lazy-loaded entities
Protection proxy	*Who* may call (access control)	`AuditedReport`; `Collections.unmodifiableList`
Remote proxy	*Where* the object lives — hides the network	`RemoteRateProxy` (stub) + `RateServer` (skeleton); RMI/gRPC (remote-call frameworks) stubs
Caching proxy	*Whether* the real call is needed — reuses results	`CachingRateProxy` (bounded LRU + TTL — time to live, how long an entry stays valid)
Logging / monitoring proxy	Records each call before/after forwarding	`LoggingHandler` (dynamic proxy)
Smart reference	Bookkeeping on access — reference counting, locking, loading on first use	`SharedDevice` handles close the device when the last one closes

GoF names four kinds — remote, virtual, protection, smart reference — differing only in what the proxy does before delegating. Caching and logging proxies are widely used variants from later sources such as refactoring.guru (GoF would file logging/counting under smart reference).

Structure

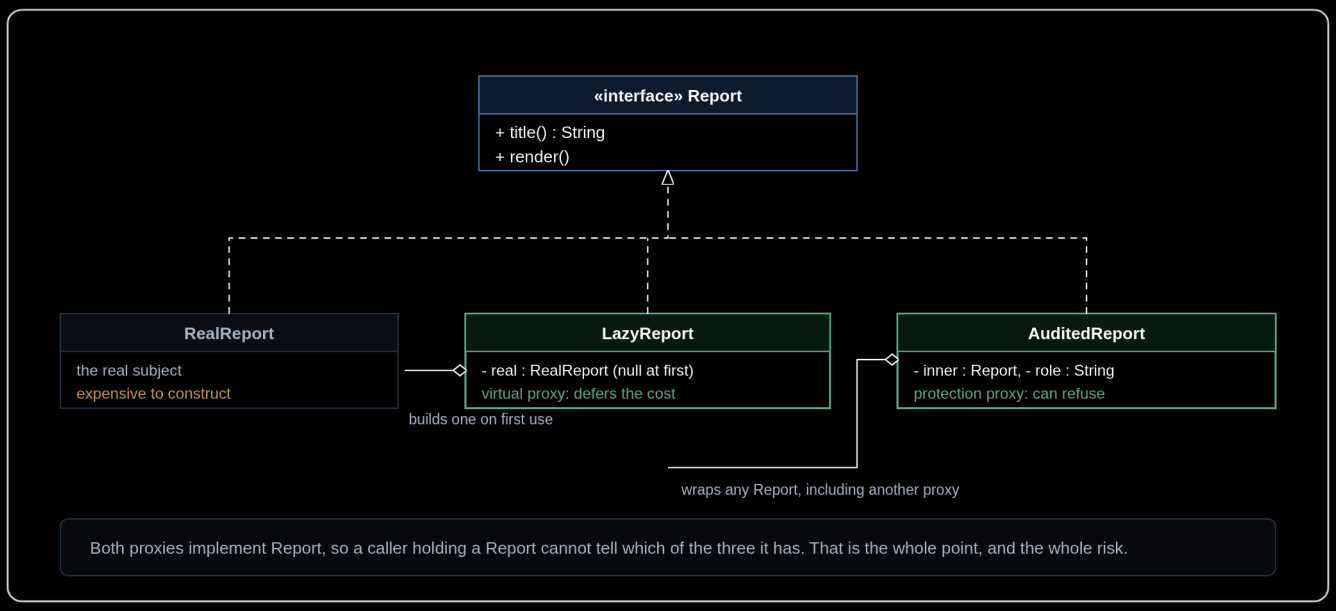
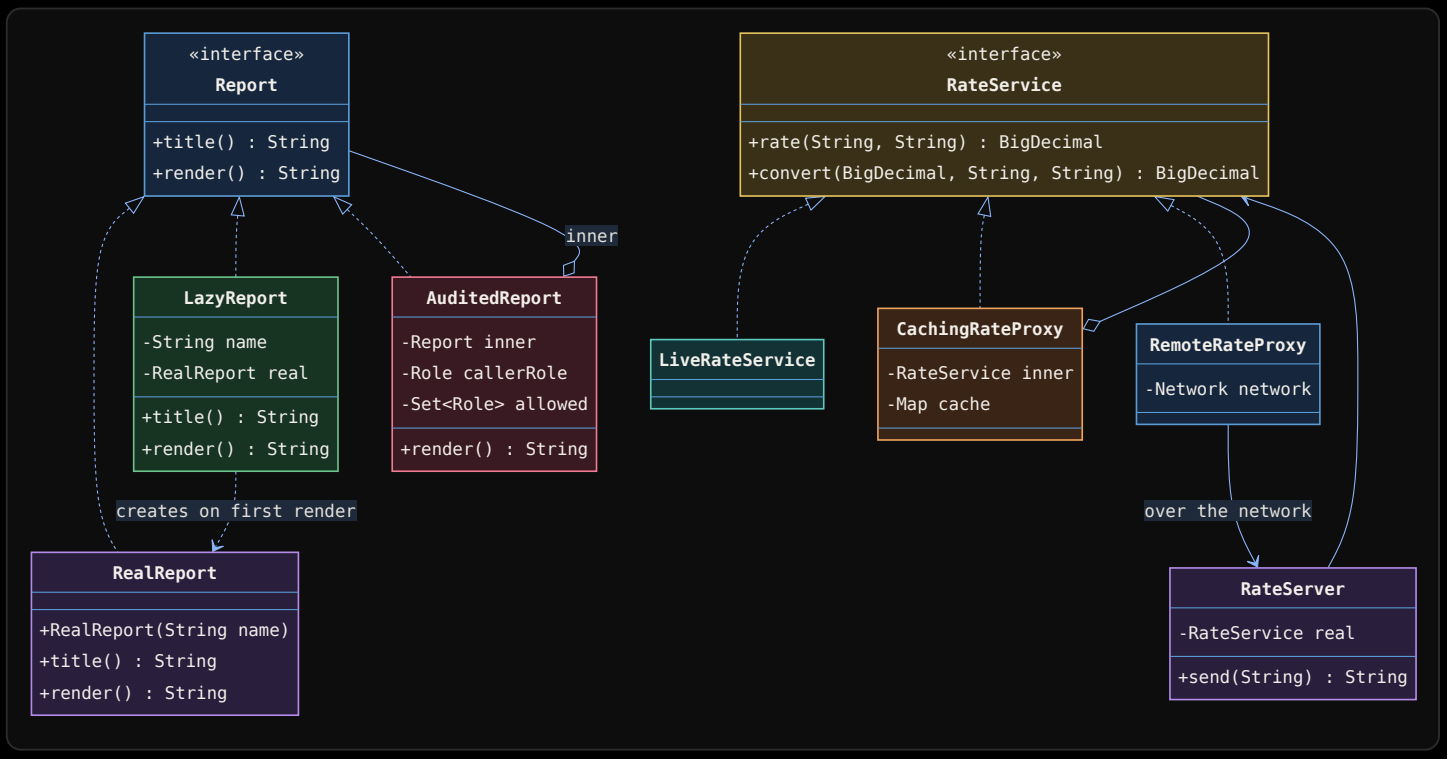


Figure — Proxy class diagram: `RealReport`, `LazyReport` (virtual proxy) and `AuditedReport` (protection proxy) all implement `Report`; `LazyReport` creates the `RealReport` itself, `AuditedReport` wraps any `Report` (course diagram)

`RealReport` does the expensive work in its constructor; `LazyReport` keeps only the name, answers `title()` itself and builds the `RealReport` on the first `render()`; `AuditedReport` checks a role first. Holding the **interface**, `AuditedReport` can guard the real report, the lazy proxy or another protection proxy (our demo wraps the lazy one). Stripped down, only one line of `LazyReport` is special — the null check that triggers construction; the rest forwards.



Java example

Full code: `code/lld/ch09/proxy/` — `Report` / `RealReport` / `LazyReport` / `AuditedReport` / `Role`; `RateService` / `LiveRateService` / `CachingRateProxy`; `Network` / `RateServer` / `RemoteRateProxy`; `Device` / `RealDevice` / `SharedDevice`; `LoggingHandler`; `Main`.

```java
// LazyReport.java — VIRTUAL PROXY: answers title() itself, builds the RealReport on first render().
// Thread-safe lazy init with double-checked locking (field must be volatile).
final class LazyReport implements Report {
    private final String name;
    private volatile RealReport real;      // created by the proxy itself, not handed in
    LazyReport(String name) { this.name = name; }

    @Override public String title()  { return name; }   // no load needed
    @Override public String render() { return realReport().render(); }

    private RealReport realReport() {
        RealReport r = real;
        if (r == null) {
            synchronized (this) {
                r = real;
                if (r == null) real = r = new RealReport(name);   // prints "[real] loading 100 MB..."
            }
        }
        return r;
    }
}

// AuditedReport.java — PROTECTION PROXY: checks permission, may REFUSE the call.
@Override public String title() { return inner.title(); }  // titles are public
@Override public String render() {
    if (!allowed.contains(callerRole)) throw new SecurityException(callerRole + " may not render " + inner.title());
    return inner.render();
}
```

```java
// RateService.java — money and rates are BigDecimal, never double; amounts rounded to 2 dp (HALF_EVEN).
interface RateService {
    BigDecimal rate(String from, String to);
    BigDecimal convert(BigDecimal amount, String from, String to);
    static BigDecimal toMoney(BigDecimal value) { return value.setScale(2, RoundingMode.HALF_EVEN); }
}
// LiveRateService (real subject, counts lookups):
public BigDecimal convert(BigDecimal amount, String from, String to) {
    return RateService.toMoney(amount.multiply(rate(from, to)));   // self-call: goes to THIS object, not through any proxy
}

// CachingRateProxy.java — bounded (LRU) and time-limited, because an unbounded cache is a leak.
this.cache = new LinkedHashMap<>(16, 0.75f, true) {   // access order = LRU
    @Override protected boolean removeEldestEntry(Map.Entry<String, CachedRate> eldest) { return size() > maxEntries; }
};
@Override public synchronized BigDecimal rate(String from, String to) {
    String key = from + "->" + to;
    CachedRate e = cache.get(key);
    long now = System.currentTimeMillis();
    if (e != null && e.expiresAt() > now) return e.value();
    BigDecimal v = inner.rate(from, to);
    cache.put(key, new CachedRate(v, now + ttlMillis));
    return v;
}

// REMOTE PROXY over a simulated in-process Network { String send(String request); }
// RateServer (server "skeleton"): unmarshals "rate|FROM|TO", returns real.rate(...).toPlainString()
// RemoteRateProxy (client "stub"):
@Override public BigDecimal rate(String from, String to) {
    String request = "rate|" + from + "|" + to;
    System.out.println("    [stub] sending '" + request + "' over the wire");
    return new BigDecimal(network.send(request));
}
```

```java
// SharedDevice.java — SMART REFERENCE: each client gets its own handle (a proxy); the real device
// opens on the first handle and closes when the LAST closes. All of refs, real and each handle's
// 'closed' flag are read/written under the SharedDevice lock.
synchronized Device acquire() {
    if (refs++ == 0) real = new RealDevice();
    System.out.println("    [ref] handle acquired, count=" + refs);
    return new Handle();
}
// Handle.close():
synchronized (SharedDevice.this) {
    if (closed) return;          // closing a handle twice must not double-decrement
    closed = true;
    refs--;
    System.out.println("    [ref] handle released, count=" + refs);
    if (refs == 0) { real.close(); real = null; }
}

// LoggingHandler.java — DYNAMIC PROXY: one InvocationHandler that logs calls on ANY interface.
static <T> T wrap(T target, Class<T> iface) {
    return (T) Proxy.newProxyInstance(iface.getClassLoader(), new Class<?>[] {iface}, new LoggingHandler(target));
}
@Override public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
    String call = method.getName() + (args == null ? "()" : Arrays.toString(args));
    System.out.println("    [proxy] -> " + call);
    try {
        Object result = method.invoke(target, args);   // forward to the real object
        System.out.println("    [proxy] <- " + call + " returned " + result);
        return result;
    } catch (InvocationTargetException e) {
        throw e.getCause();          // rethrow the real exception, not the reflection wrapper
    }
}
// Main: CachingRateProxy(live, 2, 60_000) gets USD>EUR, USD>EUR, USD>INR, EUR>INR (evicts USD>EUR), USD>EUR.
```

Output:

```
== Virtual proxy: pay only when needed ==
  created proxy; title = Q3 revenue
    [real] loading 100 MB of data for 'Q3 revenue'...
  render #1 -> Q3 revenue (1000000 rows)
  render #2 -> Q3 revenue (1000000 rows)
== Protection proxy wrapping the virtual proxy ==
    [real] loading 100 MB of data for 'Payroll'...
  finance -> Payroll (1000000 rows)
  viewer  -> refused: VIEWER may not render Payroll (nothing was loaded)
== Caching proxy (LRU, max 2 entries, 60 s TTL) ==
  5 calls, real lookups = 4
== Remote proxy (stub) ==
    [stub] sending 'rate|USD|INR' over the wire
  100 USD = 8300.00 INR
== Smart reference (reference counting) ==
    [device] opened (expensive)
    [ref] handle acquired, count=1
    [ref] handle acquired, count=2
  a -> ok:status, b -> ok:print
    [ref] handle released, count=1
    [ref] handle released, count=0
    [device] closed
== Dynamic proxy: one handler, any interface ==
    [proxy] -> title()
    [proxy] <- title() returned Q4 plan
    [proxy] -> render()
    [real] loading 100 MB of data for 'Q4 plan'...
    [proxy] <- render() returned Q4 plan (1000000 rows)
    [proxy] -> convert[100, USD, EUR]
    [proxy] <- convert[100, USD, EUR] returned 92.00
  ^ convert() called rate() on 'this', so the inner rate() call was NOT logged
  generated at run time: isProxyClass = true
```

What each part does

- `LazyReport` — **virtual proxy**: `title()` answered locally; "loading" appears **once**, at the first render.
- **Thread-safe lazy init.** Otherwise two threads calling `render()` together both build the object (harmless for a report, a real bug if construction opens a connection or charges a card). **Double-checked locking** (check

without a lock, then lock and check again — Chapter 8) with a **`volatile`** field (makes a write immediately visible to all threads) so no thread sees a half-built object.

- `AuditedReport` — **protection proxy** wrapping a virtual proxy; the viewer is refused **before** anything loads (no second "loading 'Payroll'" line).
- `RateService` uses `BigDecimal` (Java's exact decimal type) — never `double` for money — rounding to 2 places with `HALF_EVEN` ("banker's rounding": an exact half goes to the even digit).
- `CachingRateProxy` — **bounded** (LRU via access-ordered `LinkedHashMap` + `removeEldestEntry`) and **time-limited** (TTL). 5 calls → 4 lookups: the second USD→EUR hits, the last misses because it was evicted.
- `SharedDevice` — **smart reference**; all bookkeeping under one lock and `close()` is idempotent (twice = once), so concurrent or double closes can't corrupt the count.

Static vs dynamic proxies

	Static proxy	Dynamic proxy
How it's made	You hand-write a class per interface (`LazyReport`, `CachingRateProxy`)	Generated at **run time** from an interface list + a handler
Code per interface	One proxy class each; every method written out	One `InvocationHandler` for **any** interface
Type safety	Full; compiler checks each method	Calls arrive as `Method` + `Object[]` — reflective, less checked
Good for	Logic specific to one type (lazy load, access rules for a domain object)	**Cross-cutting concerns** (needs shared by many classes) on many types: logging, timing, transactions, security
JDK API	—	`java.lang.reflect.Proxy.newProxyInstance(loader, interfaces, handler)`
Limitation	Boilerplate	JDK proxies work **only for interfaces**; class-based proxies need bytecode (compiled JVM instructions) libraries (CGLIB, ByteBuddy)

How the JDK dynamic proxy works:

1. `Proxy.newProxyInstance(classLoader, new Class<?>[]{Report.class}, handler)` generates a class (named like `$Proxy0`) implementing `Report`.
2. **Every** call on it is routed to `handler.invoke(proxy, method, args)`.
3. The handler does its extra work and forwards with `method.invoke(target, args)`.
4. If the target throws, reflection wraps it in `InvocationTargetException` — **unwrap it** (`e.getCause()`) so callers see the real exception.

The self-invocation trap (and Spring AOP)

In the output, `convert` went through the logging proxy, but internally called `rate(...)` on **`this`** — the real object — so that inner call **bypassed the proxy** and was not logged. This is exactly **Spring AOP**:

- Spring implements annotations (`@` tags it reads) such as `@Transactional` (run inside a database transaction — all changes succeed or fail together), `@Cacheable`, `@Async`, `@PreAuthorize` and custom `@Aspect` advice (the code AOP runs around a method) by wrapping your bean (an object the Spring container manages) in a **proxy**; callers get the proxy injected.
- Spring uses a **JDK dynamic proxy** when the target implements at least one interface, a **CGLIB** subclass proxy otherwise (or when `proxyTargetClass = true`; Spring Boot 2.0+ defaults to CGLIB class-based proxies).
- **Self-invocation is not intercepted**: if `placeOrder()` calls `this.saveAudit()` annotated `@Transactional(propagation = REQUIRES_NEW)`, the annotation is silently ignored — the call never leaves the object.
- With CGLIB, **`final` classes cannot be proxied and `final`/`private` methods cannot be advised**.
- Fixes, in order of preference: move the method into another bean; inject the bean's own proxy (e.g. a lazily injected self-reference); or (discouraged) `AopContext.currentProxy()`, which also needs `exposeProxy = true`. AspectJ weaving (inserting the code into compiled classes at compile or load time) avoids the problem — no proxies.

Proxy vs Decorator

The hardest pair to tell apart: both implement an interface, hold another object of it and forward. GoF says implementations can be similar and the difference is **purpose**: a decorator *adds responsibilities*, a proxy

controls access.

	Proxy	Decorator
Purpose	**Controls access** to the object	**Adds behaviour** around the object
Does the call get through?	Not necessarily — may defer, cache or refuse	Normally yes — the added work runs around a forwarded call
Who creates / finds the real object?	**Often the proxy itself** (`LazyReport` builds `RealReport`; a remote proxy only knows an address)	**The caller**, who hands it in
Does the client know it has a wrapper?	Usually not — a factory/DI container hands it the proxy as if it were the real object	Usually yes — the client (or its configuration) builds the stack
How many layers?	Usually one	Often several, stacked recursively on purpose

The **creation test** is most practical — check the constructor: `LazyReport(String name)` builds its own subject; `LoggingRepository(UserRepository inner)` is given one. A protection proxy like `AuditedReport` *is* handed its subject and is structurally identical to a decorator — only intent separates them. A caching wrapper genuinely sits on the line; the more useful review question is whether its cache is bounded and invalidated.

Where proxies go wrong

- **Unsynchronised lazy creation** — two threads both construct the expensive object: sometimes harmless, sometimes a duplicate connection or double charge.
- **A caching proxy with no size limit or expiry** — it *is* a cache and needs a bound and invalidation story, or it leaks memory and serves stale data. (Ours has both.)
- **Concealing a real cost** — a local-looking remote proxy invites use inside loops. Network calls disguised as field access cause some of the nastiest distributed-system (many machines cooperating over a network) performance bugs; the cure is making the cost visible (batch APIs fetching many items per call, explicit `fetch...` names), not tuning the proxy.
- **ORM N+1 queries** (an ORM maps database rows to objects; N+1 = one query for the list plus one per item) — iterating 100 orders and touching `order.getCustomer().getName()` fires one query per lazy proxy.

Trade-offs

Pros	Cons
Pay later — costly construction only if something needs it.	**Extra indirection**; stack traces show the proxy, not the real class.
No caller changes — identical interface (Open/Closed).	**Latency (how long a call takes) and failures disguised** as an ordinary method call.
A home for cross-cutting rules on a class you cannot edit.	**Lazy creation raises a thread-safety question** easy to get wrong.
Permission checks stay outside the object, which keeps to its job.	**A cache with no eviction** leaks memory and returns stale data.

When not to use it

- **The real object is cheap** — laziness only adds a null check per call.
- **Callers need to see the cost** — sometimes the honest interface is a method named `fetchOverNetwork()`.
- **The framework already generates one** — ORMs, DI containers and service meshes (proxies placed next to every service to manage traffic between them) create proxies; stacking a hand-written one doubles the hidden behaviour.
- **You mean to add behaviour, not guard access** — a Decorator.

Common mistakes

- Non-`volatile` field in double-checked locking.
- Forgetting to unwrap `InvocationTargetException` in a dynamic proxy.
- Expecting Spring annotations to work on self-calls, `private` methods or `final` methods.
- Unbounded caching proxies.

- Using a remote proxy in a tight loop.

Real-world examples

- **ORM lazy loading** (Hibernate/JPA — Java's standard ORM API — also Entity Framework and Django) — `entityManager.getReference(...)` and lazy associations return proxies; the SQL runs the first time you touch a non-id field. The most used virtual proxy — and the source of N+1 and, in Hibernate, `LazyInitializationException` when the session (its open unit of work with the database) is already closed.
- **Spring AOP** — `@Transactional`, `@Cacheable`, `@Async`, method security: all proxies. `@Configuration` classes are CGLIB-enhanced subclasses so calling one `@Bean` method from another returns the container's singleton (unless `proxyBeanMethods = false`).
- **Remote stubs** — Java RMI stubs, gRPC and Thrift generated clients, SOAP clients: a network call behind an ordinary method.
- `java.lang.reflect.Proxy` — used by many frameworks; e.g. Spring Data repositories and MyBatis mappers are interfaces whose implementations are generated proxies.
- `Collections.unmodifiableList` — a protection proxy refusing mutation (also seen as a decorator).
- **Service meshes / sidecars** (Envoy in Istio) — intercept traffic for retries, mTLS (mutual TLS: both sides prove identity, traffic encrypted) and telemetry (metrics and traces sent for monitoring): the proxy idea at infrastructure level. **Reverse proxies / CDNs** (Nginx, Cloudflare) cache and protect origin servers (your servers holding the original content); a reverse proxy receives requests on your servers' behalf, a CDN (content delivery network) caches content close to users — see System Design Fundamentals.
- **C++** `shared_ptr` — a smart-reference proxy doing reference counting.

In modern languages

- **Java**: `java.lang.reflect.Proxy`, plus bytecode generation (CGLIB, ByteBuddy) for classes.
- **JavaScript/TypeScript**: built-in `Proxy` intercepts property access and calls.
- **Python**: `__getattr__` forwards anything not defined locally, so a proxy writes only what it intercepts; `functools.cached_property` is a virtual proxy for one field.
- **C#**: `DispatchProxy`, and `Lazy<T>` — a virtual proxy for one value.
- **C++**: `std::shared_ptr` and other smart pointers count references behind `operator->`.

Other patterns it is confused with

Looks like	Actual difference
Decorator	Same structure — see Proxy vs Decorator.
Adapter	An Adapter has a **different** interface on purpose — it exists to translate. A Proxy has the **same** interface — it exists to substitute.
Facade	A Facade offers a new, simpler interface over **several** objects. A Proxy offers the same interface over **one**.
A cache	A caching proxy is a cache with a proxy's shape. Calling it a proxy does not remove the need to design its size bound and eviction.

In the interview

30-second answer: "A Proxy is a stand-in with the same interface as the real object that controls access to it. I'd name the kind: a virtual proxy defers expensive creation, a protection proxy checks permissions, a remote proxy hides the network, a caching proxy reuses results, and a smart reference does bookkeeping like reference counting. Because the interface is identical, callers don't change — which is also the danger, since an expensive call looks cheap. Lazy creation needs thread safety, and ORM lazy-loading proxies are the classic source of N+1 queries. In Java I'd use `java.lang.reflect.Proxy` for cross-cutting concerns; that's how Spring AOP implements `@Transactional`."

Follow-ups

- *Q: Proxy vs Decorator?* — A: Proxy controls access and often creates the real object; Decorator adds behaviour to an object handed to it and normally forwards every call. Check the constructor.
- *Q: Make the virtual proxy thread-safe.* — A: Double-checked locking with a `volatile` field (as in `LazyReport`), or a `synchronized` getter if contention (threads waiting on each other for the lock) is low. (The static holder idiom only fits a *static* lazy field, not a per-instance proxy.)

- *Q: Why doesn't `@Transactional` work when I call the method from the same class?* — A: Spring's proxy only intercepts calls coming through the proxy; `this.method()` goes straight to the target.
- *Q: JDK proxy vs CGLIB?* — A: JDK proxies implement interfaces only; CGLIB subclasses, so it works without interfaces but can't handle `final` classes/methods.
- *Q: How does lazy loading cause N+1?* — A: Loading N parents then touching each one's lazy child proxy triggers N extra queries; fix with fetch joins / entity graphs / batch fetching (loading children together with parents).

Self-check

1. **Q:** Which proxy kind is `LazyReport`? `AuditedReport`? **A:** Virtual; protection.
2. **Q:** Why is the field in `LazyReport` `volatile`? **A:** So double-checked locking is safe — other threads see a fully constructed object.
3. **Q:** In the dynamic proxy demo, why wasn't the inner `rate()` call logged? **A:** `convert()` called `rate()` on `this` (the real object), bypassing the proxy — the self-invocation trap.
4. **Q:** Can `java.lang.reflect.Proxy` proxy a class with no interface? **A:** No; it needs interfaces. Use CGLIB/ByteBuddy for classes.
5. **Q:** What two properties must a caching proxy have? **A:** A size bound (eviction) and an invalidation/expiry policy.

TL;DR

- **Same interface, controlled access.** Virtual, protection, remote, caching, logging, smart reference.
- **Static** = hand-written per interface; **dynamic** = `java.lang.reflect.Proxy` + `InvocationHandler` for any interface.
- Spring AOP = proxies → beware self-invocation, `final`, `private`. Watch thread safety and hidden costs.

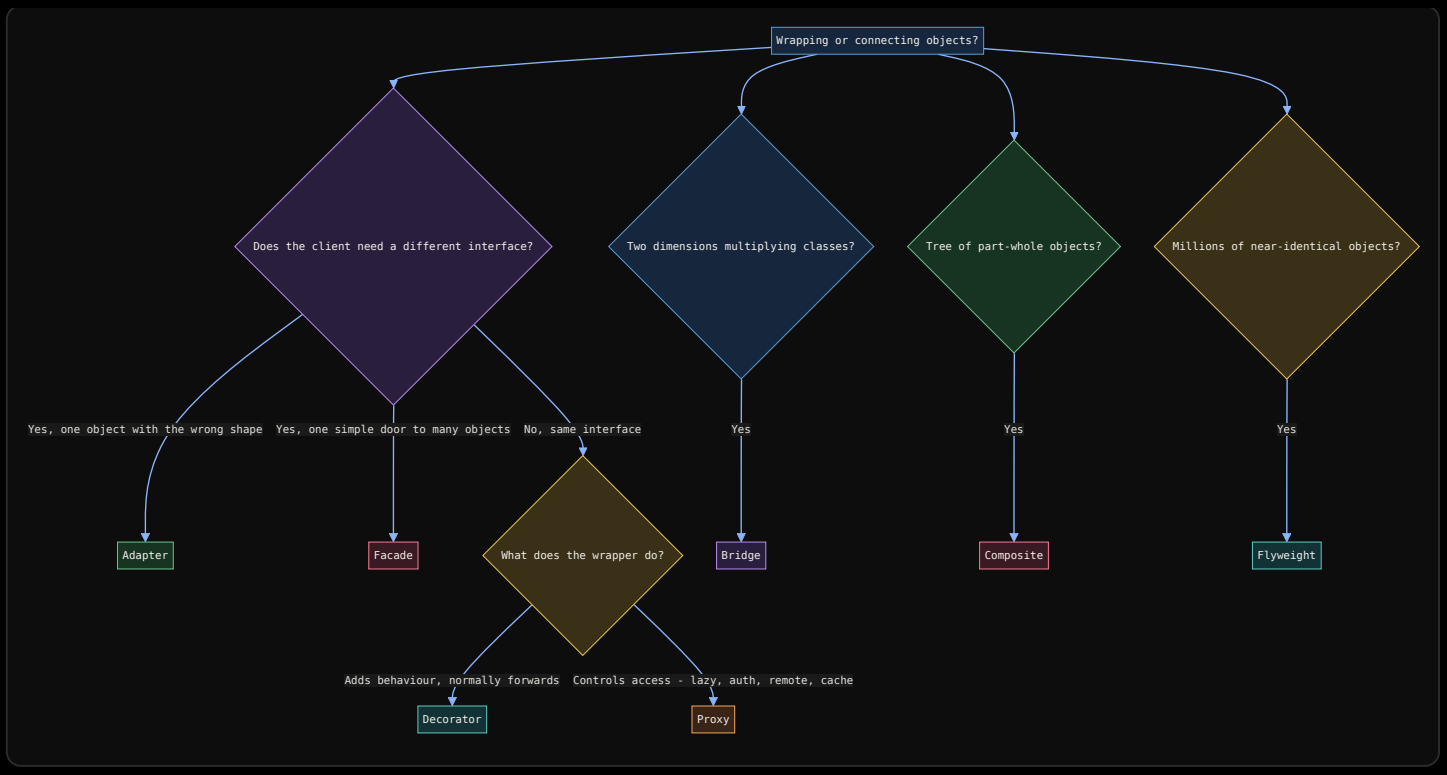
9.8 Putting it all together — comparison

Summary of all seven structural patterns

Pattern	Distinctive feature	Use it when	Example	Pros	Cons
Adapter	Bridges incompatible interfaces	You must use an existing class whose interface doesn't fit your code	Travel plug adapter; `StripeGateway`	Interoperability; reuse of existing code	Extra class and complexity
Bridge	Separates abstraction from implementation	Two dimensions vary independently; avoid permanent binding	UI shapes × platform renderers; notifications × channels	Both sides vary independently; scales by addition	More concepts up front; indirection
Composite	Part-whole tree with one interface	Treat single objects and groups uniformly	Drawing editor with shapes and groups; file system	Simpler client code; easy to add node types	Interface can become over-general
Decorator	Adds responsibilities dynamically	Extend an object's behaviour at run time, in combinations	Adding scrolling/border to a text view; Java I/O	More flexible than subclassing; no feature-bloated classes	Many small objects; hard to debug; order matters
Facade	Simple interface to a complex subsystem	Most callers need one common path through many classes	Home-theatre "watch movie" button; `OrderFacade`	Simplifies use; fewer dependencies	Can become a god object or single point of failure
Flyweight	Shares common state among many objects	Huge numbers of similar objects with little variation	Characters in a word processor; map markers	Big memory savings	Complexity; managing shared immutable state
Proxy	Placeholder that controls access	Lazy loading, access control, remote, caching, logging, smart reference	Internet proxy server; Hibernate lazy entity	Controls access; defers costly work	Indirection; hidden latency

Adapter vs Decorator vs Proxy vs Facade vs Bridge

	Adapter	Decorator	Proxy	Facade	Bridge
Intent	Convert an interface to the one the client expects	Add behaviour without changing the interface	Control access to an object	Simplify a complex subsystem	Let abstraction and implementation vary independently
Interface seen by client	**Different** from the wrapped object's (it's yours)	**Same** as wrapped	**Same** as real subject	**New, simpler** interface	Abstraction's own interface
Behaviour	Unchanged (translated)	**Extended**	Unchanged but **controlled** (may defer/cache/refuse)	Unchanged (fewer calls)	Delegated to implementor
Objects wrapped	One	One (stackable)	One	**Several**	One implementor (by reference)
Who creates the wrapped object	Usually passed in	Passed in by client	**Often the proxy itself**	Passed in / owned	Passed in at construction
Structure	Implements Target, holds Adaptee	IS-A and HAS-A Component	IS-A Subject, HAS-A Real Subject	HAS-A many subsystem objects	Abstraction HAS-A Implementor; two hierarchies
When introduced	**After the fact**, to make existing things fit	Any time a concern is added	Any time access needs controlling	When callers repeat a multi-step sequence	**Up front**, before the class explosion
Typical trigger	"The vendor API doesn't match ours"	"Sometimes we need logging, sometimes caching, sometimes both"	"It's expensive / remote / restricted"	"Every caller copies these 5 steps"	"Class names with two nouns"
JDK / framework example	`Arrays.asList`, `InputStreamReader`	`BufferedInputStream`, `Collections.synchronizedList`	`java.lang.reflect.Proxy`, Spring `@Transactional`, Hibernate lazy load	SLF4J `LoggerFactory`, service layers	JDBC `Connection` + drivers, AWT peers



Memory hooks

- **Adapter** — *"make it fit"* (changes the interface).
- **Decorator** — *"add to it"* (same interface, more behaviour, stackable).
- **Proxy** — *"guard it"* (same interface, controlled access).
- **Facade** — *"simplify it"* (one door, many rooms).
- **Bridge** — *"split it"* (two hierarchies, add instead of multiply).
- **Composite** — *"nest it"* (one interface for leaf and tree).
- **Flyweight** — *"share it"* (intrinsic shared, extrinsic passed in).

Chapter self-check

1. **Q:** Which of these patterns wrap exactly one object? **A:** Adapter, Decorator, Proxy (Bridge also holds one implementor, but by design rather than as a wrapper). Facade wraps several; Composite holds many children.
2. **Q:** Which two have identical code shape but differ in intent and timing? **A:** Adapter and Bridge (after-the-fact vs up-front); also Bridge and Strategy; also Decorator and Proxy.
3. **Q:** A wrapper returns cached results with the same interface. Decorator or Proxy? **A:** Either is defensible; the important question is whether the cache is bounded and invalidated.
4. **Q:** You have `WindowsButton`, `MacButton`, `WindowsCheckbox`, `MacCheckbox` … and a Linux port is coming. Which pattern? **A:** Bridge (widget abstraction × platform implementor) — often combined with Abstract Factory to pick the implementor.
5. **Q:** Which pattern explains why `Integer.valueOf(100) == Integer.valueOf(100)` is true? **A:** Flyweight (Integer cache).

Chapter TL;DR

- Structural patterns **compose objects into bigger structures** while keeping them flexible.
- **Adapter / Decorator / Proxy / Facade** share a shape; tell them apart by **what changes as the call passes through** and **how many objects are wrapped**.
- **Bridge** prevents class explosion across two dimensions; **Composite** makes trees uniform; **Flyweight** shares immutable state to save memory.
- In interviews: name the pattern, state the **constraint** that justifies it, name its **trap** (vendor leaks, wrong split, cycles, decorator order, god facade, mutable flyweight, self-invocation), and give a **real JDK/Spring example**.

Chapter 10 — Behavioral Design Patterns (Part 1)

> **What you will learn.** What behavioral patterns are, and six in depth: **Chain of Responsibility, Command, Iterator, Mediator, Memento** and **Interpreter** — each with a plain definition, the problem it fixes, the class diagram, runnable Java 21 code, real products and JDK (Java Development Kit — compiler, tools and standard library) classes that use it, and interview Q&A. Chapter 11 covers the other five GoF behavioral patterns (Observer, State, Strategy, Template Method, Visitor) plus important non-GoF patterns.

How to run the code in this chapter (listings below show the essential lines; full files are in `code/`)

```
cd chapters/10-patterns-behavioral-1/code
javac --release 21 -d /tmp/lld_10 $(find . -name '*.java')
java -cp /tmp/lld_10 lld.ch10.chain.Main        # also: command, iterator, mediator, memento, interpreter
```

10.0 Behavioral patterns: the big picture

What it is

Creational patterns (Chapter 8) are about **how objects are made**; structural (Chapter 9) about **how they are put together**. **Behavioral patterns** are about **how objects share work and talk**: who is responsible for what, and who calls whom — in a restaurant kitchen, not "which stations exist?" but "when an order arrives, who hears first, who passes it on, who orders the dishes, who can cancel?"

They help you:

- let objects work together flexibly and efficiently;
- cut tight coupling (classes depending heavily on each other's internals), so each is easier to change and test;
- turn tangled many-object communication into something easy to follow;
- change behaviour **while the program runs** (at run time), not only at compile time.

The eleven GoF behavioral patterns

The "Gang of Four" (GoF) book, *Design Patterns* (Gamma, Helm, Johnson, Vlissides, 1994), lists eleven; this chapter covers six, Chapter 11 the rest.

Pattern	One-line intent	Chapter
Chain of Responsibility	Hand a request down a row of handler objects; each may deal with it, stop it, or forward it.	10
Command	Wrap a request in an object, which can then be stored, queued, logged or reversed.	10
Interpreter	Represent the grammar of a small language as classes, and evaluate sentences in it.	10
Iterator	Walk through a collection one element at a time without seeing how it is stored.	10
Mediator	Route communication between many objects through one coordinator.	10
Memento	Let an object save a snapshot of its own state and later return to it, while keeping that state private.	10
Observer	When one object changes, notify all the objects that depend on it.	11
State	Let an object change its behaviour when its internal state changes.	11
Strategy	Choose one of several interchangeable algorithms at run time.	11
Template Method	Fix the skeleton of an algorithm in a base class; let subclasses fill in some steps.	11
Visitor	Add a new operation to a set of classes without changing those classes.	11

Class scope vs object scope. *Class* behavioral patterns share behaviour by **inheritance** — only **Template Method** and **Interpreter**. All others are *object* behavioral patterns using **composition** (objects holding references to others), so they can be rewired at run time.

One idea runs through all of them: take behaviour hidden inside a big method (a chain of `if`s, a hard-coded call, a loop) and turn it into an **object** you can name, pass around, swap, store and test on its own.

10.1 Chain of Responsibility

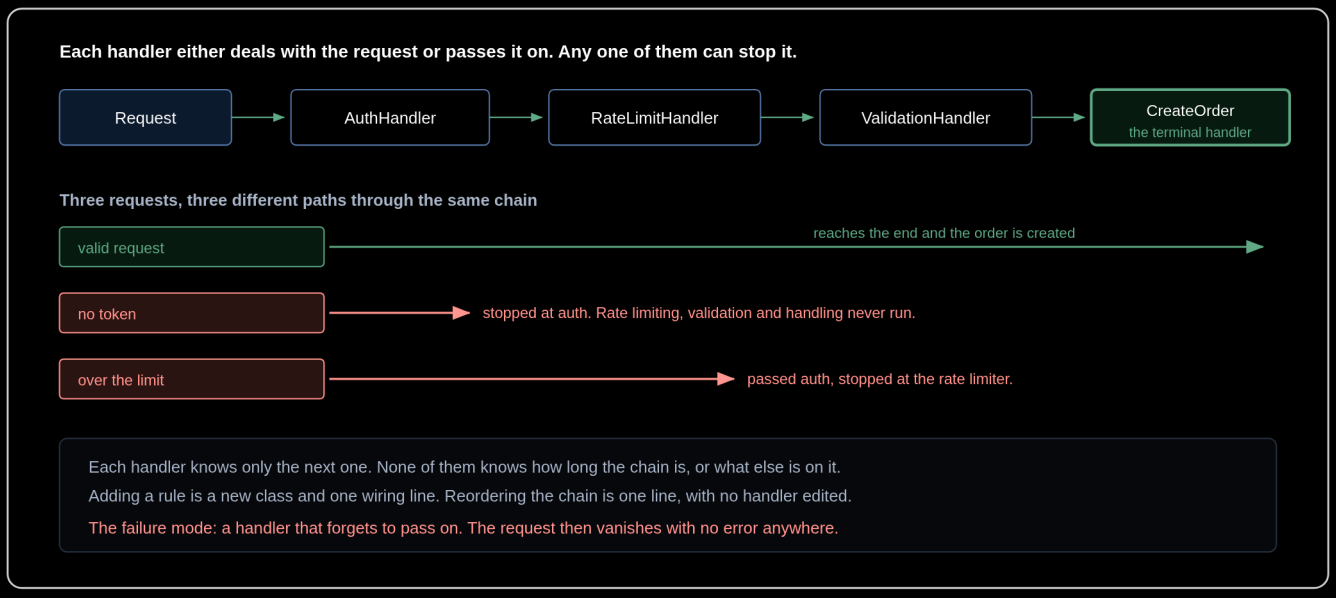


Figure — Three requests go into the same chain of handlers and each stops at a different place (course diagram)

What it is

Chain of Responsibility (CoR) sends a request through a row of **handler** objects. Each handler deals with it, rejects it (nobody later sees it), or forwards it to its successor. The sender talks only to the first handler and never learns which one finished the job — like a support line: level 1, then level 2, then an engineer, though you dialled one number.

GoF intent: *avoid tying the sender of a request to one particular receiver by giving several objects a chance to handle it; link those objects and forward the request until one handles it.* Participants: **Handler** (declares the handling method, optionally holds the successor link), **ConcreteHandler** (handles what it is responsible for, otherwise forwards), **Client** (starts the request at a handler).

In the figure, one chain (auth → rate limit → validation → create order) gets three requests: a valid one reaches the end, one with no token stops at auth, one over the limit stops at the rate limiter. Each handler knows only the next.

The problem it solves

Before an order is created a request must pass authentication (who is calling), rate limiting (too often?) and validation (is the body — the data sent — well-formed?). In one method:

- The method grows with every rule.
- The **order of checks is hidden in the body**, yet it matters: auth-then-rate-limit is a different system from rate-limit-then-auth.
- Some routes (URL paths such as `/orders`) need only two checks — soon boolean flags, then a forgotten second copy.

CoR makes **each check a small object** linked into a line. No handler knows the line's length or contents, so **the line is configuration, not logic**: a new rule is one class plus one wiring line; reordering touches only the wiring; each route gets its own line, assembled at start-up.

How it works

1. A **Handler** type with `handle(request)` and a reference to the **next** handler.
2. One **concrete handler** per check: check, then **stop** (return a result) or **pass on** (`next.handle(request)`).
3. A **terminal handler** at the end does the real work (or reports "nobody handled this").
4. **Wire** once, at start-up: `auth.setNext(rateLimit).setNext(validate).setNext(createOrder)`.
5. The client calls only the **first** handler.

Structure (UML)

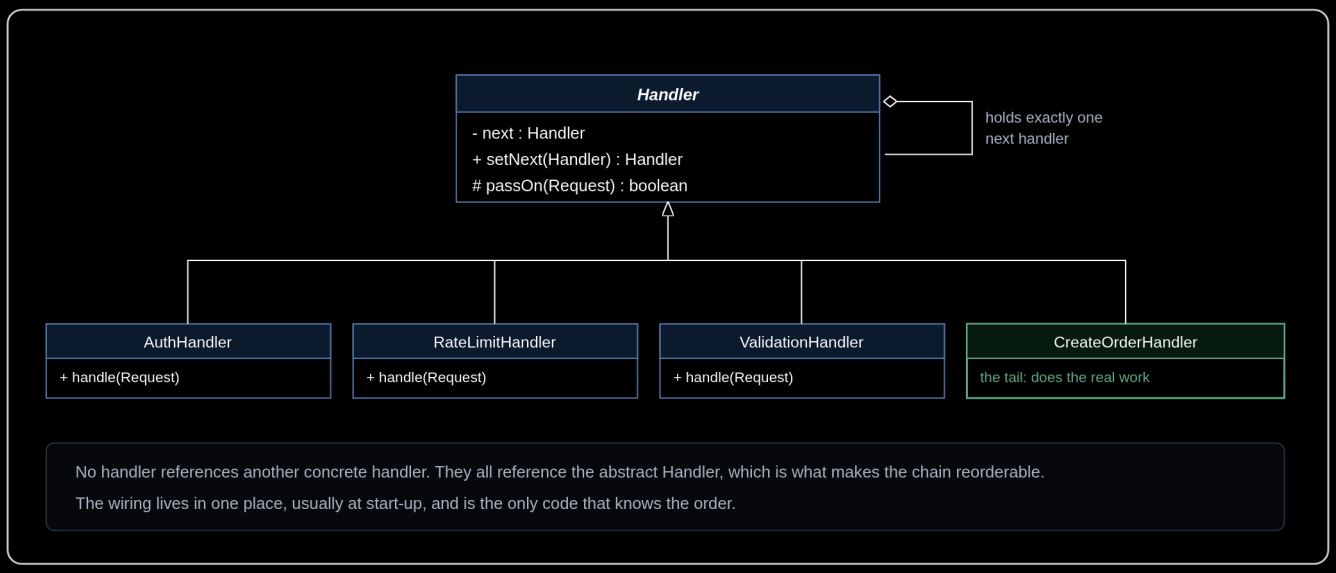
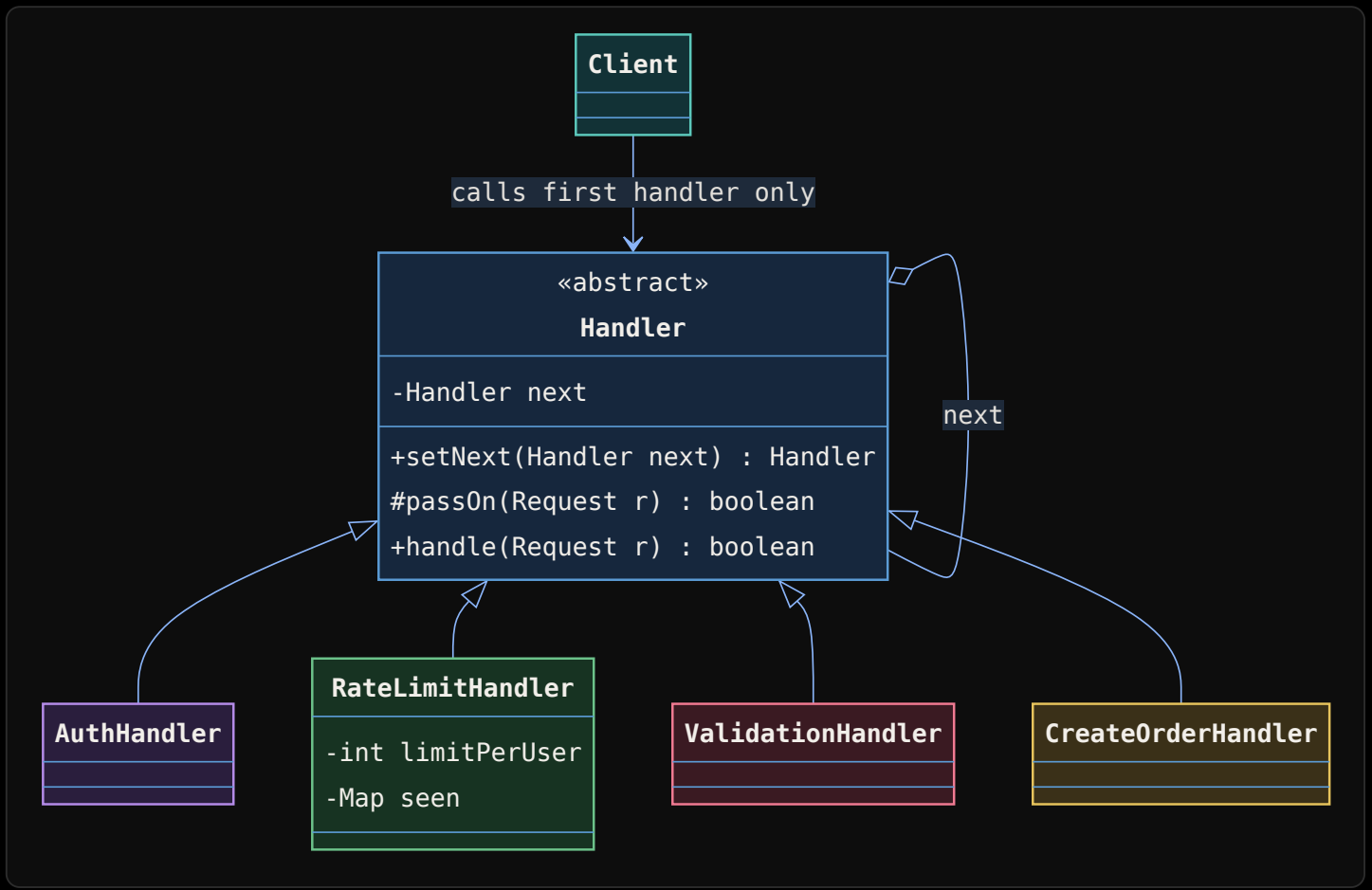


Figure — Class diagram: an abstract Handler holds `next`; concrete handlers extend it; CreateOrderHandler is the tail (course diagram)

In the class diagram (UML — Unified Modeling Language, the standard notation for classes and links), note what is **missing**: no handler refers to a *concrete* handler. Every link is typed as abstract `Handler`, so stages shuffle freely. `setNext` returns its argument, so wiring is one fluent expression (calls chained with dots, each returning the object for the next).

> **Our code vs the course pseudocode.** The course's `passOn` returns `true` ("success") when there is no next handler. Ours **throws**, because silently reporting success when the chain ran out is exactly this pattern's typical invisible failure. Either is legal; make it deliberate.



Java example

`code/lld/ch10/chain/` shows **four flavours**:

1. **Classic linked handlers** (`Handler`, `AuthHandler`, `RateLimitHandler`, `ValidationHandler`, `CreateOrderHandler`). Each returns an *explicit* outcome: `false` = rejected, `passOn(...)` = continue. `passOn` **throws** with no next handler, so a chain without a terminal fails loudly.
2. **Approval chain** (`Approver`, `Expense`). Pure CoR: the *first* approver with a high-enough limit approves; nobody after sees it. Only **one** class — the chain is data (role + limit). Amounts are `BigDecimal` (Java's exact decimal type) compared with `compareTo` — never `double` for money.
3. **Servlet-style filters** (`Filter`, `FilterChain`, `Endpoint`, `HttpRequest`) — how `jakarta.servlet.Filter` (Java's standard web-request API) works: "next" is passed **in as an argument** (`chain.doFilter(...)`). A filter can act before *and after* the rest (the timing filter), or stop the chain by not calling it (the auth filter).
4. **Logging levels** (`LogHandler`, `ConsoleLog`, `FileLog`, `PagerLog`, `Level`). A common *variant*: each handler acts if severe enough and **always** passes on, so several may act. Strictly, when nothing can stop the flow it is closer to a *pipeline* (a fixed series of steps every item passes through) — textbooks still show logging this way.

```
// Handler.java — classic base: each handler knows only "the next one".
public abstract class Handler {
    private Handler next;
    /** Links the next handler and returns it, so wiring reads as one chained expression. */
    public Handler setNext(Handler next) { this.next = next; return next; }

    /** Forwards the request. Falling off the end means the chain has no terminal handler: a wiring bug. */
    protected boolean passOn(Request request) {
        if (next == null) throw new IllegalStateException(
                getClass().getSimpleName() + " tried to pass on, but the chain has no terminal handler");
        return next.handle(request);
    }
    /** @return true if the request was fully handled, false if some handler rejected it. */
    public abstract boolean handle(Request request);
}

// AuthHandler.java — stage 1 (RateLimitHandler keys a per-user count; ValidationHandler rejects empty payloads;
// CreateOrderHandler is terminal and never passes on).
@Override public boolean handle(Request r) {
    if (r.token() == null || r.token().isBlank()) {
        System.out.println("  [auth] rejected: missing token");
        return false;                       // explicit outcome: the chain stops here
    }
    System.out.println("  [auth] ok for " + r.user());
    return passOn(r);                       // explicit outcome: continue
}

// Approver.java — first capable handler wins; the chain is data.
String approve(Expense e) {
    if (e.amountUsd().compareTo(limitUsd) <= 0) {   // compareTo, not equals: 300 and 300.00 are the same amount
        return role + " approved " + e.id() + " ($" + e.amountUsd() + ")";
    }
    if (next == null) {   // explicit terminal outcome instead of a silent "nothing happened"
        return "REJECTED " + e.id() + " ($" + e.amountUsd() + "): above every approver's limit";
    }
    System.out.println("    " + role + " (limit $" + limitUsd + ") escalates " + e.id());
    return next.approve(e);
}
```

```java
// Filter.java — same shape as jakarta.servlet.Filter: "next" is an explicit argument, so forgetting it is visible.
@FunctionalInterface
interface Filter { void doFilter(HttpRequest request, StringBuilder response, FilterChain chain); }

// FilterChain.java — one per request, because it holds a position.
void doFilter(HttpRequest request, StringBuilder response) {
    if (position < filters.size()) filters.get(position++).doFilter(request, response, this);
    else endpoint.service(request, response);   // the terminal handler
}

// LogHandler.java — VARIANT: act if severe enough, then ALWAYS pass on.
final void log(Level level, String message) {
    if (level.compareTo(threshold) >= 0) write(level, message);
    if (next != null) next.log(level, message);
}
// ConsoleLog threshold DEBUG, FileLog INFO, PagerLog ERROR.

// Main.java (wiring)
// Order is a design decision: auth BEFORE rate limiting so the limiter can key on the user,
// and so anonymous traffic cannot burn a real user's quota.
Handler pipeline = new AuthHandler();
pipeline.setNext(new RateLimitHandler(2)).setNext(new ValidationHandler()).setNext(new CreateOrderHandler());
Approver chain = new Approver("TeamLead", new BigDecimal("1000"));
chain.then(new Approver("Manager", new BigDecimal("10000"))).then(new Approver("Director", new BigDecimal("50000")));
Filter auth = (req, res, next) -> {
    if (!req.headers().containsKey("Authorization")) { res.append("401 Unauthorized"); return; }  // chain stops
    next.doFilter(req, res);
};
```

Output:

```
=== 1. API request pipeline (classic linked handlers) ===
request from alice:
  [auth] ok for alice
  [ratelimit] ok (1/2)
  [validate] ok
  [order] created order for alice: 2 x widget
  => handled = true
request from bob:
  [auth] rejected: missing token
  => handled = false
request from alice:
  [auth] ok for alice
  [ratelimit] ok (2/2)
  [validate] rejected: empty payload
  => handled = false
request from alice:
  [auth] ok for alice
  [ratelimit] rejected: alice made 3 requests, limit 2
  => handled = false

=== 2. Expense approval chain (first capable handler wins) ===
  TeamLead approved EXP-1 ($300)
    TeamLead (limit $1000) escalates EXP-2
  Manager approved EXP-2 ($7500)
    TeamLead (limit $1000) escalates EXP-3
    Manager (limit $10000) escalates EXP-3
  REJECTED EXP-3 ($120000): above every approver's limit

=== 3. Servlet-style filters (next is an explicit argument) ===
  [timing] start /orders
  [timing] end   /orders -> 200 OK orders list
  [timing] start /orders
  [timing] end   /orders -> 401 Unauthorized

=== 4. Logging levels (variant: every handler may act) ===
  log DEBUG:
    console <- DEBUG cache miss for key=42
  log ERROR:
    console <- ERROR payment service down
    app.log <- ERROR payment service down
    PAGER   <- ERROR payment service down
```

Walk through the output:

- Request 1 travels the full chain; request 2 stops at auth (limiter, validator and order code never run); request 3 passes auth and the limiter (2 of 2) and stops at validation; request 4 is alice's third, stopped by the limiter.
- `EXP-2` is escalated once, approved by the Manager. `EXP-3` exceeds every limit, so the chain returns an explicit "REJECTED" rather than vanishing.
- The timing filter prints before and after the rest. For the unauthenticated request the auth filter wrote `401` (HTTP's "not authenticated" status) and did not call `next`, so the endpoint never ran.
- `DEBUG` reaches only the console; `ERROR` reaches all three.

Trade-offs

Pros	Cons
A new stage costs one class and one wiring line; no working class is edited (Open/Closed).	To see a request's route you read the wiring plus several files.
Every stage can be unit-tested (in isolation) against a stub (minimal stand-in) successor.	When a request is refused you may step through several handlers to learn which one.
Each route or environment can get its own chain, assembled at start-up.	No compiler or framework verifies the chain is complete or correctly ordered.
The sender does not know which object will handle its request.	A handler that neither handles nor forwards makes the request vanish without error.
	Each stage adds a call; rarely matters, but can on very hot paths (code run extremely often) with long chains.

Stage order is a design decision no tool verifies. Auth before rate limiting lets the limiter count per *user*, and rejects bad credentials before they use up a real user's quota (allowed number of requests). Rate limiting first can only key on something like the IP address (the calling machine's network address) — but shields a possibly expensive auth step from floods (huge request bursts). Both are reasonable; they behave differently. **Comment the reason beside the wiring** (see `Main`) — it is recorded nowhere else.

When not to use it

- **Every request always runs every stage in a fixed order** — four calls in one method are clearer.
- **Handlers need each other's results** — not independent stages but a *pipeline* where each step reshapes a value; call steps in sequence passing an accumulating context object.
- **No terminal handler exists** — reaching the end with nothing done makes failures silent; add one first.
- **The chain is long and on a very hot path.**

Common mistakes

1. **The swallowing bug.** A handler checks and then just `return`s instead of calling `passOn` — no exception, no log; orders "sometimes don't appear". Fixes: an explicit outcome on every path (handled / rejected with reason / passed on); prefer `next` as an explicit parameter (servlet filters, Express, Go `http.Handler`) so forgetting it is visible; unit-test each handler against a stub successor recording whether it got the request.
2. **No terminal handler.** Reaching the end should be an error or a clear "not handled" — never a shrug.
3. **Wrong order, undocumented.** See trade-offs.
4. **Handlers that know about each other** (`new RateLimitHandler()` inside `AuthHandler`) — the chain is fixed in code again.
5. **Sharing a stateful chain object between requests.** `FilterChain` keeps a position, so it must be **one per request** (`Main` creates a new one each time).

Real-world examples

- **Servlet filters**: `jakarta.servlet.Filter.doFilter(request, response, chain)` — calling `chain.doFilter(...)` *is* "pass on". **Spring Security** (Spring's authentication/authorisation library) is a chain of such filters (`SecurityFilterChain`).
- **Spring MVC** `HandlerInterceptor`: `preHandle` returns `false` to stop the request.
- **HTTP middleware** (code a web framework runs around every request) in Express (`next()`), Django, Ruby's Rack, ASP.NET Core, Go (`func(http.Handler) http.Handler`, the usual Go web-service structure). **OkHttp** interceptors call `chain.proceed(request)`; **Netty**'s `ChannelPipeline` passes events handler to handler.

- **With first-class functions** (functions stored and passed like values) the chain is usually a list of `(request, next) => ...` functions; JavaScript frameworks often fold it (combine a list into one value step by step) with `Array.reduce`, and Python decorators (`@` wrappers) stacked on a view function (the function handling a web request) form a chain as syntax. Passing `next` explicitly removes the "forgot to pass on" bug by design.
- `java.util.logging`: a `LogRecord` goes to the logger's handlers and then, by default, its parent logger's handlers; `setUseParentHandlers(false)` stops that — why a parent logger's level or handler affects its children.
- **Exception handling**: each stack frame (a method call in progress) catches the exception or lets it travel up — the language's own chain.
- **DOM event bubbling** (DOM — Document Object Model, the browser's tree of page elements): a click climbs from the element through its parents; any listener can `stopPropagation()`.
- **Approval workflows** (expenses, leave, loans) by amount or role, and **customer-support escalation** tiers.

In the interview

30-second answer: "Chain of Responsibility splits a series of checks into separate handler objects, each linked only to its successor. The request moves down the line until some handler completes it or refuses it. Because only the wiring knows the order, adding, removing or reordering a rule is cheap. Servlet filters and HTTP middleware are the everyday examples. The classic bug is a handler that forgets to pass on and swallows the request, so I give every path an explicit outcome and put a terminal handler at the end."

Follow-ups:

- *How is it different from Decorator?* Both link objects sharing one interface. With **Decorator** all wrappers run and the call always reaches the inner object — the point is to *add* behaviour. With **CoR** a handler may **end** the request and later handlers never see it.
- *How is it different from Observer?* Observer tells **every** subscriber and none can block the rest; CoR ends at the **first** handler that accepts.
- *How is it different from a pipeline?* A pipeline pushes a value through **all** steps and produces a transformed result; a chain *routes* a request to whichever handler takes it. If no stage can stop the flow, it is a pipeline.
- *Combine it with Command?* Yes — the request moving through the handlers is often a Command (e.g. a job checked by validation and authorisation handlers before a worker runs it).
- *What if no handler can handle the request?* An explicit terminal handler (e.g. `NoHandlerFound`) that returns an error or logs.
- *How do you test it?* Each handler with a fake next recording whether and with what it was called; one integration test (several parts together) for the wiring order.
- *Rate limiting before or after auth?* Explain the trade-off above and document the reason next to the wiring.

Self-check

1. **Q:** What does each handler hold a reference to? **A:** Only the next handler (through the abstract `Handler` type).
2. **Q:** Name the most common CoR bug. **A:** A handler that neither handles nor passes on — the request silently disappears.
3. **Q:** Why do modern frameworks pass `next` as a parameter? **A:** A missing `next(...)` call is then easy to spot in the handler's body, and the handler can work both before and after the rest of the chain.
4. **Q:** Is a logging chain where every handler always passes on a "pure" CoR? **A:** Not strictly; since no handler can stop the flow it behaves like a pipeline. It is a common variant.
5. **Q:** When should you *not* use CoR? **A:** When every stage always runs in a fixed order, when stages depend on each other's results, or when there is no terminal handler.

TL;DR

A request walks a line of handlers; each handles, rejects, or passes on. The chain is data, so it is easy to change — but **order matters**, **always have a terminal handler**, and **never swallow a request**.

10.2 Command

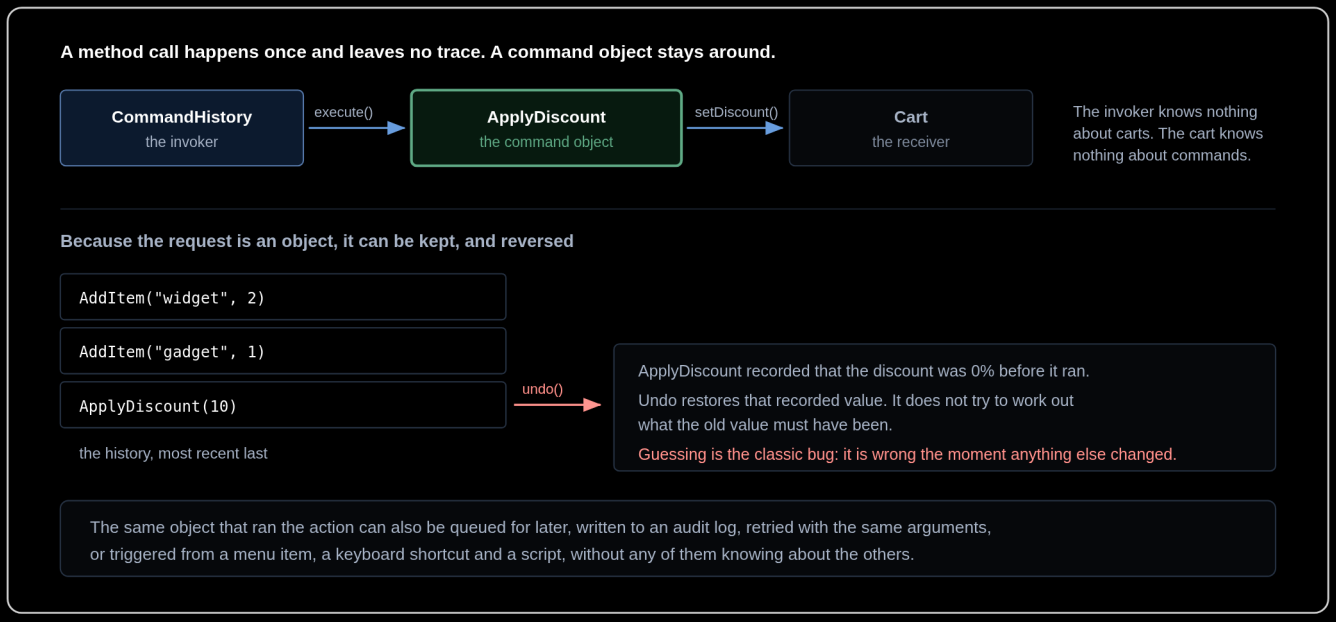


Figure — The invoker (CommandHistory) runs command objects against the receiver (Cart) and keeps them so they can be undone (course diagram). The figure assumes the discount was 0% before; our code deliberately starts at 5% to prove undo restores the recorded value.

What it is

Command packages a request ("add 2 widgets to the cart") as an **object** holding everything needed to perform it later. Once the request is a value you can **keep it in a list, put it on a queue (first-in, first-out waiting line), write it to an audit log (a permanent record of who did what), run it again, hand it to a worker thread (a background thread — independent path of execution — that runs jobs)** — and, if it knows its inverse, **reverse it**.

GoF intent: *wrap a request in an object, so clients can be given different requests, requests can be queued or logged, and operations can be undone.* Like a restaurant order slip: the waiter (invoker) writes the slip (command) and queues it; the cook (receiver) acts later; the slip can be queued, prioritised or cancelled.

The problem it solves

`cart.setDiscount(10)` runs immediately and is forgotten on return. Not enough once you need:

- **Undo** — "reverse my last two changes".
- **A queue** — run later on a worker thread, not the request thread (the one serving the user).
- **A log of intent** — which operations were *requested*, not just the final state.
- **Retry** — rerun a failed action with the same arguments.
- **One action, many triggers** — menu item, hot-key (keyboard shortcut) and script all fire the same "apply discount".

Command gives the request a body that outlives the call.

How it works

Interviewers usually want **three roles** — **invoker**, **command**, **receiver** — plus the **client** creating commands. GoF lists five participants: **Command** (interface), **ConcreteCommand** (binds a receiver to an action), **Client**, **Invoker**, **Receiver**:

Role	Job	In our code
Command	Interface with `execute()` (and optionally `undo()`).	`Command`, `UndoableCommand`
Concrete command	Stores its receiver and arguments; can perform (and reverse) exactly one operation.	`AddItemCommand`, `ApplyDiscountCommand`, `MacroCommand`
Receiver	The object whose state is really modified. Has no idea commands exist.	`Cart`
Invoker	Triggers commands and keeps a record of them. Has no idea carts exist.	`CommandHistory`, `ExecutorService`
Client	Builds a concrete command and gives it its receiver.	`Main`

1. The client creates a concrete command with the **receiver** and **arguments**, and hands it to the **invoker**.
2. The invoker calls `execute()`; if undoable, pushes it on an **undo stack** (last-in, first-out: add and remove only at the top).
3. **Undo** = pop, `undo()`, push on a **redo stack**.
4. **Redo** = pop from redo, `execute()`, push back on undo. **A fresh command empties the redo stack** — the branch you might have re-applied is replaced.

Structure (UML)

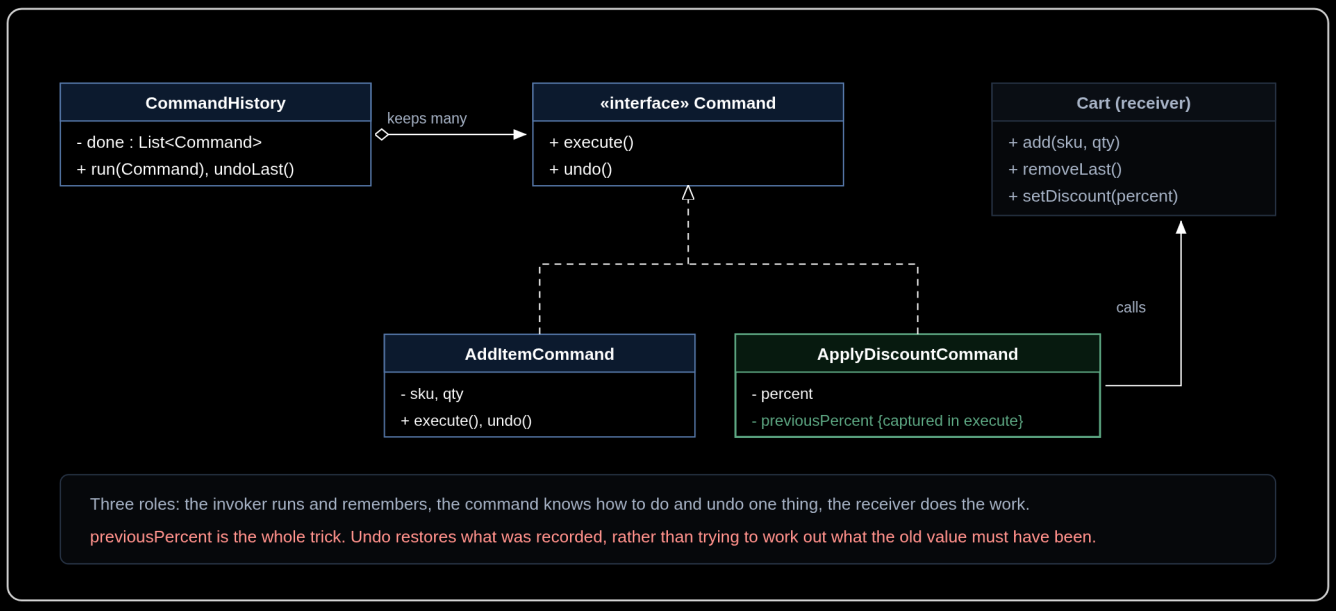
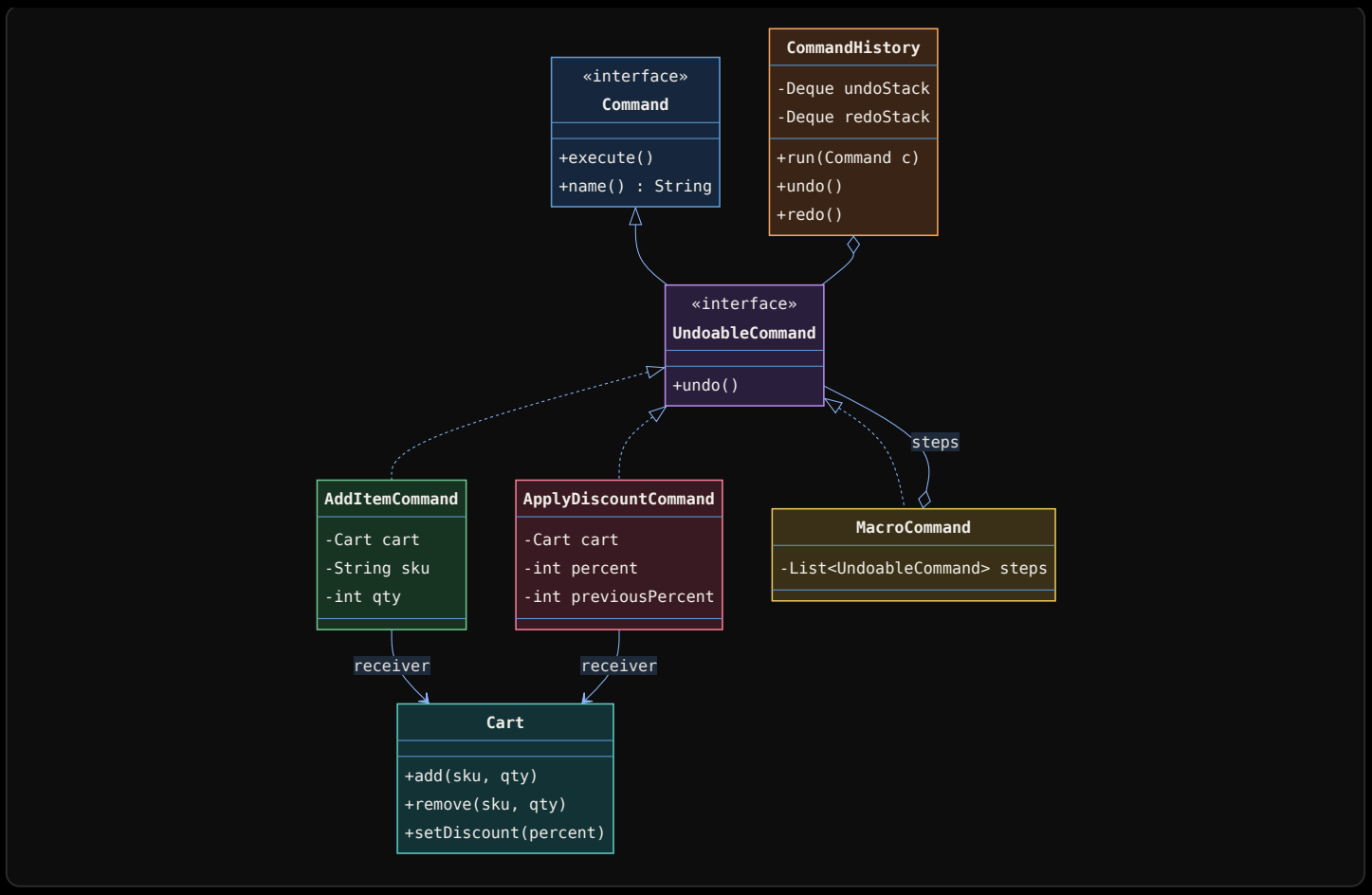


Figure — Class diagram: Command interface with execute/undo, two concrete commands, Cart as receiver, CommandHistory as invoker (course diagram; its `undoLast`/`removeLast` are named `undo`/`remove` in our code)

Each command keeps its own `Cart` reference — when it finally runs, its creator may be long gone, so the target travels inside the command with the arguments. `ApplyDiscountCommand.previousPercent` is the single field that makes its undo correct. `CommandHistory` never mentions `Cart`, so one history serves the whole application.



Java example

Full code: `code/lld/ch10/command/`. Key points:

- `Command` has **one abstract method**, so any lambda (short unnamed function) or `Runnable`-like block (Java's built-in "code to run" interface) is a command. Only *honestly* reversible commands implement `UndoableCommand`; `CommandHistory` keeps irreversible ones out of the undo stack.
- `ApplyDiscountCommand` **captures the old value in `execute()`** and restores exactly that. The cart starts at 5% to show "undo = set to 0" would be wrong.
- `MacroCommand` is a command of commands (a **Composite** of commands): steps run in order and undo in **reverse**.
- Section 4 of `Main` is the JDK's version: `Runnable` is the command interface and an `ExecutorService` (Java's thread-pool service) the invoker running commands on a worker thread.

```
@FunctionalInterface
public interface Command {                          // a request turned into an object; a lambda is a Command
    void execute();
    default String name() { return getClass().getSimpleName(); }
}
public interface UndoableCommand extends Command { void undo(); }   // only honestly reversible commands

// AddItemCommand: carries its receiver and all the arguments it needs to run later.
@Override public void execute() { cart.add(sku, qty); }
@Override public void undo()    { cart.remove(sku, qty); }   // removes exactly that line

// ApplyDiscountCommand: captures the previous value during execute so undo RESTORES, never guesses.
@Override public void execute() {
    previousPercent = cart.discountPercent();   // capture BEFORE changing
    cart.setDiscount(percent);
}
@Override public void undo() { cart.setDiscount(previousPercent); }

// MacroCommand: runs in order; undoes in REVERSE order.
@Override public void execute() { steps.forEach(Command::execute); }
@Override public void undo() { for (int i = steps.size() - 1; i >= 0; i--) steps.get(i).undo(); }
```

```java
// CommandHistory.java — the INVOKER: runs commands and remembers them. Knows nothing about carts.
public void run(Command command) {
    command.execute();
    if (command instanceof UndoableCommand u) {
        undoStack.push(u);
        redoStack.clear();     // a new action destroys the "future" you could have redone
        System.out.println("  ran  " + command.name());
    } else {
        System.out.println("  ran  " + command.name() + " (irreversible: kept out of history)");
    }
}
public void undo() {
    if (undoStack.isEmpty()) { System.out.println("  nothing to undo"); return; }
    UndoableCommand c = undoStack.pop();
    c.undo();
    redoStack.push(c);
    System.out.println("  undo " + c.name());
}
// redo(): pop redoStack, execute(), push on undoStack, print "  redo ..." (or "  nothing to redo")

// Main.java, section 4: commands on a queue
ExecutorService worker = Executors.newSingleThreadExecutor();   // one thread => FIFO order
jobs.forEach(worker::submit);
```

Output:

```
=== 1. Undo / redo ===
  ran  AddItem(widget, 2)
  ran  AddItem(gadget, 1)
  ran  ApplyDiscount(10%)
  Cart[widget x2, gadget x1] discount=10%
  undo ApplyDiscount(10%)
  undo AddItem(gadget, 1)
  Cart[widget x2] discount=5%   <- discount restored to the recorded 5%, not 0%
  redo AddItem(gadget, 1)
  Cart[widget x2, gadget x1] discount=5%
  ran  AddItem(cable, 3)
  nothing to redo

=== 2. Macro command ===
  ran  Macro:starter-kit
  Cart[widget x2, gadget x1, cable x3, keyboard x1, mouse x1] discount=15%
  undo Macro:starter-kit
  Cart[widget x2, gadget x1, cable x3] discount=5%

=== 3. Irreversible command (no undo offered) ===
  [email] receipt sent for Cart[widget x2, gadget x1, cable x3] discount=5%
  ran  SendReceipt (irreversible: kept out of history)

=== 4. Commands on a queue: Runnable + ExecutorService (the invoker) ===
  queued 3 jobs
  [worker] resize image 1
  [worker] resize image 2
  [worker] send newsletter
  all jobs done
```

Undo must restore, not recompute

Writing a discount undo as "reset to 0" or "work out what it probably was" is a **guess**: if the cart began at 5%, or another command touched the discount meanwhile, the user gets a value that never existed. Rule: `execute` **must record whatever** `undo` **will need** — usually just the fields it overwrites, not a copy of the whole object.

Undo history is strictly last-in, first-out. `AddItem, AddItem, ApplyDiscount` is undone as `ApplyDiscount`, then the second `AddItem`, then the first; any other order can produce never-existing states — hence a stack, never a randomly indexed list.

When undo is a lie. Some effects escape your system: a sent email, a charged card, a call to a partner's API (the interface a service exposes to other programs). An `undo()` there promises the impossible — worse than no undo. Honest options: keep them out of the history (our `SendReceipt`), or **compensation** — a *new* action reaching a similar end state, like a **refund** for a charge. A refund is not an undo: both events stay in the record.

Trade-offs

Pros	Cons
The request outlives the call, so undo/redo, queues, audit logs, retries and scheduling become possible.	One class per action; class count tracks action count.
The invoker knows nothing of what it runs, so one history serves all commands.	Each command must record enough to run later and reverse itself; subtle mistakes are easy.
One action object can back a menu entry, hot-key, script and API endpoint.	Extra hop when reading: to learn what a button does, find the command it builds.
Commands can be grouped into macro commands.	One faulty `undo` corrupts the whole history.

When not to use it

- **The call is simple and immediate** — a `SaveCommand` nobody undoes, queues or logs is code without benefit.
- **Undo is not really possible** — use compensation and call it that.
- **You only need deferral** — a lambda (`Runnable`) already is a one-method command; promote it to a class when you need `undo`, a readable log name, or serialization (turning it into bytes/text to store or send).
- **Commands must outlive the process** — then serialization drags in format versioning; a durable queue (survives restarts — see System Design Fundamentals) or an event log (Kafka — a durable, distributed message log — or a jobs table) handles it better.

Common mistakes

1. **Recomputing in `undo`** instead of restoring captured state.
2. **Forgetting to clear the redo stack** when a new command runs.
3. **Business logic in the invoker** (`if (cmd instanceof AddItem) ...`). The invoker only runs and remembers.
4. **Capturing a mutable object by reference** (mutable = changeable after creation) and reading it later when it has changed. Capture values.
5. **Unbounded history** — cap the depth, as editors do.

Real-world examples

- **Undo in editors**: word processors, IDEs (code editors such as IntelliJ), Photoshop. JDK: `javax.swing.undo.UndoManager` with `UndoableEdit` objects having `undo()`/`redo()`.
- **`Runnable` / `Callable`** are the JDK's command interfaces; `ExecutorService`, `ScheduledExecutorService` and `Thread` are invokers.
- **GUI actions** (graphical user interface): Swing's (Java's desktop UI toolkit) `javax.swing.Action` backs a menu item, toolbar button and shortcut at once; .NET WPF's `ICommand` likewise.
- **Event sourcing** (Chapter 11; storing every change as an event and rebuilding state by replay): recording *requested changes* rather than resulting state — the command idea for a whole system.
- **Job queues**: a Sidekiq/Celery (Ruby/Python background-job libraries) job, a Spring `@Async` task, a Kafka/RabbitMQ (message brokers) message asking a worker to "send invoice #42" — a serialized command.
- **Database migrations** (versioned schema-change scripts) with `up`/`down` (e.g. Liquibase changesets with rollback) mirror `execute`/`undo`.
- **Redux** (a JavaScript state-management library) actions are commands: the store dispatches them (invoker), reducers apply them (receiver); Redux DevTools' time travel replays them.
- **In other languages**: Python — a callable object (defines `__call__`) with an `undo` method, `functools.partial` to bind arguments; C# — UI frameworks ship `ICommand`, and a `record` gives an immutable command in one line.
- **Transactional outbox / task tables** (the job is written to a table in the same all-or-nothing transaction as the data change, so it cannot be lost) in food-delivery or e-commerce back ends store "commands" (send SMS, notify restaurant) run reliably by workers.

In the interview

30-second answer: "Command wraps a request, together with its receiver and arguments, in an object — so the request can be kept, queued, logged, retried or reversed. Three roles matter: the invoker triggers and records commands without knowing what they touch, each command can perform and reverse a single operation, and the receiver holds the state that actually changes. For undo, `execute` must capture the old value and

> `undo` restores it; history is a stack, and a new command clears redo. Irreversible actions like charging a card get compensation, not undo. In Java, `Runnable` plus an `ExecutorService` is the everyday form."

Follow-ups:

- *Command vs Strategy?* Both put behaviour in an object. **Strategy** answers *how* a job is done and is picked/replaced by the caller. **Command** captures *which* job, arguments included, held on to so it can run later or be reversed.
- *Command vs a plain lambda?* A one-method closure (a function remembering variables from where it was created) already is a command; write a class only for `undo`, a readable log name, or serialization.
- *Command vs Memento for undo?* Command stores the **action** plus a little state; undo runs the inverse. Memento stores the **whole state**; undo restores a snapshot. Command is cheaper per step; Memento is always correct. Real systems combine them (occasional snapshots, replay commands in between).
- *How do you implement redo?* A second stack; undo moves a command to it; redo re-executes and moves it back; a new command clears it.
- *How do you make a command queue thread-safe (correct with many threads)?* A `BlockingQueue` (whose `take()` waits for an item) or an `ExecutorService`; commands immutable apart from captured undo state.
- *How would you persist commands?* Serialize (JSON — a common text format — with a type field and version), store in a durable queue or table, and make them **idempotent** (running twice = once — see System Design Fundamentals) so retries are safe.

Self-check

1. **Q:** What must a command carry? **A:** Its receiver and all arguments needed to run later (and any state needed for undo).
2. **Q:** Why does `ApplyDiscountCommand` store `previousPercent`? **A:** So undo restores the real previous value instead of guessing.
3. **Q:** What happens to the redo stack when a new command runs? **A:** It is cleared.
4. **Q:** How do you "undo" a card charge? **A:** You don't; you issue a compensating action (a refund) and keep both in the record.
5. **Q:** Which JDK interface is a command? **A:** `Runnable` (and `Callable`).

TL;DR

Make the request an object: **invoker** runs it, **command** knows do/undo, **receiver** changes. Undo **restores captured state**, history is a **stack**, and irreversible actions get **compensation**. A lambda is a command when you only need deferral.

10.3 Iterator

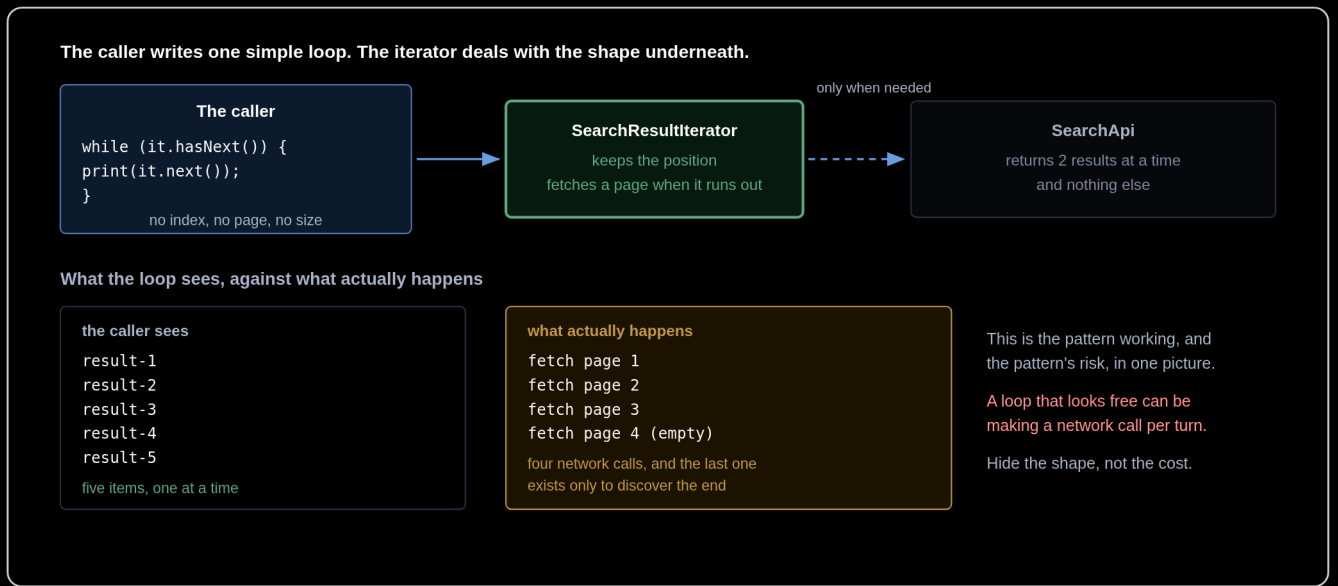


Figure — The caller writes one simple loop; the iterator fetches pages from the API only when needed (course diagram)

What it is

Iterator visits a collection's elements **one at a time, in order, without knowing how they are stored** — array, linked list (elements chained by references), tree, or a remote API returning two results per page. Like a TV remote's "next channel": no idea how channels are stored, just press *next*.

GoF intent: *give sequential access to the elements of an aggregate (GoF's word for a collection) without exposing its internal representation.* Participants: **Iterator** (interface for accessing and traversing), **ConcreteIterator** (implements it, tracks the current position), **Aggregate** (interface for creating an iterator), **ConcreteAggregate** (returns the right ConcreteIterator).

The problem it solves

- Loops using `get(i)` assume an array; move the data to a linked list, tree or paginated (one page at a time) service and every loop is rewritten.
- Many sources **cannot be indexed**: a tree has several visiting orders (depth-first — one branch fully down before the next; breadth-first — level by level); a graph (nodes joined by edges, possibly looping) must remember visited nodes; a search API hands out pages. Each caller writing its own traversal re-implements paging — some wrongly.

Iterator moves the walking logic **into a separate object** answering *any elements left?* (`hasNext`) and *give me the next* (`next`).

Key detail: the current position lives inside the iterator, never in the collection — so two traversals run together without interfering, and each `iterator()` call must build a **new** iterator rather than rewind a shared cursor (marker of the current position).

How it works

1. The **aggregate** has `iterator()` creating a new iterator; in Java it implements `Iterable<T>`.
2. The **iterator** implements `Iterator<T>`: `hasNext()`, `next()` (optionally `remove()`).
3. The iterator keeps its own **cursor** (index, stack of tree nodes, current page...).
4. The client loops `while (it.hasNext()) use(it.next());` — or `for (T x : collection)`, which the compiler turns into exactly that.

Structure (UML)

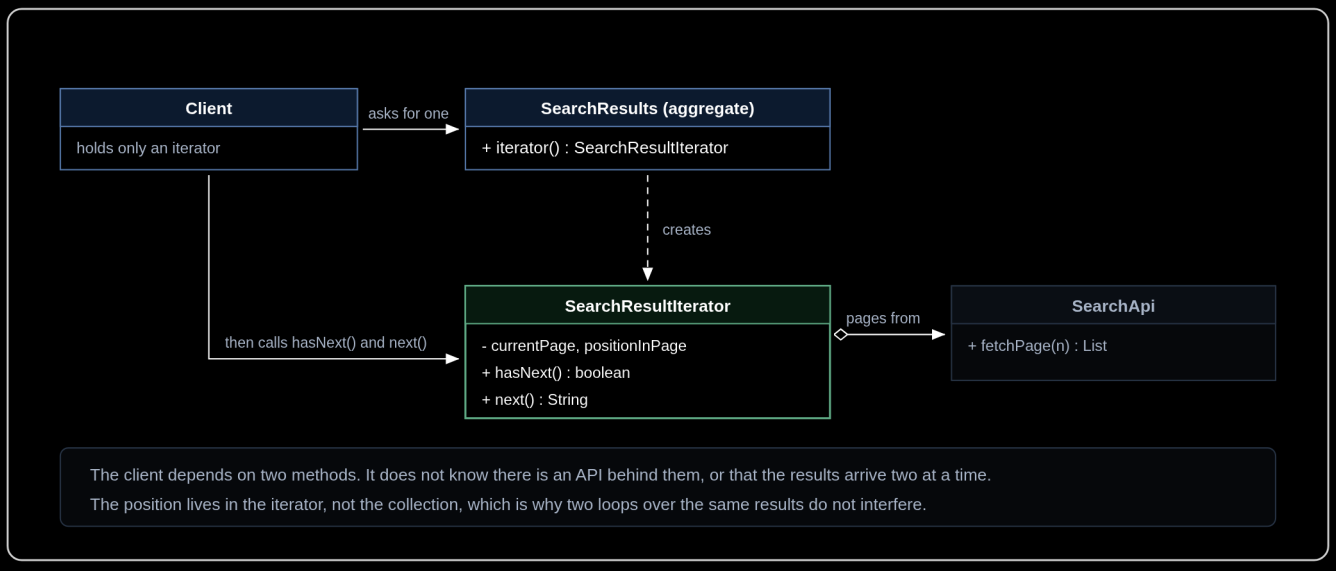
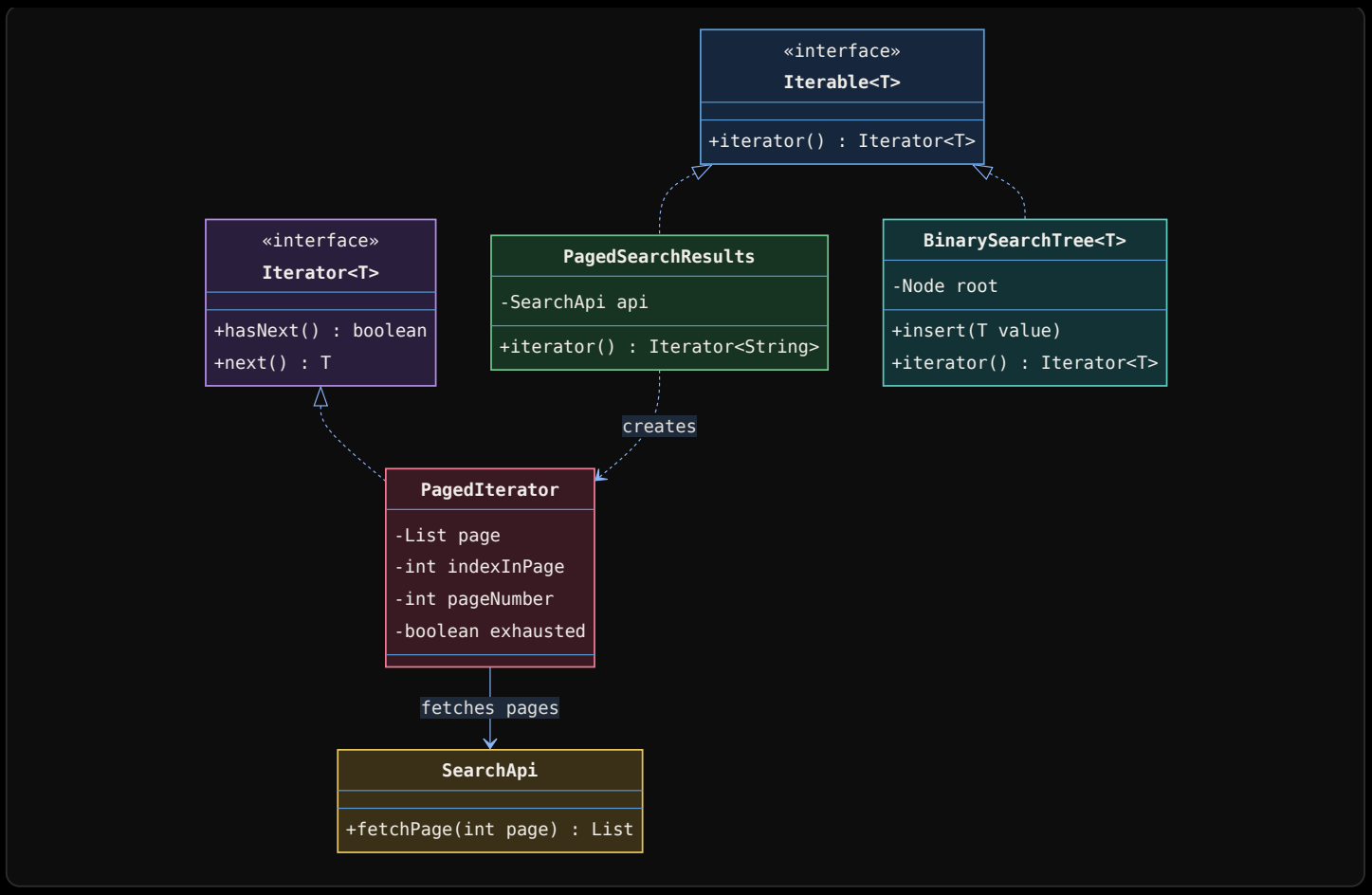


Figure — Class diagram: SearchResults (aggregate) hands out a SearchResultIterator, which calls SearchApi page by page; the client only calls hasNext/next (course diagram)

`hasNext` does real work: when the buffered page runs out it fetches another. *Lazy* iterators (producing elements only on demand) commonly behave this way — never assume `hasNext` is cheap or free of side effects (effects beyond returning a value, such as a network call).



Java example

`code/lld/ch10/iterator/` shows:

1. **A paged remote source behind a plain for-each.** `PagedSearchResults implements Iterable<String>`; its inner `PagedIterator` fetches a page only when the current one is used up and follows the `Iterator` contract (the documented rules every implementation must obey: `next()` throws `NoSuchElementException` when empty).
2. **A tree iterator.** `BinarySearchTree` hides an in-order walk (left subtree, node, right subtree — sorted order) behind `hasNext/next` with an explicit stack; two iterators over one tree run independently.
3. **External vs internal iteration.** *External*: the **client** drives (`hasNext/next`) and can `break` early. *Internal*: the **collection** drives and calls your code per element (`forEach(lambda)`, streams — Java's pipeline API for collections).
4. **Fail-fast vs fail-safe.** Structurally changing an `ArrayList` (add/remove) during a for-each *usually* makes the next `next()` throw `ConcurrentModificationException` — only **best-effort**: removing the second-to-last element makes `hasNext()` just return `false`, no exception, last element silently skipped. Safe: `Iterator.remove()` (or `removeIf`). `CopyOnWriteArrayList` iterates a **snapshot**, never throws, but the loop misses new elements. ("Fail-safe" is informal; the Javadoc — Java's official API docs — says *snapshot* and *weakly consistent*.)
5. `Spliterator` (Java 8): an iterator that can **split** itself (`trySplit`) so parallel streams process parts on different threads; reports **characteristics** such as `SIZED` and `ORDERED`.

```java
// PagedSearchResults.java — the AGGREGATE: its only job is to hand out a FRESH iterator each time.
// (SearchApi.fetchPage(page) returns 2 results per 1-based page, an empty list past the end, and counts calls.)
final class PagedSearchResults implements Iterable<String> {
    @Override public Iterator<String> iterator() { return new PagedIterator(); }

    /** The ITERATOR. The position lives here, not in the collection. */
    private final class PagedIterator implements Iterator<String> {
        private List<String> page = List.of();
        private int indexInPage = 0, pageNumber = 0;
        private boolean exhausted = false;

        @Override public boolean hasNext() {                // NOT a passive question: may hit the network
            if (indexInPage < page.size()) return true;
            if (exhausted) return false;
            page = api.fetchPage(++pageNumber);
            indexInPage = 0;
            if (page.isEmpty()) { exhausted = true; return false; }
            return true;
        }
        @Override public String next() {
            if (!hasNext()) throw new NoSuchElementException();   // the Iterator contract
            return page.get(indexInPage++);
        }
    }
}

// BinarySearchTree.iterator() — the stack holds the path still to visit: this IS the iterator's position.
private final Deque<Node<T>> stack = new ArrayDeque<>();
{ pushLeft(root); }
private void pushLeft(Node<T> n) { for (; n != null; n = n.left) stack.push(n); }
@Override public boolean hasNext() { return !stack.isEmpty(); }
@Override public T next() {
    if (stack.isEmpty()) throw new NoSuchElementException();
    Node<T> n = stack.pop();
    pushLeft(n.right);
    return n.value;
}
```

```java
// Main.java, section 4 (fail-fast vs fail-safe) and 5 (Spliterator)
for (String t : tags) if (t.equals("tmp")) tags.remove(t);       // structural change behind the iterator's back
for (String t : quiet) { visited.add(t); if (t.equals("tmp")) quiet.remove(t); }  // removes the SECOND-TO-LAST element
for (Iterator<String> i = tags.iterator(); i.hasNext(); ) if (i.next().equals("tmp")) i.remove();  // the safe way
for (String s : cow) cow.add(s + "!");                           // CopyOnWriteArrayList iterates a snapshot
Spliterator<Integer> right = nums.spliterator();
Spliterator<Integer> left = right.trySplit();                    // hands off (roughly) the first half
```

Output:

```
=== 1. Paged remote results behind a plain for-each ===
    [api] fetching page 1
  loop sees result-1
  loop sees result-2
    [api] fetching page 2
  loop sees result-3
  loop sees result-4
    [api] fetching page 3
  loop sees result-5
    [api] fetching page 4
  5 items, 4 network calls (the last one only discovers the end)

=== 2. Two independent iterators over one tree ===
  a: 20, 30, 40
  b: 20   <- b starts from the beginning: position lives in the iterator
  in-order: 20 30 40 50 60 70 80

=== 3. External vs internal iteration ===
  external: found bob, stopping early
  internal: ann
  internal: bob
  internal: cy

=== 4. Fail-fast vs fail-safe ===
  fail-fast: ConcurrentModificationException
  best-effort only: no exception, visited [java, tmp], 'lld' silently skipped
  after Iterator.remove: [java, lld]
  fail-safe CopyOnWriteArrayList: [x, y, x!, y!]

=== 5. Spliterator: an iterator that can split for parallel work ===
  SIZED=true ORDERED=true
  left half:  1 2 3 4
  right half: 5 6 7 8
  parallel stream sum = 36
```

Hide the shape, not the cost

Five results, **four** network calls: the last returns empty just to learn the data ended, since this API gives neither a total nor a "next page" token — real behaviour, not a demo bug. **A tidy loop conceals each step's cost.** The **N+1 query problem** in ORMs (object-relational mappers: rows → objects) has the same shape (one query for the list, one more *per element*). Keep the cost visible:

- **Name the type honestly** (`PagedResults`, `LazySearchResults`, not `List`).
- **Offer a batch API** with bigger pages for callers wanting everything.
- **Return the total or a next-page cursor** when the source knows it, avoiding the extra call.
- **Log or count fetches**, so a growing request count gets noticed.

Java's iterator family (quick reference)

Type	What it adds
`Iterable<T>`	`iterator()`; lets a type be used in the enhanced for-loop. Also `forEach` (internal iteration) and `spliterator()`.
`Iterator<T>`	`hasNext()`, `next()`, optional `remove()`, `forEachRemaining()`.
`ListIterator<T>`	Bidirectional: `hasPrevious()`, `previous()`, indexes, `set()`, `add()`.
`Spliterator<T>`	`tryAdvance`, `forEachRemaining`, `trySplit` for parallelism, `characteristics()` like `ORDERED`, `SIZED`, `DISTINCT`, `IMMUTABLE`.
`Enumeration<E>`	The old (Java 1.0) version: `hasMoreElements()`, `nextElement()`. Prefer `Iterator`.

Fail-fast, weakly consistent, snapshot. `ArrayList`, `HashMap` and friends have **fail-fast** iterators: they detect a structural change behind their back and throw `ConcurrentModificationException` — on a **best-effort** basis per the Javadoc, so never rely on it (demo section 4 shows an unnoticed removal). Despite the name, it is usually **one thread** changing a collection inside its own loop, not concurrency (several threads at once). `ConcurrentHashMap` iterators are **weakly consistent** (never throw, may or may not show concurrent changes). `CopyOnWriteArrayList` iterators use a **snapshot** (their `remove()` throws `UnsupportedOperationException`).

Trade-offs

Pros	Cons
Client code no longer cares whether data lives in an array, a tree or behind an API.	One more object per traversal (noticeable only in extremely hot loops).
One structure can offer several named, reusable visiting orders.	Changing the collection mid-loop is a bug unless via the iterator's `remove` or a concurrent collection.
Elements can be produced lazily, so huge or endless sequences work.	A lazy iterator owning a file or DB cursor leaks it on early `break` unless closed.
Paging, buffering and prefetching (loading data before it is asked for) hide behind an ordinary loop.	Per-step cost becomes invisible — the root of N+1 problems.

When not to use it

- **The data is already an in-memory list** — hand out the list (or its iterator); a custom iterator over an array adds nothing.
- **Callers jump around** ("I need element 4,000") — iterators only step forward; offer indexed access.
- **Everything is in memory and always consumed in full** — a plain loop is simpler and faster.
- **A step has side effects the caller must decide on** — silently loading a page is fine; silently charging a customer is not.

Common mistakes

1. **Modifying the collection while iterating** (other than `Iterator.remove`) — collect changes and apply after the loop, or `removeIf`.
2. **Leaking a resource** from a lazy iterator on early `break` — use try-with-resources (`try (...)` closes automatically; `Files.lines`, `DirectoryStream` and JDBC — Java's database API — `ResultSet` are closeable).
3. **Reusing a one-shot iterator** or stream: it is exhausted. A Java `Stream` throws `IllegalStateException`; a Python generator (a function producing values one at a time with `yield`) silently yields nothing, so the bug shows as missing data — why `iterator()` must return a *new* object.
4. **`next()` without `hasNext()`**, or a `next()` returning `null` at the end instead of throwing `NoSuchElementException`.
5. **Storing the cursor in the collection** — two loops interfere.
6. **Assuming every language fails loudly.** Java usually throws `ConcurrentModificationException`; Python may skip elements and finish with a wrong result. Record intended changes and apply them after the loop.

Real-world examples

- **Every for-each loop** in Java, Python, C#, JavaScript (`for...of`).
- **JDBC** `ResultSet` walks rows not all loaded at once; `Scanner` implements `Iterator<String>`; `Files.newDirectoryStream` returns an `Iterable` over directory entries.
- **Paginated SDKs** (a provider's client libraries): AWS (Amazon Web Services) SDK for Java v2 "paginators" and Stripe's auto-pagination give an iterable fetching pages behind the scenes — exactly section 1.
- **Java** `Stream`**s** are built on `Spliterator`s (how parallel streams split work); C#'s LINQ (built-in collection query syntax) is likewise lazy over iteration — both work on endless sequences.
- **Generators**: Python's `yield`, JavaScript's `function*`, C#'s `yield return` write an iterator as a *function*; the language remembers the position — same pattern, less code.
- **Kafka consumer** `poll()` **loops**, and **infinite feeds** (Instagram/Twitter timelines) loading the next page as you scroll.

In the interview

30-second answer: "Iterator pulls traversal out into a separate object with `hasNext` and `next`, so a caller can walk data without caring whether the data is an array, a tree or a paginated API. Each iterator keeps its own position, so two loops over the same data are independent. Rather than a home-made interface I'd implement the language's protocol — `Iterable<T>` in Java — so the native for-each works. My example would be the lazy paged case, not an array, and I'd flag the risk: when every step can hit the network, the loop hides its cost — that is how N+1 problems start."

Follow-ups:

- *Internal vs external iterator?* External: the client steps (`hasNext/next`) and can stop, pause or interleave two iterators. Internal: the collection loops and calls your function (`forEach`, streams) — simpler, easier to parallelise, harder to stop early or walk two collections in lock-step (one element from each at a time).
- *What is fail-fast?* The iterator checks a modification counter and throws `ConcurrentModificationException` if the collection was structurally changed outside it. Best-effort only.
- *How do you remove while iterating?* `Iterator.remove()` or `Collection.removeIf(predicate)` (a true/false function).
- *What is a Spliterator?* A splittable iterator used by streams; `trySplit` hands off part of the elements to another thread; characteristics tell the framework what it can optimise.
- *Iterator vs Visitor?* Iterator decides **how you move**; Visitor **what you do** at each element. Often used together.
- *Iterator vs Composite?* Composite makes leaves and branches look alike; an iterator supplies a way to visit them. Often used together.
- *Iterator vs generator?* A generator (Python `yield`, JS `function*`) is the same pattern with a function that `yield`s instead of a class.
- *Iterator vs stream?* Streams sit on iteration and add `map`, `filter`, etc.; use them to transform data, not merely visit it.
- *How is Iterator expressed in other languages?* Python: `__iter__`/`__next__` or a generator. JavaScript/TypeScript: `[Symbol.iterator]` or `function*`, enabling `for...of` and spread. C#: `IEnumerable<T>` with `yield return`. C++: `begin()`/`end()` for range-based for and the standard algorithms. Go: traditionally `Next() bool` plus a value accessor (`sql.Rows`, `bufio.Scanner`), and since Go 1.23 range-over-function iterators.

Self-check

1. **Q:** Where does the cursor live? **A:** In the iterator object.
2. **Q:** What does the enhanced for-loop need from a class? **A:** That it implements `Iterable<T>`.
3. **Q:** Why did five results take four API calls? **A:** Without a total or cursor, the iterator can only discover the end by asking for one more page.
4. **Q:** Does `CopyOnWriteArrayList`'s iterator throw on concurrent modification? **A:** No; it iterates over a snapshot taken when the iterator was created.
5. **Q:** What should `next()` do when there are no more elements? **A:** Throw `NoSuchElementException`.

TL;DR

A separate object walks the collection with `hasNext/next`; the **position lives in the iterator**. Implement `Iterable` in Java. Know **fail-fast** vs snapshot iterators, **internal vs external** iteration, and don't let a lazy loop **hide its cost**.

10.4 Mediator

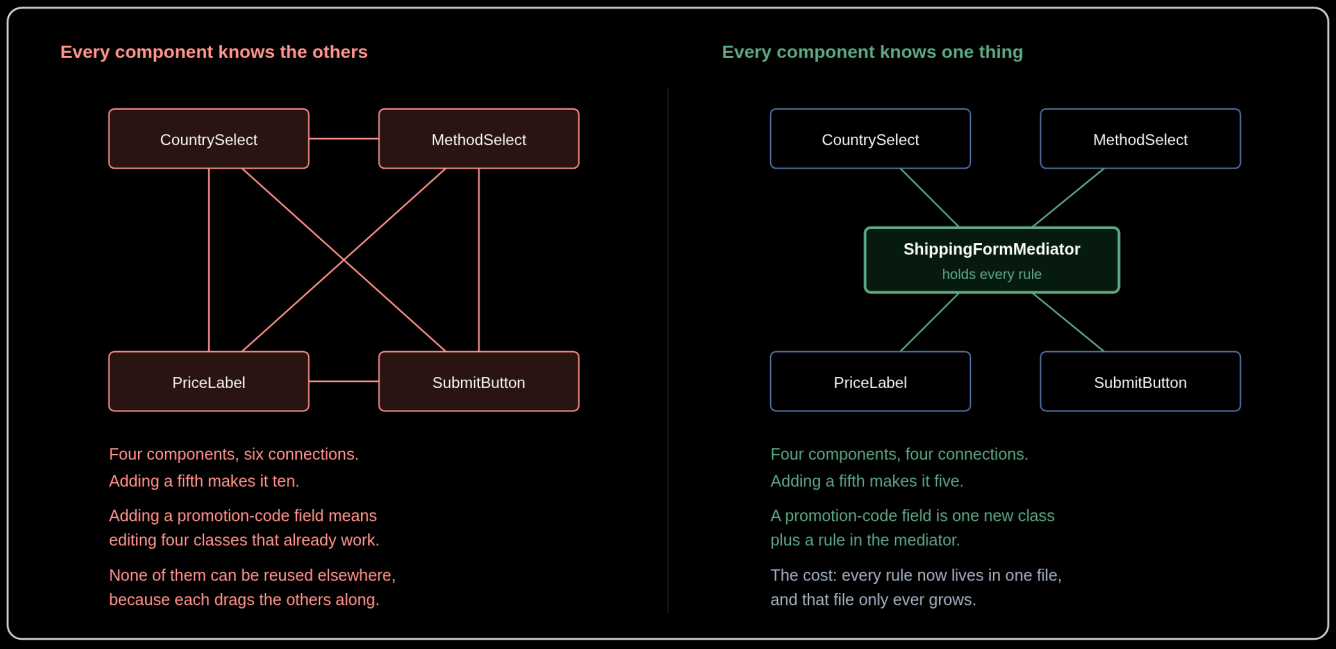


Figure — Left: four form components all know each other (six links). Right: each knows only the mediator (four links) (course diagram)

What it is

Mediator makes a group of objects talk through **one coordinator** instead of referencing each other. Each object only reports "this just happened"; the mediator works out how the rest respond — like **air traffic control**: planes talk only to the tower, which holds the spacing and ordering rules.

GoF intent: *define an object that encapsulates how a set of objects interact, so they stop referring to each other explicitly and their interaction can be varied independently.* Participants: **Mediator** (interface for talking to colleagues), **ConcreteMediator** (knows the colleagues, implements the coordination), **Colleague** classes (each knows its mediator and talks to it instead of other colleagues). Some literature calls the mediator a *controller*.

The problem it solves

A checkout form has four widgets: **country** picker, **delivery-method** picker, **price** label, **submit** button. Picking a country refreshes delivery methods; the new method changes the price; the price situation decides whether submit is clickable; some destinations offer no delivery at all.

If widgets talk directly:

- each holds references to three others — four widgets need **six** links, five need **ten** (n(n−1)/2, roughly quadratic);
- a new "promo code" field changes four tested classes;
- no widget is reusable: if `CountrySelect` calls `methodSelect.reload()`, it cannot go anywhere without `MethodSelect`.

With a mediator each widget has **one** reference: four widgets, four links; a fifth makes five. `CountrySelect` no longer knows a price label exists (or how many widgets do), so another screen can reuse it with its own mediator and rules.

The honest trade: dependencies are not removed — **they are relocated** into the mediator, a class that tends to keep growing.

How it works

1. A **Mediator** interface, e.g. `notify(sender, event)`.
2. Each **component** (colleague) holds a reference to the mediator — nothing else.
3. When something happens, a component calls `mediator.notify(this, event)`.
4. The **concrete mediator** holds all components and rules: "when the country changes, reload methods, then show the price, then enable or disable submit".
5. The mediator calls back **down** into specific components — traffic is **two-way** through one object.

Structure (UML)

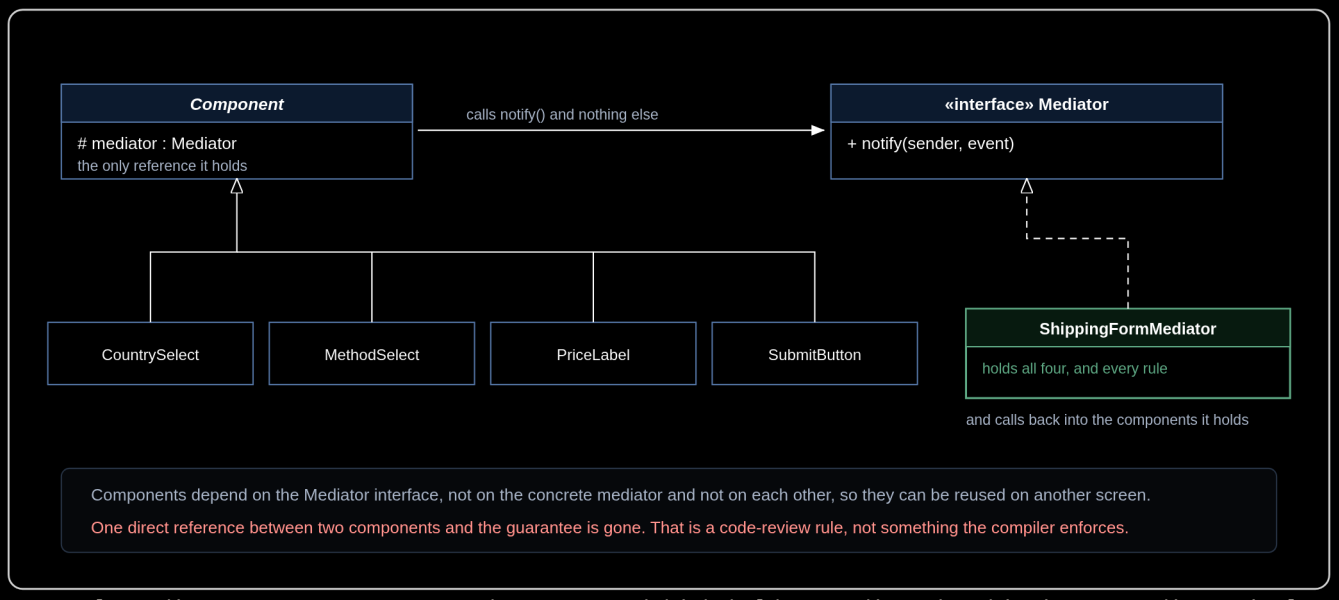
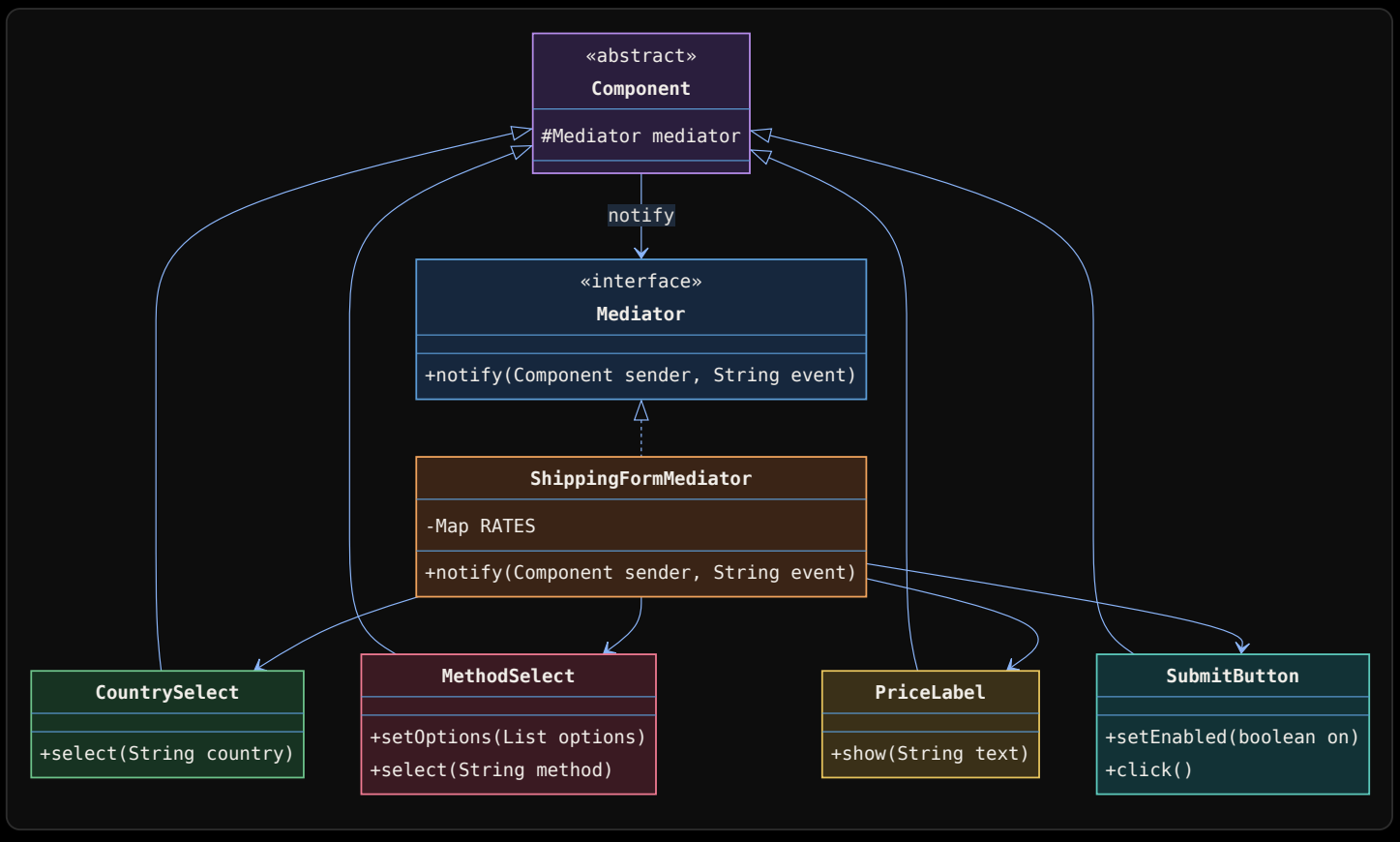


Figure — Class diagram: components extend Component (which holds a Mediator); ShippingFormMediator implements Mediator and holds all four components (course diagram)

Widgets report **upward** via `notify`; the mediator instructs the widgets it owns **downward**. Two-way traffic through one hub separates a mediator from an observer.



Java example

`code/lld/ch10/mediator/` has three mediators:

1. **Shipping form** (`ShippingFormMediator` + four components). Only `notify` knows a country choice affects the price. Components are identified by reference (`sender == country`), not strings. Rates are `BigDecimal` — money is never `double`.
2. **Chat room** (`ChatRoom`, `ChatUser`). Users never reference each other; the room routes broadcasts and private messages and applies **moderation** (filtering or blocking unwanted messages) — rules in the mediator, not the users.

3. **Air traffic control** (`ControlTower`, `Flight`). Flights only request landing and report "vacated"; the tower owns the runway and the holding queue.

```
public interface Mediator { void notify(Component sender, String event); }  // components depend on this only
public abstract class Component {                // base for every widget: its ONLY reference is the mediator
    protected Mediator mediator;
    void setMediator(Mediator mediator) { this.mediator = mediator; }
}
// CountrySelect.select(country): print, then mediator.notify(this, country);   // report up; never call a sibling
// MethodSelect.setOptions(options) defaults to the first; SubmitButton.click() notifies only if enabled.

// ShippingFormMediator.java — the ONLY place that knows how a country change affects methods, price and submit.
private static final Map<String, Map<String, BigDecimal>> RATES = Map.of(
    "Germany", ordered("standard", "5.00", "express", "15.00"),   // ordered(): LinkedHashMap keeps standard first
    "USA", ordered("standard", "8.00", "express", "20.00"),
    "Antarctica", Map.of());

@Override public void notify(Component sender, String event) {
    if (sender == country) {
        Map<String, BigDecimal> available = RATES.getOrDefault(event, Map.of());
        methods.setOptions(new ArrayList<>(available.keySet()));
        if (available.isEmpty()) {
            price.show("shipping not available");
            submit.setEnabled(false);
        } else {
            showPrice();                          // ORDER matters: reload methods first, then price
            submit.setEnabled(true);
        }
    } else if (sender == methods) {
        showPrice();
    } else if (sender == submit) {
        System.out.println("  [mediator] order placed: " + country.selected() + " / " + methods.selected());
    }
}
```

```
// ChatRoom.java — room rules (moderation, routing) live here, not in the users.
void send(ChatUser from, String to, String text) {
    if (bannedWords.stream().anyMatch(text.toLowerCase()::contains)) {
        from.receive("room", "message blocked by moderation");
        return;
    }
    if (to == null) members.values().stream().filter(u -> u != from).forEach(u -> u.receive(from.name(), text));
    else if (members.containsKey(to)) members.get(to).receive(from.name() + " (private)", text);
    else from.receive("room", to + " is not in the room");
}

// ControlTower.java — aircraft talk to the tower, never to each other.
void requestLanding(Flight f) {
    if (onRunway == null) clear(f);
    else { holding.add(f); System.out.println("  [tower] " + f.callsign() + ": hold, runway busy with " +
onRunway.callsign()); }
}
void runwayVacated(Flight f) {
    System.out.println("  [tower] " + f.callsign() + " vacated the runway");
    onRunway = null;
    Flight next = holding.poll();
    if (next != null) clear(next);
}
```

Output:

```
=== 1. Shipping form: widgets coordinated by a mediator ===
  [submit] disabled
  [country] user picked Germany
  [methods] reloaded: [standard, express], default standard
  [price] standard shipping 5.00
  [submit] enabled
  [methods] user picked express
  [price] express shipping 15.00
  [mediator] order placed: Germany / express
  [country] user picked Antarctica
  [methods] reloaded: []
  [price] shipping not available
  [submit] disabled
  [submit] click ignored: button is disabled

=== 2. Chat room ===
  asha sees <room> asha joined
  asha sees <room> ben joined
  ben sees <room> ben joined
  asha sees <room> chen joined
  ben sees <room> chen joined
  chen sees <room> chen joined
  ben sees <asha> standup in 5
  chen sees <asha> standup in 5
  chen sees <ben (private)> coffee first?
  chen sees <room> message blocked by moderation

=== 3. Air traffic control ===
  [tower] AI101: cleared to land
  [tower] BA22: hold, runway busy with AI101
  [tower] LH7: hold, runway busy with AI101
  [tower] AI101 vacated the runway
  [tower] BA22: cleared to land
  [tower] BA22 vacated the runway
  [tower] LH7: cleared to land
```

One user action ("picked Germany") updated three components — and **no component called another**.

Mediator or Observer?

Both decouple objects; the difference is **what happens to the news**.

	Mediator	Observer
What travels	An event **and** the decision on what should follow	Only the event
Where reaction logic sits	In one place: the mediator	Spread across the observers, each deciding alone
Do the parties know each other?	No — not even how many peers exist	No; the subject only knows it has a list of subscribers
Flow	Both ways: colleagues report in, the mediator calls out to them	One way: the subject publishes, observers respond
Order of reactions	Written explicitly in the mediator — a reason to choose it	Not part of the contract; observers should not depend on each other's order

Practical test: does something need coordinating? "Country changed; anyone interested, update yourself" is Observer — lighter. "Country changed: refresh methods, *then* price the new default, *then* decide submit" is sequenced coordination — Mediator. **With Observer the listeners decide; with Mediator the hub decides and gives orders.** (A mediator is often *implemented* with observer-style notifications from components.)

Trade-offs

Pros	Cons
Every widget depends on exactly one thing, so links grow linearly, not quadratically.	Every new rule lands in the mediator, so it keeps growing.
All interaction rules sit in one readable, unit-testable class.	Everyone changing the screen edits that file, so merge conflicts (clashing edits) pile up.
Widgets are reusable — they no longer drag neighbours along.	To learn why a widget changed you read the mediator, not the widget.
A new widget costs one class plus one rule.	It can quietly become a **god object** (a class that knows and does too much).

Keeping the mediator from becoming a god object:

- **Split by area** — one mediator per screen section or workflow, not per application. Two 80-line mediators beat one 160-line one.
- **Keep it to coordination.** Tax and pricing belong in their own classes; the mediator decides *what to call*, not *how to compute* (a Facade respects the same boundary).
- **Reject shortcuts in review.** One widget-to-widget reference breaks the guarantee, and the compiler won't catch it.
- **Dispatch by table, not `if`-chain.** With first-class functions keep a map from event (or sender) to handler — e.g. a Python `dict` of `{event: method}` — so each rule is easy to find.

When not to use it

- **Only two objects interact** — let them call each other.
- **Interactions are few and stable** — swapping many small links for one big central class pays off only once the tangle is large.
- **Nothing needs coordinating** — components react independently; use Observer.
- **The mediator is already a 400-line god object** — split it into smaller mediators before adding rules.

Common mistakes

1. **Components calling each other "just this once".**
2. **Computation inside the mediator** (tax, pricing), turning it into a god object.
3. **Confusing it with Facade.** A Facade sits *in front of* a subsystem whose classes are unaware of it; calls flow one way. A Mediator sits *among* peers that know it and report to it; calls flow both ways.
4. **String-typed events everywhere** (`"country"`, `"method"`) breaking silently on a typo. Use the sender reference, an enum, or typed events.

Real-world examples

- **UI controllers / view-models** coordinating widgets (MVC — Model–View–Controller — controllers, MVVM — Model–View–ViewModel — view-models, Android `ViewModel` wiring). GoF's own example is a dialog box coordinating its widgets.
- **Chat servers** (Slack, WhatsApp): clients send to the server, which routes to rooms and people and moderates.
- **Air traffic control** — the classic analogy.
- **Workflow / saga orchestrators** (a saga is a multi-step process across services, each step with its own undo; e.g. Temporal, Camunda, AWS Step Functions): a checkout orchestrator invokes payment, stock reservation and shipping in sequence and holds the recovery plan for each failure. Ride-hailing dispatch (matching riders and drivers through a central service) has the same shape.
- **Redux store** at app scale (components dispatch actions instead of calling each other; reducers hold the rules); **MediatR** in .NET popularised the name server-side, dispatching each request to its handler.
- **Message brokers / event buses** (servers delivering messages to the right consumers): services hand messages to the broker instead of calling each other; routing configuration is the mediator logic — and can become a god object too.
- `java.util.Timer` **and** `Executor`/`ExecutorService` are often listed (including by the course) as JDK mediators, standing between submitted tasks and the threads running them. A *loose* fit — they don't coordinate peer colleagues like the GoF dialog — so in an interview lead with UI controllers or orchestrators.

In the interview

> **30-second answer:** "Mediator swaps peer-to-peer references for a single hub. Widgets just report events to it; the hub owns the rules and instructs particular widgets. Four widgets go from six links to four, and a fifth gives five instead of ten. The dependencies aren't gone — they're concentrated in one class that I can read and test. The risk is a god object, so I split mediators per screen area or workflow and push computation into its own classes."

Follow-ups:

- *Mediator vs Observer?* Observer: each listener decides. Mediator: the hub decides and instructs. When the sequence of reactions matters, pick Mediator.
- *Mediator vs Facade?* Facade simplifies a subsystem for outside callers (one-way, subsystem unaware); Mediator coordinates peers that know it (two-way).
- *How does it relate to Command?* A mediator often creates and dispatches commands.
- *How do you test a mediator?* Fake components; assert which calls it made per event.
- *In microservices (a system split into small, separately deployed services)?* Orchestration (a central saga orchestrator) is mediator-style; choreography (services react to each other's events) is observer-style.

Self-check

1. **Q:** How many links do 5 fully connected components need, and how many with a mediator? **A:** 10 vs 5.
2. **Q:** Does a mediator remove coupling? **A:** No, it moves it into one class.
3. **Q:** What is the main failure mode? **A:** The mediator grows into a god object.
4. **Q:** Which way does traffic flow in a Mediator vs an Observer? **A:** Mediator two-way; Observer one-way.
5. **Q:** What should *not* go into a mediator? **A:** Business calculations — only coordination.

TL;DR

Peers talk through **one coordinator** that holds the interaction rules. Links become linear and components reusable, but the coupling **moves** into the mediator — keep it small and split it before it becomes a god object.

10.5 Memento

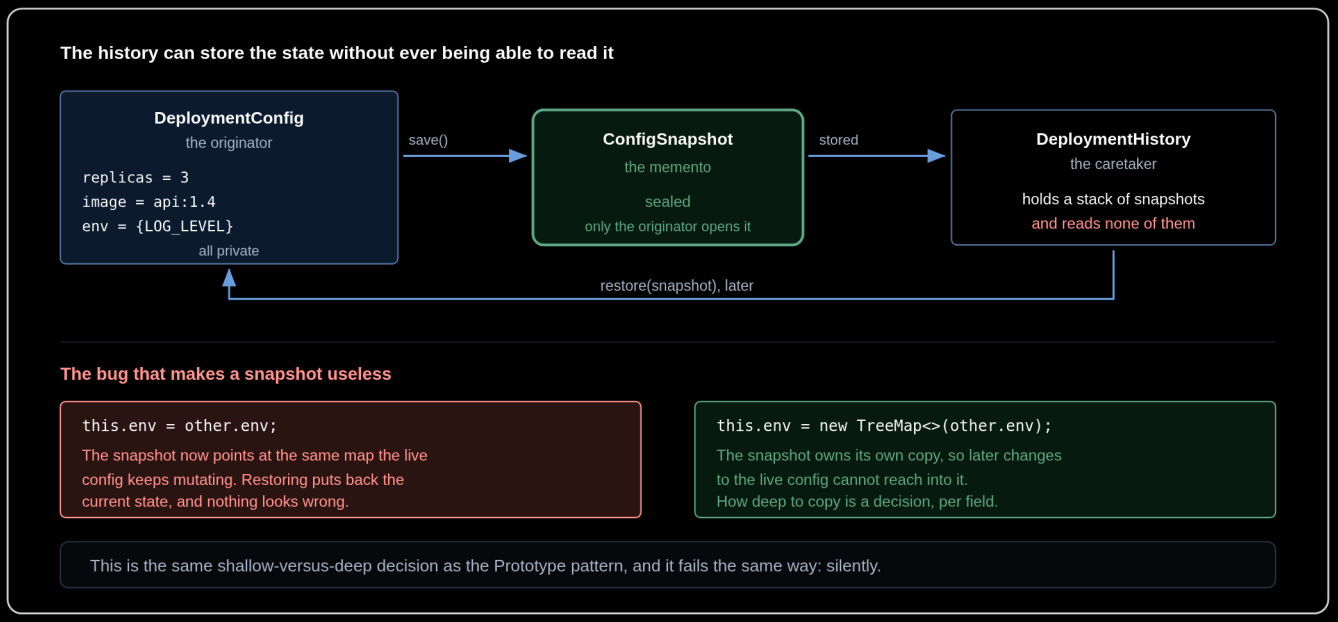


Figure — The originator (DeploymentConfig) saves a sealed snapshot; the caretaker (DeploymentHistory) stores it without being able to read it; below, the shared-map bug that makes a snapshot useless (course diagram)

What it is

Memento lets an object record its internal state in a **snapshot** and later **return** to it — while nobody else can **see** inside the snapshot. Like a **save game**: the menu lists saves by name and date and loads one, never

understanding the file's contents.

GoF intent: *without breaking encapsulation, capture and externalize an object's internal state so the object can be put back into that state later.* Participants: **Memento** (stores the state; narrow interface to the caretaker, wide one to the originator), **Originator** (creates a memento of its state and uses one to restore itself), **Caretaker** (keeps mementos safe but never operates on or examines their contents).

The problem it solves

You need a **rollback** (return to the previous working version): record the deployment config before a risky change, and restore it if the rollout (releasing a change to running servers) misbehaves. Having something *outside* read and store every field breaks two things:

1. **Encapsulation.** The history class must understand replicas (copies of the service running side by side), image names (which packaged build runs) and environment variables (named settings passed to a program). Every new config field changes the history class, and private details are exposed.
2. **Atomicity** (all-or-nothing). A field-by-field restore can stop half-way: three fields written, an exception before the fourth, and the config holds a never-real combination.

With Memento the object **takes its own snapshot** and hands out a sealed token **only it can open**. Restore is **one call into the originator**, which controls it and can make it all-or-nothing (our `restore` validates the token first, then assigns every field; nothing in between can fail).

How it works

Role	Job	In our code
Originator	Owns the state; produces snapshots (`save`) and returns to them (`restore`). No other class can look inside a snapshot.	`DeploymentConfig`
Memento	Carries the state; opaque to everyone else.	`DeploymentConfig.Snapshot` (narrow) / private `State` record (wide)
Caretaker	Decides **when** to save and restore; never looks **inside**.	`DeploymentHistory`

1. Caretaker: `snapshot = config.save(label)`, stored (usually on a stack).
2. Later, caretaker calls `config.restore(snapshot)`; the originator copies the state back in one step.

How opacity is enforced in Java. GoF describes a **narrow interface** (caretaker's view) and a **wide interface** (originator's). We use types: public interface `Snapshot` exposes only `label()`; the data is a **private nested record** `State` only `DeploymentConfig` sees. `restore` checks `instanceof State`, so a forged `Snapshot` (output section 4) is refused. (Strictly, reflection — inspecting/calling private members at run time — or a debugger can still peek; Java access control protects against honest code, not a hostile JVM.) Other options: package-private accessors in Java; the originator as a C++ `friend`; a private nested class in C#; Python and JavaScript have no true enforcement — a leading-underscore convention plus review — though a JavaScript closure *can* hide contents in a way a class field cannot.

Structure (UML)

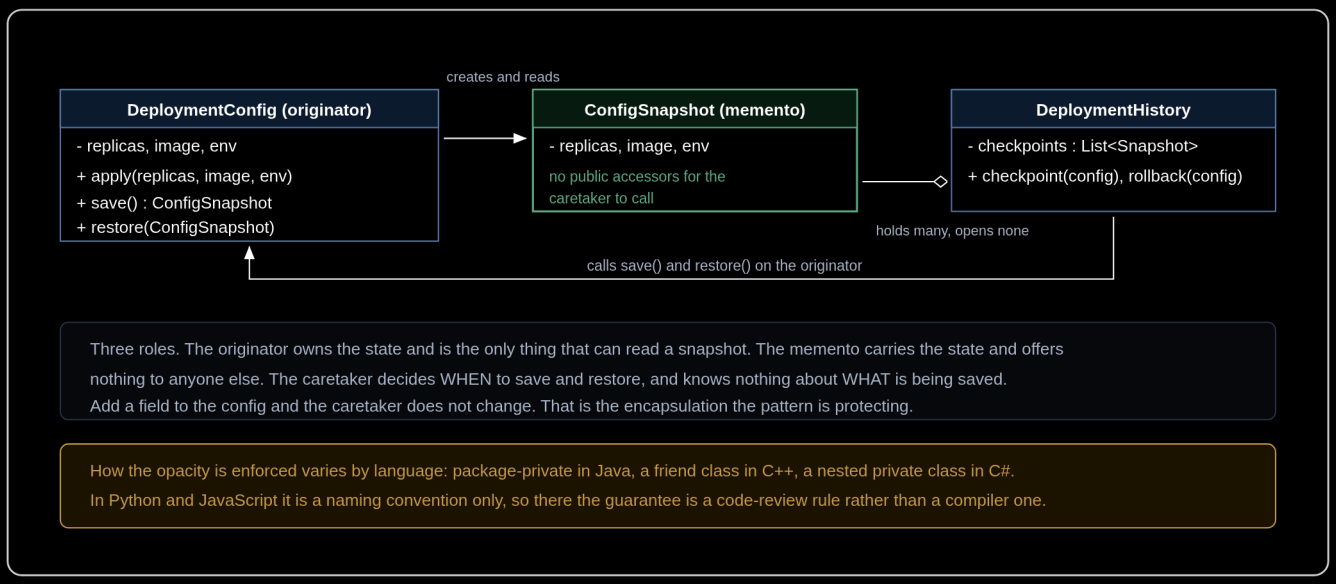
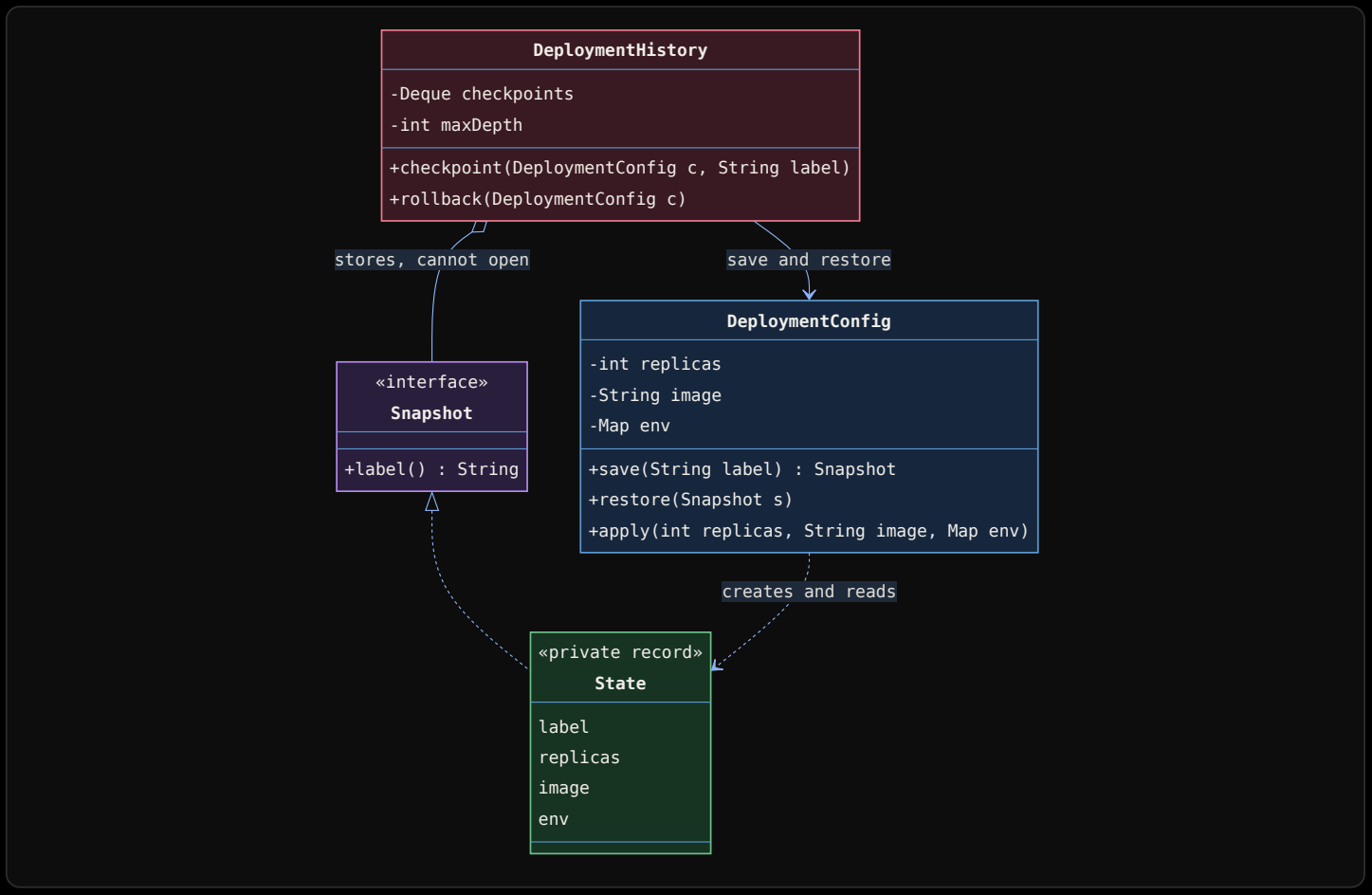


Figure — Class diagram: DeploymentConfig creates and restores ConfigSnapshot; DeploymentHistory holds a list of snapshots (course diagram)

`DeploymentHistory` never refers to a configuration field — the quick check for a real Memento: if the caretaker names the fields, you only relocated them.



Java example

Full code: `code/lld/ch10/memento/`. Notice:

- The `State` record's (immutable data class) compact constructor (no parameter list, runs on every creation) does `env = Map.copyOf(env)`, so the snapshot **owns its copy**. Section 1 changes the live map *after* saving (`FEATURE_X`) and rollback still restores the original.
- `DeploymentHistory` is **bounded** (`maxDepth`), dropping the oldest checkpoint when full.
- Section 3 shows the **shallow-copy bug** in isolation.

```java
// DeploymentConfig.java — the ORIGINATOR: owns the state and is the only class that can read a snapshot.
public final class DeploymentConfig {
    /** The NARROW interface the caretaker sees: a label, nothing else. */
    public interface Snapshot { String label(); }

    /** The WIDE interface: private, so only DeploymentConfig can open it. Immutable record. */
    private record State(String label, int replicas, String image, Map<String, String> env) implements Snapshot {
        State { env = Map.copyOf(env); }   // DEEP enough: the snapshot owns its own copy, not the live map
    }

    private int replicas;
    private String image;
    private final Map<String, String> env = new TreeMap<>();

    public Snapshot save(String label) { return new State(label, replicas, image, env); }

    /** One call restores everything: all-or-nothing. */
    public void restore(Snapshot snapshot) {
        if (!(snapshot instanceof State s)) throw new IllegalArgumentException("not a snapshot produced by
DeploymentConfig");
        apply(s.replicas(), s.image(), s.env());
    }
    // apply(replicas, image, env) assigns all fields; setEnv(key, value) mutates the live map.
}

// DeploymentHistory.java — the CARETAKER: decides WHEN, never WHAT is inside. Bounded to avoid leaks.
public void checkpoint(DeploymentConfig config, String label) {
    if (checkpoints.size() == maxDepth) {
        DeploymentConfig.Snapshot dropped = checkpoints.removeLast();   // oldest
        System.out.println("  [history] full, dropped oldest checkpoint '" + dropped.label() + "'");
    }
    checkpoints.push(config.save(label));
    System.out.println("  [history] checkpoint '" + label + "' taken");
}
public void rollback(DeploymentConfig config) {
    if (checkpoints.isEmpty()) { System.out.println("  [history] nothing to roll back to"); return; }
    DeploymentConfig.Snapshot s = checkpoints.pop();
    config.restore(s);
    System.out.println("  [history] rolled back to '" + s.label() + "'");
}

// Main.java, sections 3-4
Map<String, String> badSnapshot = liveEnv;                  // shares the live map
Map<String, String> goodSnapshot = new HashMap<>(liveEnv);  // owns a copy
config.restore(() -> "forged");                             // a Snapshot not made by the originator
```

Output:

```
=== 1. Checkpoint, bad rollout, rollback ===
  live: replicas=3 image=api:1.4 env={LOG_LEVEL=info}
  [history] checkpoint 'before 2.0' taken
  live: replicas=10 image=api:2.0 env={FEATURE_X=on, LOG_LEVEL=debug}
  [history] rolled back to 'before 2.0'
  live: replicas=3 image=api:1.4 env={LOG_LEVEL=info}   <- snapshot was a real copy

=== 2. Bounded history ===
  [history] checkpoint 'v1' taken
  [history] checkpoint 'v2' taken
  [history] full, dropped oldest checkpoint 'v1'
  [history] checkpoint 'v3' taken
  [history] rolled back to 'v3'
  [history] rolled back to 'v2'
  [history] nothing to roll back to

=== 3. The shallow-copy bug, shown in isolation ===
  bad snapshot  = {LOG_LEVEL=debug}  <- changed with the live object
  good snapshot = {LOG_LEVEL=info}

=== 4. The caretaker cannot open a snapshot ===
  caretaker can read only: label=peek
  forged snapshot refused: not a snapshot produced by DeploymentConfig
```

The bug that makes a snapshot useless

A memento holding a **pointer to the very map (or list) the originator keeps mutating** is no snapshot: the "saved" copy drifts with the live object and a restore re-applies today's state. Nothing throws — until someone notices the rollback did nothing. Always copy mutable parts (`Map.copyOf`, `new TreeMap<>(...)`, deep copies — duplicating the objects inside — of nested objects). Same shallow-vs-deep decision as Prototype (Chapter 8).

Two more things to bound: history depth — cap it and evict the oldest, since an endless list is a memory leak (memory that grows and is never released) — and **state that cannot be captured** (live sockets — open network connections — open files, data owned by another system). A snapshot claiming to cover those fails, usually long after the mistake.

Memento or Command undo?

	Memento	Command undo
Saved per step	A full copy of the state at that moment	The operation plus the little bit of state its reversal needs
How undo happens	Re-install an earlier copy	Execute the opposite operation
Memory cost per step	Proportional to the state size	Proportional to one operation
Correctness risk	Low — the copy was a genuine state	Every reverse operation must be written correctly
Prefer it when	Reversal is tricky or the state is small	Reversal is simple or the state is big

A drawing app snapshotting the whole canvas per brush stroke runs out of memory — use Command ("stroke added, here are its points"). A wizard (multi-step form) whose step 3 changes six interacting fields is the opposite — one snapshot beats six correct inverses. **Real systems combine them:** occasional snapshots, commands logged in between; to go back, restore the nearest snapshot and replay forward. Databases and event-sourced systems (Chapter 11) do exactly this.

Trade-offs

Pros	Cons
Rollback is possible while internals stay private.	Memory rises with every stored snapshot, quickly for large state.
New originator fields need no caretaker change.	Mutable parts must be deep-copied, or snapshots share live data.
The originator controls the restore — all-or-nothing, returning a state that really existed.	Some languages can hide the snapshot only by convention.
Checkpoints, wizards, save games, transactions: one mechanism.	Some state (sockets, external systems) cannot be captured at all.

When not to use it

- **Large state that changes often** — Command undo or periodic snapshots + replay.
- **State you cannot capture** (sockets, file handles, remote systems).
- **The object is immutable** — an old instance already *is* a perfect snapshot; keep a reference (why functional code rarely names this pattern).
- **Each operation has an easy exact inverse** — per-step Command undo costs less memory.

Common mistakes

1. Storing references to mutable internals (the shallow-copy bug).
2. Letting the caretaker read or edit the snapshot's fields (encapsulation lost).
3. Restoring field by field from outside (not atomic).
4. Unbounded history.
5. Snapshotting things that cannot really be restored (connections, locks).

Real-world examples

- **Database transactions** (changes that succeed or fail together): `BEGIN` ... `ROLLBACK`. The DB keeps undo logs; your code never sees them.
- **Version control**: a Git commit snapshots the tree; `git checkout` restores it, and the history (caretaker) needn't understand file contents.
- **Kubernetes Deployments** (Kubernetes runs and manages containers across machines) keep old ReplicaSets (the group of identical running copies per version) as revision history (bounded by `revisionHistoryLimit`, default 10); `kubectl rollout undo` restores a previous pod template — a bounded caretaker like ours.
- **VM / container snapshots** (VM = a whole computer simulated in software; a container is a lighter isolated, packaged process) and infrastructure state files enabling `rollback`.
- **Game saves and checkpoints**; **form wizards** with "back"; **editor undo** for small documents.
- In Java, `Serializable` or a JSON mapper is often *how* a memento is stored when it must outlive the process; a `record` gives an immutable memento in one line.
- **Other languages**: Python — `copy.deepcopy(self.__dict__)` is a quick memento, `dataclasses.replace` for immutable objects; JavaScript — `structuredClone(state)` deep-copies plain data.

In the interview

> **30-second answer:** "With Memento an object packs its state into a token that only the object itself can unpack. The originator produces and consumes snapshots, the memento holds the data, and the caretaker chooses when to save or roll back without ever inspecting the contents. Restoring is a single call the originator controls, so it can be all-or-nothing, and a new field in the object never touches the history code. I'd make sure the snapshot deep-copies mutable parts, bound the history, and compare it with Command undo: Memento stores state, Command stores actions; real systems combine periodic snapshots with a command log."

Follow-ups:

- *How do you enforce that the caretaker can't read the memento in Java?* A narrow public interface (e.g. only a label) implemented by a private nested record; or package-private accessors.
- *Memento vs Prototype?* Both copy state. A prototype clone becomes a live object you keep using; a memento does nothing alone — it exists to be handed back to the originator.
- *Memento vs a backup?* A backup targets disaster recovery (recovering systems after a major failure) and lives outside the process (disk, another region — a separate data-centre location); a memento is usually an in-memory checkpoint for undo. Same idea, very different lifetime and cost.
- *Undo for a large document?* Command undo per edit plus periodic snapshots, so undoing far back doesn't replay thousands of commands.
- *How does it relate to serialization?* Serialization is one *storage format* for a memento; the pattern is about *access* — only the originator may look inside.

Self-check

1. **Q:** Name the three roles. **A:** Originator, memento, caretaker.
2. **Q:** Who may read the memento's contents? **A:** Only the originator.
3. **Q:** Why copy the `env` map in the snapshot? **A:** Otherwise the snapshot shares the live map and changes with it.
4. **Q:** When is Command undo better than Memento? **A:** When state is large and each action has an easy inverse.
5. **Q:** What does Kubernetes' `revisionHistoryLimit` correspond to? **A:** A bounded caretaker history.

TL;DR

The object **snapshots itself**; the caretaker **stores but cannot read**; restore is **one call the originator controls**. **Deep-copy** mutable parts, **bound** the history, and choose Memento (store state) vs Command (store actions) by which is smaller.

10.6 Interpreter

(Not covered by a lesson in the course beyond its one-line entry in the behavioral-patterns table; this section is from the GoF book and standard practice.)

What it is

Interpreter represents the **grammar** of a small language as classes — **one class per grammar rule** — and evaluates a sentence by building a **tree** of those objects and asking the root to `interpret` itself. Like a calculator reading `2 + 3 * 4`: it knows `*` binds tighter, builds `2 + (3 * 4)` and works from the inside out.

- **Grammar** — the rules for valid sentences, e.g. "an expression is terms joined by `+` or `-`".
- **DSL (domain-specific language)** — a small language for one job: search filters, pricing rules, feature-flag conditions, cron schedules (a compact text format for repeating times, like "every weekday at 9").
- **AST (abstract syntax tree)** — the tree form of a sentence: operators as inner nodes, numbers/variables as leaves.
- **Terminal expression** — a leaf rule (a number, a variable).
- **Nonterminal expression** — a rule made of other expressions (addition, multiplication).
- **Context** — outside information evaluation needs, e.g. current variable values.

The problem it solves

Some problems recur as *sentences*: "price * qty − discount", "age > 18 AND country = IN", "every weekday at 9:00". Hard-coding each in Java means a redeploy (rebuilding and releasing the application) per new rule. Letting business users write rules in a tiny language and interpreting them turns rules into **data**. Interpreter is the simplest OO way to evaluate such a language when the grammar is **small**.

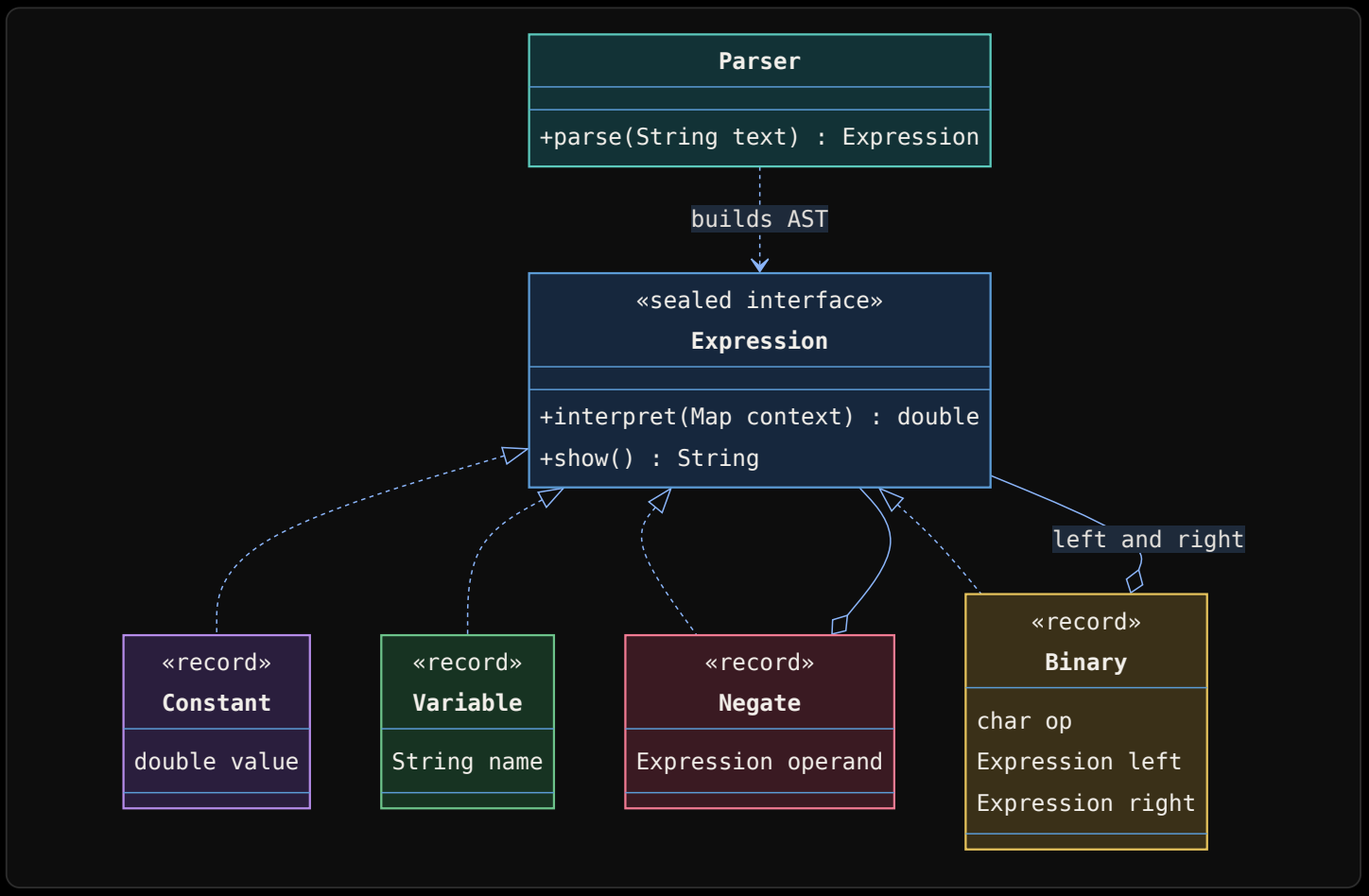
How it works

1. **Write the grammar** (here in EBNF, a standard notation):

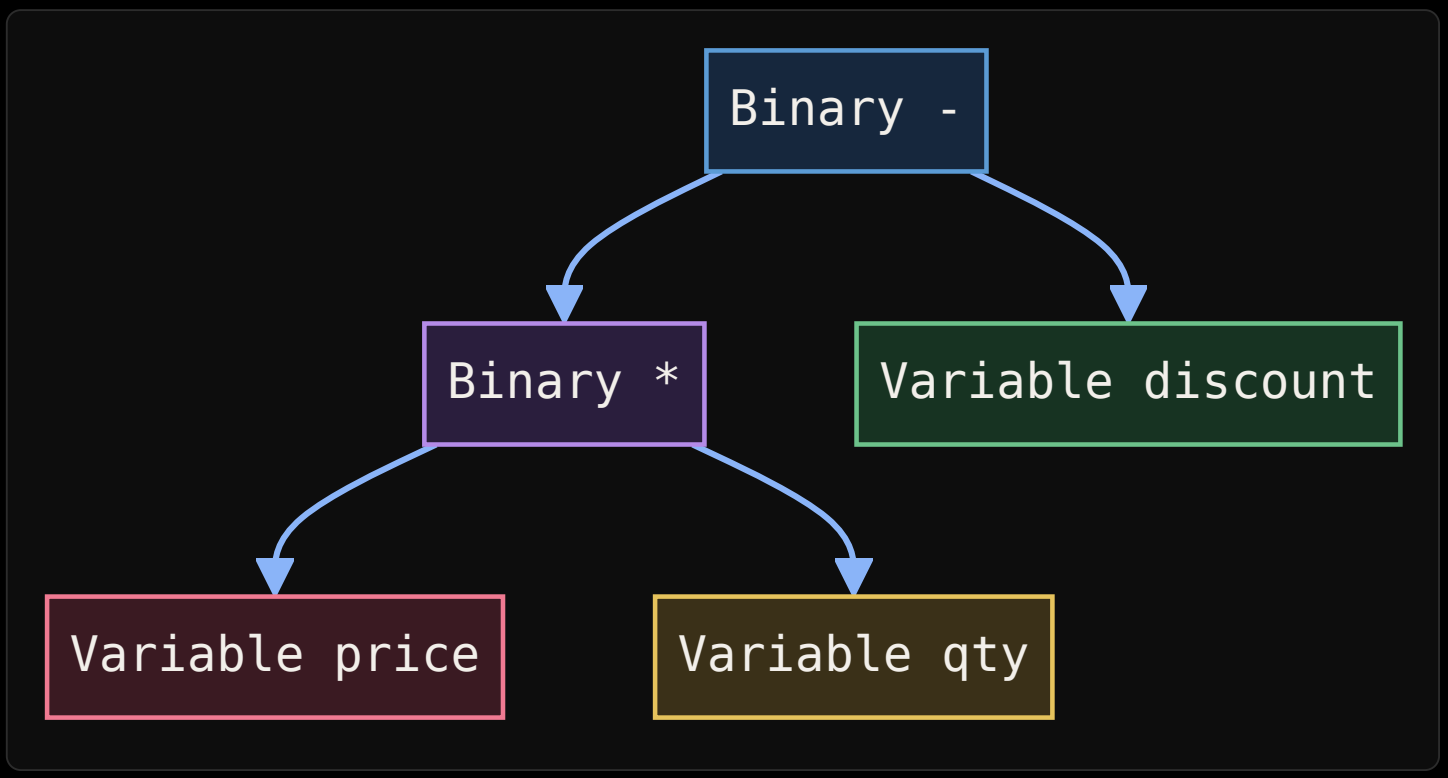
   ```
   expr   := term   (('+' | '-') term)*
   term   := factor (('*' | '/') factor)*
   factor := NUMBER | NAME | '(' expr ')' | '-' factor
   ```

2. **One class per rule**: `Constant` and `Variable` (terminals), `Negate` and `Binary` (nonterminals), all implementing `Expression.interpret(context)`.
3. **Build the AST.** GoF deliberately leaves **parsing** (reading text and building a structure such as the AST) out of the pattern; any parser works. Ours is *recursive-descent*: one method per grammar rule, calling the others.
4. **Interpret**: the root calls `interpret` on its children and combines results (*recursive* evaluation).
5. **Reuse the tree**: parse once, interpret many times with different contexts.

Structure (UML)



The AST for `price * qty - discount`:



The structure is a **Composite** (Chapter 9): leaves and inner nodes share one interface. Interpreter is essentially "Composite + an `interpret` operation".

Java example

`code/lld/ch10/interpreter/`: `Expression` is a **sealed interface** (only the four listed records may implement it), each record one grammar rule; `Parser` is recursive-descent with `expr`, `term`, `factor` mirroring the grammar. Errors (unknown variable, division by zero, bad syntax) get clear messages.

> **Why `double` here?** This is a general arithmetic calculator teaching the pattern; names like `price` are sample data. A real pricing or promotion DSL must use `BigDecimal` (with an explicit `RoundingMode` for division), as the domain code elsewhere in this chapter does — `double` cannot represent values like 0.10 exactly.

```
// Expression.java — AbstractExpression: one class per grammar rule; each knows how to interpret itself.
public sealed interface Expression
        permits Expression.Constant, Expression.Variable, Expression.Negate, Expression.Binary {
    double interpret(Map<String, Double> context);   // evaluate against variable name -> value
    String show();                                   // print the tree as fully-parenthesised text

    /** TerminalExpression: a literal number. */
    record Constant(double value) implements Expression {
        public double interpret(Map<String, Double> ctx) { return value; }
        ...
    }
    /** TerminalExpression: a variable looked up in the context. */
    record Variable(String name) implements Expression {
        public double interpret(Map<String, Double> ctx) {
            Double v = ctx.get(name);
            if (v == null) throw new IllegalArgumentException("unknown variable '" + name + "'");
            return v;
        }
        ...
    }
    /** NonterminalExpression: unary minus. */
    record Negate(Expression operand) implements Expression {
        public double interpret(Map<String, Double> ctx) { return -operand.interpret(ctx); }
        ...
    }
    /** NonterminalExpression: + - * / . Interprets its children, then combines. */
    record Binary(char op, Expression left, Expression right) implements Expression {
        public double interpret(Map<String, Double> ctx) {
            double l = left.interpret(ctx), r = right.interpret(ctx);
            return switch (op) {
                case '+' -> l + r;
                case '-' -> l - r;
                case '*' -> l * r;
                case '/' -> {
                    if (r == 0) throw new ArithmeticException("division by zero in " + show());
                    yield l / r;
                }
                default -> throw new IllegalStateException("unknown operator " + op);
            };
        }
        public String show() { return "(" + left.show() + " " + op + " " + right.show() + ")"; }
    }
}

// Parser.java — recursive descent: one method per grammar rule. Parsing is NOT part of GoF Interpreter.
private Expression expr() {
    Expression left = term();
    while (peek('+') || peek('-')) {
        char op = src.charAt(pos++);
        left = new Expression.Binary(op, left, term());   // left-associative
    }
    return left;
}
// term() is the same over '*' and '/' with factor(); factor() handles '-' factor, '(' expr ')', numbers and names,
// and errors report "... at position N in \"text\"".

// Main.java
Expression ast = Parser.parse(text);          // build the tree once...
double value = ast.interpret(context);        // ...interpret it (could be many times)
```

Output:

```
context: {a=7.0, b=3.0, discount=50.0, price=250.0, qty=3.0}
  price * qty - discount   AST ((price * qty) - discount)      = 700.0
  2 + 3 * 4                AST (2 + (3 * 4))                   = 14.0
  (2 + 3) * 4              AST ((2 + 3) * 4)                   = 20.0
  (a + b) * (a - b)        AST ((a + b) * (a - b))             = 40.0
  -a + 10 / 4              AST ((-a) + (10 / 4))               = -4.5
  price / (qty - 3)        ERROR: division by zero in (price / (qty - 3))
  price * tax              ERROR: unknown variable 'tax'
  2 + * 3                  ERROR: unexpected '*' at position 4 in "2 + * 3"

Same tree, different contexts (parse once, evaluate many):
  qty=1 -> 200.0
  qty=2 -> 450.0
  qty=4 -> 950.0
```

Operator precedence (which operator binds first: `2 + 3 * 4` = 14, `(2 + 3) * 4` = 20) comes from the **grammar**: `term` handles `*` and `/` below `expr`, so they bind tighter.

Trade-offs

Pros	Cons
Easy to change and extend the grammar: a new rule is a new class.	One class per rule: large grammars become a jungle of classes.
Each rule's evaluation is small and testable.	Tree-walking is slow compared with compiling to bytecode (compiled JVM instructions).
Rules become data (stored in a DB, edited by users).	Adding a new *operation* (pretty-print, type-check, optimise) means touching every class — that is where **Visitor** (Chapter 11) comes in.
Parse once, evaluate many times.	You still need a parser, which the pattern does not give you.

When not to use it

- **The grammar is large or complex** (a real programming language, SQL) — use a parser generator (writes a parser from a grammar file) such as **ANTLR** and a compiler/visitor design.
- **Performance matters** with millions of evaluations — compile to bytecode or use an existing engine.
- **An existing language fits** — regular expressions (text-matching patterns), SpEL (Spring's expression language), JSONPath (queries into JSON), CEL (Google's Common Expression Language), or a scripting engine (an embedded interpreter for a full language such as JavaScript).
- **Security matters.** Never let users run a *general-purpose* script engine on your server; a small restricted DSL is safer precisely because it can do only what its grammar allows.

Common mistakes

1. **Mixing parsing and evaluation** in one pass, re-parsing on every evaluation.
2. **Ignoring precedence and associativity** (associativity groups equal-precedence operators: `8 - 3 - 2` must be `(8 - 3) - 2`; our `expr` loop builds left-associative trees).
3. **Poor error messages** — always report the position.
4. **Adding many operations to every node** instead of Visitor or pattern matching (a `switch` branching on an object's type and unpacking its fields).
5. **Growing a "tiny" DSL** into an accidental programming language.

Real-world examples

- `java.util.regex.Pattern`: compiling a regex builds an internal tree of nodes that a `Matcher` walks — a classic interpreter.
- **Spring Expression Language (SpEL)**: `SpelExpressionParser` parses `#user.age > 18` into an AST of `SpelNode`s; `getValue()` evaluates it against a context. Jakarta Expression Language (EL) is similar.
- **SQL engines** evaluate `WHERE` clauses as expression trees; **spreadsheet formulas** (`=SUM(A1:A3)*2`) are a DSL.
- **Rule and pricing engines**: e-commerce promotions ("10% off if cart > 999 and category = electronics"), feature-flag targeting rules, and Google's **CEL** in Kubernetes validation rules and IAM (identity and access management — who may do what) conditions.
- **Cron expressions** (`0 9 * * MON-FRI`), e.g. Quartz's (a Java job scheduler) `CronExpression`.

In the interview

> **30-second answer:** "Interpreter models a small language's grammar as classes, one per rule — terminals like numbers and variables, nonterminals like addition. A sentence is parsed into an AST made of those objects and evaluated by calling `interpret` on the root with a context. It's really Composite plus an evaluate operation. It's great for small DSLs like pricing rules or filters; for big grammars I'd use a parser generator like ANTLR, and when I need many operations over the tree I'd switch to Visitor or sealed types with pattern matching."

Follow-ups:

- *Is parsing part of the pattern?* No. GoF assumes the tree exists; parsing is separate (hand-written recursive descent, or ANTLR).
- *Interpreter vs Visitor?* Interpreter puts evaluation **inside** node classes; Visitor takes operations **out** into separate classes. A real interpreter usually starts as the first and becomes the second when a second pass (type-check, optimise, pretty-print) arrives.
- *Interpreter vs Composite?* Composite is the structure; Interpreter uses it to evaluate a language.
- *How would you make it faster?* Cache parsed trees; constant-fold (pre-compute variable-free parts); or compile to bytecode/lambdas once.
- *How do you design a rule engine for promotions?* A small grammar (comparisons, AND/OR/NOT, cart fields), rules stored as text, parsed once into cached trees, evaluated per cart.

Self-check

1. **Q:** What is the difference between terminal and nonterminal expressions? **A:** Terminals are leaves (numbers, variables); nonterminals combine other expressions (operators).
2. **Q:** What is the "context"? **A:** External information needed for evaluation, such as variable values.
3. **Q:** Why does `2 + 3 * 4` give 14? **A:** The grammar puts `*` in `term`, below `expr`, so it binds tighter.
4. **Q:** Name a JDK class that is an interpreter. **A:** `java.util.regex.Pattern`.
5. **Q:** When should you not use it? **A:** Large grammars, performance-critical evaluation, or when an existing language fits.

TL;DR

One class per grammar rule; parse a sentence into an **AST**; evaluate by calling `interpret` recursively with a **context**. Perfect for **tiny DSLs**; for big grammars use a parser generator and Visitor.

10.7 Chapter recap: the six patterns side by side

Pattern	The object you create	The question it answers	Classic JDK / real example	Often confused with
Chain of Responsibility	A handler per check	*Who* should handle this request?	Servlet filters, logging handlers	Decorator (all layers run)
Command	An object per request	How do I store, queue or undo *this request*?	`Runnable`, `UndoManager`	Strategy (how vs what)
Iterator	A cursor object	How do I walk this without knowing its shape?	`Iterator`, `Spliterator`, `ResultSet`	Visitor (what to do at each node)
Mediator	One coordinator	How do many objects cooperate without knowing each other?	UI controllers, chat servers, saga orchestrators	Observer (broadcast only), Facade
Memento	A sealed snapshot	How do I go back to an earlier state safely?	DB rollback, Git, `kubectl rollout undo`	Command undo, Prototype
Interpreter	A node per grammar rule	How do I evaluate sentences of a small language?	`Pattern`, SpEL	Composite, Visitor

Interview combo facts to remember:

- The request flowing through a **Chain of Responsibility** is frequently a **Command** object.

- A **Mediator** often creates and dispatches **Commands**.
- **Memento + Command** together give scalable undo (snapshots + replay).
- **Iterator + Composite** walk trees; **Interpreter = Composite + evaluate**; when operations multiply, move to **Visitor** (Chapter 11).

Self-check (whole chapter)

1. **Q:** Which two GoF behavioral patterns are class-scoped (inheritance-based)? **A:** Template Method and Interpreter.
2. **Q:** A request must pass auth, then a rate limit, then validation, and any step can reject it. Which pattern? **A:** Chain of Responsibility.
3. **Q:** You need undo/redo for a text editor. Which patterns? **A:** Command (per edit), optionally Memento for periodic snapshots.
4. **Q:** Five form widgets depend on each other's values. Which pattern? **A:** Mediator.
5. **Q:** Your team needs business users to write discount rules. Which pattern? **A:** Interpreter (a small DSL), possibly with Specification (Chapter 11).

TL;DR

Behavioral patterns turn **hidden control flow** into **named objects**: handlers (CoR), requests (Command), cursors (Iterator), coordinators (Mediator), snapshots (Memento) and grammar nodes (Interpreter). Each one buys flexibility with indirection — use it when the flexibility is a real requirement, not "just in case".

Chapter 11 — Behavioral Design Patterns (Part 2) & Beyond GoF

> **What you will learn.** The remaining five **GoF** ("Gang of Four" — the four authors of the 1994 *Design Patterns*, which catalogued 23 classic patterns) behavioral patterns — **Observer, State, Strategy, Template Method, Visitor** — including the three that share one class diagram (Strategy, State, Bridge) and how to tell them apart. Then six widely used non-GoF patterns that come up constantly in interviews and real code: **Null Object, Dependency Injection, Repository, Specification, MVC** (the Head First "compound pattern") and **Event Sourcing (lite)**. Then a "which behavioral pattern do I need?" table, and finally the **architectural patterns** that shape whole systems (layered, client-server, modular monolith vs microservices, event-driven, pipe-and-filter, hexagonal/clean) and how they differ from design patterns.

How to run the code in this chapter (listings show the essential lines; full files are in `code/`)

```
cd chapters/11-patterns-behavioral-2/code
javac --release 21 -Xlint:all -d /tmp/lld_11 $(find . -name '*.java')   # compiles with 0 warnings
java -cp /tmp/lld_11 lld.ch11.observer.Main
# also: state, strategy, templatemethod, visitor, nullobject, di, repository, specification, mvc, eventsourcing, architecture
```

Chapter 10 introduced behavioral patterns — **how objects share work and talk**. The idea to keep in mind here: most of these patterns turn a conditional (`if` / `switch`) or hard-coded call into an **object** that can be swapped, subscribed, or visited.

11.1 Observer

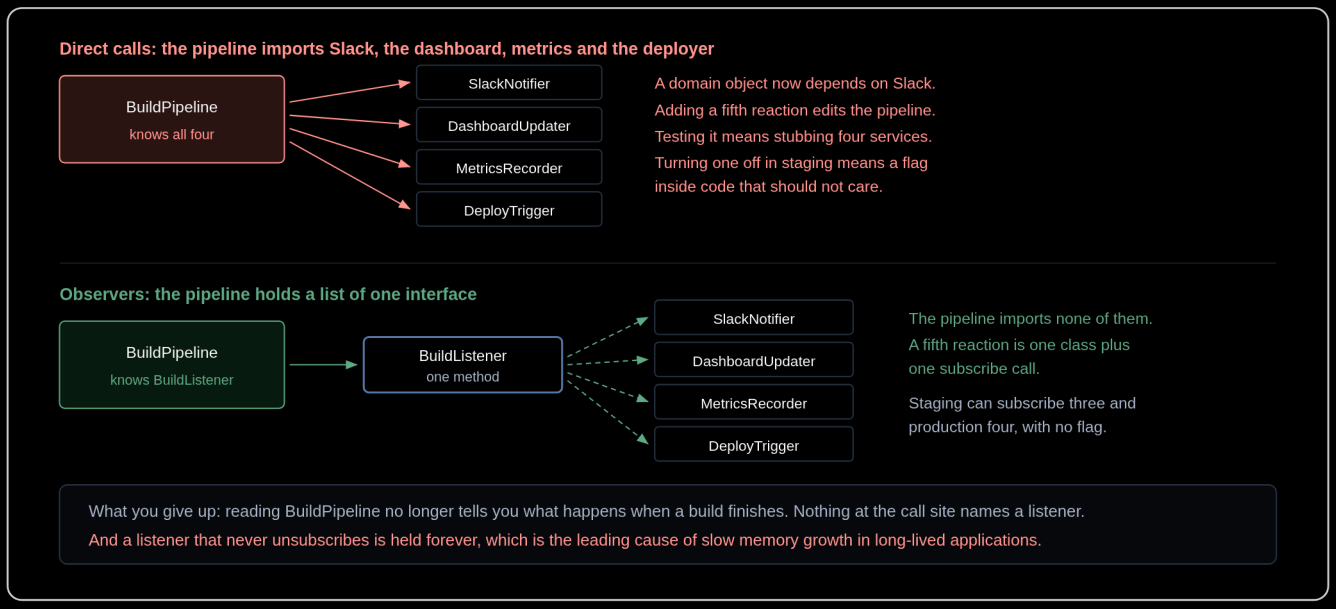


Figure — Top: the build pipeline calls Slack, dashboard, metrics and deployer directly. Bottom: it holds one list of BuildListener and imports none of them (course diagram)

What it is

Observer lets one object (the **subject**, also *publisher* or *observable*) notify many dependents (the **observers**, *subscribers* or *listeners*) when something happens — **without knowing who they are or what they do**. Like a YouTube channel: subscribers are notified of a new video, the channel doesn't care what they do with it, and anyone can unsubscribe.

GoF intent (paraphrased): a one-to-many dependency, so that when one object's state changes, every dependent is told and can update itself automatically.

GoF participants:

Participant	Role	In `lld.ch11.observer`
Subject	Knows its observers (any number) and offers attach/detach operations	`BuildPipeline` (`addListener` + the returned unsubscribe handle)
Observer	The update interface that subjects call	`BuildListener`
ConcreteSubject	Holds the state observers care about; notifies them when it changes	`BuildPipeline` (GoF split Subject into an abstract class and a concrete one; in modern Java one class usually plays both)
ConcreteObserver	Implements the update interface; may keep its own state in step with the subject	`SlackNotifier`, `DashboardUpdater`, `MetricsRecorder`, `DeployTrigger`

The problem it solves

Four reactions must follow every finished build: Slack message, dashboard refresh, metrics record, deployment. Naively:

```
void finish(int buildNumber, String result) {
    slack.post(buildNumber, result);
    dashboard.update(result);
    metrics.record(buildNumber);
    deployer.start();
}
```

The pipeline now **depends on a chat service**; a fifth reaction edits the pipeline; a **unit test** (an automated test of one class on its own) must fake four services; and disabling deploys in **staging** (the pre-production copy used for final checks) needs an environment flag in a class that should know nothing of environments.

Observer replaces four references with **one list of one interface**. **The source only announces; every listener chooses its own reaction** — nobody orchestrates, and listeners don't know each other.

How it works

1. An **observer interface** with one method, e.g. `onBuildFinished(buildNumber, result)`.
2. The **subject** keeps a list and offers subscribe/unsubscribe (classically `addListener` / `removeListener`; our `addListener` returns an unsubscribe handle).
3. On a change the subject **first finishes changing its state**, then calls each observer.
4. Observers are added and removed at run time (e.g. staging subscribes three listeners, production four).

Push vs pull — two ways to deliver the data:

	Push model	Pull model
What the observer receives	All the data about the change (e.g. a `Reading` record)	Just "I changed" plus a reference to the subject
How the observer gets data	It is in the call	It calls getters on the subject for what it needs
Pros	Simple; observers don't depend on the subject's getters	Each observer takes only what it needs; the interface doesn't change when new data is added
Cons	Sends data some observers don't need; adding data changes the interface	Observers depend on the subject; they must work out *what* changed

`java.util.Observer.update(Observable o, Object arg)` supported both (`o` for pull, `arg` for push); *Head First*'s weather station shows the same trade-off.

Structure (UML)

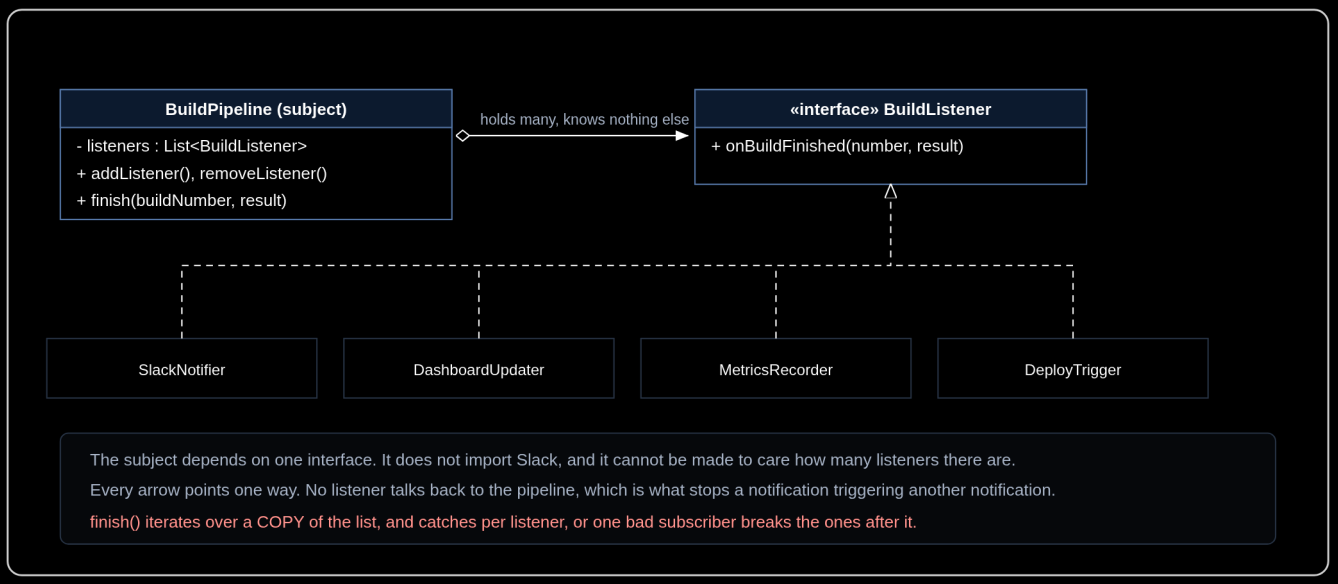
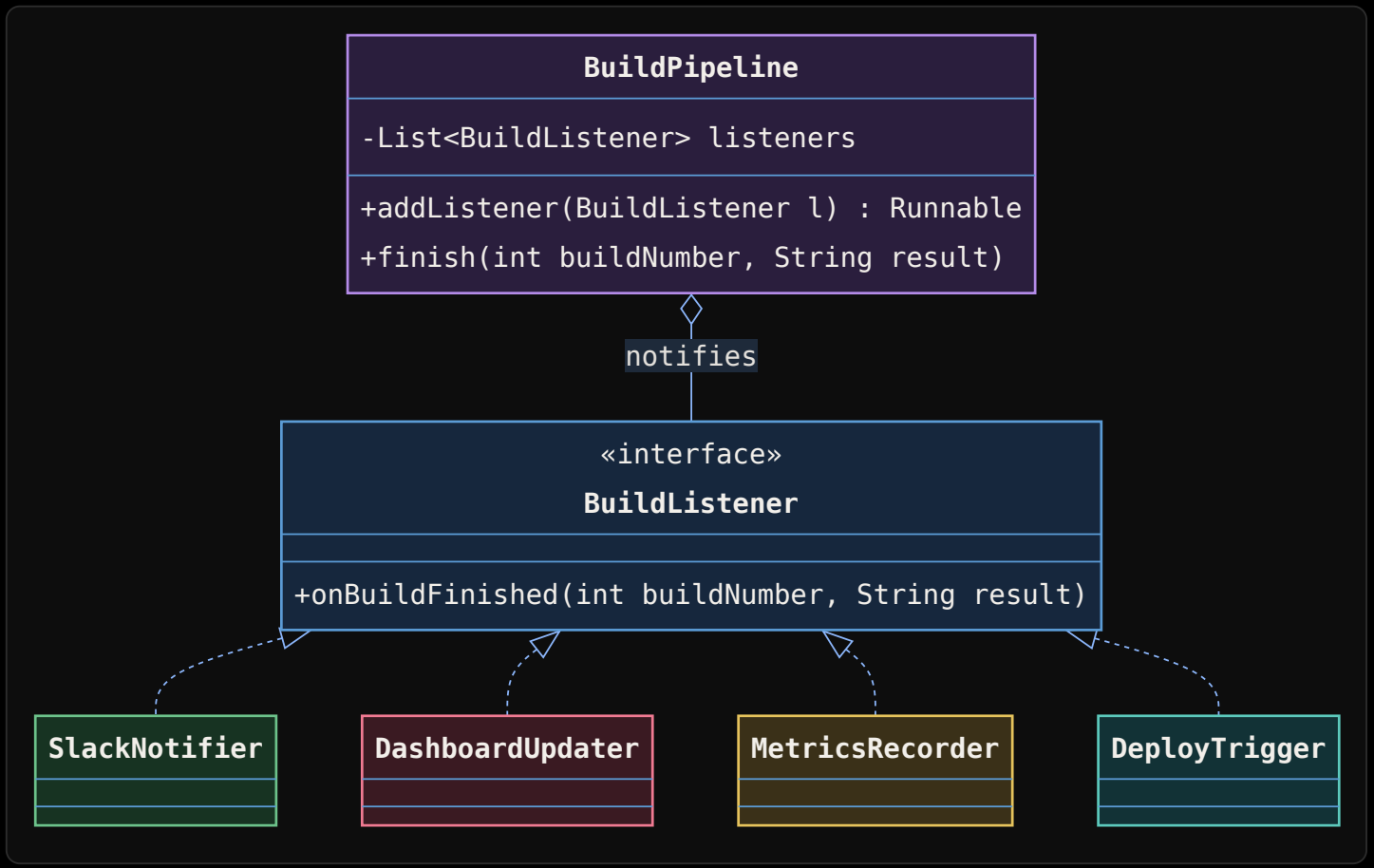


Figure — Class diagram: BuildPipeline holds a list of BuildListener; SlackNotifier, DashboardUpdater, MetricsRecorder and DeployTrigger implement it (course diagram)

In this **UML** class diagram (Unified Modeling Language — the standard box-and-arrow notation for classes and relations) all dependencies run one way: listeners never call the pipeline back, so one notification cannot trigger another.



Java example

`code/lld/ch11/observer/` shows four things:

1. **Build pipeline.** Listeners live in a `CopyOnWriteArrayList` (a JDK list copying its internal array on every write, so readers never see it change), whose iterator uses a **snapshot** — a listener unsubscribing mid-notification can't break the loop, and the list is **thread-safe** (safe across *threads* — independent paths of execution running at once). `finish` **catches exceptions per listener**, so a broken dashboard doesn't stop

metrics. `addListener` returns a `Runnable` (Java's "code to run later" interface, one method `run()`) that unsubscribes: whoever subscribes owns the unsubscribe.

2. **Push vs pull** (`WeatherStation`): one display receives the whole `Reading`; the other gets the station and pulls only the temperature.
3. **A tiny event bus** (`EventBus`): subscribers register for an event *type* (a class); publishers just publish objects. The returned `Subscription` is the unsubscribe handle, closed in a `finally` so the refunds handler is removed even if publishing throws. (`Subscription` is also `AutoCloseable` — `close()` runs automatically when a *try-with-resources* block ends — so `try (var s = bus.subscribe(...))` works, but if the body never uses `s`, `javac -Xlint` warns the resource is unreferenced; Java 22+ allows `try (var _ = ...)`; on Java 21 `try`/`finally` is the warning-free form.) The handler list is a `CopyOnWriteArrayList`, so subscribing and publishing from different threads is safe. Delivery is still **synchronous on the publisher's thread** (it waits while each handler runs in turn) and nothing is stored, so it is not a **queue** (a waiting line storing items and handing them out later, in order).
4. `java.beans.PropertyChangeSupport` (price in whole cents as an `int` — exact, never `double`), which the `Observable` **Javadoc** (official API docs generated from source comments) itself points to (the `java.beans` package) as the richer event model. It does **not** fire when old and new values are both non-null and equal. (Since Java 9 `java.beans` is in the `java.desktop` module; on the class path — the folders and JARs Java searches for classes — nothing extra is needed, but a modular app, one declaring dependencies in `module-info.java`, must `requires java.desktop`.)

```
// BuildListener.java — the OBSERVER interface: one method, so a lambda can implement it.
@FunctionalInterface
public interface BuildListener { void onBuildFinished(int buildNumber, String result); }

// BuildPipeline.java — the SUBJECT: a list of one interface; imports none of the concrete reactions.
public final class BuildPipeline {
    // CopyOnWriteArrayList iterates over a snapshot: a listener that unsubscribes
    // itself mid-notification cannot break the loop, and it is safe across threads.
    private final List<BuildListener> listeners = new CopyOnWriteArrayList<>();

    /** Returns a handle: whoever subscribes owns the matching unsubscribe (prevents leaks). */
    public Runnable addListener(BuildListener listener) {
        listeners.add(listener);
        return () -> listeners.remove(listener);
    }

    public void finish(int buildNumber, String result) {
        System.out.println("Build " + buildNumber + " finished: " + result);
        for (BuildListener l : listeners) {
            try {
                l.onBuildFinished(buildNumber, result);
            } catch (RuntimeException e) {          // one bad listener must not starve the rest
                System.out.println("  [pipeline] a listener failed: " + e.getMessage());
            }
        }
    }
}
// SlackNotifier, MetricsRecorder print; DashboardUpdater throws when offline; DeployTrigger deploys only on "success".

// WeatherStation.measure — change state FIRST, then notify both styles.
public void measure(Reading r) {
    current = r;
    pushObservers.forEach(o -> o.accept(r));      // PUSH: the data travels with the call
    pullObservers.forEach(o -> o.accept(this));   // PULL: "I changed" + a reference; observers call temperature()
}
```

```java
// EventBus.java — publishers and subscribers share only event TYPES. Synchronous, not durable (unlike Kafka).
private record Handler<E>(Class<E> type, Consumer<? super E> action) {
    void deliver(Object event) { if (type.isInstance(event)) action.accept(type.cast(event)); }
}
public interface Subscription extends AutoCloseable { @Override void close(); }
private final List<Handler<?>> handlers = new CopyOnWriteArrayList<>();

public <E> Subscription subscribe(Class<E> type, Consumer<? super E> action) {
    Handler<E> h = new Handler<>(type, action);
    handlers.add(h);
    return () -> handlers.remove(h);
}
public void publish(Object event) { handlers.forEach(h -> h.deliver(event)); }

// PricedItem.java — the JDK's recommended replacement for java.util.Observable.
public void setPriceCents(int newPrice) {
    int old = this.priceCents;
    this.priceCents = newPrice;
    support.firePropertyChange("priceCents", old, newPrice);   // no event if old == new
}

// Main.java (excerpt)
Runnable unsubscribeDeploy = pipeline.addListener(new DeployTrigger());
pipeline.finish(101, "success");        // every listener runs
unsubscribeDeploy.run();                // e.g. staging has no auto-deploy
EventBus.Subscription refunds = bus.subscribe(OrderCancelled.class, e -> ...);
try { bus.publish(new OrderPlaced("A-1", new BigDecimal("499.00"))); bus.publish(new OrderCancelled("A-1")); }
finally { refunds.close(); }            // refunds handler removed here, even on error
```

Output:

```
=== 1. Build pipeline: subscribe, unsubscribe, failing listener ===
Build 101 finished: success
  [slack] #builds: build 101 success
  [dashboard] status tile -> success
  [metrics] recorded build 101 (total 1)
  [deploy] deploying build 101
Build 102 finished: success
  [slack] #builds: build 102 success
  [dashboard] status tile -> success
  [metrics] recorded build 102 (total 2)
Build 103 finished: failed
  [slack] #builds: build 103 failed
  [pipeline] a listener failed: dashboard is offline
  [metrics] recorded build 103 (total 3)

=== 2. Push vs pull ===
  [push display] got everything: Reading[tempC=31.5, humidity=0.62, pressureHpa=1008.0]
  [pull display] asked only for temp: 31.5 C

=== 3. Event bus (typed topics, scoped unsubscribe) ===
  [email] confirmation for A-1
  [audit] OrderPlaced[orderId=A-1, amount=499.00]
  [audit] OrderCancelled[orderId=A-1]
  [refunds] start refund for A-1
  [audit] OrderCancelled[orderId=A-2]

=== 4. JDK: PropertyChangeSupport (replacement for deprecated Observable) ===
  [price watcher] phone priceCents: 49900 -> 44900
  [price watcher] phone priceCents: 44900 -> 39900
```

Why `java.util.Observer` / `Observable` were deprecated

Since **Java 9**, `java.util.Observer` and `java.util.Observable` are **deprecated** (officially "don't use in new code; may be removed") but not removed. The Javadoc's reasons: the event model is **quite limited**, the **order of notifications is unspecified**, and **state changes are not one-for-one with notifications**. Also:

- `Observable` is a **class**, not an interface — your subject must *extend* it, using up Java's **single inheritance** (one parent class only).
- `setChanged()` is **protected**, so you can't use `Observable` by composition (as a field) without subclassing.
- `update(Observable, Object)` is untyped (`Object arg`), so observers must cast.

- Thread safety is partial: methods are `synchronized` (one thread at a time, each call first taking the object's **lock** — a token only one thread can hold), but `notifyObservers` calls observers outside the lock on a snapshot, so (as a comment in the **JDK** — Java Development Kit: compiler, runtime, standard library — source admits) a newly added observer may miss a notification and a just-removed one may still get it.

Instead (the Javadoc names the first, third and fourth): `java.beans.PropertyChangeSupport` / `PropertyChangeListener` for bean properties; your own typed listener interface (as `BuildPipeline`); `java.util.concurrent` queues for reliable, ordered hand-off between threads; the `java.util.concurrent.Flow` API (Java 9) for **reactive streams** (asynchronous streams of items from a producer to consumers) with **back-pressure** (the consumer tells the producer to slow down when it can't keep up).

Trade-offs

Pros	Cons
The source no longer depends on its reactions (the pipeline no longer imports Slack).	Harder to trace: the call site gives no hint which code will run.
Reactions added/removed at run time; environments differ without flags.	**Memory leaks** (memory never freed) if an observer is never removed (the subject holds it forever).
Each reaction is testable alone.	Stack traces start inside a loop over a list; debugging is harder.
A new reaction = one class + one subscribe call (Open/Closed).	**No ordering or delivery guarantee**; a crash loses reactions not yet called.
	With synchronous observers, the subject takes as long as all listeners combined.

Observer vs Mediator, Chain of Responsibility, callbacks

Looks like	The actual difference
Mediator	Observer broadcasts one way and each observer decides; a mediator **decides what happens next**, calls components back (two-way) and owns the ordering — full table in §10.4.
Publish-subscribe with a broker (a server that stores and forwards messages)	Same idea, but the broker contributes persistence, redelivery, ordering guarantees and a network hop — none of which in-process observers offer.
Chain of Responsibility	Observer informs **all** listeners, and no listener can prevent the rest from running. A chain ends as soon as one handler accepts the request.
A callback	One callback is one observer. It becomes the Observer pattern when the number of listeners varies and they subscribe and unsubscribe at run time.

Observer outside Java. With **first-class functions** (functions stored and passed like values) a listener is a **lambda** (a short anonymous function, `x -> ...` in Java) — keep the handle or you can't unsubscribe. JavaScript has `EventTarget` / `EventEmitter`, and `AbortController` unsubscribes by signal. Python often uses a list of callables; `weakref.WeakSet` avoids the leak. C# has `event`, `IObservable<T>` and `IDisposable` subscriptions. Everywhere, **weak references** (which don't keep their target alive, so the garbage collector may free it) fix the leak, at the cost of a listener vanishing when nothing else holds it.

When not to use it

- **One listener, forever** — call it directly.
- **You need ordering or guaranteed delivery** — a queue or message broker (Kafka, RabbitMQ, SQS): same idea plus persistence, retries and a network — very different failure modes.
- **Reactions are slow** — notify **asynchronously** (hand work to another thread and return at once, e.g. via an *executor* — a pool of worker threads — or a queue) or don't use in-process observers.
- **The whole flow must be auditable/traceable** — explicit calls are kinder to readers.

Common mistakes (the four classic ways observers go wrong, plus one)

1. **No unsubscribe → memory leak.** The subject's **strong reference** (an ordinary reference keeping the object alive) holds the listener and everything it points to — a top cause of slow memory growth in long-running

apps. Every `add` needs an owner for its `remove` (or weak references, with listeners possibly vanishing).
2. **A throwing listener kills the loop** — without per-listener try/catch, listener 3 fails and 4 and 5 never run.
3. **Changing the list during notification** → `ConcurrentModificationException` (thrown when a collection changes while being looped over), or silently skipped listeners in some languages. Iterate a copy, or use `CopyOnWriteArrayList`.
4. **Listeners calling back into the subject** → cycles or endless recursion. Finish updating state before notifying; never let a notification synchronously cause another.
5. **Notifying before the state is updated**, so pull-style observers read stale data.
6. **Relying on notification order** — it is the order of subscription, a side effect of start-up code nobody decided. If "update the dashboard, then decide whether to deploy" matters, that is coordination — a Mediator.

Real-world examples

- **Every UI event handler**: Swing `ActionListener`, Android `OnClickListener`, DOM `addEventListener`.
- **JDK**: `PropertyChangeListener`, `java.util.concurrent.Flow` (`Publisher`/`Subscriber`), servlet listeners such as `HttpSessionListener`.
- **Spring** (the most widely used Java application framework): `ApplicationEventPublisher` + `@EventListener` (in-process events, optionally `@Async` or `@TransactionalEventListener`).
- **Guava** `EventBus`, **RxJava / Project Reactor / RxJS**, Kotlin `Flow`, front-end signals — observers plus composable operators (map, filter, debounce — wait until events pause, then pass on only the latest).
- **Model–view separation**: a view registers with its model, which updates freely without knowing any screen is attached (MVC in §11.7.5).
- **Distributed publish–subscribe**: Kafka topics, Google Pub/Sub, AWS SNS. An order service publishes one "order placed" event and email, inventory, analytics and recommendation services each consume it — Observer at system scale, plus durability.
- **Database triggers and change data capture** (CDC: streaming every row change from a database's internal log; Debezium is popular) — observers at the storage layer.

In the interview

> **30-second answer:** "Observer swaps a handful of hard-coded calls for a list of objects that share one listener interface. The subject just says "this happened" and every observer picks its own response, so each reaction can be plugged in, unplugged and unit-tested on its own, and the subject no longer depends on any of them. In production I iterate over a copy of the listener list, catch exceptions per listener, and make sure every subscribe has a matching unsubscribe to avoid leaks. When I need durability, retries or ordering I use a message broker instead."

Follow-ups:

- *Push or pull? Why was `Observable` deprecated? Observer vs Mediator?* See the push/pull table, the deprecation section and the comparison table above (Mediator in depth: §10.4) — when the sequence of reactions matters, pick a mediator.
- *Observer vs Pub/Sub?* In Observer the subject holds its observers directly; in publish–subscribe a **broker/event bus** sits between, so publishers and subscribers don't know each other; brokers add durability, retries and ordering.
- *How do you make it thread-safe?* `CopyOnWriteArrayList` for listeners (reads far outnumber writes), no locks held while calling listeners, an executor for slow listeners.
- *How do you avoid leaks?* Return a subscription handle tied to a lifecycle (try-with-resources, `onDestroy`, `@PreDestroy`), or use weak references.

Self-check

1. **Q:** What does the subject know about its observers? **A:** Only that they implement the observer interface.
2. **Q:** Why iterate over a copy of the listener list? **A:** So a listener that (un)subscribes during notification can't break the loop.
3. **Q:** What is the classic Observer memory leak? **A:** Listeners that are never removed stay strongly referenced by the subject.
4. **Q:** What should replace `java.util.Observable`? **A:** `PropertyChangeSupport`, a custom typed listener interface, or the `Flow` API.

5. **Q:** When is a message broker better than in-process observers? **A:** When you need durability, retries, ordering, or slow/remote consumers.

TL;DR

One subject, a list of listeners, one-way notification. **Iterate over a copy, catch per listener, always unsubscribe.** Push vs pull is a trade-off; `Observable` is deprecated; use a broker when you need guarantees.

11.2 State

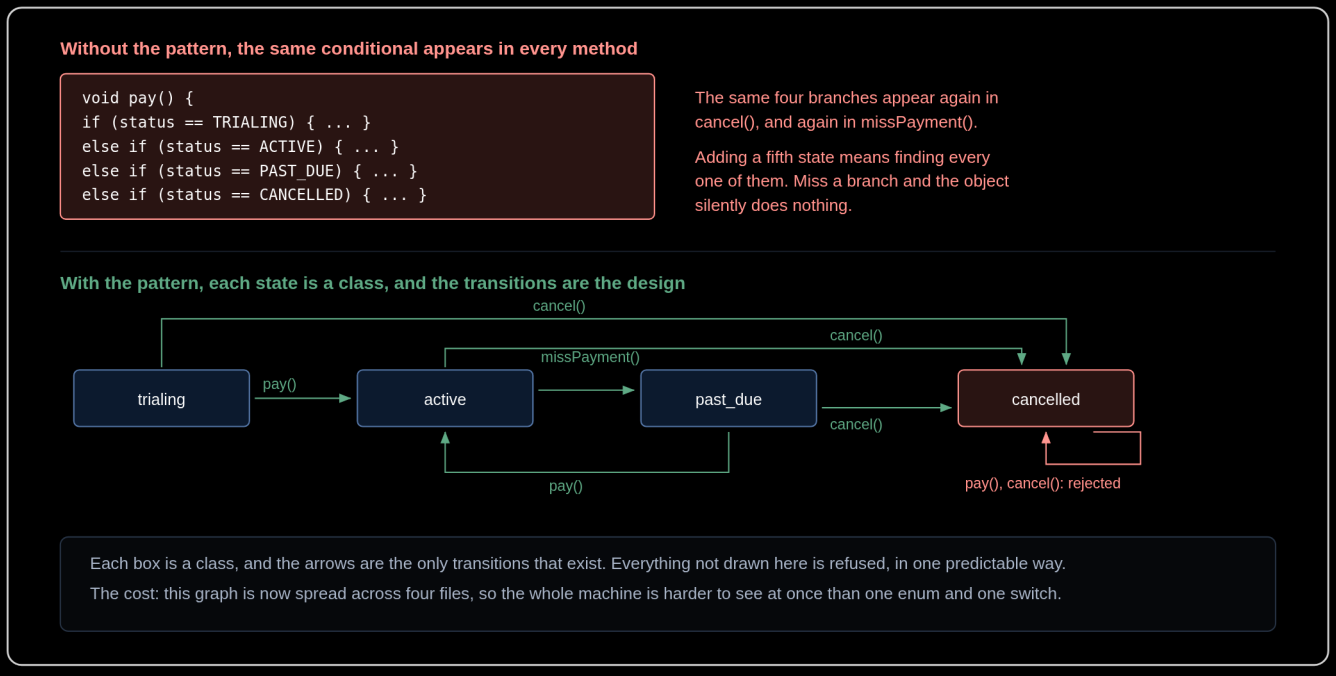


Figure — Top: the same four-branch status check repeated in every method. Bottom: each state is a class and the transitions form a graph (course diagram)

What it is

State (GoF intent, paraphrased): when an object's internal condition changes, **its behaviour changes with it**, so from outside it looks as if the object switched class. Instead of every method testing a status field, the object hands each action to a **state object**, and replaces that object when the condition changes — like a phone's ringer switch: "call received" does different things in *ring*, *vibrate* and *silent*.

Participant	Role	In `lld.ch11.state`
Context	The object clients use; holds the current state object and forwards requests to it	`Subscription`, `VendingMachine`
State	Interface declaring one method per request the context forwards	`SubscriptionState`, `VendingState`
ConcreteState	One class per state, implementing the behaviour for that state	`Trialing`, `Active`, `PastDue`, `Cancelled`; `Idle`, `HasMoney`, `SoldOut`

GoF leaves open *who* performs transitions: the context (fixed table) or the states. Letting states choose their successor, as here, is more flexible but makes them know each other.

The problem it solves

A subscription runs **trialing → active → past due → cancelled**. Naively each public method opens with the same four-way branch:

```
public void pay() {
    if (status == TRIALING)       { /* charge the card, switch to ACTIVE */ }
    else if (status == ACTIVE)    { /* renew */ }
    else if (status == PAST_DUE)  { /* settle arrears, back to ACTIVE */ }
    else if (status == CANCELLED) { /* reject */ }
}
```

`cancel()` and `missPayment()` repeat it; a fifth state means hunting every copy; one state's definition is spread across all methods; and a **forgotten branch** silently does nothing, surfacing weeks later as "subscriptions randomly stop working".

With State, **each state is a class**: the conditional vanishes (each context method is one line) and everything about a state sits in one file — what a reviewer or auditor wants.

How it works

1. A **State interface** with one method per action (`pay`, `missPayment`, `cancel`); every state answers every action, even with "not allowed".
2. One **concrete state** class per state.
3. The **context** (`Subscription`) holds the current state and delegates: `state.pay(this)`.
4. A state rules whether the action is **legal** and which state comes **next**, advancing via `context.setState(new Active())`.
5. Terminal states refuse everything, loudly, in one place.

The objects choose their own successors — the machine advances itself. That is the key difference from Strategy.

Structure (UML)

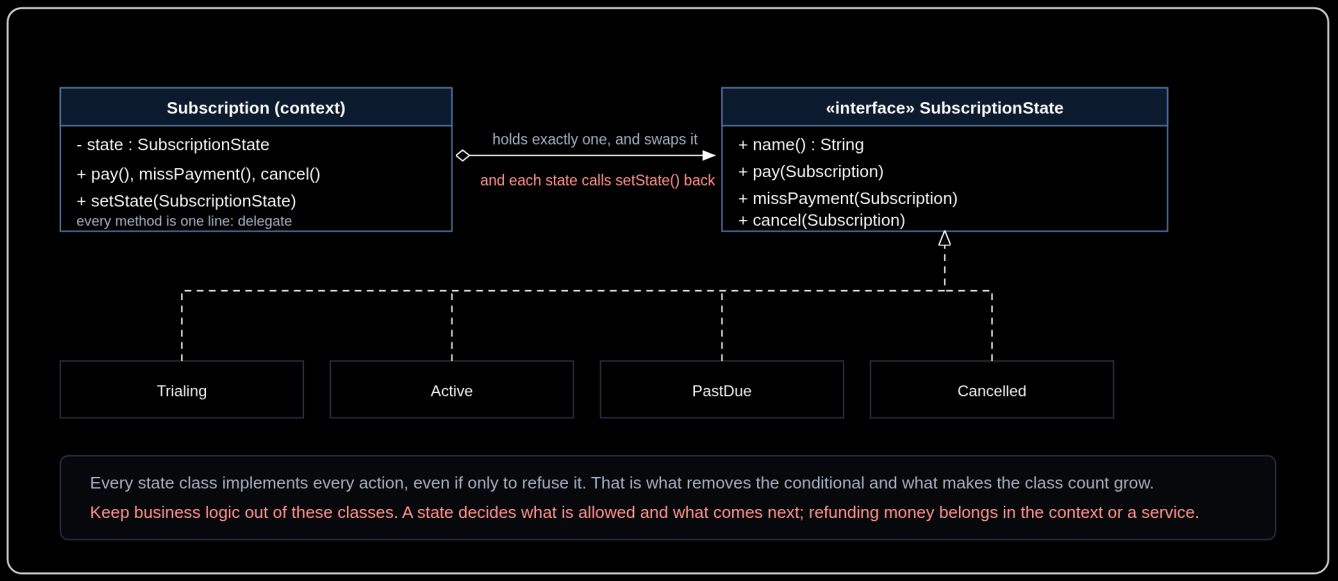
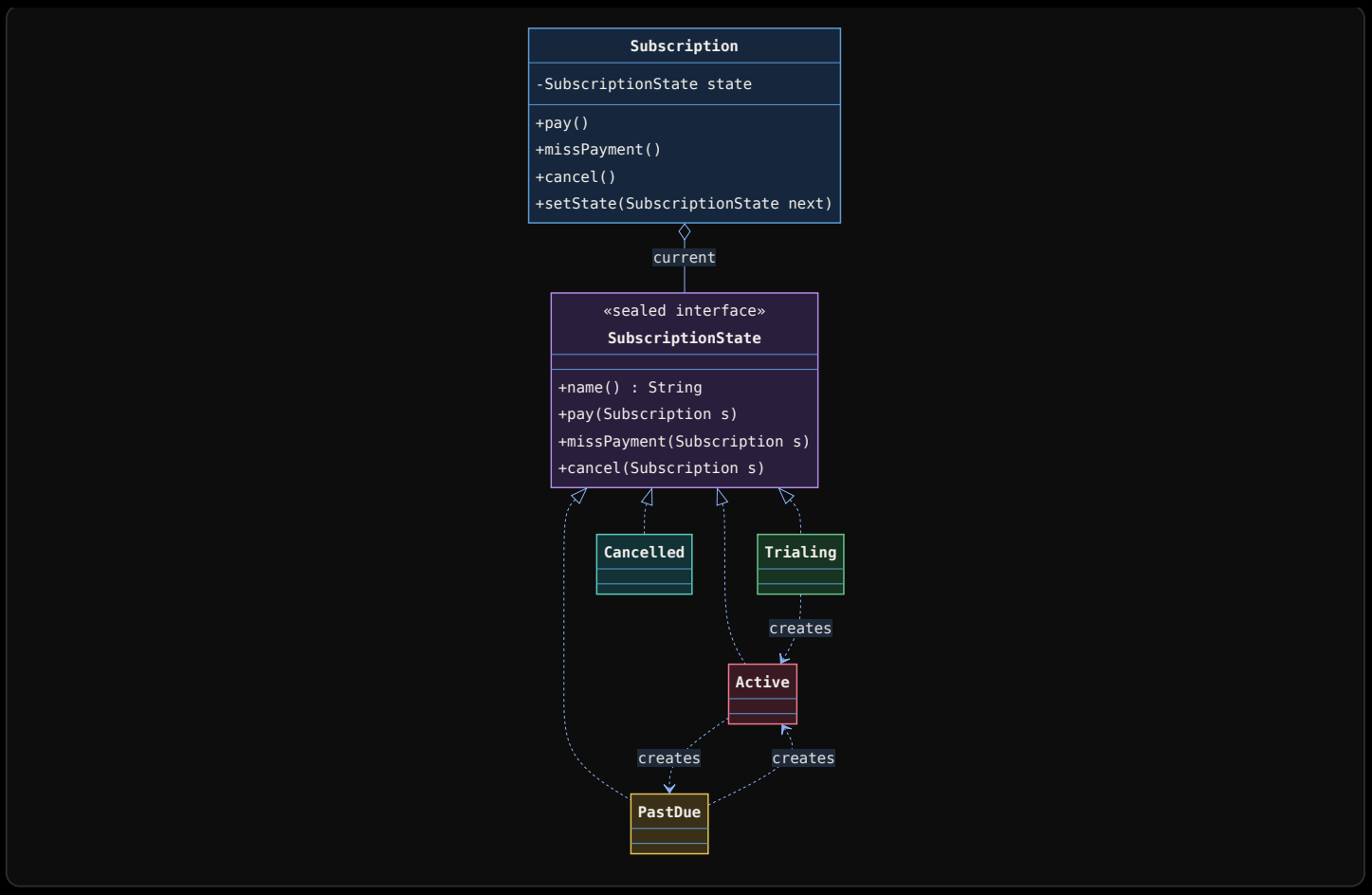
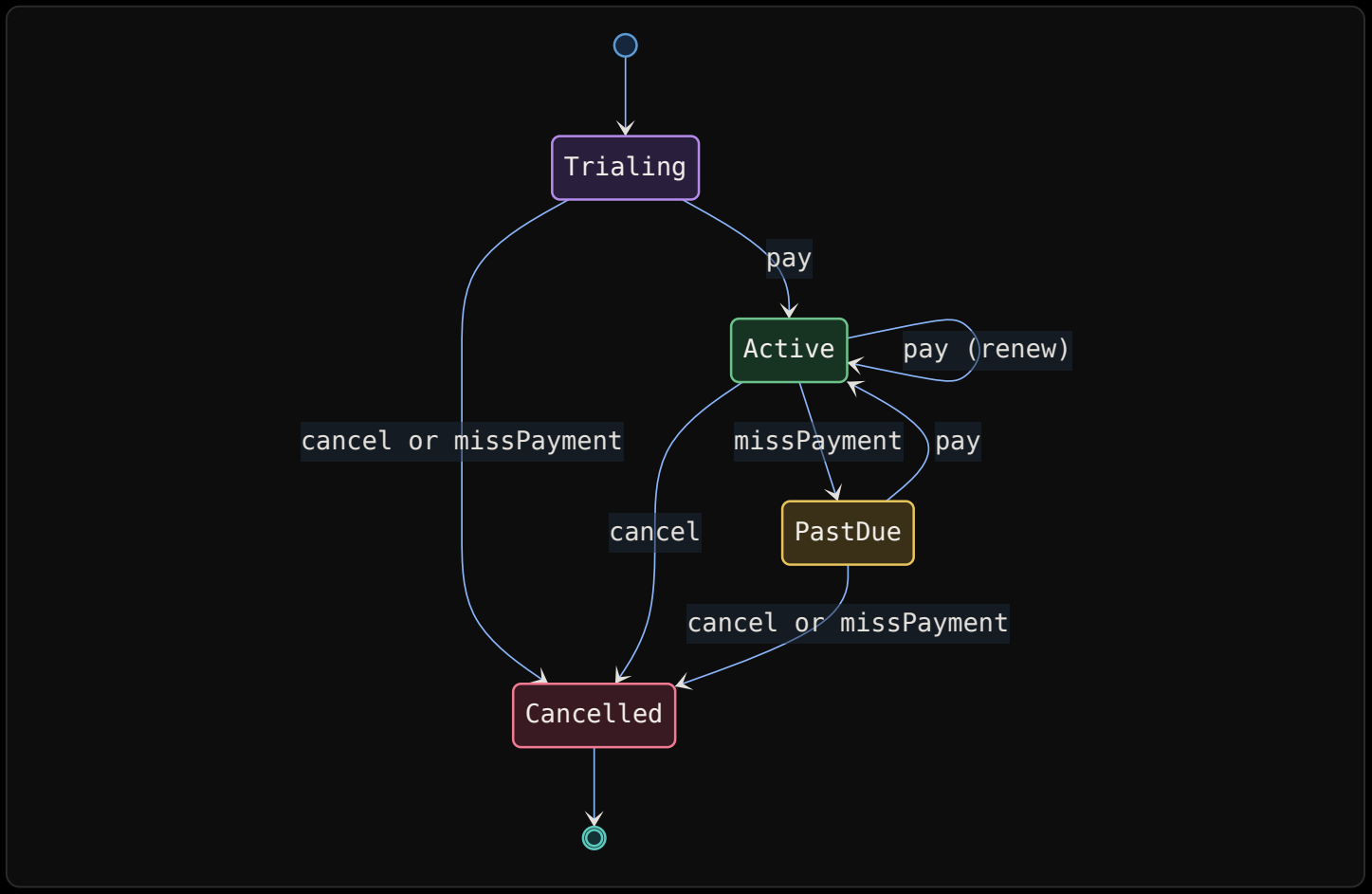


Figure — Class diagram: Subscription (context) holds a SubscriptionState; Trialing, Active, PastDue and Cancelled implement it and call setState on the context (course diagram)

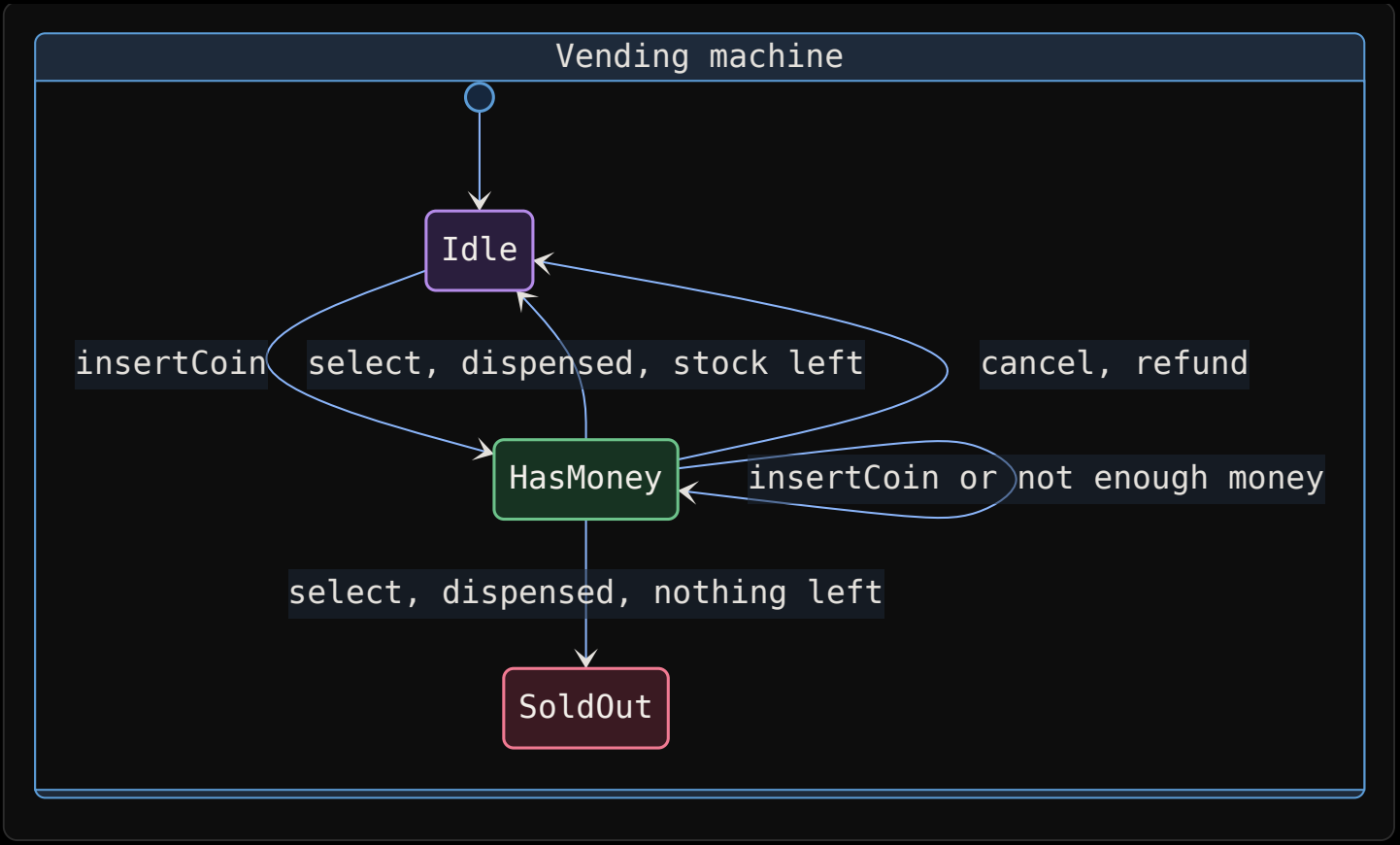
States reach **back** into the context to replace themselves, and know each other (Trialing creates an Active).



The state machine itself:



The vending machine (the traffic light, RED 30 s → GREEN 25 s → YELLOW 5 s → RED, is readable directly from its enum below):



Java example

`code/lld/ch11/state/` shows three styles:

1. **Class per state** (`Subscription` + four `record` states). `SubscriptionState` is **sealed** (lists exactly which types may implement it), so the compiler knows every state. The states have no fields, so component-less **records** (compact data classes with fields fixed at creation) are a neat, **immutable** (unchangeable after creation) choice.
2. **Enum-based state machine** (`TrafficLight`). Each constant overrides `next()`; shared behaviour is a **default method** (an interface method with a body, inherited unless overridden). The **whole machine is readable in one file** — idiomatic Java for simple machines.
3. **Vending machine** (`VendingMachine` + `Idle`, `HasMoney`, `SoldOut`). The state interface's **default methods refuse**, so each state overrides only what it allows (`SoldOut` overrides nothing). Business actions (dispensing, returning change) live in the **context**; states only rule on legality and the next state. Money is whole **cents in an** `int` (exact, unlike `double`); a real machine would use a `Money`/`BigDecimal` (Java's exact decimal class) type, as the Strategy and Visitor examples do.

```java
// SubscriptionState.java — each state implements all actions, including ones it only turns down.
public sealed interface SubscriptionState permits Trialing, Active, PastDue, Cancelled {
    String name();
    void pay(Subscription s);
    void missPayment(Subscription s);
    void cancel(Subscription s);
}

public record Trialing() implements SubscriptionState {
    public String name() { return "trialing"; }
    public void pay(Subscription s) {
        System.out.println("  [trialing] first payment taken");
        s.setState(new Active());                    // the state chooses its successor
    }
    public void missPayment(Subscription s) {
        System.out.println("  [trialing] trial ended unpaid");
        s.setState(new Cancelled());
    }
    public void cancel(Subscription s) { s.setState(new Cancelled()); }
}
// Active: pay renews; missPayment -> PastDue. PastDue: pay -> Active; missPayment -> Cancelled.

/** Terminal state: refuses everything, loudly, in one place. */
public record Cancelled() implements SubscriptionState {
    public String name() { return "cancelled"; }
    public void pay(Subscription s) { System.out.println("  [cancelled] refused: no payments after cancellation"); }
    public void missPayment(Subscription s) { System.out.println("  [cancelled] ignored: nothing is billed"); }
    public void cancel(Subscription s) { System.out.println("  [cancelled] rejected: already cancelled"); }
}

/** The CONTEXT: every public action is one line of delegation. No status switch anywhere. */
public final class Subscription {
    private SubscriptionState state = new Trialing();
    void setState(SubscriptionState next) {
        System.out.println("    " + state.name() + " -> " + next.name());
        state = next;
    }
    public void pay()         { state.pay(this); }
    public void missPayment() { state.missPayment(this); }
    public void cancel()      { state.cancel(this); }
}
```

```java
// TrafficLight.java — State the idiomatic-Java way: an enum whose constants override behaviour.
public enum TrafficLight {
    RED(30)    { TrafficLight next() { return GREEN; }  boolean pedestriansMayCross() { return true; } },
    GREEN(25)  { TrafficLight next() { return YELLOW; } },
    YELLOW(5)  { TrafficLight next() { return RED; } };
    ...
    abstract TrafficLight next();
    boolean pedestriansMayCross() { return false; }    // default for most states
}

// VendingState.java — default methods = "refuse", so each state overrides only what it allows.
interface VendingState {
    String name();
    default void insertCoin(VendingMachine m, int cents) { System.out.println("  [" + name() + "] coin rejected, returned
" + cents); }
    default void select(VendingMachine m, String code)   { System.out.println("  [" + name() + "] selection ignored"); }
    default void cancel(VendingMachine m)                { System.out.println("  [" + name() + "] nothing to cancel"); }
}
// Idle.insertCoin: m.addBalance(cents); m.setState(new HasMoney());   SoldOut overrides nothing.

// HasMoney.select — rules on legality; the business action lives in the context.
if (!m.inStock(code)) { System.out.println("  [has_money] " + code + " is empty, choose another"); return; }
int price = m.priceOf(code);
if (m.balance() < price) { System.out.println("  [has_money] " + code + " costs " + price + ", balance " + m.balance());
return; }
m.dispense(code);
m.returnChange();
m.setState(m.anythingInStock() ? new Idle() : new SoldOut());
// VendingMachine (CONTEXT) owns slots and balance; insertCoin/select/cancel delegate to the current state.
```

Output:

```
=== 1. Subscription lifecycle (class per state) ===
  [trialing] first payment taken
    trialing -> active
  [active] payment failed, grace period started
    active -> past_due
  [past_due] arrears paid
    past_due -> active
    active -> cancelled
  [cancelled] refused: no payments after cancellation
  final status: cancelled

=== 2. Traffic light (enum-based state machine) ===
  RED    for 30s, pedestrians may cross: true
  GREEN  for 25s, pedestrians may cross: false
  YELLOW for  5s, pedestrians may cross: false
  RED    for 30s, pedestrians may cross: true

=== 3. Vending machine ===
select A1
  [idle] selection ignored
insert 100
  balance 100
    idle -> has_money
select A1
  [has_money] A1 costs 150, balance 100
select B2
  [has_money] B2 is empty, choose another
insert 100
  balance 200
select A1
  >> dispensed cola
  >> change returned 50
    has_money -> sold_out
insert 25
  [sold_out] coin rejected, returned 25
```

The final subscription line shows the payoff: an **illegal action** is refused the same way every time, by code in one file.

State vs Strategy (and Bridge) — same diagram, different reasons

These patterns draw **the same class diagram** (an object holds an interface reference and forwards to it) — one of the most common interview questions, and the diagram cannot answer it:

	State	**Strategy**	**Bridge**
Who chooses the object	The **states**, which replace themselves via the context	The **caller**, from outside	Chosen once at construction, usually by configuration
How often it changes	Repeatedly — that is the point	Set once, sometimes per call	Effectively never after wiring
Do the alternatives reference one another?	**Yes** (`Trialing` builds an `Active`)	**No** — never	No
What varies	The object's current *condition*, and therefore its allowed actions	*How* one job gets done	A whole dimension of the design
Why it exists	To model a lifecycle	To make one algorithm replaceable	To stop two class hierarchies multiplying

The reliable test: do the alternatives name each other? Yes → State machine. No → Strategy (or Bridge — then ask how often the choice changes).

Looks like	The actual difference
An enum plus one `switch`	Frequently the better answer, and easy to defend in an interview. Class-per-state starts to win only once the same status switch is repeated in three or more methods.
A state-machine library (Spring Statemachine, XState)	The same idea expressed as data (states, events, guards, actions) rather than classes. Once a machine grows past roughly a dozen states, the declarative form is easier to read and to draw.

Trade-offs

Pros	Cons
Everything about one state sits in a single file.	One class per state; small classes multiply.
The repeated conditional disappears; no branch can be missed.	Transitions live in several files; no single place shows the whole machine.
Illegal actions are rejected consistently.	Each state must implement all actions, even just to say no.
A new state = a new class; the compiler lists what it must implement.	Persisting needs a name↔class mapping — keep it in exactly one place.

When not to use it

- **Two states and a handful of methods** — a boolean with an `if` reads better.
- **Only data changes, not behaviour** — store the differing value in a field.
- **A big, formal machine** (over about a dozen states with guards) — a **transition table** (`Map<(state, event), handler>`) or a library (e.g. **Spring Statemachine**), where the diagram can be the source of truth.
- **Rule of thumb from the course:** it pays off once **three or more methods repeat the same status switch**; below that, an enum with a switch is defensible.
- **Signal you've outgrown it:** you keep sketching the machine on paper to follow your own code — make the sketch (transition table or state chart) the real source of truth.

Common mistakes

1. **Successors hard-coded everywhere** — fine at 4 states, painful at 12; then let the context or a transition table own transitions.
2. **Business logic inside state classes.** A state rules on *legality* and picks the *successor*; charging cards and sending emails belong in the context or a service it calls.
3. **Not making illegal actions explicit** (silently doing nothing).
4. **Storing state objects in the DB** — store a name/enum and map it back in exactly one place.
5. **Sharing stateful state objects** between contexts — stateless states (like ours) can be shared; stateful ones cannot.

State in other languages

- **Java:** an `enum` with per-constant behaviour (`TrafficLight`) keeps the graph in one file; a `sealed` interface plus exhaustive pattern-matching `switch` gives compiler-checked coverage.
- **Kotlin / Scala:** `sealed` hierarchies with exhaustive `when` / `match`.
- **Rust:** enums with data, matched exhaustively; the **typestate** pattern makes each state a separate *type*, so an action illegal in that state is a compile error.
- **Python:** a dict mapping `(state, action)` to a handler — a compact transition table showing every transition.
- **TypeScript:** a **discriminated union** (object types each carrying a fixed tag field) plus a `switch` on the tag, checked for missing cases.

Real-world examples

- **Order lifecycles** in food delivery and e-commerce: placed → accepted → preparing → out for delivery → delivered (or cancelled); you can cancel while "placed" but usually not once "out for delivery".
- **Payment and subscription lifecycles** (Stripe statuses such as `trialing`, `active`, `past_due`, `canceled`).
- **TCP connections** (the protocol under most internet traffic, giving ordered, reliable delivery; see System Design Fundamentals) — GoF's own example: `LISTEN`, `ESTABLISHED`, `CLOSED` respond differently to the same call.

- **Workflow/approval engines** (draft → in review → approved/rejected), **editor modes**, **game characters** (idle, running, jumping).
- **Lexers/parsers** (split text into tokens, then build a structure), **vending machines, ATMs, elevators, traffic lights** — favourite **machine-coding** questions (an interview round writing working code for a small system in an hour or two; see the case-study chapters).
- **Libraries**: Spring Statemachine; the JDK's `Thread.State` is an *enum of states* (data), not the pattern.

In the interview

> **30-second answer:** "State swaps a status field, and the switch statements repeated around it, for one class per state. The context forwards each action to whichever state object it currently holds, and that object decides whether the action is legal and which state follows. Each state's rules sit together in one file, and an illegal action is refused the same way every time. It draws exactly like Strategy, but states name one another and move the machine forward on their own, while strategies are picked from outside and never reference each other. In Java, for small machines, I'd use an enum with per-constant behaviour or a sealed interface."

Follow-ups:

- *State vs Strategy?* The naming test and the "who chooses" test (table above).
- *Where should transitions live?* In the states for small machines; in the context or a transition table as it grows, so the graph is visible in one place.
- *How do you persist it?* Store the state's name (enum) in the DB; rebuild the object through one factory/mapping.
- *How do you handle concurrency?* Two requests ("cancel" and "deliver") can **race** (run together and interfere, the result depending on timing): guard transitions with a lock or, in a DB, an **optimistic** (no lock held; the write fails if someone changed the row first) `UPDATE ... WHERE status = 'PLACED'` and check the affected row count.
- *Design a vending machine.* States Idle, HasMoney, SoldOut (maybe Dispensing, Maintenance); the context owns inventory and balance; states allow or refuse actions (as in our code).

Self-check

1. **Q:** Who changes the current state in the State pattern? **A:** The state objects (via the context's `setState`).
2. **Q:** What is the "naming test"? **A:** If the alternatives reference each other, it's State; if never, it's Strategy/Bridge.
3. **Q:** When is an `enum` better than class-per-state? **A:** When the machine is small; the whole machine is visible in one file.
4. **Q:** What belongs in the context, not the state? **A:** Business actions and data (dispensing, charging, inventory).
5. **Q:** What does a sealed state interface give you? **A:** The compiler knows every state, so `switch`es over states can be checked for exhaustiveness.

TL;DR

A class for each state; the context forwards every action; each state picks what follows it. Same diagram as Strategy, but states **know each other**. Use enums for small machines, tables/libraries for big ones.

11.3 Strategy

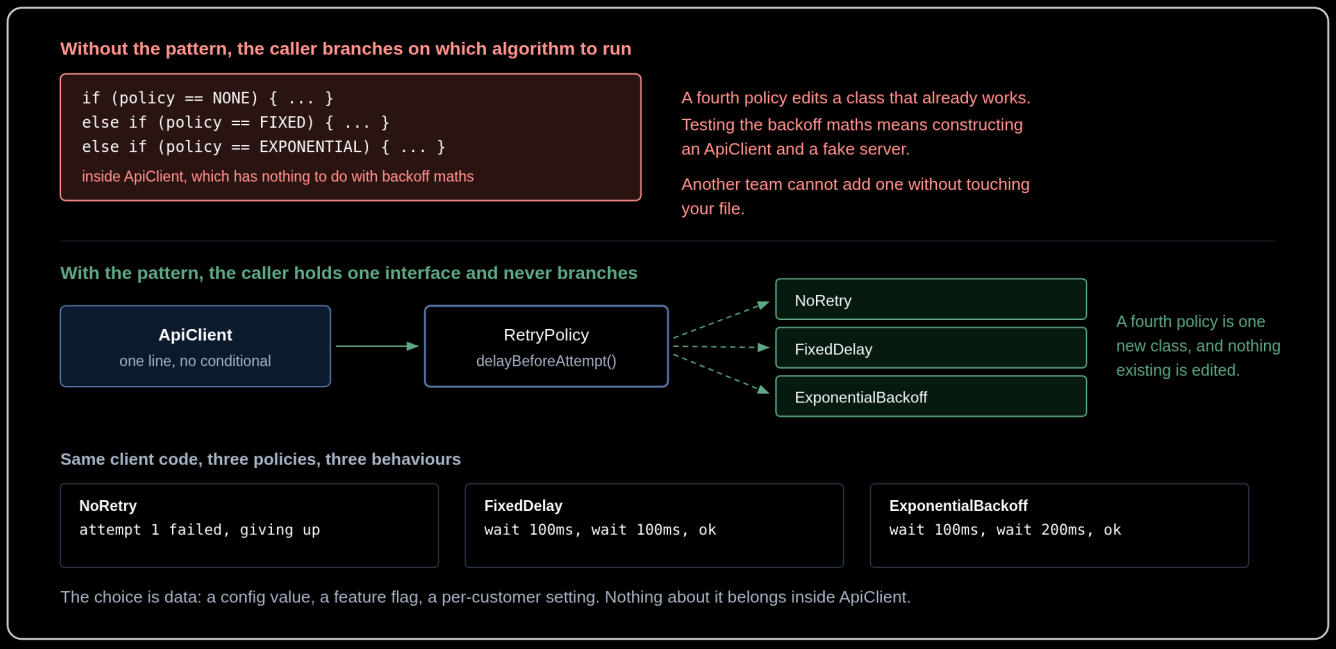


Figure — Top: ApiClient branches on which retry policy to use. Bottom: ApiClient holds one RetryPolicy interface and never branches (course diagram)

What it is

Strategy (GoF intent, paraphrased): group related algorithms into a **family**, each in its own class behind a **common interface**, interchangeable so the algorithm can change — even at run time — without touching the calling code. The object using it (the **context**) does not branch on which algorithm it has. Like getting to the airport by car, train or taxi: same goal, different algorithm, chosen by time, money and weather.

Participant	Role	In `lld.ch11.strategy`
Strategy	Interface shared by all the algorithms; the context calls only this	`RetryPolicy` (also `Comparator` in part 2)
ConcreteStrategy	One implementation per algorithm	`FixedDelay`, `ExponentialBackoff`, the `RetryPolicy.none()` lambda (the course diagram names it `NoRetry`)
Context	Is configured with a strategy object and delegates to it; may pass it the data it needs	`ApiClient`

The problem it solves

Retrying a failed request can mean no retry, a fixed delay, or **exponential backoff** (doubling the wait after each failure — 100 ms, 200 ms, 400 ms). Written directly, the branch sits inside `ApiClient`:

```
if (policy == NONE)             { /* give up */ }
else if (policy == FIXED)       { /* wait 100ms, try again */ }
else if (policy == EXPONENTIAL) { /* double the wait after each failure */ }
```

`ApiClient` now holds backoff arithmetic unrelated to calling an API; a fourth policy edits a working class; backoff maths can't be tested without a client and a fake server; another team must edit your class to contribute a policy.

Strategy gives each algorithm its own class. `ApiClient` asks its `RetryPolicy` "how long before the next attempt?" and **never checks what kind it holds**. The choice is made **outside** — configuration, a **feature toggle** (switches behaviour without redeploying), or a customer setting.

How it works

1. A **strategy interface** — small, ideally one question (`delayBeforeAttempt(attempt)`).
2. One **concrete strategy** per algorithm, holding only its own parameters.

3. The **context** receives a strategy (constructor, setter or method parameter) and calls it.
4. A **factory, configuration or DI container** (a library that creates objects and hands them their dependencies — §11.7.2) decides which — not every call site.

Structure (UML)

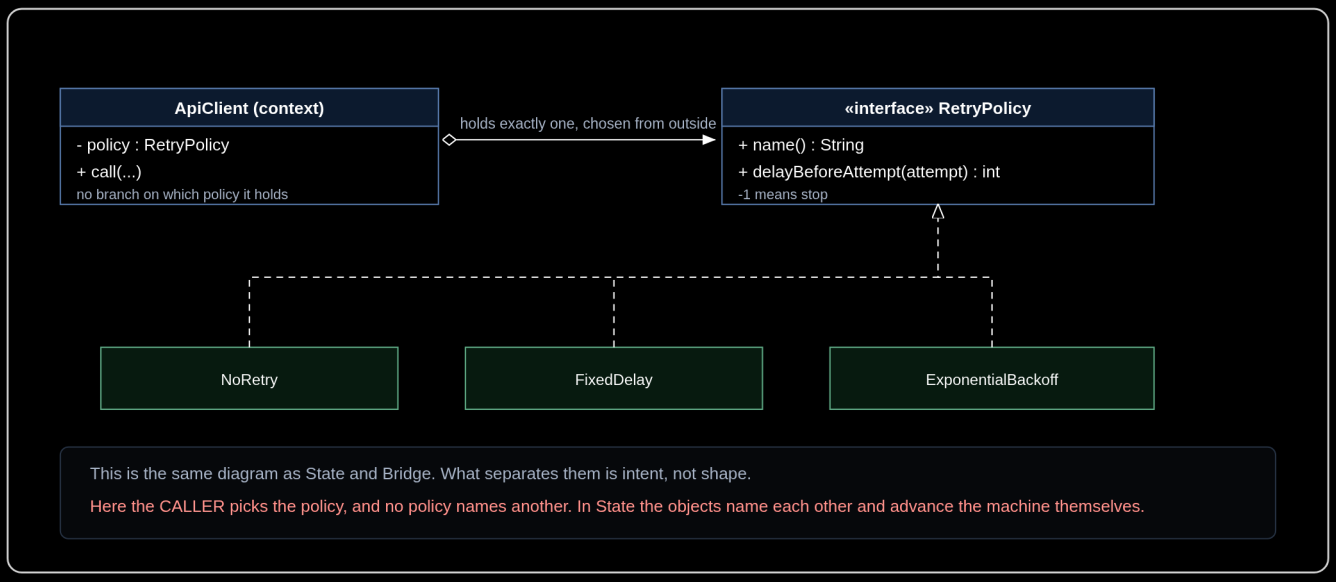
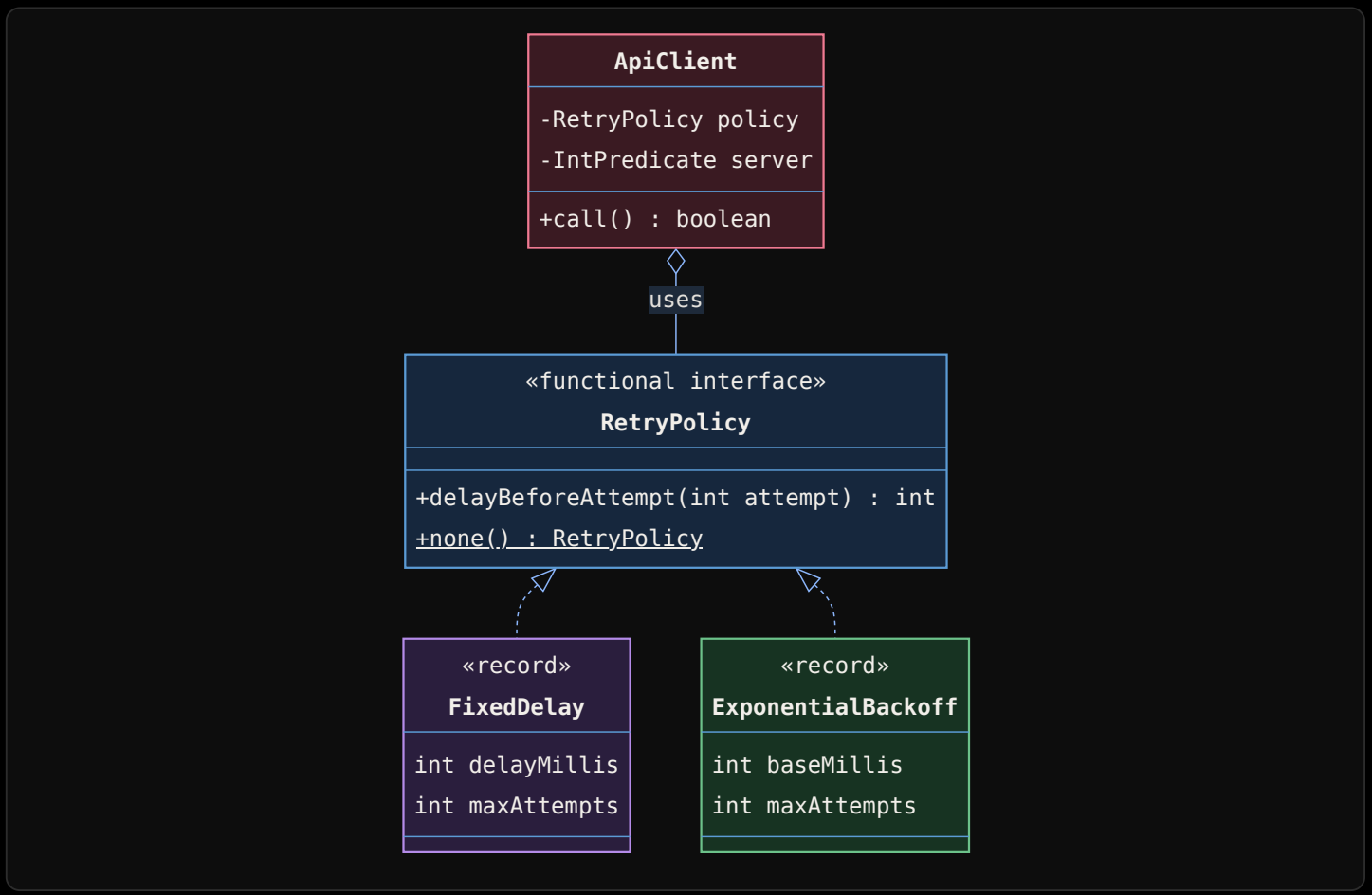


Figure — Class diagram: ApiClient holds a RetryPolicy; NoRetry, FixedDelay and ExponentialBackoff implement it (course diagram)

No strategy references another — `ExponentialBackoff` has never heard of `FixedDelay`. That absence tells Strategy from State.



Java example

`code/lld/ch11/strategy/`:

1. **Retry policies.** `RetryPolicy` has one abstract method, so `RetryPolicy.none()` is a **lambda**; configured strategies (`FixedDelay`, `ExponentialBackoff`) are **records**. `ApiClient.call()` is identical for all runs; only the injected policy differs. (Delays are printed, not slept.)
2. `Comparator` — the JDK's everyday strategy: `List.sort(comparator)` fixes the sorting algorithm and plugs in the comparison; `Comparator.comparing(...).reversed().thenComparing(...)` composes strategies.
3. **Strategy registry.** A `Map` from configuration key to a lambda (`UnaryOperator<BigDecimal>` pricing rules) — short and readable when each strategy is one line.

```
/** STRATEGY: one small question — delay in ms before the given attempt (>= 2), or STOP. */
@FunctionalInterface
public interface RetryPolicy {
    int STOP = -1;
    int delayBeforeAttempt(int attempt);
    static RetryPolicy none() { return attempt -> STOP; }
}
/** Strategies with configuration earn a real type (a record here). */
public record FixedDelay(int delayMillis, int maxAttempts) implements RetryPolicy {
    public int delayBeforeAttempt(int attempt) { return attempt > maxAttempts ? STOP : delayMillis; }
}
/** 100, 200, 400, ... ms. Knows nothing about FixedDelay: strategies never name each other. */
public record ExponentialBackoff(int baseMillis, int maxAttempts) implements RetryPolicy {
    public int delayBeforeAttempt(int attempt) { return attempt > maxAttempts ? STOP : baseMillis << (attempt - 2); }
}

/** CONTEXT: holds a policy and never asks WHICH policy it has. server: fake (attempt number -> success?). */
public boolean call() {
    for (int attempt = 1; ; attempt++) {
        if (server.test(attempt)) { System.out.println("  attempt " + attempt + ": ok"); return true; }
        int delay = policy.delayBeforeAttempt(attempt + 1);
        if (delay == RetryPolicy.STOP) { System.out.println("  attempt " + attempt + ": failed, giving up"); return false;
}
        System.out.println("  attempt " + attempt + ": failed, waiting " + delay + "ms");  // simulated, no sleep
    }
}

// Main.java, parts 2 and 3
Comparator<Product> byRatingThenName =
Comparator.comparingDouble(Product::rating).reversed().thenComparing(Product::name);
Map<String, UnaryOperator<BigDecimal>> pricing = Map.of(
    "regular", p -> p,
    "member", p -> p.multiply(new BigDecimal("0.90")).setScale(2, RoundingMode.HALF_EVEN),  // 10% off
    "flash-sale", p -> p.subtract(new BigDecimal("20.00")).max(BigDecimal.ZERO));          // flat 20.00 off
```

Extending is cheap: a policy adding random **jitter**, or one honouring a server's `Retry-After` header (an **HTTP** — the web's request/response protocol — response header saying how long to wait), is one more class (or lambda) plus one line where the policy is chosen; `ApiClient` doesn't change.

Output:

```
=== 1. Retry policies: same ApiClient code, three strategies ===
none (a lambda):
  attempt 1: failed, giving up
FixedDelay(100, 4):
  attempt 1: failed, waiting 100ms
  attempt 2: failed, waiting 100ms
  attempt 3: ok
ExponentialBackoff(100, 4):
  attempt 1: failed, waiting 100ms
  attempt 2: failed, waiting 200ms
  attempt 3: ok
ExponentialBackoff(100, 4) against a server that is always down:
  attempt 1: failed, waiting 100ms
  attempt 2: failed, waiting 200ms
  attempt 3: failed, waiting 400ms
  attempt 4: failed, giving up

=== 2. Comparator: the JDK's everyday strategy ===
  by price:          [cable, mouse, keyboard, monitor]
  by rating, name:   [keyboard, cable, mouse, monitor]

=== 3. Strategy registry: lambdas chosen by configuration ===
  regular    keyboard costs 49.99
  member     keyboard costs 44.99
  flash-sale keyboard costs 29.99
```

Lambdas as strategies

Strategy is the pattern a plain function replaces most often. Since Java 8 any interface with a **single abstract method** (a *functional interface*) can be a lambda or method reference, so many strategies need **no class**: `Comparator<T>`, `Function`, `Predicate`, `Supplier`, or your `RetryPolicy`. Keep a named class (or record) when the strategy has **configuration or state** (`FixedDelay(100, 4)`), needs a **name for logging**, or must be **discovered/registered** (Spring beans — container-managed objects — or `ServiceLoader`, the JDK's plug-in discovery mechanism).

Other languages: Python passes a function (or `functools.partial` to bind parameters); a dict from name to function is a one-line registry. JavaScript/TypeScript pass a function argument or an object of functions. Kotlin/Scala take a function-typed constructor parameter, often with a default. Go uses a one-method interface (satisfied implicitly) or a `func` field.

The registry uses `BigDecimal` for prices: money in domain code is never `double`, since binary floating point can't represent most cent values exactly.

Trade-offs

Pros	Cons
New variants without editing working code — even by other teams (Open/Closed).	One class per variant (unless lambdas), one more file to open.
Each algorithm is testable alone.	Someone must choose — ideally a factory or container, not every caller.
Behaviour configurable at run time (flag, setting, header).	The interface must fit every variant, including future ones.
The context keeps to its real job (no backoff maths in an API client).	Tempting to overuse: three one-line strategies = three classes for three lines.

When not to use it

- **Two branches, no third in sight** — an `if` reads better; introduce the pattern when a third appears.
- **One-line branches** — a `Map` from key to lambda is shorter than a class each.
- **Variants need different inputs** (one the customer, another the warehouse) — the shared interface is forced; rethink the split.
- **The choice is fixed for the program's life** — decide it when wiring (or use Bridge).

Common mistakes

1. **Every call site choosing** — the branch moved instead of disappearing; let a factory/config/DI container choose.
2. **A bloated interface** with parameters half the implementations ignore — the variants do different jobs.
3. **Strategies that know about each other** — then it is State.
4. **Stateful strategies shared between threads** — keep them immutable (records help).

Real-world examples

- `Comparator` with `Collections.sort` / `List.sort` / `TreeMap` — the textbook example.
- `ThreadPoolExecutor`**'s** (the JDK's configurable thread pool) `RejectedExecutionHandler`: `AbortPolicy`, `CallerRunsPolicy`, `DiscardPolicy`, `DiscardOldestPolicy` — what happens when the pool is full.
- **Spring Security** `PasswordEncoder` (`BCryptPasswordEncoder`, `Argon2PasswordEncoder`, ...) chosen by configuration, so upgrading the **hash** (one-way scrambling of the password, irreversible) doesn't touch login code.
- **Resilience4j** (a Java fault-tolerance library) retry `IntervalFunction` (fixed, exponential, randomized backoff) and **Polly** in .NET — exactly these retry policies.
- **AWT/Swing** `LayoutManager` (`FlowLayout`, `BorderLayout`, `GridLayout`) — a container delegates layout to a strategy.
- **Payment method at checkout** (card, UPI [India's instant bank-to-bank payment system], wallet, cash on delivery), **pricing/discount rules by customer tier**, **route modes** (drive, walk, transit) in map apps, **compression/serialization formats** (turning objects into bytes or text) chosen by a content-type header.

In the interview

> **30-second answer:** "Strategy gives every variant of one job its own class behind a common interface, and something outside — usually a factory or configuration — picks the variant at run time. The context stops branching: my API client asks its retry policy how long to wait and never asks which policy it is. Each algorithm is testable alone and new ones don't touch working code. In Java a lambda is often enough, like a `Comparator`; I keep a class when the strategy has configuration. It shares its diagram with State, but strategies never reference each other."

Follow-ups:

- *Strategy vs State / Bridge?* Strategies are picked from outside and don't know each other; states pick their successors; Bridge is a design dimension fixed at assembly (see the State-vs-Strategy-vs-Bridge table in §11.2 and §9.2). Don't use any of the three for two branches that will never become three.
- *Strategy vs Template Method?* Template Method swaps **one step** by **inheritance**; Strategy swaps **the whole algorithm** by **composition**, chosen at run time (table in §11.4). Prefer Strategy when both fit.
- *Strategy vs Command?* Strategy is *how* to do something; Command is *what* to do, with arguments, kept to run later or undo.
- *How is the strategy selected?* Config + factory (`Map<String, Strategy>`), DI container (Spring can inject a `Map<String, PaymentStrategy>` of all beans), or per request (header, customer tier).
- *Relation to Open/Closed and DIP?* The context depends on an abstraction (DIP); new strategies extend behaviour without modifying it (OCP).

Self-check

1. **Q:** Who chooses the strategy? **A:** Someone outside the context — ideally a factory/config/DI container.
2. **Q:** Why can `RetryPolicy.none()` be a lambda? **A:** `RetryPolicy` is a functional interface (one abstract method).
3. **Q:** Name two JDK strategies. **A:** `Comparator`; `RejectedExecutionHandler` (also `LayoutManager`).
4. **Q:** When is a map of lambdas better than classes? **A:** When each strategy is one line and needs no configuration.
5. **Q:** How do you tell Strategy from State in code? **A:** Strategies never reference each other.

TL;DR

Interchangeable algorithms behind one interface, chosen from outside. The context stops branching. Use lambdas for one-liners, records/classes when configured. Don't confuse with State (states know each other) or Template Method (inheritance, one step).

11.4 Template Method

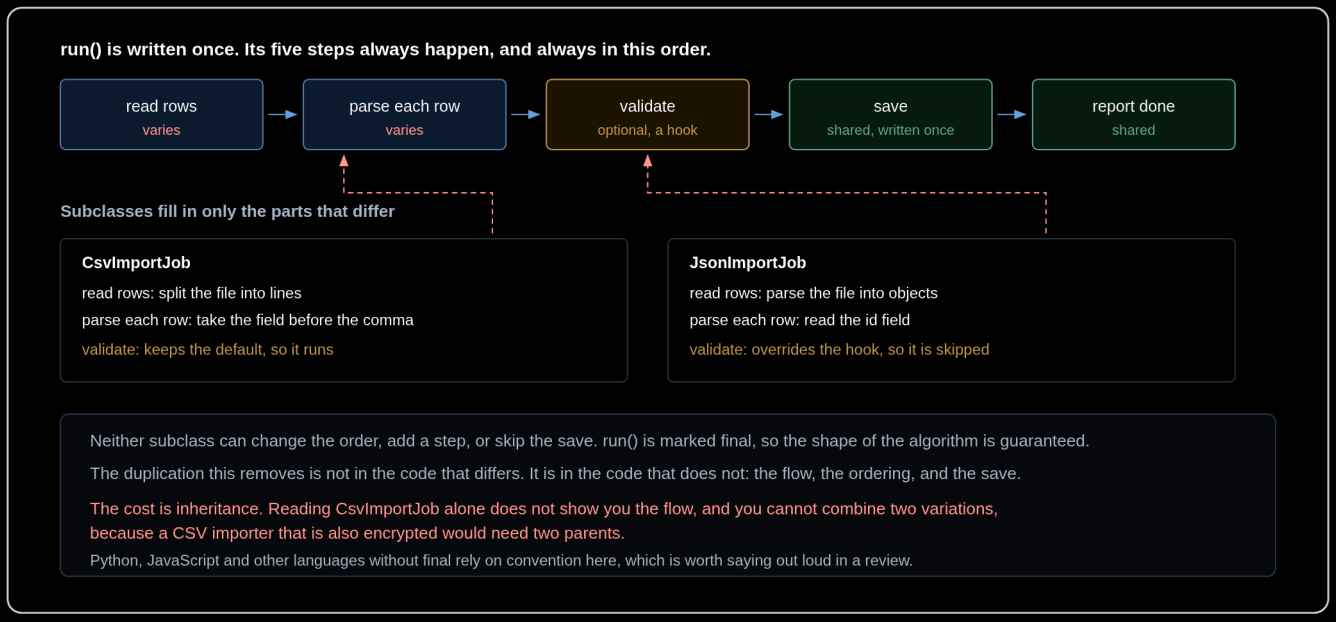


Figure — run() always performs five steps in the same order; subclasses fill in only reading and parsing, and one optional hook (course diagram)

What it is

Template Method (GoF intent, paraphrased): put the **outline of an algorithm** into one base-class operation (the *template method*) and leave certain steps for **subclasses**; they may redefine those steps but not the overall sequence. Like a recipe card with blanks — "boil water → brew ___ → pour → add ___ (if wanted)" — tea and coffee fill the blanks differently; the order never changes.

Participant	Role	In `lld.ch11.templatemethod`
AbstractClass	Declares the primitive steps (abstract or with a default) and implements the template method that calls them in order	`ImportJob`, `CaffeineBeverage`
ConcreteClass	Implements the primitive steps that differ for its variant	`CsvImportJob`, `JsonImportJob`, `Tea`, `Coffee`

The problem it solves

An import job always reads rows, parses each, validates, saves, reports "done"; only reading and parsing differ between CSV and JSON. As separate classes the **shared flow** (order, validation, saving, reporting) is copied into each; when saving must also write an audit record, three files change — and the missed one is the bug. Template Method writes the flow once in a method subclasses **cannot override** (`final`); varying steps become methods subclasses must supply.

How it works

1. Write the **template method** in an abstract class, `final` so the order is guaranteed.
2. It calls the steps in order. Three kinds:
   - **Required (abstract) steps** — every subclass must implement; the compiler enforces it.
   - **Hooks** — steps with a **default** (often empty or `true`) a subclass *may* override, e.g. `shouldValidate()`.
   - **Shared steps** — `private`, written once, not changeable.
3. Each subclass implements only what differs.

The Hollywood Principle — "Don't call us, we'll call you." (*Head First*.) The base class (high-level component) decides **when** subclass methods run; the subclass never drives the flow. That **inversion of control** is the backbone of most **frameworks** (libraries that own the program's flow and call your code at set points): **the framework runs the sequence; your class supplies only your pieces.**

Structure (UML)

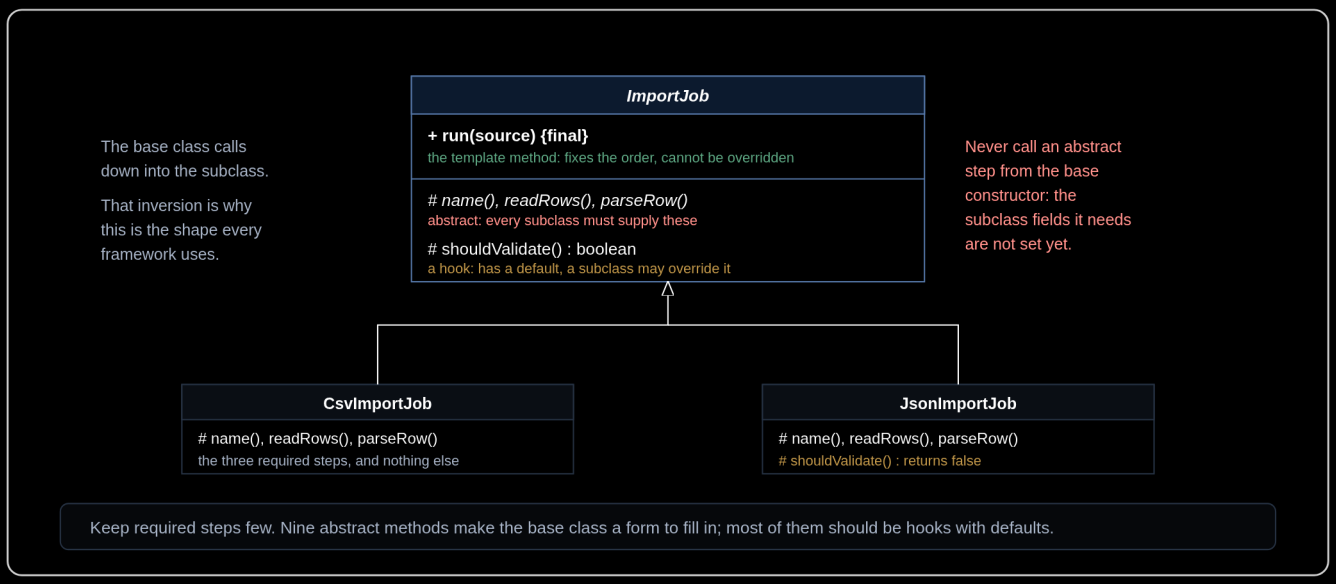
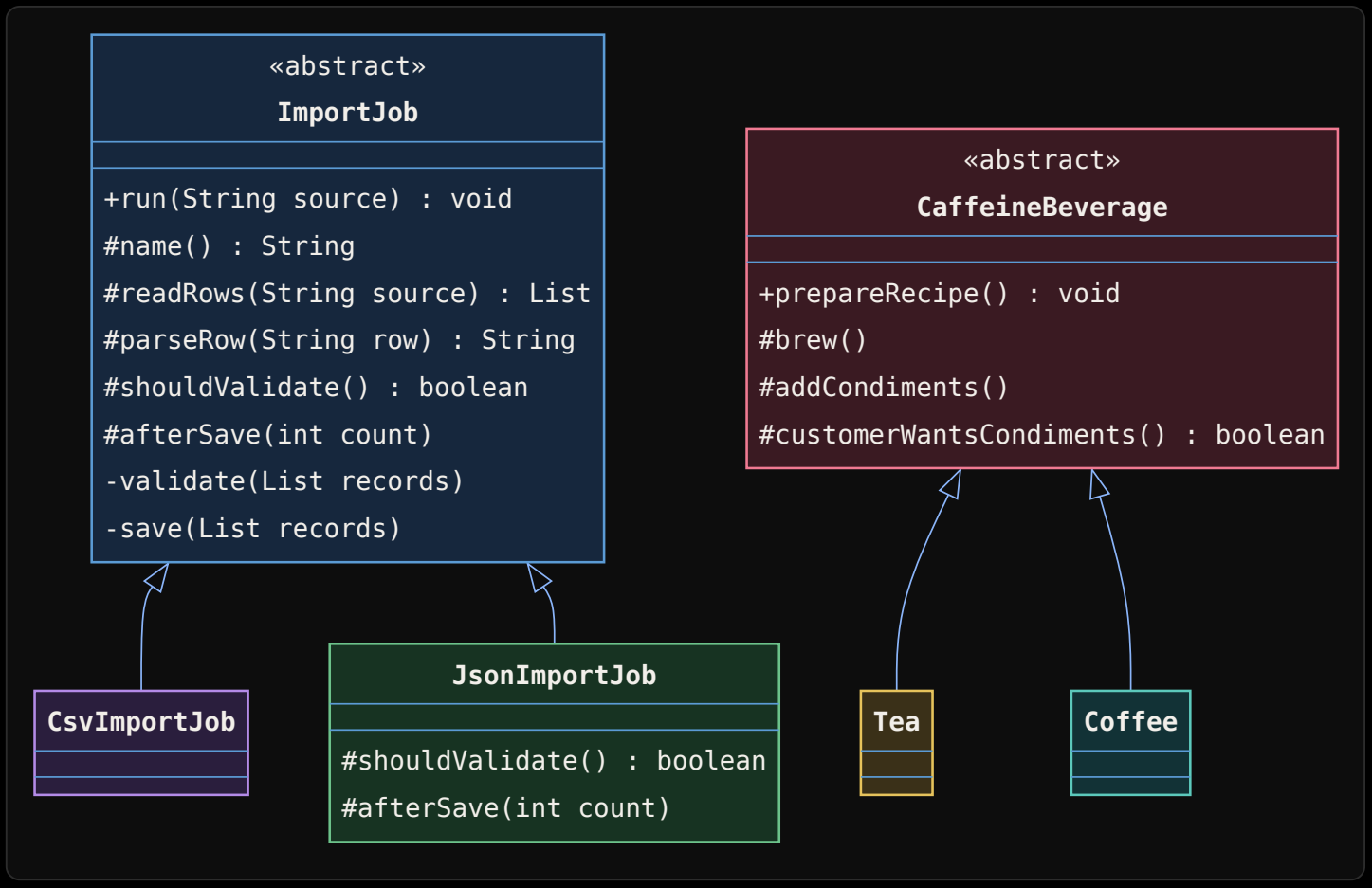


Figure — Class diagram: ImportJob with final run(), abstract steps name/readRows/parseRow, hook shouldValidate and private save; CsvImportJob and JsonImportJob override only what differs (course diagram)

Subclasses are tiny — but `CsvImportJob` alone doesn't reveal the sequence it follows; only the parent shows it. That is the trade-off.



`run()` and `prepareRecipe()` are `final` in the code (Mermaid has no marker for `final`).

Java example

`code/lld/ch11/templatemethod/`:

1. **Import jobs.** `ImportJob.run` is `final`; `name`, `readRows`, `parseRow` are **abstract** (required); `shouldValidate` and `afterSave` are **hooks** (defaults `true` and "do nothing"); `validate` and `save` are **private** shared steps. `JsonImportJob` overrides both hooks.
2. **Beverages** (*Head First*). `customerWantsCondiments()` is a hook: `Coffee(black = true)` turns off the optional step.

```java
/** ABSTRACT CLASS holding the TEMPLATE METHOD. */
public abstract class ImportJob {
    /** Template method. Marked final: subclasses cannot change the order, drop a step or insert one. */
    public final void run(String source) {
        System.out.println("Running the " + name() + " import:");
        List<String> rows = readRows(source);                // required step
        List<String> records = new ArrayList<>();
        for (String row : rows) records.add(parseRow(row));   // required step
        System.out.println("  parsed " + records.size() + " records: " + records);
        if (shouldValidate()) {                              // hook
            validate(records);
        } else {
            System.out.println("  skipping validation, source is validated upstream");
        }
        save(records);                                       // shared, fixed
        afterSave(records.size());                           // hook with an empty default
        System.out.println("  done");
    }
    // Required steps: the compiler forces every subclass to supply them.
    protected abstract String name();
    protected abstract List<String> readRows(String source);
    protected abstract String parseRow(String row);
    // Hooks: sensible defaults, overriding is optional.
    protected boolean shouldValidate() { return true; }
    protected void afterSave(int count) { }
    // Shared steps: private validate(...) and save(...) — written once, cannot be changed by subclasses.
}

/** Supplies only what differs for CSV. */
public final class CsvImportJob extends ImportJob {
    @Override protected String name() { return "CSV"; }
    @Override protected List<String> readRows(String source) { return Arrays.stream(source.split("\n")).skip(1).toList();
} // skip header
    @Override protected String parseRow(String row) { return row.split(",")[0].trim(); }
}
// JsonImportJob (toy regex parsing, no JSON library) also overrides the hooks:
//   shouldValidate() -> false;  afterSave(count) prints "[hook] notified upstream: N ids consumed"

public abstract class CaffeineBeverage {
    public final void prepareRecipe() {          // template method
        boilWater();
        brew();
        pourInCup();
        if (customerWantsCondiments()) addCondiments();
    }
    protected abstract void brew();
    protected abstract void addCondiments();
    /** Hook: "Don't call us, we'll call you" - the base class decides WHEN this runs. */
    protected boolean customerWantsCondiments() { return true; }
    ...
}
// Coffee(boolean black) overrides customerWantsCondiments() to return !black.
```

Output:

```
=== 1. Import jobs ===
Running the CSV import:
  parsed 3 records: [u1, u2, u3]
  validated 3 records, 0 blank
  saved 3 records (+ audit entry)
  done
Running the JSON import:
  parsed 2 records: [p9, p7]
  skipping validation, source is validated upstream
  saved 2 records (+ audit entry)
  [hook] notified upstream: 2 ids consumed
  done

=== 2. Beverages (hook controls an optional step) ===
tea:
  boiling water
  steeping the tea
  pouring into cup
  adding lemon
black coffee:
  boiling water
  dripping coffee through filter
  pouring into cup
```

Template Method vs Strategy

	Template Method	Strategy
What varies	**One step**; the surrounding algorithm is fixed	**The entire** algorithm
Mechanism	**Inheritance** — subclass supplies the step	**Composition** — caller passes an object in
When chosen	When the object is created (by which subclass you instantiate); fixed after that	Run time, can change per call
How many axes can vary	One (a class has one parent)	Several (hold several strategies)
What one file shows	Just the varying pieces	The complete algorithm

Strategy is usually more flexible. Template Method pays off when **most of the flow is common**, small pieces differ, and you want the compiler to **enforce the sequence**. They combine well: the base fixes the sequence but each step is a **strategy object** (or lambda) passed in rather than inherited — order enforced, no single-parent limit. Spring's `JdbcTemplate` (a helper around **JDBC**, Java's standard SQL-database API) owns the fixed flow (get connection, create statement, execute, handle exceptions, close resources) and takes your part as a **callback** (a function/object you hand over to be called back — `RowMapper`, `PreparedStatementSetter`), not a subclass.

Trade-offs

Pros	Cons
The shared flow is written once (DRY: Don't Repeat Yourself).	No single file tells the whole story: parent holds the order, child the pieces.
The order is guaranteed (`final`).	One parent only: two independent variations can't combine.
New variants are small.	Tight parent–child binding; changing one step's signature changes every subclass.
Abstract steps make the compiler report anything a subclass hasn't supplied.	Base classes swell until they demand many abstract methods and nobody wants to extend them.

When not to use it

- **The flow is two lines** — nothing to protect.
- **Variants must combine** (an encrypted CSV import needs both an "encrypted" and a "CSV" parent) — composition.
- **Behaviour must change after creation** — Strategy.
- **Subclasses override every step** — the flow was never shared.

Common mistakes

1. **Too many required steps** — a subclass should write two or three methods; make the rest hooks with defaults.
2. **Calling an overridable step from the base constructor.** It runs **before** subclass fields are initialised, so in Java the step sees `null` / `0` / `false`. (In C++ it is different but still wrong: during the base constructor a virtual call dispatches to the base version, never the override.) Call steps from the template method only.
3. **Omitting `final` on the template method** — a subclass can reorder or skip steps; the pattern becomes a mere agreement. (C++: make it non-virtual. Python and JavaScript can't enforce it — document and review.)
4. **Making steps `public`** — callers could run step 3 without 1 and 2. Keep them `protected` (or move reusable steps into a separate object).
5. **Violating Liskov** (Liskov Substitution Principle: a subclass must work wherever its parent is expected) in overrides — a hook that throws, a step that changes the contract.

Template Method in other languages

- **Java:** `final` template method, `protected abstract` required steps, `protected` hooks with a body. An interface `default` method can act as a template method, but an interface has no instance state and a default method can't be `final`, so any implementer may override it.
- **Python:** `abc.ABC` with `@abstractmethod`; no run-time `final`, so a comment (or `typing.final` for type checkers) is the only guard; a leading underscore marks internal steps.
- **TypeScript:** `abstract` classes give the compile-time check; plain JavaScript uses base methods that throw.
- **C++:** a non-virtual public template method calling `protected virtual` steps — the **non-virtual interface (NVI)** idiom.
- **Go and Rust** have no inheritance: a function taking an interface/trait with the varying steps — the Strategy shape.

Real-world examples

- **`java.io.InputStream`**: `read(byte[], int, int)` is built on abstract `read()`; subclasses supply the single-byte read (and override the bulk method for speed).
- **`java.util.AbstractList`**: implement `get(int)` and `size()` and inherit `iterator()`, `indexOf`, `contains`, `equals`, `hashCode`...
- **`HttpServlet.service()`** (Jakarta/Java Servlet) dispatches to `doGet` / `doPost` / `doPut`... by HTTP method; methods you don't override reject the request (405 Method Not Allowed on HTTP/1.1).
- **`AbstractQueuedSynchronizer`** (AQS — the JDK's building block behind `ReentrantLock`, `Semaphore`, `CountDownLatch`): `acquire()` is the template; subclasses implement `tryAcquire` / `tryRelease`.
- **Test frameworks**: JUnit (`@BeforeEach` / `@AfterEach`) and pytest (fixtures, `setup_method` / `teardown_method`) own set-up → test → tear-down; your hooks never call each other. **Mobile lifecycles**: Android `Activity` (`onCreate`, `onStart`, `onResume`...), iOS `UIViewController` (`viewDidLoad`, `viewWillAppear`...). **Maven/Gradle** (Java build tools): fixed phases, you plug in goals/tasks.
- **Sorting with a comparison**: `Arrays.sort` owns the algorithm and needs only the comparison. Old designs had elements supply it (`Comparable.compareTo`); modern APIs pass a `Comparator` — the Strategy form of the same idea.
- **Spring `JdbcTemplate`, `RestTemplate`, `TransactionTemplate`** — "template + callback" variants.

In the interview

> **30-second answer:** "Template Method puts an algorithm's outline in a single final method of a base class, and subclasses supply only the steps that differ. Steps a subclass must write are abstract; steps it may change are hooks with a default body; steps nobody should change are private. It's the Hollywood principle — the base class calls the subclass, not the other way round — which is how frameworks like servlets, JUnit and Android lifecycles work. It varies one step via inheritance; if I need run-time choice or several axes I prefer Strategy, or pass the steps in as callbacks like JdbcTemplate does."

Follow-ups:

- *What is a hook?* A step with a default implementation that subclasses may override (e.g. `shouldValidate()`) — optional control points.
- *What is the Hollywood Principle?* "Don't call us, we'll call you": high-level components call low-level ones at the right time; low-level components plug in but don't call up. It prevents "dependency rot"

(dependencies tangled in every direction, so nothing can change safely).

- *Why `final`?* It is the guarantee: without it a subclass could reorder or drop steps — only a gentleman's agreement.
- *Template Method vs Factory Method?* A factory method is often one step of a template method — the step that creates an object (GoF notes template methods frequently call factory methods). The course frames it more strongly — a factory method is a special template step whose subclass-supplied job is creation; take that as "usually lives inside a template method", not a strict rule: clients can call a factory method directly.
- *Template Method vs Decorator?* A decorator wraps an existing object at run time and can be layered; a template-method subclass is fixed when the object is built and can't be layered.
- *Template Method vs Builder?* Both impose an order, but a Builder director invokes steps on a **different** object (the builder it holds); a template method invokes them on **its own** instance via inheritance.
- *Constructor trap?* Never call overridable methods from a constructor; the subclass isn't initialised yet.

Self-check

1. **Q:** Which method is `final`? **A:** The template method (`run`, `prepareRecipe`).
2. **Q:** Difference between an abstract step and a hook? **A:** Abstract steps must be implemented; hooks have defaults and are optional.
3. **Q:** Name two JDK template methods. **A:** `InputStream.read(byte[],int,int)`, `AbstractList`'s methods built on `get`/`size`, AQS `acquire`.
4. **Q:** Why is `JdbcTemplate` more flexible than a classic template method? **A:** It takes the varying step as a callback object (composition), not a subclass.
5. **Q:** What goes wrong if the base constructor calls an abstract step? **A:** The subclass fields aren't initialised yet.

TL;DR

Fixed order in a `final` base method; subclasses fill steps; hooks are optional steps. Hollywood principle. Inheritance-based and compile-time — prefer Strategy/callbacks when you need flexibility.

11.5 Visitor

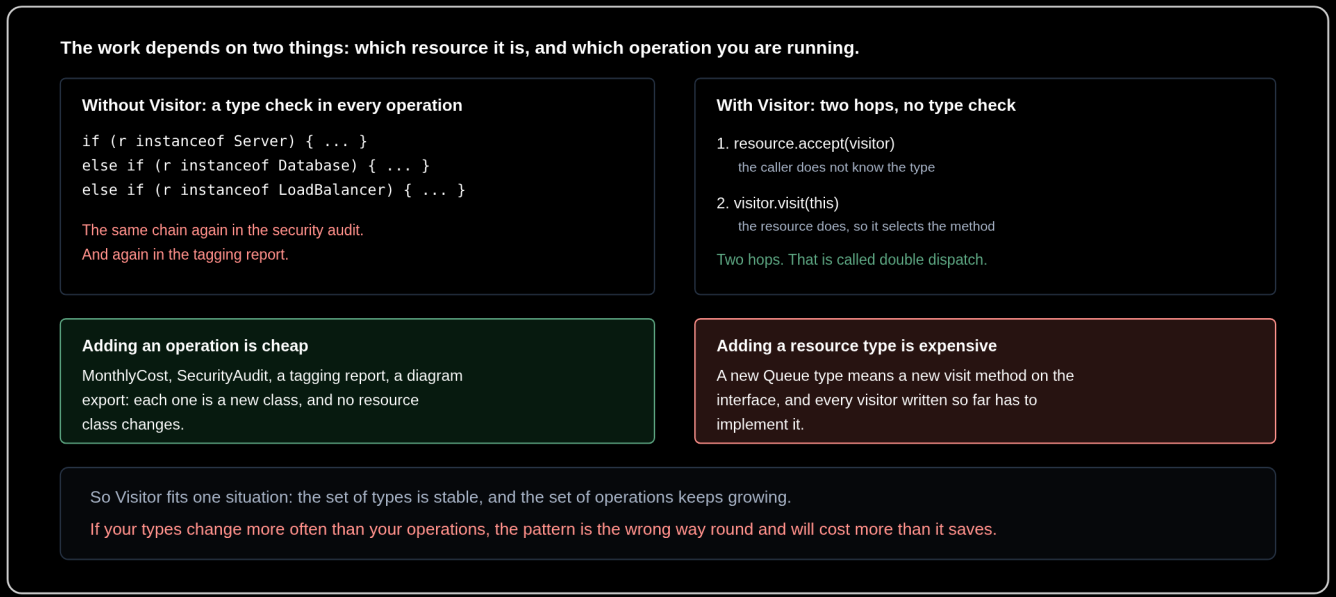


Figure — Without Visitor, every operation repeats an instanceof chain; with Visitor, two calls (accept, then visit) select the right code with no type check (course diagram)

What it is

Visitor lets you **add new operations** over a group of classes **without touching them**. Each operation is a **visitor** class with one method per element kind; each element keeps one tiny `accept(visitor)` whose body is `visitor.visit(this)`. Like a tax inspector who brings the rules for "restaurant", "shop" or "factory" — next year a fire-safety inspector visits with different rules, and the businesses don't change.

GoF intent (paraphrased): represent an operation on the elements of an object structure as a separate object, so you can define a new operation without changing the element classes.

Participant	Role	In `lld.ch11.visitor`
Visitor	Declares one `visit` operation per ConcreteElement class	`PlanVisitor`
ConcreteVisitor	Implements every `visit`; one class per operation, often accumulating a result	`MonthlyCost`, `SecurityAudit`
Element	Declares `accept(visitor)`	`Resource`
ConcreteElement	Implements `accept` as `visitor.visit(this)`	`Server`, `Database`, `LoadBalancer`, `ResourceGroup`
ObjectStructure	Holds the elements and lets a visitor reach each of them (a list or a Composite)	the `ResourceGroup` tree built in `Main`

The problem it solves

An infrastructure plan has three resource kinds — server, database, load balancer — a **stable list**. **Operations never stop arriving**: monthly bill, security review, tag report, diagram export...

- Operations **inside** the classes → `Server` holds pricing, security, tagging and drawing code; unrelated teams edit one file.
- Operations **outside** with type checks → an `instanceof` chain in each; a new resource kind means finding every chain, and the compiler won't say which you missed.

Visitor gives each operation its own class and uses **double dispatch** to pick the right method with no type check.

How it works — double dispatch

Java chooses **which override** (a subclass's replacement of an inherited method) runs from the **run-time type of the receiver** — the object before the dot (single dispatch) — but **which overload** (same-named methods with different parameter types: `visit(Server)` vs `visit(Database)`) at **compile time**, from the argument's **static type** (the declared type the compiler sees). Holding a `Resource r`, `visitor.visit(r)` can't compile to the right overload — the compiler only knows "Resource".

Visitor makes **two calls**:

1. `resource.accept(visitor)` — dispatch #1 on the **resource's** run-time type: `Server.accept` runs.
2. Inside it, `visitor.visit(this)` — `this` has static type `Server`, so the compiler picks `visit(Server)`; dispatch #2 happens on the **visitor's** run-time type (`MonthlyCost` or `SecurityAudit`).

The method that runs depends on **two** types — element and visitor: **double dispatch**.

Structure (UML)

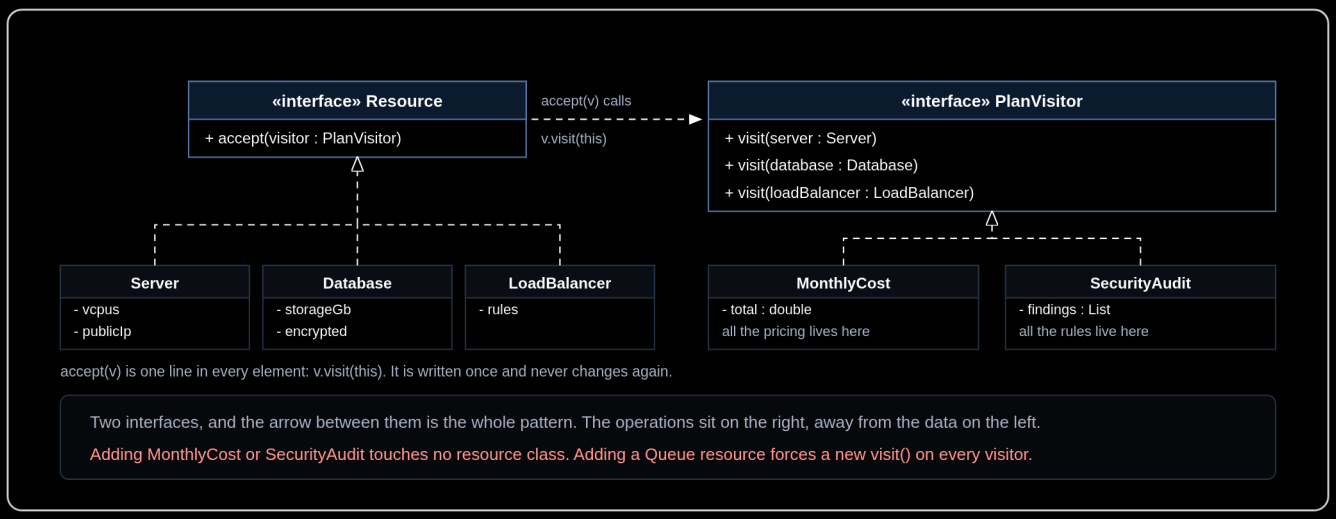
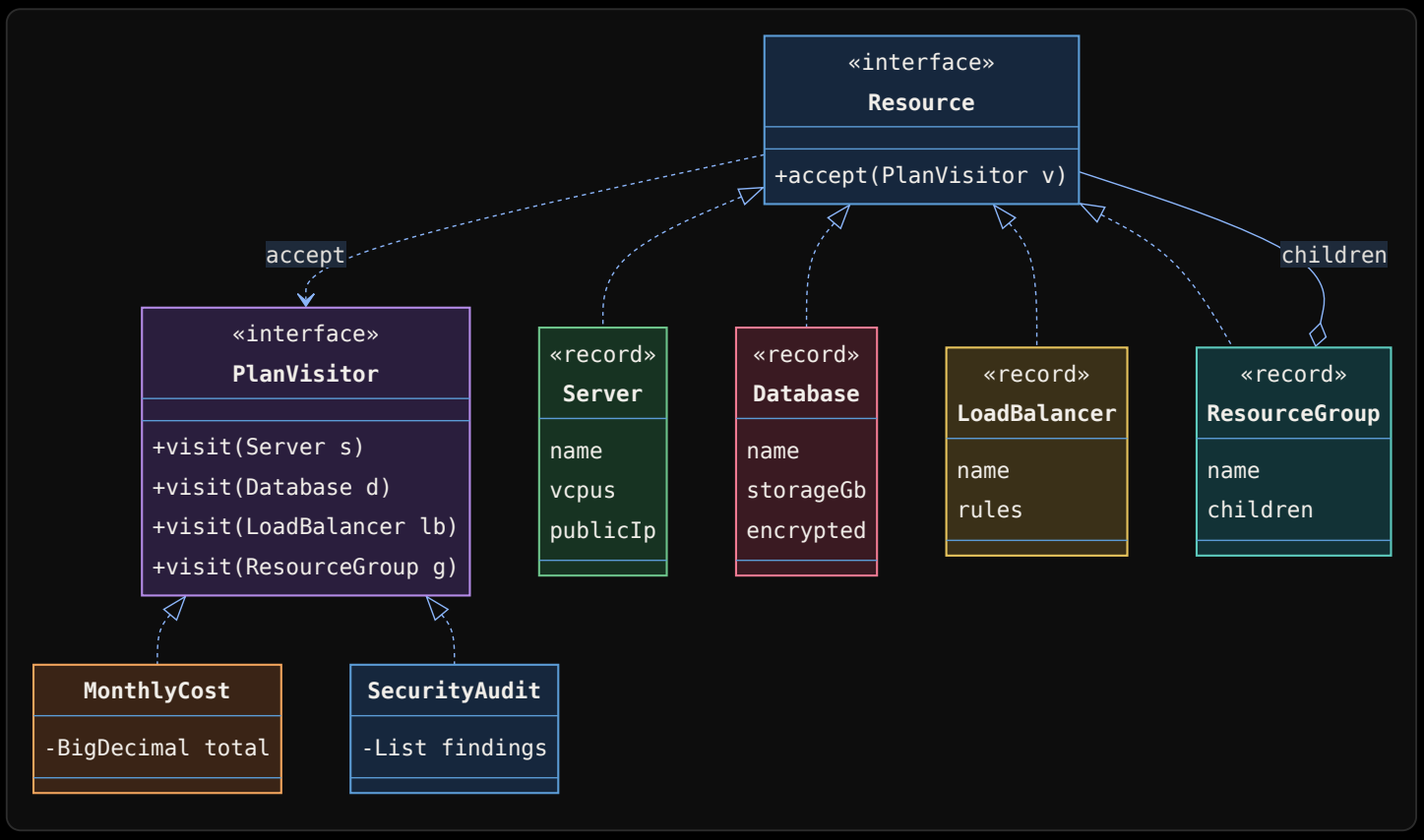


Figure — Class diagram: Resource with accept; Server, Database, LoadBalancer implement it; PlanVisitor has one visit per resource type; MonthlyCost and SecurityAudit implement it (course diagram; it shows `total : double`, our Java code uses `BigDecimal` for money)

The arrows point opposite ways: each element depends on the visitor interface (`accept` takes a visitor), while each visitor depends on every element type (a `visit` for each) — why **adding an element type is expensive**.



Java example

`code/lld/ch11/visitor/` has **both** styles over one domain:

1. **Classic Visitor.** Elements are **records** (visitors read data via accessors, not public fields). `ResourceGroup` is a **Composite** (group and single item share one interface, so a tree is handled uniformly): its `accept` visits itself then its children. (A tree is optional — Visitor needs only a stable set of element types and works over a flat list; Composite is just the most common route in.) Prices are `BigDecimal`, never `double`. `MonthlyCost` and `SecurityAudit` each keep all rules in one file and touch no resource class; a new visitor instance per run because they accumulate.
2. **Java 21 alternative — sealed interface + pattern matching** (`ModernPlan`). `Infra` is `sealed ... permits Vm, Sql, Lb`. `monthlyCost` is a `switch` with **no** `default` — the compiler checks it is **exhaustive** (covers every case), so a new permitted record breaks every switch that forgot it (Visitor's safety net). `audit` uses **record patterns** (deconstruction, `case Sql(String n, int gb, boolean enc)`) and **guards** (`when !enc`). A guarded case never counts toward exhaustiveness (the compiler can't know the guard is sometimes true), so `audit` also needs the unguarded `case Vm vm` / `Sql db` / `Lb lb` lines.

```java
/** ELEMENT: the only thing every resource must provide is accept. */
public interface Resource { void accept(PlanVisitor visitor); }

/** VISITOR: one visit per element type. This list is what makes new types expensive. */
public interface PlanVisitor {
    void visit(Server server);
    void visit(Database database);
    void visit(LoadBalancer loadBalancer);
    void visit(ResourceGroup group);
}

public record Server(String name, int vcpus, boolean publicIp) implements Resource {
    public void accept(PlanVisitor v) { v.visit(this); }   // 2nd dispatch: overload picked by static type Server
}
// Database(name, storageGb, encrypted), LoadBalancer(name, rules): the same one-line accept.

/** COMPOSITE element: visits itself, then passes the visitor to every child. */
public record ResourceGroup(String name, List<Resource> children) implements Resource {
    public void accept(PlanVisitor v) {
        v.visit(this);
        for (Resource child : children) child.accept(v);
    }
}

/** Operation 1: ALL pricing rules in one file. New instance per run (it accumulates). */
public final class MonthlyCost implements PlanVisitor {
    static final BigDecimal PER_VCPU = new BigDecimal("24.00"), DB_BASE = new BigDecimal("40.00"),
            PER_GB = new BigDecimal("0.25"), LB_BASE = new BigDecimal("18.00"), PER_RULE = new BigDecimal("1.50");
    private BigDecimal total = BigDecimal.ZERO;
    public void visit(Server s)        { total = total.add(PER_VCPU.multiply(BigDecimal.valueOf(s.vcpus()))); }
    public void visit(Database d)      { total =
total.add(DB_BASE.add(PER_GB.multiply(BigDecimal.valueOf(d.storageGb())))); }
    public void visit(LoadBalancer lb) { total =
total.add(LB_BASE.add(PER_RULE.multiply(BigDecimal.valueOf(lb.rules())))); }
    public void visit(ResourceGroup g) { /* groups are free */ }
    ...
}
// Operation 2, SecurityAudit, added later without touching a resource class: findings for a public IP,
// an unencrypted database, a load balancer with no rules, an empty group.
```

```java
/** Java 21: a SEALED hierarchy + pattern-matching switch. No accept(), no visitor interface. */
public sealed interface Infra permits Vm, Sql, Lb {}
public record Vm(String name, int vcpus, boolean publicIp) implements Infra {}
public record Sql(String name, int storageGb, boolean encrypted) implements Infra {}
public record Lb(String name, int rules) implements Infra {}

/** Operation 1 as a plain function. No 'default' branch, on purpose. Same prices as MonthlyCost. */
public static BigDecimal monthlyCost(Infra r) {
    return switch (r) {
        case Vm vm -> MonthlyCost.PER_VCPU.multiply(BigDecimal.valueOf(vm.vcpus()));
        case Sql db -> MonthlyCost.DB_BASE.add(MonthlyCost.PER_GB.multiply(BigDecimal.valueOf(db.storageGb())));
        case Lb lb -> MonthlyCost.LB_BASE.add(MonthlyCost.PER_RULE.multiply(BigDecimal.valueOf(lb.rules())));
    };
}
/** Operation 2 with record deconstruction patterns and guards. */
public static Optional<String> audit(Infra r) {
    return switch (r) {
        case Vm(String n, int cpus, boolean pub) when pub -> Optional.of(n + ": server has a public IP");
        case Sql(String n, int gb, boolean enc) when !enc -> Optional.of(n + ": storage not encrypted");
        case Lb(String n, int rules) when rules == 0 -> Optional.of(n + ": load balancer has no rules");
        case Vm vm -> Optional.empty();     // no finding
        case Sql db -> Optional.empty();
        case Lb lb -> Optional.empty();
    };
}
// Main: plan = ResourceGroup("prod", [Server web-1 4 vCPU public, Server worker-1 2, Database orders-db 200 GB
// unencrypted, LoadBalancer edge-lb 4 rules, empty ResourceGroup staging]); plan.accept(new MonthlyCost()) ...
```

Output:

```
=== 1. Classic Visitor (double dispatch) over a composite plan ===
  monthly cost: 258.00
  finding: web-1: server has a public IP
  finding: orders-db: storage not encrypted
  finding: staging: empty group

=== 2. Java 21: sealed interface + pattern matching switch ===
  monthly cost: 258.00
  finding: web-1: server has a public IP
  finding: orders-db: storage not encrypted
```

Both styles agree (96.00 + 48.00 + 90.00 + 24.00 = 258.00; the empty `staging` group is free). The modern one needs no `accept` or visitor interface; each operation is a function.

The one question that decides it: the expression problem

	Add a new operation	**Add a new type**
Methods on the classes (plain OO)	Edit every class	One new class
Visitor (or sealed + switch)	One new visitor/function	Edit the visitor interface **and every visitor** (compiler finds them)

Ask: **which changes more often — types or operations?** Settled types + growing operations → Visitor (or sealed types with pattern matching). Growing types → operations on the classes. No ordinary OO design makes both cheap: the **expression problem**.

Open/Closed trade-off: Visitor makes the element hierarchy **closed for modification but open for new operations** — at the price of being **closed to new element types** (each forces edits to all visitors).

Sealed interfaces + pattern matching (Java 17/21) as the modern alternative: sealed classes arrived in Java 17 (JEP 409 — a JEP, JDK Enhancement Proposal, is the numbered design document for a Java feature); pattern matching for `switch` and record patterns were finalised in Java 21 (JEPs 441 and 440). Together they give Visitor's main advantage, the exhaustiveness check, with far less code. The exact rules:

- A `switch` **expression** must always be exhaustive. A `switch` **statement** must be too once it uses patterns or `case null` (old-style statements over constants need not be).
- Over a `sealed` type, covering every permitted subtype with an **unguarded** pattern makes the switch exhaustive with no `default`. Guarded cases (`when ...`) don't count.
- Over a type that is **not** sealed, the compiler can't know all subtypes, so you need a `default` (or a total pattern such as `case Infra i`) — and the check is gone.
- If a new subtype appears at run time without recompiling the switch (separate compilation), the compiler-generated fallback throws `MatchException`.
- **Dominance:** cases are tried top to bottom, and a case that can never match because an earlier one covers it is a compile error — why the guarded `case Vm(...) when pub` must come **before** the unguarded `case Vm vm`.
- **`null`:** a pattern `switch` without `case null` throws `NullPointerException` on a `null` selector; `default` alone does not match `null`.

Write a classic Visitor when the element hierarchy **is not and cannot be sealed** — typically classes **owned by another team or library** that already expose `accept` (libraries such as ASM — reads and rewrites Java **bytecode**, the compiled instructions the JVM (Java Virtual Machine) runs — or javac's tree API ship their own visitor interfaces) — when you need a **walk with per-node callbacks** over a big tree, or when each operation should be a separately testable, injectable class.

Visitor outside Java. Kotlin: sealed class + exhaustive `when`. Python: `match` with class patterns, or `functools.singledispatch` (dispatch on argument type, no `accept`). C++: `std::variant` + `std::visit` — the pattern as a library feature. Rust and Swift: enums with associated values plus `match` / `switch`; a missing case is a build error. Languages without overloading (Python, JavaScript) name methods `visitServer`, `visitDatabase`, ... instead of overloading `visit`.

Trade-offs

Pros	Cons
Adding an operation = writing one class; elements stay as they are.	Adding an element type changes every existing visitor.
Related logic stays together (all pricing in one file).	Double dispatch is indirect; `accept` tells you nothing about what runs.
The compiler reports any element type a visitor hasn't handled.	Elements often must expose their data (weaker encapsulation).
Elements stay small: data + one-line `accept`.	Ceremony for small sets of types.

When not to use it

- **New types arrive often** — each edits every visitor.
- **A tiny model** (three element types, one operation) — a type check or plain method is simpler.
- **The operation concerns one type only** — it's a method.
- **Your language checks switch exhaustiveness** (Java 21 sealed types, Kotlin `when`, Rust `match`) — use that.

Common mistakes

1. Visitor with two element types.
2. **Public** element fields for visitors (use accessors/records).
3. **Reusing a stateful visitor** (`MonthlyCost`) across runs without resetting — create one per run.
4. A `default` branch in a sealed-type switch — it **turns off** the exhaustiveness check you wanted.
5. Forgetting the composite's `accept` must forward to children (or not deciding where traversal lives: element, visitor, or an Iterator).
6. **A visitor where a sealed-type `switch` would do** — with checked exhaustiveness (Java 21, Kotlin, Rust, Swift) you get the same safety for a fraction of the code.

Real-world examples

- **Compilers and linters** (tools flagging suspicious code): an **AST** (abstract syntax tree — the tree a compiler builds from source) has fixed node kinds and endless passes (type check, optimise, format, lint rules). The JDK's `com.sun.source.tree.TreeVisitor` (javac tree API) and `javax.lang.model.element.ElementVisitor` (annotation processing) are visitors; ANTLR (a parser generator) generates visitor classes for your grammar.
- **Bytecode libraries**: ASM's `ClassVisitor` / `MethodVisitor`.
- `java.nio.file.FileVisitor` with `Files.walkFileTree` — named after the pattern and often cited, but strictly a **tree-walk callback** interface (`preVisitDirectory`, `visitFile`, `visitFileFailed`, `postVisitDirectory`): no `accept`, no double dispatch over element classes. The *spirit* (operation kept outside the file-system objects) is Visitor's; the mechanism is closer to a Template Method walk calling your hooks.
- **Document models**: a **DOM** (Document Object Model — the element tree of a web page or XML document) has fixed node kinds and ever more operations — serialise to HTML/PDF, extract plain text, collect links; **infrastructure tools** computing cost, policy and diagrams over resource plans (this example); **query planners** (the database component deciding how to run a SQL query: cost estimation, rewriting, EXPLAIN).

In the interview

> **30-second answer:** "Visitor pulls an operation out of the element classes and into a separate class that has a method for each element type. Each element has a one-line `accept` that calls `visitor.visit(this)`, which gives double dispatch: the method chosen depends on both the element type and the visitor type, so no `instanceof` chains. It makes adding operations cheap and adding types expensive — the expression problem — so it fits stable type sets like ASTs. In Java 21 I'd usually use a sealed interface with an exhaustive pattern-matching switch instead, which gives the same compiler check with much less code."

Follow-ups:

- *Explain double dispatch.* Java picks overloads at compile time from static types; `accept` runs on the element's run-time type, inside it `this` has the concrete static type so the right `visit` overload is chosen; then the visitor's run-time type picks the implementation.

- *Why not just call `visitor.visit(resource)`?* With static type `Resource` the compiler can't choose `visit(Server)`; it needs `visit(Resource)`, which doesn't exist.
- *Visitor vs Iterator / Strategy / Composite / Interpreter?* Iterator decides the route, Visitor the work at each stop; a strategy is one algorithm ignoring element types, a visitor dispatches on type; Composite treats tree and node alike, Visitor adds operations over a stable type set (tree or not); Interpreter keeps evaluation **in** each node, Visitor moves it **out** (interpreters switch once a second pass arrives). Often combined.
- *How do you add a new element type?* Add it to the `visit` list (or `permits` clause) and fix every visitor/switch the compiler flags.

Self-check

1. **Q:** What is the body of every `accept` method? **A:** `visitor.visit(this)`.
2. **Q:** What change is cheap with Visitor, and what is expensive? **A:** New operations are cheap; new element types are expensive.
3. **Q:** What Java 21 features replace most visitors? **A:** Sealed interfaces + pattern-matching `switch` (with record patterns), checked for exhaustiveness.
4. **Q:** Why no `default` in `monthlyCost`'s switch? **A:** A `default` would hide missing cases; without it the compiler forces every permitted type to be handled.
5. **Q:** Where does the traversal live for a composite? **A:** Here, in the composite's `accept` (it visits itself, then its children).

TL;DR

Operations become visitor classes; elements just `accept`; double dispatch picks the method. Cheap new operations, expensive new types (expression problem). In Java 21, prefer **sealed interfaces + exhaustive pattern matching**.

11.6 GoF behavioral patterns: quick-reference summary

Key features, when to apply each, an example, pros and cons (from the course summary plus this book).

Pattern	Distinctive feature	Apply when	Example	Pros	Cons
Chain of Responsibility	Passes a request along a chain of handlers	Several objects may handle a request and the handler isn't known in advance	Help systems, HTTP middleware (code run before/after each web request)	Decouples sender and receiver	A request may go unhandled; long chains can be inefficient and slow to debug
Command	Wraps a request as an object	Actions must be queued, logged, retried or undone	GUI actions, job queues	Decouples invoker from receiver; undo/redo	Many small command classes
Interpreter	One class per grammar rule	Evaluating sentences of a small language	Regex, rule DSLs (small domain-specific languages)	Easy to extend the grammar	Unmanageable for large grammars
Iterator	Sequential access without exposing representation	You need a standard way to traverse any collection	Browsing a playlist	Simple client code; hides the structure	Hidden costs; state handling in lazy iterators
Mediator	Centralises communication between peers	Many objects interact in tangled ways	Chat application, air traffic control	Fewer dependencies; reusable components	The mediator can become a god object (one class that knows and does too much)
Memento	Captures and externalises state without breaking encapsulation	Undo, checkpoints, rollback	Editor undo, save games	Preserves encapsulation; atomic restore	Memory-heavy for big/frequent snapshots
Observer	Notifies dependents of state changes	A change in one object must reach others	Live stock/data feeds	Subject and observers are loosely coupled	Leaks, unexpected performance costs, no ordering
State	Behaviour changes with internal state	An object's behaviour depends on a state that changes at run time	Game modes, order lifecycles	Localises state-specific behaviour	More classes; graph spread across files
Strategy	Algorithm chosen at run time	Several ways to do one task, chosen dynamically	Compression algorithms, retry policies	Swap algorithms; isolate each	More classes; someone must choose
Template Method	Algorithm skeleton in a base class, steps in subclasses	The structure is fixed but some steps vary	Frameworks' lifecycles	Code reuse; enforced order	Inheritance rigidity; harder to follow
Visitor	Separates operations from the objects they work on	Many operations over a stable set of types	Processing document elements, compiler passes	Easy to add operations; related code together	Adding element types is hard; bloated visitor interface

---

11.7 Beyond GoF: patterns every LLD interview expects

The GoF book (1994) is not the whole story. These come from later sources — *Patterns of Enterprise Application Architecture* (Fowler, 2002), *Domain-Driven Design* (Evans, 2003 — DDD, shaping code around the business domain and its vocabulary), *Head First Design Patterns*, and decades of framework practice — and appear in almost every case study in this book.

11.7.1 Null Object

What it is

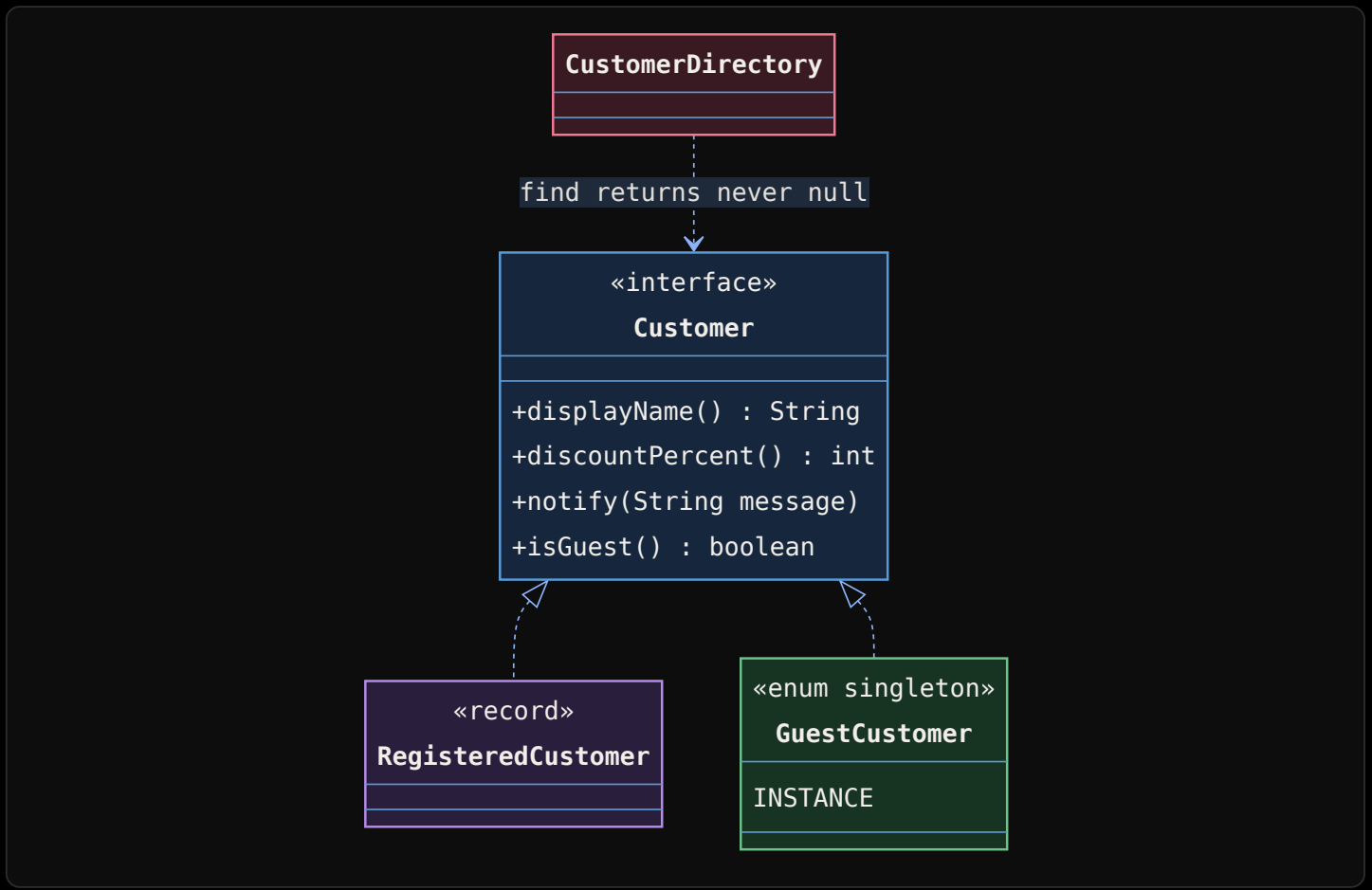
A **Null Object** is a real object implementing an interface with **neutral, do-nothing behaviour**, used instead of `null` to mean "no object here", so callers need no `if (x != null)` checks — like a "guest" badge that walks the lobby like anyone else but opens no special doors.

The problem it solves

Returning `null` for "not found" forces **every caller** to remember a null check; one forgotten check → `NullPointerException` (NPE — thrown when calling a method on `null`) in production, and the checks clutter business logic.

How it works

1. An interface (`Customer`).
2. The real class (`RegisteredCustomer`) and a **null class** (`GuestCustomer`) returning neutral values (0% discount) or doing nothing (empty `notify`).
3. Return the null object where you'd return `null`. Stateless null objects can be a **singleton** (one shared instance — here an `enum` constant).
4. If a caller truly must know, offer `isGuest()` / `isNull()` — sparingly.



Java example

Full code: `code/lld/ch11/nullobject/`.

```java
public interface Customer {
    String displayName();
    int discountPercent();
    void notify(String message);
    default boolean isGuest() { return false; }
}
// RegisteredCustomer(name, discountPercent, email) is a record; notify prints an email.

/** NULL OBJECT: a real, do-nothing implementation with neutral values. Stateless -> enum singleton. */
public enum GuestCustomer implements Customer {
    INSTANCE;
    public String displayName() { return "guest"; }
    public int discountPercent() { return 0; }            // neutral value
    public void notify(String message) { }                // does nothing, on purpose
    @Override public boolean isGuest() { return true; }   // for the rare caller that must know
}

// CustomerDirectory
/** Style 1: never return null; return the Null Object instead. */
public Customer find(String id) { return byId.getOrDefault(id, GuestCustomer.INSTANCE); }
/** Style 2: make absence explicit in the type; the caller chooses the fallback. */
public Optional<Customer> lookup(String id) { return Optional.ofNullable(byId.get(id)); }

// Main.checkout — no "if (customer != null)" anywhere: the Null Object answers every call safely.
BigDecimal payable = price.multiply(BigDecimal.valueOf(100 - c.discountPercent()))
                          .divide(HUNDRED, 2, RoundingMode.HALF_EVEN);
System.out.printf("  %-6s pays %s%n", c.displayName(), payable);
c.notify("thanks for your order");
```

Output:

```
=== Null Object ===
  Asha   pays 90.00
    email to asha@example.com: thanks for your order
  guest  pays 100.00
=== Optional (absence visible in the signature) ===
  lookup(unknown-id) present? false -> using guest
```

`checkout` has no null checks, and the guest's `notify` silently does nothing. (Prices are `BigDecimal` with an explicit rounding mode, never `double`.)

Trade-offs, when not to use it, mistakes

Pros	Cons
Removes repetitive null checks; fewer NPEs.	Can **hide real errors** — a missing object that *should* exist fails silently.
Polymorphism instead of conditionals.	One more class per interface.
Easy to test; a natural default.	Callers may not realise they got a "nothing" object.

- **Don't use it** when absence is an **error** (throw) or the caller must decide what absence means (return `Optional<T>` — a JDK container holding a value or empty).
- **Null Object vs `Optional`:** Null Object hides absence behind neutral behaviour; `Optional` makes absence **visible in the type** and lets each caller choose (`orElse`, `orElseThrow`). `Optional` for lookup return values; Null Object for legitimately optional **collaborators** (a logger, listener, discount policy).
- **Common mistakes:** a null object with **side effects** or non-neutral values; a mutable null-object singleton; overusing `isNull()` checks (the conditionals come back).

Real-world examples

`Collections.emptyList()` / `emptyMap()` (empty, not null); `InputStream.nullInputStream()`, `OutputStream.nullOutputStream()`, `Reader.nullReader()`, `Writer.nullWriter()` (Java 11); SLF4J's (a common Java logging API) `NOPLogger`; a "no-op" (does-nothing) `MeterRegistry`/tracer in **observability** libraries (metrics, logs and traces showing what a running system does).

In the interview

> **30-second answer:** "Null Object replaces `null` with an implementation that does nothing or returns neutral values, so callers never need null checks — like returning an empty list instead of null. I use it for

optional collaborators like loggers or discount policies, and `Optional` for lookups where the caller should decide. I avoid it when absence is actually an error, because it would hide the bug."

Follow-up — is it a GoF pattern? No; Bobby Woolf described it (*Pattern Languages of Program Design 3*), and Fowler's *Refactoring* has it as "Introduce Special Case" (earlier "Introduce Null Object").

Self-check. Q: What does `Collections.emptyList()` have to do with this? A: It's a null object for lists — safe to iterate, never null. Q: When should you throw instead? A: When the object must exist and its absence is a bug.

TL;DR: Return a harmless "nothing" object instead of `null`; use `Optional` when the caller must choose; throw when absence is a bug.

11.7.2 Dependency Injection (DI)

What it is

Dependency Injection: a class **receives** its dependencies from outside instead of **creating** them with `new` or looking them up — the most common way to implement **Inversion of Control (IoC)**, where the framework or a "composition root" builds and wires objects, not the objects themselves. Martin Fowler settled the name in his 2004 article on IoC containers. Like a kitchen supplied with vegetables rather than growing them: change supplier, recipes unchanged.

The problem it solves

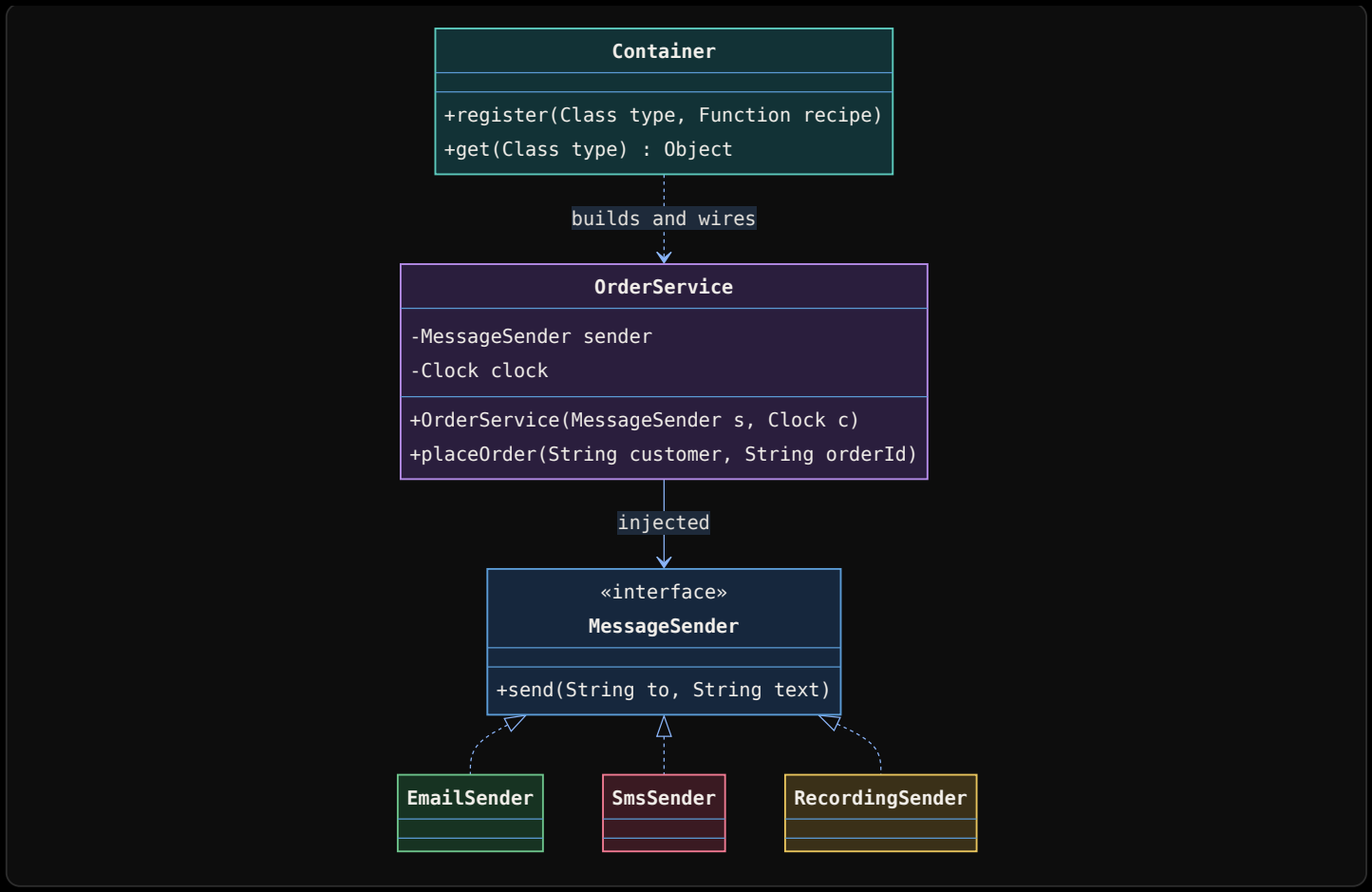
```
class OrderService {
    private final EmailSender sender = new EmailSender();   // hard-wired
}
```

It can never send SMS, can't be unit-tested without really sending email, and hides its dependencies. DI fixes all three: depend on an **interface** (`MessageSender`), receive it through the **constructor**, let someone else choose the implementation.

How it works

1. Declare dependencies as **constructor parameters** typed by **interfaces** (Dependency Inversion Principle, Chapter 4).
2. A **composition root** (usually `main`, or a DI container at start-up) creates concrete objects and passes them in.
3. Tests pass **fakes** (simple stand-ins used only in tests).

Style	How	Use for
Constructor injection	Dependencies are constructor parameters, stored in `final` fields	Required dependencies — the default choice
Setter injection	`setX(...)` after construction	Optional dependencies or ones that can change
Field injection	Framework writes directly into fields (`@Autowired` annotation on a field)	Avoid: hides dependencies and makes testing harder



Java example

Full code: `code/lld/ch11/di/`. `OrderService` receives a `MessageSender` and a `java.time.Clock` — injecting the clock makes time **testable** (`Clock.fixed`). `Container` is a toy IoC container: register a *recipe* per type; `get` builds on demand (recipes may ask the container for their own dependencies) and caches singletons.

```
/** CONSTRUCTOR INJECTION: declares what it needs, never calls 'new' on collaborators; even the clock is injected. */
public final class OrderService {
    private final MessageSender sender;
    private final Clock clock;
    public OrderService(MessageSender sender, Clock clock) { this.sender = sender; this.clock = clock; }
    public void placeOrder(String customer, String orderId) {
        LocalTime now = LocalTime.now(clock);
        sender.send(customer, "order " + orderId + " placed at " + now);
    }
}
// EmailSender and SmsSender print; RecordingSender (test double) adds "to|text" to a list —
// only possible because the dependency is injected.

/** A toy IoC container. Real ones (Spring, Guice, Dagger) add scanning, scopes, proxies and lifecycles. */
public <T> T get(Class<T> type) {
    Object existing = singletons.get(type);
    if (existing != null) return type.cast(existing);
    Function<Container, ?> recipe = recipes.get(type);
    if (recipe == null) throw new IllegalStateException("no binding for " + type.getSimpleName());
    T created = type.cast(recipe.apply(this));   // the recipe may ask the container for its own deps
    singletons.put(type, created);
    return created;
}

// Main.java
Clock fixed = Clock.fixed(Instant.parse("2026-01-15T09:30:00Z"), ZoneOffset.UTC);
new OrderService(new EmailSender(), fixed).placeOrder("asha@example.com", "A-1");   // manual DI at the composition root
c.register(OrderService.class, k -> new OrderService(k.get(MessageSender.class), k.get(Clock.class)));
```

Output:

```
=== 1. Manual DI: the composition root wires everything ===
  [smtp] to asha@example.com: order A-1 placed at 09:30
  [sms] to +91-90000-00000: order A-2 placed at 09:30

=== 2. In a unit test: inject a fake ===
  fake recorded: [ben|order T-9 placed at 09:30]

=== 3. A toy IoC container ===
  [smtp] to chen@example.com: order A-3 placed at 09:30
  same instance each time: true
```

Trade-offs, when not to use it, mistakes

Pros	Cons
Swap implementations without editing the class (OCP, DIP).	More indirection: "where does this come from?"
Easy unit tests with fakes.	Containers add "magic" (reflection — code inspecting classes at run time; proxies — generated wrapper objects) and start-up cost.
Dependencies are explicit in the constructor.	Misconfiguration may only show at start-up (or run time).

- **Don't** inject value objects or simple data (`new Money(...)` is fine), or add single-implementation interfaces "just for DI" when no test or variation needs them.
- **Common mistakes:** the **Service Locator** anti-pattern (classes call `container.get(...)` themselves — dependencies hidden again); field injection; **a constructor with 8+ dependencies** (the class does too much — split it); **circular dependencies** (A needs B while B needs A).

Real-world examples

Spring (`@Component` + constructor injection, `ApplicationContext` as container), **Google Guice**, **Dagger/Hilt** (compile-time DI on Android), **Jakarta CDI**, **Micronaut/Quarkus** (compile-time DI for fast start-up). Spring can inject a `Map<String, PaymentStrategy>` of all strategy beans — DI + Strategy together.

In the interview

30-second answer: "Dependency injection means a class receives its collaborators through its constructor, typed by interfaces, instead of creating them. It's how we apply Dependency Inversion and Inversion of Control: a composition root or a container like Spring wires the graph. The benefits are swappable implementations and easy testing with fakes — I even inject the Clock. I prefer constructor injection, avoid field injection and service locators, and treat a huge constructor as a **design smell** (a surface sign that something deeper is wrong)."

Follow-ups: **DI vs IoC?** IoC is the general idea (the framework calls you / controls object creation); DI is one way to achieve it. **DI vs Service Locator?** A locator: the class asks (hidden); DI: dependencies are handed in (visible). **DI vs Factory?** A factory creates objects on request; DI supplies ready-made dependencies — containers use factories internally.

Self-check. Q: Why inject `Clock`? A: To make time-dependent code deterministic in tests. Q: Which injection style is preferred? A: Constructor injection.

TL;DR: Don't `new` your collaborators — **receive them** (constructor, interfaces). Wire at the composition root or with a container.

11.7.3 Repository

What it is

A **Repository** is a **collection-like interface for loading and saving domain objects**, in domain language (`findByEmail`, `findActive`), that **hides how and where data is stored** (SQL — relational databases; NoSQL — non-relational key-value or document stores; files; memory). Described by Fowler (*PoEAA*) and central to Evans's *DDD*. Like a library front desk: ask for "books by this author", never mind which shelf or branch.

The problem it solves

SQL, connection handling and mapping code scattered through business classes make the rules hard to read, untestable without a database, and tied to one storage technology.

How it works

1. Define the interface in the **domain layer** (code holding business rules): `UserRepository` with `save`, `findById`, `findByEmail`, ... returning `Optional` for "maybe not found".
2. Implement it in the **infrastructure layer** (code talking to databases, networks, files): `InMemoryUserRepository` for tests, a JDBC/JPA one (JPA, the Java Persistence API, maps objects to tables) for production.
3. Services depend on the **interface** (injected — see DI).



(`JdbcUserRepository` illustrates a production implementation; the demo includes the in-memory one.)

Java example

Full code: `code/lld/ch11/repository/`.

```java
public record User(String id, String email, boolean active) {}   // domain object, knows nothing about storage

/** REPOSITORY: a collection-like interface for aggregates, in DOMAIN language. */
public interface UserRepository {
    User save(User user);
    Optional<User> findById(String id);
    Optional<User> findByEmail(String email);
    List<User> findActive();
    boolean deleteById(String id);
}

/** In memory (tests, prototypes). A JDBC/JPA one would implement the same interface. */
public final class InMemoryUserRepository implements UserRepository {
    private final Map<String, User> store = new ConcurrentHashMap<>();
    public User save(User user) { store.put(user.id(), user); return user; }
    public Optional<User> findById(String id) { return Optional.ofNullable(store.get(id)); }
    public Optional<User> findByEmail(String email) {
        return store.values().stream().filter(u -> u.email().equalsIgnoreCase(email)).findFirst();
    }
    public List<User> findActive() { return
store.values().stream().filter(User::active).sorted(Comparator.comparing(User::id)).toList(); }
    public boolean deleteById(String id) { return store.remove(id) != null; }
}

/** Business logic talks to the repository interface only: no SQL, no maps, no files. */
public String signUp(String id, String email) {
    if (users.findByEmail(email).isPresent()) return "rejected: " + email + " already registered";
    users.save(new User(id, email, true));
    return "created " + id;
}
public void deactivate(String id) { users.findById(id).ifPresent(u -> users.save(new User(u.id(), u.email(), false))); }
// Main: repo = new InMemoryUserRepository() (swap for a JDBC version in production); sign up u1, u2, u3 (ASHA@...).
```

Output:

```
created u1
created u2
rejected: ASHA@example.com already registered
active users: [User[id=u1, email=asha@example.com, active=true]]
findById(u9): Optional.empty
deleteById(u2): true
```

Trade-offs, when not to use it, mistakes

Pros	Cons
Business logic is storage-agnostic and testable in memory.	Another layer; trivial CRUD (create, read, update, delete) apps may not need it.
One place for queries per aggregate.	Can leak storage concerns (lazy loading — fetching data only when first touched; transactions — changes that succeed or fail together) anyway.
Swap storage (SQL → NoSQL) behind the interface.	"Generic repository for everything" becomes a thin, useless wrapper.

- **Don't** create one per table; one per **aggregate root** (DDD: the entry-point object of a cluster, e.g. `Order` with its `OrderLine`s).
- **Repository vs DAO:** a DAO (Data Access Object) is table/record-oriented, close to the database; a repository is domain-oriented, behaving like a collection of aggregates. In small apps they often look the same.
- **Common mistakes:** returning `null` instead of `Optional`; exposing SQL types (`ResultSet`); business rules in the repository; dozens of `findByXAndYOrZ` methods — use **Specification** (next).
- **Thread-safety note:** `ConcurrentHashMap` (the JDK's thread-safe hash map) makes each single call safe, but `SignupService.signUp` is **check-then-act** (the check's answer can be stale by the time you act) — `findByEmail`, then `save` — so two concurrent sign-ups with one email can both pass. A real system enforces uniqueness with a database **unique constraint** (rejects a duplicate column value), or an atomic `putIfAbsent` keyed by email in memory.

Real-world examples

Spring Data `CrudRepository` / `JpaRepository` (derived query methods like `findByEmail` generated from method names), Micronaut Data, Hibernate/JPA behind them; Android Room DAOs wrapped in repositories (Android architecture guidance).

In the interview

> **30-second answer:** "A repository is a collection-like interface for aggregates in domain language — save, findById, findByEmail — that hides storage. Services depend on the interface, so business logic is testable with an in-memory implementation and storage can change. In machine-coding rounds I always put an in-memory repository behind an interface, so the design is ready for a database later."

Self-check. Q: What does a repository return when nothing is found? A: `Optional.empty()` (or an empty list). Q: One repository per table? A: No, per aggregate root.

> **TL;DR:** A **domain-language collection interface** over storage; services use the interface; start in-memory, swap later.

11.7.4 Specification

What it is

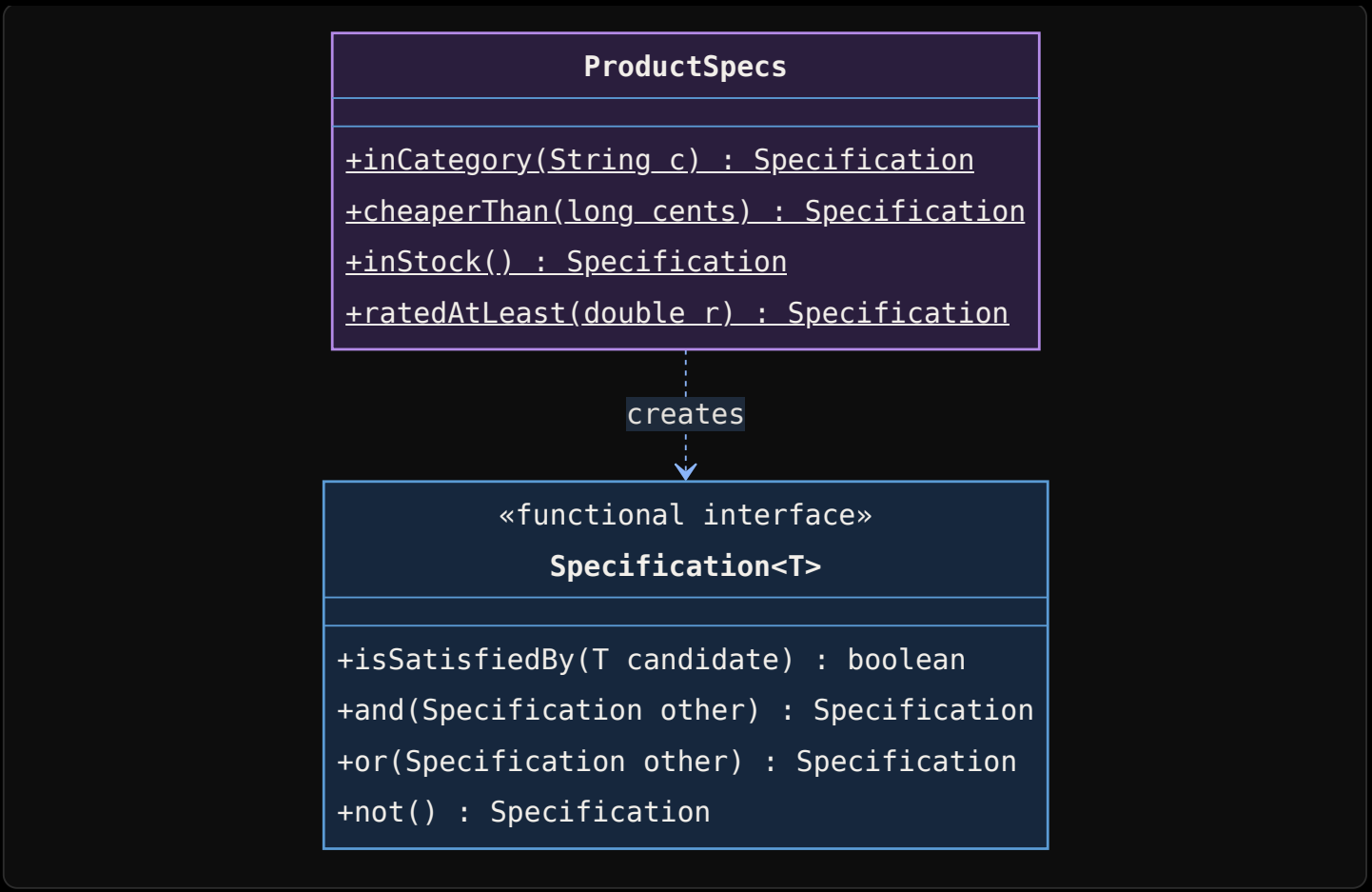
A **Specification** is a **business rule packaged as an object** with one question — `isSatisfiedBy(candidate)` — **combinable** with `and`, `or`, `not`. Introduced by Eric Evans and Martin Fowler. Like a shopping site's filter panel: "Electronics" ✓, "under ₹5,000" ✓, "in stock" ✓ — separate rules combined as ticked.

The problem it solves

"Bargain electronics = category electronics AND price < 50.00 AND in stock" gets duplicated across queries, validations and UI filters, or becomes a method explosion (`findByCategoryAndPriceLessThanAndStockGreaterThan...`). Specification gives each rule a **name**, makes it **reusable** (selection, validation, building to order) and **composable**.

How it works

1. `Specification<T>` with `isSatisfiedBy(T)`; default `and`, `or`, `not` return new composite specifications (the **Composite** pattern + **Interpreter**-like evaluation).
2. Small named factories (`inStock()`, `cheaperThan(...)`).
3. Combine where needed; pass to a repository or a stream filter.



Java example

Full code: `code/lld/ch11/specification/`.

```java
/** SPECIFICATION: a business rule as a small, named, combinable object. */
@FunctionalInterface
public interface Specification<T> {
    boolean isSatisfiedBy(T candidate);
    default Specification<T> and(Specification<T> other) { return c -> isSatisfiedBy(c) && other.isSatisfiedBy(c); }
    default Specification<T> or(Specification<T> other)  { return c -> isSatisfiedBy(c) || other.isSatisfiedBy(c); }
    default Specification<T> not()                       { return c -> !isSatisfiedBy(c); }
}

public record Product(String name, String category, long priceCents, int stock, double rating) {}

/** Named rules, in the language of the business. Each is tiny and testable alone. */
public static Specification<Product> inCategory(String c) { return p -> p.category().equals(c); }
public static Specification<Product> cheaperThan(long cents) { return p -> p.priceCents() < cents; }
public static Specification<Product> inStock() { return p -> p.stock() > 0; }
public static Specification<Product> ratedAtLeast(double r) { return p -> p.rating() >= r; }

// Main.java — the same rule objects are reused for querying AND for validation.
Specification<Product> bargainElectronics = inCategory("electronics").and(cheaperThan(5_000)).and(inStock());
Specification<Product> recommendable = ratedAtLeast(4.5).and(inStock());
Specification<Product> needsRestock = inStock().not();
// select(catalog, spec) = catalog.stream().filter(spec::isSatisfiedBy).map(Product::name).toList()
```

Output:

```
bargain electronics: [usb-cable, keyboard]
recommendable:       [keyboard, novel]
needs restock:       [headphones, cookbook]
books or top-rated:  [keyboard, novel, cookbook]
is 'novel' recommendable? true
```

Trade-offs, when not to use it, mistakes

Pros	Cons
Rules are named, reusable and testable one by one.	In-memory specs don't translate to SQL automatically — you need a query-building version (e.g. JPA Criteria) for large data.
Combining rules replaces a method explosion.	Overkill for one or two fixed filters.
The same rule serves querying and validation.	Deeply nested combinations can be hard to debug.

- Java's `Predicate<T>` (the JDK's yes/no test on a value, with `and`, `or`, `negate`) already is a specification; a named `Specification` type adds domain meaning and a place for SQL translation.
- **Common mistake:** filtering millions of rows in memory — push specifications down to the database.

Real-world examples

Spring Data JPA `Specification<T>` (`where(...).and(...)`, `JpaSpecificationExecutor.findAll(spec)`), e-commerce search filters, eligibility rules (loan, coupon, delivery-slot), access-control policies.

In the interview

> **30-second answer:** "A specification is a named business rule object with `isSatisfiedBy`, combinable with and/or/not. It removes duplicated conditions and the explosion of repository finder methods, and the same rule can filter a query and validate an object. In Java, `Predicate` is the building block; Spring Data JPA has a `Specification` that translates to SQL."

Self-check. Q: What three combinators does a specification usually have? A: `and`, `or`, `not`. Q: Which pattern does combining specifications resemble? A: Composite (and Interpreter for evaluation).

> **TL;DR: Rules as small combinable objects**; reuse them for querying and validation.

11.7.5 MVC — Model–View–Controller (the Head First "compound pattern")

What it is

MVC splits an interactive application into three roles:

- **Model** — data and business rules; knows nothing about screens.
- **View** — displays the model and receives user input.
- **Controller** — interprets input and tells the model what to change.

Created by Trygve Reenskaug for Smalltalk at Xerox PARC in the late 1970s. *Head First* calls MVC a **compound pattern** — several patterns working together:

Pattern inside MVC	Where
Observer	The model notifies views (and sometimes controllers) when its state changes; views re-render.
Strategy	The view delegates "what should this input do?" to a controller object; swap the controller to change behaviour (e.g. read-only mode).
Composite	The view is a tree of nested components (panels containing labels, lists, buttons) rendered uniformly.

Analogy: a restaurant — kitchen and pantry are the **model**, menu and plates the **view**, the waiter passing your order to the kitchen the **controller**.

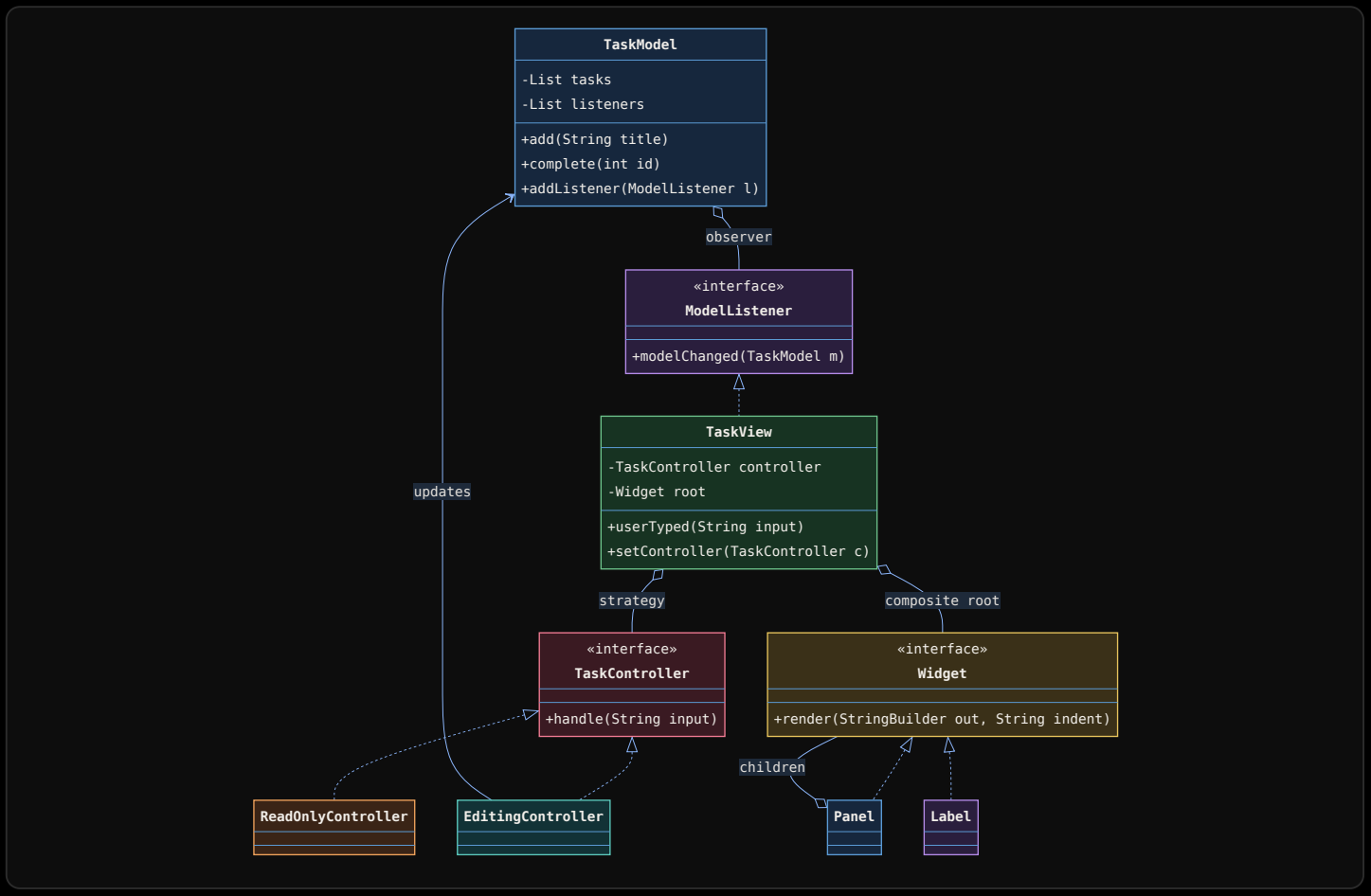
The problem it solves

With UI code, input handling and business rules mixed, you can't change the screen without risking the rules, can't show the same data two ways, and can't test the rules without a UI.

How it works

1. The user acts on the **view** (`userTyped("add ...")`).

2. The view hands the input to its **controller** (strategy).
3. The controller calls the **model** (`model.add(title)`); it does **not** redraw anything.
4. The model changes and **notifies its listeners** (observer).
5. The view re-renders its **composite** widget tree from the model.



Java example

A console to-do app (full code: `code/lld/ch11/mvc/`). `TaskModel` knows only `ModelListener`s. `TaskView` implements `ModelListener`, renders a **composite** of `Widget.Panel`/`Widget.Label`, and forwards input to a swappable `TaskController`; switching to `ReadOnlyController` changes behaviour without touching view or model.

```java
/** MODEL: state + business rules. Knows NOTHING about views or controllers, only listeners. */
public final class TaskModel {
    public void add(String title) {
        if (title.isBlank()) throw new IllegalArgumentException("title must not be blank");
        tasks.add(new Task(nextId++, title, false));
        fireChanged();
    }
    // complete(id) replaces the task with a done copy and fires, or throws "no task <id>"
    private void fireChanged() { listeners.forEach(l -> l.modelChanged(this)); }
}

/** COMPOSITE: the view is a tree of widgets rendered the same way. */
public interface Widget {
    void render(StringBuilder out, String indent);
    record Label(Supplier<String> text) implements Widget { ... }
    record Panel(String title, List<Widget> children) implements Widget {
        public void render(StringBuilder out, String indent) {
            out.append(indent).append("+ ").append(title).append('\n');
            children.forEach(c -> c.render(out, indent + "  "));
        }
    }
}

/** CONTROLLER as a STRATEGY: interprets input and updates the MODEL; it does not redraw anything. */
public void handle(String input) {
    try {
        if (input.startsWith("add ")) model.add(input.substring(4));
        else if (input.startsWith("done ")) model.complete(Integer.parseInt(input.substring(5).trim()));
        else view.showMessage("unknown command: " + input);
    } catch (IllegalArgumentException e) {        // NumberFormatException is a subclass
        view.showMessage("error: " + e.getMessage());
    }
}
// ReadOnlyController.handle: view.showMessage("read-only mode, ignored: " + input)

/** VIEW: observes the model, renders it (composite), forwards user input to its controller (strategy). */
public void userTyped(String input) { System.out.println("> " + input); controller.handle(input); }
@Override public void modelChanged(TaskModel model) {     // Observer callback: re-render from the model
    lastSeen = model;
    StringBuilder out = new StringBuilder();
    root.render(out, "  ");
    System.out.print(out);
}

// Main.java
model.addListener(view);                                  // Observer: model -> view
view.setController(new EditingController(model, view));  // Strategy: view -> controller
```

Output:

```
> add write chapter 11
  + My Tasks
    [ ] 1. write chapter 11
    + Summary
      1 open
> add review diagrams
  + My Tasks
    [ ] 1. write chapter 11
    [ ] 2. review diagrams
    + Summary
      2 open
> done 1
  + My Tasks
    [x] 1. write chapter 11
    [ ] 2. review diagrams
    + Summary
      1 open
> done 42
  (error: no task 42)
-- switching the view to a read-only controller --
> add sneaky task
  (read-only mode, ignored: add sneaky task)
```

Trade-offs, variants and mistakes

Pros	Cons
Separation of concerns; business rules testable without UI.	More classes and indirection for tiny apps.
Several views on one model (list, chart, widget).	Update flows via observers can be hard to trace.
Swap controllers/views independently.	Controllers tend to get "fat" if business logic leaks into them.

- **Web MVC is different.** In **Spring MVC** (a "Model 2" design) a **front controller** (`DispatcherServlet` — a *servlet* is a Java class handling HTTP requests inside a web server) routes each request to a `@Controller` method, which fills a model and returns a view name; the view renders once per request and does **not** observe the model — no long-lived observer link over HTTP.
- **Relatives: MVP** (Model–View–Presenter: a passive view the presenter updates) and **MVVM** (Model–View–ViewModel: the view binds to observable properties of a view-model — Android Jetpack `ViewModel` + `LiveData` / `StateFlow`, WPF). MVVM's view-model also plays a **mediator** role.
- **Common mistakes:** business logic in controllers ("fat controller"); views changing the model directly; models importing UI classes.

Real-world examples

Spring MVC, Ruby on Rails, Django (calls it MTV), ASP.NET MVC, Swing (each component has a model, e.g. `JTable` + `TableModel` with `TableModelListener`), iOS UIKit (`UIViewController`), Android (MVVM).

In the interview

> **30-second answer:** "MVC separates the model — data and rules — from the view that shows it and the controller that interprets input. Head First calls it a compound pattern: the model is an Observer subject for the views, the controller is a Strategy the view delegates to, and the view is a Composite of widgets. On the web, like Spring MVC, it becomes request-driven with a front controller, and the view doesn't observe the model."

Self-check. Q: Which three patterns make up MVC? A: Observer, Strategy, Composite. Q: Does the controller redraw the view? A: No — it updates the model, and the model's notification makes the view redraw.

> **TL;DR: Model** (state + rules, observable) → **View** (composite, observes) → **Controller** (strategy that interprets input).

11.7.6 Event Sourcing (lite)

What it is

Event Sourcing stores **every change as an immutable event** ("ItemAdded", "ItemRemoved", "CheckedOut") in an **append-only log** (you can only add at the end, never edit) and derives current state by **replaying** them — instead of storing only the latest state and overwriting it. It is here because it **borrows Command, Memento and Observer ideas at system level** (events look like recorded commands, snapshots are mementos, projections observe the log), and appears in design interviews about carts, wallets, orders and audit trails. Like a bank passbook: the transaction list is the truth; the balance can always be recomputed.

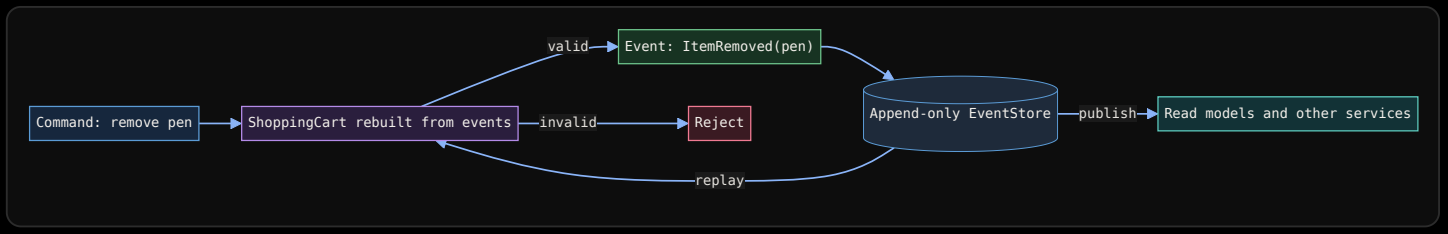
The problem it solves

With "update in place" history is lost: you can't answer "what did the cart look like before checkout?", audit who changed what, or rebuild a **read model** (a copy of data shaped for fast queries) after a bug. With events you keep full history and can **time-travel**, **audit**, and **build new views** by replaying.

How it works

1. **Command side:** load the aggregate by replaying its events; **validate** the command against current state; if valid, **emit an event** (a fact, past tense).
2. **Append** it to the store — never update or delete events.
3. **Apply**: a **pure function** (output depends only on inputs; changes nothing else) updates state from an event — no validation, no side effects; events are facts.

4. **Read side:** replay (all or the first *n* events) for state at any point; publish events to **observers** building read models (this split is **CQRS** — Command Query Responsibility Segregation).
5. For long streams, periodic **snapshots** (Memento!) and replay only newer events.



Java example

Full code: `code/lld/ch11/eventsourcing/`. A **sealed** `CartEvent` hierarchy; `ShoppingCart.apply` is an exhaustive pattern-matching `switch` (a `switch` *statement* with patterns must be exhaustive in Java 21, so a new event type `apply` forgets is a compile error); `EventStore` is an append-only in-memory log with `synchronized` methods, safe across threads. Prices are `BigDecimal`.

```
/** Facts that HAPPENED (past tense). Immutable; never updated or deleted. */
public sealed interface CartEvent permits CartEvent.ItemAdded, CartEvent.ItemRemoved, CartEvent.CheckedOut {
    record ItemAdded(String sku, int qty, BigDecimal unitPrice) implements CartEvent {}
    record ItemRemoved(String sku) implements CartEvent {}
    record CheckedOut() implements CartEvent {}
}

/** Append-only log per stream (cart id). The log IS the source of truth. */
public synchronized void append(String streamId, CartEvent e) { streams.computeIfAbsent(streamId, k -> new ArrayList<>
()).add(e); }
public synchronized List<CartEvent> read(String streamId) { return List.copyOf(streams.getOrDefault(streamId, List.of()));
}

/** ShoppingCart: aggregate whose current state is DERIVED by replaying events. */
public static ShoppingCart replay(List<CartEvent> history, int n) {    // first n events: "time travel"
    ShoppingCart cart = new ShoppingCart();
    history.stream().limit(n).forEach(cart::apply);
    return cart;
}
/** Pure state transition: no validation, no side effects. Events are facts. */
private void apply(CartEvent e) {
    switch (e) {
        case CartEvent.ItemAdded a -> lineTotals.merge(a.sku(), a.unitPrice().multiply(BigDecimal.valueOf(a.qty())),
BigDecimal::add);
        case CartEvent.ItemRemoved r -> lineTotals.remove(r.sku());
        case CartEvent.CheckedOut c -> checkedOut = true;
    }
}
/** Command side: validate against current state, THEN emit an event. */
public CartEvent decideRemove(String sku) {
    if (checkedOut) throw new IllegalStateException("cart already checked out");
    if (!lineTotals.containsKey(sku)) throw new IllegalArgumentException(sku + " not in cart");
    return new CartEvent.ItemRemoved(sku);
}

// Main.java
store.append(cartId, new CartEvent.ItemAdded("book", 2, new BigDecimal("12.99")));
store.append(cartId, new CartEvent.ItemAdded("pen", 5, new BigDecimal("0.99")));
ShoppingCart current = ShoppingCart.replay(store.read(cartId), Integer.MAX_VALUE);
store.append(cartId, current.decideRemove("pen"));      // validated, then recorded
store.append(cartId, new CartEvent.CheckedOut());
```

Output:

```
event log:
  ItemAdded[sku=book, qty=2, unitPrice=12.99]
  ItemAdded[sku=pen, qty=5, unitPrice=0.99]
  ItemRemoved[sku=pen]
  CheckedOut[]
state after each event (replay = time travel):
  after 0: [] total=0.00
  after 1: [book] total=25.98
  after 2: [book, pen] total=30.93
  after 3: [book] total=25.98
  after 4: [book] total=25.98 CHECKED-OUT
command rejected: cart already checked out
```

Trade-offs, when not to use it, mistakes

Pros	Cons
Complete audit trail and time travel.	More complex than CRUD; a learning curve for the team.
Rebuild or add read models at any time.	**Event schema evolution** (versioning old events) is hard.
Natural fit with event-driven architectures.	Reads need projections (read models built from events); eventual consistency (the read side catches up after a short delay) between write and read sides.

- **Don't use it** for simple CRUD without audit or history needs.
- **Common mistakes:** validation in `apply` (replaying old facts may then fail); events named as commands (`RemoveItem` instead of `ItemRemoved`); mutable events; derived data stored inside events; no snapshots for very long streams; **appending without an expected-version check** — two commands decided on the same old state can both append. Real event stores take an `expectedVersion` on append and reject the second writer (optimistic concurrency); the toy `EventStore` omits it.

Real-world examples

Bank ledgers and payment systems (balances derived from ledger entries); Git (its history of commits); **Axon Framework** and **EventStoreDB**; Kafka as an event log with long or unlimited **retention** (how long messages are kept) — not *compacted* topics, which keep only the latest record per key and so discard the history event sourcing needs; shopping carts and order histories where "what happened when" matters.

In the interview

> **30-second answer:** "Event sourcing stores every change as an immutable, past-tense event in an append-only log and derives state by replaying. Commands are validated against the replayed state and then produce events; `apply` just folds events into state. You get audit and time travel, and can build new read models by replaying; the costs are complexity, schema evolution and eventual consistency. Snapshots keep replay fast — that's Memento plus a command log."

Self-check. Q: Where does validation happen? A: On the command side, before emitting the event — never in `apply`. Q: How do you keep replay fast? A: Periodic snapshots.

> **TL;DR: Store facts, not state**; rebuild by replay; validate commands, never events; snapshot long streams.

11.8 Choose the right behavioral pattern

Start from the **symptom** in the code or requirement, not the pattern name.

If you see / need...	Use	Not to be confused with
A request must pass several independent checks; any may reject it; different routes need different checks	**Chain of Responsibility**	Decorator (every layer always runs)
Undo/redo, queue, schedule, retry, or log *requested actions*	**Command**	Strategy (how, not what)
Evaluate user-written rules/expressions in a small language	**Interpreter** (+ Specification)	Visitor (for many operations over the AST)
Walk a collection/tree/paged API without exposing structure	**Iterator** (`Iterable`)	Visitor (what to do at each node)
Many objects depend on each other's changes and the *order* of reactions matters	**Mediator**	Observer (no coordination)
Roll back to an earlier state without exposing internals	**Memento**	Command undo (store actions instead)
One change must notify an open-ended set of independent listeners	**Observer** (or a message broker for durability)	Mediator
Three or more methods repeat one `switch` over a status; which actions are legal depends on that status	**State**	Strategy (chosen from outside)
Several interchangeable algorithms for one job, chosen by config/user/run time	**Strategy** (often a lambda)	State, Template Method
Several classes share a long fixed flow and differ in a few steps	**Template Method** (or template + callbacks)	Strategy
Many new operations over a stable set of types	**Visitor**, or **sealed types + pattern matching** in Java 21	Iterator, Strategy
`null` checks everywhere for an optional collaborator	**Null Object** (or `Optional` for lookups)	—
Classes create their own collaborators; hard to test	**Dependency Injection**	Service Locator (anti-pattern)
Business logic mixed with storage code	**Repository**	DAO (table-oriented)
The same business rule duplicated in queries and validation; finder-method explosion	**Specification**	—
UI, input handling and business rules tangled together	**MVC / MVVM**	—
Need full history, audit, time travel of changes	**Event Sourcing** (+ snapshots)	Plain audit log

Three questions that settle the famous look-alikes:

1. **Do the alternatives reference each other?** Yes → **State**. No → **Strategy** (or Bridge if chosen once at wiring).
2. **Can one participant stop the flow?** Yes → **Chain of Responsibility**. Every participant always runs → **Decorator** / pipeline. Everyone is told, nobody stops anyone → **Observer**.
3. **Does a central object decide what happens next?** Yes → **Mediator**. Each listener decides for itself → **Observer**.

In the interview (chapter-level)

"Which patterns have you used in production?" Pick concrete ones and name the *trade-off*: "Strategy for payment methods, chosen by a map injected by Spring; Observer via Spring events for sending emails after an order, moved to Kafka when we needed retries; State for the order lifecycle, with the DB update guarded by `WHERE status = ?` to avoid races."

"When would you avoid patterns?" When only two branches exist and no third is expected; when a lambda or an enum does the job; when the indirection costs more readability than it buys flexibility. Patterns are a vocabulary for trade-offs, not a checklist.

Self-check (whole chapter)

1. **Q:** An order moves placed → accepted → preparing → delivered and "cancel" is only legal in early stages. Pattern? **A:** State (enum or class per state).

2. **Q:** Checkout must support card, UPI, wallet, and new methods next quarter. Pattern? **A:** Strategy, selected by a factory/DI map.
3. **Q:** After an order is placed, send email, update analytics and notify the restaurant; more reactions will come. Pattern? **A:** Observer in-process, or pub/sub via a broker for reliability.
4. **Q:** A compiler needs type-checking, optimisation and pretty-printing passes over a fixed AST. Pattern? **A:** Visitor, or sealed interfaces + pattern matching in Java 21.
5. **Q:** Three report generators share "load → filter → format → export" but differ in format. Pattern? **A:** Template Method (or a fixed pipeline taking a formatting strategy).
6. **Q:** What three GoF patterns make up MVC? **A:** Observer, Strategy, Composite.

TL;DR

- **Observer** broadcasts; **Mediator** coordinates; **CoR** lets someone stop the flow.
- **State** vs **Strategy**: same diagram — states know each other and advance themselves; strategies are chosen from outside.
- **Template Method** = inheritance, one step varies; **Strategy** = composition, whole algorithm varies.
- **Visitor** = cheap new operations, expensive new types; Java 21 **sealed + switch** usually replaces it.
- Beyond GoF: **Null Object**, **DI**, **Repository**, **Specification**, **MVC**, **Event Sourcing** — the everyday glue of real systems and machine-coding answers.

11.9 Architectural patterns, and how they differ from design patterns

A **design pattern** (everything above) arranges a handful of classes to solve one recurring problem inside a program. An **architectural pattern** (also called an **architectural style**) fixes the shape of the whole system: which large parts it has, which part may call or depend on which, and how they communicate (method call, network request or message). Design patterns sit *inside* the parts that an architectural pattern defines. For example, a hexagonal service (§11.9.7) uses Repository (§11.7.3) and Dependency Injection (§11.7.2) inside it.

	Design pattern	Architectural pattern
Scope	A few classes	A whole application or system
Unit	Class, interface, object	Layer, module, component, service, process
Main concern	Responsibility and collaboration between objects	Dependency direction, deployment, communication, data ownership
Cost of changing it later	A refactoring of a few files	A re-architecture, often months of work
Examples	Strategy, Observer, Repository	Layered, microservices, event-driven, hexagonal

Why an LLD interview cares. Your answer still says where the classes live ("controller, service, repository" or "domain core with ports"), and follow-ups such as "would you split this into microservices?" are architectural.

11.9.1 The styles at a glance

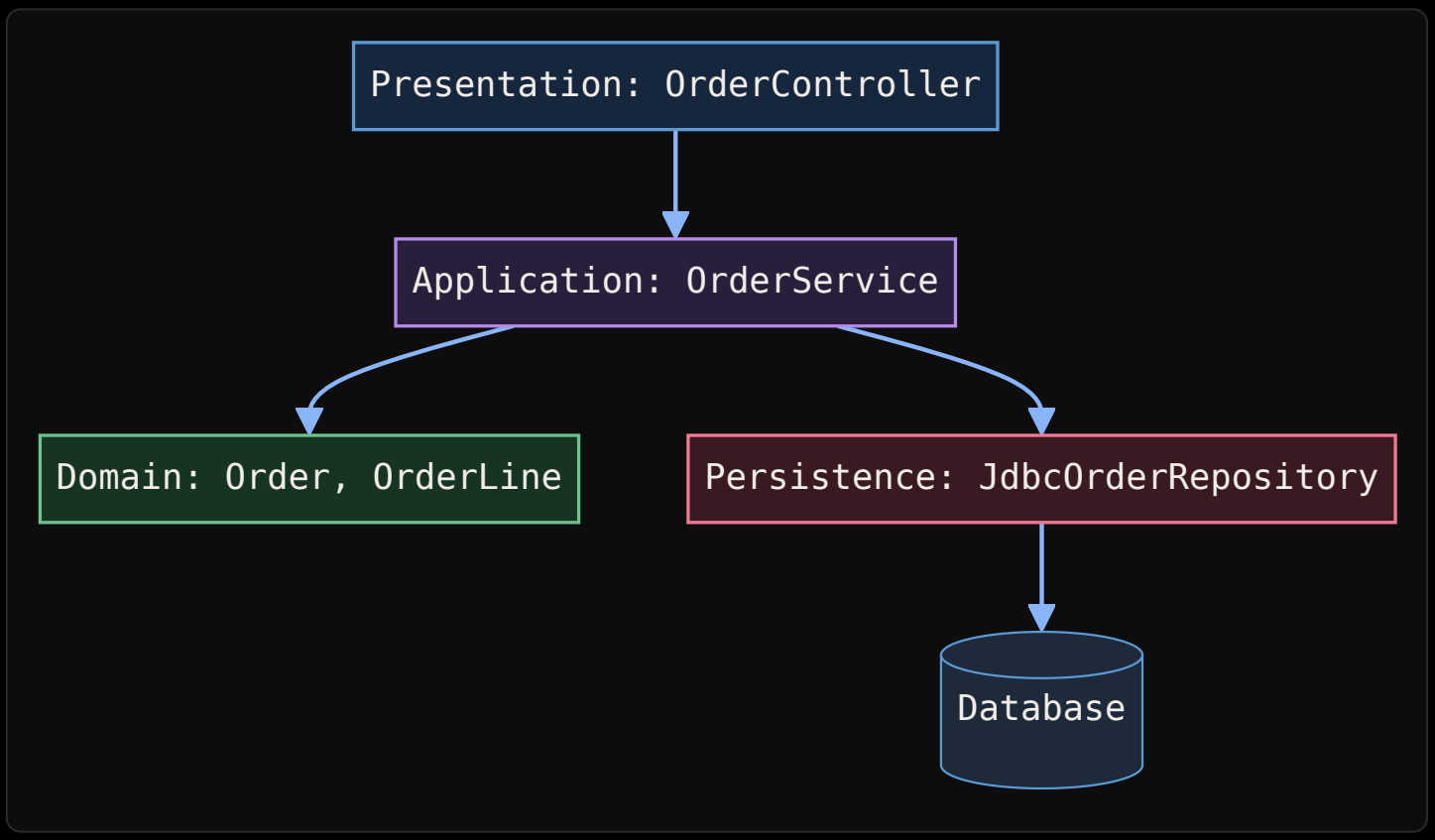
Style	Core idea	Main cost	How it shows up in an LLD answer
Layered (n-tier)	Horizontal layers; each depends only on layers below	Domain shaped by storage; features cut through every layer	`Controller -> Service -> Repository` packages
Client-server	Clients request; a central server owns data and rules	Server is a bottleneck until scaled out	The API boundary: what the server re-validates
Modular monolith	One deployable split into modules with enforced boundaries	One release train, one shared failure fate	Packages per business capability, interfaces between them
Microservices	Independently deployable services, each owning its data	Network failures, distributed data, operations	Say where you *would* split and why
Event-driven	Components react to published events instead of calling each other	Hard to trace; duplicates, ordering, eventual consistency	Observer in-process, a broker between services
Pipe-and-filter	Data flows through independent transforming steps	Shared state between steps is awkward	`Filter<I, O>` plus composition
Hexagonal / clean	Domain core in the middle; I/O behind ports and adapters	More interfaces and mapping code	No framework imports in the domain; repository and gateway ports
MVC / MVP / MVVM	Separate UI state, display and input	Extra classes for small screens	§11.7.5; only the UI edge of a system

The sections below add one diagram per style and say when to use it. §11.9.9 is one runnable program that combines hexagonal, pipe-and-filter and broker-style event-driven.

11.9.2 Layered (n-tier)

What it is. Code is split into horizontal **layers**; each layer serves the one above and depends only on layers below it. The usual four are: **presentation** (HTTP controllers, UI), **application** or service (use cases, transactions), **domain** (business rules) and **persistence** or data access (repositories, SQL). A **tier** is a *physical* separation (a separate process or machine), whereas a layer is a *logical* one. "Three-tier" usually means browser, application server and database. **Strict layering** allows calls only to the layer directly below. **Relaxed layering** allows skipping layers, for example a controller reading a repository directly. Relaxed

layering is convenient but erodes the structure. The diagram shows the common relaxed form, where the service calls both the domain and the repository.



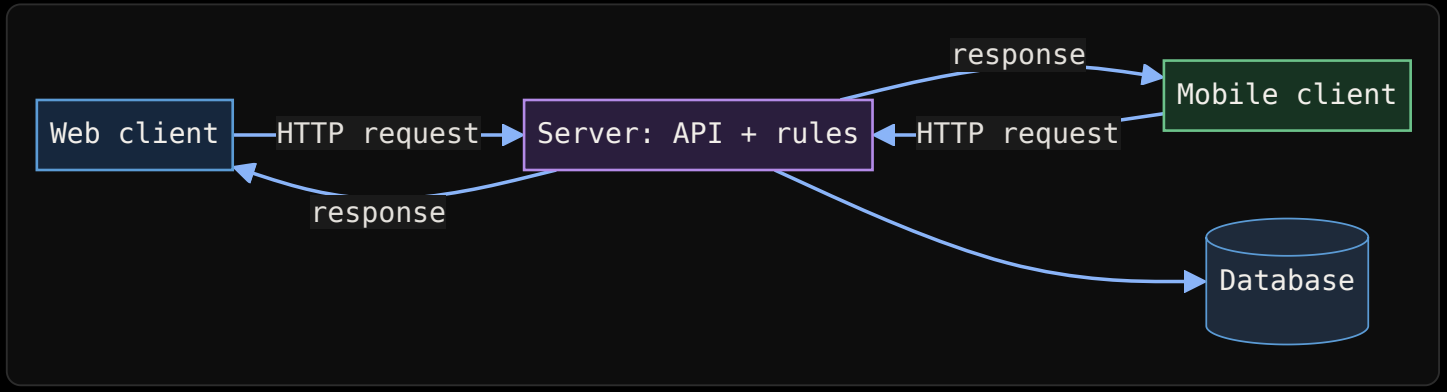
Use it when the app is mostly forms over data, the team is new to the domain, or a framework (Spring, Rails, Django) already imposes it.

Trade-offs. It is understood by everyone and gives separation of concerns for free. However, the dependency arrow points *down to the database*, so the domain ends up shaped by tables. One feature ("add a discount") touches every layer. And when services only forward calls ("sinkhole" layers), the layers add indirection without value. Hexagonal architecture (§11.9.7) keeps the layers but turns the bottom arrow around.

In an LLD interview. This is the default machine-coding layout: `controller` (or `Main`), `service` (use cases), `repository` (interfaces with in-memory implementations), `model` (rules where they belong, never in the controller).

11.9.3 Client-server (and peer-to-peer)

What it is. A **server** owns a resource (data, computation) and waits for requests. **Clients** start requests and present results. The two communicate through a protocol (HTTP, gRPC, a database wire protocol). The server is the single source of truth and the place where rules are enforced. Never trust validation done only on the client. The contrasting style is **peer-to-peer (P2P)**: every node is both client and server, as in BitTorrent or a blockchain. P2P removes the central bottleneck, but it makes consistency, discovery and trust much harder.



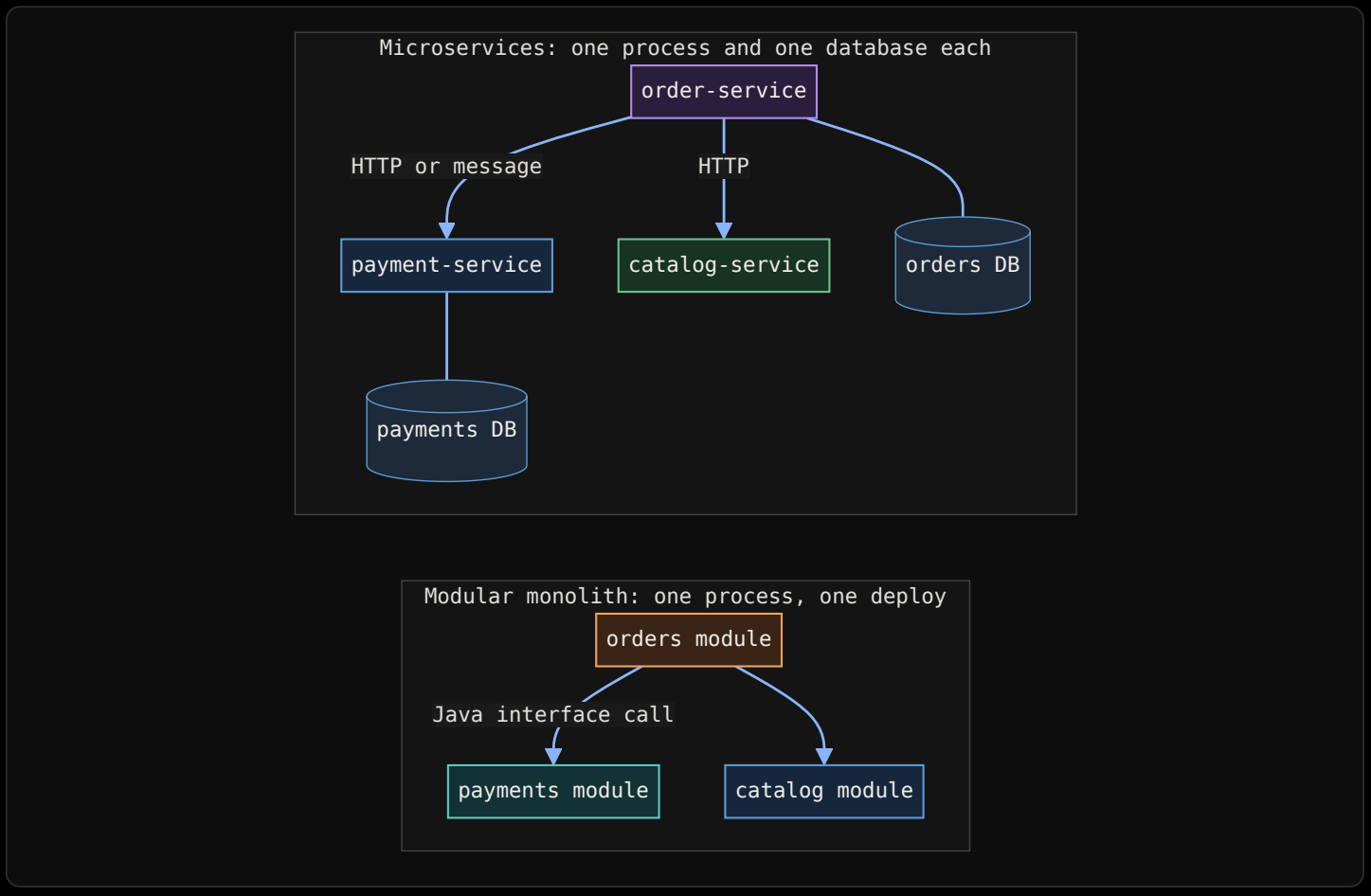
Use it when many users share data that needs one authority (almost every business app). P2P is used for file sharing, sync without a central host, and decentralised ledgers.

Trade-offs. Central control makes rules, security and backups simple. The server can be scaled out (several instances behind a load balancer), but it is a bottleneck and a failure point until you do so. **Stateless** servers (no per-client memory between requests; session data lives in a token or a shared store) are much easier to scale than stateful ones.

In an LLD interview. Client-server mostly appears as the API boundary of the system you design. Examples: "the client sends `bookSeat(showId, seatId)` and the server re-checks availability under a lock", or "the elevator panel is a client of the controller". State which side owns the truth and which side validates.

11.9.4 Monolith, modular monolith and microservices

What they are. A **monolith** is one deployable unit (one JAR, one process) containing all features. A **modular monolith** is still one deployable, but it is divided into **modules** by business capability (orders, payments, catalogue). Each module exposes a small public interface and hides everything else. Its boundaries are enforced by packages, Java modules (JPMS, the Java Platform Module System) or build rules. **Microservices** split the system into independently deployable services. Each runs in its own process, owns its own database (no other service reads its tables) and talks to the others over the network. The usual split follows a **bounded context**, a domain-driven-design term for a part of the business where words have one consistent meaning.



Use a modular monolith by default. It gives most of the structural benefit at none of the network cost. **Move a module out into a service** only when you have a concrete reason: a separate team that needs to release independently, very different scaling (a search module needing 50 instances while the rest needs 2), a different technology, or fault isolation.

Trade-offs of microservices.

- **Gains:** independent deploys and scaling, smaller codebases per team, and a crash in one service does not take down the others.
- **Costs:** every call can be slow or fail, so you need timeouts, retries and **idempotency** (repeating a request has no extra effect). There are no cross-service database transactions, so a multi-step business action needs a **saga** (a series of local transactions, each with a compensating undo step). You also take on **eventual consistency** (other services see a change after a delay), distributed tracing, and many more

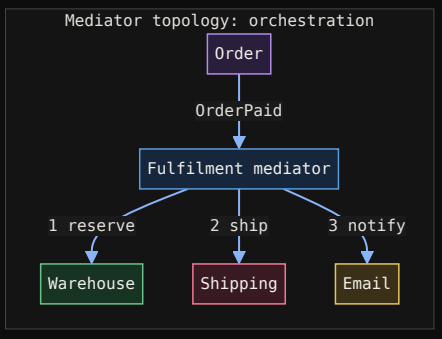
deployments to run. A **distributed monolith** is the worst case: services that must be deployed together because they share a database or call each other synchronously in long chains.

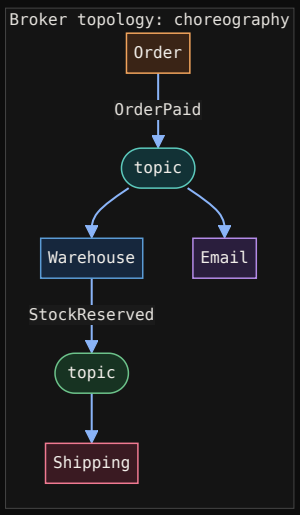
In an LLD interview. You design one service or a modular monolith. When asked "how would this scale to microservices?", point to your module interfaces (`PaymentGateway`, `NotificationSender`) and say that each could become a network client with the same interface, adding timeouts, retries and idempotency keys. That is the payoff of designing to interfaces.

11.9.5 Event-driven architecture: broker vs mediator topology

What it is. Components communicate by producing and consuming **events** (immutable facts such as `OrderPaid`) instead of calling each other directly. A producer does not know who consumes its events, so new reactions can be added without touching it. This is Observer (§11.1) lifted to system level. Between processes, a **message broker** (Kafka, RabbitMQ, a cloud queue) stores and delivers the events. Chapter 20 §20.6 designs such a pub-sub system. Event-driven systems come in two **topologies** (shapes of how events flow):

- **Broker topology.** There is no central coordinator. Each consumer subscribes to the events it cares about, does its work and may publish new events, so the flow emerges as a chain. Highly decoupled and scalable, but no single place knows the whole flow, and error handling is spread out. Also called **choreography**.
- **Mediator topology.** A central **mediator** (an orchestrator or workflow engine) receives the initial event and sends ordered commands to each step ("reserve stock", then "charge card", then "ship"). It decides what happens next and handles failures and compensation. The flow is easier to see and control, but the mediator couples to every step and can become a bottleneck. Also called **orchestration**. This is Mediator (§10.4) at system level.





Use it when one action has many independent reactions (notifications, analytics, search indexing), when systems must be integrated loosely, or when work should continue after the user gets a response. Choose the **broker** topology for independent reactions, and the **mediator** for a multi-step business process with ordering and compensation, such as order fulfilment or loan approval.

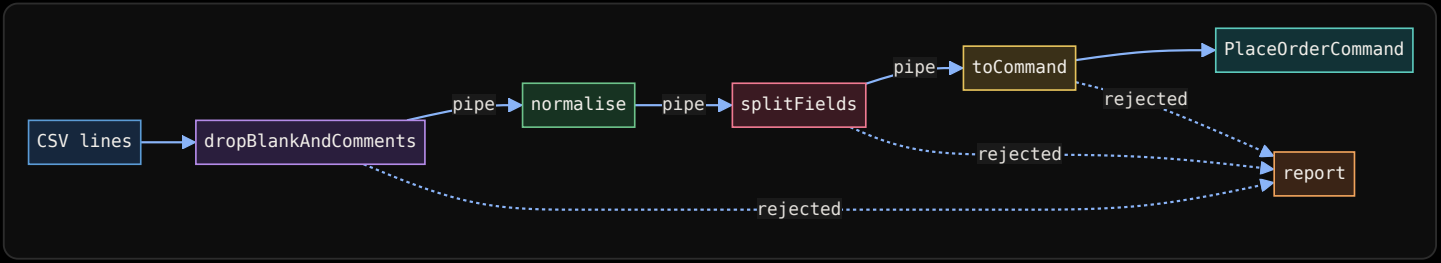
Trade-offs.

- **Gains:** loose coupling, easy extension, consumers that scale and fail independently, and a natural audit trail when combined with Event Sourcing (§11.7.6).
- **Costs:** the call flow is invisible in the code. Most brokers deliver **at least once**, so consumers must be idempotent. Ordering is guaranteed only per partition or key. Consumers see changes late (eventual consistency). And you need **dead-letter queues** (where messages that keep failing are parked for inspection).

In an LLD interview. In-process, it is an `EventBus` or Observer with typed events. Between services, it is a broker with topics. The §11.9.9 demo is a broker-topology bus behind an `EventPublisher` port. Name the delivery guarantee and the idempotency strategy, for example "consumers keep processed event IDs".

11.9.6 Pipe-and-filter

What it is. Processing is split into **filters**: independent steps that each take an input, transform, enrich, validate or drop it, and emit an output. Filters are connected by **pipes** that carry each filter's output to the next filter. A filter knows nothing about its neighbours, only the data format on its input and output, so filters can be reordered, reused and tested alone. Pipes can be plain function composition (synchronous), or bounded queues between threads (each filter on its own thread, with **backpressure**, meaning a full queue slows down the producer). Unix shell pipelines (`grep | sort | uniq`), Java Streams, compiler phases and servlet filter chains are all this style.



Use it when the work is a sequence of transformations on a stream of items: parsing and import, ETL (extract, transform, load), log processing, image or audio processing, request middleware.

Trade-offs. Each filter is simple, testable and replaceable, and with queues the stages run in parallel. The costs: steps that need shared context (for example "reject duplicates across lines") need a stateful filter or a side store; interactive request-reply flows fit poorly; and each step adds conversion or queue overhead.

Versus Chain of Responsibility (§10.1). In a CoR, each handler may *stop* the request, and usually one handler handles it. In a pipeline, *every* filter normally runs and *transforms* the data, and a rejection is just one possible output. Versus Decorator (§9.4): a decorator wraps one object with the *same* interface, whereas filters may change the data type at each step (`String -> List<String> -> PlaceOrderCommand`).

In an LLD interview. It appears in log processors, file-import jobs, request validation, rate-limiter middleware and the logging-framework case in Chapter 20 §20.5. Model it as `interface Filter<I, O>` with a composition method, as in §11.9.9.

11.9.7 Hexagonal architecture (ports and adapters) and clean architecture

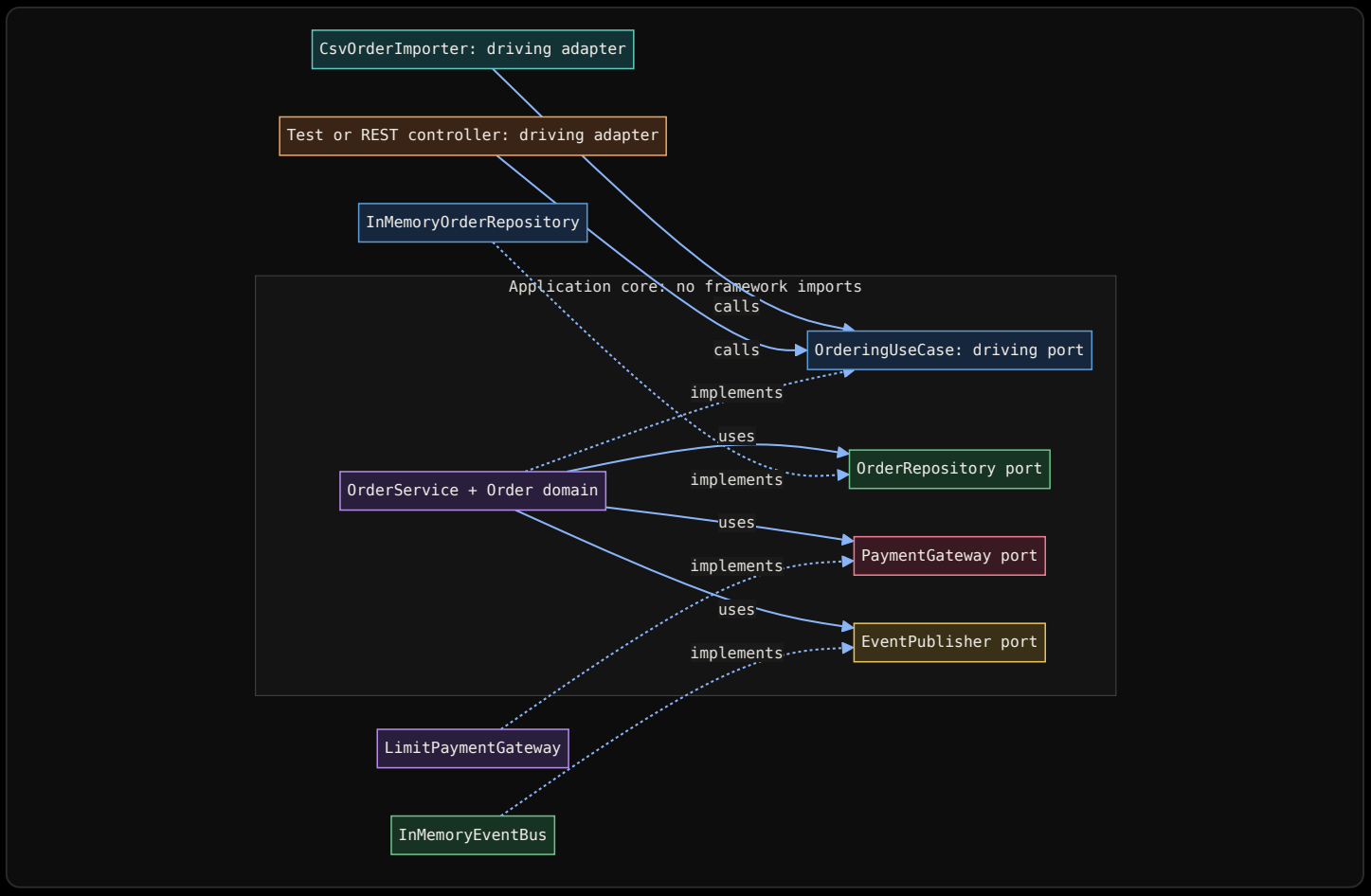
What it is. **Hexagonal architecture** (Alistair Cockburn, 2005) puts the **application core** (domain model plus use cases) in the middle and pushes all I/O to the edge. The core talks to the outside only through **ports**, which are interfaces *owned by the core*. **Adapters** implement or call those ports for one specific technology. There are two kinds of each:

- **Driving (primary, inbound) side.** Something outside *drives* the app: an HTTP controller, a CLI, a CSV importer, a test. The core declares a **driving port** (a use-case interface such as `OrderingUseCase`), and the driving adapter calls it.
- **Driven (secondary, outbound) side.** The app needs something outside: a database, a payment provider, a message broker. The core declares a **driven port** (such as `OrderRepository` or `PaymentGateway`), and a driven adapter implements it.

All source-code dependencies point *inward*. Adapters depend on ports, and the core imports no adapter, framework or I/O class. This is the Dependency Inversion Principle (Chapter 4 §4.10) applied to a whole

application. A **composition root** (one place, usually `main` or the DI container's configuration) wires concrete adapters to the core.

Clean architecture (Robert C. Martin, 2012) and **onion architecture** (Jeffrey Palermo, 2008) are the same idea drawn as concentric circles: entities, then use cases, then interface adapters, then frameworks and drivers. They share one **dependency rule**: source code may depend only on circles further in. Treat hexagonal, onion and clean as one style with different vocabulary.



Use it when the business rules are valuable and long-lived, when you want fast tests without databases or networks, or when infrastructure is likely to change (swap the database, add a second payment provider, add a gRPC entry point next to REST).

Trade-offs.

- **Gains:** the core is testable with in-memory adapters, technology decisions can be postponed or reversed, and every new entry point is just another adapter.
- **Costs:** more interfaces and **mapping code** (converting between adapter types such as JSON or table rows and core types). A trivial CRUD app gains little. Ports designed around one vendor's API ("leaky ports") bring the coupling back.

Hexagonal vs layered. Both separate concerns. In layered architecture the domain depends *down* on persistence. In hexagonal, persistence depends *in* on the domain's port, and "top" and "bottom" become simply "driving side" and "driven side".

In an LLD interview. This is what a strong machine-coding answer usually *is*, even when unnamed: domain classes with no I/O, `XxxRepository` and `XxxGateway` interfaces, in-memory implementations, and a `Main` that wires them together. Say the name and the dependency rule out loud. It shows that you know why the interfaces exist.

11.9.8 Where MVC, MVP and MVVM fit

MVC, MVP and MVVM (§11.7.5) are **presentation patterns**: they organise the *UI edge*, not the whole system. In a layered system MVC is the presentation layer (Spring MVC's "model" is often just data for a template; the rules live below). In hexagonal terms a controller, presenter or view-model is a **driving adapter** that turns user gestures into calls on a driving port. So "MVC vs hexagonal" is not a choice: MVC at the edge, hexagonal for the core.

11.9.9 Java example: a hexagonal order service fed by a pipe-and-filter importer

One small program shows three styles working together:

- **Hexagonal.** `domain` holds `Order` and its rules. `port` holds the interfaces the core owns: one driving port (`OrderingUseCase`) and four driven ports. `app.OrderService` implements the use case using only ports and domain. `adapter` holds the technology-specific code. `Main` is the composition root. The diagram in §11.9.7 uses these exact names.
- **Pipe-and-filter.** The driving adapter `CsvOrderImporter` parses CSV lines through four `Filter`s composed with `then()`. Syntax errors are dropped by the filters. Business errors (unknown product, declined payment) are decided by the core.
- **Event-driven (broker topology).** The core publishes `OrderPaid` or `PaymentFailed` through the `EventPublisher` port. `InMemoryEventBus` delivers each event to whoever subscribed, so the warehouse, email and audit handlers are unknown to `OrderService`.

The dependency rule can be checked mechanically. No file in `domain`, `port` or `app` imports anything from `adapter` or `pipeline`: `grep -r "architecture.adapter\|architecture.pipeline" domain port app` prints nothing.

Full code: `code/lld/ch11/architecture/` (packages `domain`, `port`, `app`, `adapter`, `pipeline`, plus `Main`). The essential lines:

```java
// domain — pure Java: no imports from ports, adapters, frameworks or I/O.
/** Value object: one priced line of an order. Immutable, compared by value. */
public record OrderLine(String sku, int qty, BigDecimal unitPrice) {
    public OrderLine {
        if (qty <= 0) throw new IllegalArgumentException("quantity must be positive for " + sku);
        if (unitPrice.signum() < 0) throw new IllegalArgumentException("negative price for " + sku);
    }
    public BigDecimal lineTotal() { return unitPrice.multiply(BigDecimal.valueOf(qty)); }
}
public enum OrderStatus { PENDING_PAYMENT, PAID, PAYMENT_FAILED }

public final class Order {   // DOMAIN entity
    /** Factory that enforces the invariant "an order has at least one line". */
    public static Order place(String id, String customerId, List<OrderLine> lines) {
        if (lines.isEmpty()) throw new IllegalArgumentException("an order needs at least one line");
        return new Order(id, customerId, List.copyOf(lines));
    }
    public BigDecimal total() { return lines.stream().map(OrderLine::lineTotal).reduce(BigDecimal.ZERO, BigDecimal::add);
}
    public void markPaid() { requirePending(); status = OrderStatus.PAID; }
    public void markPaymentFailed() { requirePending(); status = OrderStatus.PAYMENT_FAILED; }
    ...
}
/** Domain events: facts the core announces. Past tense, immutable. */
public sealed interface OrderEvent permits OrderEvent.OrderPaid, OrderEvent.PaymentFailed {
    String orderId();
    record OrderPaid(String orderId, String customerId, BigDecimal total) implements OrderEvent {}
    record PaymentFailed(String orderId, String customerId, BigDecimal total) implements OrderEvent {}
}
```

```java
// port — interfaces OWNED BY THE CORE.
/** DRIVING (inbound) port: what the outside world may ask the application to do. */
public interface OrderingUseCase {
    OrderReceipt place(PlaceOrderCommand command);
    Optional<OrderReceipt> find(String orderId);
}
/** Input to the driving port. Plain data: no HTTP, CSV or JSON types leak in. */
public record PlaceOrderCommand(String customerId, List<Item> items) {
    public record Item(String sku, int qty) {}
    public PlaceOrderCommand { items = List.copyOf(items); }
}
/** Output: a read-only view, so callers never hold the mutable Order. */
public record OrderReceipt(String orderId, OrderStatus status, BigDecimal total) {}

// DRIVEN (outbound) ports: the core needs these; it does not care how they are implemented.
public interface OrderRepository { void save(Order order); Optional<Order> findById(String id); }
public interface ProductCatalog { Optional<BigDecimal> priceOf(String sku); }      // prices never come from the caller
@FunctionalInterface
public interface PaymentGateway { boolean charge(String customerId, BigDecimal amount); }  // one method: a test can pass a lambda
public interface EventPublisher { void publish(OrderEvent event); }                // the core does not know who listens
```

```java
// app/OrderService.java — APPLICATION core: implements the driving port, uses only driven ports and the domain.
@Override
public OrderReceipt place(PlaceOrderCommand cmd) {
    List<OrderLine> lines = cmd.items().stream()
        .map(i -> new OrderLine(i.sku(), i.qty(), catalog.priceOf(i.sku())
            .orElseThrow(() -> new IllegalArgumentException("unknown product " + i.sku()))))
        .toList();
    Order order = Order.place("o-" + nextId.getAndIncrement(), cmd.customerId(), lines);

    boolean paid = payments.charge(order.customerId(), order.total());
    if (paid) order.markPaid(); else order.markPaymentFailed();
    orders.save(order);
    events.publish(paid
        ? new OrderEvent.OrderPaid(order.id(), order.customerId(), order.total())
        : new OrderEvent.PaymentFailed(order.id(), order.customerId(), order.total()));
    return receiptOf(order);
}
```

```java
// adapter — DRIVEN adapters: InMemoryOrderRepository (ConcurrentHashMap), InMemoryCatalog (fixed price list),
// LimitPaymentGateway (approves charges up to a limit, printing "[payment] charge ... -> approved/declined"), and:
/** A toy BROKER (event-driven, broker topology): publishers and subscribers know only the bus and event types.
 *  Synchronous and in memory; Kafka or RabbitMQ would add durability and async delivery. */
public final class InMemoryEventBus implements EventPublisher {
    private record Subscription<T>(Class<T> type, Consumer<? super T> handler) {
        void offer(Object event) { if (type.isInstance(event)) handler.accept(type.cast(event)); }
    }
    private final List<Subscription<?>> subscriptions = new CopyOnWriteArrayList<>();
    public <T extends OrderEvent> void subscribe(Class<T> type, Consumer<? super T> handler) { subscriptions.add(new
Subscription<>(type, handler)); }
    @Override public void publish(OrderEvent event) { subscriptions.forEach(s -> s.offer(event)); }
}
```

```java
// pipeline — what a filter emits: a value for the next filter, or a rejection that ends the run.
public sealed interface Result<T> permits Result.Ok, Result.Rejected {
    record Ok<T>(T value) implements Result<T> {}
    record Rejected<T>(String reason) implements Result<T> {}
}
/** PIPE-AND-FILTER: a filter turns one input into one output (or rejects it). then() is the PIPE.
 *  Filters never call each other directly, so they can be reordered, reused and tested alone. */
@FunctionalInterface
public interface Filter<I, O> {
    Result<O> process(I input);
    static <I, O> Filter<I, O> of(Filter<I, O> filter) { return filter; }
    default <R> Filter<I, R> then(Filter<O, R> next) {
        return input -> switch (process(input)) {
            case Result.Ok<O> ok -> next.process(ok.value());
            case Result.Rejected<O> rejected -> new Result.Rejected<>(rejected.reason());
        };
    }
}

// adapter/CsvOrderImporter.java — DRIVING adapter: CSV lines ("customer, sku:qty, sku:qty") -> calls on the driving port.
// Syntax checks live here; business rules (known products, payment) live in the core.
private static final Filter<String, PlaceOrderCommand> PIPELINE =
    Filter.of(CsvOrderImporter::dropBlankAndComments)   // rejects "blank line" / "comment"
          .then(CsvOrderImporter::normalise)            // strip, lower-case, remove whitespace
          .then(CsvOrderImporter::splitFields)          // rejects "expected a customer and at least one item in '...'"
          .then(CsvOrderImporter::toCommand);           // rejects "bad item '...', expected sku:qty" (qty must match [1-
9][0-9]{0,3})
// importLines: Rejected -> "line n dropped by filter: <reason>"; Ok -> ordering.place(cmd), printing the receipt
// or "line n rejected by core: <message>" on IllegalArgumentException.
```

```
// Main.java — COMPOSITION ROOT: the only class that knows every adapter. It wires them to the core.
var catalog = new InMemoryCatalog(Map.of(
    "book", new BigDecimal("12.99"), "pen", new BigDecimal("0.99"), "lamp", new BigDecimal("89.00")));
var bus = new InMemoryEventBus();
bus.subscribe(OrderEvent.OrderPaid.class, e -> System.out.println("    [warehouse] reserve stock for " + e.orderId()));
bus.subscribe(OrderEvent.OrderPaid.class, e -> System.out.println("    [email] to " + e.customerId() + ": order " +
e.orderId() + " confirmed"));
bus.subscribe(OrderEvent.class, e -> System.out.println("    [audit] " + e));

OrderingUseCase ordering = new OrderService(
    new InMemoryOrderRepository(), catalog, new LimitPaymentGateway(new BigDecimal("100.00")), bus);

new CsvOrderImporter(ordering).importLines(List.of(
    "# customer, sku:qty, ...", "  C-1, Book:2,  pen:3", "", "c-2, lamp:2", "c-3, pen:zero", "c-4, kettle:1", "c-5"));
// part 2: ordering.find("o-1"), ordering.find("o-9")
// part 3: swap a driven adapter — same core, a lambda that declines every payment:
OrderingUseCase declining = new OrderService(new InMemoryOrderRepository(), catalog, (customer, amount) -> false, bus);
```

Output:

```
== 1. Driving adapter: CSV -> pipe-and-filter -> port -> core -> events ==
  line 1 dropped by filter: comment
  line 2 -> PlaceOrderCommand[customerId=c-1, items=[Item[sku=book, qty=2], Item[sku=pen, qty=3]]]
    [payment] charge c-1 28.95 -> approved
    [warehouse] reserve stock for o-1
    [email] to c-1: order o-1 confirmed
    [audit] OrderPaid[orderId=o-1, customerId=c-1, total=28.95]
  line 2 receipt: OrderReceipt[orderId=o-1, status=PAID, total=28.95]
  line 3 dropped by filter: blank line
  line 4 -> PlaceOrderCommand[customerId=c-2, items=[Item[sku=lamp, qty=2]]]
    [payment] charge c-2 178.00 -> declined
    [audit] PaymentFailed[orderId=o-2, customerId=c-2, total=178.00]
  line 4 receipt: OrderReceipt[orderId=o-2, status=PAYMENT_FAILED, total=178.00]
  line 5 dropped by filter: bad item 'pen:zero', expected sku:qty
  line 6 -> PlaceOrderCommand[customerId=c-4, items=[Item[sku=kettle, qty=1]]]
  line 6 rejected by core: unknown product kettle
  line 7 dropped by filter: expected a customer and at least one item in 'c-5'
== 2. Another driving adapter (a test) calls the same port ==
  find(o-1) = Optional[OrderReceipt[orderId=o-1, status=PAID, total=28.95]]
  find(o-9) = Optional.empty
== 3. Swap a driven adapter: same core, a lambda that declines every payment ==
    [audit] PaymentFailed[orderId=o-1, customerId=c-9, total=0.99]
  OrderReceipt[orderId=o-1, status=PAYMENT_FAILED, total=0.99]
```

What to notice.

- **Validation is split by responsibility.** `pen:zero` never reaches the core, because a pipe-and-filter step rejects bad syntax. `kettle` is syntactically fine but is rejected by the core, because only the catalogue knows which products exist. Prices come from the `ProductCatalog` port, never from the caller.
- **The core is unchanged when adapters change.** Part 3 replaces the payment adapter with a one-line lambda. That is how a unit test runs the core without a payment provider.
- **The bus is the broker topology.** `OrderService` publishes one event, and three independent handlers react. Adding an SMS handler is one `subscribe` call in `Main`.
- **Concurrency and production gaps.** The repository (`ConcurrentHashMap`), the bus (`CopyOnWriteArrayList`) and the ID counter (`AtomicLong`) are thread-safe one call at a time. `place` itself is not atomic, however: it charges, then saves, then publishes. A crash after the charge but before the save loses the order. Production code would save the order as `PENDING_PAYMENT` first, call the gateway with an **idempotency key** (the order ID) so that a retry cannot charge twice, and publish events through an **outbox** (events written in the same database transaction as the order and relayed to the broker afterwards). The bus is also synchronous: a slow handler slows down `place`. A real broker delivers asynchronously and at least once.

In the interview (architectural patterns)

"What architecture would you use?" A strong 30-second answer: "A modular monolith with a hexagonal core: the domain and use cases have no framework imports, and repositories, payment and notifications sit behind ports with in-memory adapters, so the design is testable now and each adapter can later become a network client. Inside, side effects such as email and analytics hang off domain events on an event bus, broker style. If one capability needs to scale or ship independently, its module boundary is already the future service boundary."

Common follow-ups (answers in the sections above): layered or hexagonal (§11.9.7), why not microservices from day one (§11.9.4), choreography or orchestration (§11.9.5), pipeline or chain of responsibility (§11.9.6).

Mistakes to avoid: naming an architecture without saying the trade-off; drawing microservices for a 90-minute machine-coding problem; letting framework annotations or SQL types into domain classes; ports shaped like one vendor's API; events named as commands (`SendEmail` instead of `OrderPaid`).

Self-check (architectural patterns)

1. **Q:** What is the difference between a layer and a tier? **A:** A layer is a logical code separation. A tier is a physical one (a separate process or machine).
2. **Q:** In hexagonal architecture, who owns `OrderRepository`, and who implements it? **A:** The core owns the port. A driven adapter (in-memory, JDBC) implements it.
3. **Q:** A loan application must pass a credit check, then a fraud check, then manual approval, and a rejection must undo earlier steps. Broker or mediator topology? **A:** Mediator (orchestration), because there is ordering plus compensation.
4. **Q:** Why should most new systems start as a modular monolith? **A:** You get enforced boundaries without network failures, distributed transactions or operations overhead. Services can be extracted later along the module boundaries.
5. **Q:** How do MVC and hexagonal relate? **A:** MVC organises the UI edge. Its controller is a driving adapter of the hexagonal core, so the two are complementary.
6. **Q:** In §11.9.9, why is `pen:zero` rejected by a filter but `kettle` by the core? **A:** The first is a syntax error, an adapter concern. The second is a business rule (a known product), a core concern.

TL;DR

- **Design patterns** arrange classes. **Architectural patterns** arrange the whole system: its parts, dependency directions and communication.
- **Layered:** controller, then service, then repository; simple, but the domain depends on storage. **Client-server:** the server owns the truth and the validation.
- **Modular monolith first**, microservices only for team, scaling or isolation reasons. Microservices cost network failures, sagas and eventual consistency.
- **Event-driven:** a **broker** (choreography) for independent reactions, a **mediator** (orchestration) for ordered processes. Consumers must be idempotent.
- **Pipe-and-filter:** independent transforming steps joined by pipes, where every step runs. In a chain of responsibility, any handler may stop the request.
- **Hexagonal / clean / onion:** a core with ports, technology in adapters, and dependencies pointing inward (DIP at system scale). MVC/MVP/MVVM (§11.7.5) are driving adapters at the UI edge.

Chapter 12 — Concurrency & Multithreading for LLD (Java)

> **Why this chapter exists.** In a low-level design (LLD) interview you will often hear: *"Now two users click **Book** on the same seat at the same moment. What happens?"* A design with no answer is incomplete. This chapter teaches, from zero, the ideas and Java tools to answer it well. Chapter 13 applies them to the classic concurrency interview problems; chapters 14–21 use them inside full case studies.

How to read this chapter. Each concept follows: *What it is → The problem it solves → How it works → Java example → Trade-offs (incl. when not to use it) → Common mistakes → Real-world examples → In the interview → TL;DR.* All code is Java 21, standard library only, in `code/lld/ch12/<topic>/Main.java`; listings below keep the essential lines, and every `Main` compiles and runs with the output shown. From `chapters/12-concurrency-concepts/code`:

```
javac --release 21 -d /tmp/lld_12 $(find . -name '*.java')
java -cp /tmp/lld_12 lld.ch12.threads.Main
```

Map of the chapter

Part	Sections	Code package
A. Foundations	12.1 Concurrency vs parallelism · 12.2 Processes vs threads · 12.3 Thread lifecycle · 12.4 Creating threads	`threads`
B. What goes wrong	12.5 Race conditions & critical sections · 12.6 Java Memory Model (visibility, happens-before, `volatile`)	`race`
C. Locks	12.7 Mutex / `synchronized` · 12.8 `ReentrantLock` · 12.9 Try-lock & timed locking · 12.10 Read-write locks & `StampedLock` · 12.11 Coarse vs fine-grained locking	`locks`
D. Coordination	12.12 Semaphores · 12.13 Condition variables · 12.14 `CountDownLatch` & `CyclicBarrier`	`coordination`
E. Lock-free	12.15 Compare-and-swap & atomics · 12.16 Concurrent collections	`atomics`, `collections`
F. Liveness failures	12.17 Deadlock · 12.18 Livelock · 12.19 Starvation	`deadlock`
G. Patterns	12.20 Signaling · 12.21 Thread pool · 12.22 Producer-consumer · 12.23 Reader-writer · 12.24 Immutability & thread confinement	`patterns`
H. Interviews	12.25 Concurrency in LLD interviews (parking spots, seat booking, inventory)	`lldcases`
	12.26 Choosing the right tool · 12.27 Chapter self-check · 12.28 Chapter TL;DR	

Words you will see everywhere (defined once here):

Term	Meaning in one line
Thread	One independent sequence of instructions running inside a program.
Shared state	Data (fields, objects) that more than one thread can read or write.
Thread-safe	A class that behaves correctly no matter how threads interleave, with no extra locking by callers.
Atomic	Happens as one indivisible step: other threads see either "before" or "after", never "halfway".
Lock / mutex	A token only one thread can hold at a time. Code that needs the token waits for it.
Critical section	A piece of code that touches shared state and must not run in two threads at once.
Contention	Many threads wanting the same lock at the same time, so they queue up.
Blocking	A thread stops and waits (for a lock, I/O, a signal) instead of using the CPU.
Liveness	"Something good eventually happens" — the program keeps making progress (no deadlock, livelock, starvation).
Safety	"Nothing bad ever happens" — no corrupted data, no double booking.

Part A — Foundations

12.1 Concurrency vs parallelism

What it is

- **Concurrency** is about *dealing with* many things at once: several tasks whose lifetimes overlap, possibly taking turns on one **CPU core** (one independent processing unit inside the processor; a modern CPU has several).
- **Parallelism** is about *doing* many things at once: tasks literally execute at the same instant on different cores.

One chef switching between three simmering dishes is concurrent; three chefs each cooking one dish is parallel.

The problem it solves

Programs wait a lot — for disks, networks, databases, users. Concurrency does useful work while one task waits (better *responsiveness* and *throughput* — work finished per second; see System Design Fundamentals). Parallelism makes CPU-heavy work (sorting, image processing) finish sooner by using all cores (*speed-up*).

How it works

1. The OS *scheduler* (the part of the operating system that decides which thread runs next) gives each runnable thread a short time slice on a core.
2. When the slice ends, or the thread blocks (for **I/O** — disk or network reads — a lock, `sleep`), the scheduler performs a **context switch**: it saves that thread's **registers** (the CPU's few, fastest storage slots) and loads another's.
3. On one core, fast switching creates the *illusion* of simultaneity → concurrency. On many cores, threads really run at the same instant → parallelism.

	Concurrency	Parallelism
Core idea	Structure: many tasks in progress	Execution: many tasks running at the same instant
Needs multiple cores?	No	Yes
Main benefit	Overlapping waiting (I/O-bound work: mostly waiting on disk/network)	Speed on CPU-bound work (mostly computing)
Java tools	Threads, executors, virtual threads, `CompletableFuture`	Fork/Join, parallel streams, `Arrays.parallelSort`
Typical example	A web server handling 10,000 connections	Sorting 100 million numbers

Java example

§12.4's `Main`: 10,000 tasks that each *sleep* 50 ms finish in well under a second on virtual threads (concurrency hides waiting). Chapter 13's merge sort uses **Fork/Join** (the JDK framework that splits a task into subtasks run on all cores) for parallelism.

Trade-offs

- Concurrency adds complexity (races, deadlocks) and overhead (context switches). Parallelism is capped by **Amdahl's law**: if a fraction s of the work is inherently sequential, speed-up with N cores can never exceed $1 / (s + (1 - s)/N)$. With 10% sequential work, even infinite cores give at most 10×.
- **Avoid** for small, fast, CPU-light tasks (splitting costs more than it saves). Single-threaded is a valid choice when it removes all locking: Redis (an in-memory key-value database) executes commands on one main thread; Node.js (a server-side JavaScript runtime) runs JavaScript on one **event-loop** thread — a single loop that handles ready events one at a time and never blocks.

Common mistakes

- Saying "concurrent" when you mean "parallel" (and vice versa).
- Expecting more threads to always mean more speed. Past the core count, CPU-bound work gets *slower* from switching.

Real-world examples

- **Concurrency:** Tomcat and Jetty (Java web servers) serve requests with a **pool** of threads (a fixed set of reusable workers); Netty (a Java networking library) and Node.js serve thousands of connections with a few event-loop threads.
- **Parallelism:** `Arrays.parallelSort`, parallel streams, Spark/Hadoop (distributed data-processing frameworks) splitting data across cores and machines.

In the interview

> **30-second answer:** "Concurrency is about structure — several tasks in progress, possibly interleaved on one core. Parallelism is about execution — tasks running at the same instant on several cores. Concurrency helps I/O-bound work; parallelism helps CPU-bound work."

Follow-ups:

- *Concurrency without parallelism?* Yes — a single core time-slicing several threads. *Parallelism without concurrency?* In a narrow sense: SIMD (one instruction on many data items) runs in parallel inside one task.
- *How many threads for a pool?* CPU-bound: about the core count (`Runtime.getRuntime().availableProcessors()`). I/O-bound: roughly cores × (1 + wait time / compute time).
- *A chat server with 50,000 idle connections?* A concurrency problem — most connections wait, not compute.
- *What limits parallel speed-up?* The sequential part (Amdahl) plus coordination overhead.

TL;DR

Concurrency = dealing with many things (structure). Parallelism = doing many things at once (execution). I/O-bound → concurrency; CPU-bound → parallelism.

12.2 Processes vs threads

What it is

- A **process** is a running program with its *own* memory space (**heap** — where objects live — code, open files). Processes cannot see each other's memory.
- A **thread** is a unit of execution *inside* a process. Threads share the process's heap, but each has its own **stack** (local variables, call frames) and **program counter** (address of the next instruction).

The problem it solves

Threads let one program do several things at once while sharing data cheaply. Processes give strong isolation: one crashing does not corrupt the others.

How it works

1. The OS creates a process with a private **virtual address space** (its own view of memory, mapped by the OS onto real RAM); it starts with one thread (`main` in Java).
2. More threads share heap objects; each gets its own stack.
3. Threads communicate through shared objects (with synchronization); processes through **pipes** (one-way byte channels), **sockets** (network-style connection endpoints), shared-memory segments or message queues (IPC).

	Process	Thread
Memory	Private address space	Shares the process's heap; own stack
Creation cost	High (new address space)	Lower (platform thread ≈ an OS thread with ~1 MB stack reserved by default on 64-bit JVMs — Java Virtual Machines, which run Java bytecode); virtual threads are much cheaper still
Communication	IPC: sockets, pipes, files	Shared objects — fast, but needs synchronization
Failure isolation	Strong	Weak: an unhandled error can bring down the whole process (e.g. `OutOfMemoryError`)
Context switch	Heavier (address-space switch)	Lighter
Java	`ProcessBuilder`, `Runtime.exec`	`Thread`, executors

Java example

```
Process p = new ProcessBuilder("ls", "-l").inheritIO().start();   // new PROCESS (separate memory)
p.waitFor();
Thread t = new Thread(() -> System.out.println("same heap, different stack"));   // new THREAD
t.start();
t.join();
```

Trade-offs

Threads share fast, but every shared mutable field is a potential bug. Processes isolate safely, but communication is slower and needs **serialization** (turning objects into bytes). Prefer processes or **containers** (lightweight isolated environments such as Docker) when isolation matters more than speed — untrusted plugins, a crash-prone **native library** (machine code, e.g. C, called from Java).

Common mistakes

- Thinking local variables are shared. Each thread has its own stack; only objects reachable from shared references are shared.
- One platform thread per request without limit — each has a large stack and an OS thread behind it. Use a pool, or virtual threads.

Real-world examples

- **Google Chrome** runs tabs in separate *processes* so one crashing page does not take down the browser.
- **PostgreSQL** uses one process per connection; **MySQL** one thread per connection.

- A **JVM** is one process; application threads, **garbage-collector** threads (the GC frees unused memory) and **JIT-compiler** threads (Just-In-Time: compiles hot bytecode to machine code at run time) all live inside it.

In the interview

> **30-second answer:** "A process is an isolated program with its own memory; a thread is a path of execution inside a process that shares its heap but has its own stack. Threads are cheaper and share data easily but need synchronization; processes are isolated and communicate through IPC."

Follow-ups:

- *What does a thread own privately?* Its stack, program counter, and `ThreadLocal` values (variables holding a separate copy per thread).
- *Why are threads cheaper to switch?* They share the address space, so the CPU's memory mappings stay the same.
- *Two threads each declare `int x = 0;` in a method and increment it — race?* No, locals live on each stack. *Same object's field?* Yes, unless synchronized.

TL;DR

Process = isolated, private memory. Thread = worker inside it sharing the heap, owning only its stack. Share cheaply, synchronize carefully.

12.3 Thread lifecycle & states

What it is

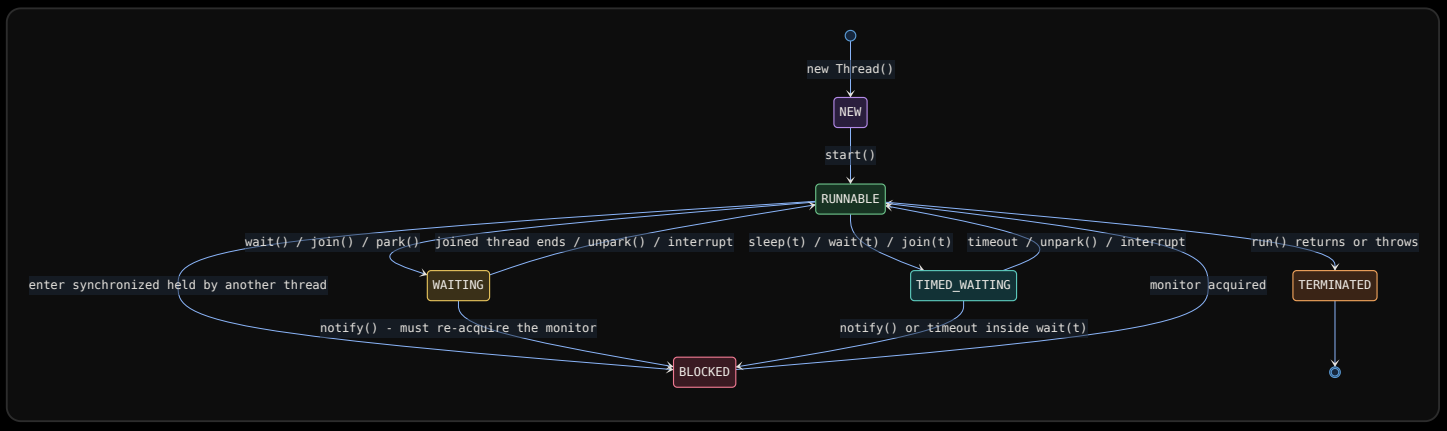
`Thread.getState()` returns one of the six values of `java.lang.Thread.State`:

State	Meaning
`NEW`	Created with `new Thread(...)`, `start()` not called yet.
`RUNNABLE`	Running on a CPU or ready to run. Java does not distinguish "running" from "ready"; a thread blocked in most I/O calls also shows as `RUNNABLE`.
`BLOCKED`	Waiting to *enter* a `synchronized` block/method whose monitor (the object's built-in lock, §12.7) another thread holds.
`WAITING`	Waiting with no time limit for another thread: `Object.wait()`, `Thread.join()`, `LockSupport.park()` (the low-level call that suspends a thread; `ReentrantLock`, `Condition`, `CountDownLatch` use it).
`TIMED_WAITING`	Same with a timeout: `Thread.sleep(ms)`, `wait(ms)`, `join(ms)`, `parkNanos`, `tryLock(timeout)`.
`TERMINATED`	`run()` has returned or thrown. A thread cannot be restarted.

The problem it solves

Knowing the states lets you read **thread dumps** (a snapshot of every thread's state and stack, via `jstack` or `jcmd <pid> Thread.print`) and diagnose hangs: many threads `BLOCKED` on one monitor → lock contention or **deadlock** (threads waiting on each other forever, §12.17); many `WAITING` on a queue → idle pool.

How it works



- `start()` moves `NEW` → `RUNNABLE`; calling it twice throws `IllegalThreadStateException`.
- A thread woken from `wait()` must re-acquire the monitor, so it may pass through `BLOCKED` before `RUNNABLE`.
- **Interruption.** `t.interrupt()` sets a flag. If `t` is in `sleep`, `wait`, `join` (or other interruptible blocking calls) it wakes with `InterruptedException` and the flag is cleared. Interruption is *cooperative*: the thread decides how to react. `Thread.stop()` is unsafe and no longer works on modern JDKs.

Java example

`threadStates()` in §12.4's `Main.java` drives one thread into each state and prints `getState()` (the "Thread.State tour" output lines).

Trade-offs

It's a model, not a choice — but `RUNNABLE` hides whether the thread is *actually* on a CPU or stuck in native I/O. Never use `getState()` for program logic ("if WAITING then…"): the state changes between check and action. Use it for monitoring and debugging only.

Common mistakes

- Calling `run()` instead of `start()`: the code runs on the *current* thread.
- Swallowing `InterruptedException` silently. Rethrow it or restore the flag with `Thread.currentThread().interrupt()`.
- Treating `BLOCKED` and `WAITING` as the same: `BLOCKED` = waiting for a *monitor*; `WAITING` = waiting for a *signal*.

Real-world examples

A `jstack` dump showing 200 Tomcat worker threads `BLOCKED` on one `synchronized` logger or connection-pool method is the classic sign of a coarse lock bottleneck.

In the interview

> **30-second answer:** "Six states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED. BLOCKED means waiting for a monitor lock; WAITING/TIMED_WAITING mean waiting for another thread's action, optionally with a timeout."

Follow-ups:

- *Is there a RUNNING state?* No — running and ready are both `RUNNABLE`.
- *State during `ReentrantLock.lock()` when the lock is taken?* `WAITING` (it parks via `LockSupport`) — `BLOCKED` is only for intrinsic monitors. Inside `latch.await()`: `WAITING`; inside `Thread.sleep(1000)`: `TIMED_WAITING`.
- *Restart a terminated thread?* No; create a new one (or submit the task to a pool).

TL;DR

NEW → RUNNABLE ↔ (BLOCKED | WAITING | TIMED_WAITING) → TERMINATED. BLOCKED = waiting for a monitor; WAITING = waiting for a signal. Use `start()`, never `run()`, and respect interrupts.

12.4 Creating threads: `Thread`, `Runnable`, `Callable`, `ExecutorService`, virtual threads

What it is

Way	What you write	Returns a value?	Use when
Extend `Thread`	`class T extends Thread { run() }`	No	Almost never — mixes the task with the worker
`Runnable`	`() -> { ... }` passed to `new Thread(r)` or a pool	No	A fire-and-forget task
`Callable<V>`	`() -> value` (may throw checked exceptions — ones the compiler forces you to catch or declare)	Yes, via `Future<V>`	You need a result or an exception back
`ExecutorService`	`pool.submit(task)`	`Future<V>`	Default choice: reuse threads, limit concurrency
Virtual threads (Java 21)	`Thread.ofVirtual().start(r)` / `Executors.newVirtualThreadPerTaskExecutor()`	via `Future`	Very many *blocking* (I/O-bound) tasks

A `Runnable` is the job description, a `Thread` the worker, an `ExecutorService` the staffing agency, a `Future` the receipt you collect the result with.

The problem it solves

An OS thread per task is expensive and unbounded. Executors separate **what** to run (task) from **how** it runs (pooling, queueing, scheduling). Virtual threads remove the cost problem for blocking code, allowing a simple "one thread per request" style again.

How it works

1. Write the task as `Runnable` (no result) or `Callable<V>`; submit it to an `ExecutorService` (e.g. `Executors.newFixedThreadPool(n)`) and get a `Future<V>`.
2. `future.get()` blocks until the result is ready (or rethrows the task's exception wrapped in `ExecutionException`).
3. When done, `shutdown()` (stop accepting tasks) and `awaitTermination(...)`; or, since Java 19, use **try-with-resources** (a `try (...)` block that closes its resource automatically) — `ExecutorService.close()` waits for tasks to finish.
4. **Virtual threads** (final in Java 21, JEP 444 — a JEP, JDK Enhancement Proposal, is the numbered design document for a Java feature) are lightweight JVM-managed threads. Many run on a small pool of OS "carrier" threads (a `ForkJoinPool` sized to the core count by default). When a virtual thread blocks on I/O, `sleep`, or a `java.util.concurrent` lock, the JVM *unmounts* it and the carrier runs another. The exception is **pinning**: a virtual thread that cannot unmount keeps its carrier busy while blocked. On Java 21–23 this happens when blocking inside `synchronized` (including `Object.wait()`) or a native method; JDK 24 (JEP 491) removed the `synchronized` case, leaving mainly native/foreign-function calls (and blocking inside a class initializer).

Java example

Essential lines of `code/lld/ch12/threads/Main.java` (full file: also the `GreeterThread extends Thread` and `Runnable` demos, and the `awaitState` polling helper):

```
ExecutorService pool = Executors.newFixedThreadPool(2);
try {
    Future<Integer> future = pool.submit(() -> IntStream.rangeClosed(1, 100).sum()); // returns at once
    System.out.println("3) Callable+Future  -> sum 1..100 = " + future.get());       // get() blocks
    List<Callable<String>> jobs = List.of(() -> "A", () -> "B", () -> "C");
    for (Future<String> f : pool.invokeAll(jobs)) results.add(f.get());              // input order kept
} finally {
    pool.shutdown();                                  // stop accepting work
    pool.awaitTermination(5, TimeUnit.SECONDS);       // wait for running tasks
}

// 10,000 blocking tasks: each sleep parks its virtual thread, freeing the carrier OS thread.
try (ExecutorService vexec = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10_000; i++) fs.add(vexec.submit(() -> { Thread.sleep(50); return 1; }));
    for (Future<Integer> f : fs) sum += f.get();
}                                                     // close() waits for all tasks

// threadStates(): drive threads into each state and print getState()
synchronized (monitor) {                              // main holds the monitor...
    blocked = new Thread(() -> { synchronized (monitor) { } }, "blocked");
    blocked.start();
    awaitState(blocked, Thread.State.BLOCKED);        // polls: state changes are not instant
    System.out.println("waiting to enter synchronized : " + blocked.getState());
}
```

`t.start()` creates a new call stack; `t.join()` waits and also makes everything `t` wrote visible to the joiner (a *happens-before* edge, §12.6). Output (full program):

```
=== Creating threads ===
1) extends Thread   -> hello from greeter-thread
2) Runnable         -> hello from runnable-thread
3) Callable+Future  -> sum 1..100 = 5050
4) ExecutorService  -> invokeAll results [A, B, C]
=== Virtual threads (Java 21) ===
5) virtual thread   -> isVirtual=true
10,000 tasks that each sleep 50 ms completed: 10000 (well under 10,000 x 50 ms = 500 s: true)
=== Thread.State tour ===
after new Thread()            : NEW
inside Thread.sleep(10s)      : TIMED_WAITING
after run() returns           : TERMINATED
inside latch.await()          : WAITING
waiting to enter synchronized : BLOCKED
busy running on a CPU         : RUNNABLE
```

Trade-offs

	Platform thread pool	Virtual threads
Cost per thread	High (OS thread + large stack)	Very low (heap-allocated stack that grows as needed)
How many	Hundreds to a few thousand	Millions
Best for	CPU-bound work (size ≈ cores)	Blocking I/O-bound work
Limits concurrency?	Yes, by pool size	No — add a `Semaphore` if a downstream resource needs a limit
Pooling	Yes	No — never pool virtual threads; create one per task

Avoid: virtual threads for CPU-heavy computation (no extra cores — use a fixed pool or Fork/Join); on Java 21–23, blocking inside `synchronized` on a virtual thread (pinned — prefer `ReentrantLock` around blocking calls there); `Executors.newCachedThreadPool()` for unbounded incoming traffic (unlimited platform threads).

Common mistakes

- Forgetting `shutdown()`: non-daemon pool threads keep the JVM alive after `main` ends (a **daemon** thread is a background thread that does not keep the JVM running).
- Ignoring the `Future` of a submitted task: its exception is captured there and lost if you never call `get()`.
- `Executors.newFixedThreadPool` for untrusted load: its queue is **unbounded** (`LinkedBlockingQueue`), so a burst can exhaust memory. Build a `ThreadPoolExecutor` with a bounded queue (§12.21).

Real-world examples

- `Executors` factory methods are the textbook **Factory** pattern in the JDK.
- **Spring** (the most widely used Java application framework): `@Async` runs methods on a `TaskExecutor` (commonly `ThreadPoolTaskExecutor`); Spring Boot 3.2+ runs request handling on virtual threads with `spring.threads.virtual.enabled=true`; Tomcat and Jetty can use virtual threads too.

In the interview

> **30-second answer:** "I never manage raw threads in business code. I describe work as `Runnable` / `Callable` and submit it to an `ExecutorService` sized for the workload: about cores for CPU-bound work, a bounded queue with a rejection policy for incoming traffic. For many blocking I/O tasks on Java 21, I use virtual threads — one per task — and protect scarce resources with a semaphore" (a counter of permits capping how many threads proceed at once, §12.12).

Follow-ups:

- *`Runnable` vs `Callable`?* `Callable` returns a value and may throw checked exceptions.
- *`submit` vs `execute`?* `execute(Runnable)` returns nothing and uncaught exceptions go to the thread's handler; `submit` returns a `Future` that captures the exception.
- *What does `CompletableFuture` add?* Non-blocking composition — `thenApply`, `thenCombine`, `allOf`, `exceptionally` — without `get()` at each step.
- *`thread.run()`?* Runs synchronously on the current thread. *Why not pool virtual threads?* They are cheap; pooling adds nothing and hides one-thread-per-task. Limit concurrency with a semaphore.

TL;DR

Tasks = `Runnable`/`Callable`. Run them with an `ExecutorService`, get results via `Future`. Java 21 virtual threads make one-thread-per-blocking-task cheap. Always shut pools down.

Part B — What goes wrong

12.5 Race conditions & critical sections

What it is

- A **race condition** is a bug where the result depends on the unlucky timing (interleaving) of threads.
- A **critical section** is code that touches shared mutable state and must therefore run in only one thread at a time.
- A **data race** (the precise Java Memory Model term) is two threads accessing the same variable, at least one writing, with no happens-before ordering between them (§12.6).

Example: two people share an account with ₹100; both check the balance at once, both see ₹100, both withdraw ₹80 — the bank paid ₹160 from ₹100. "Check, then withdraw" was the critical section.

The problem it solves

Knowing *which* code needs protection and *why*. The two classic shapes:

Shape	Example	Why it breaks
Read-modify-write	`count++`	It is really `tmp = count; tmp = tmp + 1; count = tmp`. Two threads read the same old value; one update is lost.
Check-then-act	`if (!map.containsKey(k)) map.put(k, v);` or `if (seat.isFree()) seat.book(me);`	The condition can become false between the check and the act.

How it works

1. Find all **shared mutable state** (fields reachable by more than one thread and changed after construction).
2. Find every **invariant** (a rule that must always hold) involving it — "a seat has at most one owner", "balance ≥ 0".

3. Mark the code that reads and then updates that state as a **critical section**.
4. Make each critical section **atomic**: a lock (`synchronized`, `ReentrantLock`), a single atomic operation (`AtomicInteger.incrementAndGet`, `ConcurrentHashMap.putIfAbsent`), or remove the sharing (immutability — objects that never change; confinement — data only one thread touches; §12.24).

Java example

Essential lines of `code/lld/ch12/race/Main.java` (4 threads × 250,000 increments each, joined before reading):

```
static class UnsafeCounter implements Counter {        // BROKEN: count++ is 3 steps
    private int count;
    public void increment() { count++; }
}
static class SynchronizedCounter implements Counter {  // Fix 1: mutex (intrinsic lock)
    private int count;
    public synchronized void increment() { count++; }
    public synchronized int get() { return count; }    // reads need the lock too (visibility)
}
static class AtomicCounter implements Counter {        // Fix 2: lock-free CAS (§12.15)
    private final AtomicInteger count = new AtomicInteger();
    public void increment() { count.incrementAndGet(); }
}

// Check-then-act fixed with ONE atomic call: only succeeds if the seat is still free (0)
AtomicInteger seatOwner = new AtomicInteger(0);
Thread a = new Thread(() -> seatOwner.compareAndSet(0, 1));
Thread b = new Thread(() -> seatOwner.compareAndSet(0, 2));

static class Worker implements Runnable {               // visibility (§12.6)
    private volatile boolean running = true;           // without volatile the loop may never see 'false'
    public void run() { while (running) iterations++; }
    void stop() { running = false; }
}
```

Output (lost updates change every run — that unpredictability *is* the race):

```
expected                : 1000000
unsafe counter          : LOST 680044 updates
synchronized counter    : 1000000
atomic counter          : 1000000
seat has exactly one owner: true
volatile flag stopped worker: true
```

Trade-offs

Fix	Pros	Cons
Lock (`synchronized` / `ReentrantLock`)	Protects any multi-step, multi-variable invariant	Threads wait; risk of deadlock if nested
Atomic variable / atomic collection method	Fast, no blocking, no deadlock	Only covers one variable or one map entry
Immutability / confinement	No synchronization at all	Must redesign so state isn't shared or changed

Don't lock data that is never shared (locals, thread-confined objects) or never changed after safe construction.

Common mistakes

- Protecting writes but not reads — a reader can see stale or half-updated state.
- Two *separate* thread-safe calls assumed atomic together: `if (!map.containsKey(k)) map.put(k,v)` on a `ConcurrentHashMap` is still a race. Use `putIfAbsent` / `computeIfAbsent`.
- Locking on different objects in different methods — each lock protects nothing against the other.
- Believing a race is fixed because tests pass. Races show up rarely and under load.

Real-world examples

Double-booking in ticketing, overselling in **flash sales** (short, heavily promoted sales of limited stock), duplicate payments on a double-clicked "Pay" — all check-then-act races, solved with atomic operations, locks, or database constraints: unique indexes, `SELECT ... FOR UPDATE` (a SQL read that locks the rows it returns), optimistic version columns (a version checked on every update, so a write based on stale data fails).

In the interview

> **30-second answer:** "A race condition is when correctness depends on thread timing. The two common shapes are read-modify-write, like `count++`, and check-then-act, like 'if seat is free then book it'. I find the shared mutable state and its invariants, and make each critical section atomic — with a lock, an atomic operation like `putIfAbsent` or `compareAndSet`, or by removing sharing."

Follow-ups:

- *Is `volatile int count; count++` safe?* No — `volatile` gives visibility, not atomicity.
- *Is `ConcurrentHashMap` enough to make my class thread-safe?* Only if every invariant lives in single map operations; multi-key invariants still need a lock or careful design.
- *The race in `if (cache.get(k) == null) cache.put(k, load(k));`?* Check-then-act; both threads may load and put. Use `computeIfAbsent`.

TL;DR

Race = result depends on timing. Watch for read-modify-write and check-then-act. Make every critical section atomic, including reads.

12.6 Java Memory Model basics: visibility, happens-before, `volatile`

What it is

The **Java Memory Model (JMM)** (Java Language Specification, chapter 17) defines *when a write by one thread is guaranteed to be seen by a read in another*. Without the right synchronization a thread may see a stale value — or never see the update.

- **Visibility**: will thread B *see* what thread A wrote? Each thread may work from **CPU caches** (small, fast memories next to each core) and registers rather than main memory.
- **Reordering**: compilers, the JIT and CPUs may reorder instructions as long as a *single* thread can't tell. Other threads can.
- **Happens-before (HB)**: if action X *happens-before* Y, X's effects are visible to Y and X appears ordered before Y.
- `volatile`: a field modifier making reads/writes of that field visible to all threads and ordered (no caching in registers, no harmful reordering around it).

The problem it solves

`while (running) iterations++;` with a *non-volatile* `running` may never stop: the JIT may read `running` once and turn the loop into `while (true)`. Such bugs are invisible in code review unless you know the JMM.

How it works

The key **happens-before rules** (quotable in an interview):

Rule	Meaning
Program order	Within one thread, each action HB the next one.
Monitor lock	An **unlock** of a monitor HB every later **lock** of the *same* monitor.
Volatile	A **write** to a volatile field HB every later **read** of that field.
Thread start	`t.start()` HB every action in thread `t`.
Thread join	Every action in `t` HB another thread's successful return from `t.join()`.
Transitivity	If A HB B and B HB C, then A HB C.
Final fields	Strictly a separate guarantee (JLS §17.5), not an HB edge: once a constructor finishes without leaking `this`, any thread that obtains a reference sees the correct values of its `final` fields (and of objects reachable through them, as of the end of the constructor) — even if the reference was shared without synchronization.
`java.util.concurrent`	E.g. putting into a `BlockingQueue` HB taking it out; `countDown()` HB a return from `await()`; `Future` completion HB `get()` returning.

Apply them: for each shared variable, is every write-then-read pair connected by an HB edge? Same lock → yes. A single independent flag or reference → `volatile` is enough. A compound update (`x++`, check-then-act) → `volatile` is **not** enough; use a lock or atomic.

Safe publication. An object is *safely published* when its reference becomes visible through an HB edge: a `volatile` field (or `AtomicReference`), a `final` field of a properly constructed object, a static initializer, a concurrent collection, or a write under a lock the reader also takes. Otherwise a reader might see the reference but not the fully constructed object.

Double-checked locking (lazy Singleton — a class with exactly one shared instance, created on first use) is only correct if the field is `volatile`:

```
class Config {
    private static volatile Config instance;          // volatile is essential
    static Config get() {
        Config c = instance;
        if (c == null) {
            synchronized (Config.class) {
                c = instance;
                if (c == null) instance = c = new Config();
            }
        }
        return c;
    }
}
```

Simpler and just as lazy: the *holder idiom* — `private static class Holder { static final Config I = new Config(); }` — class initialization is thread-safe by the JLS.

Java example

`Worker` in §12.5: `private volatile boolean running`. The write `running = false` in `main` HB the worker's next read, so the loop exits — output `volatile flag stopped worker: true`.

Trade-offs

Tool	Visibility	Atomic compound ops	Blocking	Cost
`volatile`	Yes	No	No	Low
`synchronized` / `Lock`	Yes (for code under the same lock)	Yes	Yes	Medium
`Atomic*`	Yes	Yes, for one variable	No	Low–medium
`final` (immutable)	Yes, after construction	N/A — never changes	No	Free

Don't use `volatile` for anything combining several steps or variables.

Common mistakes

- "It works on my laptop" — x86 CPUs (Intel/AMD) reorder less than ARM ones (Apple Silicon, AWS Graviton). Code without HB edges may fail only on other hardware or after **JIT warm-up** (once hot code is compiled and optimised).
- Leaking `this` from a constructor (registering a listener, starting a thread) — others may see a half-built object, and final-field guarantees are lost.
- Synchronizing the writer only. The reader must use the *same* lock.

Real-world examples

- `volatile boolean running` shutdown flags in server loops and schedulers.
- `ConcurrentHashMap`, `AtomicInteger`, `CopyOnWriteArrayList` use volatile reads/writes and CAS (§12.15) internally to give HB guarantees without reader locks.
- `String` is immutable with `final` fields, so strings are shared freely.

In the interview

> **30-second answer:** "The Java Memory Model says a thread is only guaranteed to see another thread's write if there's a happens-before edge — unlock-then-lock of the same monitor, a volatile write-then-read, thread start/join, or a concurrent-collection handoff. `volatile` gives visibility and ordering for a single variable but not atomicity; for compound actions I use locks or atomics."

Follow-ups:

- *Why must the double-checked-locking field be volatile?* Otherwise the reference write can be reordered before the constructor's writes, so another thread sees a non-null, half-initialized object.
- *Does `volatile` make `long`/`double` writes atomic?* Yes — the JLS lets non-volatile 64-bit writes split into two 32-bit halves (rare on 64-bit JVMs, but allowed); volatile ones cannot.
- *A sets non-volatile `ready = true` after preparing data; B spins on `ready`. Two bugs?* B may never see `true`, or may see `true` but stale data (reordering).
- *Which JDK classes give HB for you?* Locks, `Atomic*`, concurrent collections, `BlockingQueue`, `CountDownLatch`, `Future`, `ExecutorService.submit`.

TL;DR

Writes are only guaranteed visible across threads through happens-before edges. `volatile` = visibility + ordering for one variable, not atomicity. Locks = visibility + atomicity. Publish objects safely.

Part C — Locks

12.7 Mutex (mutual exclusion) and `synchronized`

What it is

A **mutex** (mutual-exclusion lock) can be held by at most one thread. Every Java object has a built-in mutex, its **intrinsic lock** or **monitor**, acquired by `synchronized`:

```
public synchronized void deposit(int amt) { balance += amt; }   // locks 'this'
public static synchronized void log() { ... }                    // locks the Class object
void transfer() { synchronized (lock) { ... } }                  // locks 'lock'
```

The problem it solves

Makes a critical section **atomic** (no interleaving) and **visible** (the next thread to take the lock sees everything done before the unlock).

How it works

1. At `synchronized (obj)`, a thread takes `obj`'s monitor if free; otherwise it goes `BLOCKED` until free.
2. Leaving the block — normally *or by exception* — releases the monitor automatically.

3. Intrinsic locks are **reentrant**: the holder can re-enter `synchronized` code on the same object without blocking itself.

Java example

`SynchronizedCounter` (§12.5). Lock on a **private final** object, not `this`, so outside code can't take your lock:

```
private final Object lock = new Object();
void add(Item i) { synchronized (lock) { items.add(i); } }
```

Trade-offs

Pros	Cons
Simplest syntax; auto-release even on exceptions	No timeout, no `tryLock`, not interruptible while waiting
JVM-optimised (uncontended locking is cheap)	One wait-set (threads parked in `wait()`) per object — only one "condition"
Reentrant	No fairness option (fair = longest-waiting thread first); on Java 21–23 pins virtual threads when blocking inside

Need a timeout, interruptible waiting, fairness, several condition queues, or read/write separation → `ReentrantLock` / `ReentrantReadWriteLock`.

Common mistakes

- Synchronizing on a *changing* reference (`synchronized (list)` after `list = new ArrayList<>()`) — threads lock different objects.
- Synchronizing on **boxed values** (objects like `Integer` wrapping a primitive) or string literals (`synchronized ("LOCK")`, `synchronized (Integer.valueOf(1))`) — possibly shared JVM-wide, causing mysterious contention or deadlocks.
- Slow I/O while holding the lock.

Real-world examples

`Vector`, `Hashtable`, `StringBuffer` and `Collections.synchronizedList(...)` wrap every method in `synchronized` — thread-safe per call but slow under contention, hence replaced by `ArrayList` / `HashMap` / `StringBuilder` plus concurrent collections.

In the interview

> **30-second answer:** "A mutex lets one thread at a time into a critical section. In Java, `synchronized` uses the object's monitor: it's reentrant, released automatically, and gives visibility. I lock on a private final object and keep the section short."

Follow-ups:

- *What does a static synchronized method lock?* The `Class` object (e.g. `Counter.class`) — a different lock from instance methods. Synchronized methods on *different* objects don't block each other.
- *Is `synchronized` slow?* Uncontended, it's cheap. Under heavy contention any lock is slow — reduce sharing or use finer locks.
- *Does an exception inside release the lock?* Yes.

TL;DR

Mutex = one holder at a time. `synchronized` is Java's built-in, reentrant, auto-releasing mutex. Lock a private final object; keep the section short.

12.8 `ReentrantLock` and reentrancy

What it is

`java.util.concurrent.locks.ReentrantLock` is an explicit mutex with `synchronized`'s meaning plus extras. **Reentrant** means the holder can acquire it again (a hold count goes up) and must release it as many times.

The problem it solves

- Without reentrancy, a locked method calling another locked method of the same object would **deadlock with itself**.
- Beyond `synchronized`: `tryLock()` / `tryLock(timeout)`, `lockInterruptibly()`, optional **fairness** (`new ReentrantLock(true)`), multiple `Condition`s per lock, and queries like `isLocked()`, `getHoldCount()`.

How it works

`lock()` acquires (hold count 0 → 1) or waits; the same thread calling `lock()` again → 2, no waiting; each `unlock()` decrements, and at 0 a waiter may proceed. **Always** release in `finally`:

```
lock.lock();
try {
    // critical section
} finally {
    lock.unlock();
}
```

Java example

Essential lines of `code/lld/ch12/locks/Main.java` (also used by §12.9–12.10):

```
static class Account {                                  // reentrancy
    private final ReentrantLock lock = new ReentrantLock();
    void withdraw(int amount) {
        lock.lock();
        try { if (canWithdraw(amount)) balance -= amount; }   // calls another locked method
        finally { lock.unlock(); }
    }
    boolean canWithdraw(int amount) {
        lock.lock();                                    // same thread re-acquires: no self-deadlock
        try { System.out.println("hold count inside nested call = " + lock.getHoldCount());
              return balance >= amount; }
        finally { lock.unlock(); }
    }
}

// tryLock with timeout (§12.9): main holds 'printer'; the worker gives up after 100 ms
if (printer.tryLock(100, TimeUnit.MILLISECONDS)) {
    try { System.out.println("worker: got the printer"); } finally { printer.unlock(); }
} else System.out.println("worker: printer busy for 100 ms, doing something else instead");

static class ConfigStore {                              // read-write lock (§12.10)
    private final ReadWriteLock rw = new ReentrantReadWriteLock();
    private final Lock r = rw.readLock(), w = rw.writeLock();
    String get(String k) { r.lock(); try { return map.get(k); } finally { r.unlock(); } }
    void put(String k, String v) { w.lock(); try { map.put(k, v); } finally { w.unlock(); } }
}
// demo: 3 readers wait at a CyclicBarrier(3) WHILE holding the read lock -
// if the read lock were exclusive, the barrier would time out.

static class Point {                                    // StampedLock optimistic read (§12.10)
    private final StampedLock sl = new StampedLock();
    double distanceFromOrigin() {
        long stamp = sl.tryOptimisticRead();            // no locking, just a version number
        double cx = x, cy = y;                          // copy fields to locals
        if (!sl.validate(stamp)) {                      // a writer slipped in: real read lock
            stamp = sl.readLock();
            try { cx = x; cy = y; } finally { sl.unlockRead(stamp); }
        }
        return Math.hypot(cx, cy);
    }
}
```

Output (full program):

```
=== Reentrancy ===
hold count inside nested call = 2
balance = 70
=== tryLock with timeout ===
worker: printer busy for 100 ms, doing something else instead
tryLock() now (free)  = true
=== ReadWriteLock: readers share ===
3 readers held the read lock at the same time: true
=== StampedLock optimistic read ===
distance = 5.0
```

Trade-offs

Feature	`synchronized`	`ReentrantLock`
Release	Automatic	Manual (`finally`) — easy to forget
Try / timeout	No	`tryLock()`, `tryLock(t, unit)`
Interruptible wait	No	`lockInterruptibly()`
Fairness	No	Optional (`true` = longest waiter first; lower throughput)
Conditions	One wait set	Many `Condition` objects
Reentrant	Yes	Yes

If you need none of the extras, `synchronized` is shorter and can't be left locked by mistake.

Common mistakes

- `lock()` *inside* the `try` — if `lock()` threw, `finally` would `unlock()` a lock you don't hold (`IllegalMonitorStateException`). Call `lock()` just before `try`.
- Missing `unlock()` on an early `return` or exception path — the lock stays held forever.
- Fairness "to be safe" — fair locks are noticeably slower; use only when starvation is real.

Real-world examples

`ArrayBlockingQueue`, `LinkedBlockingQueue` and `ThreadPoolExecutor` use `ReentrantLock` internally (`CopyOnWriteArrayList` did up to Java 8; since Java 9 it uses `synchronized` on an internal lock object). `ReentrantLock`, `ReentrantReadWriteLock`, `Semaphore` and `CountDownLatch` are built on `AbstractQueuedSynchronizer` (AQS — the JDK's framework for building locks and synchronizers); `CyclicBarrier` on a `ReentrantLock` + `Condition`.

In the interview

> **30-second answer:** "`ReentrantLock` is an explicit mutex with the same semantics as `synchronized` plus try-lock with timeout, interruptible acquisition, optional fairness, and multiple conditions. Reentrant means the owner can re-acquire it; the hold count must drop to zero to release. I always unlock in `finally`."

Follow-ups:

- *Why would a non-reentrant lock deadlock?* A holder calling another method needing the same lock waits for itself forever.
- *Is `StampedLock` reentrant?* No — a thread holding its write lock that calls `writeLock()` or `readLock()` again blocks forever. It also has no owner and no `Condition`s.
- *`lock()` 3 times, `unlock()` 2 times — free?* No; hold count is 1. *`new ReentrantLock(true)`?* Waiters get the lock in roughly FIFO (first in, first out) order, at a throughput cost.

TL;DR

`ReentrantLock` = `synchronized` + tryLock/timeout/interruptible/fairness/conditions. Owner can re-enter. `lock(); try { … } finally { unlock(); }` always.

12.9 Try-lock & timed locking

What it is

- `tryLock()` — take the lock **only if free right now**; returns `true` / `false` immediately.
- `tryLock(timeout, unit)` — wait up to the timeout, then return `false`. Interruptible.

The problem it solves

Plain `lock()` waits forever: hung requests, permanent deadlocks. Try-lock lets a thread **give up and do something else** — fail fast, retry later, return "busy" — and breaks the *hold and wait* condition of deadlock (§12.17).

How it works

1. `tryLock(timeout)`. If `true`: `try { work } finally { unlock(); }`.
2. If `false`: fallback — return an error, retry with **back-off** (waiting a short while before trying again), or release other locks you hold.
3. With several locks: if any `tryLock` fails, **release everything you hold**, back off for a random short time, retry.

Java example

`locks/Main.java` (§12.8): the worker's `printer.tryLock(100, TimeUnit.MILLISECONDS)` returns `false` while `main` holds it, so it prints the fallback. `transferTryLock` in `deadlock/Main.java` (§12.17) uses try-lock plus random back-off to avoid deadlock.

Trade-offs

Pros	Cons
No infinite waits; can report "busy"	Caller must handle failure paths
Breaks deadlock (hold-and-wait)	Retry loops without random back-off can **livelock**
Timeouts give predictable **latency** (time one operation takes)	Choosing the timeout is a judgement call

When failing isn't acceptable and waiting is fine (a background **batch job** — bulk processing with no user waiting) plain `lock()` is simpler.

Common mistakes

- Calling `unlock()` after `tryLock` returned `false` → `IllegalMonitorStateException`.
- Retrying immediately in a tight loop — burns CPU and can livelock with another thread doing the same.
- Expecting untimed `tryLock()` to respect fairness — it *barges* (takes a free lock even if others are queued); `tryLock(0, unit)` respects fairness.

Real-world examples

- **Distributed locks** (shared by many servers, kept in an external store) follow the same idea — try for a bounded time, give up, or let the lock expire: Redis `SET key value NX PX 30000` (set only if absent, expire after 30,000 ms), Redisson's (a Java Redis client) `tryLock(wait, lease, unit)`, ZooKeeper (a cluster coordination service) recipes.
- Ticketing seat holds: if someone else's checkout has the seat, you're told immediately.

In the interview

> **30-second answer:** "`tryLock` with a timeout lets a thread give up instead of waiting forever. I use it when latency matters or when I need several locks: if I can't get all of them, I release what I hold, back off randomly and retry — which prevents deadlock."

Follow-ups:

- *Why random back-off?* Threads retrying on the same schedule can collide forever (livelock); randomness breaks the symmetry.
- *`tryLock(0, SECONDS)`?* Tries once without waiting but, unlike `tryLock()`, respects a fair lock's queue.

TL;DR

Try-lock = acquire only if available (optionally within a timeout). Handle `false`, release everything on failure, back off randomly.

12.10 Read-write locks: `ReadWriteLock` and `StampedLock`

What it is

A **read-write lock** is a pair: a shared **read lock** (many readers at once) and an exclusive **write lock** (one writer, no readers). Java has `ReentrantReadWriteLock` and, since Java 8, `StampedLock` (adds cheap *optimistic reads*).

The problem it solves

With a plain mutex, readers block each other though reading changes nothing. For **read-heavy** data (configuration, **caches** — fast copies of data kept for quick reads — price catalogues) reads can proceed in parallel.

How it works

1. Reader: `readLock().lock()` succeeds if no writer holds the write lock; many readers may hold it.
2. Writer: `writeLock().lock()` waits until there are **no readers and no writer**.
3. `ReentrantReadWriteLock`: a writer may **downgrade** (take the read lock while holding the write lock, then release the write lock); a reader may **not upgrade** (asking for the write lock while holding the read lock deadlocks — it waits for all readers, including itself).
4. `StampedLock` optimistic read: `long s = tryOptimisticRead()` (no locking) → copy fields to locals → `validate(s)`; if a writer intervened, `validate` returns `false` and you retry with a real `readLock()`.

Java example

`ConfigStore` and `Point` in `locks/Main.java` (§12.8); `3 readers held the read lock at the same time: true` proves readers share.

Trade-offs

	`synchronized` / `ReentrantLock`	`ReentrantReadWriteLock`	`StampedLock`
Parallel readers	No	Yes	Yes (optimistic reads take no lock at all)
Reentrant	Yes	Yes	**No**
Conditions	Yes	Write lock only	No
Overhead	Lowest	Higher bookkeeping	Low for optimistic reads; more complex code
Best when	Short sections or write-heavy	Reads ≫ writes and reads are not trivially short	Very read-heavy, tiny critical sections (e.g. coordinates)

Avoid for write-heavy or very short sections (the bookkeeping can make it *slower* than a mutex), or when a copy-on-write snapshot is simpler — an immutable map in a `volatile` field replaced on write (§12.23).

Common mistakes

- Upgrading read → write in `ReentrantReadWriteLock` (self-deadlock).
- Modifying state while holding only the *read* lock.
- Using optimistic-read data before `validate`, or reading fields directly instead of copying to locals first.
- Writer starvation in non-fair mode under a constant stream of readers (use `new ReentrantReadWriteLock(true)` if it matters; `StampedLock` gives no fairness guarantee either).

Real-world examples

In-memory caches, configuration holders, routing tables read on every request and updated rarely. `ConcurrentHashMap` goes further: reads take no lock.

In the interview

> **30-second answer:** "A read-write lock lets many readers in together but gives writers exclusive access. It helps when reads vastly outnumber writes. `ReentrantReadWriteLock` allows downgrade but not upgrade; `StampedLock` adds optimistic reads that take no lock and are validated afterwards, but it's not reentrant."

Follow-ups:

- *Thread-safe config service?* A RW lock around a `HashMap`, or an immutable `Map` in a `volatile` field replaced atomically on reload (readers never block).
- *Two readers and a writer arrive?* The readers run together; the writer runs alone.
- *Why copy to locals in an optimistic read?* So after `validate` succeeds you use a consistent snapshot, not fields that may change after the check.

TL;DR

Many readers OR one writer. Use for read-heavy shared data. No upgrade in `ReentrantReadWriteLock`; `StampedLock` = optimistic, non-reentrant.

12.11 Coarse-grained vs fine-grained locking

What it is

- **Coarse-grained locking**: one lock protects a lot of state (the whole parking lot).
- **Fine-grained locking**: many locks, each protecting a small piece (per floor, per spot, or per **hash bucket** — one slot of the internal array a hash map spreads entries over).

The problem it solves

The right granularity balances **simplicity and safety** against **throughput under contention**.

How it works

1. Start coarse: one lock around the whole object. Measure.
2. If contention is the bottleneck, split state into **independent** parts whose invariants don't cross, each with its own lock (**lock splitting**).
3. For a big collection, use **lock striping**: N locks; item *i* uses lock `hash(i) % N` (Java 7's `ConcurrentHashMap` had 16 segments by default; chapter 13 builds a striped map).
4. When an operation needs several fine locks (booking seats A1 and A2), acquire them in a **global order** to avoid deadlock.
5. Keep critical sections short: slow work (I/O, computation) *outside* the lock.

	Coarse-grained	Fine-grained
Simplicity	High	Lower — more locks, ordering rules
Concurrency	Low: one thread at a time	High: threads on different parts run together
Deadlock risk	Low (one lock)	Higher (many locks) — needs lock ordering
Overhead	One lock	Memory and bookkeeping for many locks
Cross-part invariants	Easy	Hard — may need to take several locks

Java example

Coarse: `SynchronizedCounter` (§12.5). Fine: `Show` in `lldcases/Main.java` (§12.25) has **one lock per seat**, taken in sorted order; chapter 13's `StripedHashMap` has one lock per stripe.

Trade-offs

See the table. Rule of thumb: *correct first, then fine-grained only where **profiling** (measuring where time is really spent) shows contention*. Don't split when an invariant spans the parts ("total seats booked ≤ capacity" across all rows) unless you also handle the cross-part check.

Common mistakes

- Premature fine-graining → complexity and deadlocks without real benefit.
- Locking fine-grained parts in inconsistent order.
- Fine locks that still funnel through one shared counter, which becomes the **bottleneck** (the one slow point limiting the system) — use `LongAdder`, a counter spreading updates over several cells (§12.15).

Real-world examples

- `ConcurrentHashMap` (Java 8+) locks only the head node of one bin for writes.
- Databases: row-level locks (fine) vs table-level locks (coarse).
- Python's GIL (Global Interpreter Lock) is a famous coarse lock: simple, but one thread runs Python **bytecode** (the interpreter's compiled instructions) at a time.

In the interview

> **30-second answer:** "I start with one lock for correctness, then refine where contention matters: per-floor or per-spot locks for a parking lot, per-show or per-seat locks for booking, striping for maps. Finer locks raise throughput but need a consistent lock order to avoid deadlock."

Follow-ups:

- *Per-show or per-seat locking for BookMyShow (an Indian movie-ticketing site)?* Per-show is simple and usually enough (contention is per show). Per-seat allows more parallelism, but multi-seat bookings must lock seats in sorted order.
- *Lock striping?* A fixed array of N locks, each item mapped to one by hash — fine-grained locking with a bounded lock count.

TL;DR

Coarse = simple, low throughput. Fine = parallel, complex, deadlock-prone. Start coarse, refine with measurements, and order your locks.

Part D — Coordination

12.12 Semaphores

What it is

A **semaphore** holds a number of **permits**. `acquire()` takes one (waiting if none are left); `release()` gives one back. `java.util.concurrent.Semaphore` implements it. With 1 permit it behaves like a mutex, *except* it has no owner — any thread may release it.

The problem it solves

- **Limiting concurrency**: at most N threads use a resource (DB connections, **API** calls — calls to another service's programming interface — GPU slots).
- **Signaling** between threads: 0 permits means "wait until someone signals" (used heavily in chapter 13).

How it works

1. `new Semaphore(n)` — n permits, **non-fair** by default: a thread calling `acquire()` may *barge* ahead of waiting threads. `new Semaphore(n, true)` is **fair**: permits go to waiters in FIFO order (at some throughput cost). Even on a fair semaphore the untimed `tryAcquire()` barges; `tryAcquire(0, TimeUnit.SECONDS)` respects fairness.
2. `acquire()` — if permits > 0, decrement and continue; else wait. `release()` — increment and wake a waiter. Also `tryAcquire()`, `tryAcquire(timeout, unit)`, `acquire(k)`.
3. Always `release()` in `finally` after a *successful* `acquire()`.

Java example

Essential lines of `code/lld/ch12/coordination/Main.java` (it also holds the §12.13–12.14 demos):

```
Semaphore connections = new Semaphore(3);                 // at most 3 concurrent "DB connections"
for (int i = 0; i < 10; i++) pool.submit(() -> {          // 10 tasks on a 10-thread pool
    connections.acquire();                                // blocks when 0 permits left
    try {
        maxSeen.accumulateAndGet(inUse.incrementAndGet(), Math::max);   // peak concurrent users
        Thread.sleep(30);                                 // "run a query"
        inUse.decrementAndGet();
    } finally { connections.release(); }                  // give the permit back
});
```

Output (full program; later lines belong to §12.13–12.14):

```
=== Semaphore: at most 3 concurrent DB connections ===
10 queries done, never more than 3 at once: true
=== wait/notifyAll: one-slot mailbox ===
received in order: [m1, m2, m3]
=== Lock + Condition: strict turn-taking ===
PING PONG PING PONG PING PONG
=== CountDownLatch: wait for 3 services to start ===
all services ready: true (count now 0)
=== CyclicBarrier: 3 workers, 2 phases ===
phase 1 complete: all 3 workers arrived
phase 2 complete: all 3 workers arrived
```

Trade-offs

Pros	Cons
Simple way to cap concurrency	No owner: a bug can release more than acquired (permits grow)
Works as a signal (0 permits)	Doesn't protect data by itself — only counts

Don't use a semaphore as a mutex around shared data when a lock would do — locks have ownership checks and reentrancy.

Common mistakes

- Releasing on a path where `acquire` never succeeded (e.g. `acquire` interrupted) → permits leak upward.
- Not releasing on exceptions → permits slowly disappear, the system stalls.

Real-world examples

- **Bulkhead pattern** (isolating resources per dependency, like a ship's watertight compartments, so one failure can't sink everything): Resilience4j's `SemaphoreBulkhead` (and Hystrix's — Netflix's older fault-tolerance library — semaphore isolation) cap concurrent calls to a dependency.
- **Connection pools** (open database connections reused across requests) bound how many are handed out (HikariCP, a popular JDBC pool, enforces its maximum pool size).
- With virtual threads, a `Semaphore` is the recommended way to limit calls to a slow **downstream** service (one your code calls).

In the interview

> **30-second answer:** "A semaphore is a permit counter: acquire takes a permit or waits, release returns one. I use it to cap concurrent access to a limited resource — a bulkhead — or with zero initial permits as a signal from one thread to another. Unlike a lock it has no owner."

Follow-ups:

- *Binary semaphore vs mutex?* Both allow one holder, but a mutex has an owner (only the holder may unlock; usually reentrant); a binary semaphore can be released by any thread and isn't reentrant.
- *`new Semaphore(0)`: what does `acquire()` do?* Waits until some thread calls `release()` — a signal.

TL;DR

Semaphore = N permits. Cap concurrency or signal. No ownership, so release carefully in `finally`.

12.13 Condition variables: `wait/notify` and `Condition`

What it is

A **condition variable** lets a thread *sleep until some condition about shared state becomes true*, releasing the lock while it sleeps, and lets another thread *wake it* after changing that state.

- Built-in: `Object.wait()`, `notify()`, `notifyAll()` — inside `synchronized` on the same object.
- Explicit: `Condition c = lock.newCondition(); c.await(); c.signal(); c.signalAll();` — while holding that `ReentrantLock`.

The problem it solves

Waiting for a state change without **busy-waiting** (spinning in a loop wasting CPU) and without missing the moment it happens.

How it works

The canonical pattern — memorize it:

```
synchronized (lock) {                // or: lock.lock(); try {
    while (!conditionHolds()) {      // ALWAYS a while loop, never if
        lock.wait();                 //   cond.await();  releases the lock while waiting
    }
    doAction();                      // condition is true AND we hold the lock
    lock.notifyAll();                //   cond.signal()/signalAll() if we changed state others wait for
}                                    // } finally { lock.unlock(); }
```

`wait()` / `await()` **atomically releases the lock** and sleeps; on waking the thread **re-acquires the lock** before returning, then re-checks the condition.

Why the `while` loop? (a) **Spurious wakeups** — the JLS and the `Condition` docs allow a thread to wake without being notified. (b) Another thread may grab the state between the notify and your re-acquisition. (c) With `notifyAll`, several threads wake but only one can succeed.

`notify` vs `notifyAll`: `notify()` wakes one arbitrary waiter. If different kinds of threads (producers and consumers) wait on one monitor, `notify()` can wake the wrong kind and everyone sleeps forever. Use `notifyAll()` with a single monitor, or separate `Condition`s (`notFull`, `notEmpty`) and targeted `signal()`.

Java example

From `coordination/Main.java` — output `received in order: [m1, m2, m3]` and `PING PONG PING PONG PING PONG` (§12.12):

```
static class Mailbox<T> {                                // one slot: wait/notifyAll
    private T item;                                      // guarded by 'this'
    public synchronized void put(T x) throws InterruptedException {
        while (item != null) wait();                     // ALWAYS wait in a loop
        item = x;
        notifyAll();                                     // wake the taker(s)
    }
    public synchronized T take() throws InterruptedException {
        while (item == null) wait();
        T x = item; item = null;
        notifyAll();                                     // wake the putter(s)
        return x;
    }
}
// TurnTable (ReentrantLock + Condition): two threads alternate PING / PONG
lock.lock();
try {
    while (pingsTurn != iAmPing) turnChanged.await();    // releases lock while waiting
    out.add(word);
    pingsTurn = !pingsTurn;
    turnChanged.signalAll();
} finally { lock.unlock(); }
```

Trade-offs

	`wait/notify`	`Lock` + `Condition`
Conditions per lock	One wait set	Many (e.g. `notFull`, `notEmpty`)
Targeted wake-up	No — use `notifyAll`	Yes — `signal()` on the right condition
Timed wait	`wait(ms)`	`await(t, unit)`, `awaitNanos` (returns time left)
Verbosity	Less	More (manual unlock)

Usually you don't need raw condition variables: `BlockingQueue`, `Semaphore`, `CountDownLatch`, `CompletableFuture` solve the common waiting problems. Write your own only for custom coordination (interview problems, custom synchronizers).

Common mistakes

- `if` instead of `while` around `wait()`.
- `wait()` / `notify()` without holding that object's monitor → `IllegalMonitorStateException`; or calling `wait()` on one object while synchronized on another.
- Changing state *without* notifying → waiters sleep forever (a "lost wakeup").

Real-world examples

`ArrayBlockingQueue` uses one `ReentrantLock` with two `Condition`s, `notEmpty` and `notFull` — the pattern built by hand in chapter 13.

In the interview

> **30-second answer:** "A condition variable lets a thread release a lock and sleep until another thread changes the state and signals it. I always wait in a while loop that re-checks the condition, because of spurious wakeups and competing threads. With one monitor I use notifyAll; with ReentrantLock I use separate Conditions and targeted signal."

Follow-ups:

- *Why call `wait()` while holding the lock?* So check-then-sleep is atomic; otherwise a notify could land between your check and your wait and be lost.
- *What does `wait()` do with the monitor?* Releases it while sleeping and re-acquires it before returning.

TL;DR

Condition variable = "release lock, sleep until signalled, re-acquire, re-check". Always `while`, prefer `notifyAll` or separate `Condition`s.

12.14 `CountDownLatch` and `CyclicBarrier`

What it is

- **`CountDownLatch(n)`**: a one-time gate. Threads call `await()` until `countDown()` has been called *n* times. Cannot be reset.
- **`CyclicBarrier(n, action)`**: a reusable meeting point. Each of *n* threads calls `await()`; when the *n*th arrives, the optional barrier action runs once, all are released, and the barrier resets.

The problem it solves

Coordinating *phases*: "start only after all services are up" (latch), "all workers finish step 1 before anyone starts step 2" (barrier), "release all test threads at the same instant" (latch with count 1 — used throughout this book to maximise contention).

How it works

	CountDownLatch	CyclicBarrier
Who waits	Any thread(s) calling `await()`	The *n* participating threads themselves
Who counts	Anyone calling `countDown()` (need not wait)	Arrival at `await()`
Reusable	No	Yes (auto-reset after each trip)
Extra	Count reaching 0 is permanent: later `await()` calls return at once. One thread may call `countDown()` several times	Barrier action runs once per trip (by the last thread to arrive); if one party is interrupted/times out, the barrier **breaks** and others get `BrokenBarrierException` (`reset()` repairs it)
Happens-before	Actions before `countDown()` HB actions after the matching `await()` returns	Actions before `await()` HB the barrier action, which HB actions after the other parties return from `await()`

`Phaser` (Java 7) generalises both: parties register/deregister dynamically across many phases.

Java example

From `coordination/Main.java` — output `all services ready: true (count now 0)`, `phase 1 complete…`, `phase 2 complete…` (§12.12):

```
CountDownLatch ready = new CountDownLatch(3);
for (String svc : List.of("db", "cache", "search"))
    new Thread(() -> { /* ...warm up... */ ready.countDown(); }, svc).start();
boolean allUp = ready.await(2, TimeUnit.SECONDS);       // one-shot: cannot be reset

CyclicBarrier barrier = new CyclicBarrier(3,            // action runs once per trip
        () -> System.out.println("phase " + phase.incrementAndGet() + " complete: all 3 workers arrived"));
// each of 3 workers, twice: sleep a random time, then barrier.await()  (reusable every phase)
```

Trade-offs

Latch: dead simple, one-shot. Barrier: reusable but fragile — one failing party breaks it for all. For "wait for these N tasks and collect results", `ExecutorService.invokeAll` or `CompletableFuture.allOf(...)` is usually clearer.

Common mistakes

- Waiting forever because a worker threw before `countDown()` — put `countDown()` in `finally`; use `await(timeout)`.
- A barrier with fewer threads than parties → everyone waits forever.

Real-world examples

- Test harnesses release N threads together with a `CountDownLatch(1)` to maximise races (JCStress — OpenJDK's harness for testing concurrency correctness — uses stronger tools, same idea).
- Application start-up: wait for caches, DB pools and message consumers before accepting traffic.

In the interview

> **30-second answer:** "`CountDownLatch` is a one-shot gate that opens after N countDowns; `CyclicBarrier` is a reusable rendezvous where N threads wait for each other each phase, optionally running an action when all arrive."

Follow-ups:

- *Can a latch be reset?* No — create a new one or use `CyclicBarrier` / `Phaser`.
- *3 workers syncing at the end of every simulation step?* `CyclicBarrier(3)`.

TL;DR

Latch = one-shot "wait for N events". Barrier = reusable "wait for N threads". Put `countDown` in `finally`; use timeouts.

Part E — Lock-free

12.15 Compare-and-swap (CAS) & atomics

What it is

Compare-and-swap is a single CPU instruction (`LOCK CMPXCHG` on x86; LL/SC — load-linked/store-conditional — or CAS instructions on ARM): "if the value at this address is still `expected`, replace it with `new` and report success; otherwise change nothing and report failure." Java exposes it through `java.util.concurrent.atomic` — `AtomicInteger`, `AtomicLong`, `AtomicBoolean`, `AtomicReference`, `AtomicIntegerArray`, `LongAdder` — and through `VarHandle`.

The problem it solves

Atomic updates **without locks** (**lock-free**: no thread ever blocks another, and some thread always makes progress): no blocking, no deadlock, usually faster under low-to-moderate contention. Lock-free data structures and the JDK's own concurrency utilities are built on it.

How it works

The **CAS retry loop**: read the current value → compute the new value → `compareAndSet(current, new)`; if it fails, another thread changed the value — start again. Helpers run the loop for you: `incrementAndGet`, `addAndGet`, `updateAndGet(fn)`, `accumulateAndGet(x, fn)` (the function may run more than once, so it must be side-effect-free).

`LongAdder` spreads updates over several internal cells when contended and sums them on `sum()` — much faster than `AtomicLong` for hot counters (metrics), but `sum()` is not an atomic snapshot.

ABA problem. A value goes A → B → A; a thread that read A earlier sees A again and its CAS succeeds, though the state changed. It matters for pointer-based structures (reusing nodes in a lock-free stack). Fix: attach a version — `AtomicStampedReference` (value + int stamp) or `AtomicMarkableReference`. (In Java, garbage collection prevents the classic node-reuse ABA for freshly allocated nodes, which is why the Treiber stack below is safe.)

Java example

Essential lines of `code/lld/ch12/atomics/Main.java`:

```
static int casIncrement(AtomicInteger a) {              // what incrementAndGet does inside
    while (true) {
        int current = a.get();                          // 1. read
        int next = current + 1;                         // 2. compute
        if (a.compareAndSet(current, next)) return next; // 3. write only if unchanged
    }                                                   // 4. someone won the race: retry
}

static class LockFreeStack<T> {                          // Treiber stack: one AtomicReference
    private record Node<T>(T value, Node<T> next) { }
    private final AtomicReference<Node<T>> head = new AtomicReference<>();
    void push(T v) {
        Node<T> oldHead, newHead;
        do { oldHead = head.get(); newHead = new Node<>(v, oldHead); }
        while (!head.compareAndSet(oldHead, newHead));
    }
    T pop() {
        Node<T> oldHead;
        do { oldHead = head.get(); if (oldHead == null) return null; }
        while (!head.compareAndSet(oldHead, oldHead.next()));
        return oldHead.value();
    }
}

highWater.accumulateAndGet(i, Math::max);                // atomic "max"
LongAdder hits = new LongAdder();                        // 8 threads x 100,000 increments

AtomicReference<String> ref = new AtomicReference<>("A"); // ABA
String seen = ref.get(); ref.set("B"); ref.set("A");
ref.compareAndSet(seen, "C");                            // true: plain CAS can't tell
AtomicStampedReference<String> sref = new AtomicStampedReference<>("A", 0);
// read value + stamp 0, then set("B", 1), set("A", 2):
sref.compareAndSet(seen2, "C", seenStamp, seenStamp + 1); // false: the stamp moved
```

Output (full program; 4 threads in each of the first three demos):

```
hand-written CAS loop counter = 400000
atomic max                    = 999
lock-free stack popped        = 40000
LongAdder sum                 = 800000
plain CAS after A->B->A succeeds   : true
stamped CAS after A->B->A succeeds : false
```

Trade-offs

	Locks	CAS / atomics
Blocking	Yes	No (threads retry)
Deadlock	Possible	Impossible (no locks)
Scope	Any number of variables	One variable/reference per CAS
Under heavy contention	Threads sleep (saves CPU)	Many failed retries (wastes CPU) — use `LongAdder` or striping
Difficulty	Moderate	Hard for anything beyond counters/flags/one reference

Invariants spanning several variables (moving money between two accounts) can't use a single CAS — use a lock, or put all fields into one immutable object and CAS the reference (as chapter 13's lock-free token bucket does).

Common mistakes

- Separate `get()` then `set()` on an atomic → still a race. Use `compareAndSet`/`updateAndGet`.
- Side effects inside `updateAndGet` lambdas (they may run several times).
- `AtomicLong` for a very hot metrics counter where `LongAdder` scales better.

Real-world examples

- `ConcurrentHashMap` inserts into empty bins with CAS; `ConcurrentLinkedQueue` is a lock-free (Michael–Scott) queue; AQS (behind `ReentrantLock`) CASes its state.

- Optimistic locking in databases and ORMs (Object-Relational Mappers — libraries mapping objects to tables; JPA `@Version`, `UPDATE ... WHERE version = ?`) is CAS at the database level.
- LMAX Disruptor (a high-performance inter-thread messaging library) uses CAS-based sequences for its lock-free **ring buffer** (a fixed-size array used as a circular queue).

In the interview

> **30-second answer:** "CAS atomically updates a value only if it still equals what I read; otherwise I retry. Java's atomic classes wrap it, so counters, flags and single references can be updated without locks and without deadlock. It covers one variable at a time, can suffer ABA — fixed with a stamped reference — and under heavy contention I prefer `LongAdder`."

Follow-ups:

- *Is lock-free always faster?* No. Under heavy contention retries waste CPU; a lock that parks threads can be better.
- *Lock-free update of two fields?* Put them in an immutable object and CAS an `AtomicReference` to it.
- *A failed `compareAndSet` means?* Another thread changed the value after you read it; re-read and retry.

TL;DR

CAS = "update only if unchanged", retried in a loop. Atomics give lock-free, deadlock-free updates to one variable. Watch ABA; use `LongAdder` for hot counters.

12.16 Concurrent collections

What it is

Thread-safe collections in `java.util.concurrent` built for high concurrency (unlike the old fully-`synchronized` `Vector` / `Hashtable`):

Need	Use	Key behaviour
Map	`ConcurrentHashMap`	Lock-free reads; writes lock only one bin; atomic `putIfAbsent`, `computeIfAbsent`, `compute`, `merge`; **no null keys/values**; *weakly consistent* iterators (below)
Sorted map/set	`ConcurrentSkipListMap` / `Set`	Lock-free, sorted, O(log n) (cost grows very slowly with size)
List read ≫ write	`CopyOnWriteArrayList` / `Set`	Each write copies the whole array; iteration uses a snapshot (never throws CME, `ConcurrentModificationException`)
Queue (non-blocking)	`ConcurrentLinkedQueue` / `Deque`	Lock-free; `poll()` returns null when empty
Queue (blocking)	`ArrayBlockingQueue` (bounded), `LinkedBlockingQueue` (optionally bounded), `PriorityBlockingQueue`, `DelayQueue`, `SynchronousQueue` (no capacity: hand-off), `LinkedTransferQueue`	`put` waits when full, `take` waits when empty; `offer` / `poll` with timeouts

Two `ConcurrentHashMap` guarantees worth quoting exactly:

- **`computeIfAbsent` / `compute` / `merge` are atomic.** The whole call is performed atomically, and for `computeIfAbsent` the mapping function is invoked **exactly once if the key is absent, otherwise not at all** — two racing threads never both create a value for one key. Some updates by other threads (at least those hitting the same bin) block while the function runs, so keep it short and never touch the same map inside it. (The default `ConcurrentMap.computeIfAbsent` used by some other maps, e.g. `ConcurrentSkipListMap`, may call the function more than once and discard results — only the final put is atomic.)
- **Iterators are weakly consistent.** They never throw `ConcurrentModificationException`, may proceed during modification, return each element at most once, and reflect the map's state at some point at or after creation — later updates *may or may not* show. `size()` / `isEmpty()` are likewise only estimates while updates are in flight.

The problem it solves

Wrapping a `HashMap` in one lock **serialises** every access (one at a time). Concurrent collections let threads work on different parts simultaneously and offer **atomic compound operations** so common patterns need no external lock.

How it works

1. Pick the collection by access pattern (table above).
2. Replace check-then-act sequences with one atomic method:
   - `if (!m.containsKey(k)) m.put(k, v)` → `m.putIfAbsent(k, v)`
   - `m.put(k, m.get(k) + 1)` → `m.merge(k, 1, Integer::sum)`
   - lazy creation → `m.computeIfAbsent(k, key -> new ...)`
3. Hand-offs between threads → a `BlockingQueue` (producer-consumer, §12.22).
4. *Compound operations across several calls are still not atomic.*

Java example

Essential lines of `code/lld/ch12/collections/Main.java`:

```
for (String w : words) wordCount.merge(w, 1, Integer::sum);       // 4 threads x 1000 reps: exact totals
ordersByUser.computeIfAbsent("alice", k -> new CopyOnWriteArrayList<>()).add("order-1");  // create once
seats.putIfAbsent("A1", "bob");      // null = you won; the existing value = someone else has it

CopyOnWriteArrayList<String> listeners = new CopyOnWriteArrayList<>(List.of("L1", "L2"));
for (String l : listeners) listeners.add(l + "-new");             // iterates a snapshot: no CME

BlockingQueue<Integer> q = new ArrayBlockingQueue<>(2);           // bounded = back-pressure
q.offer(3);                          // false when full;  q.poll(50, MILLISECONDS) -> null when empty
```

Output (full program):

```
word counts (4 threads x 1000 reps): {apple=12000, banana=8000, cherry=4000}
alice's orders: [order-1, order-2]
first putIfAbsent returns : null
second putIfAbsent returns: bob  (seat already bob's)
iterated 2 items; list now [L1, L2, L1-new, L2-new]
offer 1: true, offer 2: true, offer 3: false (full)
take -> 1, poll -> 2, poll(50ms) on empty -> null
```

Trade-offs

Collection	Great for	Bad for
`ConcurrentHashMap`	General shared maps, caches, counters	Multi-key atomic operations; `size()` is only an estimate under concurrent updates
`CopyOnWriteArrayList`	Listener lists, rarely-changed config	Frequent writes (O(n) copy each)
`Collections.synchronizedMap`	Legacy code, low contention	High contention; iteration still needs manual `synchronized(map)`
`ArrayBlockingQueue`	Bounded hand-off with back-pressure	Very high throughput (single lock for put/take)
`LinkedBlockingQueue`	Higher throughput (separate put/take locks)	Default constructor is **unbounded** (capacity `Integer.MAX_VALUE`)

Thread-confined objects are faster as plain `HashMap` / `ArrayList`; a transaction across several keys is clearer with one lock (or a database).

Common mistakes

- Slow work or calling back into the same map inside `computeIfAbsent` / `compute` — the function runs with the bin locked; recursive updates can throw `IllegalStateException` or deadlock.
- `null` in a `ConcurrentHashMap` → `NullPointerException`.
- Unbounded queues that silently grow until `OutOfMemoryError`.

Real-world examples

- Caffeine's (a Java caching library) cache is built on `ConcurrentHashMap`; Guava's (Google's core Java library) `LocalCache` uses a segmented design derived from Java 7's `ConcurrentHashMap`.
- Guava's `EventBus` keeps each event type's subscribers in a `CopyOnWriteArraySet` — listener lists are the textbook copy-on-write case.
- Log4j2's async **appenders** (components writing log events to a destination) and other logging frameworks hand events to a writer thread through queues.

In the interview

> **30-second answer:** "I use `ConcurrentHashMap` for shared maps and its atomic methods — `putIfAbsent`, `computeIfAbsent`, `merge` — instead of check-then-act. `CopyOnWriteArrayList` for read-mostly lists like listeners. `BlockingQueue`, preferably bounded, to hand work between threads."

Follow-ups:

- *How does Java 8+ `ConcurrentHashMap` achieve concurrency?* Lock-free (volatile) reads. Writes CAS into empty bins, otherwise `synchronized` on the bin's first node; long bins become **red-black trees** (self-balancing sorted binary trees); resizing is cooperative across threads. (Java 7 used 16 lock-striped segments by default.)
- *Why no nulls?* `get(k) == null` must unambiguously mean "absent"; you can't follow up with `containsKey` atomically.
- *Atomic word count?* `map.merge(word, 1, Integer::sum)`. *Why no CME on a COW list?* It iterates an immutable snapshot.

TL;DR

`ConcurrentHashMap` + atomic methods for maps, COW lists for read-mostly lists, bounded `BlockingQueue` for hand-offs. Single calls are atomic; sequences of calls are not.

Part F — Liveness failures

12.17 Deadlock

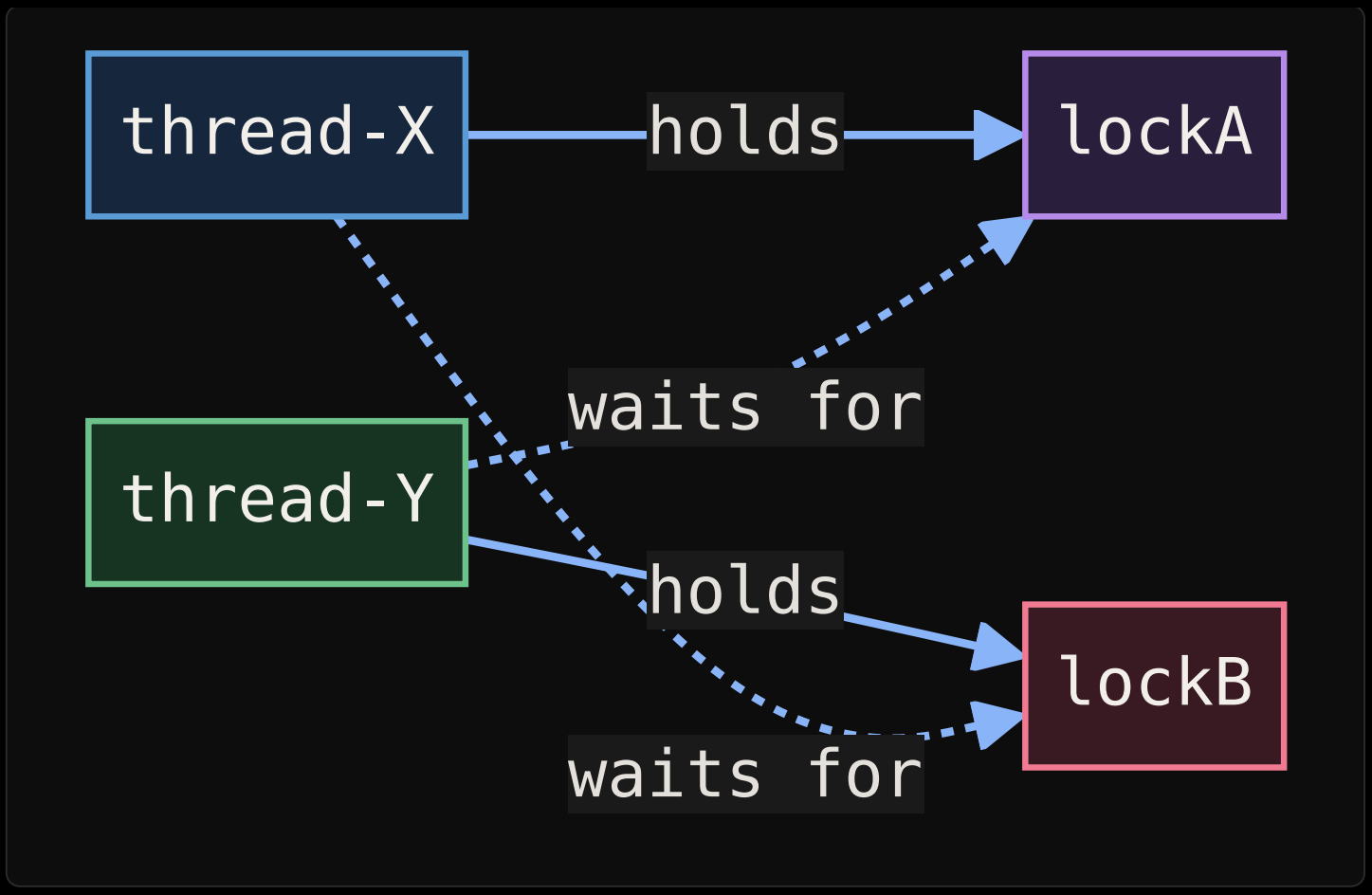
What it is

A **deadlock** is when two or more threads each wait forever for a resource another one holds. Nobody can proceed.

The problem it solves

Understanding deadlock lets you *design it out*. It requires all four **Coffman conditions** (Coffman et al., 1971) at once — break **any one** and deadlock is impossible:

#	Condition	Meaning	How to break it
1	**Mutual exclusion**	A resource can be held by only one thread	Lock-free/immutable data, or shareable resources (read locks)
2	**Hold and wait**	A thread holds one resource while waiting for another	Acquire all at once, or `tryLock` and release everything on failure
3	**No preemption**	Resources can't be taken away; only the holder releases	Timeouts (`tryLock(timeout)`), lock **leases** (locks that expire automatically)
4	**Circular wait**	A cycle: T1 waits for T2's lock, T2 waits for T1's	**Global lock ordering** (always lock lower id first)



How it works

1. **Prevention by lock ordering (most common):** a total order over locks (account id, seat id, `System.identityHashCode` with a tie-breaker lock); always acquire in that order.
2. **Prevention by try-lock:** `tryLock(timeout)` each lock; on failure release all held locks, wait a random short time, retry.
3. **Avoid nested locks:** call alien code (listeners, callbacks) *outside* your lock.
4. **Detection:** thread dumps (`jstack` / `jcmd`) print "Found one Java-level deadlock"; programmatically, `ThreadMXBean.findDeadlockedThreads()` (the JDK's management interface for inspecting threads).
5. **Avoidance** (Banker's algorithm — grant a request only if every thread can still finish) exists in OS theory but is rarely used in application code.

Java example

Essential lines of `code/lld/ch12/deadlock/Main.java`:

```java
// Real deadlock: opposite lock order; a CyclicBarrier(2) forces the bad interleaving every time.
Thread x = new Thread(() -> { synchronized (lockA) { await(bothHoldOne); synchronized (lockB) { } } }, "thread-X");
Thread y = new Thread(() -> { synchronized (lockB) { await(bothHoldOne); synchronized (lockA) { } } }, "thread-Y");
x.setDaemon(true); y.setDaemon(true);                    // daemon: the JVM can still exit
long[] ids = ManagementFactory.getThreadMXBean().findDeadlockedThreads();   // polled until non-null

// Fix 1: global lock ordering (smaller id first) - circular wait impossible
static void transferOrdered(Account from, Account to, int amount) {
    Account first = from.id < to.id ? from : to, second = from.id < to.id ? to : from;
    first.lock.lock();
    try {
        second.lock.lock();
        try { if (from.balance >= amount) { from.balance -= amount; to.balance += amount; } }
        finally { second.lock.unlock(); }
    } finally { first.lock.unlock(); }
}

// Fix 2: tryLock + random back-off - breaks "hold and wait"
while (true) {
    if (from.lock.tryLock(1, TimeUnit.MILLISECONDS)) {
        try {
            if (to.lock.tryLock(1, TimeUnit.MILLISECONDS)) {
                try { /* move money */ return; } finally { to.lock.unlock(); }
            }
        } finally { from.lock.unlock(); }              // give up what we hold
    }
    // Without random back-off both threads could retry in lock-step forever (LIVELOCK).
    Thread.sleep(0, ThreadLocalRandom.current().nextInt(1, 100_000));
}
```

Output (full program; 2 × 50,000 opposite-direction ordered transfers, 2 × 5,000 try-lock transfers — money is conserved):

```
deadlock detected between: [thread-X, thread-Y]  (states BLOCKED, BLOCKED)
lock ordering: finished, total money = 2000
tryLock + back-off: finished, total money = 2000
```

Trade-offs

Strategy	Pros	Cons
Lock ordering	Simple, zero runtime cost	Needs a stable global order; hard across modules
Try-lock + back-off	Works without an order; bounded waits	Retry logic; possible livelock without randomness
Single coarse lock	No deadlock among your locks	Lower throughput
Lock-free / immutable	No locks at all	Harder to design

You always want to prevent deadlock — but don't add complicated detection/recovery in application code when simple lock ordering would prevent it.

Common mistakes

- Calling a listener or another object's synchronized method while holding your own lock (hidden nested locking).
- Ordering by `hashCode()` without handling equal hashes.
- *Resource* deadlocks: a pool of 10 threads where each task submits a sub-task to the **same** pool and waits for it — when all 10 wait, nothing runs (thread-starvation deadlock).

Real-world examples

- Databases (MySQL InnoDB, PostgreSQL) detect deadlocks between **transactions** (groups of operations that succeed or fail as one unit) and abort one as a victim — "detect and recover".
- Bank transfer systems lock accounts in id order — lock ordering.

In the interview

> **30-second answer:** "Deadlock needs four conditions: mutual exclusion, hold-and-wait, no preemption, and circular wait. I break circular wait with a global lock order — e.g. lock seats or accounts in ascending

id — or break hold-and-wait with tryLock with timeout and release-and-retry. I avoid calling external code while holding a lock, and I'd detect it in production with thread dumps."

Follow-ups:

- *Two accounts with the same id?* Same account → one lock (transfer to self is a no-op or rejected).
- *Find a deadlock in production?* `jstack <pid>` or `jcmd <pid> Thread.print`; monitoring can call `ThreadMXBean.findDeadlockedThreads()`.
- *Which condition does lock ordering break?* Circular wait. *`tryLock` + release?* Hold and wait (and effectively no-preemption, by giving up).

TL;DR

Deadlock = cycle of waiting. Needs all 4 Coffman conditions. Break one: lock ordering (circular wait) or tryLock + release (hold and wait). Don't call alien code under a lock.

12.18 Livelock

What it is

A **livelock** is when threads are not blocked — they keep running and reacting to each other — but make **no progress** (two people in a corridor both stepping aside the same way, again and again).

The problem it solves

Recognising it. Livelocks often appear *as a side effect of deadlock avoidance*: threads that both `tryLock`, fail, release and retry at the same rhythm can collide forever.

How it works (and how to fix it)

Thread 1 takes A, tries B, fails, releases A; thread 2 takes B, tries A, fails, releases B; both retry immediately → same collision forever. **Fixes:** random back-off (different waits break the symmetry), exponential back-off, a consistent lock order (no retries needed), or a priority/tie-breaker (e.g. the older transaction wins).

Java example

`transferTryLock` in `deadlock/Main.java` sleeps a **random** number of nanoseconds (`ThreadLocalRandom`) after a failed attempt; without it the threads could, in principle, fail in lock-step forever.

Trade-offs

Random back-off makes progress very likely but adds latency; exponential back-off adapts to contention. Never design retry loops that react only to each other with no randomness or tie-breaker.

Common mistakes

- Fixed retry delays (`sleep(10)` for everyone).
- Retrying forever without a limit or alert.

Real-world examples

- **Ethernet (CSMA/CD)** (classic wired-network rule: listen before sending, detect collisions) uses *binary exponential back-off* (doubling the maximum random wait after each collision) for exactly this reason.
- AWS SDK (Amazon Web Services client libraries) and Resilience4j retry policies use exponential back-off **with jitter** (a small random amount added to each wait) to stop **retry storms** (crowds of clients retrying at the same moment).

In the interview

30-second answer: "Livelock is when threads keep reacting to each other without progress — like two people side-stepping in a corridor. It often comes from symmetric retry logic; I fix it with randomized or exponential back-off, or remove retries by using a lock order."

Follow-up: *Thread states, deadlock vs livelock?* Deadlocked threads are blocked/waiting; livelocked threads are `RUNNABLE` but achieve nothing.

TL;DR

Livelock = busy but no progress. Break symmetry with jittered back-off or a tie-breaker.

12.19 Starvation (and fairness)

What it is

Starvation is when a thread waits indefinitely because others keep getting the resource first, even though the system as a whole progresses.

The problem it solves

Recognising it. Causes: unfair (barging) locks under heavy contention, a writer behind a constant stream of readers in a read-write lock, low thread priority, a long task hogging a single-thread executor.

How it works (fixes)

1. **Fair** locks/semaphores where it matters: `new ReentrantLock(true)`, `new Semaphore(n, true)`, `new ReentrantReadWriteLock(true)` (fair = longest waiter next).
2. Short critical sections, so the lock is free more often.
3. Separate pools for slow and fast work (bulkheads).
4. Don't rely on `Thread.setPriority` — its effect is OS-dependent.

Java example

Chapter 13's Dining Philosophers `Waiter` uses `new Semaphore(n - 1, true)` so no philosopher waits forever for a seat.

Trade-offs

Fairness prevents starvation but reduces throughput (more context switches, no barging). Don't make every lock fair "just in case" — only where starvation is seen or arguable.

Common mistakes

Confusing starvation with deadlock: in starvation *someone* progresses; in deadlock *nobody* does.

Real-world examples

OS schedulers use *aging* (raising the priority of long-waiting processes); Linux's schedulers (CFS, and its successor EEVDF since **kernel** — the core of the OS — 6.6) aim to give every runnable task a fair share of CPU.

In the interview

> **30-second answer:** "Starvation is when a thread never gets a resource because others always win — e.g. writers behind endless readers. Fair locks, short critical sections and separate pools fix it, at some throughput cost." (Writer starvation in `ReentrantReadWriteLock`: construct it fair.)

TL;DR

Starvation = some thread never gets a turn. Fairness, short sections, separate pools.

Part G — Concurrency patterns

The next five patterns share one runnable file, `code/lld/ch12/patterns/Main.java`; each section quotes its essential lines. Output of the full program:

```
=== Signaling: B must run after A ===
A: charge payment
B: send email (after payment)
=== Thread pool ===
hand-made pool ran 20 tasks on 3 threads
ThreadPoolExecutor + CompletableFuture: total = 118
=== Producer-consumer ===
consumed sum of 1..1000 = 500500 (expected 500500)
=== Immutability ===
o1 = [pen], o2 = [pen, book]
=== Thread confinement (ThreadLocal) ===
Hello, T1
Hello, T2
```

12.20 Signaling pattern

What it is

One thread **notifies** another that an event happened so the second can proceed: "B must run after A" (a relay baton hand-over).

The problem it solves

Enforcing ordering between threads without busy-waiting (spinning on a flag).

How it works

1. Create a signal object in the "not yet" state: `new Semaphore(0)`, `new CountDownLatch(1)`, a `CompletableFuture`, or a flag + condition variable.
2. The waiter calls `acquire()` / `await()` / `join()`; the signaller calls `release()` / `countDown()` / `complete(value)`.
3. The waiter wakes; the signal also provides happens-before, so it sees everything A did.

Tool	Reusable	Carries a value	Good for
`Semaphore(0)`	Yes (each release = one signal)	No	Ping-pong between threads (FooBar, chapter 13)
`CountDownLatch(1)`	No	No	One-time "ready/go"
`CompletableFuture`	No	Yes	"Here is the result"
Condition variable	Yes	Via shared state	Complex conditions

Java example

B starts first but must wait; A's line always prints first:

```java
Semaphore aDone = new Semaphore(0);                    // 0 permits = "not signalled yet"
Thread b = new Thread(() -> { aDone.acquire(); log.add("B: send email (after payment)"); });
Thread a = new Thread(() -> { log.add("A: charge payment"); aDone.release(); });
b.start(); a.start();
```

Trade-offs

Semaphores are compact but easy to mis-count; latches can't be reset; futures are clean but single-use. If the "signal" is really "here's a work item", use a `BlockingQueue` (producer-consumer).

Common mistakes

Signalling before the waiter waits, with a tool that *forgets* signals (plain `notify()` without a state flag = lost wakeup). Semaphores and latches remember.

Real-world examples

`CompletableFuture` chains in Spring WebFlux (Spring's non-blocking web framework)/Netty **callbacks** (functions handed to a library so it can call you back later); start-up gates; "order paid → send confirmation" flows inside one service.

In the interview

"Signaling means one thread waits for an event raised by another. I use a zero-permit semaphore for repeated hand-offs, a latch for one-time events, or a `CompletableFuture` when a result travels with the signal." *Why is a bare `notify()` risky?* If it fires before the other thread waits, the signal is lost; pair it with a state variable checked in a loop.

TL;DR

Signal = "you may go now". Semaphore(0), latch, or future. Make sure signals can't be lost.

12.21 Thread pool pattern

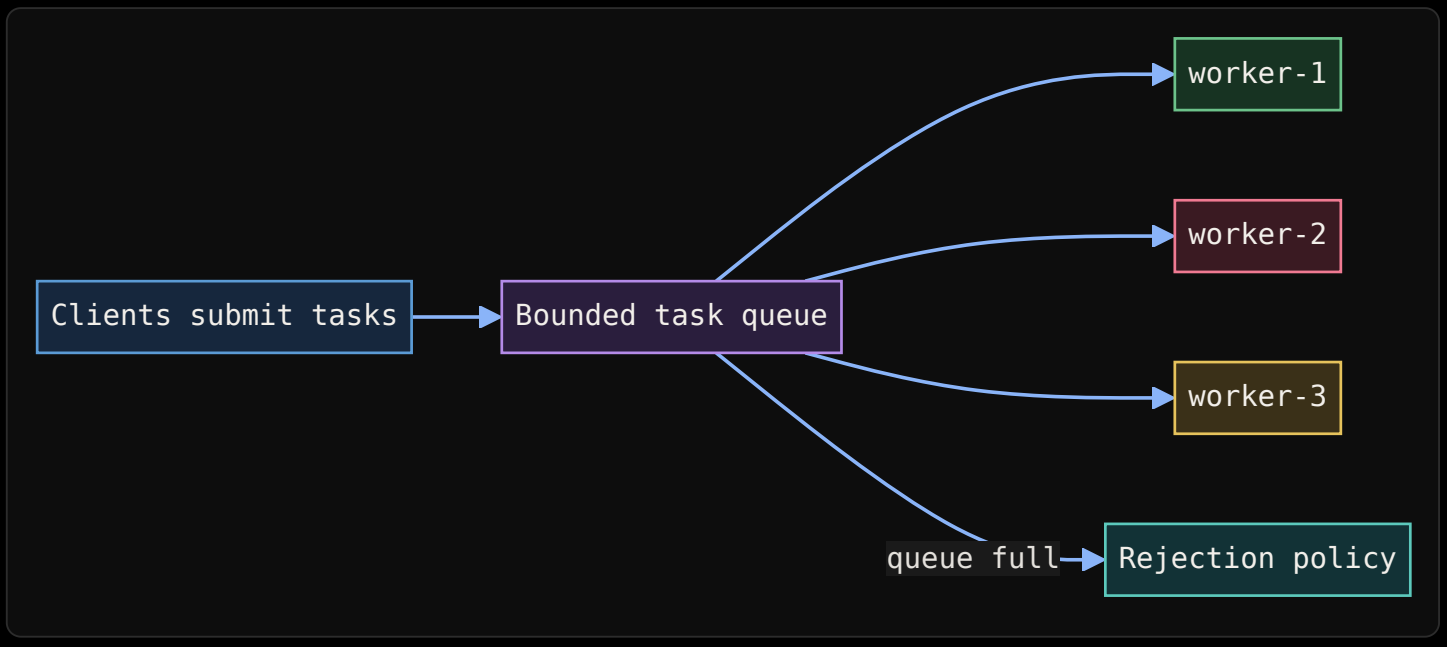
What it is

A fixed set of **worker threads** takes tasks from a shared **queue** and runs them, reusing threads instead of creating one per task.

The problem it solves

Creating/destroying threads per task is expensive, and unbounded concurrency can overwhelm CPU, memory and downstream systems. A pool bounds concurrency and **amortises** thread creation (spreads its one-time cost over many tasks).

How it works



1. Workers loop: `task = queue.take(); task.run();`; `submit` enqueues.
2. `ThreadPoolExecutor` rules: while fewer than **core** threads run, a new thread per task; then tasks go into the **queue**; if the queue is full, extra threads up to **max**; if still full, the **rejection policy** runs (`AbortPolicy` throws — default; `CallerRunsPolicy` runs it on the submitting thread, slowing producers; `DiscardPolicy`; `DiscardOldestPolicy`).
3. Shutdown: stop accepting, let workers drain the queue, exit (the hand-made pool uses one "**poison pill**" — a marker task meaning "stop now" — per worker). "Is it shut down? then enqueue" in `submit` is a check-then-act, so `submit` and the pill-adding step hold the same lock — otherwise a task could land *behind* the pills and never run.

Sizing: CPU-bound ≈ number of cores; I/O-bound ≈ cores × (1 + wait/compute) — or virtual threads.

Java example

```
static class SimpleThreadPool {                          // hand-made, to see the moving parts
    private static final Runnable POISON = () -> { };
    private final BlockingQueue<Runnable> tasks = new LinkedBlockingQueue<>();
    private boolean shutdown;                            // guarded by 'this'
    // each worker: while (true) { Runnable r = tasks.take(); if (r == POISON) return; r.run(); }
    synchronized void submit(Runnable r) {
        if (shutdown) throw new RejectedExecutionException("pool is shut down");
        tasks.add(r);
    }
    void shutdownAndWait() throws InterruptedException {
        synchronized (this) {
            if (shutdown) return;
            shutdown = true;
            for (int i = 0; i < workers.size(); i++) tasks.add(POISON);   // one pill per worker
        }
        for (Thread w : workers) w.join();               // wait OUTSIDE the lock
    }
}

ThreadPoolExecutor exec = new ThreadPoolExecutor(        // production version
        2, 4, 30, TimeUnit.SECONDS,                      // core, max, idle time of extra threads
        new ArrayBlockingQueue<>(100),                   // bounded: no OOM from a huge backlog
        new ThreadPoolExecutor.CallerRunsPolicy());      // when full, submitter runs it (back-pressure)
CompletableFuture<Integer> price = CompletableFuture.supplyAsync(() -> 100, exec);
CompletableFuture<Integer> tax = CompletableFuture.supplyAsync(() -> 18, exec);
price.thenCombine(tax, Integer::sum).join();             // 118
```

Trade-offs

Choice	Effect
Bounded queue	Protects memory; needs a rejection policy
Unbounded queue (`newFixedThreadPool`)	Never rejects, may run out of memory, and max-pool-size is never used
`CallerRunsPolicy`	Natural back-pressure: submitter slows down
Many pools (bulkheads)	Isolation between workloads; more threads overall

For huge numbers of blocking tasks on Java 21, virtual-thread-per-task is simpler. For recursive CPU work, use `ForkJoinPool` (**work stealing**: an idle thread takes queued tasks from a busy thread's queue).

Common mistakes

- Tasks waiting for other tasks in the same bounded pool (thread-starvation deadlock).
- Swallowed exceptions in `submit`ted tasks; forgetting `shutdown()`.
- `ThreadLocal`s in pooled threads not cleared (values leak between tasks).

Real-world examples

Tomcat's connector thread pool (`maxThreads`, default 200), Spring's `ThreadPoolTaskExecutor`, database connection pools (same idea for connections), `ForkJoinPool.commonPool()` behind parallel streams and `CompletableFuture.supplyAsync` without an executor.

In the interview

"A thread pool reuses a bounded set of workers fed by a queue. I size it by workload type, use a bounded queue with an explicit rejection policy for back-pressure, and give slow dependencies their own pool as a bulkhead." *When are threads beyond `corePoolSize` created?* Only when the queue is full (and fewer than max exist).

TL;DR

Workers + queue + rejection policy. Bound everything. Size by CPU vs I/O.

12.22 Producer-consumer pattern

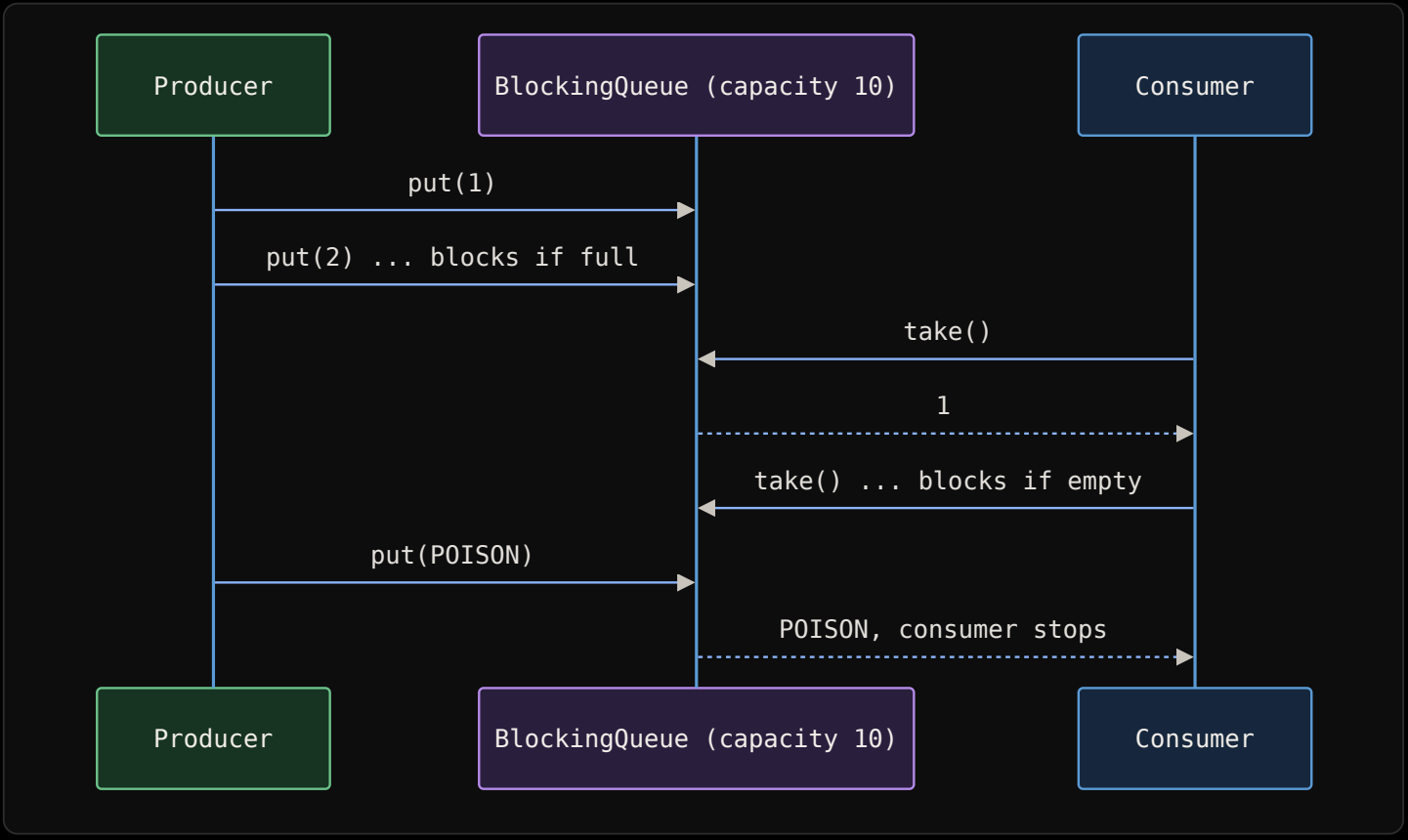
What it is

Producers put work items into a shared **buffer**; **consumers** take them out and process them. The buffer decouples their speeds.

The problem it solves

Producers and consumers run at different rates. A bounded buffer smooths bursts and applies **back-pressure** (producers slow down when consumers fall behind), with neither side knowing about the other.

How it works



Share a bounded `BlockingQueue`; producers `put` (waits when full), consumers `take` (waits when empty). Shutdown: each consumer must learn when to stop — a **poison pill** per consumer, interrupting consumers, or a "done" flag plus `poll(timeout)`.

Java example

1 producer, 2 consumers, capacity 10, two poison pills — the sum is exactly 500500. Chapter 13 builds the blocking queue itself two ways.

```
BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(10);       // producers block when full
// each consumer:
for (int x = queue.take(); x != POISON; x = queue.take()) total.addAndGet(x);
// producer:
for (int i = 1; i <= 1_000; i++) queue.put(i);
for (int c = 0; c < CONSUMERS; c++) queue.put(POISON);             // one pill per consumer
```

Trade-offs

Pros	Cons
Decouples rates, enables back-pressure	Adds latency (queueing)
Easy to scale consumers	Ordering across multiple consumers isn't preserved
Clear separation of concerns	Need a shutdown protocol

When the caller needs the result immediately and synchronously, a direct call is simpler.

Common mistakes

- Unbounded buffer → no back-pressure → memory blow-up.
- One poison pill for N consumers → N−1 consumers hang forever.
- Busy-polling with `poll()` in a loop instead of blocking `take()`.

Real-world examples

Apache **Kafka** (a distributed, durable message log) is producer-consumer across machines: topics are the buffer, **consumer groups** (consumers splitting a topic's messages between them) scale consumers. Inside the Kafka Java producer, application threads append to an in-memory accumulator and a background sender thread ships batches. Also logging async appenders, and `ThreadPoolExecutor` itself (submitters produce, workers consume).

In the interview

"Producer-consumer decouples producers from consumers with a bounded blocking queue: put waits when full, take waits when empty, giving natural back-pressure. I stop consumers with one poison pill each, and scale consumers independently." *Why bounded?* So a slow consumer pushes back instead of the queue growing until memory runs out.

TL;DR

Bounded `BlockingQueue` between producers and consumers. Back-pressure for free. One poison pill per consumer.

12.23 Reader-writer pattern

What it is

Many readers may access shared data at the same time, but a writer needs exclusive access.

The problem it solves

Read-mostly data (config, catalogues, routing tables) shouldn't be serialised through one mutex.

How it works

1. **Read-write lock** (§12.10): `readLock` for readers, `writeLock` for writers; fair mode if writers could starve.
2. **Copy-on-write snapshot**: an immutable object in a `volatile` field. Readers just read the field (no lock); a writer builds a *new* immutable copy and swaps the reference (under a lock or `AtomicReference.updateAndGet` if several writers may race).

```
private volatile Map<String, String> config = Map.of();       // immutable snapshot
String get(String k) { return config.get(k); }                  // readers: no lock
synchronized void put(String k, String v) {                      // writers: serialized
    Map<String, String> copy = new HashMap<>(config);
    copy.put(k, v);
    config = Map.copyOf(copy);                                   // publish new snapshot
}
```

	RW lock	Copy-on-write snapshot
Reader cost	Acquire shared lock	One volatile read
Writer cost	Exclusive lock, in-place update	Copy whole structure
Readers see	Latest data	A consistent snapshot (maybe slightly old)
Best for	Frequent-ish writes, big data	Rare writes, small-to-medium data

Java example

`ConfigStore` (RW lock) in `locks/Main.java`; the snapshot above; `CopyOnWriteArrayList` in `collections/Main.java`.

Trade-offs

See the table; RW-lock readers must not upgrade to write. For write-heavy data a plain mutex or `ConcurrentHashMap` is better.

Common mistakes

Mutating the shared snapshot in place (breaks every reader's assumptions); upgrading read → write locks.

Real-world examples

Feature-flag and configuration clients that reload periodically and swap an immutable snapshot; Linux kernel RCU (read-copy-update) is the same idea at OS level.

In the interview

"For read-heavy data I use either a read-write lock or, if writes are rare, an immutable snapshot in a volatile field that writers replace — readers never block." *What guarantees readers see a fully built snapshot?* The `volatile` write of the reference HB later volatile reads, and the snapshot is immutable.

TL;DR

Many readers, one writer. RW lock, or immutable snapshot + volatile swap.

12.24 Immutability & thread confinement

What it is

Two ways to be thread-safe **without any synchronization**:

- **Immutability**: state can't change after construction (all fields `final`, no setters, no leaking mutable internals). `String`, `Integer`, `LocalDate`, `BigDecimal`, records with immutable components, `List.of(...)`.
- **Thread confinement**: data only one thread ever touches — local variables (stack confinement), `ThreadLocal<T>` (one copy per thread), or one thread owning a structure (event loop, **actor** — an object with private state processing messages from its mailbox one at a time).

The problem it solves

Races need *shared mutable* state. Remove "mutable" (immutability) or "shared" (confinement) and races are impossible.

How it works

Immutable class checklist:

1. Make the class `final` (or a `record`) so subclasses can't add mutability.
2. All fields `private final`.
3. No setters; "modifiers" return new objects (`withItem(...)`).
4. **Defensive-copy** mutable inputs in the constructor (`List.copyOf`, so the caller's later changes can't reach you) and don't expose mutable internals.
5. Don't let `this` escape during construction.

Confinement options: keep data in locals; `ThreadLocal.withInitial(...)` for per-thread helpers (formatters, buffers) — call `remove()` in pooled threads; single-writer design (one thread owns the state, others message it

through a queue).

Java example

```
record Order(String id, List<String> items) {
    Order { items = List.copyOf(items); }                // caller's later add("hacked") can't change o1
    Order withItem(String item) {                        // "modify" = return a new object
        List<String> copy = new ArrayList<>(items); copy.add(item);
        return new Order(id, copy);
    }
}
private static final ThreadLocal<StringBuilder> BUFFER = ThreadLocal.withInitial(StringBuilder::new);
static String render(String name) {
    StringBuilder sb = BUFFER.get();                     // each thread has its own builder
    sb.setLength(0);
    return sb.append("Hello, ").append(name).toString();
}
```

Trade-offs

	Immutability	Confinement
Cost	New object per change (GC pressure — more garbage to collect)	Per-thread copies; ThreadLocal leaks if not cleaned
Sharing	Share freely	Not shareable by definition
Reasoning	Easiest possible	Easy, if confinement is really enforced

Very large objects changed at high frequency cost too much to copy — use a concurrent structure.

Common mistakes

- `final List<String> items` is not immutable if the list is mutable and exposed.
- `ThreadLocal` in pools without `remove()` — values leak into the next task; with many virtual threads heavy `ThreadLocal`s waste memory (*scoped values* are the alternative: a **preview** — an experimental opt-in feature — in Java 21, final in Java 25 via JEP 506).
- Sharing `SimpleDateFormat` (mutable, not thread-safe) instead of immutable `DateTimeFormatter`.

Real-world examples

- `java.time` types and `DateTimeFormatter` are immutable and thread-safe.
- **Netty** and **Vert.x** confine each connection to one event-loop thread, so handlers need no locks; Redis's single main thread is confinement at system level.
- Spring's `@Transactional` (runs a method inside a database transaction) keeps the current transaction in `ThreadLocal`s (via `TransactionSynchronizationManager`).

In the interview

"The easiest thread-safety is none needed: make value objects immutable — final fields, defensive copies, 'with' methods — and confine mutable state to one thread with locals, ThreadLocal, or a single owner thread fed by a queue." *Is a class with* `private final Date created` *immutable?* Not unless it copies the `Date` in and out — `Date` is mutable. Prefer `Instant`.

TL;DR

No shared mutable state → no races. Immutable objects share freely; confined objects aren't shared.

Part H — Concurrency in LLD interviews

12.25 How concurrency shows up in LLD interviews (parking, seat booking, inventory)

What it is

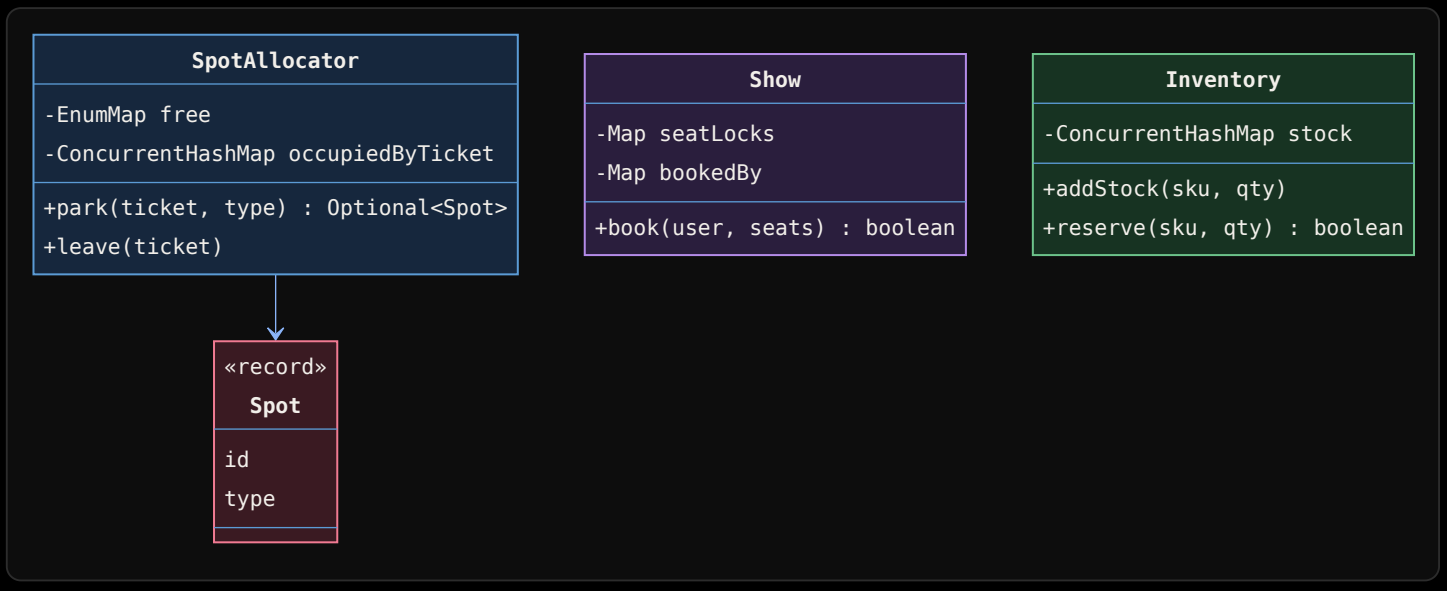
Most LLD case studies have a **contended resource** — a parking spot, a movie seat, a hotel room, a limited stock item, an elevator request, a wallet balance. The interviewer checks that your design prevents double allocation and still performs.

The problem it solves

Turning a single-threaded class diagram into a **thread-safe design** with the smallest correct synchronization.

How it works — a 6-step recipe

1. **Name the shared mutable state** (free spots, seat → owner map, stock counts).
2. **State the invariant** ("a spot has at most one car", "stock ≥ 0", "a seat has at most one booking").
3. **Find the critical operation** (usually check-then-act: *find free → assign*).
4. **Pick the smallest tool that makes it atomic:**
   - one item claimed → atomic op: `queue.poll()`, `map.putIfAbsent`, `AtomicBoolean.compareAndSet`, `AtomicInteger` CAS loop;
   - several items all-or-nothing → per-item locks acquired **in sorted order** (or one per-show/per-floor lock);
   - across processes/servers → database constraints/transactions, optimistic versioning, or a distributed lock.
5. **Choose granularity:** per floor/show/SKU (stock-keeping unit — one distinct product) locks, not one global lock.
6. **Handle time:** temporary holds with expiry (seat held 10 minutes during payment), timeouts, and **idempotent** retries (repeating has the same effect as doing it once; see System Design Fundamentals).



Java example

Essential lines of `code/lld/ch12/lldcases/Main.java`:

```
static class SpotAllocator {                                // 1. Parking: many gates at once
    // One lock-free queue of FREE spots per type. poll() is atomic: two gates never get the same spot.
    private final Map<SpotType, ConcurrentLinkedQueue<Spot>> free = new EnumMap<>(SpotType.class);
    private final ConcurrentHashMap<String, Spot> occupiedByTicket = new ConcurrentHashMap<>();
    Optional<Spot> park(String ticket, SpotType type) {
        Spot s = free.get(type).poll();
        if (s != null) occupiedByTicket.put(ticket, s);
        return Optional.ofNullable(s);
    }
    void leave(String ticket) {
        Spot s = occupiedByTicket.remove(ticket);           // atomic: a double exit is harmless
        if (s != null) free.get(s.type()).offer(s);
    }
    // ConcurrentLinkedQueue.size() is O(n) and only an estimate under change: reports, not logic.
}

static class Show {                                         // 2. Seats: all-or-nothing, one lock per seat
    boolean book(String user, Set<String> seats) {
        List<ReentrantLock> held = new ArrayList<>();
        try {
            for (String id : new TreeSet<>(seats)) {        // global order A1 < A2 < A3: no deadlock
                ReentrantLock l = seatLocks.get(id);
                if (l == null) return false;                // unknown seat
                l.lock(); held.add(l);
            }
            for (String id : seats) if (bookedBy.containsKey(id)) return false;   // someone got one first
            for (String id : seats) bookedBy.put(id, user);
            return true;
        } finally {
            for (int i = held.size() - 1; i >= 0; i--) held.get(i).unlock();
        }
    }
}

static class Inventory {                                    // 3. Stock: never oversell
    boolean reserve(String sku, int qty) {                  // CAS loop: decrement only if enough
        AtomicInteger s = stock.get(sku);
        if (s == null) return false;
        while (true) {
            int cur = s.get();
            if (cur < qty) return false;
            if (s.compareAndSet(cur, cur - qty)) return true;
        }
    }
}
// raceAll(n, task): n tasks on a pool released together by a CountDownLatch(1) to maximise contention
```

Alice books `{A3, A4}` and Bob `{A4, A3}` at the same moment: exactly one wins, no deadlock, both seats get the same owner. Output (full program):

```
parking: 100 cars raced for 10 spots -> parked 10, distinct spots 10, free left 0
booking: 50 users wanted seat A2 -> successful bookings 1
booking: alice {A3,A4} vs bob {A4,A3} (opposite order) -> winners 1, A3 and A4 same owner: true
inventory: 200 buyers, 50 in stock -> sold 50, left 0
```

Trade-offs

Resource	In-memory approach	Across many servers
Parking spot	Lock-free queue per type/floor, or per-floor lock	DB row update `WHERE status='FREE'` + check affected rows
Seat	Per-show or per-seat locks in sorted order; hold with TTL (time-to-live, an expiry)	Unique constraint on (show, seat); short-lived holds in Redis (`SET NX PX`)
Inventory	`AtomicInteger` CAS / `compute`	`UPDATE stock SET qty = qty - 1 WHERE sku = ? AND qty >= 1` ; reservations with expiry

Don't bolt `synchronized` onto every method of every class — interviewers read that as "doesn't know where the contention is". Protect the few operations that touch contended state.

Common mistakes

- A global `synchronized` on `ParkingLot.park()` — correct but serialises every gate.
- Checking availability with one call and booking with another (check-then-act).
- Locking seats in the order the user clicked them (deadlock between two multi-seat bookings).
- Forgetting that in-memory locks don't work across multiple app servers.

Real-world examples

- **BookMyShow / Ticketmaster-style** checkout: seats are *held* for a few minutes while you pay; if payment doesn't complete, the hold expires and the seats return — a lock with a lease.
- **Flash sales** (limited-stock phone launches) decrement stock atomically, often in Redis (`DECR` / Lua scripts, which Redis runs atomically) or with conditional SQL updates.
- **Uber-style dispatch**: offering one driver to one rider at a time is the same "claim a resource atomically" problem.

In the interview

> **30-second answer:** "The critical section is 'find a free X and assign it'. Inside one JVM I make that a single atomic step — a lock-free queue poll for spots, per-seat locks taken in sorted order for multi-seat booking, a CAS decrement for stock. Across servers I'd push it into the database with a conditional update or unique constraint, and add holds with expiry for seats during payment."

Follow-ups (with answers):

- *Two users book the same seat at the same millisecond?* Only one `putIfAbsent`/locked check-and-book succeeds; the other gets "seat unavailable".
- *Release a seat if payment fails or the user leaves?* Store a hold with an expiry; a scheduler or lazy check frees expired holds (chapter 13's TTL cache).
- *Why not one global lock?* Correct but no parallelism: bookings for different shows would wait for each other.
- *Service on 10 machines?* JVM locks don't cross processes: DB transactions/constraints, optimistic locking with a version column, or a distributed lock with a lease.
- *Which Coffman condition does sorting seat ids break?* Circular wait. *Why is `queue.poll()` safe for spots?* A single atomic remove; each spot can be removed by only one thread.

TL;DR

Find the contended resource and its invariant, make "check + claim" one atomic step with the smallest tool, keep locks fine-grained and ordered, and move to DB/distributed mechanisms when there's more than one server.

12.26 Choosing the right tool (cheat sheet)

I need to…	Use
Increment a shared counter	`AtomicInteger` / `AtomicLong`; `LongAdder` if very hot
Stop a loop from another thread	`volatile boolean` flag or `interrupt()`
Make a multi-step update atomic	`synchronized` or `ReentrantLock`
Give up if the lock is busy	`tryLock(timeout)`
Many readers, few writers	`ReentrantReadWriteLock`, `StampedLock`, or immutable snapshot in `volatile`
Limit concurrent access to N	`Semaphore(N)`
Wait for a condition on shared state	`Lock` + `Condition` (or `wait/notifyAll`) in a `while` loop
Wait for N events once	`CountDownLatch`
N threads meet each phase	`CyclicBarrier` / `Phaser`
Hand work between threads	Bounded `BlockingQueue`
Shared map	`ConcurrentHashMap` with `putIfAbsent` / `compute` / `merge`
Read-mostly list	`CopyOnWriteArrayList`
Run tasks	`ExecutorService` (bounded); virtual threads for blocking I/O
Split CPU work	`ForkJoinPool` / parallel streams
Combine async results	`CompletableFuture`
Avoid synchronization entirely	Immutability, thread confinement

12.27 Chapter self-check

1. **Q:** Concurrency vs parallelism in one line each? **A:** Concurrency = many tasks in progress (structure); parallelism = many tasks executing at the same instant (hardware).
2. **Q:** Name the six `Thread.State` values. **A:** NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
3. **Q:** Why is `volatile` not enough for `count++`? **A:** It gives visibility, not atomicity of read-modify-write.
4. **Q:** Three happens-before rules? **A:** Unlock → later lock of the same monitor; volatile write → later read; `start()` → thread's actions; thread's actions → `join()` return.
5. **Q:** Why always `while` around `wait()`? **A:** Spurious wakeups and competing threads; the condition must be re-checked.
6. **Q:** Four Coffman conditions? **A:** Mutual exclusion, hold and wait, no preemption, circular wait.
7. **Q:** Deadlock vs livelock vs starvation? **A:** Nobody progresses and all wait / everybody runs but nobody progresses / some progress while one never does.
8. **Q:** `CountDownLatch` vs `CyclicBarrier`? **A:** One-shot count of events vs reusable meeting of N threads.
9. **Q:** What is ABA and its fix? **A:** Value changes A→B→A so CAS can't notice; add a version (`AtomicStampedReference`).
10. **Q:** How to allocate a parking spot to exactly one of many gates without a lock? **A:** Keep free spots in a concurrent queue and `poll()` — an atomic remove.

12.28 Chapter TL;DR

- Races come from **shared mutable state**; fix by making critical sections atomic, or by removing sharing/mutability.
- Visibility needs **happens-before** edges: locks, `volatile`, start/join, concurrent utilities.
- Toolbox: `synchronized` → `ReentrantLock` (try/timed/fair/conditions) → RW locks → semaphores → latches/barriers → atomics/CAS → concurrent collections.
- Liveness: prevent deadlock with **lock ordering** or **tryLock + back-off**; add jitter against livelock; fairness against starvation.
- Patterns: signaling, thread pool, producer-consumer, reader-writer, immutability/confinement.

- In LLD interviews: identify the contended resource, keep "check + claim" atomic, lock fine-grained and in order, and say what changes when there are many servers.

Chapter 13 — Classic Concurrency Problems

> **Goal.** Chapter 12 gave you the toolbox. This chapter uses it on the concurrency problems interviewers actually ask — from LeetCode's (a popular coding-practice website) "Concurrency" section (FooBar, Zero-Even-Odd, FizzBuzz, H2O, Dining Philosophers) to "design a thread-safe X" **machine-coding** rounds (write working code for a small system in an hour or two): TTL cache, concurrent hash map, blocking queue, Bloom filter, parallel merge sort, **rate limiter** (a component capping how many requests a client may make per second).

Layout of each problem: *Problem → Approach → Java solution (+ real output) → Why it's correct → Pitfalls → Real-world examples → In the interview → TL;DR.*

All code is Java 21, standard library only, in `code/lld/ch13/<problem>/Main.java`. Listings below show the solution classes; package/imports, the `start(...)` thread helper and the `main` test harness are in the files. Run from `chapters/13-concurrency-problems/code`:

```
javac --release 21 -d /tmp/lld_13 $(find . -name '*.java')
java -cp /tmp/lld_13 lld.ch13.foobar.Main
```

Where the order of events varies between runs, the demo *verifies* the result and prints `true/false`, so the output shown is what you get every time.

> **Key terms (built in Chapter 12):** **thread-safe** = correct however threads interleave; a **race** = result depends on timing. A **lock / monitor** is a token one thread holds at a time (`synchronized` uses every object's built-in *monitor*; `ReentrantLock` is the explicit version). A **semaphore** counts permits (`acquire()` takes one or waits, `release()` returns one). A **condition variable** (`wait/notifyAll`, or `Condition`) lets a thread sleep, releasing its lock, until another changes the state and wakes it. A **barrier** (`CyclicBarrier`) holds N threads until all arrive. **Atomic / CAS**: an atomic step happens all at once or not at all; compare-and-swap updates a value only if it still equals what you read — the basis of **lock-free** code (no locks; a loser retries). **Happens-before**: the Java Memory Model rule guaranteeing one thread's writes are visible to another (same lock, `volatile`, semaphore release → acquire). **Deadlock / livelock / starvation**: waiting on each other forever / retrying with no progress / one thread never gets its turn.

Problem map — which tool solves what

#	Problem	Core idea	Main tools
13.1	Print FooBar Alternately	Pass a "baton" between 2 threads	`Semaphore` ×2 (or monitor + turn flag)
13.2	Print Zero Even Odd	A dispatcher thread decides who goes next	`Semaphore` ×3
13.3	Fizz Buzz Multithreaded	4 threads, one shared counter, each waits for "its" numbers	`synchronized` + `wait/notifyAll`
13.4	Building H2O	Group threads into sets of 2 H + 1 O	`Semaphore` (slots) + `CyclicBarrier` (group)
13.5	Dining Philosophers	Avoid deadlock over shared forks	Lock ordering, or a "waiter" semaphore
13.6	Thread-safe cache with TTL	Expiring entries, no stampede	`ConcurrentHashMap`, `compute`, scheduled sweeper
13.7	Concurrent HashMap (simplified)	Lock striping (N locks, each guarding a slice of the data)	`ReentrantLock[]`, all-stripe resize
13.8	Bounded Blocking Queue	Wait when full / empty	`wait/notifyAll` **and** `Lock` + 2 `Condition` s
13.9	Concurrent Bloom Filter	Lock-free bit set	`AtomicLongArray` + atomic OR
13.10	Multi-threaded Merge Sort	Divide and conquer on cores	`ForkJoinPool`, `RecursiveAction`
13.11	Rate limiter (token bucket)	Atomic refill-and-take	`synchronized`, or CAS on an immutable state

Reusable mental checklist for any concurrency problem

1. What is the **shared state**? (a turn flag, a counter, a map, a buffer)
2. What **ordering or invariant** must hold? ("foo before bar", "never more than 2 H in a molecule", "never more than capacity items")
3. Who **waits**, for **what condition**, and who **signals** it?
4. Pick the primitive: turn-taking → semaphores; "wait until predicate" → condition variable; groups → barrier; mutual exclusion → lock; single-variable update → atomic.
5. Check **liveness** (every thread eventually makes progress): can anyone wait forever? Does every thread exit at the end?

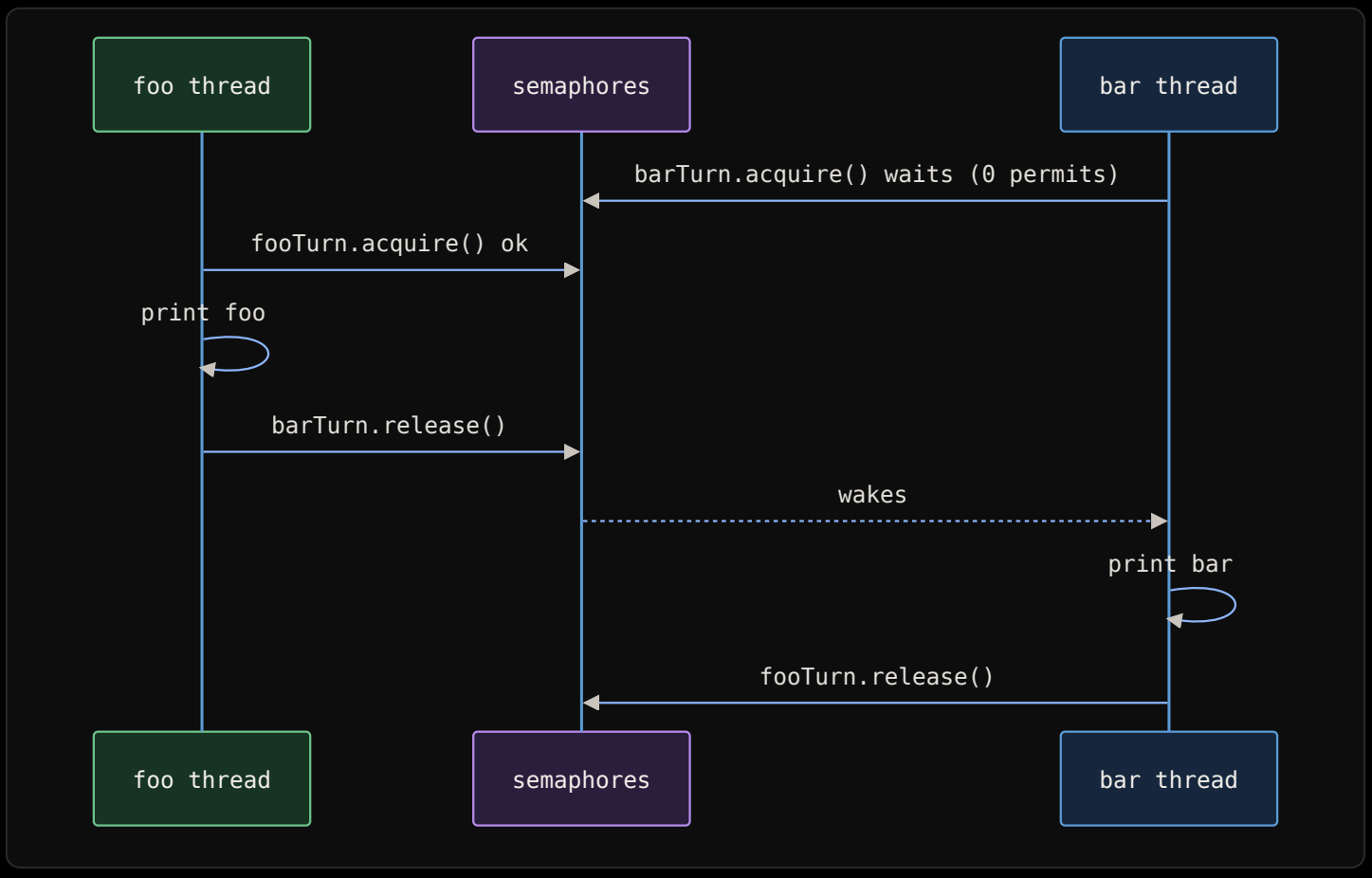
13.1 Print FooBar Alternately (LeetCode 1115)

Problem

One `FooBar` object is shared by two threads. Thread A calls `foo()`, printing `"foo"` n times; thread B calls `bar()`, printing `"bar"` n times. Output must be exactly `"foobar"` × n, whichever thread starts first.

Approach

Pure **signaling** (§12.20): a baton passes back and forth. `fooTurn = Semaphore(1)` (foo may go first), `barTurn = Semaphore(0)` (bar must wait). `foo`: acquire `fooTurn`, print, release `barTurn`; `bar`: the mirror image. Alternative: one monitor with a boolean `fooTurn`, `while (!fooTurn) wait();`, then flip it and `notifyAll()`.



Java solution

```java
// code/lld/ch13/foobar/Main.java
static class FooBarSemaphore {                          // Solution 1: semaphores as a baton
    private final int n;
    private final Semaphore fooTurn = new Semaphore(1);     // foo may go first
    private final Semaphore barTurn = new Semaphore(0);     // bar must wait
    FooBarSemaphore(int n) { this.n = n; }

    void foo(Runnable printFoo) throws InterruptedException {
        for (int i = 0; i < n; i++) {
            fooTurn.acquire();       // wait for my turn
            printFoo.run();
            barTurn.release();       // hand the baton to bar
        }
    }
    void bar(Runnable printBar) throws InterruptedException {
        for (int i = 0; i < n; i++) {
            barTurn.acquire();
            printBar.run();
            fooTurn.release();
        }
    }
}

static class FooBarMonitor {                            // Solution 2: monitor + turn flag
    private final int n;
    private boolean fooTurn = true;                     // guarded by 'this'
    FooBarMonitor(int n) { this.n = n; }

    void foo(Runnable printFoo) throws InterruptedException {
        for (int i = 0; i < n; i++) {
            synchronized (this) {
                while (!fooTurn) wait();
                printFoo.run();
                fooTurn = false;
                notifyAll();
            }
        }
    }
    // bar(): identical with `while (fooTurn) wait();` and `fooTurn = true;`
}
// main: n = 4, starts the bar thread FIRST on purpose, appends to a StringBuffer (synchronized)
```

Output:

```
semaphores : foobarfoobarfoobarfoobar
monitor    : foobarfoobarfoobarfoobar
both correct: true
```

Why it's correct

- **Invariant:** permits in `fooTurn` + `barTurn` always total exactly 1 (each thread releases the *other* semaphore only after acquiring its own). At most one thread prints, and turns strictly alternate.
- Starting `bar` first doesn't matter: it blocks on `barTurn` (0 permits) until `foo` has printed.
- **Visibility:** a `release()` happens-before the `acquire()` it enables, so each thread sees the other's appends.
- **Termination:** both loops run exactly *n* times, and each acquire has a matching release.

Pitfalls

- One semaphore for both threads — a thread can re-acquire its own release and print twice.
- `if` instead of `while` in the monitor version, or `notify()` in more complex variants.
- **Busy-waiting** (a loop re-checking a flag) on a `volatile` flag (writes immediately visible to other threads) works but burns a CPU core.

Real-world examples

Strict two-party hand-offs: ping-pong protocols, request/acknowledge stages, **double-buffering** (one buffer is filled while the other is read, then they swap) where a renderer and a producer alternate buffers.

In the interview

> **30-second answer:** "Two semaphores as a baton: foo's starts at 1, bar's at 0. Each thread acquires its own, prints, and releases the other's. The total permits are always one, so they alternate."

Follow-ups:

- *Three threads printing A, B, C in order?* Three semaphores in a ring: A(1), B(0), C(0); each releases the next.
- *Without semaphores?* A monitor with a turn variable and `wait/notifyAll`, or `Lock` + `Condition`.
- *What if `barTurn` starts with 1?* bar can print first → output may begin with "bar".

TL;DR

Two semaphores, one permit total, passed back and forth.

13.2 Print Zero Even Odd (LeetCode 1116)

Problem

Three threads share a `ZeroEvenOdd(n)` object. `zero()` may only print 0, `even()` only even numbers, `odd()` only odd numbers. Together they print `0 1 0 2 0 3 … 0 n` — e.g. *n* = 5 → `0102030405`.

Approach

The zero thread is a **dispatcher**: after printing the 0 for position *i*, it releases `oddTurn` if *i* is odd, else `evenTurn`; that thread prints its number and hands control back via `zeroTurn`. Initial permits: `zeroTurn = 1`, `oddTurn = 0`, `evenTurn = 0`. Zero loops *n* times; odd over 1, 3, 5…; even over 2, 4, 6….

Java solution

```java
// code/lld/ch13/zeroevenodd/Main.java
static class ZeroEvenOdd {
    private final int n;
    private final Semaphore zeroTurn = new Semaphore(1);
    private final Semaphore oddTurn = new Semaphore(0);
    private final Semaphore evenTurn = new Semaphore(0);
    ZeroEvenOdd(int n) { this.n = n; }

    void zero(IntConsumer print) throws InterruptedException {
        for (int i = 1; i <= n; i++) {
            zeroTurn.acquire();
            print.accept(0);
            if (i % 2 == 1) oddTurn.release(); else evenTurn.release();   // dispatcher decides
        }
    }
    void odd(IntConsumer print) throws InterruptedException {
        for (int i = 1; i <= n; i += 2) {
            oddTurn.acquire();
            print.accept(i);
            zeroTurn.release();
        }
    }
    void even(IntConsumer print) throws InterruptedException {
        for (int i = 2; i <= n; i += 2) {
            evenTurn.acquire();
            print.accept(i);
            zeroTurn.release();
        }
    }
}
// main: n in {1, 2, 5, 10}, threads started even, odd, zero
```

Output:

```
n=1 -> 01
n=2 -> 0102
n=5 -> 0102030405
n=10 -> 01020304050607080910
```

Why it's correct

- Exactly one semaphore holds a permit at any moment → exactly one thread may print.
- The zero thread's index *i* decides the parity of the next number, so the right thread wakes.
- Each loop count matches the permits that thread receives: zero *n*, odd ⌈n/2⌉, even ⌊n/2⌋. Nobody waits forever and everyone exits (for *n* = 1, even's loop runs zero times).

Pitfalls

- Odd/even threads computing "the next number" from a shared counter without synchronization.
- Loop bounds that don't match the permits (even looping `n/2 + 1` times) → a thread waits forever; the program hangs.

Real-world examples

Dispatcher/worker designs where one coordinator routes each step to a specialised worker (an interpreter routing **opcodes** — numeric codes of individual instructions — a scheduler assigning alternating tasks).

In the interview

> **30-second answer:** "Three semaphores. Zero starts with one permit. After each 0 it releases odd or even depending on the index, and they release zero back. One permit total in the system, so the order is forced."

Follow-ups:

- *One lock + condition instead?* A shared `state` (ZERO/ODD/EVEN) and `current` number; each thread waits `while (state != mine)`, acts, updates state, `signalAll()`.
- *For n = 4, how many times does `odd()` acquire?* Twice (1 and 3).

TL;DR

Zero is the dispatcher; one permit circulates among three semaphores.

13.3 Fizz Buzz Multithreaded (LeetCode 1195)

Problem

Four threads share a `FizzBuzz(n)` object and together output 1..n: multiples of 15 → `fizzbuzz` (thread C), multiples of 3 only → `fizz` (A), multiples of 5 only → `buzz` (B), everything else → the number (D).

Approach

One shared counter `current` guarded by the object's monitor; each thread runs the same loop with its own predicate — the **condition variable** pattern (§12.13):

1. `while (current <= n && !mine(current)) wait();`
2. If `current > n` → `notifyAll()` and exit.
3. Print, `current++`, `notifyAll()` (the next number may belong to anyone).

A helper `runWhile(predicate, action)` avoids writing the loop four times (DRY: Don't Repeat Yourself).

Java solution

```java
// code/lld/ch13/fizzbuzz/Main.java
static class FizzBuzz {
    private final int n;
    private int current = 1;                            // guarded by 'this'
    FizzBuzz(int n) { this.n = n; }

    /** Wait until 'current' is mine (or we're done), act, advance, wake everyone. */
    private void runWhile(IntPredicate mine, IntConsumer action) throws InterruptedException {
        while (true) {
            synchronized (this) {
                while (current <= n && !mine.test(current)) wait();
                if (current > n) { notifyAll(); return; }     // done: wake others so they exit too
                action.accept(current);
                current++;
                notifyAll();                                  // the next number may belong to anyone
            }
        }
    }
    void fizz(Runnable p)     throws InterruptedException { runWhile(i -> i % 3 == 0 && i % 5 != 0, i -> p.run()); }
    void buzz(Runnable p)     throws InterruptedException { runWhile(i -> i % 5 == 0 && i % 3 != 0, i -> p.run()); }
    void fizzbuzz(Runnable p) throws InterruptedException { runWhile(i -> i % 15 == 0, i -> p.run()); }
    void number(IntConsumer p) throws InterruptedException { runWhile(i -> i % 3 != 0 && i % 5 != 0, p); }
}
// main: n = 15; output list touched only inside the monitor, read after join()
```

Output:

```
1, 2, fizz, 4, buzz, fizz, 7, 8, fizz, buzz, 11, fizz, 13, 14, fizzbuzz
```

Why it's correct

- The predicates **partition** 1..n (each number satisfies exactly one), so exactly one thread acts per value.
- `current` is read and written only inside `synchronized (this)` → no races, full visibility; the output list too, and `join()` makes it visible to `main`.
- `notifyAll()`, not `notify()`: four different predicates wait on one monitor; `notify()` could wake a thread whose predicate is false, which waits again while the right thread is never woken → hang.
- **Termination:** once `current` exceeds *n*, the last `notifyAll` wakes every waiter, each sees `current > n`, and all four exit.

Pitfalls

- `notify()` instead of `notifyAll()` → lost wake-ups, random hangs.
- Missing the `current > n` exit check in the wait loop → threads wait forever after the last number.
- Overlapping predicates (fizz = `i % 3 == 0` without excluding 15) → two threads compete for 15.

Real-world examples

"Workers wait until shared state matches their role" — pipeline stages each waiting for a job to reach their stage in a shared state machine.

In the interview

> **30-second answer:** "One monitor, one shared counter. Each thread waits in a while loop until the counter matches its predicate or passes n, prints, increments and notifies all. The predicates partition the numbers, so exactly one thread acts per number."

Follow-ups:

- *Semaphore version?* The number thread dispatches, releasing the fizz/buzz/fizzbuzz semaphore for the next value — more efficient (no **thundering herd**: many threads woken when only one can proceed) but more code.
- *Why `notifyAll`?* Threads wait for *different* conditions on one monitor; only waking all guarantees the right one wakes.

TL;DR

Shared counter + per-thread predicate + `while`-wait + `notifyAll`.

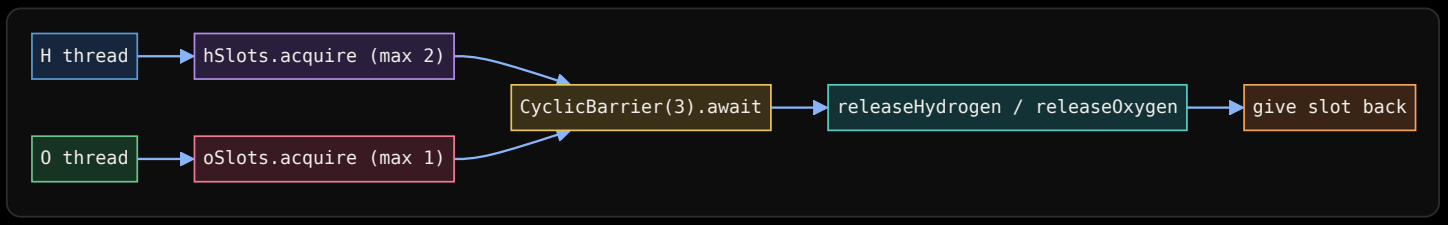
13.4 Building H2O (LeetCode 1117)

Problem

Hydrogen and oxygen threads arrive in any order and must pass a barrier in groups of three forming one water molecule: exactly **2 H + 1 O**. All threads of one molecule pass before any thread of the next.

Approach

- **Slots** — `hSlots = Semaphore(2)`, `oSlots = Semaphore(1)`: at most 2 H and 1 O are "in" the current molecule.
- **Group** — `CyclicBarrier(3)`: the three atoms wait for each other, then all go.
- A thread returns its slot **after** it has been released (printed), so the next molecule forms only once this one is finished.



Java solution

```java
// code/lld/ch13/h2o/Main.java
static class H2O {
    private final Semaphore hSlots = new Semaphore(2);         // at most 2 H in the current molecule
    private final Semaphore oSlots = new Semaphore(1);         // at most 1 O
    private final CyclicBarrier bond = new CyclicBarrier(3);   // wait until all 3 atoms are present

    void hydrogen(Runnable releaseHydrogen) throws InterruptedException {
        hSlots.acquire();
        try { bond.await(); } catch (BrokenBarrierException e) { throw new IllegalStateException(e); }
        releaseHydrogen.run();
        hSlots.release();                                      // free my slot only after I have "left"
    }
    void oxygen(Runnable releaseOxygen) throws InterruptedException {
        oSlots.acquire();
        try { bond.await(); } catch (BrokenBarrierException e) { throw new IllegalStateException(e); }
        releaseOxygen.run();
        oSlots.release();
    }
}
// main: 16 H + 8 O shuffled with Random(7); since HHO, HOH and OHH are all valid,
// it checks every consecutive group of 3 in the output instead of printing it.
```

Output:

```
arrival order   : OHOOOHHHHOHHHHHOOHHHHHOH
molecules formed: 8, every group of 3 has 2 H + 1 O: true
```

Why it's correct

- The slot semaphores let **at most 2 H and 1 O** reach the barrier per round; the barrier needs 3 parties, so each trip is exactly `H, H, O`.
- **No mixing:** a new molecule needs all three slots free, and each is freed only after its thread printed — so the next group can't trip the barrier (and print) until all three of the previous group have printed.
- The barrier resets automatically (it's *cyclic*) for the next molecule.

Pitfalls

- Releasing the slot **before** printing → a new H may print before the previous O, mixing molecules.
- Semaphores without the barrier → the 2 H and 1 O aren't forced to leave "together".
- Thread counts not exactly 2:1 → the last threads wait forever (the problem guarantees they are).

Real-world examples

Batching: gather exactly N items of certain types before dispatching (matchmaking 2 players + 1 server slot; assembling an order needing parts from different producers).

In the interview

> **30-second answer:** "Two semaphores cap the molecule at 2 H and 1 O; a CyclicBarrier of 3 makes the three wait for each other; each atom returns its slot only after it's released, so molecules never interleave."

Follow-ups:

- *Without a barrier?* A lock with counters of waiting H and O; when 2 H + 1 O are available, the thread completing the set signals exactly those. More code, same idea.
- *Why* `hSlots` *= 2, not 1?* A molecule needs two hydrogens inside the barrier at once.

TL;DR

Slots (semaphores) decide *who* can join; the barrier decides *when* they leave; free slots only after leaving.

13.5 Dining Philosophers (LeetCode 1226)

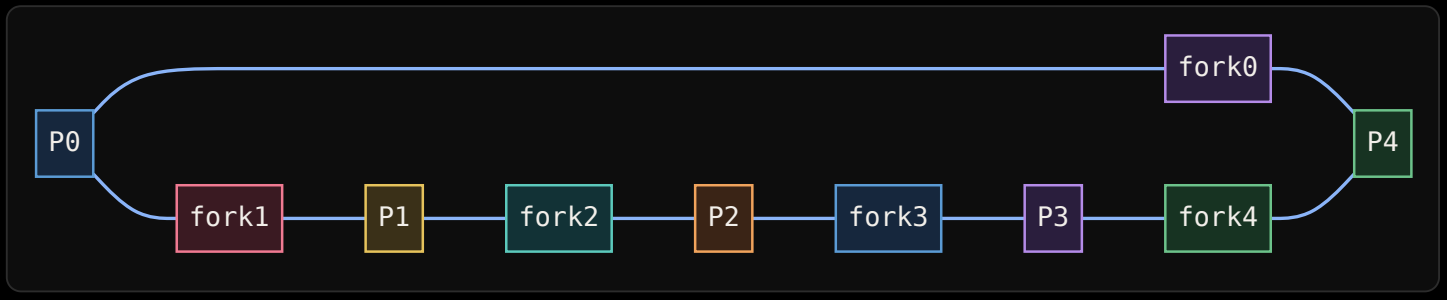
Problem

Five philosophers sit at a round table with one fork between each pair (5 forks). To eat, a philosopher needs **both** adjacent forks. If everyone picks up their left fork first, everyone may hold one fork and wait forever for the other — **deadlock** (all four **Coffman conditions** hold — mutual exclusion, hold-and-wait, no preemption, circular wait; §12.17).

Approach

Break a Coffman condition:

1. **Resource ordering** (breaks circular wait): number forks 0–4; always pick up the **lower-numbered** fork first. Philosopher 4 (forks 4 and 0) picks fork 0 first, so the cycle is impossible.
2. **Waiter / limited seats** (also breaks circular wait): a semaphore lets at most **n − 1** philosophers try at once — 4 people, 5 forks, so at least one gets both. Make it **fair** to prevent starvation.
3. Others: `tryLock` both forks and put down on failure with random **back-off** (a random short wait before retrying); the Chandy–Misra algorithm for distributed settings.



Java solution

```java
// code/lld/ch13/philosophers/Main.java
interface Table { void eat(int p, Runnable eatAction) throws InterruptedException; }

static class OrderedForks implements Table {              // Fix 1: lower-numbered fork first
    private final ReentrantLock[] forks;                  // one ReentrantLock per fork
    public void eat(int p, Runnable eatAction) {
        int left = p, right = (p + 1) % forks.length;
        ReentrantLock first = forks[Math.min(left, right)], second = forks[Math.max(left, right)];
        first.lock();
        try {
            second.lock();
            try { eatAction.run(); } finally { second.unlock(); }
        } finally { first.unlock(); }
    }
}

static class Waiter implements Table {                    // Fix 2: at most n-1 try at once
    private final ReentrantLock[] forks;
    private final Semaphore seats;                        // = new Semaphore(n - 1, true): fair, nobody starves for a
seat
    public void eat(int p, Runnable eatAction) throws InterruptedException {
        seats.acquire();
        try {
            ReentrantLock left = forks[p], right = forks[(p + 1) % forks.length];
            left.lock();
            try {
                right.lock();
                try { eatAction.run(); } finally { right.unlock(); }
            } finally { left.unlock(); }
        } finally { seats.release(); }
    }
}
// dine(table, 5, 10_000): 5 threads "think" (Thread.onSpinWait) then eat; counts meals in an AtomicIntegerArray
```

Output:

```
resource ordering, meals per philosopher: [10000, 10000, 10000, 10000, 10000]
waiter (4 seats),  meals per philosopher: [10000, 10000, 10000, 10000, 10000]
```

Why it's correct

- **Ordering:** every philosopher acquires forks in increasing index order. A waiting cycle would need someone holding a higher fork while waiting for a lower one — impossible.
- **Waiter:** with at most 4 competing for 5 forks, by the **pigeonhole principle** (more items than boxes → some box gets two) at least one gets both forks, eats and releases — progress is always possible. The fair semaphore hands out seats in FIFO (first in, first out) order.
- Forks are released in `finally`, so an exception can't leave one held. 5 × 10,000 meals complete.

Pitfalls

- "Everyone picks left then right" → deadlock.
- `tryLock` both forks and retry immediately without random back-off → livelock.
- Unfair strategies can starve a philosopher in theory. The fair semaphore makes *seat* access first-come-first-served; for strict no-starvation on the forks too, make the fork locks fair (`new ReentrantLock(true)`), at a throughput cost.

Real-world examples

Any task needing **two or more locks** at once: bank transfers (two accounts), moving an item between inventories, DB transactions updating several rows — solved by ordering (account id), or by the DB's deadlock detection aborting one transaction.

In the interview

> **30-second answer:** "Naively everyone takes the left fork and waits for the right — circular wait, deadlock. I break circular wait by always taking the lower-numbered fork first, or by a waiter semaphore allowing

> only n−1 philosophers to try; fairness prevents starvation."

Follow-ups:

- *Which Coffman condition does each fix break?* Both break circular wait; the tryLock variant breaks hold-and-wait.
- *Maximum parallelism?* With 5 forks, at most 2 philosophers eat at once.
- *With ordering, who behaves "differently"?* The one between forks 4 and 0 — they pick fork 0 (right) first.

> **TL;DR**
>
> Break circular wait: global fork order, or allow only n−1 diners.

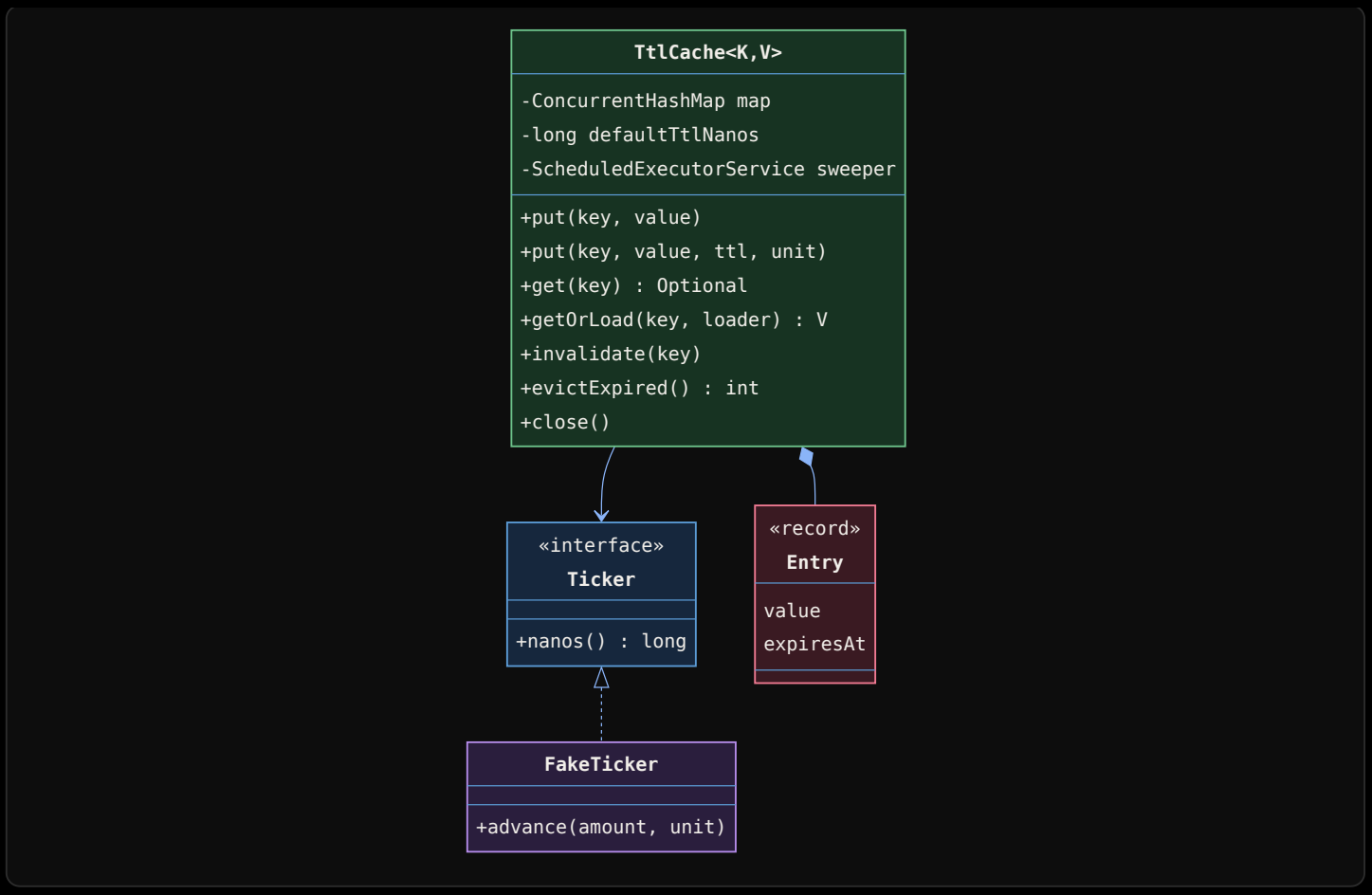
13.6 Thread-safe cache with TTL

Problem

Design an in-memory key-value **cache** (a fast store keeping copies of data for quick reads; see System Design Fundamentals) used by many threads, where each entry expires after a **time-to-live** (TTL). Requirements: `put(k, v)` (default or per-entry TTL); `get(k)` returns nothing for expired entries; `getOrLoad(k, loader)` loads on a miss without a **cache stampede** (many threads loading the same key at once); expired entries are eventually removed from memory; testable without sleeping.

Approach

- Immutable `Entry(value, expiresAt)` records in a `ConcurrentHashMap`.
- **Lazy expiry**: `get` checks `expiresAt`; if expired, removes it with the **conditional** `remove(key, entry)`, so it never deletes a fresher value another thread just put.
- **Active expiry**: a **daemon** (background thread that doesn't keep the JVM running) `ScheduledExecutorService` (a JDK pool running tasks on a timer) periodically calls `evictExpired()`, so keys nobody reads again don't leak memory.
- **No stampede**: `getOrLoad` uses `map.compute(...)`, atomic per key — concurrent callers for one key wait and the loader runs once.
- **Testability**: time comes from an injected `Ticker` (`FakeTicker` in tests) — **Dependency Inversion** (depend on an interface, so the real clock can be swapped for a fake).


TtlCache<K,V>
-ConcurrentHashMap map
-long defaultTtlNanos
-ScheduledExecutorService sweeper
+put(key, value)
+put(key, value, ttl, unit)
+get(key) : Optional
+getOrLoad(key, loader) : V
+invalidate(key)
+evictExpired() : int
+close()
«interface»
Ticker
+nanos() : long
«record»
Entry
value
expiresAt
FakeTicker
+advance(amount, unit)

Java solution

```java
// code/lld/ch13/ttlcache/Main.java
@FunctionalInterface interface Ticker { long nanos(); }      // FakeTicker: an AtomicLong + advance(amount, unit)

static final class TtlCache<K, V> implements AutoCloseable {
    private record Entry<V>(V value, long expiresAt) { }
    private final ConcurrentHashMap<K, Entry<V>> map = new ConcurrentHashMap<>();
    private final long defaultTtlNanos;
    private final Ticker ticker;
    private final ScheduledExecutorService sweeper;        // null = lazy expiry only

    TtlCache(long ttl, TimeUnit unit, Ticker ticker, long sweepEveryMillis) {
        this.defaultTtlNanos = unit.toNanos(ttl);
        this.ticker = ticker;
        if (sweepEveryMillis > 0) {
            sweeper = Executors.newSingleThreadScheduledExecutor(r -> {
                Thread t = new Thread(r, "ttl-sweeper");
                t.setDaemon(true);                          // never blocks JVM shutdown
                return t;
            });
            // 'this' escapes before the constructor ends - see "Why it's correct".
            sweeper.scheduleAtFixedRate(this::evictExpired, sweepEveryMillis, sweepEveryMillis, TimeUnit.MILLISECONDS);
        } else sweeper = null;
    }

    private boolean expired(Entry<V> e) { return ticker.nanos() - e.expiresAt() >= 0; }   // overflow-safe

    void put(K key, V value, long ttl, TimeUnit unit) {    // put(key, value) uses the default TTL
        Objects.requireNonNull(key); Objects.requireNonNull(value);
        map.put(key, new Entry<>(value, ticker.nanos() + unit.toNanos(ttl)));
    }

    Optional<V> get(K key) {
        Entry<V> e = map.get(key);
        if (e == null) return Optional.empty();
        if (expired(e)) {
            map.remove(key, e);   // conditional: never deletes a FRESH value another thread just put
            return Optional.empty();
        }
        return Optional.of(e.value());
    }

    /** Read-through: compute() locks the key's bin, so concurrent misses for one key load once.
     *  The loader must be fast and must not touch this cache. */
    V getOrLoad(K key, Function<? super K, ? extends V> loader) {
        return map.compute(key, (k, old) ->
                old != null && !expired(old) ? old : new Entry<>(loader.apply(k), ticker.nanos() + defaultTtlNanos)
        ).value();
    }

    void invalidate(K key) { map.remove(key); }

    int evictExpired() {                                   // active expiry; safe alongside readers/writers
        int removed = 0;
        for (Map.Entry<K, Entry<V>> me : map.entrySet())
            if (expired(me.getValue()) && map.remove(me.getKey(), me.getValue())) removed++;
        return removed;
    }
    int rawSize() { return map.size(); }                   // includes not-yet-swept expired entries
    @Override public void close() { if (sweeper != null) sweeper.shutdownNow(); }
}
// main: fake clock (100 ms default TTL, 30 ms OTP); 16 threads released by a latch call getOrLoad("user:7");
// then a real-clock cache (30 ms TTL, sweep every 10 ms) with 1,000 puts and a 200 ms sleep.
```

Output:

```
t=50ms  session=alice, otp=<expired>
t=110ms session=<expired>
before sweep raw size=5, swept=5, after=0
16 concurrent misses -> loader ran 1 time(s), all got same value: true
real clock: size right after puts=1000
real clock: size after 200 ms (sweeper ran)=0
```

Why it's correct

- All shared state is in one `ConcurrentHashMap` of immutable records: a reader sees the old or the new entry, never a half-written one.
- `remove(key, e)` removes only if the mapping is still *that exact entry*; a fresh replacement survives.
- `compute` runs the remapping function atomically per key, so 16 simultaneous misses cause one load.
- Expiry uses `now - expiresAt >= 0`, not `now >= expiresAt`, which stays correct even if `System.nanoTime()` values **overflow** (wrap around past the largest `long`) — the JDK recommends comparing nano times by subtraction.
- The sweeper is a daemon and is shut down in `close()`.
- The constructor hands `this::evictExpired` to the sweeper before returning (`this` escapes) — normally a bug (§12.6). It's safe here only because the fields it uses (`map`, `ticker`) are already set and submitting to an executor is a happens-before edge; in production prefer a **static factory method** (a static method that builds and returns the object) that starts the sweeper after construction.

Pitfalls

- `System.currentTimeMillis()` for TTL: wall-clock time can jump (NTP — Network Time Protocol — corrections). `System.nanoTime()` is **monotonic** (only moves forward) for durations.
- A slow loader inside `compute` blocks other keys sharing the hash bin, and a loader touching the same map can throw or deadlock. For slow loads store a `CompletableFuture<V>` (a placeholder an async task fills later) instead — `computeIfAbsent(k, key -> supplyAsync(...))` — so the bin lock is held only briefly.
- Only lazy expiry → memory leak for keys never read again. Only active expiry → `get` may return a stale value between sweeps. Use both.
- No size limit: a real cache needs a maximum size and an **eviction policy** (which entry to throw out when full: LRU = least recently used, LFU = least frequently used).

Real-world examples

- **Caffeine** (the de-facto Java caching library): `expireAfterWrite` / `expireAfterAccess`, size-based eviction, and a stampede-free loading cache — the production version of this design. Guava's (Google's core Java library) `CacheBuilder` is its predecessor.
- **Redis** `EXPIRE` / `SET … EX`: lazy expiry on access plus periodic active sampling of keys with TTLs.
- Sessions, OTPs (one-time passwords), DNS caches (DNS maps names to IP addresses), and seat holds in booking systems (a hold is a TTL entry).

In the interview

> **30-second answer:** "A ConcurrentHashMap of immutable entries holding value and expiry time. Get checks expiry lazily and removes with a conditional remove; a daemon scheduled sweeper removes expired keys nobody reads. getOrLoad uses compute so concurrent misses load once. Time is injected for testing."

Follow-ups (with answers):

- *Max size with LRU?* An access-ordered `LinkedHashMap` under a lock, or a `ConcurrentHashMap` plus a concurrent **deque** (double-ended queue) of keys updated on access — or just Caffeine (Window-TinyLFU).
- *Refresh without blocking readers?* "Refresh-ahead": serve the old value while one thread reloads asynchronously as the entry nears expiry.
- *Distributed?* Redis/Memcached (shared in-memory cache servers) with TTLs, plus a small local cache in front for hot keys.
- *Why `remove(key, e)` not `remove(key)`?* To avoid deleting a value written between our read and our remove.

TL;DR

CHM (`ConcurrentHashMap`) + immutable (value, expiry) entries; lazy + scheduled expiry; `compute` for **single-flight** loads (one load per key, shared by every waiting caller); injected clock.

13.7 Design a (simplified) Concurrent HashMap — lock striping

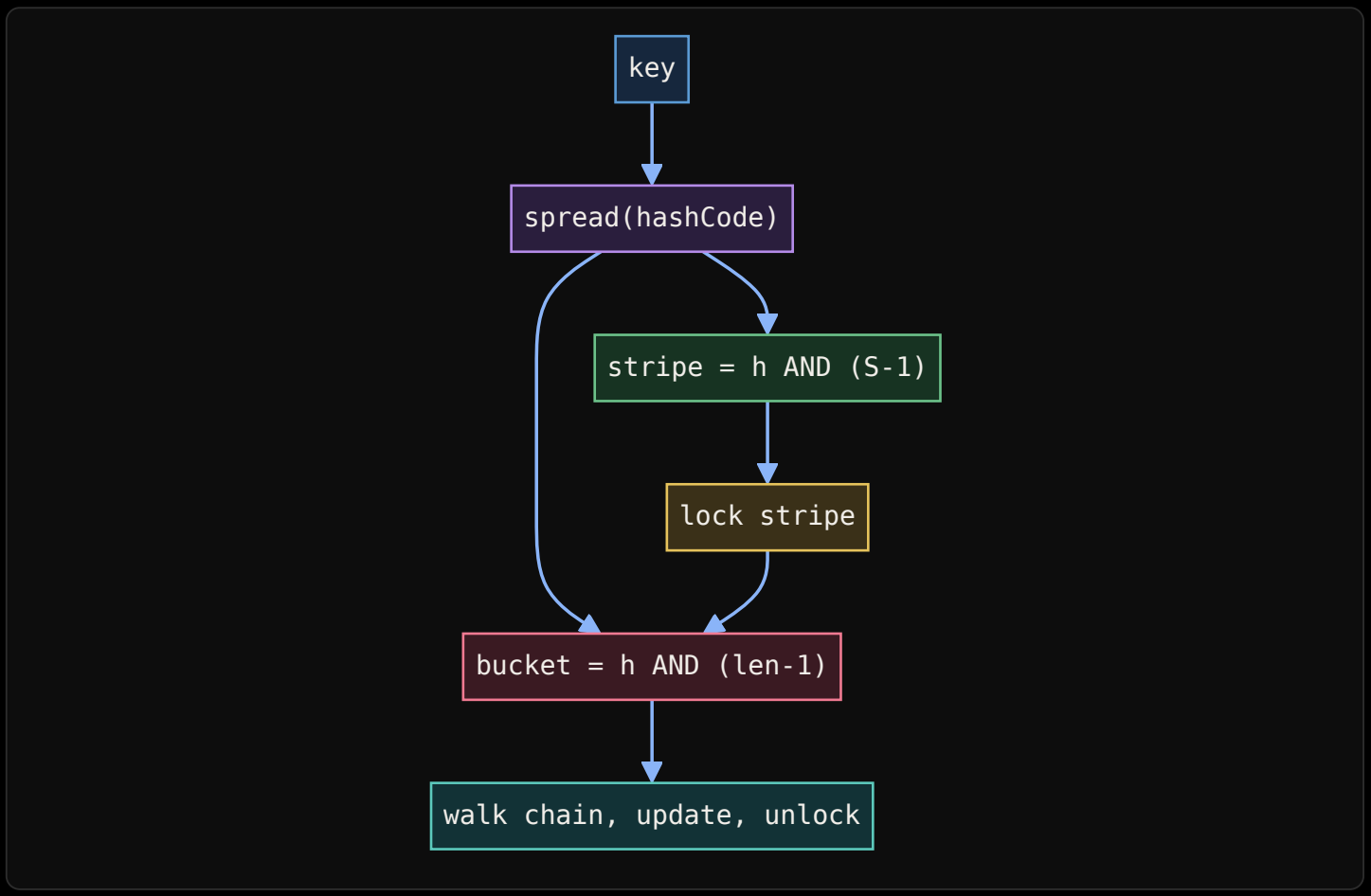
Problem

A hash map many threads can use at once — `get`, `put`, `remove`, `compute`, automatic resizing — faster than one global lock around a `HashMap`.

Approach

Lock striping (§12.11):

- Buckets: an array of linked lists (**separate chaining**: each bucket lists all entries whose hash lands there).
- Stripes: a fixed array of `S` locks (power of two). Hash *h* uses stripe `h & (S − 1)` and bucket `h & (tableLength − 1)`. The stripe depends only on the hash — **not** the table length — so a key's stripe never changes on resize.
- Each operation locks only its stripe; different stripes run in parallel.
- **Resize** (size > 0.75 × buckets): acquire **all** stripe locks in index order (global order → no deadlock), re-check, **rehash** (recompute every key's bucket) into a table twice as big, publish via a `volatile` field, release in reverse.
- Size tracked with an `AtomicInteger`.



Java solution

```
// code/lld/ch13/stripedmap/Main.java  (Java 7's ConcurrentHashMap used this idea with "segments")
static final class StripedHashMap<K, V> {
    private static final class Node<K, V> { final K key; final int hash; V value; Node<K, V> next; /* ctor */ }

    private volatile Node<K, V>[] table;        // replaced (never mutated in place) only while ALL locks are held
    private final ReentrantLock[] stripes;
    private final AtomicInteger size = new AtomicInteger();
    private static final float LOAD_FACTOR = 0.75f;
    // constructor: both sizes powers of two, and stripeCount <= initialBuckets
    //              (guarantees every bucket maps to exactly one stripe), else IllegalArgumentException

    private static int spread(Object key) {
        int h = key.hashCode();
        return (h ^ (h >>> 16)) & 0x7fffffff;    // mix high bits in, keep it non-negative
    }
    private ReentrantLock stripeFor(int hash) { return stripes[hash & (stripes.length - 1)]; }  // never changes on resize
    private static int indexFor(int hash, int len) { return hash & (len - 1); }

    V get(K key) {
        int h = spread(Objects.requireNonNull(key));
        ReentrantLock lock = stripeFor(h);
        lock.lock();
        try {
            Node<K, V>[] t = table;                      // stable while we hold a stripe lock
            for (Node<K, V> n = t[indexFor(h, t.length)]; n != null; n = n.next)
                if (n.hash == h && n.key.equals(key)) return n.value;
            return null;
        } finally { lock.unlock(); }
    }
    V put(K key, V value) { Objects.requireNonNull(value); return doCompute(key, (k, old) -> value).oldValue(); }
    V compute(K key, BiFunction<? super K, ? super V, ? extends V> remap) { return doCompute(key, remap).newValue(); }
    // remove(key): same locking as get(); unlinks the node and decrements size

    private record Result<V>(V oldValue, V newValue) { }
    private Result<V> doCompute(K key, BiFunction<? super K, ? super V, ? extends V> remap) {
        int h = spread(Objects.requireNonNull(key));
        ReentrantLock lock = stripeFor(h);
        boolean grew = false;
        V old, nv;
        lock.lock();
        try {
            Node<K, V>[] t = table;
            int i = indexFor(h, t.length);
            Node<K, V> prev = null, n = t[i];
            while (n != null && !(n.hash == h && n.key.equals(key))) { prev = n; n = n.next; }
            old = n == null ? null : n.value;
            nv = remap.apply(key, old);
            if (n != null && nv != null) n.value = nv;                          // update in place
            else if (n != null) {                                               // remap returned null: remove
                if (prev == null) t[i] = n.next; else prev.next = n.next;
                size.decrementAndGet();
            } else if (nv != null) {                                            // insert at bucket head
                t[i] = new Node<>(key, h, nv, t[i]);
                grew = size.incrementAndGet() > t.length * LOAD_FACTOR;
            }
        } finally { lock.unlock(); }
        if (grew) resize();                                                     // never resize holding one stripe
        return new Result<>(old, nv);
    }

    private void resize() {                      // stop-the-world: every stripe lock IN ORDER, rehash, publish
        for (ReentrantLock l : stripes) l.lock();
        try {
            Node<K, V>[] old = table;
            if (size.get() <= old.length * LOAD_FACTOR) return;               // another thread already resized
            Node<K, V>[] nt = (Node<K, V>[]) new Node[old.length * 2];
            for (Node<K, V> head : old)
                for (Node<K, V> n = head; n != null; ) {
                    Node<K, V> next = n.next;
                    int j = indexFor(n.hash, nt.length);
                    n.next = nt[j]; nt[j] = n;
                    n = next;
                }
```

```
            table = nt;
        } finally {
            for (int i = stripes.length - 1; i >= 0; i--) stripes[i].unlock();
        }
    }
}
// main: basic ops; 8 threads x 20,000 puts from 16 buckets; 8 threads x 10,000 compute() on 10 hot keys; concurrent removes
```

Output:

```
put a=1, put a=2 returns old=1, get a=2, remove a=2, get a=null
8 threads x 20,000 puts -> size=160000, missing=0, table grew from 16 to 262144 buckets
atomic compute on 10 hot keys: each=8000, total=80000 (expected 80000)
after concurrent removes size=0
```

Why it's correct

- **Every bucket belongs to exactly one stripe.** *S* and the table length are powers of two with *S* ≤ length, so the stripe index (`h & (S-1)`) is the low bits of the bucket index (`h & (len-1)`). All keys in a bucket share one stripe lock; two threads never modify one chain at once.
- **Resize safety:** resize holds *every* stripe lock, so nothing else is in progress; it re-checks the load (another thread may have resized) and publishes via a `volatile` write. A later operation acquires a stripe lock the resizer released → happens-before → it sees the new table and moved nodes.
- **No deadlock:** normal operations hold one lock; resize takes all locks in ascending order, only *after* the caller released its own stripe.
- **Atomic compute:** read-modify-write of one key happens entirely under its stripe lock (80,000 exact increments).

Pitfalls

- Deriving the stripe from the *current table's bucket index* — after a resize a key could move to another lock while a thread holds the old one.
- Resizing while holding one stripe and then taking all → deadlock with another resizer. Release first.
- Reading `table` without a lock or `volatile` → a stale table after resize.
- Allowing `null` keys/values makes `get() == null` ambiguous (the JDK's `ConcurrentHashMap` forbids them too).

How the real `ConcurrentHashMap` goes further

Aspect	This design	JDK 8+ `ConcurrentHashMap`
Reads	Lock the stripe	Lock-free (volatile reads of nodes)
Write locking	One of S stripe locks	CAS into empty bins; `synchronized` on the bin's first node otherwise
Long chains	Linked list	Converted to a red-black tree beyond 8 nodes (when table ≥ 64)
Resize	Stop-the-world (all locks; everything else pauses)	Incremental: threads help transfer bins concurrently
Size	`AtomicInteger`	`LongAdder`-style counter cells

Real-world examples

Java 7's `ConcurrentHashMap` used **segments** (16 by default) — the same striping idea. Guava's `Striped<Lock>` offers striped locks for your own data; Caffeine uses striped buffers to record reads.

In the interview

> **30-second answer:** "An array of buckets plus a small array of locks; each key maps to a stripe by its hash bits, so different stripes work in parallel. Every bucket belongs to exactly one stripe. Resize takes all stripe locks in order, rehashes and publishes the new table through a volatile field."

Follow-ups:

- *Lock-free `get`?* Make `Node.value` and `next` volatile, publish nodes safely, and never mutate a chain so a reader could see it half-done (the JDK copies/reorders carefully during transfer).
- *How many stripes?* About the number of concurrently writing threads (JDK 7 default 16); more = less contention, more memory.
- *Why must the stripe count divide the table length?* So every bucket maps to exactly one stripe (stripe index = low bits of bucket index).

> **TL;DR**
>
> N locks for M buckets, stripe by hash bits, resize under all locks in order, publish via volatile.

13.8 Thread-safe bounded Blocking Queue (two ways)

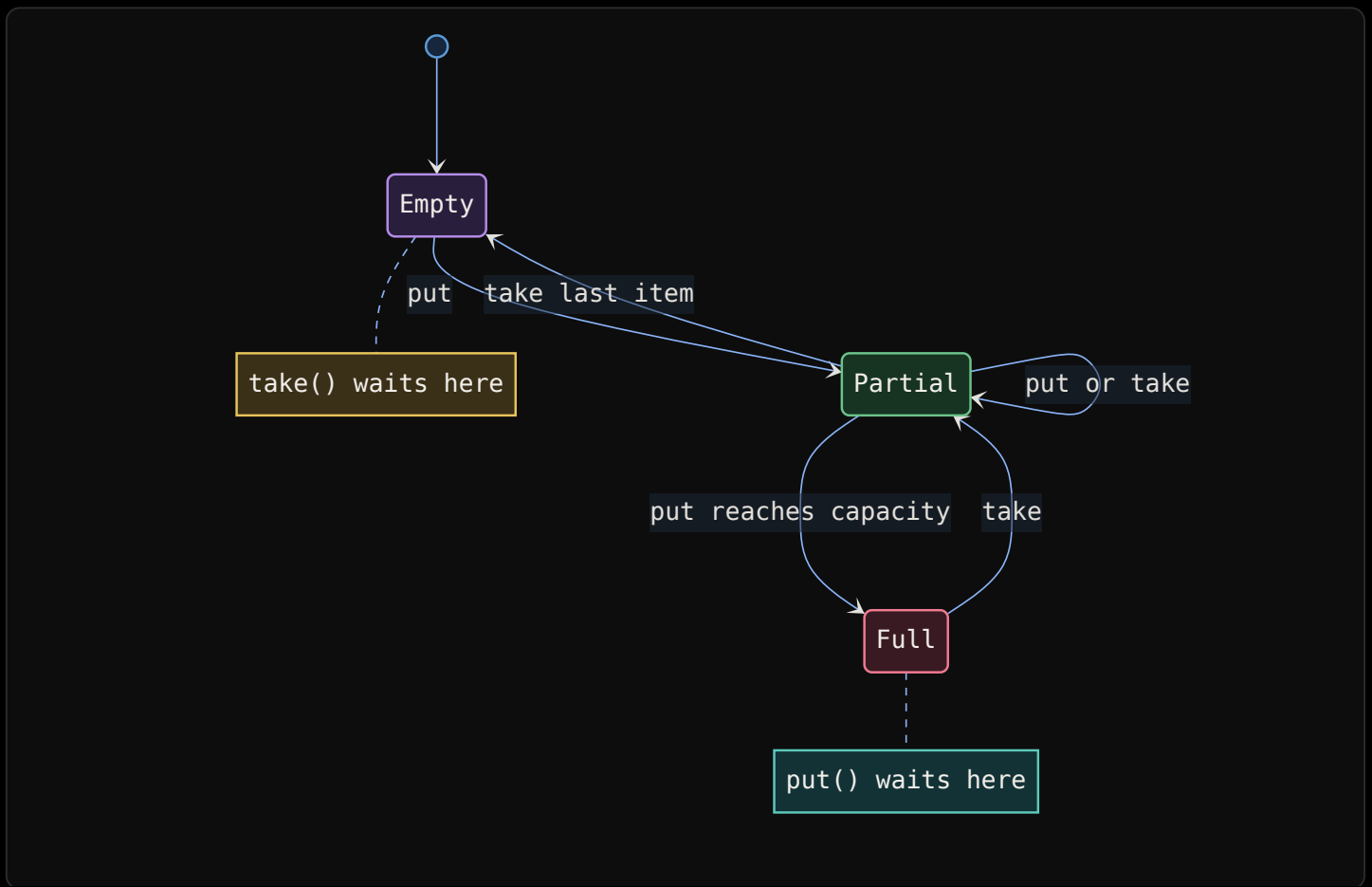
Problem

A fixed-capacity FIFO queue for producer-consumer use: `put` blocks while full, `take` blocks while empty, plus `offer(item, timeout)` and `poll(timeout)` that give up after a timeout.

Approach

Storage: a **circular array** (its end wraps around to the start; `head`, `tail`, `count`). Two synchronisation designs:

	Way 1: monitor	**Way 2: `ReentrantLock` + 2 `Condition`s**
Lock	`synchronized` methods	`lock.lockInterruptibly()`
Wait	`while (full) wait();`	`while (full) notFull.await();`
Wake	`notifyAll()` — producers and consumers share one wait set	`notEmpty.signal()` after enqueue, `notFull.signal()` after dequeue
Timed wait	`TimeUnit.timedWait(this, nanos)` + deadline loop	`awaitNanos(nanos)` returns the time left
Efficiency	Wakes everyone (thundering herd)	Wakes exactly one thread of the right kind



Java solution

```java
// code/lld/ch13/blockingqueue/Main.java - both implement put/take/offer(timeout)/poll(timeout)/size
// Way 1: one monitor shared by producers AND consumers -> must use notifyAll()
static final class MonitorQueue<T> implements BoundedBlockingQueue<T> {
    private final Object[] items;
    private int head, tail, count;                          // all guarded by 'this'

    public synchronized void put(T item) throws InterruptedException {
        while (count == items.length) wait();
        enqueue(item);
        notifyAll();
    }
    public synchronized T take() throws InterruptedException {
        while (count == 0) wait();
        T x = dequeue();
        notifyAll();
        return x;
    }
    public synchronized T poll(long timeout, TimeUnit unit) throws InterruptedException {
        long deadline = System.nanoTime() + unit.toNanos(timeout);
        while (count == 0) {
            long left = deadline - System.nanoTime();
            if (left <= 0) return null;
            TimeUnit.NANOSECONDS.timedWait(this, left);      // wait(millis, nanos) under the hood
        }
        T x = dequeue();
        notifyAll();
        return x;
    }
    // offer(item, timeout): mirror image of poll - waits while full, returns false on timeout
    private void enqueue(T item) {
        if (item == null) throw new NullPointerException();  // null is our "timed out" signal
        items[tail] = item;
        tail = (tail + 1) % items.length;                    // circular buffer
        count++;
    }
    private T dequeue() {
        T x = (T) items[head];
        items[head] = null;                                  // let GC reclaim it
        head = (head + 1) % items.length;
        count--;
        return x;
    }
}

// Way 2: two condition queues -> a targeted signal() is enough (how ArrayBlockingQueue is built)
static final class ConditionQueue<T> implements BoundedBlockingQueue<T> {
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();

    public void put(T item) throws InterruptedException {
        if (item == null) throw new NullPointerException();
        lock.lockInterruptibly();
        try {
            while (count == items.length) notFull.await();
            enqueue(item);                                   // enqueue() ends with notEmpty.signal(): wake ONE consumer
        } finally { lock.unlock(); }
    }
    public boolean offer(T item, long timeout, TimeUnit unit) throws InterruptedException {
        if (item == null) throw new NullPointerException();
        long nanos = unit.toNanos(timeout);
        lock.lockInterruptibly();
        try {
            while (count == items.length) {
                if (nanos <= 0) return false;
                nanos = notFull.awaitNanos(nanos);           // returns the time still left
            }
            enqueue(item);
            return true;
        } finally { lock.unlock(); }
    }
    // take()/poll(timeout): mirror images waiting on notEmpty; dequeue() ends with notFull.signal()
```

```
}
// stressTest: capacity 2, 3 producers x 20,000 items, 3 consumers; then poll/offer with 30 ms timeouts
```

Output:

```
wait/notifyAll  : consumed 60000 items, sum ok=true, left in queue=0
wait/notifyAll  : poll(30ms) on empty -> null
wait/notifyAll  : offer(30ms) on full -> false
Lock+Conditions : consumed 60000 items, sum ok=true, left in queue=0
Lock+Conditions : poll(30ms) on empty -> null
Lock+Conditions : offer(30ms) on full -> false
```

Why it's correct

- All fields are accessed only under the single lock → no races, plus visibility.
- **Waiting in `while` loops** handles spurious wake-ups and "someone else took the item first".
- **Way 1** must use `notifyAll()`: with one wait set, `notify()` might wake a producer when a consumer was needed (or vice versa), and the right thread might never wake → a deadlock-like hang.
- **Way 2** can `signal()`: producers wait only on `notFull`, consumers only on `notEmpty`; each enqueue enables exactly one more consumer, each dequeue exactly one producer.
- Timed versions track the *remaining* time, so repeated spurious wakeups can't extend the total wait.
- Stress test at capacity 2 (maximum blocking): all 60,000 items consumed, sum correct, queue empty.

Pitfalls

- `if` instead of `while`; `notify()` with one monitor shared by producers and consumers.
- Forgetting `items[head] = null` after dequeue → the array keeps references alive (a leak for large objects).
- Accepting `null` items when `null` means "timed out" in `poll`.
- `lock()` instead of `lockInterruptibly()` makes cancellation harder (the JDK's queues use `lockInterruptibly` in blocking methods).

Real-world examples

`java.util.concurrent.ArrayBlockingQueue` is built exactly like Way 2 (one lock, `notEmpty` / `notFull`). `LinkedBlockingQueue` uses **two** locks (`putLock`, `takeLock`) so a producer and a consumer work at the same time. Thread pools, logging frameworks and Kafka's producer internals all rely on blocking hand-off.

In the interview

> **30-second answer:** "A circular array with head, tail and count guarded by one lock. put waits while count == capacity, take waits while count == 0, both in while loops. With synchronized I must use notifyAll because both sides share one wait set; with ReentrantLock I use two Conditions, notFull and notEmpty, and signal just the right side."

Follow-ups:

- *Higher throughput?* A two-lock linked queue (separate put/take locks, atomic count) like `LinkedBlockingQueue`, or a lock-free ring buffer (LMAX Disruptor, a high-performance Java messaging library) for single producer/consumer.
- *Unbounded?* Drop the "full" condition — but then there's no back-pressure.
- *Why is `signal()` safe in Way 2 but `notify()` unsafe in Way 1?* Separate condition queues guarantee the woken thread is of the right kind.

TL;DR

Circular buffer + one lock + "wait while full/empty" in loops. Monitor → `notifyAll`; Lock → `notFull` / `notEmpty` + `signal`.

13.9 Concurrent Bloom Filter

Problem

A thread-safe **Bloom filter** — a compact probabilistic set answering *"definitely not present"* or *"probably present"* — that many threads add to and query at once.

Approach

How a Bloom filter works:

1. An array of m bits, all 0.
2. **Add**: compute k hash positions and set those bits to 1.
3. **Query**: the same k positions — if **any** bit is 0 the item was **definitely never added**; if all are 1 it was **probably** added (other items may have set them → a *false positive*).
4. No false negatives, no deletion (clearing a bit could erase other items; a *counting* Bloom filter with small counters supports deletes).

Sizing for n expected items and target false-positive rate p: bits $m = \lceil -n \cdot \ln p / (\ln 2)^2 \rceil$; hashes k = round((m / n) · ln 2). For n = 80,000, p = 1% → $m \approx$ 766,805 bits (≈ 93.6 KiB — kibibytes of 1,024 bytes; the demo prints 93 because it uses integer division) and k = 7 — about 9.6 bits per item, whatever the item size.

Double hashing (Kirsch–Mitzenmacher): compute $h1$, $h2$ once and use positions $h1 + i \cdot h2$ for i = 0..k−1 — as good as k independent hashes in practice.

Thread safety without locks: bits live in an `AtomicLongArray` (64 bits per word); setting a bit is an **atomic OR**, `getAndAccumulate(word, mask, (a, b) -> a | b)` (a CAS loop inside), so two threads setting different bits of one word both succeed. Queries are plain volatile reads: bits only go 0 → 1, so a concurrent query can at worst miss an item whose `add` hasn't finished (it hadn't happened yet); after `add` returns, every later query sees its bits (happens-before of atomic operations).

Java solution

```java
// code/lld/ch13/bloomfilter/Main.java
static final class ConcurrentBloomFilter {
    private final AtomicLongArray words;     // m bits packed into 64-bit words
    private final int numBits, numHashes;    // m, k

    ConcurrentBloomFilter(int expectedItems, double falsePositiveRate) {
        double ln2 = Math.log(2);
        long m = (long) Math.ceil(-expectedItems * Math.log(falsePositiveRate) / (ln2 * ln2));
        if (m > Integer.MAX_VALUE - 63) throw new IllegalArgumentException("too large");
        this.numBits = (int) m;
        this.numHashes = Math.max(1, (int) Math.round((double) m / expectedItems * ln2));
        this.words = new AtomicLongArray((numBits + 63) / 64);
    }

    /** Returns true if at least one bit changed (i.e. it was definitely new). */
    boolean add(String item) {
        long base = fnv1a64(item);
        long h1 = fmix64(base), h2 = fmix64(base ^ 0x9E3779B97F4A7C15L) | 1;   // h2 odd
        boolean changed = false;
        for (int i = 0; i < numHashes; i++) {
            int bit = (int) Math.floorMod(h1 + i * h2, (long) numBits);        // double hashing
            long mask = 1L << (bit & 63);
            long before = words.getAndAccumulate(bit >>> 6, mask, (cur, m) -> cur | m);  // atomic OR
            if ((before & mask) == 0) changed = true;
        }
        return changed;
    }

    boolean mightContain(String item) {                    // same h1/h2 as add()
        for (int i = 0; i < numHashes; i++) {
            int bit = (int) Math.floorMod(h1 + i * h2, (long) numBits);
            if ((words.get(bit >>> 6) & (1L << (bit & 63))) == 0) return false;   // definitely absent
        }
        return true;
    }
    // fnv1a64: 64-bit FNV-1a over the chars; fmix64: MurmurHash3's finalizer (a strong bit-mixer)
}
// main: 8 threads x 10,000 adds; check all for false negatives; probe 100,000 unseen keys
```

Output:

```
sized for n=80000, p=1%: m=766805 bits (93 KiB), k=7 hashes
false negatives after 8-thread insert: 0
false-positive rate on 100,000 unseen keys: 1.01% (target 1%)
add("user-0-0") again changes bits? false
```

Why it's correct

- **No lost updates:** a plain `words[i] |= mask` on a `long[]` is read-modify-write, so two threads could overwrite each other's bits → false negatives. The atomic OR retries on conflict.
- **Monotonic state:** bits never return to 0 — no ABA, no locks.
- **Deterministic result:** OR is commutative, so the final bit array is the same whatever the interleaving — the measured false-positive rate (≈1%, matching the formula's target) is identical every run.

Pitfalls

- Non-atomic bit setting → false negatives (the one thing a Bloom filter must never produce).
- Poor hashes (`String.hashCode()` with `i * 31` offsets) → clustered bits and much higher false-positive rates. Mix bits (FNV-1a + MurmurHash3's finalizer — two fast, non-cryptographic hash functions).
- Adding far more than *n* items → the false-positive rate climbs fast; resize by creating a larger filter (or a "scalable" chain of filters).
- `Math.floorMod` on `long` to avoid negative indices.

Real-world examples

- **Apache Cassandra, HBase, RocksDB, LevelDB**: a Bloom filter per **SSTable** (Sorted String Table — an immutable sorted data file on disk)/file to skip disk reads for absent keys.
- **Google Guava** `BloomFilter` (`put`/`mightContain` thread-safe since Guava 23 using atomic long arrays); **Redis** (RedisBloom module / Redis Stack) `BF.ADD`/`BF.EXISTS`.
- Is a username taken, or a URL already crawled, before hitting the database; Medium has described using Bloom filters to avoid recommending already-read articles.

In the interview

> **30-second answer:** "A bit array of size m with k hash positions per item from double hashing; add sets k bits, query checks them — any zero means definitely absent. For concurrency I store bits in an AtomicLongArray and set them with an atomic OR; since bits only go from 0 to 1, reads need no lock. m and k come from n and the target false-positive rate."

Follow-ups:

- *Deletions?* Counting Bloom filter (small counters, atomic increments/decrements) or a Cuckoo filter (an alternative that supports deletion).
- *Distributed?* Keep it in Redis (RedisBloom) or rebuild per node from a shared source.
- *Can it say "absent" for an added item?* No — no false negatives (assuming atomic, completed adds). *Bits per element for 1% FPR (false-positive rate)?* About 9.6.

TL;DR

Bits + k hashes; "no" is certain, "yes" is probable. `AtomicLongArray` + atomic OR = lock-free and correct.

13.10 Multi-threaded Merge Sort (Fork/Join)

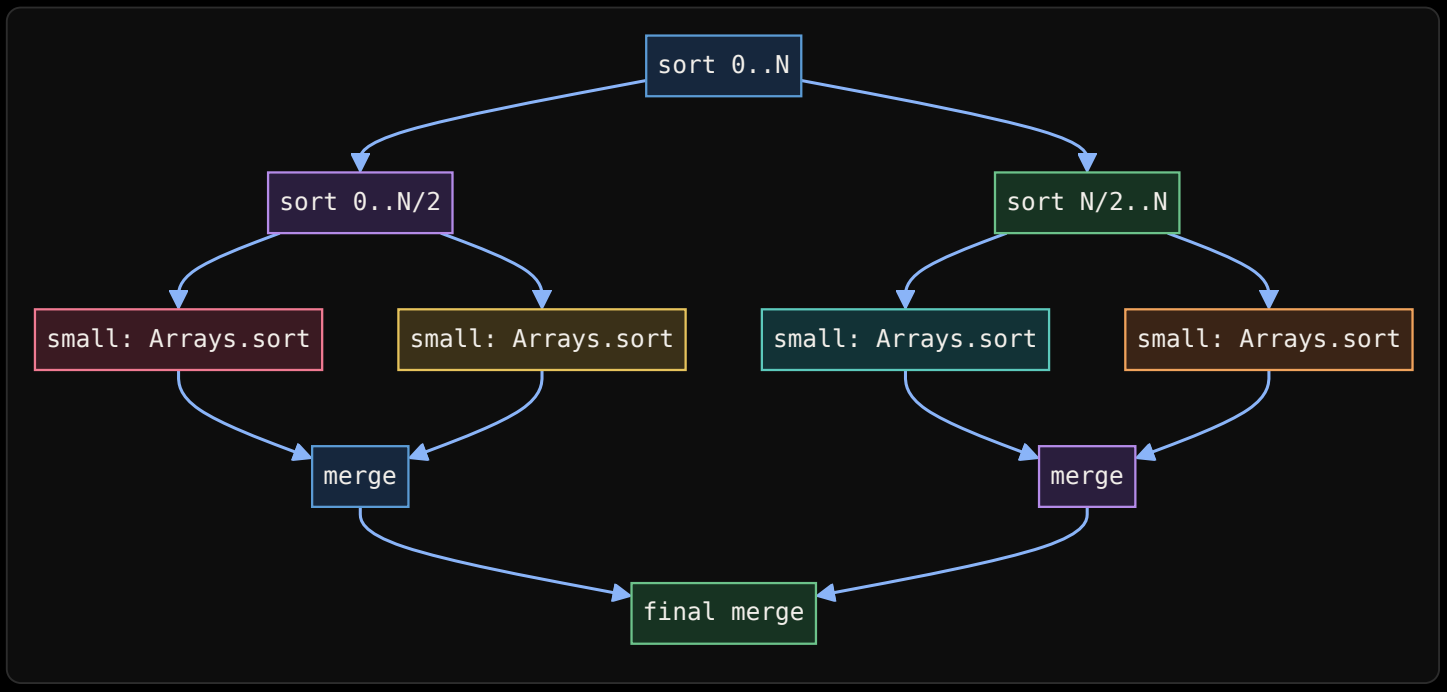
Problem

Sort a large `int[]` using all CPU cores.

Approach

Merge sort is **divide and conquer**: sort both halves *independently* (in parallel), then merge. Java's **Fork/Join framework** (`ForkJoinPool`, `RecursiveAction`/`RecursiveTask`) is built for this:

1. Range small (≤ 8,192 elements) → sort sequentially (`Arrays.sort`); splitting further costs more than it saves.
2. Otherwise split at `mid = (lo + hi) >>> 1` and `invokeAll(left, right)` — fork both halves, wait for both.
3. Merge using one **shared scratch buffer**; each task uses only its own `[lo, hi)` slice, so concurrent tasks share nothing.
4. **Work stealing:** each pool thread has its own task deque; idle threads steal from the other end of busy threads' deques, keeping all cores busy.



Java solution

```java
// code/lld/ch13/mergesort/Main.java
static final class ParallelMergeSort extends RecursiveAction {
    static final int THRESHOLD = 1 << 13;   // below ~8K elements, splitting costs more than it saves
    private final int[] a, buf;             // buf = one shared scratch array; tasks use disjoint ranges
    private final int lo, hi;               // sorts a[lo, hi)

    @Override protected void compute() {
        if (hi - lo <= THRESHOLD) { Arrays.sort(a, lo, hi); return; }    // sequential base case
        int mid = (lo + hi) >>> 1;                                        // overflow-safe midpoint
        invokeAll(new ParallelMergeSort(a, buf, lo, mid),                 // fork both halves, wait for both
                  new ParallelMergeSort(a, buf, mid, hi));
        merge(mid);
    }
    private void merge(int mid) {
        if (a[mid - 1] <= a[mid]) return;                                 // already in order: skip
        System.arraycopy(a, lo, buf, lo, hi - lo);
        int i = lo, j = mid, k = lo;
        while (i < mid && j < hi) a[k++] = buf[i] <= buf[j] ? buf[i++] : buf[j++];   // <= keeps it stable
        while (i < mid) a[k++] = buf[i++];                                // right leftovers already in place
    }
}
static void parallelSort(int[] a) {
    ForkJoinPool.commonPool().invoke(new ParallelMergeSort(a, new int[a.length], 0, a.length));
}
// main: a small array, then 4,000,000 seeded random ints checked against Arrays.sort and Arrays.parallelSort
```

Output:

```
small: [3, 3, 9, 10, 27, 38, 43, 82]
4000000 random ints: matches Arrays.sort = true
Arrays.parallelSort agrees       = true
```

Why it's correct

- The two subtasks touch **disjoint** ranges of `a` and `buf` → no data race.

- `invokeAll` returns only when both subtasks completed; Fork/Join's `join` establishes happens-before, so the parent's merge sees the children's writes.
- Ties take from the left run (`<=`), so the merge is **stable** (equal elements keep their order; matters for objects, not ints). The early exit `a[mid-1] <= a[mid]` skips already-ordered halves.

Pitfalls

- No threshold → millions of tiny tasks; overhead dominates.
- A new buffer per merge → heavy GC (garbage collection, the JVM's automatic freeing of unused memory); allocate once.
- `fork()` on both halves and `join()` in the wrong order; blocking I/O inside Fork/Join tasks (it's meant for CPU work).
- `(lo + hi) / 2` can overflow for huge arrays; `>>> 1` avoids it.
- Expecting speed-up on tiny arrays or one core (Amdahl's law: the sequential part — here the final merge — caps total speed-up).

Real-world examples

- `Arrays.parallelSort` (Java 8+) splits, sorts pieces in parallel on the common Fork/Join pool and merges (the demo cross-checks against it); parallel streams (`list.parallelStream()`) run on `ForkJoinPool.commonPool()`.
- Big-data sorts (Hadoop MapReduce's — map over data in parallel, then combine — sort/shuffle, Spark's `sortBy`) apply divide-sort-merge across machines.

In the interview

> **30-second answer:** "A RecursiveAction splits the range in half until it's below a threshold, sorts small pieces sequentially, then merges using one preallocated buffer. invokeAll runs both halves in parallel on the ForkJoinPool, whose work stealing keeps cores busy. Subtasks touch disjoint ranges, so no locks are needed."

Follow-ups:

- *`RecursiveAction` vs `RecursiveTask`?* `RecursiveTask<V>` returns a value (parallel sum); `RecursiveAction` returns nothing (in-place sort).
- *Parallel merge too?* Split the merge by binary-searching the median of one run in the other — how fully parallel merge sorts scale further.
- *Why is a shared `buf` safe?* Concurrent tasks use non-overlapping index ranges.

TL;DR

Split until small, sort sequentially, merge; Fork/Join + work stealing; disjoint ranges = no locks.

13.11 Rate limiter — token bucket (thread-safe)

Problem

Limit how often clients may call an **API** (Application Programming Interface — here, a web service clients send requests to): allow short **bursts** up to *B* requests and a sustained rate of *R* per second, correctly under many concurrent requests, per client (user id / API key — a secret string identifying the caller / IP address).

Approach

Token bucket:

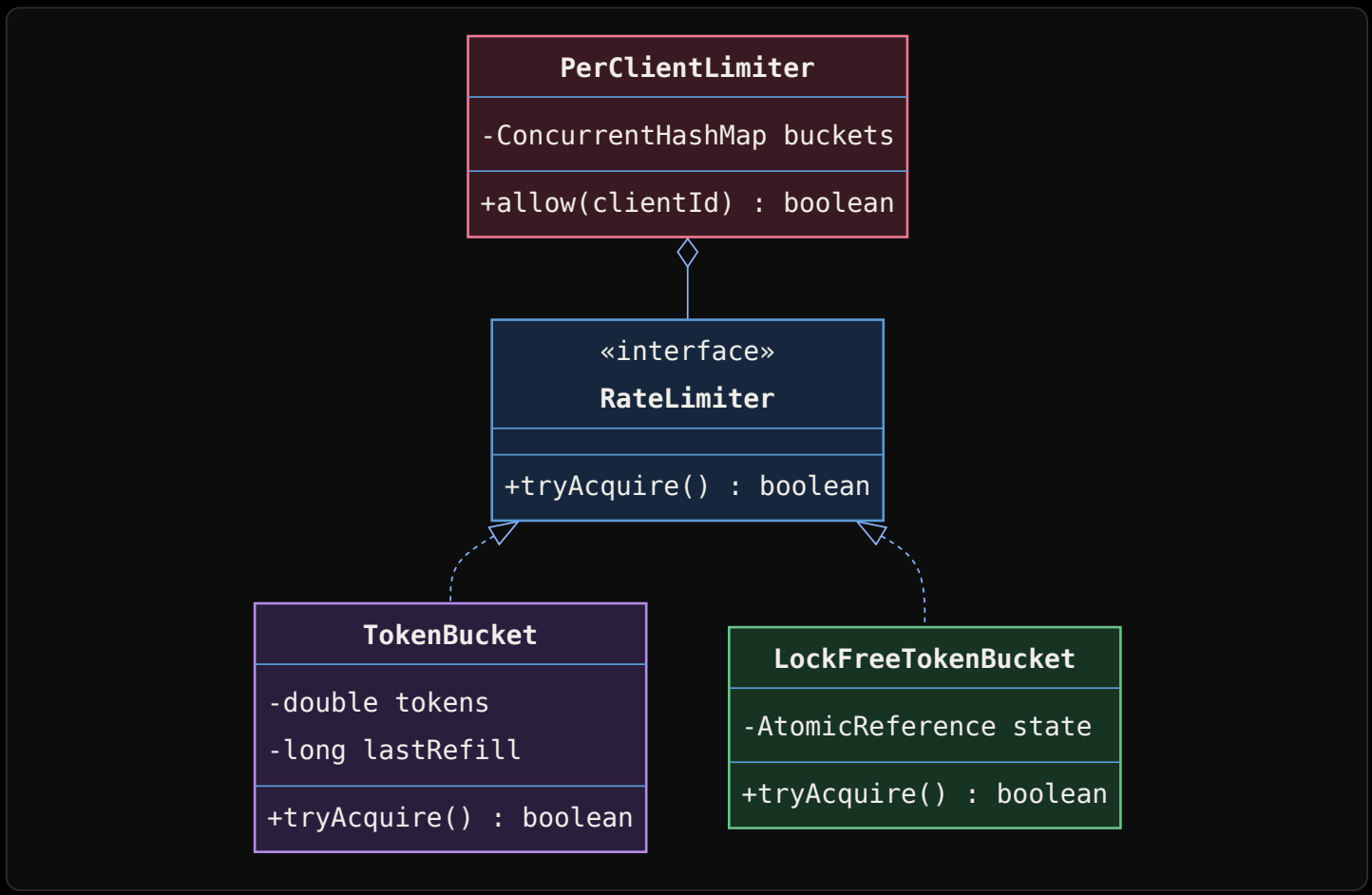
1. A bucket holds up to *capacity* tokens (the allowed burst) and starts full.
2. Tokens drip in at *refillRate* per second, never exceeding capacity.
3. Each request takes one token; none → reject (HTTP 429 "Too Many Requests"; HTTP is the web's request/response protocol — see System Design Fundamentals).
4. Refill **lazily**: store `tokens` and `lastRefill`; on each request add `elapsed × rate`. No background thread.

Thread safety: "refill + check + take" must be **one atomic step**, or two threads can both see 1 token and both take it.

- Version 1: `synchronized tryAcquire()` — simple and correct.

- Version 2: **lock-free** — `(tokens, lastRefill)` in one immutable `State` record inside an `AtomicReference`, CAS the whole state (the "several fields → one immutable object → one CAS" trick from §12.15).
- Per client: `ConcurrentHashMap.computeIfAbsent(clientId, …)` creates exactly one bucket per client.
- Time comes from an injected `Ticker`, so the demo is deterministic.

Algorithm	Bursts	Memory	Notes
Token bucket	Allows up to capacity	2 numbers	Most common for APIs
Leaky bucket	Smooths to constant output rate	Queue	Good for shaping traffic (evening out bursts)
Fixed window counter	Allows 2× burst at window edges	1 counter	Simplest
Sliding window log	Exact	One timestamp per request	Memory-heavy
Sliding window counter	Approximate, smooth	2 counters	Good compromise



Java solution

```java
// code/lld/ch13/ratelimiter/Main.java  (Ticker/FakeTicker as in 13.6)
interface RateLimiter { boolean tryAcquire(); }

static final class TokenBucket implements RateLimiter {
    private final double capacity, tokensPerNano;          // tokensPerNano = refillPerSecond / 1e9
    private final Ticker ticker;
    private double tokens;          // guarded by 'this'; starts full (= capacity)
    private long lastRefill;        // guarded by 'this'

    public synchronized boolean tryAcquire() {
        long now = ticker.nanos();
        tokens = Math.min(capacity, tokens + (now - lastRefill) * tokensPerNano);   // refill + acquire: ONE atomic step
        lastRefill = now;
        if (tokens >= 1) { tokens -= 1; return true; }
        return false;
    }
}

static final class LockFreeTokenBucket implements RateLimiter {  // same algorithm, one immutable state + CAS
    private record State(double tokens, long lastRefill) { }
    private final AtomicReference<State> state;            // starts at new State(capacity, ticker.nanos())

    public boolean tryAcquire() {
        while (true) {
            State cur = state.get();
            long now = Math.max(ticker.nanos(), cur.lastRefill());   // never move time backwards
            double refilled = Math.min(capacity, cur.tokens() + (now - cur.lastRefill()) * tokensPerNano);
            boolean ok = refilled >= 1;
            State next = new State(ok ? refilled - 1 : refilled, now);
            if (state.compareAndSet(cur, next)) return ok;            // lost the race? recompute and retry
        }
    }
}

static final class PerClientLimiter {                      // one bucket per user id / API key / IP
    private final ConcurrentHashMap<String, RateLimiter> buckets = new ConcurrentHashMap<>();
    private final Supplier<RateLimiter> factory;
    boolean allow(String clientId) { return buckets.computeIfAbsent(clientId, k -> factory.get()).tryAcquire(); }
}
// main: capacity 5, refill 2/s, fake clock; then 100 latch-released threads after an idle hour; per-client demo
```

Output:

```
[synchronized] capacity 5, refill 2/s
  t=0.0s  7 requests : YYYYYnn
  t=1.0s  3 requests : YYn
  t=1.25s 1 request  : n  (only 0.5 token so far)
  t=1.5s  1 request  : Y
  after 1 idle hour  : allowed 5 of 100 concurrent requests (burst capped at 5)
[lock-free] capacity 5, refill 2/s
  t=0.0s  7 requests : YYYYYnn
  t=1.0s  3 requests : YYn
  t=1.25s 1 request  : n  (only 0.5 token so far)
  t=1.5s  1 request  : Y
  after 1 idle hour  : allowed 5 of 100 concurrent requests (burst capped at 5)
per-client (3 burst): alice YYYn, bob YY  (alice's spam doesn't hurt bob)
```

Why it's correct

- **Atomic decision:** `TokenBucket` refills, checks and decrements inside one `synchronized` method. `LockFreeTokenBucket` computes the new state from one snapshot and installs it with `compareAndSet`; if another thread changed it first, the CAS fails and it recomputes — each token is granted at most once.
- **Burst cap:** tokens are clamped to `capacity`, so after an idle hour only 5 of 100 simultaneous requests pass.
- **Rate:** at 2 tokens/s, after 1 s exactly 2 new tokens; at +0.25 s only 0.5 → rejected; at +0.5 s → 1 → allowed.
- **Per-client isolation:** `computeIfAbsent` is atomic, so two concurrent first requests can't create two buckets (doubling the limit).
- The lock-free version clamps `now` with `Math.max(..., lastRefill)` so a thread that read the clock slightly earlier can't move time backwards.

Pitfalls

- Separate `refill()` and `take()` calls → race; two threads spend the same token.
- A background thread per bucket adding tokens → thousands of threads/timers; lazy refill needs none.
- `currentTimeMillis()` (wall clock can jump) instead of `nanoTime()`.
- An unbounded map of buckets → memory leak for one-off clients; evict idle buckets (e.g. with the TTL cache of 13.6).
- Multi-server: an in-memory limiter limits only per server. For a global limit, keep the bucket in **Redis** (an in-memory store shared by all servers), updated atomically with a Lua script (Redis runs it as one uninterruptible step), or use a sliding-window counter in Redis.

Real-world examples

- **Stripe** has described using token buckets (backed by Redis) for its API rate limits.
- **Amazon API Gateway** (AWS's managed front door for APIs) **throttling** (deliberately rejecting or delaying excess requests) is configured with a *rate* and a *burst* — token-bucket parameters.
- **Guava** `RateLimiter`, **Bucket4j** (token bucket for Java, with Redis/JCache — the standard Java caching API — back-ends), **Resilience4j** `RateLimiter`; NGINX's (a web server and reverse proxy) `limit_req` uses a leaky-bucket approach.

In the interview

> **30-second answer:** "Token bucket: capacity for bursts, refill rate for the average. I refill lazily from elapsed nanoTime on each request, and make refill-check-take one atomic step — synchronized, or lock-free by CASing an immutable state record. A ConcurrentHashMap gives one bucket per client via computeIfAbsent; across servers I'd move the bucket into Redis with a Lua script."

Follow-ups (with answers):

- *When should the client retry?* Compute `(1 − tokens) / rate` seconds and return it in a `Retry-After` header (tells the client how many seconds to wait).
- *Limits per plan (free vs paid)?* The factory builds buckets with capacity/rate from the client's plan (Strategy pattern).
- *Block instead of rejecting?* Compute the wait and sleep/park until a token exists (Guava's `acquire()` does this).
- *Capacity 5, rate 2/s, empty at t=0 — how many pass at t=3s?* min(5, 3 × 2) = 5.
- *Why one immutable `State`?* `tokens` and `lastRefill` must change together; one CAS on one reference updates both atomically.

TL;DR

Bucket of tokens: capacity = burst, rate = average. Lazy refill; refill+take atomic (lock or CAS on immutable state); one bucket per client; Redis for multi-server.

13.12 Chapter summary

Problem	Key primitive	The one invariant to state in the interview
FooBar	2 semaphores	Exactly one permit exists across both
Zero Even Odd	3 semaphores	Zero dispatches; one permit circulates
FizzBuzz	monitor + `notifyAll`	Predicates partition 1..n
H2O	2 semaphores + barrier	≤2 H and ≤1 O per round; slot freed after release
Dining Philosophers	ordered locks / n−1 semaphore	No circular wait
TTL cache	CHM + `compute` + conditional remove	Never return or keep expired data; load once per key
Striped map	lock array	Each bucket belongs to exactly one stripe
Blocking queue	lock + 2 conditions	0 ≤ count ≤ capacity; wait in loops
Bloom filter	atomic OR	Bits only go 0 → 1; no false negatives
Merge sort	Fork/Join	Subtasks touch disjoint ranges
Rate limiter	lock or CAS on immutable state	Refill + take is one atomic step

13.13 Chapter self-check

1. **Q:** In FooBar, why doesn't it matter which thread starts first? **A:** The one without a permit blocks until the other releases it.
2. **Q:** In FizzBuzz, what bug appears with `notify()`? **A:** The wrong thread may be woken and go back to sleep; the right one never wakes → hang.
3. **Q:** In H2O, why release the slot after printing? **A:** So the next molecule cannot form (and print) until this one has fully left.
4. **Q:** Two ways to prevent Dining Philosophers deadlock? **A:** Always take the lower-numbered fork first; or allow at most n−1 philosophers to try at once.
5. **Q:** Why does the TTL cache use `remove(key, entry)`? **A:** To avoid deleting a fresh value written concurrently.
6. **Q:** Why must the striped map's stripe depend only on the hash? **A:** So a key's lock doesn't change when the table resizes.
7. **Q:** When is `signal()` safe instead of `signalAll()`? **A:** When each condition queue holds only threads waiting for the same thing and one state change enables exactly one waiter.
8. **Q:** What would a non-atomic OR do to a Bloom filter? **A:** Lose bits → false negatives.
9. **Q:** Why a threshold in parallel merge sort? **A:** Small tasks cost more to schedule than to sort.
10. **Q:** How do you make a token bucket global across 10 servers? **A:** Store it in Redis and update it atomically with a Lua script.

13.14 Chapter TL;DR

- **Ordering puzzles** (FooBar, ZeroEvenOdd, H2O): semaphores as batons, barriers for groups; keep "exactly one permit" style invariants.
- **Predicate waiting** (FizzBuzz, blocking queue): lock + `while (!condition) wait/await` + `notifyAll` or targeted `signal`.
- **Multi-lock problems** (philosophers): break circular wait with ordering or limiting.
- **Thread-safe data structures** (cache, map, queue, Bloom filter, rate limiter): immutable entries, atomic compound operations, striping, CAS on immutable state, and injected clocks for testable time.
- **Parallel algorithms** (merge sort): Fork/Join with thresholds and disjoint data.